Università degli Studi di Napoli
Federico II

A.A. 2026-2027

Reti e Programmazione Distribuita

Indice

0 Prefazione 16

Come è organizzata questa dispensa 17
Convenzioni tipografiche 17
Testi di riferimento e materiale 18

I Fondamenti e architettura delle reti 19

1 Reti, topologie e categorie 20

Che cos'è una rete di calcolatori 20
A cosa servono le reti 21
Client e server 21
Client e server sono processi 22
Peer-to-peer: niente ruoli fissi 22
Internet: una rete di reti 23
Il collante: l'Internet Protocol (IP) 25
Breve storia di Internet 26
Di cosa è fatta una rete: dispositivi, collegamenti, host 26
I dispositivi dell'infrastruttura 27
Gli host 27
La rete come grafo 28
Le cinque componenti di una comunicazione 28
Che cos'è un protocollo 29
Rappresentazione e flusso dei dati 29
Rate, banda e throughput 30
Pacchetti e store-and-forward 30
Code e perdite 31
Commutazione di circuito e commutazione di pacchetto 32
Commutazione di circuito 32
Commutazione di pacchetto 32
Perché Internet ha scelto i pacchetti 33
Collegamenti punto-punto e multipunto 33
Topologie di rete 34
Bus 35
Anello (ring) 35
Stella (star) 35
Albero (tree) 36
Linea (line) 36
Maglia (mesh) 36
Topologie ibride 37
Confronto tra le topologie 38
Categorie di reti per estensione 38
PAN — Personal Area Network 39
LAN — Local Area Network 39
MAN — Metropolitan Area Network 40
WAN — Wide Area Network 41
WAN eterogenee e internetwork 43

Internet Service Provider 43
Chi si collega con chi, e chi paga 44
Collegarsi a un ISP 45
Tecnologie di accesso residenziale 46
Riepilogo 46

2 Stratificazione, modelli di riferimento e incapsulamento 47

Il Web è un'applicazione 47
Cosa attraversa una singola richiesta HTTP 47
Internet vista "dai bulloni" e come servizio 48
La vista "nuts and bolts" 48
Internet come infrastruttura di servizi 49
Protocolli e standard 49
Chi scrive gli standard 50
La stratificazione 50
Moduli, interfacce e incapsulamento delle funzioni 51
La stratificazione non è un'idea delle reti 51
Cosa guadagniamo, e cosa paghiamo 51
Servizio, interfaccia e protocollo 52
Strati omologhi: comunicazione virtuale e fisica 53
Servizi con e senza connessione 55
Servizio orientato alla connessione 55
Servizio senza connessione 55
Affidabilità 56
Sei tipi di servizio 57
Le primitive di servizio 57
Il modello di riferimento ISO/OSI 58
Non tutti i nodi implementano tutti gli strati 59
Incapsulamento e decapsulamento 60
Sotto il proprio strato, il payload è opaco 61
I sette strati, uno per uno 61
1. Livello fisico (Physical) 61
2. Livello di collegamento (Data Link) 61
3. Livello di rete (Network) 62
4. Livello di trasporto (Transport) 62
5. Livello di sessione (Session) 62
6. Livello di presentazione (Presentation) 63
7. Livello applicativo (Application) 63
Perché OSI non ha vinto 63
L'apocalisse dei due elefanti 63
Cosa non va nel modello TCP/IP 64
De iure e de facto 64
I cinque strati usati nel resto del corso 64
La stratificazione oltre Internet 65
Riepilogo 66

II Il livello applicativo 67

3 Il livello applicativo: architetture e servizi 68

Che cos'è un'applicazione di rete 68
Creare un'applicazione di rete 69
Architetture delle applicazioni di rete 70
Architettura client-server 70
Architettura peer-to-peer (P2P) 71
Comunicazione tra processi 72
I socket 72
Indirizzare i processi: indirizzo e identificatore 73
Un indirizzo appartiene a un'interfaccia 74
IPv4 e l'indirizzo che vuol dire "qui" 74
Un nome non è un indirizzo 75

I servizi di trasporto disponibili alle applicazioni 75
Che cosa Internet non garantisce 76
I protocolli di livello applicativo 76
Un protocollo non è l'applicazione 77
Protocolli applicativi comuni 77
FTP e FTPS 77
Due connessioni: controllo e dati 78
Una sessione FTP 78
Riepilogo 79

4 Il Web e il protocollo HTTP 80
Dal Web a HTTP 80
Le URL 81
Pagine e oggetti 81
Prima del primo messaggio HTTP 82
Come funziona HTTP 82
Il protocollo di trasporto sottostante 82
HTTP è stateless 83
Connessioni persistenti e non persistenti 83
Esempio di connessione non persistente 84
Connessioni in sequenza e in parallelo 84
Il costo in RTT 84
Il three-way handshake 85
Il conto totale 85
Connessioni persistenti e pipelining 86
Confronto 86
La connessione che si chiude mentre la usi 87
HTTP/2 e HTTP/3 87
Il blocco in testa alla fila 87
HTTP/2: frame intercalati su una sola connessione 87
Priorità e server push 88
HTTP/3 e QUIC 88
Laboratorio: il primo GET con Wireshark 88
Installare Wireshark 88
Catturare un GET 89
Riepilogo 89

5 HTTP: messaggi, cookie e caching 90
Il formato dei messaggi HTTP 90
Il formato della richiesta 90
I metodi HTTP 91
Esempio di richiesta 91
Il formato della risposta 92
I codici di stato 92
Esempio di risposta 93
Parlare HTTP a mano 93
I cookie 93
Le quattro componenti 94
Se il file dei cookie viene cancellato 95
Usi e problemi dei cookie 95
Il caching Web 95
Il funzionamento 96
Perché usare le cache 96
Il GET condizionale 97
Il caching nel browser 98
Oltre il caching: i proxy istituzionali 98
Perché l'HTTP in chiaro non si vede più 99
Laboratorio: una risposta che non sta in un pacchetto 99
Riepilogo 100

6 Posta elettronica e applicazioni peer-to-peer 101
La posta elettronica 101
SMTP 102
Il flusso completo di una mail 102

Dettagli del protocollo 102
Il messaggio e il suo passato a 7 bit 103
Accedere alla posta 103
POP3 104
IMAP 104
HTTP (webmail) 104
Peer-to-peer 105
Distribuire un file: due modi 105
BitTorrent 106
Il ciclo di vita di un peer 106
Il tracker 106
Le politiche del client 107
Riepilogo 108

7 Il DNS 109
Il problema: nomi e indirizzi 109
Perché non un repository centralizzato 109
Lo spazio dei nomi DNS 110
I nomi di dominio 111
I resource record 111
I name server e le zone 112
La risoluzione dei nomi 113
La gerarchia dei server 113
Il server DNS locale (resolver) 114
Query ricorsive e iterative 114
Il caching DNS 115
I messaggi DNS 116
Altri servizi del DNS 116
Aliasing 116
Distribuzione del carico 116
nslookup: interrogare il DNS a mano 116
La risoluzione iterativa fatta a mano 118
dig: leggere il messaggio DNS 119
I flag 119
Chiedere un tipo 119
Fare il server locale: partire dalla radice 120
Come un record entra nel database 121
La risposta che nessun server autorevole ha inviato 121
Laboratorio: cinque domande a un dominio vero 121
Riepilogo 122

III Programmazione di rete 123

8 File descriptor, socket e un server TCP sequenziale 124
Programmi, processi e protocolli 124
Tutto è un file: il descriptor 124
A che cosa punta un descriptor 125
read e write 125
read può restituire meno di quanto chiesto 125
write può scrivere meno di quanto chiesto 126
Quando una chiamata fallisce: −1 ed errno 126
Il socket 126
Dominio, tipo, protocollo 127
Le quattro fasi, con e senza connessione 128
Un indirizzo per l'estremo 128
Le chiamate lato server 128
bind(): un indirizzo locale 128
listen() e accept() 129
Il server ha due tipi di socket 129

Le chiamate lato client 129
Leggere e scrivere su un socket connesso 129
close(), e come il peer capisce che ce ne siamo andati 130
Il client e il server, per intero 130
Provarli sulla stessa macchina 131
Un socket trasporta byte, non tipi 132
Codificare un messaggio come testo 132
Dove finisce un messaggio? 132
Un client alla volta 133
Il progetto: una chat in nove versioni 133
Riepilogo 135

9 Indirizzi, ordine dei byte e struttura di client e server 136

La forma dei due programmi 136
Un indirizzo è un IP e una porta 137
Quali porte si possono usare 137
Una connessione sono quattro numeri, non due 137
L'ordine dei byte 138
Lo stesso numero, due disposizioni in memoria 138
L'ordine di rete e quattro funzioni 138
Quali campi viaggiano davvero 139
Anche i nostri dati 139
Riempire gli indirizzi 139
Da una stringa puntata a quattro byte 139
L'indirizzo del server e quello del client 140
Riavviare il server: le opzioni dei socket 140
Lo stesso codice su IPv6 140
Dal nome all'indirizzo 141
getaddrinfo: da un nome a indirizzi pronti 141
Provare la lista finché uno funziona 141
Le stesse ricerche dalla shell 142
HTTP scritto a mano 142
Quello che abbiamo scritto è un payload 143
Esercizi 144
Riepilogo 145

10 Processi: fork, ereditarietà dei descriptor e segnali 146

Un processo, un pid, un genitore 146
fork: una chiamata, due ritorni 147
Che cosa eredita il figlio 147
I descriptor dopo la fork 148
Anche il socket connesso viene ereditato 148
Chi chiude che cosa 149
La vita di un processo 149
Tre modi di finire, e lo stato 149
wait e waitpid 150
Gli zombie 150
I segnali 151
Ignorare, catturare, o farsi uccidere 151
Che cosa può fare un handler 152
Raccogliere i figli con SIGCHLD 152
Inviare segnali: kill 152
Fermare il server di proposito 153
exec: il processo diventa un altro programma 153
Esercizi 154
Riepilogo 155

11 I thread 156

Un processo, più flussi di controllo 156
Che cosa è privato e che cosa è condiviso 157
Che cosa compra la condivisione, e che cosa costa 157
Concorrenza non è parallelismo 157
Dove sono conosciuti i thread 158

Creare un thread 158
La funzione di partenza e il suo unico argomento 159
Due thread e un main 159
La fine di un thread 160
pthread_join 160
Thread detached 161
Un thread per connessione 161
Il prezzo della condivisione 162
Il contatore che perde gli incrementi 162
Non tutte le funzioni possono essere chiamate da due thread 162
Thread e segnali 162
Cancellare un thread, e perché un server lo fa di rado 163
Quanto costa un thread, e quanti se ne possono avere 163
Esercizi 164
Riepilogo 165

12 SINCRONIZZAZIONE 166
Lo stato condiviso di un server 166
Il mutex 166
Un mutex protegge dati, non righe di codice 167
Per quanto tempo tenere un lock 168
Due lock, due thread, nessun progresso 168
Tipi di mutex, e mutex fra processi 168
Le variabili di condizione 169
Che cosa fa pthread_cond_wait con il mutex 169
signal o broadcast 170
Un produttore, due consumatori 170
Aspettare con una scadenza 171
Il buffer limitato 171
Avviare i thread, e fermarli 173
I semafori 173
Semaforo o variabile di condizione 174
Limitare la concorrenza con un semaforo contatore 174
Una globale privata di ogni thread 175
Esercizi 176
Riepilogo 177

13 SERVER CONCORRENTI: PROCESSI E THREAD 178
Che cosa rende concorrente un server 178
Un processo per client 179
Venti client insieme 180
Far parlare i processi 180
La pipe 180
I figli che riferiscono al genitore 182
dup e dup2 182
Parentela, un nome, un indirizzo 183
Una FIFO: server e client 183
Memoria condivisa attraverso una fork 184
Un thread per client 184
Lo stato che i thread condividono 185
Processi o thread, per un server 186
Quanto costa «uno per client» 187
Esercizi 188
Riepilogo 191

14 MULTIPLEXING DELL'I/O, SERVER A EVENTI E POOL DI THREAD 192
Aspettare su più di un descriptor 192
Gli insiemi di descriptor 193
Quando un descriptor è pronto 193
Il timeout 193
select consuma gli insiemi 194
Un client che ascolta due descriptor 194
Un solo flusso di controllo, molte connessioni 195
In questo ciclo nulla può bloccarsi 196

Il client lento, finalmente 197
Quanto costa select 197
Un insieme fisso di worker 198
Quanti worker, e che cosa succede quando la coda si riempie 199
Tre forme di server 200
Un mittente, molti destinatari 200
Gli indirizzi multicast 201
Entrare in un gruppo 201
Inviare a un gruppo 202
Chi rende selettivo il multicast 202
Esercizi 203
Riepilogo 207

15 PROTOCOLLI APPLICATIVI, DELIMITAZIONE E AFFIDABILITÀ SU UDP 208
Le quattro domande, rivolte al nostro protocollo 208
Testo o byte 208
Codificare un messaggio come testo, per bene 209
Codificare un messaggio come byte 209
Dove finisce un messaggio 210
Leggere e scrivere esattamente n byte 210
Leggere un frame 211
Il protocollo della chat, messo per iscritto 211
Saper dire di no 212
Un server che sopravvive ai propri client 212
Una chiamata bloccante interrotta da un segnale 212
La write che uccide il server 213
Arrendersi in tempo 213
Un protocollo che si può scrivere a mano 214
Comunicare senza connessione 214
Un server di eco UDP 215
Una sendto, un datagramma 216
connect su un socket a datagrammi 216
Affidabilità scritta a mano 216
Che cosa fa la rete ai nostri datagrammi 216
Che cosa bisogna aggiungere 216
La ritrasmissione che nessuno aveva chiesto 217
Numeri di sequenza: scartare, ma confermare di nuovo 217
Esercizi 218
Riepilogo 222

IV IL LIVELLO DI TRASPORTO 223

16 IL LIVELLO DI TRASPORTO E UDP 224
Dall'applicazione al trasporto 224
Trasporto e rete: processi contro host 225
Le quattro responsabilità del livello di trasporto 225
La consegna processo-a-processo 226
Le porte 226
nmap 227
Multiplexing e demultiplexing in azione 228
In UDP 228
In TCP 229
Un esempio con HTTP 230
UDP 230
Perché usare UDP invece di TCP 231
Chi usa cosa 231
Il formato del segmento UDP 232
Il checksum UDP 232
Sotto il trasporto: ritardi, code e perdite 233
Trasmissione non è propagazione 233

Intensità di traffico, e da dove vengono le perdite 234
Ritardo da un capo all'altro e throughput 234
Riepilogo 235

17 Il trasferimento dati affidabile 236

Il problema 236
Stop-and-wait con corruzione dei pacchetti 237
I due problemi dello stop-and-wait ingenuo 237
Con gli ACK numerati il NAK non serve più 238
Stop-and-wait con perdita dei pacchetti 238
La stima del timeout 239
Il modello "ad acqua" 240
Le prestazioni dello stop-and-wait 240
Il pipelining 241
Go-Back-N 242
Il ricevente in GBN 242
Selective Repeat 243
Il vincolo sui numeri di sequenza 245
Un'ipotesi fatta fin dall'inizio 246
Riepilogo 246

18 TCP: segmenti, numeri di sequenza e ritrasmissione 247

TCP a confronto con UDP 247
Una connessione senza niente in mezzo 247
I buffer TCP 248
La Maximum Segment Size (MSS) 248
Il segmento TCP 249
Numeri di sequenza e di acknowledgment 249
Conferme cumulative, e un segmento in anticipo 250
Il caso full-duplex 251
Da dove parte la numerazione 252
Il mittente TCP in tre eventi 252
Un timer, non uno per segmento 252
Il timeout di ritrasmissione 252
Raddoppiare il timeout 253
La stima dell'RTT 253
Il timeout 254
Fast retransmit 255
Le conferme duplicate 255
Go-Back-N o Selective Repeat? 256
Riepilogo 257

19 TCP: la gestione della connessione 258

Il problema 258
Il problema dei due eserciti 258
L'apertura della connessione 259
I dettagli 260
Perché il numero iniziale è estratto a caso 260
Diagramma degli stati (apertura) 261
Gli stessi stati, visti dal nostro codice 261
Quanto costa l'handshake 261
Il SYN flood e i SYN cookie 262
La chiusura della connessione 263
Mezza chiusura: una direzione alla volta 263
Diagramma degli stati (chiusura) 264
TIME WAIT 264
Come TCP si salva dalla perdita di pacchetti 265
In laboratorio: una connessione vista da vicino 266
Riepilogo 267

20 Controllo di flusso e controllo di congestione 268

Il problema comune: i buffer che traboccano 268

Il controllo di flusso 268
Le formule 269
Sedici bit non bastano più 270
Il problema della congestione 270
Scenario 1: buffer infinito 271
Scenario 2: buffer finito e ritrasmissioni 272
Scenario 3: la perdita al secondo salto 272
I due approcci al controllo di congestione 273
ECN: due bit e due flag 273
Il controllo di congestione di TCP 274
1. Regolazione del ritmo: la congestion window 274
2. Rilevazione della congestione: i loss event 274
3. Aggiustamento del ritmo: l'algoritmo di Jacobson 275
Le fasi dell'algoritmo 275
Slow start 275
Congestion avoidance 276
Fast recovery 276
La macchina a stati del controllo di congestione 277
Il quadro completo delle transizioni 278
Che cosa esegue davvero la nostra macchina 279
Equità 279
Che cosa c'è sotto la nostra write 280
Analizzare il traffico TCP con Wireshark 281
Riepilogo 282

V IL LIVELLO DI RETE 283

21 IL LIVELLO DI RETE E I ROUTER 284
Dal trasporto alla rete 284
Che servizi offre il livello di rete 285
I router nella rete 286
Tipi di router 287
Data plane e control plane 287
Centralizzato o decentralizzato 288
Le componenti di un router 288
Il forwarding e la longest prefix matching 289
Match-plus-action 291
La switching fabric 291
Le code sulle porte 292
Quando non c'è più posto 292
Quanto buffer basta 292
La schedulazione dei pacchetti 293
FIFO (First-Come-First-Served) 293
Priority queuing 294
Round-robin queuing 294
Riepilogo 295

22 INDIRIZZAMENTO IP E IL PROTOCOLLO IPV4 296
Il protocollo IP 296
Gli indirizzi IPv4 296
L'indirizzo appartiene all'interfaccia 297
Il subnetting 297
La subnet mask 297
Quali maschere sono valide 298
Verificare se due host sono nella stessa subnet 299
Router e subnet 299
Leggere la propria configurazione 301
L'indirizzamento su Internet 302
Indirizzamento classful 302
Indirizzamento classless (CIDR) 302

Subnet interne a un blocco 303
L'aggregazione degli indirizzi 303
Come ottenere un blocco di indirizzi 304
Il datagramma IPv4 305
La frammentazione IPv4 306
Come funziona la frammentazione 306
Non frammentare: la scoperta dell'MTU di percorso 307
Riepilogo 308

23 ICMP e la diagnosi di una rete 309

Parlare della rete, non attraverso la rete 309
L'errore torna con la sua causa allegata 310
Quando non si invia alcun messaggio 310
ping 311
Un'applicazione senza livello di trasporto 311
Dove va il tempo 312
traceroute 312
Come sa di essere arrivato 313
Due varianti della stessa idea 313
Il nome risolto prima della prima sonda 314
traceroute e la frammentazione IP 314
Destinazione irraggiungibile, e chi lo dice 315
I messaggi caduti in disuso 315
Una scala di domande 315
Chi c'è su questa rete 316
DNS, VPN e spoofing: un caso di studio 317
La nostra chat vista da un livello più in basso 317
Esercizi 319
Riepilogo 321

24 DHCP, NAT e IPv6 322

L'assegnazione degli indirizzi IP 322
Il funzionamento del DHCP 322
Il DHCP relay agent 324
Leggere che cosa ha dato il lease 324
Un lease è temporaneo, e la connessione anche 324
Il problema della visibilità 324
Il NAT 325
Un esempio di NAT 325
Il limite del NAT 326
Che cosa costa la traduzione 326
Gli indirizzi IP riservati 327
Esempio completo: configurare una LAN 327
1. Il lato WAN del router 328
2. Il lato LAN del router 328
3. Gli host 329
IPv6 330
I vantaggi di IPv6 330
Il datagramma IPv6 331
Cosa manca rispetto a IPv4 331
Dalla IPv4 alla IPv6 332
Perché questo livello è quello difficile da cambiare 332
Riepilogo 332

25 Il piano di controllo: routing, sistemi autonomi e BGP 333

Introduzione al routing 333
Il flooding 334
Formalizzazione del problema 334
L'algoritmo Distance Vector 335
L'equazione di Bellman 335
Lo pseudocodice 336
Complessità, pro e contro 337
Il problema del count-to-infinity 337
Come si sarebbe evitato 338

L'algoritmo Link State 339
La fase di scoperta 339
Lo pseudocodice 339
Un esempio completo e la ricostruzione del cammino 340
Complessità, pro e contro 340
DV contro LS 341
I sistemi autonomi 341
OSPF: link state dentro un AS 342
BGP: il protocollo che deve essere uguale per tutti 342
eBGP e iBGP 343
Che cosa porta una rotta 343
Scegliere fra le rotte 343
La policy decide prima del costo 344
Perché due protocolli 344
Un indirizzo, molte macchine 344
Mettere un'azienda su Internet 344
Riepilogo 345

VI IL LIVELLO DI COLLEGAMENTO 346

26 FRAMING E RILEVAZIONE DEGLI ERRORI 347
Dalla rete al collegamento e al livello fisico 347
Un esempio di segnale: la codifica NRZ 348
L'ultimo incapsulamento 348
I servizi del livello di collegamento 348
1. Framing 348
2. Accesso al collegamento (link access) 349
3. Consegna affidabile 349
4. Rilevazione e correzione degli errori 349
L'implementazione: la NIC 349
Software e hardware 350
Le tecnologie fisiche 350
I metodi di framing 350
Conteggio dei byte 350
Byte di flag e byte stuffing 351
Bit di flag e bit stuffing 351
Violazioni della codifica fisica 352
La rilevazione degli errori 352
Il controllo di parità 352
La parità bidimensionale 353
Il CRC 353
L'aritmetica modulo 2 354
Un esempio completo 354
I CRC standard 355
La distanza di Hamming 355
Riepilogo 356

27 IL SOTTOLIVELLO MAC E L'ACCESSO MULTIPLO 357
I due tipi di collegamento 357
Le collisioni 357
I protocolli ad accesso multiplo 358
Protocolli a partizionamento del canale 358
TDMA 358
FDMA 358
CDMA 359
Protocolli ad accesso casuale 360
ALOHA puro 360
Slotted ALOHA 360
CSMA e CSMA/CD 361

Protocolli a turni 363
Polling 363
Token passing 363
Terminali nascosti ed esposti 364
MACA 364
Riepilogo 365

28 Indirizzi MAC, switch, ARP ed Ethernet 366
Gli indirizzi MAC 366
Come lavorano gli indirizzi MAC 366
Indirizzi e livelli 367
Gli switch 367
Il self-learning 368
La LAN commutata 368
Il protocollo ARP 369
Un esempio di ARP 369
ARP e il gateway 370
Il frame Ethernet 371
Riepilogo 372

VII Sicurezza di rete 373

29 Attacchi alle reti e fondamenti di crittografia 374
Perché la sicurezza di rete 374
I malware 374
Gli attacchi DoS 375
Il packet sniffing 375
L'IP spoofing 376
Le proprietà di una comunicazione sicura 376
Introduzione alla crittografia 377
Le chiavi e il principio di Kerckhoffs 377
I problemi della crittoanalisi 377
Cifrari per sostituzione 378
Cifrari per trasposizione 378
La crittografia simmetrica 378
Il problema dello scambio delle chiavi 379
Il one-time pad 379
Due principi fondamentali 380
Algoritmi a chiave simmetrica: il DES 380
Le modalità di cifratura: il CBC 381
L'integrità dei messaggi 381
Primo tentativo: accodare l'hash 382
Il MAC: hash con segreto condiviso 382
La crittografia asimmetrica 382
RSA 383
Lo scambio di chiavi Diffie-Hellman 384
Le autorità di certificazione 384
Riepilogo 385

30 Autenticazione, SSL/TLS e sicurezza operativa 386
L'autenticazione degli estremi 386
Tentativi progressivi di autenticazione 386
Tentativo 1: controllare l'indirizzo IP 386
Tentativo 2: una password segreta 387
Tentativo 3: una password cifrata 387
La soluzione: il nonce 388
SSL e TLS 388
SSL: l'handshake 389
SSL: la derivazione delle chiavi 390
SSL: il trasferimento dati 390

I firewall 391
Il filtraggio tradizionale dei pacchetti 391
Le access control list 392
Il limite: il filtraggio è stateless 392
Il filtraggio stateful 392
L'application gateway 393
Gli Intrusion Detection System 394
La zona demilitarizzata (DMZ) 395
Riepilogo 396

VIII ESERCIZI 397

31 ESERCIZI SUI FONDAMENTI 398
Ritardi e tempi di trasmissione 398
Commutazione di circuito e di pacchetto 400
Topologie 401
Stratificazione e incapsulamento 402

32 ESERCIZI SUL LIVELLO APPLICATIVO 406
Preparare Wireshark 406
HTTP con Wireshark 407
Leggere i messaggi HTTP 410
Tempi di risposta 411
DNS con nslookup 412
DNS con Wireshark 414
DNS: domande d'esame 416
Socket 418
Domande di teoria 419

33 ESERCIZI SUL LIVELLO DI TRASPORTO 420
Il checksum 420
Prestazioni: stop-and-wait e pipelining 421
Go-Back-N e Selective Repeat 422
Numeri di sequenza e di acknowledgment in TCP 423
Stima dell'RTT e timeout 425
Controllo di flusso 425
Controllo di congestione 426
TCP con Wireshark 428

34 ESERCIZI SUL LIVELLO DI RETE 430
Subnet mask e appartenenza alle subnet 430
CIDR e longest prefix matching 432
Progettare un piano di indirizzamento 434
DHCP e NAT 435
Frammentazione IP 436
traceroute, ICMP e IP con Wireshark 437
Routing 439

35 ESERCIZI SUL COLLEGAMENTO E SULLA SICUREZZA 442
Controllo di parità 442
Il CRC 443
La distanza di Hamming 446
Framing 448
Accesso multiplo 449
Switch e ARP 450
Crittografia 452
Firewall 454

36 Domande di ripasso 456
Fondamenti 456
Livello applicativo 459
Livello di trasporto 460
Livello di rete 462
Livello di collegamento 463
Sicurezza 465
Consigli per la preparazione 466

Prefazione

Estratto

Il seguente documento rappresenta una raccolta di appunti, riorganizzata e migliorata nelle vesti grafiche, del corso di Reti e Programmazione Distribuita del professor Andrea Calì, per il corso di laurea in Informatica A.A. 2026-2027, della Federico II di Napoli.

L'obiettivo del corso è mostrare come due programmi, ovunque si trovino nel mondo, riescano a scambiarsi informazione. Si parte dalle applicazioni che usiamo ogni giorno e si scende, uno strato alla volta, fino ai bit che viaggiano sul filo: una rete non è una scatola magica, ma una pila di protocolli, ciascuno con un problema preciso da risolvere. Lungo il percorso si scrivono programmi in C sui socket e si chiude con la sicurezza e la privatezza.

Si tiene a specificare che questo documento non ha lo scopo di **sostituire libri di testo**, note del professore o lezioni frontali, nonostante siano tutte fonti utilizzate per la stesura, ma quello di integrare al materiale più complesso una rilettura degli argomenti da un punto di vista meno tecnico e quindi più lento nel soffermarsi nei passaggi critici.

Si ringraziano **Valentino Bocchetti** e **Pasquale Miranda** ideatori e gestori del progetto *Unina Docs*,

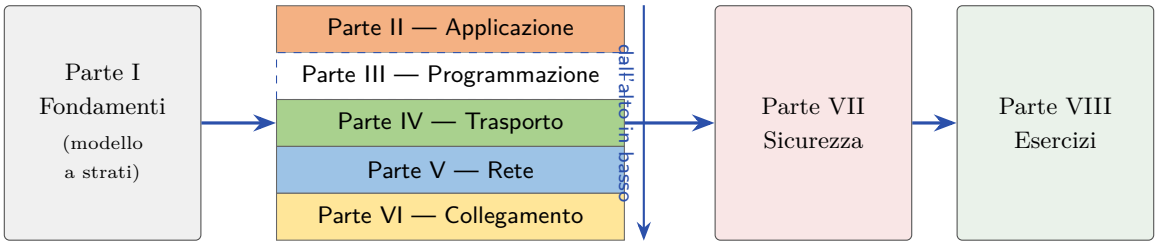
Si ringraziano inoltre tutti i revisori che hanno contribuito al documento con consigli e proposte di modifica, invitando a dare feedback in caso di errori.

Si ringraziano in fine i lettori nella speranza che questo lavoro sia stato utile.

Redazione a cura di **Fabio Rosiello**

Come è organizzata questa dispensa

Il corso segue un approccio **top-down**: si parte dal livello che l'utente percepisce (le applicazioni: il Web, la posta, il DNS) e si scende progressivamente verso l'hardware (il livello di collegamento e il livello fisico). La dispensa rispetta lo stesso ordine ed è divisa in otto parti. Subito dopo il livello applicativo c'è una parte di **programmazione**: è il punto in cui si smette di guardare i protocolli dall'esterno e si scrivono, in C, i programmi che li parlano.



- **Parte I — Fondamenti**: che cos'è una rete, com'è fatta, come si classifica, e soprattutto il *modello a strati* che regge tutto il resto del corso.
- **Parte II — Livello applicativo**: architetture client-server e P2P, HTTP, posta elettronica e DNS.
- **Parte III — Programmazione di rete**: i socket in C, dai descriptor a un server TCP; indirizzi e ordine dei byte; processi, thread e sincronizzazione; server concorrenti, `select` e pool di thread; la progettazione di un protocollo applicativo e l'affidabilità scritta a mano su UDP. Il filo conduttore è una chat, riscritta di capitolo in capitolo.
- **Parte IV — Livello di trasporto**: UDP, il problema del trasferimento affidabile, TCP in tutte le sue componenti (numeri di sequenza, timeout, connessione, controllo di flusso e di congestione).
- **Parte V — Livello di rete**: router, indirizzamento IP, subnetting e CIDR, DHCP, NAT, IPv6, ICMP e gli algoritmi di routing.
- **Parte VI — Livello di collegamento**: framing, rilevazione degli errori (parità, CRC, distanza di Hamming), protocolli ad accesso multiplo, Ethernet, switch e ARP.
- **Parte VII — Sicurezza**: attacchi, crittografia simmetrica e asimmetrica, integrità dei messaggi, autenticazione, SSL/TLS, firewall e IDS.
- **Parte VIII — Esercizi**: esercizi svolti divisi per argomento, esercitazioni di laboratorio con Wireshark e domande di ripasso.

Una dispensa che cresce con il corso

La dispensa segue le trasparenze del corso di *Reti e Programmazione Distribuita* (A.A. 2026/27), pubblicate nel repository GitHub del corso, insieme alle registrazioni e alle lavagne delle lezioni. Le parti I–III sono allineate alle lezioni 1–14; le parti successive, nate dal materiale dell'edizione precedente (*Reti di Calcolatori I*), vengono riviste sulle lezioni 15–32.

Convenzioni tipografiche

Nel testo si usano alcuni riquadri ricorrenti:

Definizione

I riquadri rossi contengono le **definizioni** formali: sono i concetti da saper enunciare con precisione all'esame.

Nota

I riquadri verdi contengono **approfondimenti**, analogie e osservazioni utili a capire meglio, ma non strettamente indispensabili.

Attenzione

I riquadri gialli segnalano **errori tipici** o punti in cui è facile confondersi.

Esempio 1 — Struttura di un esempio

Gli esempi sono racchiusi tra due righe orizzontali sottili e sono numerati all'interno di ogni capitolo.

Esercizio 1 — Struttura di un esercizio

Gli esercizi sono racchiusi tra righe doppie. Nella Parte VIII si trovano tutti gli esercizi svolti.

Nelle figure i livelli della pila protocollare hanno sempre lo stesso colore, così da riconoscerli a colpo d'occhio:

Testi di riferimento e materiale

Il corso fa riferimento principalmente a due testi, entrambi citati nelle lezioni:

- **J.F. Kurose, K.W. Ross**, *Computer Networking: A Top-Down Approach*. È il testo che dà l'impostazione top-down del corso e da cui provengono molti esempi su HTTP, TCP e IP.
- **A.S. Tanenbaum, N. Feamster, D.J. Wetherall**, *Computer Networks*. Più prolisso ma ricchissimo di esempi, anche non strettamente tecnici; usato soprattutto per la classificazione delle reti, il livello di collegamento e la crittografia.

Le trasparenze sono in inglese (la terminologia di rete è nata in inglese ed è più semplice impararla così); in questa dispensa i termini tecnici sono riportati sia in italiano sia in inglese. Le trasparenze sono un **supporto** alla lezione, non un sostituto dei libri.

Il laboratorio

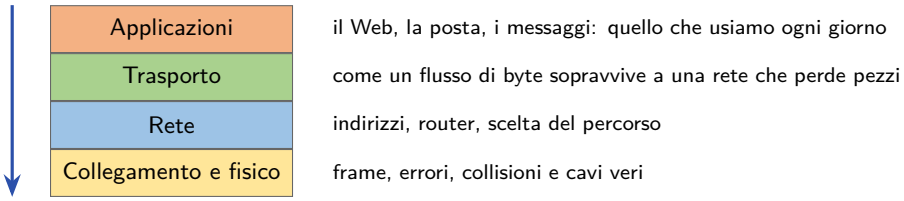
Per le esercitazioni viene fornito un **container Docker** già pronto, con tutto il software necessario (compilatore, strumenti di rete, Wireshark). Un container è una sorta di macchina virtuale leggera che condivide il kernel con la macchina ospite: se qualcosa si rompe, basta buttarlo via e ricrearlo, senza toccare il proprio sistema.

Parte I

Fondamenti e architettura delle reti

Reti, topologie e categorie

Questo primo capitolo è *terminologico*: introduce il vocabolario con cui parleremo per tutto il resto del corso. Non è difficile, ma non va sottovalutato: termini come *host*, *link*, *throughput*, *commutazione di pacchetto*, *ISP* torneranno in ogni capitolo, e all'esame vanno usati con precisione.



Il corso parte dall'alto (le applicazioni) e scende verso il basso; lungo il percorso si scrivono programmi in C sui socket, e si chiude con sicurezza e privatezza. Oggi, però, si risponde a due domande più semplici: **di cosa è fatta una rete** e **come si classificano le reti**.

Che cos'è una rete di calcolatori

Rete di calcolatori

Una **rete di calcolatori** è una collezione di dispositivi di calcolo **interconnessi** e **autonomi**. Due dispositivi sono interconnessi se possono **scambiarsi informazione**.

Le due parole chiave vanno prese sul serio.

- **Autonomi**: nessun dispositivo comanda gli altri. Ognuno ha la propria capacità di calcolo e funziona per conto suo: se un nodo si spegne, gli altri continuano a lavorare.
- **Interconnessi**: esiste un mezzo attraverso cui i dispositivi si scambiano bit. Può essere rame, fibra ottica o onde radio: alla definizione *non importa*.

Esempio 1 — Autonomo o no?

Un disco SSD collegato con due metri di cavo USB al vostro portatile **non** forma una rete con il portatile: è una periferica, completamente controllata dal calcolatore a cui è attaccata. Il vostro smartphone e il portatile collegati allo stesso Wi-Fi, invece, sono due dispositivi autonomi che si scambiano informazione: formano una rete. Anche un piccolo sensore che fa misure e le trasmette è un dispositivo autonomo. "Dispositivo di calcolo" non significa necessariamente "computer" in senso stretto.

Le reti, a loro volta, possono essere collegate tra loro formando reti più grandi. L'esempio per eccellenza è **Internet**, che è appunto una *rete di reti*.

L'unica trasmissione "vera" è quella fisica

Quando scaricate un film da un server, a voi sembra che il vostro programma parli direttamente con il programma sul server. In realtà l'informazione viaggia solo come **fenomeno fisico** (impulsi elettrici, luce, onde radio). Tutte le altre "conversazioni" tra programmi sono *virtuali*: poggiano sulla trasmissione fisica. Questa idea diventerà centrale nel prossimo capitolo, con il modello a strati.

A cosa servono le reti

Gli usi principali delle reti sono:

- **Accesso all'informazione**: pagine Web, basi di dati remote, file condivisi.
- **Comunicazione tra persone**: posta elettronica, messaggistica, videochiamate, VoIP.
- **Commercio elettronico**: acquisti online, home banking.
- **Intrattenimento**: streaming audio e video, giochi multiutente.
- **Internet of Things (IoT)**: oggetti "non informatici" connessi alla rete, come elettrodomestici, termostati, tapparelle, sensori, contatori dell'energia.

CLIENT E SERVER

Uno dei concetti fondamentali delle reti è la coppia **client–server** (cliente e servitore). L'idea è la stessa del bar: il cliente chiede un caffè, il barista lo serve. Il barista è sempre lì, pronto, e serve tanti clienti uno dopo l'altro.

Client e server

Un **server** è un'entità che **possiede un'informazione o offre un servizio** e resta in attesa di richieste. Un **client** è un'entità che **richiede esplicitamente** quell'informazione o quel servizio al server. Un singolo server gestisce tipicamente le richieste di centinaia o migliaia di client.

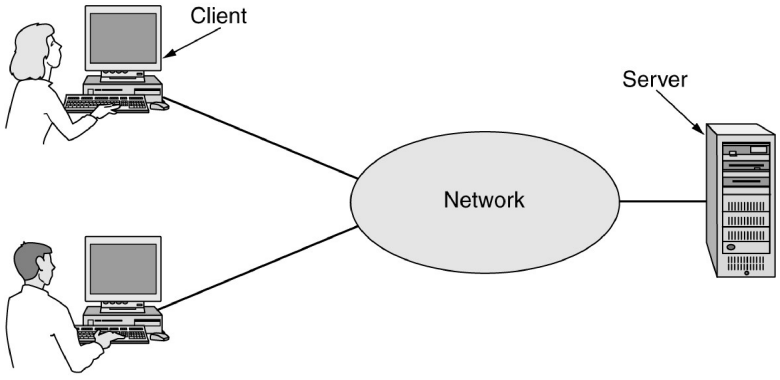


Figura 1.1 – Una rete con due client e un server (da Tanenbaum).

Esempi che usate già tutti i giorni: il *browser* è un client, il *server Web* dell'università è un server; il programma di posta sul telefono è un client, il server di posta di Ateneo, che custodisce la vostra casella e le vostre credenziali, è un server.

Client e server sono processi

Nelle figure disegniamo client e server come macchine, ma chi comunica davvero non è una macchina: è un **processo**, cioè un programma in esecuzione su quella macchina.

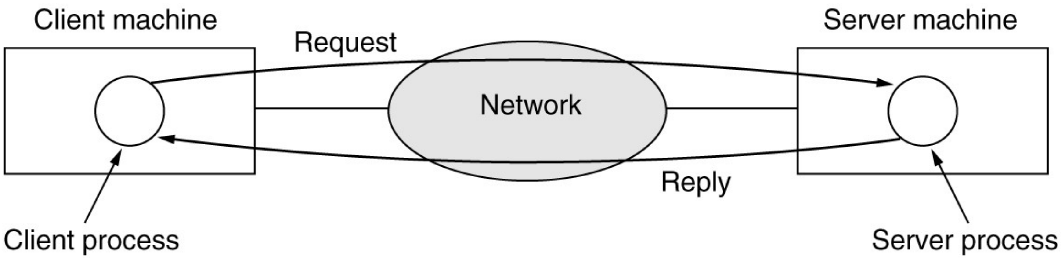


Figura 1.2 – La comunicazione avviene tra un processo client e un processo server.

Lo schema di interazione è sempre lo stesso: il processo client **invia un messaggio di richiesta** e poi **attende una risposta**; il processo server riceve la richiesta, esegue un'azione (per esempio legge un file dal disco) e **invia la risposta**.

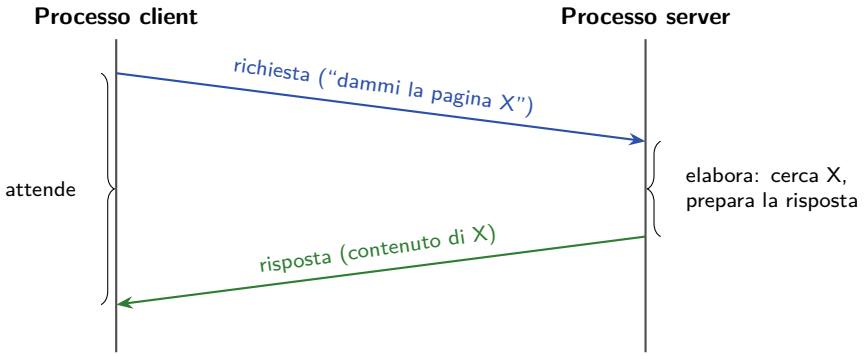


Figura 1.3 – Lo scambio richiesta–risposta tra due processi. Il tempo scorre verso il basso.

La forma di (quasi) tutto ciò che scriveremo

Due processi, uno per parte, che si scambiano messaggi secondo regole concordate: è la forma di quasi tutti i programmi che scriveremo in questo corso, a partire da quelli in C con i socket.

Peer-to-peer: niente ruoli fissi

L'architettura client-server non è l'unica possibile.

Peer-to-peer (P2P)

In un sistema **peer-to-peer** (da pari a pari) non esistono client e server fissi: ogni partecipante (*peer*) può agire **sia da client sia da server**. Non esiste un database centrale di ciò che è disponibile: ogni peer chiede ai peer che conosce.

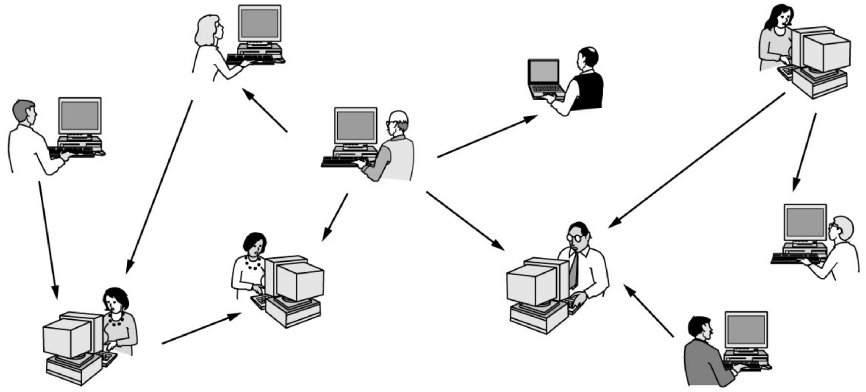


Figura 1.4 – In un sistema peer-to-peer ogni nodo può chiedere e fornire servizi.

Poiché non c'è un elenco centrale, i protocolli P2P devono prevedere dei meccanismi per **scoprire** gli altri partecipanti e ciò che offrono. In cambio, il carico è distribuito: ogni peer che scarica un file può anche ridistribuirlo ad altri (lo vedremo con BitTorrent).

	Client-server	**Peer-to-peer**
Ruoli	fissi: chi chiede e chi serve	ogni peer fa entrambe le cose
Punto centrale	il server (sempre acceso, indirizzo noto)	nessuno, o solo di supporto
Scalabilità	il server deve reggere tutti i client	ogni nuovo peer aggiunge capacità
Problemi tipici	costo e guasto del server	scoperta dei peer, sicurezza, affidabilità
Esempi	Web, posta elettronica	BitTorrent, alcune parti di VoIP

Tabella 1.1 – Confronto tra le due architetture.

INTERNET: UNA RETE DI RETI

Anche se la usiamo tutti i giorni, non è affatto ovvio che cosa sia Internet.

Internet

Internet è una **rete di reti** (*internetwork*): un insieme di reti **indipendenti**, con tecnologie diverse e gestite da organizzazioni diverse, collegate tra loro attraverso i **fornitori di accesso** (ISP) in modo da comportarsi come **un'unica rete**. Il nome stesso lo dice: *inter-net*, rete tra reti.

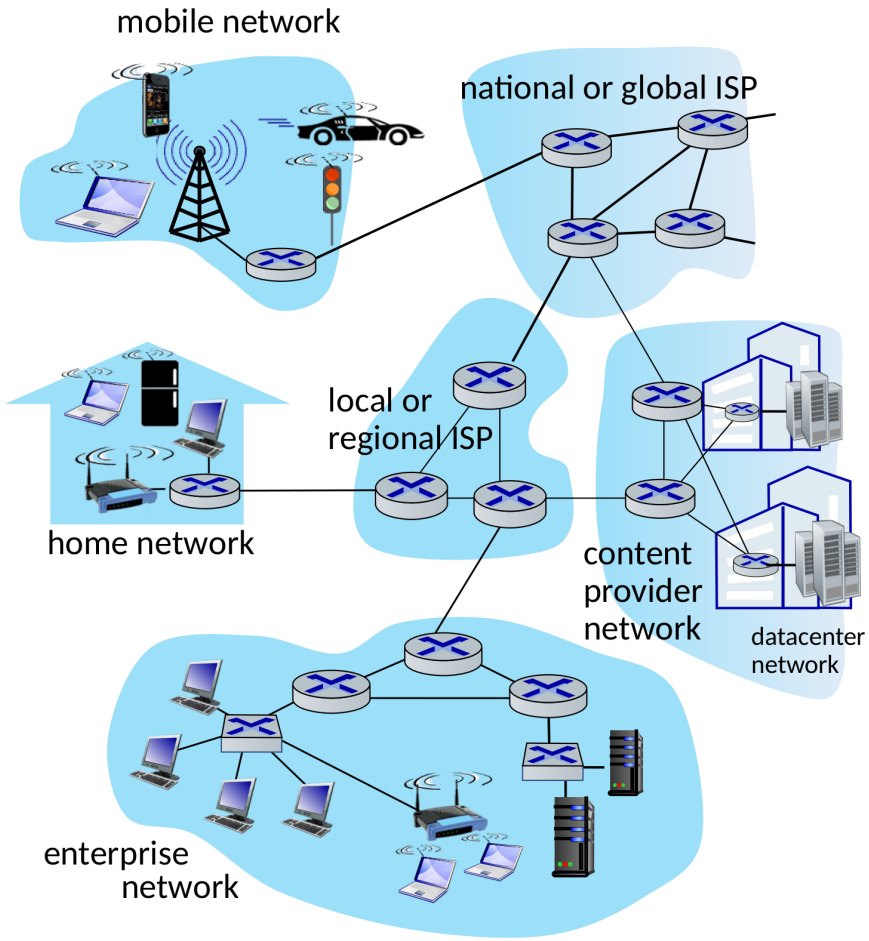


Figura 1.5 – Reti domestiche, aziendali, mobili e di data center, collegate attraverso gli ISP (da Kurose).

Guardiamo la figura partendo da casa nostra:

- La **rete di casa** (*home network*) collega telefoni, portatili, stampante, televisore. È gestita dal **router di casa**, in modo indipendente: il vostro fornitore di accesso non sa e non si occupa di come sono organizzati i dispositivi dentro casa vostra.
- Il router di casa si collega alla rete di un **ISP locale o regionale**, quello a cui pagate l'abbonamento.
- Allo stesso modo, la **rete aziendale** (*enterprise network*, per esempio la rete dell'Università) e la **rete mobile** del vostro operatore telefonico sono reti a sé, ciascuna con la propria gestione.
- Gli ISP locali e regionali sono a loro volta collegati a **ISP nazionali o globali**, che trasportano il traffico tra regioni e continenti.
- Accanto a loro ci sono le reti dei **content provider** e dei **data center** (Google, Netflix, …), che ospitano i servizi che usiamo.

Ogni rete è gestita e integrata nella rete dell'ISP a cui si appoggia; gli ISP sono integrati in ISP più grandi, e così via, fino a formare un'unica struttura: Internet.

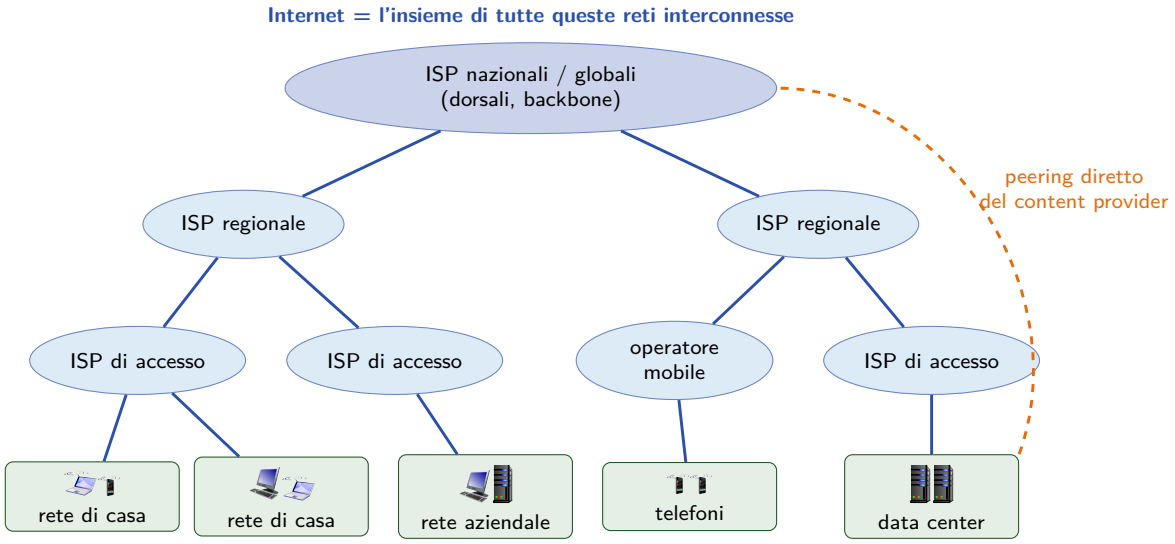


Figura 1.6 – Ogni rete si appoggia a un ISP, e gli ISP si appoggiano a ISP più grandi: il risultato è una rete di reti.

La cosa straordinaria è che, nonostante tecnologie, velocità, protocolli e contratti diversi, tutto *funziona come se fossimo sulla stessa rete*. Potete chiamare un parente in Nuova Zelanda: i vostri dati attraversano decine di reti diverse, ma l'applicazione non se ne accorge. Non è magia, è un'abilissima progettazione dei protocolli.

Il collante: l'Internet Protocol (IP)

Il protocollo che rende uniforme questa enorme eterogeneità è l'**Internet Protocol (IP)**.

Internet Protocol (IP)

L'**Internet Protocol** è il protocollo che permette di consegnare pacchetti di dati (**datagrammi**) da un host sorgente a un host destinazione attraverso una rete di reti. Ogni interfaccia collegata a Internet ha un **indirizzo IP**, e ogni datagramma porta con sé l'indirizzo del mittente e quello del destinatario. I router leggono l'indirizzo di destinazione e inoltrano il datagramma, salto dopo salto, verso la rete giusta.

Tre caratteristiche di IP da ricordare fin da ora:

- **È comune a tutti**: qualunque tecnologia ci sia sotto (Ethernet, Wi-Fi, fibra, 4G/5G) e qualunque applicazione ci sia sopra (Web, posta, videochiamate), in mezzo c'è sempre IP. È l'unico punto su cui tutti devono essere d'accordo.
- **Nasconde le differenze**: le applicazioni vedono un'unica rete, non le decine di reti attraversate.
- **È "best effort"**: IP fa del suo meglio per consegnare i datagrammi, ma non garantisce né che arrivino, né che arrivino in ordine. Le garanzie, quando servono, vengono aggiunte sopra (da TCP, come vedremo).

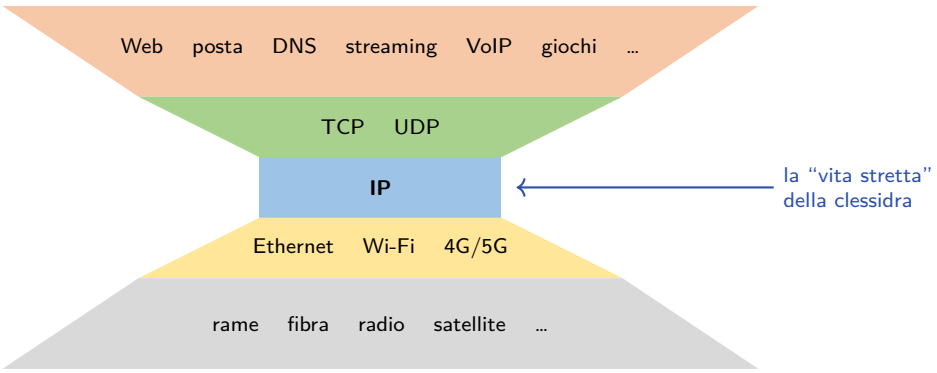


Figura 1.7 – La clessidra di Internet: tante applicazioni sopra, tante tecnologie sotto, un solo protocollo in mezzo.

Studieremo IP in dettaglio nella Parte V; per ora basta sapere che è lui a tenere insieme Internet.

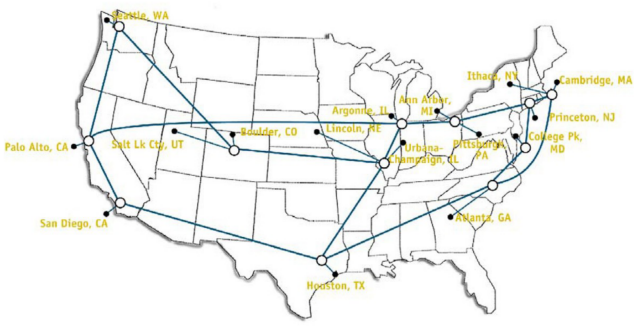
Breve storia di Internet

L'idea di una rete "universale" fu teorizzata da alcuni ricercatori (il più noto è J.C.R. Licklider) già negli anni '50–'60. Le tappe da ricordare sono poche.

Figura 1.8 – Le tappe principali: da rete di ricerca a Internet commerciale.

- **1969** — **ARPANET**. L'agenzia ARPA del Dipartimento della Difesa statunitense finanzia una rete di ricerca per università e centri di ricerca. Alla fine del 1969 ha quattro nodi: UCLA, SRI (Stanford Research Institute), UC Santa Barbara e Utah. Reti simili nascevano in modo indipendente negli USA e in Europa.
- **1974** — **TCP/IP**. Robert Kahn e Vinton Cerf progettano il *Transmission Control Protocol* (TCP) e l'*Internet Protocol* (IP), i protocolli su cui poggia ancora oggi Internet.
- **1986** — **NSFNET**. La National Science Foundation finanzia una dorsale basata su TCP/IP. Si va verso l'idea di rete di reti, ma sia ARPANET sia NSFNET vietano il traffico commerciale.
- **1990** — ARPANET viene dismessa.
- **1995** — anche NSFNET viene dismessa e con lei cade l'ultima restrizione al traffico commerciale: nella seconda metà degli anni '90 nasce l'Internet che usiamo.

(a) ARPANET nel 1969



(b) la dorsale NSFNET nel 1992

Figura 1.9 – Due fotografie della rete prima dell'Internet commerciale.

Perché la storia conta

Internet non è stata progettata come un sistema unico e coerente: è cresciuta **aggregando reti preesistenti e diverse tra loro**. Per questo i suoi protocolli sono progettati per tollerare l'eterogeneità. Prima di Internet esistevano reti non integrate tra loro (per esempio il Minitel francese), lente e poco pervasive, che non ebbero lo stesso impatto.

Di cosa è fatta una rete: dispositivi, collegamenti, host

Fisicamente, una rete è composta da due grandi famiglie di elementi.

Infrastruttura e host

- L'**infrastruttura** è l'insieme dei **dispositivi di rete** e dei **collegamenti** (*link*) tra loro. È dedicata esclusivamente a far comunicare i dispositivi, non a eseguire applicazioni per gli utenti.
- Gli **host**, detti anche **sistemi terminali** (*end systems*), sono le macchine che **eseguono le applicazioni** e che producono o consumano i dati.

I dispositivi dell'infrastruttura

Nelle figure di rete si usano dei simboli standard, in larga parte introdotti da Cisco, per indicare i dispositivi. Conviene impararli subito.

Simbolo	Dispositivo	Che cosa fa
	Router	Collega **reti diverse** e inoltra i pacchetti verso la rete di destinazione, scegliendo ogni volta da quale collegamento farli uscire (in base all'indirizzo IP). È il dispositivo che tiene insieme Internet.
	Access point	Punto di accesso **senza fili**: collega i dispositivi Wi-Fi alla rete cablata, facendo da "ponte" tra le onde radio e il cavo.
	Switch	Collega più dispositivi **della stessa rete locale**. Riceve un frame su una porta e lo inoltra **solo** sulla porta a cui è collegato il destinatario (in base all'indirizzo fisico, detto MAC).
	Hub	Il "fratello povero" dello switch: tutto ciò che riceve su una porta lo **ripete su tutte le altre**, senza guardare a chi è destinato. Oggi è quasi del tutto sostituito dagli switch.

Tabella 1.2 – I principali dispositivi di rete e i loro simboli.

Il router di casa fa tante cose insieme

Il "router" che l'operatore vi installa in casa è in realtà un apparecchio che contiene *più* dispositivi: un router (verso l'ISP), uno switch (le porte Ethernet sul retro), un access point (il Wi-Fi) e spesso anche il modem. Nelle figure li disegneremo separati.

Gli host

Laptop Server Smartphone PC

Sono host il portatile, lo smartphone, il PC, il server del sistema informativo di Ateneo, il Raspberry che a casa fa da server per i file: tutte macchine che **eseguono applicazioni**.

Commutatore di pacchetto (packet switch)

Router e switch sono entrambi **commutatori di pacchetto**: **ricevono** un pacchetto di dati da un collegamento e lo **inoltrano** su un altro collegamento.

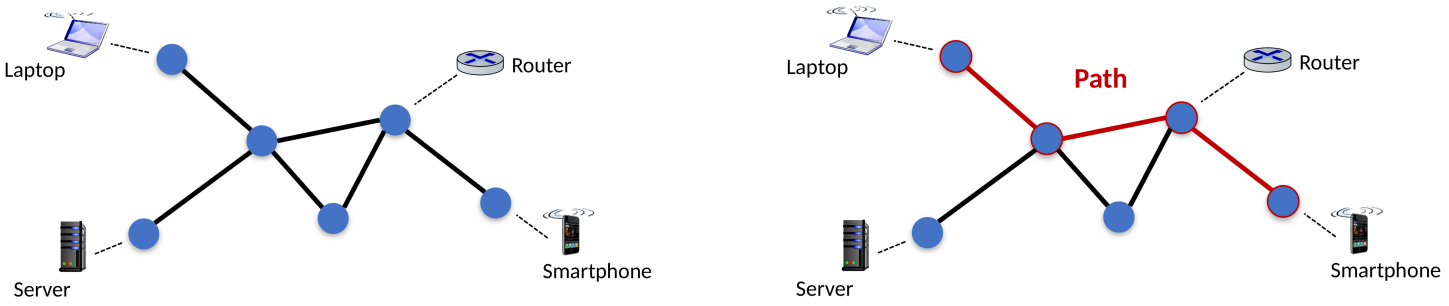
Nodo non vuol dire host

Un router è un **nodo** della rete ma **non è un host**: non esegue applicazioni per gli utenti. Come vedremo nel prossimo capitolo, non implementa nemmeno tutti gli strati della pila protocollare.

Le reti condividono il vocabolario con la **teoria dei grafi**.

Nodi, collegamenti e cammini

I dispositivi connessi alla rete sono i **nodi**; le connessioni tra nodi sono i **collegamenti** (*link*, o canali di comunicazione). Una sequenza di nodi e collegamenti che porta da un estremo all'altro è un **cammino** (*path*). Gli **host** sono i nodi speciali che forniscono o usano servizi.



(a) la rete come grafo

(b) un cammino tra due host

Figura 1.10 – Idealmente gli host sono le foglie del grafo e i nodi intermedi inoltrano i dati.

Idealmente gli host stanno sulle **foglie** del grafo, mentre i nodi intermedi (router, switch) si occupano di inoltrare i dati (nella pratica non sempre è così). Se telefonate a un parente in Nuova Zelanda non esiste un cavo diretto tra i due telefoni: i dati seguono un cammino attraverso molti nodi. A meno di stendere un cavo dedicato, cosa che alcune organizzazioni con esigenze di sicurezza o affidabilità fanno davvero.

Le cinque componenti di una comunicazione

Ogni comunicazione di dati, qualunque sia la tecnologia usata, ha cinque componenti.

Figura 1.11 – Le cinque componenti: (1) messaggio, (2) mittente, (3) destinatario, (4) mezzo, (5) protocollo.

1. **Messaggio**: i dati da trasmettere.
2. **Mittente** (*sender*): l'entità che invia il messaggio.
3. **Destinatario** (*receiver*): l'entità che deve ricevere il messaggio.
4. **Mezzo** (*medium*): il canale fisico attraverso cui il messaggio viaggia.
5. **Protocollo**: l'insieme di regole, note a entrambi, che fanno funzionare lo scambio.

L'ultima componente è la più importante: senza un protocollo condiviso, i bit che arrivano al destinatario sono solo rumore.

Protocollo

Un **protocollo** definisce il **formato** e l'**ordine** dei messaggi scambiati tra due o più entità comunicanti, e le **azioni** intraprese quando un messaggio viene inviato o ricevuto.

I protocolli esistono anche nella vita di tutti i giorni. Quando rispondete al telefono dite "Pronto" (in un'azienda si risponde con il nome della ditta); chi chiama aspetta che l'altro parli per primo; mentre uno parla, l'altro non dovrebbe parlare. Chiedere l'ora a uno sconosciuto ha la stessa struttura: un saluto, un saluto di risposta, poi la domanda vera.

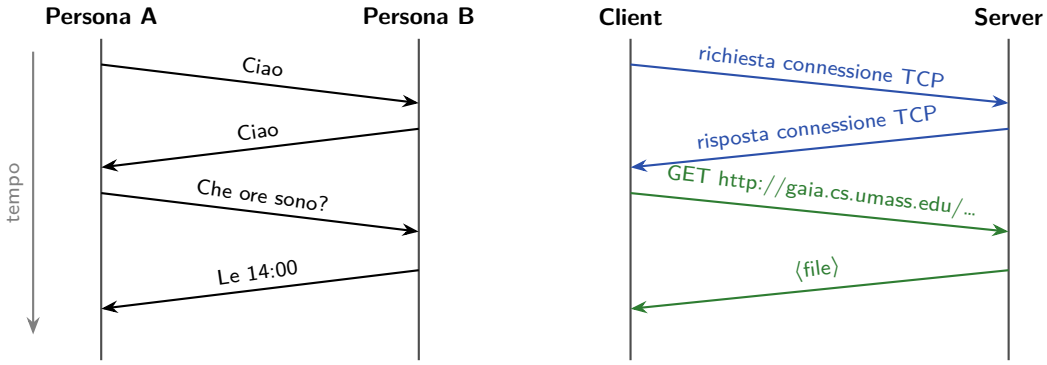


Figura 1.12 – Un protocollo umano e un protocollo di rete hanno la stessa forma: apertura, richiesta, risposta.

- Il **formato** dice come sono fatti i messaggi (come interpretare i bit che arrivano, quali sono dati e quali controllo).
- L'**ordine** dice chi parla per primo e come si risponde.
- Le **azioni** dicono cosa fare quando arriva un messaggio: se chiedete un file, il server deve andarlo a prendere dal proprio file system e trasmetterlo.

Protocolli diversi: nessuna comunicazione

Due entità che usano protocolli diversi non riescono a svolgere alcun lavoro utile, anche se sono perfettamente collegate. Tutto ciò che nel corso coinvolge due parti remote è governato da un protocollo.

Rappresentazione e flusso dei dati

I dati possono essere testo, numeri, immagini, audio o video: sul mezzo trasmissivo sono **sempre bit**. Ciò che cambia, a seconda della comunicazione, è la **direzione** in cui i bit possono fluire su un collegamento.

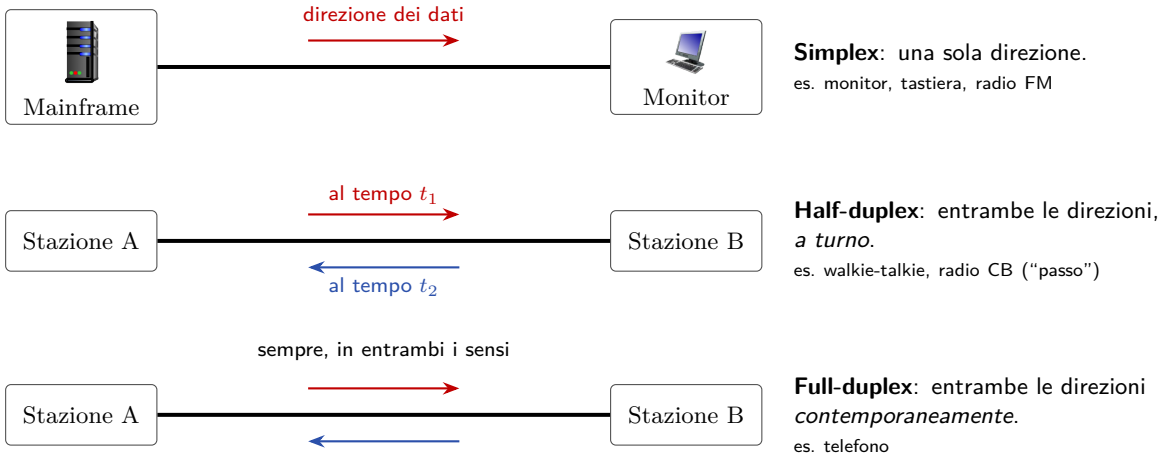


Figura 1.13 – Le tre modalità di flusso dei dati su un collegamento.

- **Simplex**: monodirezionale. Un estremo trasmette sempre, l'altro riceve sempre.
- **Half-duplex**: bidirezionale ma a turni. Quando A trasmette a B, B non può trasmettere ad A, e viceversa. Con le radio CB si parla, si dice "passo" e si lascia il canale all'altro.
- **Full-duplex**: bidirezionale e simultaneo, come in una telefonata.

Rate, banda e throughput

Tre grandezze facili da confondere, tutte misurate in bit al secondo (bit/s o b/s).

Transmission rate, bandwidth, throughput

- Il **transmission rate** (velocità di trasmissione) è il **massimo** che un **dispositivo** riesce a trasmettere.
- La **bandwidth** (larghezza di banda) è il **massimo** che un **cammino** (collegamenti e nodi insieme) riesce a trasportare.
- Il **throughput** è la quantità di dati che **sta effettivamente fluendo** lungo quel cammino in un certo istante.

I primi due sono *tetti*, il terzo è una *misura*. Il throughput non supera mai la banda.

L'analogia del tubo per innaffiare

Il tubo per innaffiare il giardino sopporta al massimo un certo flusso d'acqua: è la sua *banda*. Ma non siete obbligati ad aprire il rubinetto al massimo: l'acqua che sta passando adesso è il *throughput*. E se il tubo ha un tratto più stretto, tutto il flusso è limitato da quel tratto.

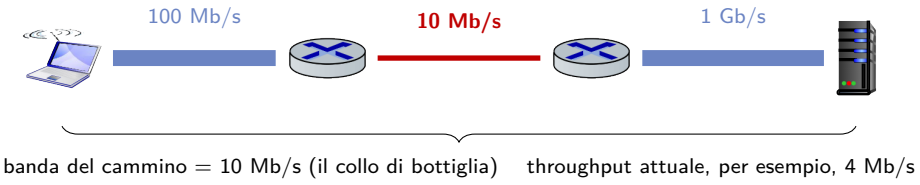


Figura 1.14 – La banda di un cammino è limitata dal suo collegamento più lento.

Due significati di "banda"

In questo capitolo *banda* indica una **velocità** (bit/s). Quando arriveremo al livello fisico la stessa parola indicherà anche una **larghezza in frequenza** (Hz): l'intervallo di frequenze usato dal segnale elettromagnetico. Sono concetti legati ma diversi.

PACCHETTI E STORE-AND-FORWARD

I messaggi lunghi non viaggiano "in un pezzo solo": vengono spezzati in **pacchetti**, ciascuno con un proprio **header** (intestazione), proprio come ogni lettera ha la sua busta con l'indirizzo.

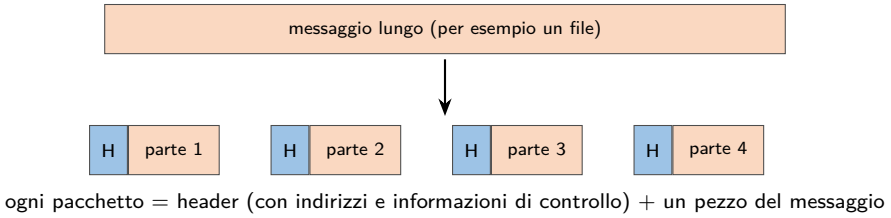


Figura 1.15 – Un messaggio viene diviso in pacchetti, ciascuno con il proprio header.

Perché dividere? Gestire un flusso continuo di bit sarebbe complicatissimo: se si verifica un errore bisognerebbe ritrasmettere tutto. Con i pacchetti si ritrasmette **solo il pacchetto danneggiato**, e più comunicazioni possono condividere lo stesso collegamento alternando i propri pacchetti.

Tempo di trasmissione e store-and-forward

- Un pacchetto di L bit su un collegamento di rate R bit/s richiede $\dfrac{L}{R}$ secondi per essere trasmesso.
- Un commutatore di pacchetto è **store-and-forward** (memorizza e inoltra): deve **ricevere l'intero pacchetto** prima di poterne trasmettere il primo bit sul collegamento successivo.
- Attraverso N collegamenti di rate R, senza altro traffico, un pacchetto impiega $N \cdot \dfrac{L}{R}$ secondi.

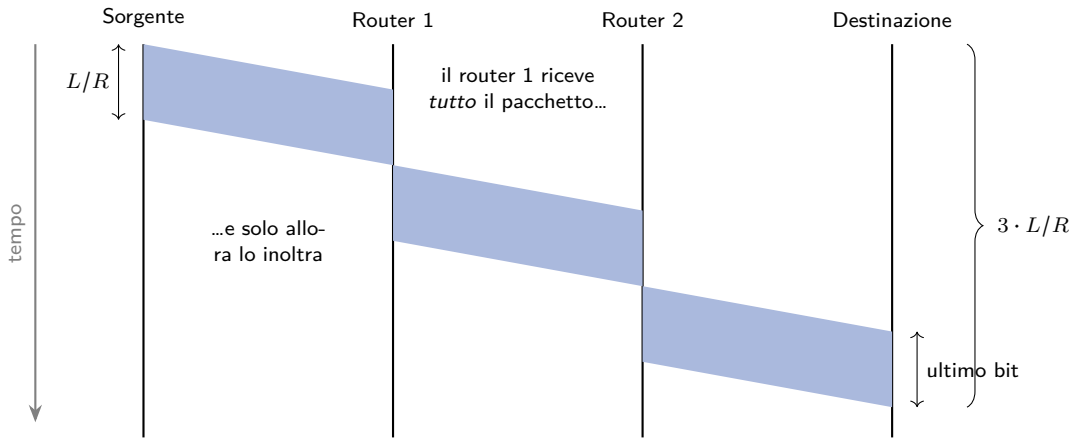


Figura 1.16 – Store-and-forward su tre collegamenti: ogni nodo aspetta l'intero pacchetto prima di inoltrarlo.

Esempio 2 — Quanto ci mette un pacchetto

Un pacchetto di $L = 10\,000$ bit attraversa $N = 3$ collegamenti da $R = 1$ Mb/s. Ogni trasmissione dura $L/R = 10^4/10^6 = 0{,}01$ s, quindi il ritardo complessivo di trasmissione è $3 \cdot 0{,}01 = 0{,}03$ s (trascurando la propagazione e le code).

Code e perdite

Ogni collegamento in uscita da un commutatore ha una **coda** (*buffer*): tutto ciò che deve essere trasmesso si mette in fila.

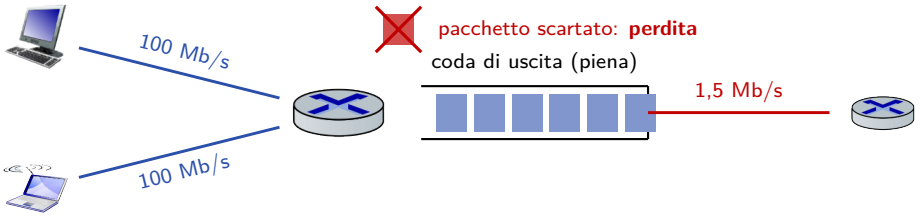


Figura 1.17 – Se i pacchetti arrivano più velocemente di quanto il collegamento riesca a smaltirli, si accodano; a coda piena vengono scartati.

Ritardo di accodamento e perdita

Se i pacchetti arrivano più velocemente di quanto il collegamento in uscita riesca a trasmetterli, **aspettano in coda** (ritardo di accodamento). Se la coda è piena, il pacchetto in arrivo viene **scartato**: è la **perdita di pacchetti** (*packet loss*).

Da qui nascono le perdite su cui costruiremo gran parte del corso: TCP, per esempio, esiste proprio per recuperarle.

Commutazione di circuito e commutazione di pacchetto

Ci sono due modi fondamentali di far attraversare una rete ai dati.

Commutazione di circuito

Commutazione di circuito (circuit switching)

Prima di trasmettere, la rete **riserva** lungo tutto il cammino una certa capacità di trasmissione, e la **mantiene dedicata** per tutta la durata della sessione; alla fine il circuito viene rilasciato. È il modello della rete **telefonica** tradizionale.

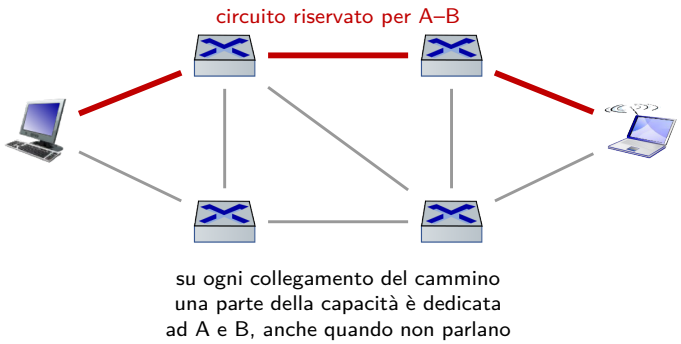


Figura 1.18 – Commutazione di circuito: un cammino e la sua capacità vengono riservati per l'intera sessione.

Come si "ritaglia" un circuito da un collegamento condiviso da più utenti? In due modi:

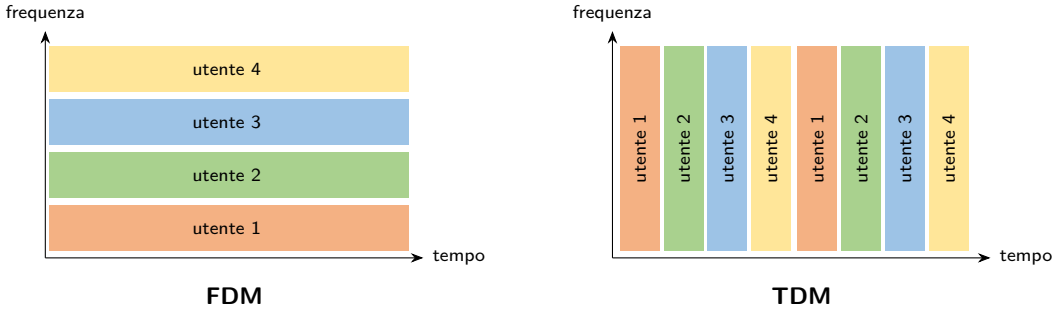


Figura 1.19 – FDM: ogni utente ha una propria banda di frequenze, sempre. TDM: ogni utente ha tutta la banda, ma solo nel proprio intervallo di tempo (*slot*), che si ripete ciclicamente.

- **FDM** (*Frequency Division Multiplexing*): lo spettro di frequenze del collegamento viene diviso in bande, e ogni circuito ne riceve una (come le stazioni radio).
- **TDM** (*Time Division Multiplexing*): il tempo viene diviso in trame, ogni trama in intervalli (*slot*), e ogni circuito riceve uno slot per trama.

Commutazione di pacchetto

Commutazione di pacchetto (packet switching)

La rete **non riserva nulla** in anticipo. Ogni pacchetto viene inviato indipendentemente e usa i collegamenti **su richiesta**; se un collegamento è occupato, il pacchetto **attende in coda**. È il modello di Internet.

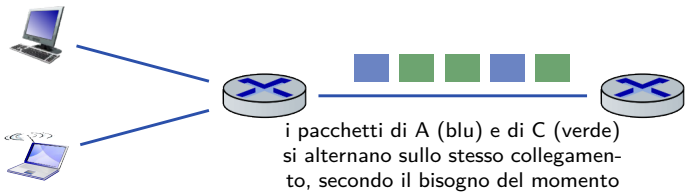


Figura 1.20 – Commutazione di pacchetto: la capacità del collegamento è condivisa dinamicamente (multiplexing statistico).

Perché Internet ha scelto i pacchetti

Il traffico dei calcolatori è **a raffica** (*bursty*): brevi periodi di intensa attività (si carica una pagina) alternati a lunghi silenzi (la si legge). Riservare un circuito significherebbe tenere capacità inutilizzata per la maggior parte del tempo.

Esempio 3 — 10 utenti contro 35 utenti

Un collegamento da 1 Mb/s è condiviso da utenti che, quando sono attivi, trasmettono a 100 kb/s, ma sono attivi solo il 10% del tempo.

- Con la **commutazione di circuito** a ogni utente vanno riservati 100 kb/s sempre: il collegamento serve al massimo $1\,000/100 = \mathbf{10}$ utenti.
- Con la **commutazione di pacchetto** si possono servire **35** utenti: la probabilità che più di 10 di loro siano attivi *nello stesso istante* è circa 0,0004. Quasi sempre, quindi, il collegamento basta per tutti, con le stesse prestazioni percepite.

	Circuito	**Pacchetto**
Riserva di risorse	sì, per tutta la sessione	no, uso su richiesta
Prestazioni	garantite, costanti	variabili: code, ritardi, perdite
Uso della capacità	spreco nei silenzi	efficiente (multiplexing statistico)
Fase preliminare	instaurazione del circuito	nessuna
Esempio	rete telefonica tradizionale	Internet

Tabella 1.3 – Commutazione di circuito e di pacchetto a confronto.

Anche la voce viaggia a pacchetti

Quando telefonate con un'app di messaggistica non viene riservato alcun cammino: la vostra voce viene codificata e spedita a pacchetti (tipicamente con UDP, che vedremo), sperando che arrivino in tempo per ricostruire l'audio dall'altra parte.

COLLEGAMENTI PUNTO-PUNTO E MULTIPUNTO

Punto-punto e multipunto

- In un collegamento **punto-punto** esiste un collegamento **dedicato** tra **esattamente due** dispositivi, cablato o senza fili.
- In un collegamento **multipunto** (o *broadcast*) **più dispositivi condividono lo stesso collegamento**, cablato o senza fili: quando uno trasmette, tutti gli altri lo sentono.

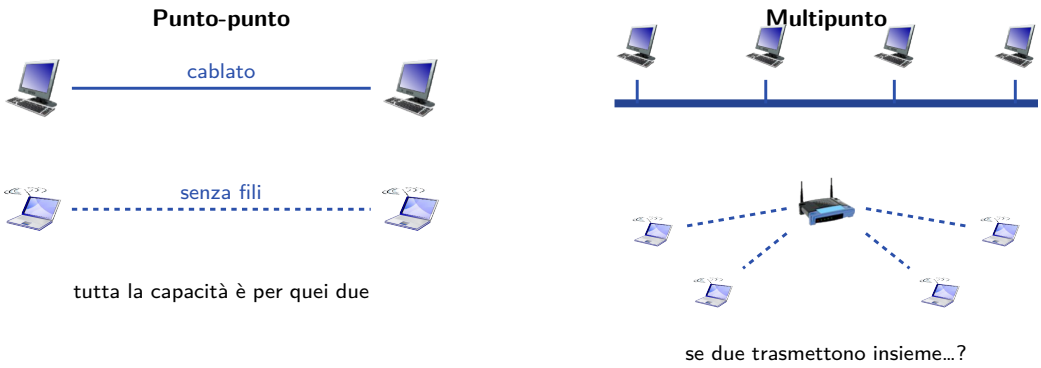


Figura 1.21 – Nel punto-punto il collegamento è di due soli dispositivi; nel multipunto il mezzo (un cavo condiviso o l'etere) è di tutti.

Il problema del mezzo condiviso

Se due dispositivi su un collegamento multipunto trasmettono nello stesso istante, i segnali si sovrappongono e diventano incomprensibili: è una **collisione**. A gestire l'accesso al mezzo condiviso è un sottostrato del livello di collegamento chiamato **MAC** (*Medium Access Control*), che studieremo nella Parte VI.

La distinzione non è solo fisica: pesa anche sul progetto dei protocolli dei livelli superiori. Notate che la rete Wi-Fi di casa, dal punto di vista logico, è un mezzo condiviso: tutti i dispositivi "parlano nell'aria" vicino allo stesso access point.

Topologie di rete

Topologia di rete

La **topologia** di una rete è la disposizione dei suoi nodi e dei suoi collegamenti: **chi è collegato fisicamente con chi**, indipendentemente da ciò che gira sopra.

Perché si chiama "topologia"

Il nome viene dal greco *tópos*, "luogo". In matematica la topologia studia le proprietà che non cambiano deformando un oggetto senza strapparlo: **le distanze non contano**, conta solo **chi è connesso con chi**. Allo stesso modo, la topologia di una rete ignora quanto sono lunghi i cavi e dove stanno fisicamente gli apparati.

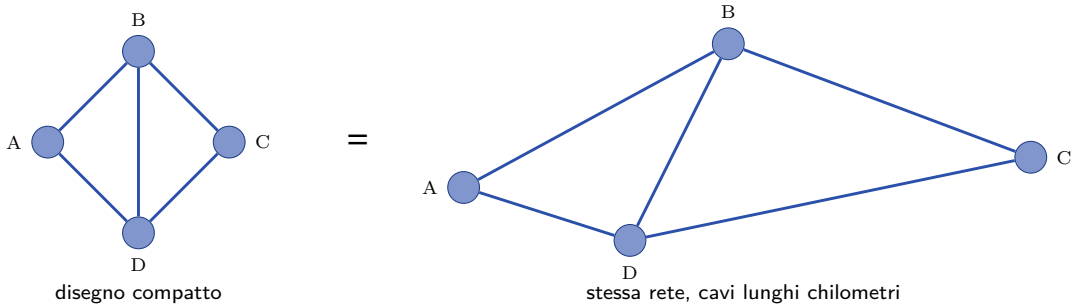


Figura 1.22 – Due disegni della stessa topologia: le lunghezze cambiano, i collegamenti no.

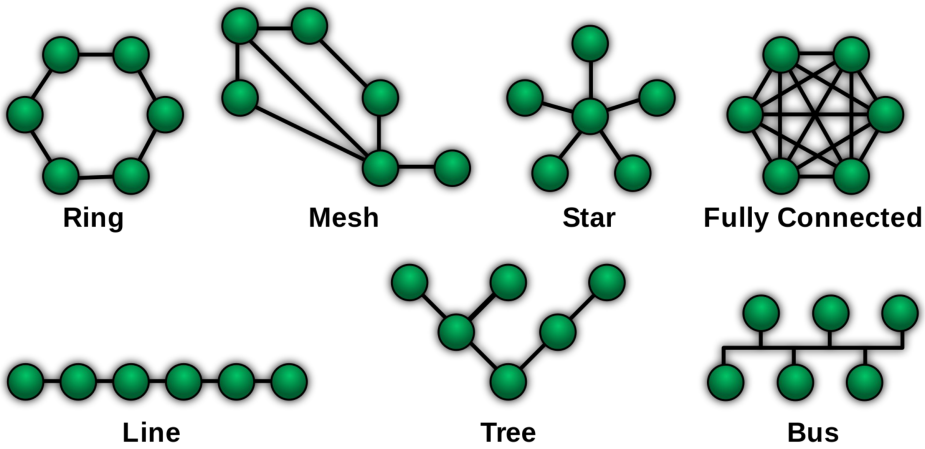
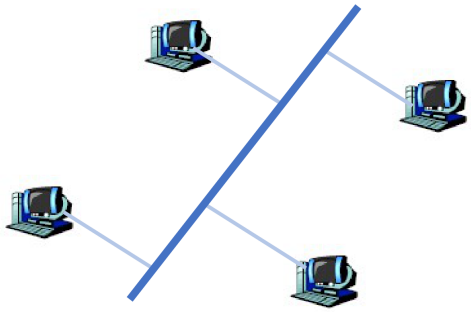


Figura 1.23 – Le topologie più comuni: anello, maglia, stella, completamente connessa, linea, albero, bus.

Bus

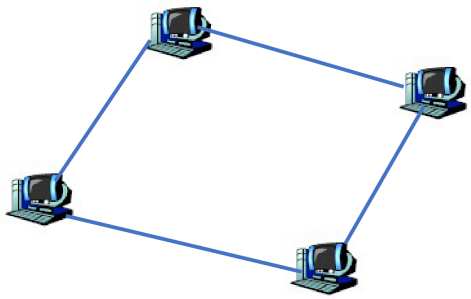


Ogni host si collega a **un unico cavo condiviso** (il *bus* o *backbone*): ogni trasmissione raggiunge tutti, e le **collisioni sono possibili**. È la topologia delle prime reti Ethernet.

- ✔ semplice ed economica, adatta a reti piccole;
- ✔ se si guasta il collegamento di un singolo host, gli altri non se ne accorgono;
- ✖ se si **interrompe il bus**, la rete si spezza in due.

Collegamenti: 1 backbone e n collegamenti di accesso $(1 + n)$.

Anello (ring)

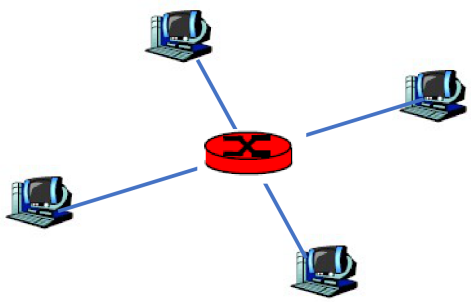


Ogni host è collegato punto-punto con **esattamente altri due** (quello che lo precede e quello che lo segue); il segnale viene inoltrato lungo l'anello, da un nodo all'altro, fino a destinazione. Un esempio storico è il *Token Ring* di IBM.

- ✔ economica, prestazioni migliori del bus;
- ✖ aggiungere un nodo significa "aprire" l'anello;
- ✖ come in una catena, l'anello più debole blocca tutto: un nodo malfunzionante può compromettere l'intera rete.

Collegamenti: n collegamenti duplex.

Stella (star)

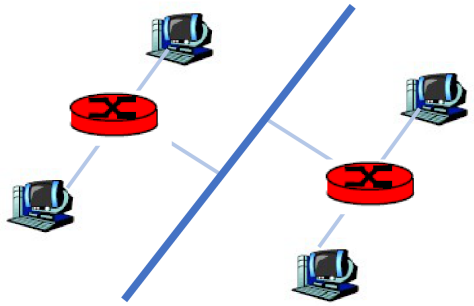


Gli host sono collegati **solo a un controllore centrale** (hub, switch o router), che smista tutti i messaggi; non ci sono collegamenti diretti tra host. Il router di casa, con i dispositivi collegati ad esso, forma proprio una stella.

- ✔ economica, semplice e **scalabile**: aggiungere un host = aggiungere un collegamento;
- ✔ se un host si guasta, gli altri continuano a funzionare;
- ✖ ogni host deve poter raggiungere il controllore;
- ✖ il controllore è un **single point of failure**: se si guasta, non funziona più nulla.

Collegamenti: 1 controllore e n collegamenti duplex.

Albero (tree)



Più stelle collegate attraverso una dorsale (un bus): è la forma che ha quasi ogni rete reale di un edificio (una stella per piano, collegate da una dorsale).

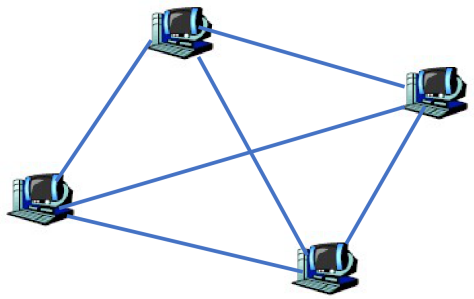
- ✔ versatile, scalabile, ben supportata dai produttori di hardware e software;
- ✖ più difficile da configurare di una singola stella: i controllori devono coordinarsi;
- ✖ eredita la debolezza della dorsale: se si interrompe, si stacca un intero sottoalbero.

Linea (line)

Figura 1.24 – Topologia a linea (o a catena): n nodi e $n-1$ collegamenti.

I nodi sono disposti in sequenza, ciascuno collegato al precedente e al successivo (ma, a differenza dell'anello, il primo e l'ultimo non si chiudono). Servono solo $n-1$ collegamenti, ma i messaggi tra nodi lontani attraversano tutti quelli in mezzo e un solo guasto divide la rete.

Maglia (mesh)



Gli host sono collegati punto-punto tra loro, **senza gerarchia**:

- **full mesh** (completamente connessa): ogni nodo è collegato a tutti gli altri;
- **partial mesh**: ogni nodo è collegato solo ad alcuni.

- ✔ collegamenti **dedicati**: niente contesa come nel bus, buona robustezza;
- ✔ se un collegamento cade, il traffico prende un'altra strada;
- ✖ ogni nodo ha bisogno di tante porte quanti sono i suoi vicini;
- ✖ **non scala**: una full mesh ha $\frac{n(n-1)}{2}$ collegamenti.

Perché una full mesh ha $n(n-1)/2$ collegamenti

In una full mesh ogni nodo è collegato a **tutti gli altri** $n-1$ nodi.

1. Ogni nodo ha quindi $n-1$ collegamenti (e $n-1$ porte). Sommando su tutti gli n nodi si ottengono $n(n-1)$ "estremità di collegamento".
2. Ma ogni collegamento ha **due** estremità: il collegamento A–B è stato contato una volta partendo da A e una volta partendo da B.
3. Dividendo per 2 si ottiene il numero di collegamenti: $\frac{n(n-1)}{2}$.

In modo equivalente, il numero di collegamenti è il numero di **coppie non ordinate** di nodi, cioè $\binom{n}{2}$; oppure si può contare aggiungendo i nodi uno alla volta: il secondo nodo porta 1 collegamento nuovo, il terzo 2, il quarto 3, …, l'n-esimo $n-1$, e $1+2+\cdots+(n-1)=\dfrac{n(n-1)}{2}$.

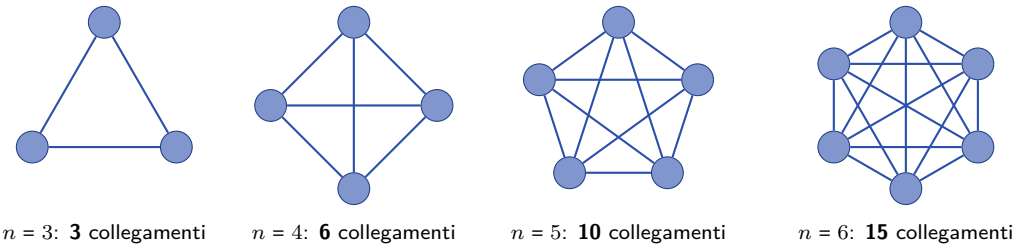


Figura 1.25 – Full mesh con 3, 4, 5 e 6 nodi: i collegamenti crescono con il quadrato dei nodi.

Esempio 4 — La crescita quadratica

Con $n = 10$ nodi servono $\dfrac{10\cdot 9}{2} = 45$ collegamenti; con $n = 100$ ne servono $\dfrac{100\cdot 99}{2} = 4\,950$, e ogni nodo avrebbe bisogno di 99 porte. Ecco perché la full mesh è impraticabile oltre poche decine di nodi, e perché gli ISP non si collegano "tutti con tutti" (lo rivedremo parlando di IXP).

Topologie ibride

Le reti reali **mescolano** le topologie: ogni parte è scelta per il proprio segmento, e il risultato non è nessuna delle forme componenti.

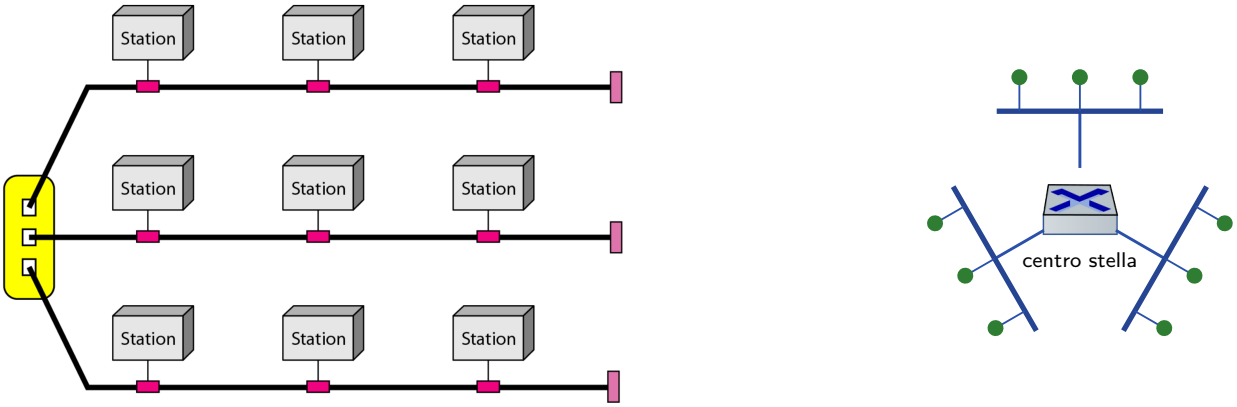


Figura 1.26 – Una dorsale a stella che collega tre reti a bus: una topologia ibrida.

Nell'esempio il centro della stella non è collegato a singoli host, ma a tre *bus*, ciascuno con i propri host. Le reti della TV via cavo, come vedremo, hanno una struttura ibrida di questo tipo (albero di bus).

Topologia	Collegamenti	Se si guasta un host	Se si guasta il mezzo
Bus	$1 + n$	nessun effetto sugli altri	la rete si divide
Anello	n	l'anello può interrompersi	l'anello può interrompersi
Stella	n	nessun effetto sugli altri	solo quell'host resta isolato
Albero	n + dorsale	nessun effetto sugli altri	si isola un intero sottoalbero
Linea	$n - 1$	la linea si spezza	la linea si spezza
Mesh (full)	$n(n-1)/2$	il traffico usa altri collegamenti	il traffico usa altri collegamenti

Tabella 1.4 – Robustezza e costo delle topologie (n = numero di host).

Robustezza contro costo

La robustezza si compra con i collegamenti, e i collegamenti si comprano con porte e cavi: ogni rete reale è un compromesso tra le ultime due colonne della tabella e la seconda.

E la riservatezza?

Una domanda frequente: una topologia come il bus, dove tutti ricevono tutto, è insicura? La riservatezza **non** dipende dalla topologia: è garantita da protocolli crittografici che funzionano a prescindere da chi può ascoltare le trasmissioni. Lo vedremo nella Parte VII.

CATEGORIE DI RETI PER ESTENSIONE

Le reti si classificano in base alla **distanza** che coprono. Non è un dettaglio: la distanza determina le **tecnologie** utilizzabili (quanto costa un cavo, quanto ritardo si accumula, quanti errori ci sono).

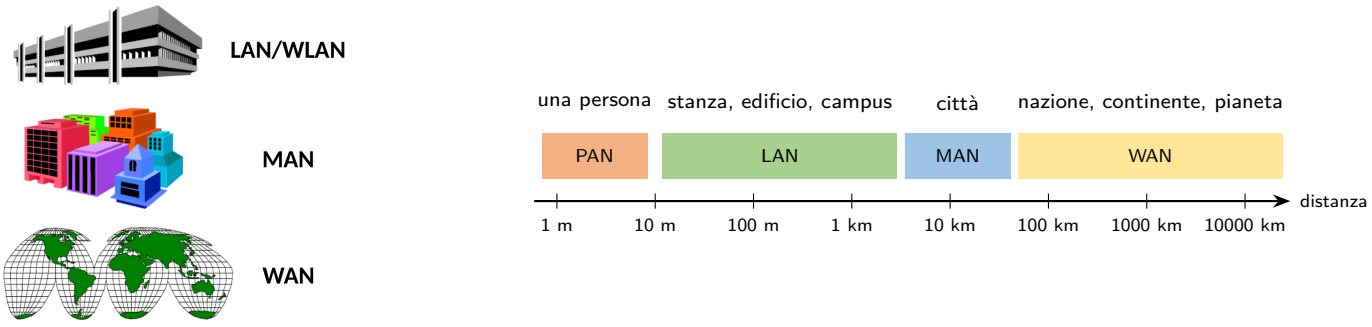


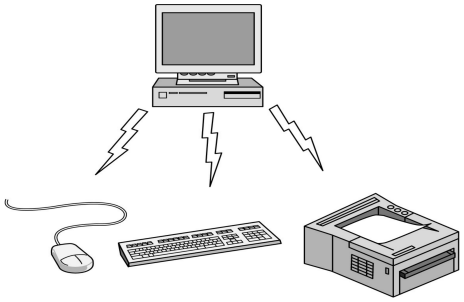
Figura 1.27 – Le categorie di reti per estensione (i confini sono indicativi).

- **PAN** (*Personal Area Network*): attorno a una persona, pochi metri.
- **LAN** (*Local Area Network*): un edificio o un campus.
- **MAN** (*Metropolitan Area Network*): una città.
- **WAN** (*Wide Area Network*): una nazione, un continente o più.

Zone grigie

La classificazione si fa “a spanne”: i confini non sono rigidi e una rete può stare a cavallo di due categorie. L’importante è capire l’ordine di grandezza e le conseguenze tecnologiche.

PAN --- Personal Area Network



Una PAN collega i dispositivi nel raggio d’azione di una persona: cuffie, tastiera, mouse, orologio, stampante. L’esempio tipico è **Bluetooth**, una rete senza fili a corto raggio che elimina i cavi da cercare e collegare. Bluetooth funziona in modalità **master–slave**: il calcolatore (master) dice alle periferiche (slave) quando e su quale frequenza possono trasmettere.

Figura 1.28 – Una PAN Bluetooth.

LAN --- Local Area Network

LAN

Una **LAN** è una rete **privata** all’interno di un edificio o di un campus. Le sue dimensioni sono limitate, quindi il **tempo di trasmissione nel caso peggiore è noto** in anticipo; ha **bassa latenza**, **pochi errori** e oggi velocità da 100 Mb/s a decine di Gb/s.

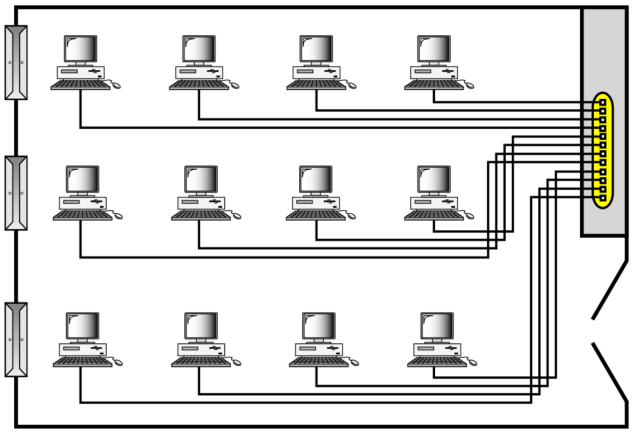


Figura 1.29 – Una LAN che collega dodici computer a uno switch.

Le LAN possono essere **senza fili** o **cablate**:

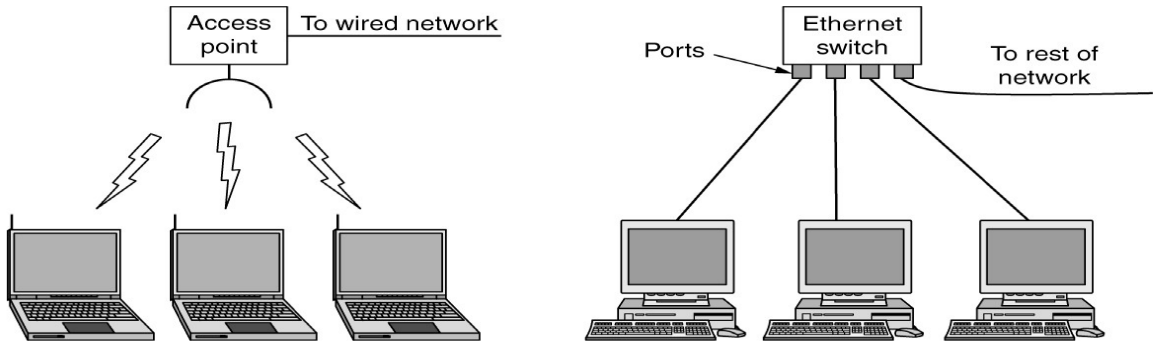


Figura 1.30 – A sinistra una rete wireless 802.11: ogni macchina parla con un access point, che inoltra i dati verso la rete cablata. A destra una rete Ethernet commutata: ogni macchina ha il proprio cavo verso uno switch, che inoltra i frame in base all'indirizzo.

- **Wireless LAN** (WLAN): lo standard è **IEEE 802.11**, il Wi-Fi di casa.
- **LAN cablata**: tipicamente **Ethernet commutata** (*switched Ethernet*), con il classico cavo di rete. Se il Wi-Fi in camera va male, un cavo Ethernet collegato al router garantisce una banda più stabile.

La rete di casa

La rete domestica è una LAN, ma è **la LAN con i requisiti più difficili**:

- viene installata da qualcuno che non leggerà il manuale: deve **funzionare da sola**, appena tirato fuori il router dalla scatola (vedremo i protocolli, come DHCP, che lo rendono possibile);
- i dispositivi vengono comprati uno alla volta, nel corso degli anni, da produttori diversi, e devono comunque **funzionare insieme**;
- i margini di guadagno sono bassi, mentre sicurezza, affidabilità e interoperabilità costano;
- un elettrodomestico insicuro non è un file perso: è una serratura che qualcun altro può aprire.

IoT: prima un problema di sicurezza

È nelle case che l'Internet of Things arriva davvero (televisore, frigorifero, telecamere, tapparelle), ed è per questo che l'IoT è un problema di **sicurezza** prima ancora che di rete.

MAN --- Metropolitan Area Network

MAN

Una **MAN** copre l'area di una **città**. Collega tra loro edifici, quartieri o abitazioni di un'area urbana, tipicamente con un'infrastruttura posseduta da un unico operatore.

L'esempio più noto è la **rete della televisione via cavo**, molto diffusa in Nord America.

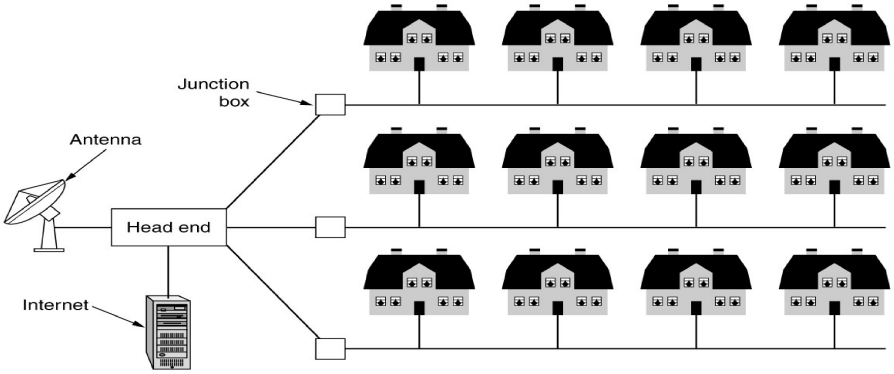


Figura 1.31 – Una MAN via cavo: televisione e Internet entrano nella stessa *head-end* e vengono distribuite alle case.

- Al centro c'è la **head-end** (*cable head-end*, o *cable modem termination system*): riceve i canali TV dall'**antenna** e, da quando la rete è usata anche per Internet, è collegata a **Internet**.
- Dalla head-end partono cavi che, attraverso delle **scatole di giunzione** (*junction box*), raggiungono le strade; lungo ogni strada le case sono collegate allo stesso cavo, come su un **bus**. Nel complesso la rete è un **albero di bus**.
- La rete era nata per distribuire la televisione **in una sola direzione**. È stata poi trasformata in un servizio Internet **bidirezionale** usando le parti dello spettro di frequenze lasciate libere dai canali TV.

Altri esempi di MAN sono le reti in fibra ottica che collegano le sedi di un'università o gli uffici pubblici di una città.

WAN --- Wide Area Network

Una WAN copre una nazione o un continente. L'esempio classico del Tanenbaum è un'azienda con tre uffici in Australia: Perth, Melbourne e Brisbane.

Host e subnet in una WAN

In una WAN:
- le macchine che eseguono le applicazioni sono gli **host**;
- tutto ciò che trasporta i loro messaggi, cioè **linee di trasmissione e router**, forma la **subnet** (sottorete di comunicazione). La subnet di solito è posseduta da qualcun altro, tipicamente una compagnia di telecomunicazioni.

Due significati di "subnet"

Qui *subnet* (terminologia del Tanenbaum) indica l'infrastruttura di trasporto di una WAN. Nella Parte V la stessa parola indicherà un *blocco di indirizzi IP* (subnetting). Non confondete i due usi.

La complessità di una WAN sta soprattutto nel **modo** in cui si realizzano i collegamenti tra sedi lontane. Ci sono tre soluzioni tipiche.

1. Linee dedicate (leased lines)

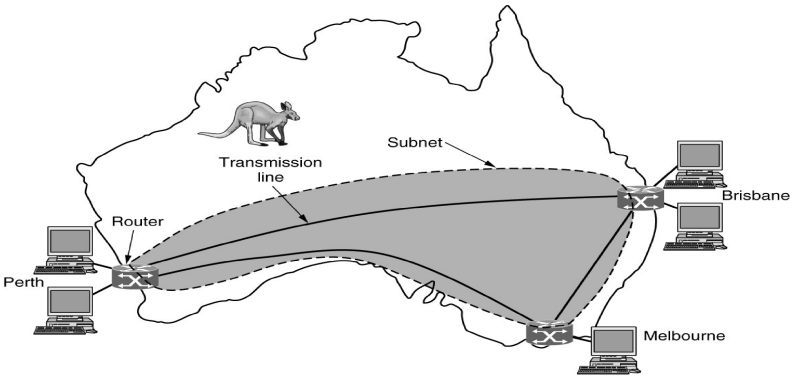


Figura 1.32 – Tre uffici collegati da linee affittate: la subnet (in grigio) è fatta di router e linee di trasmissione.

L'azienda **affitta linee di trasmissione** da un operatore e le collega con i propri router. La capacità delle linee è sua, garantita e prevedibile; ma è la soluzione più **costosa** e meno flessibile (aggiungere una sede significa affittare nuove linee). Stendere i propri cavi attraverso l'Australia, del resto, non è alla portata di quasi nessuno.

2. Collegamenti virtuali su Internet (VPN)

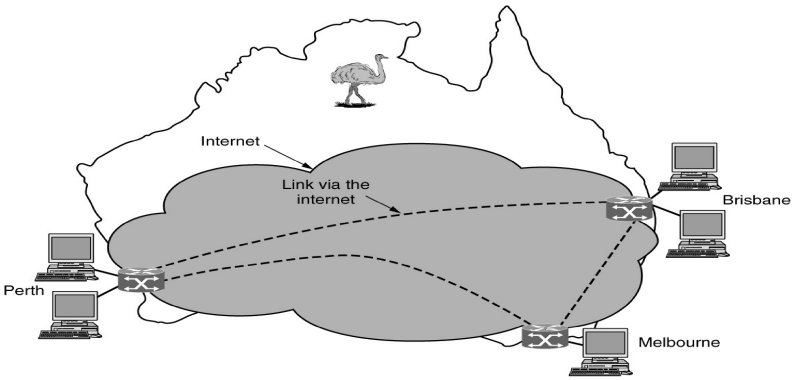


Figura 1.33 – La stessa WAN realizzata come rete privata virtuale (VPN) sopra Internet.

Invece di affittare linee, gli uffici si collegano con **collegamenti virtuali** che viaggiano sopra la normale connettività Internet: per la WAN è come se esistesse una linea diretta tra le sedi, ma in realtà i dati attraversano la rete di reti. È una **VPN** (*Virtual Private Network*). È **economica e flessibile**, ma la capacità **non è più nostra**: le prestazioni seguono quello che sta facendo Internet in quel momento, senza garanzie di velocità né di affidabilità.

3. Attraverso la rete di un ISP

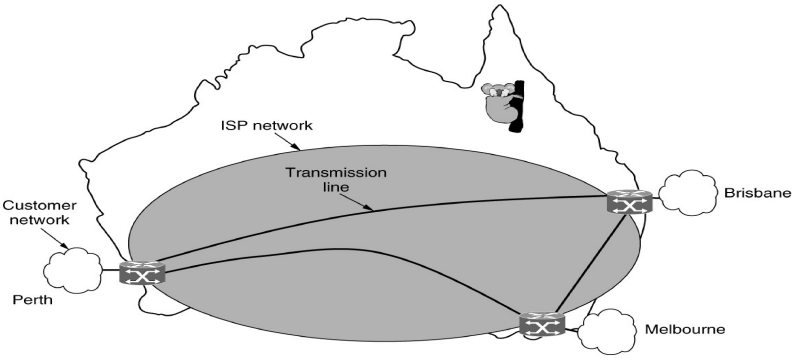


Figura 1.34 – Gli uffici collegati attraverso la rete di un ISP.

Le reti delle sedi si collegano a un **Internet Service Provider**, e la **rete dell'ISP** trasporta il traffico tra le sedi e verso il resto di Internet. L'azienda non gestisce linee proprie: se ne occupa l'ISP, che possiede o affitta le proprie dorsali (anche intercontinentali) e offre il servizio secondo un contratto.

	Linee dedicate	**VPN su Internet**	**Rete di un ISP**
Chi gestisce le linee	l'azienda (affitta)	nessuno in particolare	l'ISP
Capacità garantita	sì	no	secondo contratto
Costo	alto	basso	medio
Flessibilità	bassa	alta	media

Tabella 1.5 – I tre modi di realizzare una WAN.

WAN eterogenee e internetwork

Una WAN grande raramente usa una sola tecnologia: nello stesso disegno convivono dorsali commutate, collegamenti punto-punto, hub e LAN.

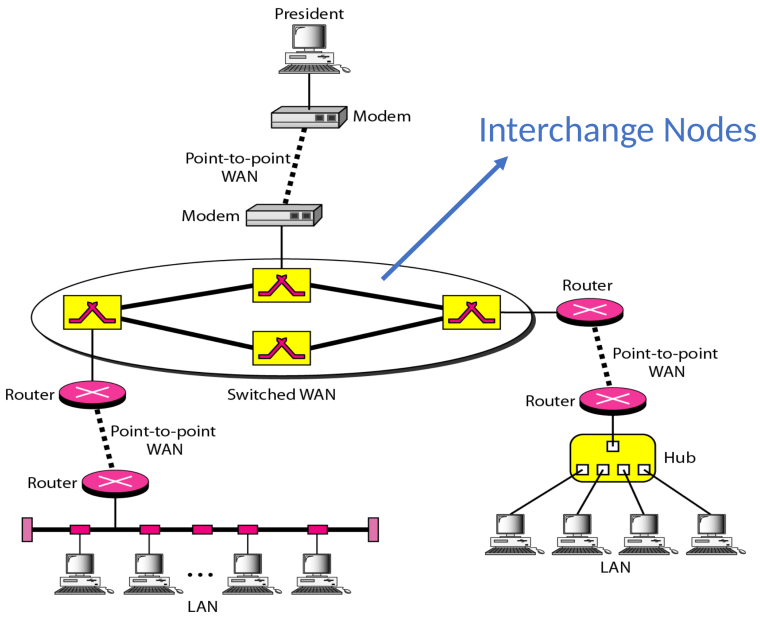


Figura 1.35 – Una WAN eterogenea: WAN commutata, WAN punto-punto, LAN e i nodi di interscambio che le raccordano.

Internetwork, gateway e router

Un insieme di **reti distinte**, gestite in modo **indipendente** e collegate tra loro è una **internetwork**. Il dispositivo che unisce due reti di tipo diverso, e "traduce" tra loro, è un **gateway**; quando il gateway opera al livello di rete si chiama **router**.

Internet Service Provider

Internet Service Provider (ISP)

Un **ISP** è un'organizzazione che fornisce i servizi per **accedere**, usare o partecipare a Internet. Può essere commerciale, di proprietà di una comunità, no-profit, oppure privato (per esempio un'azienda o un'università).

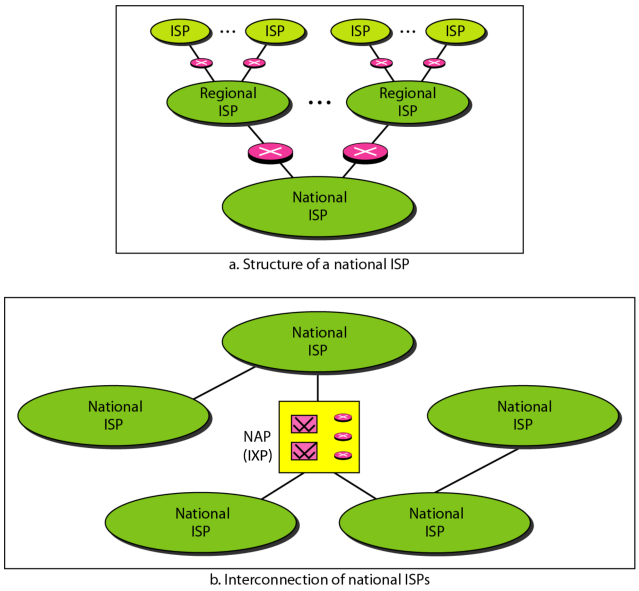


Figura 1.36 – (a) Struttura di un ISP nazionale: ISP regionali e locali appesi a quello nazionale. (b) ISP nazionali interconnessi attraverso un NAP/IXP.

La rete di un ISP è organizzata in modo **gerarchico**:

- un **PoP** (*Point of Presence*) è il punto, uno o più router, in cui i clienti si collegano al fornitore: il vostro router di casa si collega a un PoP del vostro ISP, da cui partono collegamenti più capaci;
- gli **ISP di accesso** coprono aree locali;
- gli **ISP regionali** coprono aree più grandi;
- gli **ISP nazionali** coprono intere nazioni;
- gli ISP si scambiano traffico negli **IXP** (*Internet Exchange Point*), detti anche **NAP** (*Network Access Point*): punti di incontro, spesso gestiti da un soggetto terzo neutrale.

Chi si collega con chi, e chi paga

- Collegare ogni ISP a ogni altro formerebbe una **full mesh**: con migliaia di ISP, servirebbero milioni di collegamenti. Impossibile.
- Quindi un ISP **cliente** paga un ISP **fornitore** per il transito del traffico, a ogni livello della gerarchia (l'ISP di accesso paga il regionale, il regionale paga il nazionale).
- Due ISP **allo stesso livello** possono invece fare **peering**: si collegano direttamente e si scambiano il traffico reciproco, di solito senza pagarsi.
- Un **IXP** è un edificio con l'hardware necessario perché molti ISP facciano peering contemporaneamente.
- Un ISP può fare **multi-homing**: comprare connettività da due fornitori diversi, così da continuare a funzionare se uno dei due si guasta.

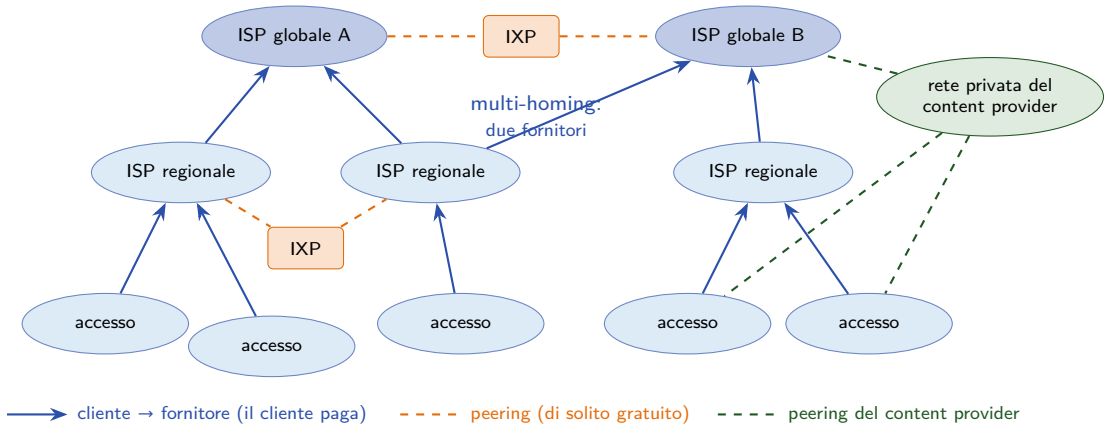


Figura 1.37 – La struttura economica di Internet: gerarchia di clienti e fornitori, peering negli IXP e reti private dei content provider.

I content provider

I grandi fornitori di contenuti (Google, Netflix, Meta, …) costruiscono **reti private globali** e fanno peering **direttamente con gli ISP di livello basso**, scavalcando i livelli superiori. Così spendono meno (non pagano il transito) e controllano direttamente come i loro contenuti arrivano agli utenti.

COLLEGARSI A UN ISP

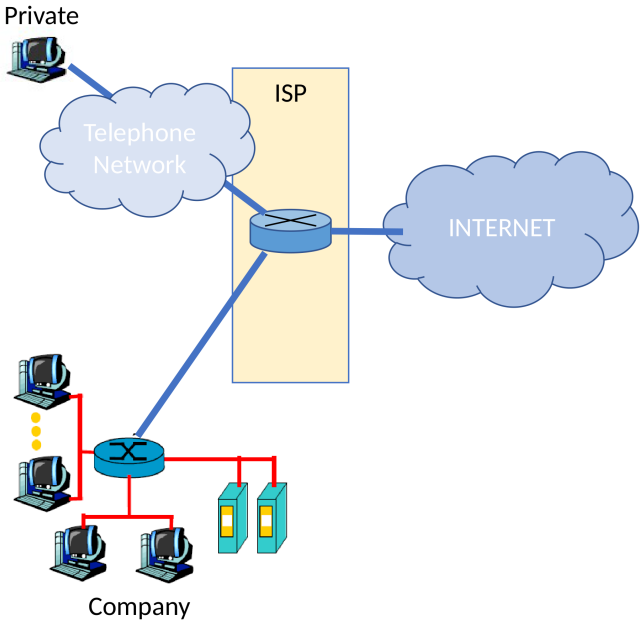


Figura 1.38 – In alto una connessione privata attraverso la rete telefonica; in basso un'azienda con un collegamento dedicato all'ISP.

- **Accesso residenziale**: un **modem** (o un **ONT**, per la fibra) raggiunge il fornitore sfruttando l'infrastruttura esistente, cioè la rete telefonica o quella via cavo.
- **Accesso aziendale**: un'azienda medio-grande ha molto più traffico, quindi compra un **collegamento dedicato** direttamente verso l'ISP e collega la propria rete dietro un **router** aziendale, che dialoga con un router dell'ISP.

Figura 1.39 – Il percorso da casa a Internet: il primo salto fuori da casa appartiene già all'ISP.

In entrambi i casi, **il primo salto fuori dall'edificio appartiene a qualcun altro**: subito fuori dalla rete di casa o dell'azienda c'è un ISP, che va pagato per i servizi che fornisce.

Tecnologia	Su cosa viaggia	Da ricordare
DSL / ADSL	il doppino telefonico esistente	la velocità **cala con la distanza** dalla centrale
Cavo (HFC)	fibra fino alla strada, cavo coassiale fino in casa	l'ultimo tratto è **condiviso** con i vicini
FTTH	fibra fino in casa, ONT in casa	centinaia di Mb/s, fino al gigabit e oltre
Wireless fisso	radio 5G da una stazione base	nessun cavo da posare

Tabella 1.6 – Le tecnologie di accesso residenziale oggi.

- Sono tutte **asimmetriche**: la velocità in *download* (verso casa) è maggiore di quella in *upload* (verso Internet), perché da casa si scarica molto più di quanto si invii.
- Le cifre pubblicizzate sono un **tetto**; e su un mezzo condiviso, quel tetto è condiviso.
- Nella fibra non c'è (de)modulazione del segnale come nel modem tradizionale: per questo l'apparecchio si chiama **ONT** (*Optical Network Terminal*) e non modem.

Un po' di storia delle velocità di accesso

Modem analogico	56 kb/s
ISDN (*Integrated Services Digital Network*)	128 kb/s
ADSL (*Asymmetric Digital Subscriber Line*)	da 1 a 20 Mb/s
Doppino in rame di nuova generazione	da 10 a 100 Mb/s
Fibra ottica (tramite ONT)	da 50 Mb/s a decine di Gb/s

RIEPILOGO

In breve

- Una rete è una collezione di dispositivi **autonomi** che possono **scambiarsi informazione**: **host** ai bordi, **commutatori di pacchetto** (router e switch) in mezzo.
- Comunicano **processi**: un client chiede, un server risponde; nel P2P ogni peer fa entrambe le cose.
- **Internet** è una rete di reti indipendenti, integrate attraverso gli ISP e tenute insieme da **IP**.
- I dati viaggiano in **pacchetti**, memorizzati e inoltrati (*store-and-forward*), che si accodano quando il collegamento è occupato e vengono persi quando la coda è piena.
- Internet usa la **commutazione di pacchetto** perché il traffico è a raffica.
- La **topologia** dice chi è collegato con chi; la **scala** dice se la rete è PAN, LAN, MAN o WAN.
- Gli ISP formano una gerarchia di clienti e fornitori, con **peering** negli IXP.

Una sola richiesta Web attraversa decine di queste reti, con tecnologie e gestori diversi. Come si tiene sotto controllo una tale complessità? La risposta è l'argomento del prossimo capitolo: la **stratificazione**.

Stratificazione, modelli di riferimento e incapsulamento

Questo è forse il capitolo più importante di tutto il corso: introduce il **modello a strati**, la chiave di lettura di tutto ciò che verrà dopo. Ogni parte successiva della dispensa si occuperà di uno strato preciso.

Il Web è un'applicazione

Partiamo da un chiarimento: **il Web non è Internet**. Il Web è *una* delle applicazioni che girano sopra Internet; la posta elettronica, lo streaming, la messaggistica, il trasferimento di file e i giochi sono altre applicazioni, e ciascuna chiede alla rete qualcosa di leggermente diverso. Nel capitolo precedente abbiamo visto di cosa è fatta una rete e come i pezzi sono collegati. Ora la domanda è:

Che cosa deve esistere, e funzionare, perché una pagina compaia nel browser?

La risposta a questa domanda è la struttura di tutto il resto del corso.

Cosa attraversa una singola richiesta HTTP

Un browser sull'host A chiede una pagina al server Web B. Tra i due c'è un bel po' di roba:

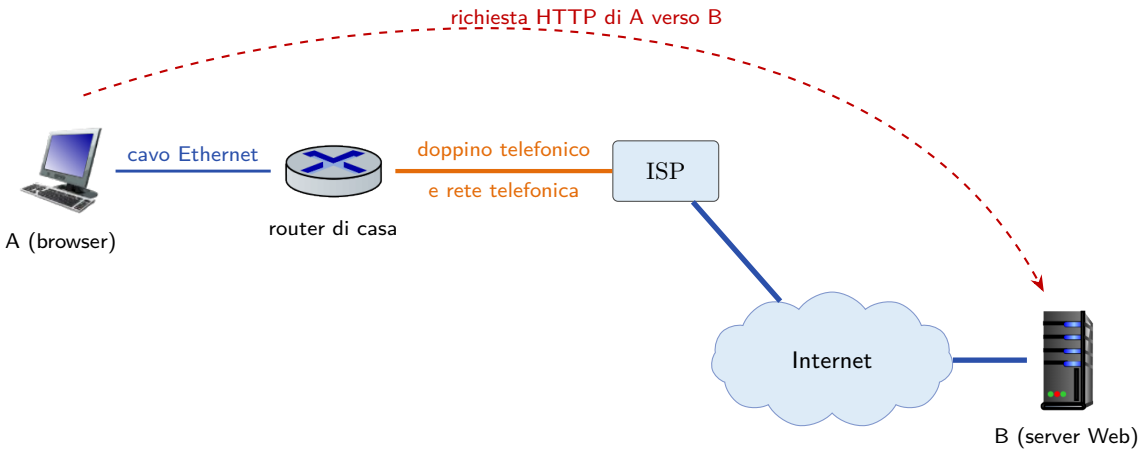


Figura 2.1 – Una richiesta HTTP attraversa mezzi diversi, apparati diversi e una maglia di reti di cui nessuno dei soggetti coinvolti conosce l'interno.

- Ogni apparato sul cammino è **hardware più software**, costruito da un produttore diverso.
- Ogni tratto è un **mezzo diverso**, con i propri errori e la propria velocità.
- Nessuno può progettare, o anche solo descrivere, un sistema del genere **tutto in una volta**.

Divide et impera

Si spezza il sistema complesso in **sottosistemi**, ciascuno con un compito proprio e un modo ben definito di chiedere aiuto al successivo; poi si progetta un sottosistema alla volta. Nelle reti questa scomposizione ha un nome: **stratificazione** (*layering*).

Internet vista "dai bulloni" e come servizio

Prima di entrare negli strati, ricapitoliamo i due modi complementari di guardare Internet.

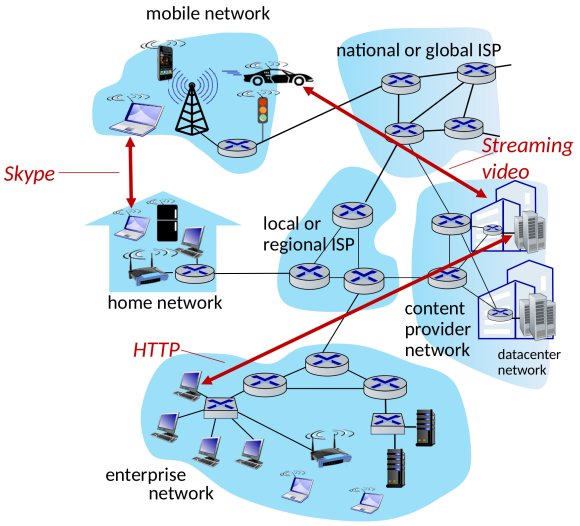
La vista "nuts and bolts"

Dal punto di vista dei suoi componenti fisici (i "dadi e bulloni"), Internet è fatta di:

- **miliardi di dispositivi di calcolo** connessi, gli **host** o *end system*, che eseguono applicazioni di rete e stanno ai **bordi** (*edge*) di Internet;
- **commutatori di pacchetto** (*packet switch*), che inoltrano pacchetti (blocchi di dati): router e switch;
- **collegamenti di comunicazione**: fibra, rame, radio, satellite, ciascuno con il proprio *transmission rate*;
- **reti**: collezioni di dispositivi, router e collegamenti gestite da una singola organizzazione.

Figura 2.2 – Non solo computer: frigoriferi, cornici digitali, telecamere, pacemaker, materassi sensorizzati, orologi… sono tutti host di Internet.

In questa vista i **protocolli sono ovunque**: controllano l'invio e la ricezione dei messaggi (HTTP per il Web, TCP, IP, Wi-Fi, 4G, Ethernet, i protocolli proprietari di Skype, …).



Vista dall'alto, Internet non è fatta di cavi: è un'**infrastruttura che fornisce servizi alle applicazioni** (Web, video, posta, videoconferenza, commercio elettronico, social media, elettrodomestici connessi).

- Fornisce anche un'**interfaccia di programmazione**: i "ganci" (*hooks*) con cui un'applicazione si aggancia a Internet per inviare e ricevere dati.
- Come il servizio postale (posta ordinaria, raccomandata, espresso), offre **opzioni di servizio**: veloce o economico, garantito o no.

Sono proprio quei ganci che programmeremo, in C, con i socket.

Figura 2.3 – Protocolli e servizi in azione sulla rete di reti.

L'applicazione vede solo un servizio

Le applicazioni non devono preoccuparsi di come i dati attraversano la rete: il flusso dei dati viene offerto loro **come servizio** dallo strato inferiore (il trasporto). È esattamente questo il senso della stratificazione.

PROTOCOLLI E STANDARD

Ricordiamo la definizione del capitolo precedente: un **protocollo** fissa il **formato** e l'**ordine** dei messaggi, e le **azioni** da compiere quando un messaggio viene inviato o ricevuto. Vale per le persone come per le macchine.

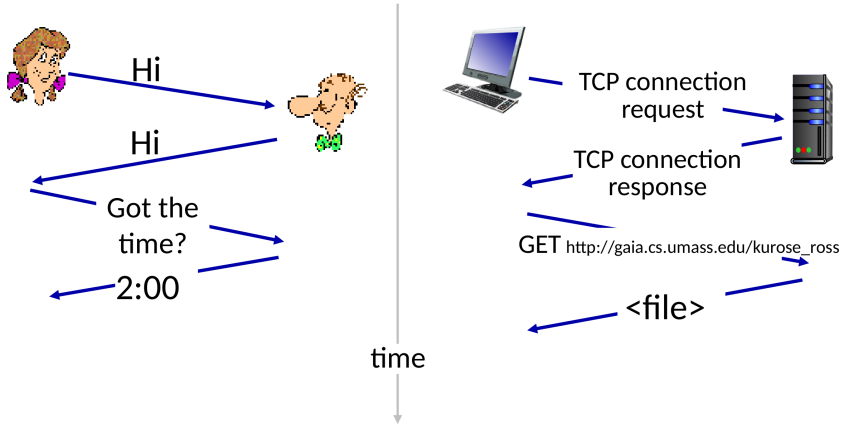


Figura 2.4 – Stessa forma: un saluto, una risposta, poi la richiesta vera e propria.

Chi scrive gli standard

- L'**IETF** (*Internet Engineering Task Force*) è una comunità aperta di esperti che definisce gli standard e i protocolli di Internet.
- Lavora in **working group aperti**, ciascuno dedicato a un aspetto di Internet.
- Ogni gruppo pubblica documenti numerati detti **RFC** (*Request For Comments*), che descrivono e definiscono protocolli, formati e metodi. Per esempio IPv4 è definito nell'**RFC 791**, del 1981. Il nome è rimasto per ragioni storiche: nascevano come inviti alla discussione e sono diventati gli standard di Internet.
- L'**ISO** (*International Standards Organization*) funziona in modo molto diverso: è un'organizzazione non basata su trattati internazionali, i cui membri sono gli enti nazionali di standardizzazione. È da qui che nasce il modello **OSI**.

Come decide Internet

Gli standard di Internet non nascono da un voto in commissione, ma dal principio *rough consensus and running code*: consenso di massima e codice che funziona. Una specifica che nessuno implementa non diventa uno standard.

LA STRATIFICAZIONE

Le reti sono complesse e fatte di molti pezzi: host, router, collegamenti su mezzi diversi, applicazioni, protocolli, hardware, software. C'è speranza di organizzarne la struttura? Sì: la risposta architetturale è una **pila di strati**, ciascuno costruito su quello sottostante.

Modello a strati

Un approccio architetturale in cui la comunicazione è realizzata da una pila di **strati** (*layer*) tali che:
- ogni strato è un **livello di astrazione**: nasconde *come* svolge il proprio compito;
- ogni strato **usa i servizi dello strato inferiore** e **fornisce servizi allo strato superiore**;
- tra due strati adiacenti c'è un'**interfaccia**: l'insieme delle operazioni che lo strato inferiore mette a disposizione di quello superiore.

La cima della pila è rivolta alle applicazioni (alto livello), il fondo all'hardware (basso livello).

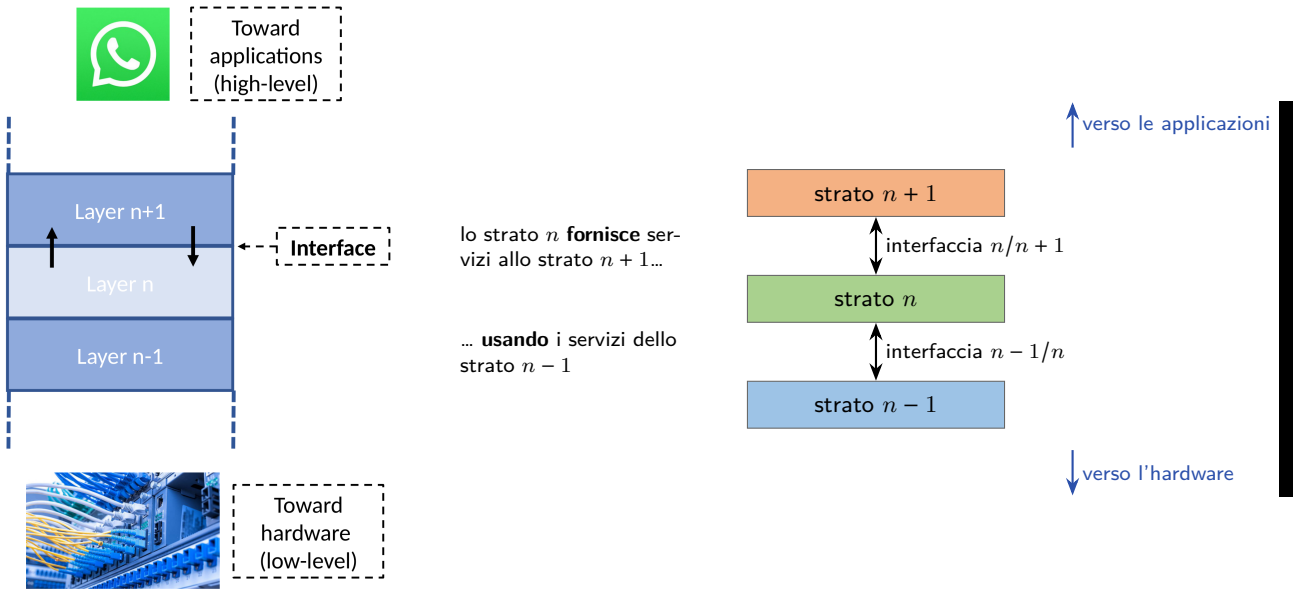


Figura 2.5 – Ogni strato si appoggia a quello sotto e serve quello sopra, attraverso un'interfaccia.

Moduli, interfacce e incapsulamento delle funzioni

L'idea non è nuova: è la stessa che si usa per progettare qualunque software di una certa dimensione. Quando un sistema viene diviso in sottosistemi, per ciascuno si stabilisce un'**interfaccia**, cioè l'insieme dei servizi che il modulo offre all'esterno; *come* quei servizi siano realizzati resta nascosto dentro il modulo. È l'**incapsulamento delle funzioni**: ogni modulo può chiamare i servizi di un altro per il proprio funzionamento, senza sapere nulla della sua implementazione. Chi ha scritto classi in C++ o in Java ha già fatto esattamente questo: metodi pubblici all'esterno, dettagli privati all'interno.

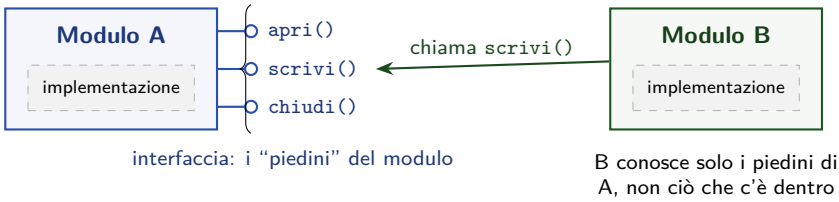


Figura 2.6 – Ogni modulo espone all'esterno i propri servizi (i "piedini") e nasconde l'implementazione: è il disegno fatto alla lavagna per introdurre gli strati.

Nell'architettura a strati questa idea diventa più **rigida**: i moduli sono impilati, e ogni strato offre i propri servizi allo strato **immediatamente superiore** e usa quelli dello strato **immediatamente inferiore** per implementarli. Lo strato in cima è quello delle applicazioni (o offerto alle applicazioni); quello più in basso è l'hardware, lo strato fisico.

Architetture chiuse e aperte

Nella concezione di base l'architettura a strati è **chiusa**: uno strato può rivolgersi soltanto a quello subito sotto. In alcuni casi, per efficienza o per necessità, la si **apre**, permettendo a uno strato di usare direttamente servizi di strati non adiacenti. È un'eccezione, e ha un prezzo: i moduli diventano meno indipendenti.

La stratificazione non è un'idea delle reti

Pile di strati le usate già, ovunque un sistema sia diventato troppo grande per essere progettato in un pezzo solo.

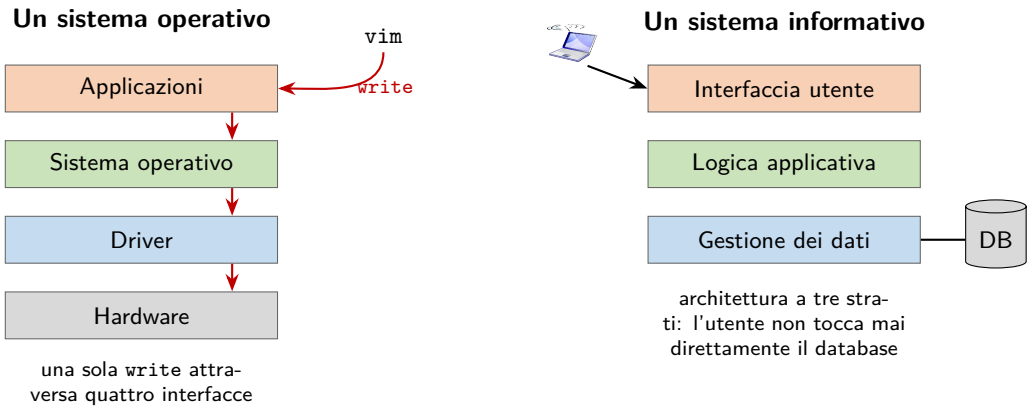


Figura 2.7 – Due pile di strati che conoscete già.

Le reti sono semplicemente il caso più grande di tutti.

Cosa guadagniamo, e cosa paghiamo

- **Struttura**: una struttura esplicita permette di **dare un nome** ai pezzi del sistema e alle loro relazioni; è un *modello di riferimento* di cui discutere.
- **Modularità**: l'implementazione di uno strato può cambiare senza che nient'altro si muova, **purché il servizio offerto resti lo stesso**. Sostituire tutte le linee telefoniche con canali satellitari è una modifica interna a un solo strato: gli strati superiori non se ne accorgono.

Esempio 1 — La modularità nella vita di tutti i giorni

- Cambiate stampante, e con lei il **driver**: il sistema operativo continua a mandare le stampe esattamente come prima, e le applicazioni non si accorgono di nulla.
- State scrivendo un messaggio collegati al Wi-Fi di casa, poi uscite e il telefono passa alla **rete cellulare**. L'applicazione di messaggistica si rivolge sempre al sistema operativo nello stesso modo: del cambio di tecnologia si occupano gli strati inferiori.

L'analogia del viaggio aereo

Un viaggio in aereo è una pila di servizi: biglietto (acquisto / reclamo), bagagli (imbarco / ritiro), gate (imbarco / sbarco), pista (decollo / atterraggio), rotta dell'aereo. Ogni "strato" dell'aeroporto di partenza ha il suo corrispondente in quello di arrivo, e se cambia la procedura ai gate il resto del sistema non se ne accorge.

Il prezzo dell'architettura a strati

La domanda "*layering considered harmful?*" non è retorica. Alla lavagna il prezzo è stato riassunto in tre voci:

- **maggiore complessità**: tutte le **interfacce** tra gli strati vanno progettate e implementate, e ogni strato aggiunge le proprie informazioni di controllo (**overhead**);
- **eventuali ridondanze**: uno strato può rifare qualcosa che uno strato inferiore fa già, per esempio la rilevazione e la correzione degli errori, svolta sia su ogni singolo collegamento sia da un estremo all'altro. Più calcolo, a volte inutile;
- **rigidità**: le informazioni interne di ogni strato sono **nascoste**. Quando poi uno strato ha bisogno di un'informazione che sta in un altro (succede spesso nelle reti), le strade sono due: **riprogettare** gli strati, oppure adottare soluzioni "*quick and dirty*", rapide, efficaci e un po' sporche, come vedremo per esempio con l'uso delle porte.

È un compromesso: si accetta un po' di inefficienza in cambio di un'enorme semplicità di progetto.

Servizio, interfaccia e protocollo

Tre concetti da tenere rigorosamente separati.

Servizio e protocollo

Un **servizio** è l'insieme delle operazioni che uno strato offre allo strato superiore: dice **che cosa** fa lo strato, mai **come** lo fa.
Un **protocollo** è l'insieme delle regole sul formato e sul significato dei messaggi scambiati con l'**entità paritaria** (*peer entity*), cioè lo stesso strato sull'altra macchina.

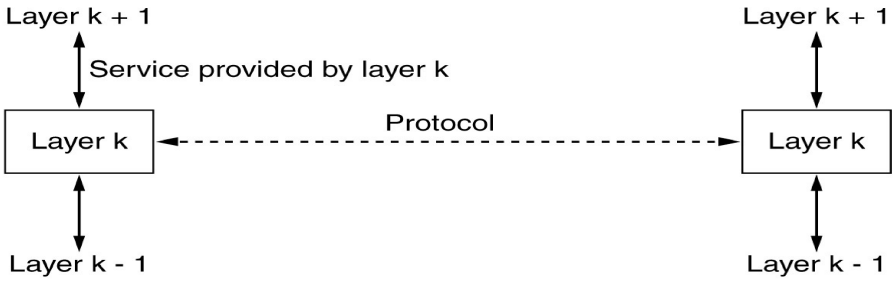


Figura 2.8 – I servizi stanno sulle interfacce verticali, i protocolli sullo scambio orizzontale.

L'**interfaccia** è il punto in cui il servizio viene offerto: l'insieme delle operazioni che lo strato inferiore rende disponibili a quello superiore. In sintesi:

Concetto	Direzione	Risponde alla domanda
Servizio	verticale (tra strati adiacenti)	*che cosa* offre lo strato?
Interfaccia	verticale	*come si chiede* il servizio?
Protocollo	orizzontale (tra strati omologhi)	*come* lo strato realizza il servizio?

Entità e protocolli

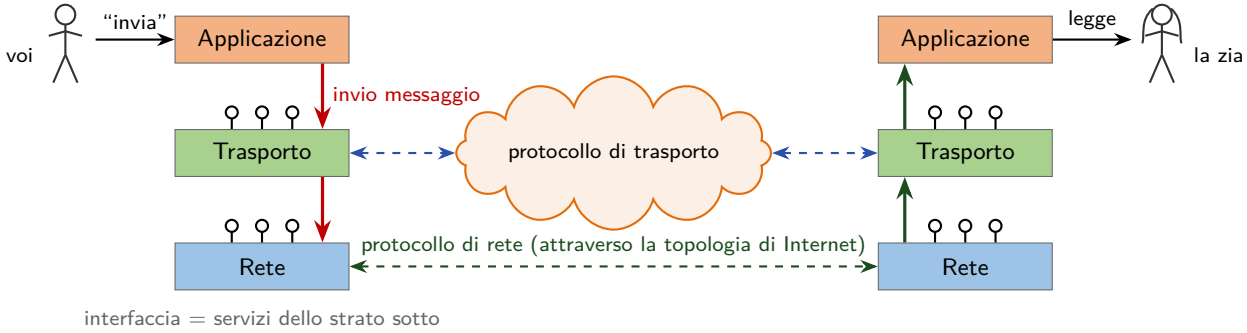
Le **entità** (gli elementi attivi di uno strato, hardware o software) **usano i protocolli per implementare i servizi** che offrono.

Servizio e protocollo sono indipendenti

Uno strato può cambiare completamente protocollo senza che gli strati superiori se ne accorgano, purché il servizio offerto resti lo stesso. Confondere servizio e protocollo è un errore tipico.

Esempio 2 — Un messaggio alla zia

Premete "invia" sulla vostra applicazione di messaggistica. L'applicazione non trasmette nulla da sé: chiama il servizio di invio che il **livello di trasporto** del telefono offre attraverso la propria interfaccia, e da quel momento se ne disinteressa. Il livello di trasporto dialoga, con il **protocollo di trasporto**, con l'entità di trasporto sul telefono della zia; per farlo usa i servizi del **livello di rete**, che a sua volta dialoga con il livello di rete dall'altra parte, occupandosi di attraversare la topologia di Internet.



Tra uno strato e quello sotto, **nello stesso telefono**, c'è un'**interfaccia** (i piedini); tra due entità dello stesso strato, **su telefoni diversi**, c'è un **protocollo** (le frecce tratteggiate).

Strati omologhi: comunicazione virtuale e fisica

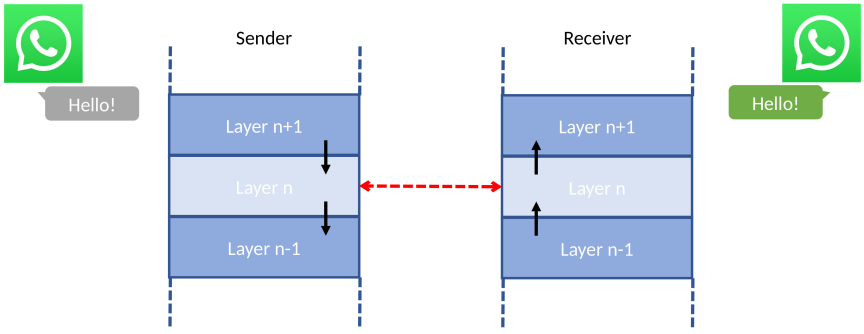


Figura 2.9 – Lo strato n del mittente dialoga logicamente con lo strato n del destinatario.

Comunicazione orizzontale

Quando due dispositivi comunicano, **ogni strato del mittente comunica con lo stesso strato del destinatario**, e con nient'altro, per mezzo di uno specifico protocollo.

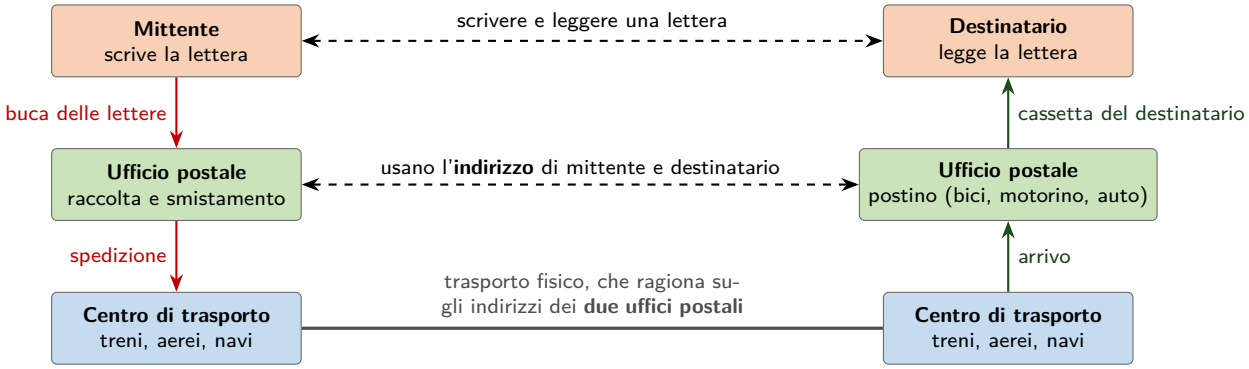
- Quella freccia orizzontale **non è un cavo**: è una conversazione *logica*.
- I dati **scendono** realmente lungo la pila del mittente e **risalgono** quella del destinatario. Solo al livello fisico qualcosa viaggia davvero.
- La catena si ripete a ogni strato: per fornire i propri servizi uno strato deve comunicare con il suo pari, e per farlo chiama i servizi dello strato inferiore, che a sua volta comunica con il proprio pari usando un protocollo **diverso**, e così via fino al protocollo fisico, che manda segnali su un mezzo.

Tutte queste trasmissioni sono virtuali

Nelle reti c'è comunicazione tra due macchine a **ogni** strato, con i relativi protocolli. Ma tutte queste trasmissioni sono **virtuali**, salvo quella a livello fisico, che è l'unica a comportare effettivi fenomeni fisici. Il livello di trasporto del mittente "crede" di parlare direttamente con il trasporto del destinatario, ma i suoi dati scendono lungo tutta la pila, viaggiano sul filo e risalgono dall'altra parte.

Esempio 3 — L'analogia postale

Il modello a strati funziona come il servizio postale: ogni strato fa un solo lavoro, si affida allo strato sotto e parla il proprio "linguaggio"; ogni indirizzo è letto dallo strato per cui è stato scritto, e da nessun altro.



- **Strato superiore**: mittente e destinatario vedono soltanto due buche per le lettere, una in cui infilare la lettera e una da cui ritirarla. I loro "protocolli" sono scrivere e leggere una lettera.
- **Strato intermedio**: gli uffici postali raccolgono la lettera e, dall'altra parte, la fanno consegnare dal postino. Il loro linguaggio sono gli **indirizzi** del mittente e del destinatario.
- **Strato inferiore**: treni, aerei e navi portano fisicamente la posta da un ufficio all'altro, e ragionano solo in termini di uffici di provenienza e di destinazione.

Tanenbaum propone un'analogia altrettanto efficace: un filosofo che detta una lettera a un traduttore, il quale la passa a una segretaria che la spedisce; dall'altra parte la catena si ripercorre al contrario, e ogni figura "parla" solo con la figura corrispondente.

Servizi con e senza connessione

Prima di vedere gli strati concreti, serve una distinzione **ortogonale** a quella degli strati: quella tra servizi **orientati alla connessione** e servizi **senza connessione**.

Servizio orientato alla connessione

Servizio orientato alla connessione

Modellato sul **sistema telefonico**: l'utente del servizio prima **stabilisce** una connessione, poi la **usa**, infine la **rilascia**.

- La connessione si comporta come un **tubo**: ciò che entra da un'estremità esce dall'altra, normalmente **nello stesso ordine**.
- L'apertura permette una **negoziazione** dei parametri (dimensione massima dei messaggi, qualità del servizio, …): una parte propone, l'altra accetta o fa una controproposta.
- Una connessione che riserva anche delle **risorse**, per esempio una velocità fissa, si chiama **circuito**: un cammino sulla rete su cui viaggiano tutti i dati della sessione.
- Conviene quando c'è una quantità significativa di dati da scambiare.

E se si ha un solo messaggio breve da inviare, vale la pena aprire e chiudere una connessione?

Servizio senza connessione

Servizio senza connessione (connectionless)

Modellato sul **sistema postale**: non si stabilisce nulla prima di trasmettere. Ogni messaggio porta con sé l'**indirizzo completo** di destinazione e viene **gestito individualmente** dai nodi intermedi.

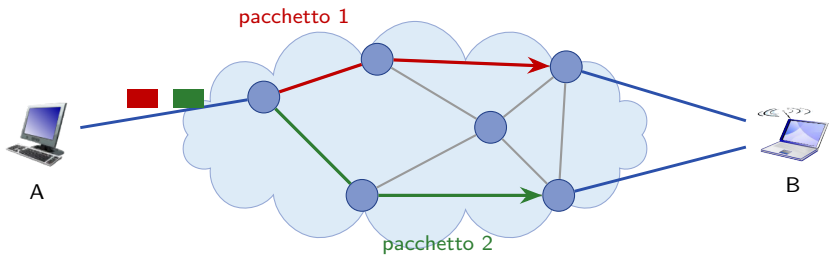


Figura 2.10 – In un servizio senza connessione i pacchetti tra gli stessi due estremi possono seguire strade diverse.

- Tra gli stessi due estremi la rete offre più cammini, quindi due messaggi possono seguirne di diversi e **arrivare fuori ordine**.
- Nella commutazione **store-and-forward** un nodo riceve un messaggio **per intero** prima di inoltrarlo al nodo successivo.
- Nella commutazione **cut-through** un nodo comincia a inoltrare il messaggio **prima** di averlo ricevuto completamente (appena ha letto l'indirizzo di destinazione).

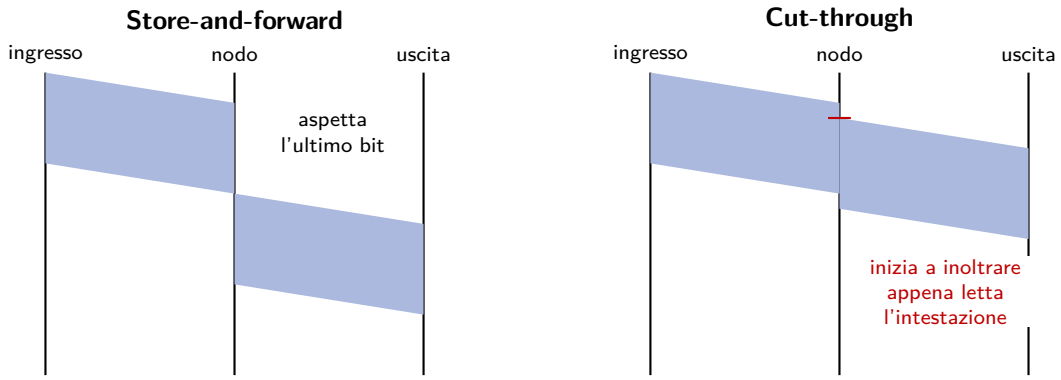


Figura 2.11 – Store-and-forward contro cut-through: il secondo riduce il ritardo, ma può inoltrare anche messaggi che si riveleranno danneggiati.

Datagram

Un servizio **senza connessione e inaffidabile**, cioè senza conferma di ricezione (*unacknowledged*). Il nome richiama il *telegramma*, che anch'esso non prevedeva ricevuta di ritorno.

Affidabilità

Affidabilità

Un servizio è **affidabile** se il ricevente **conferma** (*acknowledgement*, ACK) ciò che ha ricevuto, così che il mittente sappia che è arrivato. L'affidabilità è **ortogonale** alla connessione: sia i servizi con connessione sia quelli senza possono essere affidabili o no.

Perché mai si dovrebbe rinunciare all'affidabilità?

- Le conferme costano **overhead e ritardo**, e a volte il prezzo è troppo alto.
- L'affidabilità può semplicemente **non essere disponibile**: Ethernet non la fornisce, e sono gli strati superiori a dover recuperare gli errori.
- Nel **Voice over IP** confermare ogni pacchetto e ripristinarne l'ordine sarebbe non solo inutile ma dannoso: un ritardo di tre o quattro secondi rende una telefonata inutilizzabile, mentre una breve interruzione dell'audio si rimedia ripetendo una frase. Tutte le applicazioni VoIP cercano di **ridurre il ritardo** e accettano i buchi: l'inaffidabilità è la scelta *giusta*, non un compromesso. I parametri della comunicazione vanno adattati alle esigenze dell'applicazione.

Affidabile sopra inaffidabile

La maggior parte dei servizi affidabili che vedremo nel corso è costruita **sopra** un servizio inaffidabile: TCP (affidabile) sopra IP (inaffidabile) ne è l'esempio principale.

Sei tipi di servizio

Tipo	Servizio	Esempio
Orientato alla connessione	Flusso di messaggi affidabile	Sequenza di pagine
	Flusso di byte affidabile	Download di un film
	Connessione inaffidabile	Voice over IP
Senza connessione	Datagram inaffidabile	Posta elettronica "spazzatura"
	Datagram con conferma	Messaggi con le "spunte" di consegna
	Richiesta-risposta (*request-reply*)	Interrogazione a un database

Tabella 2.1 – Sei servizi comuni, con e senza connessione.

Le due righe da ricordare

Il **flusso di byte affidabile** è il servizio che ci darà **TCP**; il **datagram inaffidabile** è quello che ci darà **UDP**.

Le primitive di servizio

Un servizio si specifica formalmente tramite le **primitive**, cioè le operazioni con cui un processo lo richiede. Quando la pila protocollare è implementata nel sistema operativo, le primitive sono **chiamate di sistema** (*system call*).

Primitiva	Significato
`LISTEN`	si blocca in attesa di una connessione in ingresso
`CONNECT`	stabilisce una connessione con un peer in attesa
`ACCEPT`	accetta una connessione in ingresso da un peer
`RECEIVE`	si blocca in attesa di un messaggio in ingresso
`SEND`	invia un messaggio al peer
`DISCONNECT`	termina la connessione

Tabella 2.2 – Le primitive di un semplice servizio orientato alla connessione.

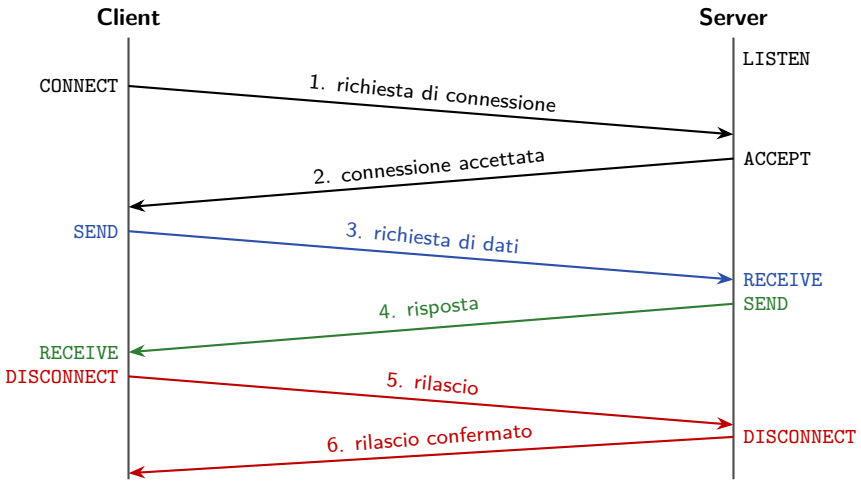


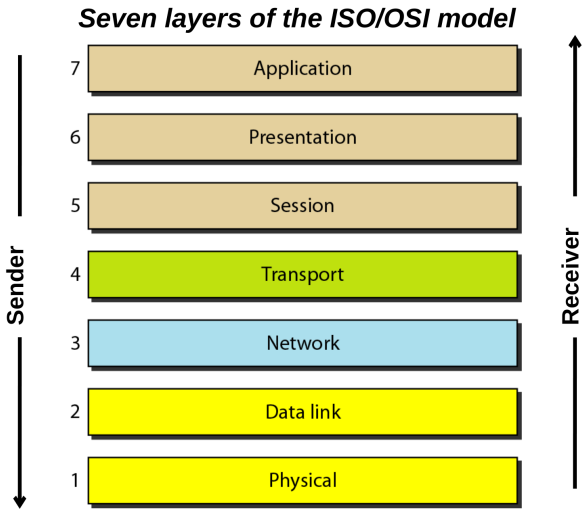
Figura 2.12 – Un'interazione client-server realizzata con le sei primitive.

Le ritroveremo in C

Guardatele bene: sono esattamente l'interfaccia su cui scriveremo i programmi in C nel capitolo sui socket: `listen()`, `connect()`, `accept()`, `send()`, `read()`, `close()`.

Il modello di riferimento ISO/OSI

Negli anni ’80 l’**ISO** definì un modello a strati di riferimento per le reti di calcolatori: il modello **OSI** (*Open Systems Interconnection*), cioè sistemi *aperti* alla comunicazione con altri sistemi.



- **Sette strati**, ciascuno creato dove serviva un’**astrazione diversa**.
- Il suo contributo più importante è aver reso **servizi**, **interfacce** e **protocolli** tre concetti **esplicitamente separati**.
- Il mittente scende lungo la pila, il destinatario la risale.

Figura 2.13 – I sette strati del modello ISO/OSI.

#	Strato	Compito
7	Applicazione	servizi all’utente: login remoto, trasferimento file, posta, Web
6	Presentazione	rappresentazione dei dati: traduzione tra codifiche, cifratura, compressione
5	Sessione	controllo del dialogo (chi parla, quando e per quanto) e checkpoint
4	Trasporto	consegna end-to-end al processo giusto (porte), segmentazione, controllo di flusso e degli errori
3	Rete	consegna dei pacchetti dall’host sorgente all’host destinazione: indirizzamento logico e routing
2	Collegamento	consegna dei frame da un nodo al successivo (un hop), controllo degli errori e accesso al mezzo
1	Fisico	trasmissione dei singoli bit sul mezzo

Tabella 2.3 – I sette strati OSI, dall’alto verso il basso.

Un trucco mnemonico

Dal basso verso l’alto: **F**isico, **C**ollegamento, **R**ete, **T**rasporto, **S**essione, **P**resentazione, **A**pplicazione. In inglese si usa la frase “***P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way*” (Physical, Data link, Network, Transport, Session, Presentation, Application).

Non tutti i nodi implementano tutti gli strati

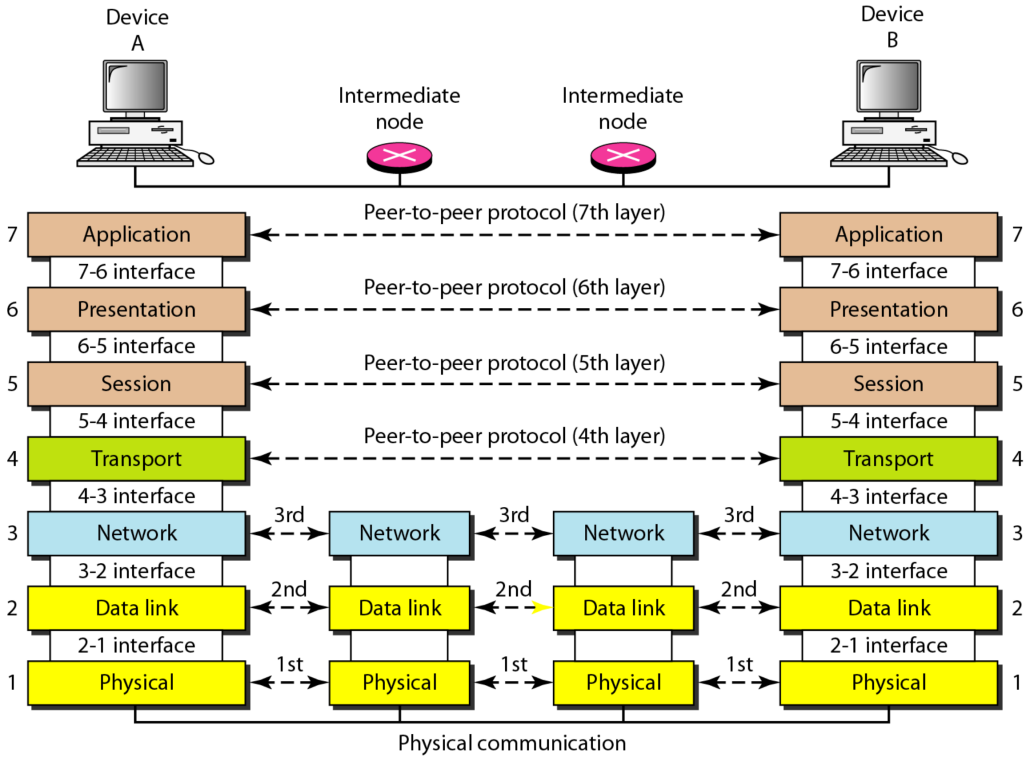


Figura 2.14 – I nodi intermedi implementano solo i primi tre strati; solo gli host implementano l'intera pila.

- Solo i **sistemi terminali** (host) implementano l'intera pila, fino all'applicazione.
- I **nodi intermedi** (switch, router) implementano solo due o tre strati, i cosiddetti *media layer*.
- Niente sopra il livello di rete: un router non ha alcun motivo di leggere il vostro messaggio.

Trasporto e rete: dove gira il software

A livello di **trasporto** la comunicazione è tra **processi**, dal punto di vista logico è punto-punto, e il software è eseguito **sugli host**. A livello di **rete** la comunicazione è tra **host**: si risolve il problema dell'instradamento dei pacchetti attraverso la topologia della rete, e il software è eseguito prevalentemente **sui nodi della rete**. Un router non ha bisogno del trasporto perché non gli interessa a quale processo sia destinato il pacchetto: gli basta sapere a quale host deve arrivare.

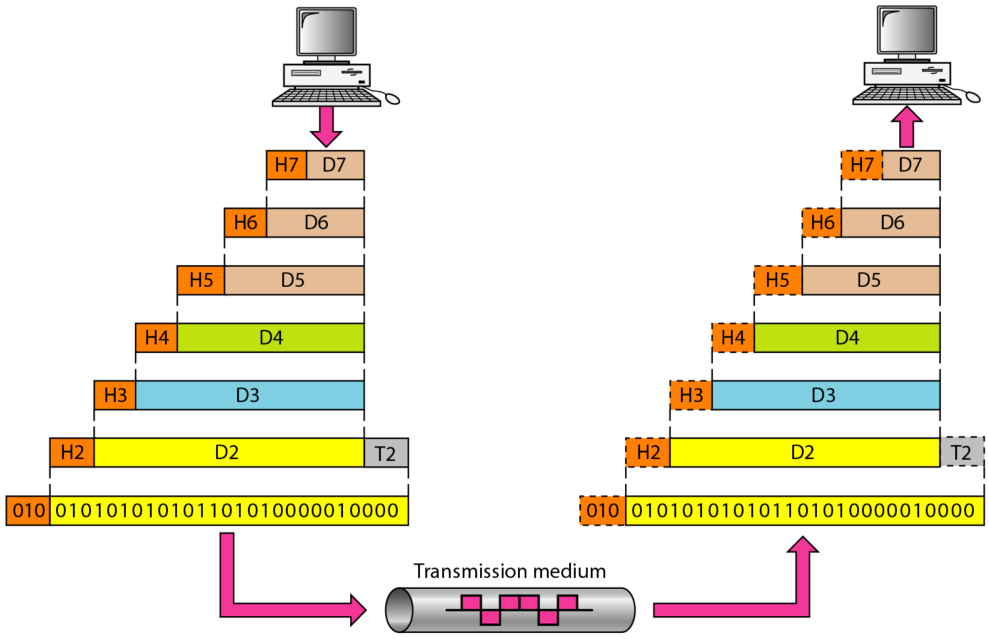


Figura 2.15 – Ogni strato del mittente aggiunge un header (e a volte un trailer); ogni strato del destinatario toglie il proprio.

Incapsulamento e decapsulamento

- **Incapsulamento** (scendendo, *top-down*): ogni strato del mittente aggiunge a ciò che riceve dallo strato superiore un proprio campo di controllo, sotto forma di un **header** (H) in testa e talvolta di un **trailer** (T) in coda.
- **Decapsulamento** (salendo, *bottom-up*): ogni strato del destinatario rimuove e interpreta **esattamente** il campo aggiunto dal suo pari, e passa il resto verso l'alto.

Un'unità di dati, a qualunque strato, è quindi sempre **header + payload**, e il **payload** è l'unità dello strato superiore.

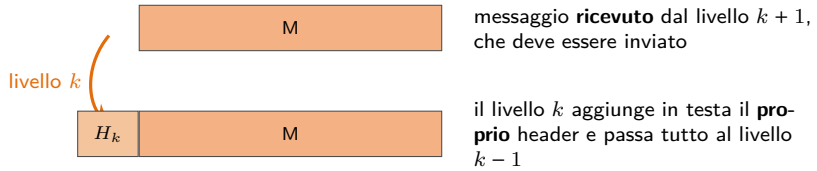


Figura 2.16 – Il passo elementare dell'incapsulamento, come alla lavagna: M arriva dall'alto e diventa il payload di $H_k + M$.

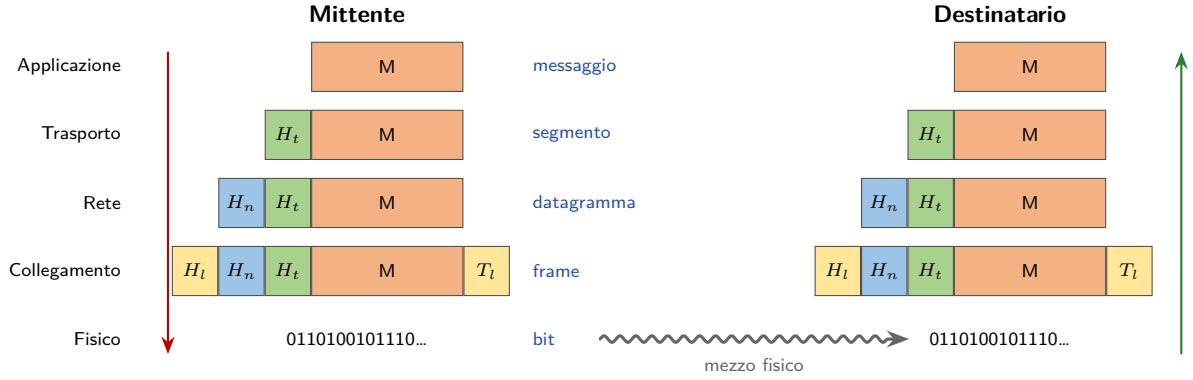


Figura 2.17 – Incapsulamento nella pila a cinque strati: ogni strato tratta tutto ciò che riceve dall'alto come un blocco unico a cui aggiungere il proprio header.

Sotto il proprio strato, il payload è opaco

Quando il messaggio di livello 7 scende, tutto ciò che viene dall'alto (header e contenuto insieme) diventa il **payload** del livello 6.

- Il livello 6 aggiunge il proprio header e passa tutto al livello 5, che fa lo stesso, e così via.
- Il livello 6 **non sa** di trasportare HTML, e non deve interessargli: per lui il payload è una **sequenza di byte opaca**.
- È proprio questo che rende gli strati **indipendenti**: uno strato che ispezionasse il proprio payload smetterebbe di funzionare appena cambia lo strato superiore.

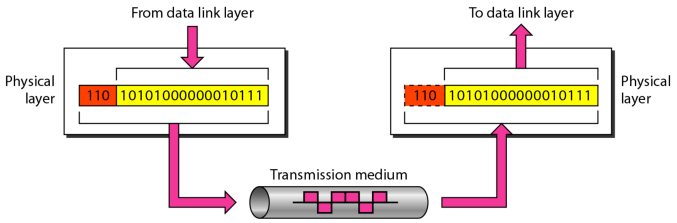
Il destinatario ripercorre la sequenza al contrario, e ogni strato toglie soltanto il proprio header.

Lo tocchiamo con mano

Quando, nella parte sulla programmazione di rete, scriveremo a mano una richiesta HTTP, vedremo che un messaggio a livello di applicazione viene semplicemente inviato come **payload** di un messaggio di trasporto, il cui contenuto è del tutto irrilevante ("trasparente") per il livello di trasporto.

I SETTE STRATI, UNO PER UNO

1. Livello fisico (Physical)



È responsabile di **spostare i singoli bit** da un nodo al successivo lungo il mezzo. Si occupa di:

- caratteristiche fisiche di interfacce e mezzi (connettori, cavi, segnali elettrici);
- configurazione della linea (punto-punto o multipunto) e topologia fisica;
- modalità di trasmissione (simplex, half-duplex, full-duplex);
- rappresentazione dei bit, velocità (data rate) e sincronizzazione dei bit tra i due estremi.

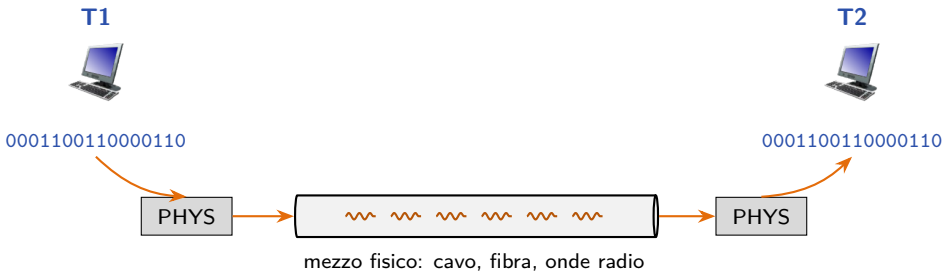
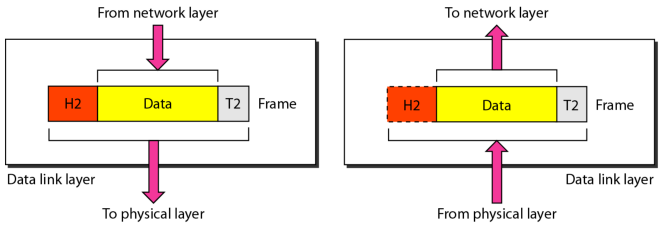


Figura 2.18 – Il livello fisico: la sequenza di bit di T1 diventa un segnale sul mezzo e ricompare identica (se tutto va bene) in T2.

2. Livello di collegamento (Data Link)



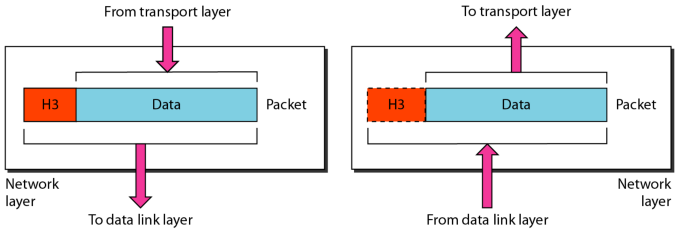
È responsabile di **spostare i frame da un nodo al successivo**: un solo salto (*hop*), non l'intero cammino. Si occupa di:

- organizzare i dati in **frame**, blocchi di dimensione limitata (tipicamente qualche centinaio di byte), con header H_2 e trailer T_2;
- **controllo di flusso** e **controllo degli errori**, tramite la *frame check sequence* (FCS) nel trailer;
- **controllo dell'accesso** al mezzo quando è condiviso.

Sottostrato MAC

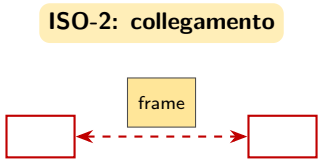
Il livello di collegamento contiene il **sottostrato MAC** (*Medium Access Control*), che si occupa della **gestione dell'accesso a un unico mezzo di trasmissione** da parte di più stazioni **in competizione** tra loro. Ethernet ne è l'esempio più noto.

3. Livello di rete (Network)

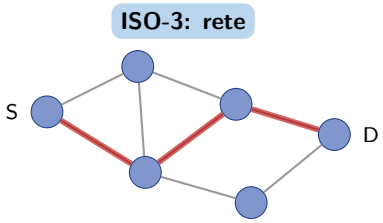


È responsabile di **consegnare i pacchetti dall'host sorgente all'host destinazione**, lungo l'intero cammino. Si occupa di:

- attraversare la **topologia**: la rete è un grafo di nodi con più di una strada possibile;
- **indirizzamento logico** (gli indirizzi IP);
- **routing**: decidere, nodo per nodo, da quale collegamento far uscire il pacchetto (tabelle di instradamento).



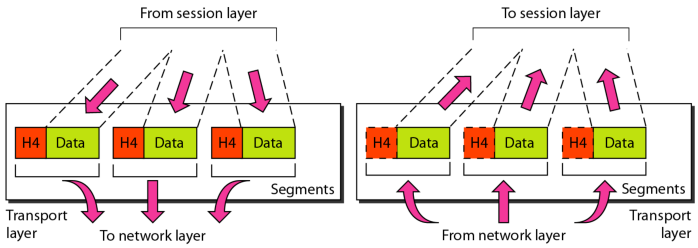
ci sono **solo due nodi** che comunicano tra loro; viaggiano frame di qualche centinaio di byte



chi trasmette attraverso una **topologia complessa**? Il livello di rete, usando i servizi del collegamento su ogni salto

Figura 2.19 – Il collegamento lavora su un solo salto tra due nodi; la rete compone i salti in un cammino dalla sorgente alla destinazione.

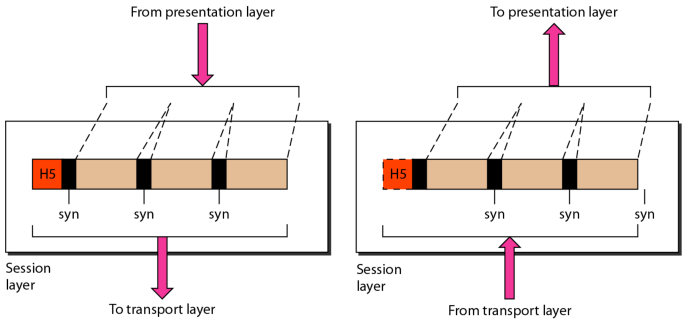
4. Livello di trasporto (Transport)



È responsabile di **consegnare un messaggio da un estremo all'altro** e al **programma giusto** sull'host di destinazione. Si occupa di:

- **indirizzamento tramite porte**: il livello di rete trova l'*host*, il trasporto trova il *processo*;
- controllo della connessione (con o senza connessione);
- **segmentazione e riassemblaggio** dei messaggi in segmenti;
- controllo di flusso e degli errori, da un estremo all'altro.

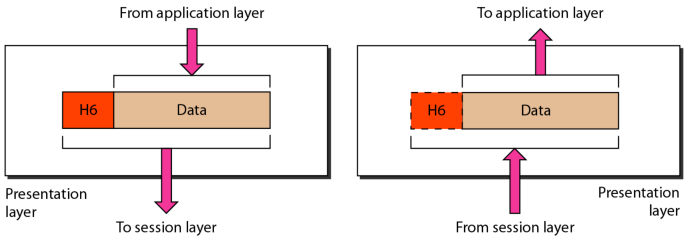
5. Livello di sessione (Session)



È responsabile del **controllo del dialogo**: stabilisce, mantiene e sincronizza l'interazione tra i due sistemi.

- **chi parla, quando e per quanto**;
- **punti di sincronizzazione** (*checkpoint*, i `syn` della figura): uno scambio interrotto riprende dall'ultimo checkpoint invece che dall'inizio.

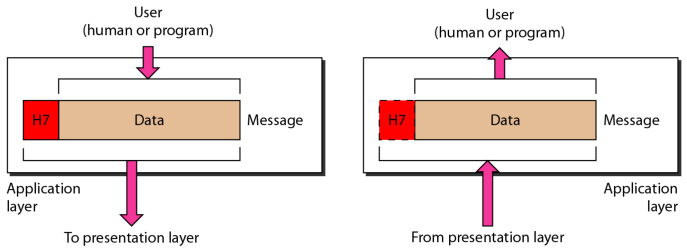
6. Livello di presentazione (Presentation)



È responsabile della **rappresentazione dei dati**, perché ciò che scrive una parte sia ciò che legge l'altra:

- **traduzione** tra codifiche diverse (il caso storico è ASCII contro EBCDIC);
- **cifratura** e decifratura;
- **compressione**.

7. Livello applicativo (Application)



È responsabile di **fornire servizi all'utente** (una persona o un programma), ed è dove vivono i protocolli che già conoscete per nome:

- login remoto (terminale virtuale di rete);
- trasferimento e accesso ai file;
- posta elettronica;
- accesso al World Wide Web.

È lo strato che apriremo per primo, perché è quello che si vede.

PERCHÉ OSI NON HA VINTO

Il modello OSI è lo standard *de iure*, ma i suoi protocolli non hanno avuto successo. Le ragioni sono quattro:

- **Cattivo tempismo** (*bad timing*): quando comparvero i protocolli OSI, TCP/IP era già diffuso nelle università di ricerca, e nessun produttore voleva sostenere un secondo stack.
- **Cattiva progettazione** (*bad design*): sia il modello sia i protocolli avevano difetti. Sette strati erano un numero "politico": sessione e presentazione sono quasi vuoti, collegamento e rete sono sovraccarichi.
- **Cattive implementazioni** (*bad implementations*): le prime erano enormi, goffe e lente, e la cattiva reputazione è rimasta anche dopo che i prodotti erano migliorati.
- **Cattiva politica** (*bad politics*): era percepito come una creatura dei ministeri delle telecomunicazioni europei, della Comunità Europea e poi del governo statunitense, mentre TCP/IP arrivava gratis insieme a Berkeley UNIX.

Il **modello** OSI è sopravvissuto ed è ancora il modo in cui si parla di reti; i suoi **protocolli** no.

L'apocalisse dei due elefanti

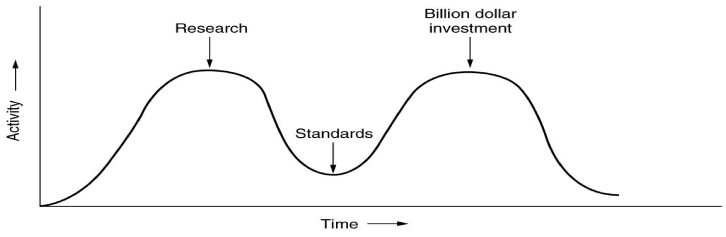


Figura 2.20 – L'attività su un argomento: prima la ricerca, poi il miliardo di dollari di investimenti.

Quando nasce un nuovo argomento, l'attività ha due "gobbe" (i due elefanti): prima un picco di **ricerca**, poi, anni dopo, un picco di **investimenti industriali**. Tra le due c'è una conca.

- Uno standard scritto **troppo presto** congela un argomento che nessuno ha ancora capito.
- Uno standard scritto **troppo tardi** viene ignorato da chi ha già investito in altro.

Per evitare il cattivo tempismo, gli standard vanno scritti **nella conca tra le due gobbe**. OSI non ci riuscì: arrivò quando TCP/IP aveva già occupato il secondo elefante.

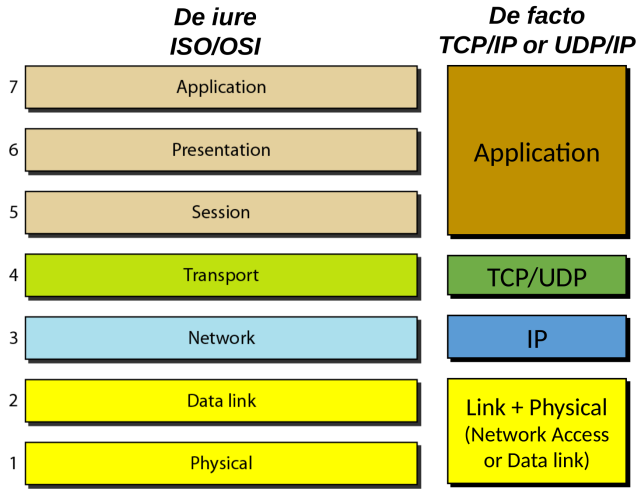
Cosa non va nel modello TCP/IP

Per correttezza, anche il *modello* TCP/IP ha i suoi difetti:

- **non distingue chiaramente** i concetti di servizio, interfaccia e protocollo, cioè proprio ciò che OSI aveva fatto bene;
- **non è generale**: non si riesce a usarlo per descrivere altre pile di protocolli (provate a descrivere Bluetooth con il modello TCP/IP);
- il suo **livello di collegamento non è un vero strato**, ma un'interfaccia tra l'host e il collegamento trasmissivo;
- **non distingue** tra livello fisico e livello di collegamento, che sono due compiti completamente diversi.

I suoi **protocolli**, però, erano progettati con cura, ben implementati e distribuiti **gratuitamente**.

De iure e de facto



- **De iure**: ISO/OSI è il modello standard per le reti di calcolatori: completo, dettagliato e complesso.
- **De facto**: i protocolli che girano davvero su Internet e sulle reti locali sono **TCP**, **UDP** e **IP** (la *Internet Protocol Suite*):
  - **TCP**: *Transmission Control Protocol*;
  - **UDP**: *User Datagram Protocol*;
  - **IP**: *Internet Protocol*.

In pratica non sono rivali: teniamo il **modello** per ragionare e i **protocolli** per lavorare.

Figura 2.21 – Corrispondenza tra la pila ISO/OSI e la pila TCP/IP.

IP è il collante

Il protocollo IP è il "collante" di tutta la rete di reti che chiamiamo Internet. È l'unico punto in cui tutti sono obbligati a mettersi d'accordo: sopra IP ci possono essere TCP o UDP, sotto IP Ethernet, Wi-Fi o qualunque altra cosa. È la "vita stretta" della clessidra vista nel capitolo 1.

I cinque strati usati nel resto del corso

Da qui in avanti il corso conta **cinque strati**, non sette: è il modello OSI senza sessione e presentazione, i cui pochi compiti reali vengono svolti direttamente dalle applicazioni. I nomi sono quelli del Kurose.

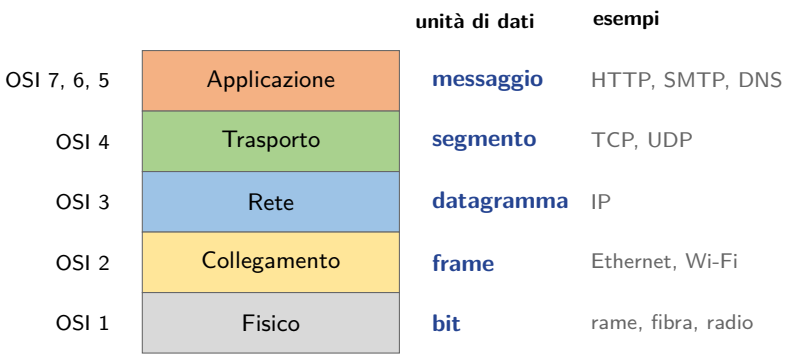


Figura 2.22 – La pila a cinque strati: corrispondenza con OSI, nome dell'unità di dati ed esempi di protocolli.

Strato	Corrisponde a OSI	Unità di dati
Applicazione	7, 6 e 5	messaggio
Trasporto	4	segmento
Rete	3	datagramma
Collegamento	2	frame
Fisico	1	bit

Tabella 2.4 – I cinque strati del corso.

Numeri OSI e nomi del corso

I numeri OSI (“livello 3”, “livello 2”) restano in uso perché li usano tutti nel settore, e li useremo anche noi quando serve. Ma la pila che studiamo, dall’alto verso il basso, è quella a cinque strati.

La stratificazione oltre Internet

Esistono altre pile di protocolli, scelte a seconda del contesto: **RS-232** per una linea seriale, **CAN** all’interno di un veicolo o in automazione industriale. Sono molto più semplici, e sono comunque stratificate.

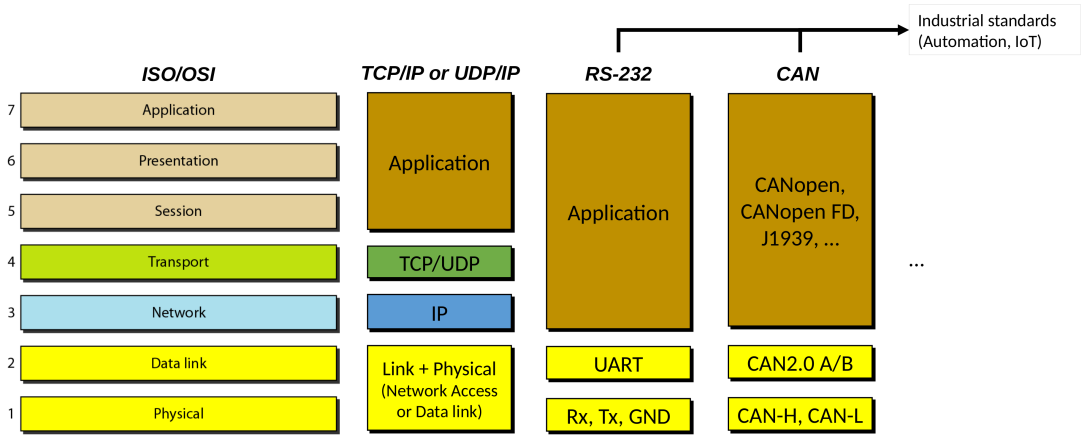


Figura 2.23 – Modelli a strati in altri contesti (standard industriali, automazione, IoT).

Strato OSI	TCP/IP o UDP/IP	RS-232	CAN
7–5	Applicazione	Applicazione	CANopen, CANopen FD, J1939, …
4	TCP/UDP		
3	IP		
2	Link + Fisico	UART	CAN 2.0 A/B
1		Rx, Tx, GND	CAN-H, CAN-L

Tabella 2.5 – In RS-232 e CAN i livelli di collegamento e fisico sono distinti, e tutto il resto è “applicazione”.

Riepilogo

In breve

- Una richiesta attraversa troppi pezzi per essere progettata tutta insieme: il sistema si divide in **strati** (*divide et impera*).
- Uno strato offre un **servizio** verso l'alto attraverso un'**interfaccia**, e usa un **protocollo** in orizzontale con il suo pari. Le due cose sono indipendenti.
- La comunicazione tra pari è **logica**: i dati scendono una pila e risalgono l'altra, e solo il livello fisico sposta davvero qualcosa.
- I servizi possono essere **con** o **senza connessione**, **affidabili** o no: TCP darà un flusso di byte affidabile, UDP un datagram inaffidabile.
- L'**incapsulamento** rende possibile tutto questo: ogni strato tratta ciò che trasporta come byte opachi.
- OSI ci ha dato sette strati con cui ragionare, TCP/IP i protocolli; nel corso usiamo **cinque strati** con i nomi del Kurose: messaggio, segmento, datagramma, frame, bit.

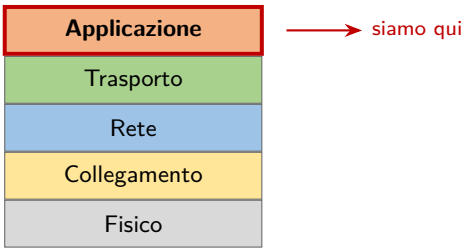
Nella prossima parte apriamo la cima della pila, dove stanno le applicazioni.

Parte II

Il livello applicativo

Il livello applicativo: architetture e servizi

Cominciamo la discesa lungo la pila partendo, come vuole l'approccio top-down, dallo strato più alto: quello che l'utente vede.



Abbiamo l'intera pila; ora apriamo lo strato in cima.

- Un'applicazione **non sposta dati**: li consegna allo strato sotto e li ritrova all'altro capo.
- Il trasporto le offre (se lo chiede) un **flusso di dati affidabile**: niente perso, niente fuori ordine.
- Due applicazioni su due host si comportano **come se parlassero direttamente**, purché seguano lo stesso protocollo.

Che cosa resta da fare al programmatore?

Tutto ciò che sta **sopra** quel flusso: **quali messaggi** esistono, **che cosa significano** e **chi parla per primo**. È esattamente ciò che definisce un protocollo applicativo.

Che cos'è un'applicazione di rete

Applicazione di rete

Un'**applicazione di rete** è composta da **più programmi** che girano su **end system diversi** (host) e che **comunicano tra loro attraverso la rete**.

Esempio 1 — Un'applicazione Web

Un'applicazione Web è fatta di due programmi distinti:

- il **browser**, in esecuzione sull'host dell'utente (desktop, laptop, tablet, smartphone);
- il **server Web**, in esecuzione sull'host che ospita il sito.

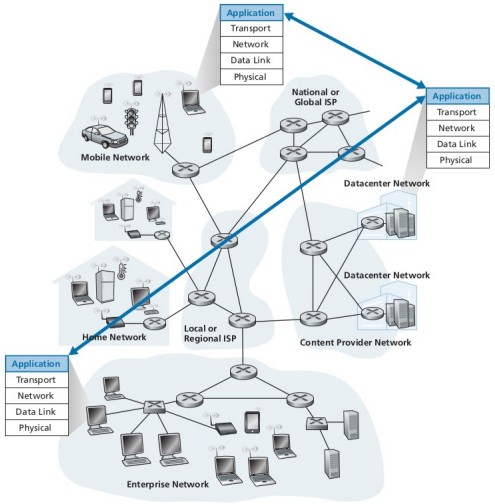
Possono essere scritti in linguaggi diversi e girare su sistemi operativi diversi: l'unica cosa che devono condividere è il **protocollo** (HTTP).

Qualche applicazione di rete tipica: social network, Web, messaggistica, posta elettronica, giochi multiutente, streaming video (YouTube, Netflix), condivisione di file P2P, voice over IP, videoconferenza, ricerca su Internet, login remoto.

Il Web è tecnicamente un'applicazione

Nel linguaggio comune "Internet" e "Web" sono usati come sinonimi, ma il Web è **una delle applicazioni** che girano sopra Internet, non è Internet.

Creare un'applicazione di rete



Creare un'applicazione di rete significa scrivere programmi che:

- girano su **end system diversi**, magari scritti in linguaggi diversi o per sistemi operativi diversi;
- comunicano attraverso la rete (per esempio tramite **socket**) usando uno specifico **protocollo**.

Notate nella figura che la pila completa (applicazione, trasporto, rete, collegamento, fisico) c'è **solo sugli host** ai bordi della rete.

Figura 3.1 – Le applicazioni girano solo sugli end system.

Non si scrive software per il nucleo della rete

Non c'è bisogno di scrivere software per i dispositivi del *network core* (router, switch):

- i dispositivi di rete **non eseguono applicazioni utente**;
- esistono librerie specifiche (i **socket**) che implementano le funzionalità di rete.

È una conseguenza diretta del modello a strati: l'applicazione vede soltanto il servizio che le offre il trasporto, e ignora come sia realizzato. Router e switch non eseguono applicazioni utente, e non vi lascerebbero installarne una. Questa restrizione **non è un limite**: è proprio il motivo per cui nuove applicazioni nascono e si diffondono così in fretta, senza toccare l'infrastruttura.

ARCHITETTURE DELLE APPLICAZIONI DI RETE

Architettura applicativa

L'**architettura** di un'applicazione di rete è progettata dallo **sviluppatore** dell'applicazione e determina come l'applicazione è strutturata sui vari end system. Le due architetture fondamentali sono **client-server** e **peer-to-peer**.

Architettura applicativa e architettura di rete

Non confondete l'*architettura dell'applicazione* (decisa dallo sviluppatore: client-server o P2P) con l'*architettura della rete* (la pila a cinque strati di Internet, che è fissata e offre all'applicazione un insieme di servizi).

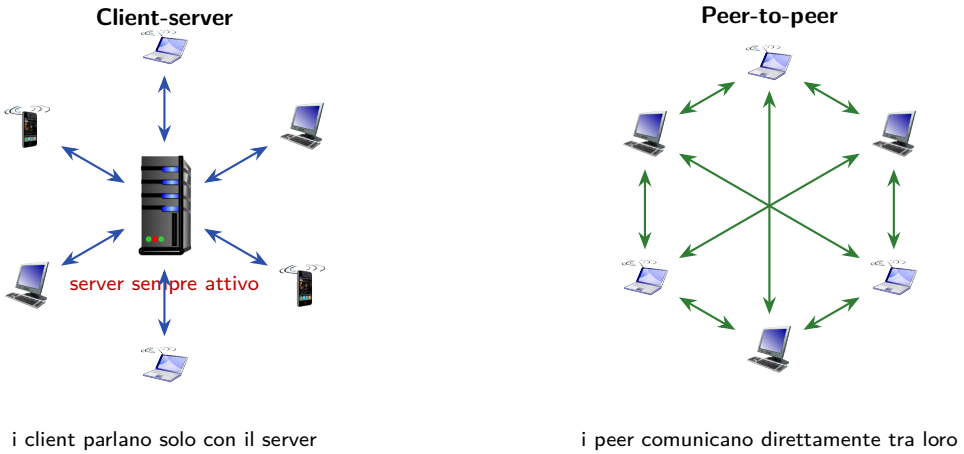


Figura 3.2 – Le due architetture applicative fondamentali.

Architettura client-server

Architettura client-server

I nodi sono **eterogenei**: esiste un host **sempre attivo** (il *server*) che fornisce servizi in risposta alle richieste di molti altri host (i *client*).

- I client **non comunicano direttamente** tra loro: due browser non si scambiano mai nulla.
- Il server ha un **indirizzo fisso e ben noto** (un nome o un indirizzo IP): un client può sempre contattarlo inviandogli un pacchetto di richiesta.
- **Il client parla per primo**: il server aspetta di essere contattato, e risponde.
- Spesso servono **molti server** per reggere tutte le richieste; il client di solito non se ne accorge e li percepisce come un server unico.

I server multipli possono essere organizzati in tre modi:

- **raggruppati in data center**, con un numero enorme di server in un unico luogo: vanno alimentati, raffreddati, mantenuti e ben collegati;
- **sparsi come server distribuiti** in tutto il mondo: vanno interconnessi e coordinati;
- organizzati in **data center distribuiti**, combinando le due soluzioni: è ciò che fanno davvero i grandi fornitori.

E il fornitore continua a pagare, mese dopo mese, l'interconnessione e la banda che esce da quegli edifici.

La conseguenza del client-server

Tutto ciò che l'applicazione fa passa per il server, che è quindi insieme il **limite alla scalabilità** (quanti client si possono servire) e il **punto di guasto unico** (*single point of failure*): se il server cade, il servizio non esiste più per nessuno.

Esempio 2 — Il Web come applicazione client-server

C'è un server Web sempre attivo che riceve richieste dai browser degli utenti. Quando riceve la richiesta di un oggetto (una pagina, un'immagine), risponde inviando l'oggetto richiesto. Il server è raggiungibile da tutti gli host, perché il suo indirizzo è noto, ed è acceso quando lo si interroga. Richiesta, risposta, e un server che c'era già prima che qualcuno chiedesse: è tutta qui la forma del client-server.

Architettura peer-to-peer (P2P)

Architettura peer-to-peer

I nodi sono (idealmente) **omogenei**: l'applicazione sfrutta la **comunicazione diretta** tra coppie di host collegati in modo **intermittente**, detti *peer*. I peer non sono di proprietà del fornitore del servizio, ma sono computer controllati dagli utenti.

- È usata spesso per applicazioni ad **alta intensità di traffico** (condivisione di file).
- Non c'è un server unico con indirizzo fisso: ogni peer ha il proprio indirizzo, che può anche cambiare.
- È **auto-scalabile** (*self-scalability*): ogni peer genera carico chiedendo un file, ma aggiunge anche **capacità** servendolo ad altri. La capacità cresce insieme alla domanda: nessun progetto client-server ha questa proprietà.
- È **economica**: non servono infrastrutture né grande banda da parte del fornitore.
- Ha però problemi di **sicurezza**, **prestazioni** e **affidabilità**: nessuno possiede le macchine coinvolte.

Il P2P puro è raro: le architetture ibride

Le applicazioni P2P pure sono rare: la maggior parte ha un'architettura **ibrida**. In molte applicazioni di messaggistica e di telefonia su Internet un **server** tiene traccia degli indirizzi IP degli utenti, ma i messaggi da utente a utente viaggiano **direttamente** tra gli host, senza passare per server intermedi. Nella condivisione di file il server può anche tenere l'elenco dei file disponibili, per velocizzare la ricerca (così facevano Napster e, in parte, BitTorrent con i tracker).

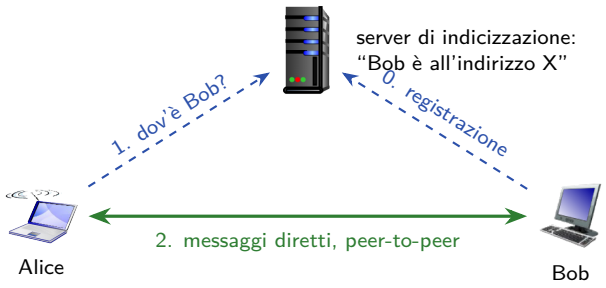


Figura 3.3 – Un'architettura ibrida: il server serve solo a trovarsi, poi i peer parlano direttamente.

COMUNICAZIONE TRA PROCESSI

Come abbiamo già detto, chi comunica **non è un programma e non è una macchina**, ma un **processo**: un programma in esecuzione, su macchine diverse e potenzialmente con sistemi operativi diversi.

- Nel Web il processo del browser scambia messaggi con il processo del server Web.
- Nella condivisione di file P2P un file passa dal processo di un peer al processo di un altro peer.

Processo client e processo server

In ogni coppia di processi che comunicano:
- il **processo client** è quello che **inizia** la comunicazione;
- il **processo server** è quello che **attende** di essere contattato.

Nel Web il browser è il client e il server Web è il server. Nel P2P il peer che scarica fa da client e quello che carica fa da server: in un'applicazione P2P i processi possono **cambiare ruolo**, e lo stesso peer può essere client su una connessione e server su un'altra.

I socket

La maggior parte delle applicazioni è fatta di coppie di processi che si mandano messaggi attraverso la rete sottostante. Il punto in cui un processo "consegna" i messaggi alla rete ha un nome preciso.

Socket

Seguendo il modello a strati, un **socket** è l'**interfaccia software tra il livello applicativo e il livello di trasporto**: è la "porta" attraverso cui un processo invia e riceve messaggi sulla rete. È anche l'**API** (*Application Programming Interface*) tra l'applicazione e la rete.

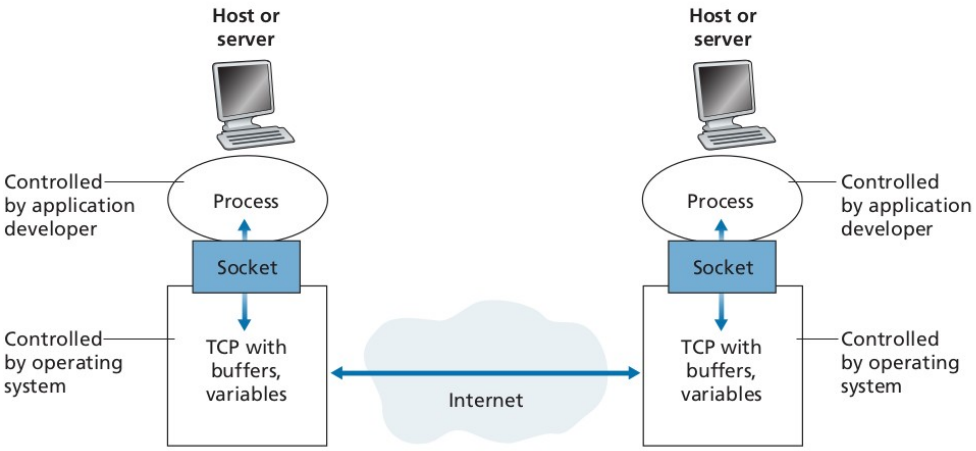
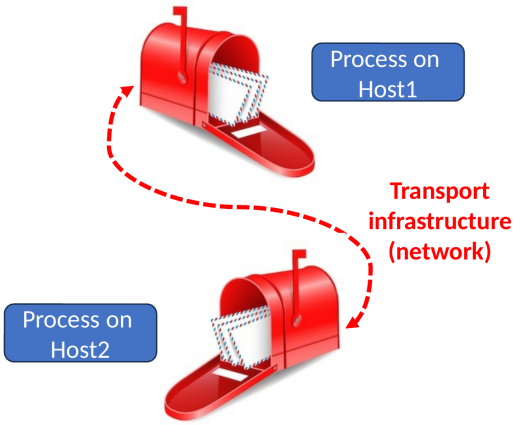


Figura 3.4 – Il processo è controllato dallo sviluppatore; ciò che sta sotto il socket (TCP con i suoi buffer e le sue variabili) è controllato dal sistema operativo.

Esempio 3 — I socket come cassette postali



- Un processo che vuole mandare un messaggio a un processo su un altro host **lo mette nella cassetta postale** (il socket).
- Il processo mittente dà per scontato che, dall'altra parte della sua porta, ci sia un'**infrastruttura di trasporto** che porterà il messaggio fino alla cassetta del destinatario.
- Quando il messaggio arriva all'host di destinazione, passa attraverso la cassetta (il socket) del processo ricevente.

Chi imbuca una lettera non deve sapere come funziona il sistema postale.

Figura 3.5 – Un processo è una casa e il suo socket è la porta: il mittente spinge fuori il messaggio e dà per scontato che qualcosa, fuori, lo porti a destinazione.

Cosa controlla lo sviluppatore

Lo sviluppatore ha il **controllo completo** di tutto ciò che sta sul lato applicativo del socket, ma **pochissimo controllo** sul lato del trasporto. Può soltanto:

1. **scegliere il protocollo di trasporto** (TCP o UDP);
2. eventualmente impostare qualche parametro del trasporto (dimensione massima dei buffer, dei segmenti, …).

Scelto il protocollo, l'applicazione viene costruita dando per scontati i servizi che quel protocollo offre. La scelta si fa **una volta, all'inizio**, e tutto il resto ne discende: è una **decisione di progetto**, non un dettaglio di configurazione.

Indirizzare i processi: indirizzo e identificatore

Riprendendo l'analogia postale, un processo ha bisogno di un **indirizzo** per ricevere messaggi. Ma su un host girano contemporaneamente molte applicazioni di rete: per individuare il processo destinatario servono **due** informazioni.

1. Un **indirizzo dell'host**, per trovare la macchina giusta sulla rete.
2. Un **identificatore del processo**, per trovare il processo giusto dentro quella macchina.

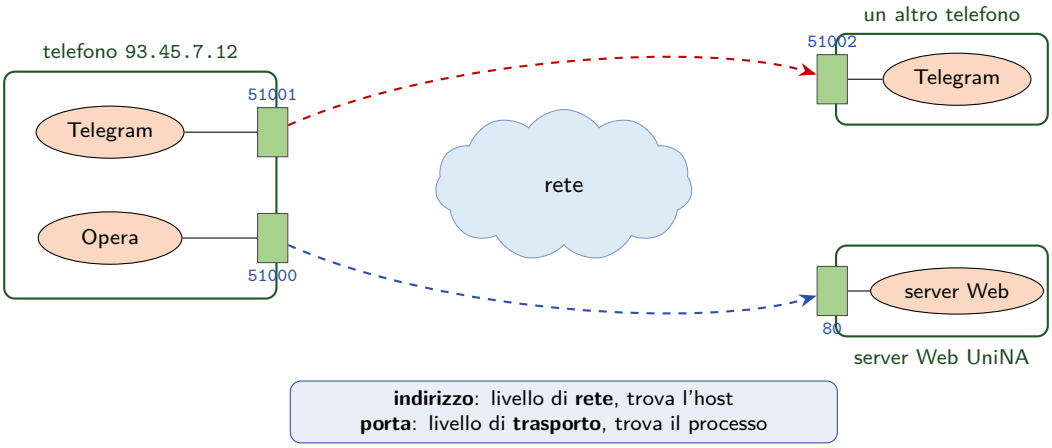


Figura 3.6 – Un telefono con la messaggistica e il browser aperti, che parlano con due destinazioni diverse nello stesso momento. Entrambi i flussi partono dallo stesso indirizzo: a tenerli separati, a entrambi i capi, è la porta.

Indirizzo IP e numero di porta

In Internet:

- un **host** è identificato da un **indirizzo IP** (in IPv4, una quantità a 32 bit, scritta come quattro numeri tra 0 e 255, per esempio `143.225.161.30`);
- un **processo** è identificato da un **numero di porta** (16 bit).

L'indirizzo lavora a livello di **rete** (instradamento), l'identificatore a livello di **trasporto**.

Le applicazioni più diffuse hanno **numeri di porta assegnati per convenzione**: un server Web ascolta sulla porta **80** (443 per HTTPS), un server di posta SMTP sulla porta **25**. L'elenco ufficiale delle porte ben note è mantenuto dalla **IANA** (*Internet Assigned Numbers Authority*). È questa convenzione che permette a un client di contattare un servizio mai visto prima: senza, ogni client dovrebbe sapere in anticipo "a quale porta bussare".

Un indirizzo appartiene a un'interfaccia

Un portatile è collegato contemporaneamente via cavo Ethernet e via Wi-Fi: quanti indirizzi ha? **Due**, uno per interfaccia. Un indirizzo IP non dà il nome a una macchina: dà il nome a **un punto di collegamento a una rete**.

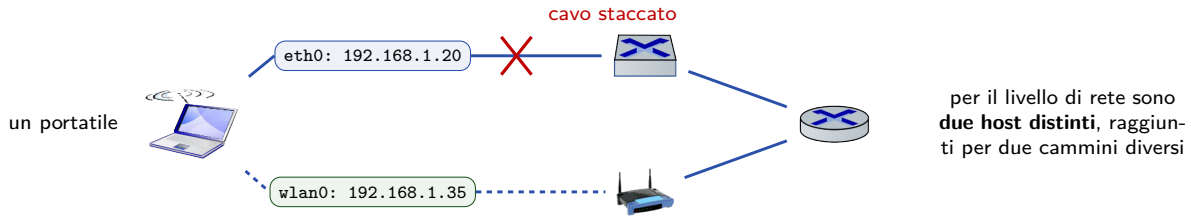


Figura 3.7 – Staccando il cavo, l'indirizzo di `eth0` smette di rispondere, mentre quello di `wlan0` continua a funzionare.

Ci torneremo quando gli indirizzi smetteranno di appartenere alle macchine e cominceranno ad appartenere alle **reti** (capitolo sull'indirizzamento IP).

IPv4 e l'indirizzo che vuol dire "qui"

Indirizzo IPv4 e localhost

Un indirizzo **IPv4** è una quantità a **32 bit**, scritta come quattro numeri decimali tra 0 e 255 separati da punti, uno per byte. L'indirizzo `127.0.0.1` (`localhost`) è speciale: non indica una macchina remota ma **questa** macchina. Due processi locali comunicano attraverso di esso esattamente come farebbero attraverso la rete, ma **nulla lascia l'host**.

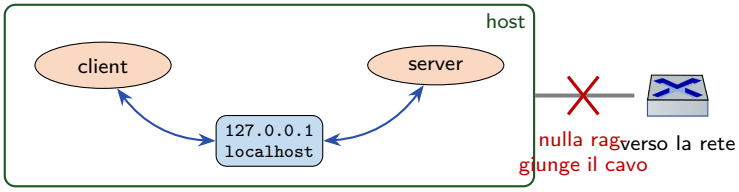


Figura 3.8 – Client e server sulla stessa macchina: è così che proverete i vostri programmi prima di avere un secondo host.

Un nome non è un indirizzo

Nessuno scrive `143.225.161.30`: si scrive `www.unina.it`, e i due designano lo stesso server Web.

- Il **nome** è per le persone: leggibile, stabile, e sopravvive al cambio della macchina che c'è sotto.
- L'**indirizzo** è ciò su cui lavora il livello di rete: è ciò che serve all'instradamento per consegnare.
- Il **DNS** (*Domain Name System*) traduce l'uno nell'altro. È un **database distribuito** (nessuna macchina lo contiene tutto) ed è a sua volta un'**applicazione** che gira sulla rete: lo apriremo tra poco.

I SERVIZI DI TRASPORTO DISPONIBILI ALLE APPLICAZIONI

Il livello di trasporto, sotto l'applicazione, offre protocolli che realizzano su richiesta alcune proprietà, dette **qualità del servizio** (QoS, *Quality of Service*).

- **Affidabilità** del trasferimento dati: i dati inviati da un estremo arrivano **corretti e completi** all'altro. Alcune applicazioni (file, posta, transazioni bancarie) non tollerano perdite; altre (audio e video) tollerano qualche perdita.
- **Throughput**: il ritmo, in bit/s, a cui il processo mittente può consegnare bit al processo ricevente. Alcune applicazioni multimediali hanno bisogno di un throughput minimo.
- **Temporizzazione** (*timing*): ogni bit immesso nel socket arriva al socket del destinatario entro un certo tempo. Serve a telefonia, videoconferenza, giochi in tempo reale.
- **Sicurezza**: cifratura e decifratura dei messaggi, integrità, autenticazione.

Scegliere un protocollo di trasporto significa scegliere tra queste quattro cose. La domanda da farsi è: **quale delle quattro serve davvero alla mia applicazione?**

Applicazione	Perdite	Throughput	Sensibile al tempo
Trasferimento file	nessuna	elastico	no
Posta elettronica	nessuna	elastico	no
Documenti Web	nessuna	elastico, pochi kb/s	no
Telefonia su Internet	tollerate	da pochi kb/s a 1 Mb/s	sì, centinaia di ms
Videoconferenza	tollerate	da 10 kb/s a 5 Mb/s	sì, centinaia di ms
Audio/video registrato	tollerate	come sopra	sì, pochi secondi
Giochi interattivi	tollerate	da pochi kb/s a 10 kb/s	sì, centinaia di ms

Tabella 3.1 – Che cosa chiedono al trasporto le applicazioni reali.

Throughput elastico

Un'applicazione ha un throughput **elastico** se usa tutto il throughput che trova: di più è meglio, ma anche con meno funziona (più lentamente).

Internet offre alle applicazioni **due** protocolli di trasporto:

	TCP (*Transmission Control Protocol*)	**UDP** (*User Datagram Protocol*)
Connessione	**sì**: handshake tra i processi prima dei dati	**no**: si invia e basta
Affidabilità	trasferimento **affidabile** e ordinato	nessuna garanzia di consegna
Controlli	di flusso e di congestione	nessuno
Servizio	flusso di byte affidabile	datagram inaffidabile, "minimale"
Non offre	garanzie di tempo e di throughput minimo	garanzie di tempo e di throughput minimo

Tabella 3.2 – I due servizi di trasporto di Internet.

TCP rallenta il mittente quando la rete è congestionata: lo fa **per il bene della rete**, non della vostra applicazione.

UDP non è un TCP difettoso

UDP è ciò che resta quando si **rifiuta di pagare** per le garanzie: niente handshake, niente conferme, niente rallentamenti imposti. Per un'applicazione che deve rispondere in fretta, o che gestisce da sé le perdite, è la scelta giusta.

Che cosa Internet non garantisce

Dei quattro servizi elencati all'inizio, i protocolli di trasporto di Internet ne forniscono **uno solo**: l'affidabilità (con TCP).

- **Nessuna garanzia di throughput né di tempo**: né TCP né UDP le offrono. Le applicazioni sensibili al tempo funzionano lo stesso perché sono **progettate per farcela**, non perché la rete le aiuti.
- **Nessuna sicurezza**: ciò che si spinge nel socket viaggia così come è stato scritto, password compresa.

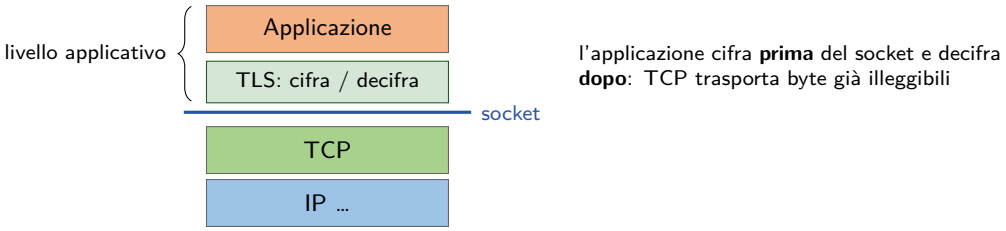


Figura 3.9 – TLS non è un terzo protocollo di trasporto: è un miglioramento di TCP, implementato nel livello applicativo.

I PROTOCOLLI DI LIVELLO APPLICATIVO

Protocollo di livello applicativo

Un **protocollo applicativo** definisce come i processi di un'applicazione, in esecuzione su end system diversi, si scambiano i messaggi. In particolare definisce:

1. i **tipi** di messaggi scambiati (per esempio richieste e risposte);
2. la **sintassi** dei vari tipi di messaggio: quali campi ci sono e come sono delimitati;
3. la **semantica** dei campi: il significato dell'informazione che contengono;
4. le **regole** che stabiliscono quando e come un processo invia messaggi e risponde.

I protocolli possono essere **aperti** (le regole sono pubbliche, tipicamente in un RFC: HTTP, SMTP, DNS) oppure **proprietari**, deliberatamente chiusi (come quello di Skype o di molte app di messaggistica).

Un protocollo non è l'applicazione

Il Web è un'applicazione; HTTP ne è **un pezzo**.

Figura 3.10 – Il Web ha bisogno di un formato per i documenti (HTML), di browser e di server: HTTP fissa soltanto formato e ordine dei messaggi tra browser e server.

Lo stesso vale per il video on demand: server, fatturazione, l'app sul telefono, e **DASH** per i messaggi. Perché insistere sulla distinzione? Perché il protocollo è **l'unica parte su cui entrambi i capi devono concordare**: tutto il resto ciascuna parte lo costruisce come vuole.

Protocolli applicativi comuni

Ecco alcuni protocolli applicativi di uso comune, con il trasporto che usano:

Protocollo	Descrizione	Porta	Trasporto
DHCP	*Dynamic Host Configuration Protocol*: assegna gli indirizzi IP	67, 68	UDP
DNS	*Domain Name System*: traduce i nomi in indirizzi IP	53	UDP, TCP
HTTP / HTTPS	*HyperText Transfer Protocol (Secure)*: trasferisce pagine Web	80 / 443	TCP
SMTP / SMTPS	*Simple Mail Transfer Protocol (Secure)*: invia la posta	25 / 465	TCP
SNMP	*Simple Network Management Protocol*: gestisce i dispositivi di rete	161	UDP
Telnet / SSH	*Teletype Network / Secure SHell*: riga di comando su host remoti	23 / 22	TCP
FTP / FTPS	*File Transfer Protocol (Secure)*: trasferisce file	21 / 990	TCP

Tabella 3.3 – Protocolli applicativi comuni e relative porte ben note.

Le versioni "Secure"

Molte applicazioni scambiano informazioni sensibili (credenziali, dati personali). Sulle reti pubbliche i messaggi vanno protetti: da qui le varianti sicure (HTTPS, SMTPS, FTPS, SSH) di quasi ogni protocollo. Telnet, per esempio, trasmette tutto in chiaro, password comprese; SSH offre lo stesso servizio (una riga di comando remota) con tutto cifrato. Su una rete pubblica, usare la variante in chiaro è un errore.

FTP e FTPS

FTP

L'**FTP** (*File Transfer Protocol*) è uno dei protocolli più antichi di Internet (prima versione nel 1971) e serve a **trasferire file** tra host attraverso la rete, secondo il modello client-server.

- Nella versione base il trasferimento avviene **in chiaro**: nome utente, password e file viaggiano leggibili da chiunque intercetti il traffico. Va quindi usato in reti locali o private.
- La versione sicura **FTPS** (*FTP Secure*) protegge nome utente e password e cifra i contenuti.
- Sia FTP sia FTPS hanno due componenti: il **protocollo**, che specifica i comandi (elencare, prendere, cancellare file, …), e un'**applicazione** che lo implementa, con un lato client e un lato server.

Due connessioni: controllo e dati

A differenza dei protocolli visti finora, FTP **non** trasporta comandi e dati sulla stessa connessione.

- La **connessione di controllo**, sulla porta **21**, si apre per prima e resta aperta per tutta la sessione: solo comandi e risposte.
- Ogni trasferimento (un `get`, un `put`, persino un `ls`) apre una **seconda connessione** solo per i dati, che viene chiusa alla fine del trasferimento.
- Tenere il controllo separato dai dati si chiama segnalazione **fuori banda** (*out-of-band*).

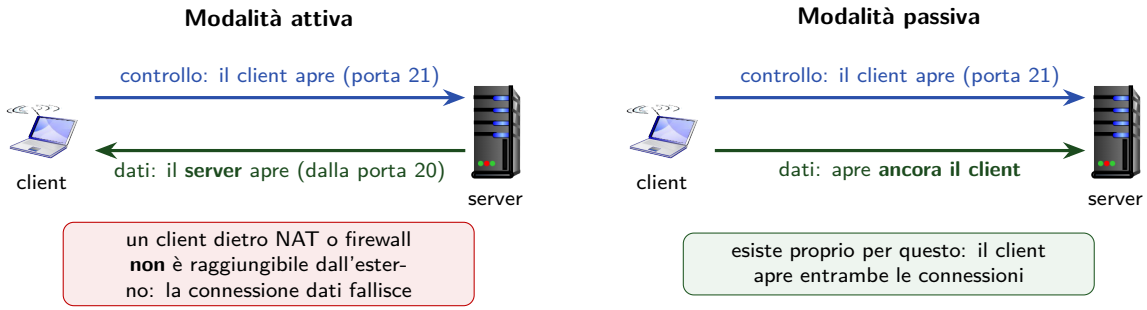


Figura 3.11 – Chi apre la connessione dati? In modalità attiva il server si ricollega al client; in modalità passiva è il client ad aprire entrambe.

Una sessione FTP

Sul server un **demone** attende in background le connessioni; sul client si usa il programma `ftp`.

```
$ sudo apt install vsftpd          # lato server, una volta sola
$ systemctl status vsftpd          # il demone e' in esecuzione?

$ ftp 143.225.161.30               # lato client
Name: student
Password:
230 Login successful.
ftp> ls                            # da qui in poi, il prompt ftp>
ftp> get report.pdf                # dal server al client
226 Transfer complete.
ftp> bye                           # bye o quit, non exit
```

I codici numerici (230, 226) sono le **risposte** del server sulla connessione di controllo: 230 conferma l'accesso, 226 la fine del trasferimento.

Il demone

Il lato server è implementato come **demone** (*daemon*): un programma che gira in background e resta in attesa che i client si connettano. Quasi ogni server che scriverete in questo corso ha questa forma: un demone, in attesa.

Comando	Effetto
`help`	elenca i comandi FTP disponibili
`cd` / `lcd`	cambia directory sulla macchina remota / locale
`ls`	mostra file e directory nella directory remota corrente
`pwd`	stampa la directory di lavoro remota
`mkdir` / `rmdir`	crea / rimuove una directory remota
`delete`	cancella un file nella directory remota
`get`	copia un file dal server alla macchina locale (*download*)
`put`	copia un file dalla macchina locale al server (*upload*)

Tabella 3.4 – I comandi FTP principali.

Due comandi, `get` e `put`, sono la ragione per cui il protocollo esiste; il resto è navigazione. Attenzione a `lcd`: è l'unico che agisce sulla **vostra** macchina.

Riepilogo

In breve

- Un'applicazione di rete è fatta di programmi su **end system diversi**; il nucleo della rete non esegue applicazioni.
- Architetture: **client-server** (server sempre attivo, indirizzo noto) e **P2P** (peer intermittenti, scalabile); nella pratica spesso **ibride**.
- Comunicano **processi**, attraverso i **socket**: l'interfaccia tra applicazione e trasporto.
- Un processo si raggiunge con **indirizzo IP** (host, livello di rete) + **numero di porta** (processo, livello di trasporto).
- Un indirizzo IP appartiene a un'**interfaccia**; `127.0.0.1` indica l'host stesso; il DNS traduce i nomi (per le persone) in indirizzi (per la rete).
- Le applicazioni chiedono al trasporto affidabilità, throughput, tempi, sicurezza; Internet offre **TCP** (connessione, affidabile) e **UDP** (senza connessione, minimale), e garantisce solo l'affidabilità: niente throughput, tempi o sicurezza (TLS si aggiunge sopra TCP, a livello applicativo).
- Un protocollo applicativo definisce tipi, sintassi, semantica dei messaggi e regole di scambio, ed è solo **un pezzo** dell'applicazione.
- **FTP** usa due connessioni: controllo (porta 21, per tutta la sessione) e dati (una per trasferimento); modalità attiva e passiva.

Il Web e il protocollo HTTP

Dal Web a HTTP

Fino ai primi anni '90 Internet era usata soprattutto da ricercatori, docenti e studenti universitari: per collegarsi a host remoti (Telnet), trasferire file (FTP), leggere e inviare news e posta elettronica. Applicazioni utilissime, ma che lasciavano Internet sconosciuta fuori dal mondo accademico. Nei primi anni '90 arrivò il **World Wide Web**, la prima applicazione Internet a conquistare il grande pubblico.

World Wide Web

Il **World Wide Web** (WWW, o semplicemente Web) è una collezione di informazioni (documenti, immagini, video, audio, …) accessibili attraverso Internet secondo uno specifico protocollo, l'**HTTP** (*HyperText Transfer Protocol*).

La comunicazione HTTP è tipicamente **client-server**:

- un **programma client** traduce le richieste dell'utente in messaggi HTTP: è implementato nei **browser** (Chrome, Firefox, Edge, Safari, …);
- un **programma server** esegue la richiesta HTTP e restituisce una risposta HTTP.

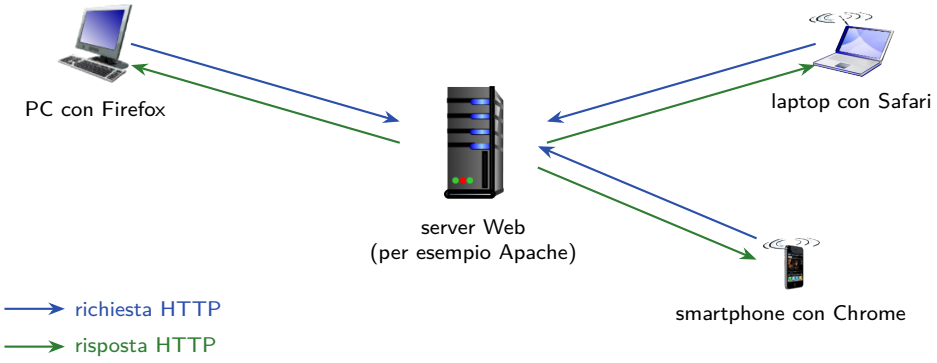


Figura 4.1 – Client diversi, con browser diversi, parlano con lo stesso server perché condividono il protocollo HTTP.

HTTP definisce principalmente due cose: la **struttura dei messaggi** e **come client e server se li scambiano**.

Cosa HTTP non definisce

HTTP non stabilisce come vadano implementati client e server, né come i contenuti vadano **visualizzati**. Il modo in cui una pagina appare dipende dal browser: due browser possono mostrare la stessa pagina in modi un po' diversi (uno aspetta che sia tutta caricata, l'altro la mostra pezzo per pezzo).

Il Web e i suoi protocolli sono la piattaforma di applicazioni come YouTube, la posta via Web (Gmail) e gran parte delle app per dispositivi mobili, social network compresi.

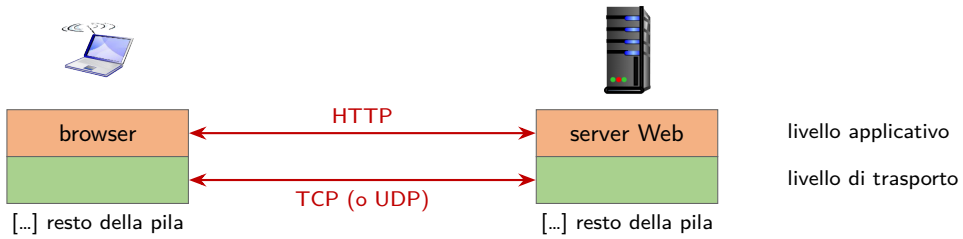


Figura 4.2 – HTTP è il protocollo tra browser e server Web; sotto, il trasporto è affidato a TCP (o, con HTTP/3, a UDP).

LE URL

Risorsa e URL

Le informazioni sul Web si chiamano **risorse** (o **oggetti**) e sono identificate da un **URL** (*Uniform Resource Locator*), una stringa con la forma generale:

`[protocollo]://[userinfo@][host][:porta][/percorso][?query][#frammento]`

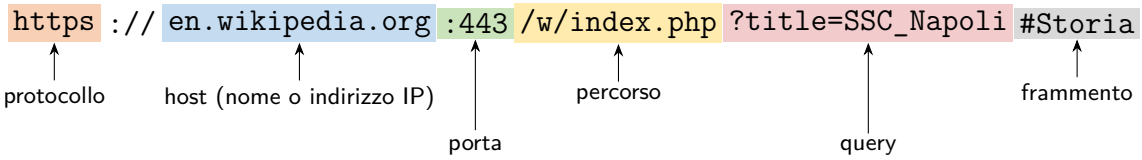


Figura 4.3 – Anatomia di un URL.

- `[protocollo]`: il protocollo con cui accedere alla risorsa (HTTP, HTTPS, FTP, …).
- `[userinfo]` (opzionale): informazioni sull'utente, come `nome:password@`. È **deprecato** per ovvi motivi di sicurezza.
- `[host]`: il nome o l'indirizzo IP del server.
- `[porta]` (opzionale): la porta da usare; di solito si ricava dal protocollo (80 per HTTP, 443 per HTTPS).
- `[percorso]`: dove si trova la risorsa dentro il server (`/percorso/della/risorsa`).
- `[query]`: preceduta da `?`, specifica eventuali richieste (parametri).
- `[frammento]`: preceduto da `#`, identifica un elemento dentro la risorsa (una sezione, un modulo).

Esempio 1 — Due URL scomposti

`http://www.someSchool.edu/someDepartment/picture1.gif`

- protocollo: `http`; host: `www.someSchool.edu`; percorso: `/someDepartment/picture1.gif`.

`https://en.wikipedia.org/w/index.php?title=SSC_Napoli`

- protocollo: `https`; host: `en.wikipedia.org`; percorso: `/w/index.php`; query: `?title=SSC_Napoli`.

In questo caso la query produce la stessa pagina di `https://en.wikipedia.org/wiki/SSC_Napoli`.

Pagine e oggetti

La maggior parte delle pagine Web è formata da un **file HTML di base** (*HyperText Markup Language*) e da vari **riferimenti** ad altri oggetti (immagini, video, script, fogli di stile), ciascuno con il proprio URL.

Esempio 2 — Quanti oggetti ha una pagina

Una pagina con un testo HTML e cinque immagini JPEG ha **sei oggetti**: il file HTML di base più le cinque immagini. Il browser riceve prima l'HTML, vi trova i riferimenti alle immagini e le chiede una a una.

Una pagina, quindi, non è un trasferimento ma **diversi** trasferimenti: ed è proprio il modo di organizzarli il tema del resto del capitolo.

Prima del primo messaggio HTTP

Tra il momento in cui si preme Invio e quello in cui parte la prima richiesta succedono già parecchie cose:

1. il browser **scompone l'URL**: protocollo, host, porta, percorso;
2. l'host è un **nome**, e verso un nome non si può aprire nessuna connessione: il **DNS** fornisce l'indirizzo IP corrispondente (lo vedremo nel capitolo dedicato);
3. solo a questo punto si apre una **connessione TCP** verso quell'indirizzo, sulla porta 80 o 443;
4. su quella connessione viaggia la prima **richiesta HTTP**, con il percorso;
5. il browser legge l'HTML, trova gli oggetti a cui rimanda e ricomincia.

Il tempo che non contiamo

Anche la risoluzione del nome costa tempo, ma nei conti in RTT che seguono non viene mai inclusa: si parte sempre da quando l'indirizzo del server è già noto.

COME FUNZIONA HTTP

HTTP definisce come i client Web chiedono oggetti ai server Web e come i server li trasferiscono. Quando l'utente chiede un oggetto (per esempio cliccando un link), il browser invia messaggi di **richiesta HTTP** per gli oggetti della pagina; il server risponde con messaggi di **risposta HTTP** che contengono gli oggetti.

HTTP evolve continuamente per bilanciare affidabilità e prestazioni:

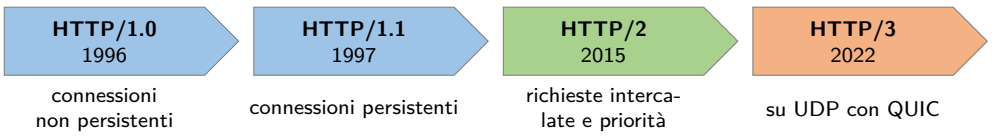


Figura 4.4 – Le versioni di HTTP.

Il protocollo di trasporto sottostante

HTTP usa principalmente **TCP** come protocollo di trasporto; dal 2022, con **HTTP/3**, può usare anche **UDP** (circa il 30% del traffico HTTP viaggia oggi su UDP).

1. Il client HTTP **apre una connessione** TCP con il server.
2. Stabilita la connessione, browser e server comunicano attraverso i loro **socket**: il client scrive richieste nel proprio socket e legge risposte dal proprio socket; il server fa il contrario.

Perché storicamente TCP? Perché offre servizi utilissimi, soprattutto il **trasferimento affidabile**: garantisce che richieste e risposte arrivino integre a destinazione (se possibile). Ma i servizi di TCP rallentano la comunicazione: per questo HTTP/3 usa UDP con sopra il protocollo **QUIC** (*Quick UDP Internet Connections*), che reimplementa l'affidabilità sopra UDP ed è stimato essere circa il 30% più veloce.

Il vantaggio della stratificazione, visto da HTTP

Qui si vede uno dei grandi vantaggi dell'architettura a strati: le applicazioni HTTP non devono preoccuparsi di raggiungibilità, perdite di dati, ritrasmissioni. Possono essere più semplici, sia logicamente sia computazionalmente, delegando gran parte del lavoro agli strati sottostanti.

HTTP È STATELESS

La semplicità è fondamentale per un server che riceve moltissime richieste al secondo (in media un server gestisce circa 1000 richieste HTTP al secondo, e l'infrastruttura di un grande motore di ricerca centinaia di migliaia di interrogazioni al secondo).

Protocollo stateless

HTTP è **senza stato** (*stateless*): il server **non conserva alcuna informazione** sulle interazioni passate con un certo client.

Esempio 3 — La stessa richiesta due volte

Se un client chiede lo stesso oggetto due volte di seguito, il server non considera la seconda richiesta ridondante: rimanda semplicemente l'oggetto, perché ha completamente dimenticato la prima.

Non tutte le applicazioni sono stateless: molte sono **stateful** (con stato). Vedremo come HTTP riesca comunque a "ricordarsi" degli utenti usando i **cookie**.

CONNESSIONI PERSISTENTI E NON PERSISTENTI

Client e server possono comunicare a lungo, scambiandosi molte coppie richiesta–risposta (una di seguito all'altra, a intervalli regolari o in modo intermittente). Con un trasporto orientato alla connessione (TCP o QUIC) ci sono due possibilità.

Connessioni persistenti e non persistenti

- **Connessione non persistente**: per **ogni** coppia richiesta–risposta si apre una **nuova** connessione, che viene chiusa subito dopo.
- **Connessione persistente**: tutte le richieste e le risposte viaggiano sulla **stessa** connessione.

Le prime versioni di HTTP (HTTP/1.0) usavano per default connessioni non persistenti; oggi il default sono le connessioni persistenti, ma client e server possono essere configurati per usare quelle non persistenti.

Esempio di connessione non persistente

Chiediamo una pagina fatta di un file HTML di base e **10 immagini JPEG**, tutte sullo stesso server, all'URL `http://www.someschool.edu/someDepartment/home.index`.

1. Il client HTTP **apre una connessione TCP** verso `www.someschool.edu` sulla porta **80**: ci sono ora due socket, uno sul client e uno sul server.
2. Il client invia nel proprio socket una **richiesta HTTP** che contiene il percorso `/someDepartment/home.index`.
3. Il server riceve la richiesta, recupera l'oggetto (da RAM o disco), lo incapsula in una **risposta HTTP** e la invia nel proprio socket.
4. Il server dice a TCP di **chiudere** la connessione; TCP la chiuderà davvero quando sarà sicuro che il client ha ricevuto la risposta integra.
5. Il client riceve la risposta e la connessione termina. Il messaggio dice che l'oggetto è un file HTML: il client lo estrae, lo esamina e trova i riferimenti ai 10 JPEG.
6. I passi 1–4 vengono ripetuti **per ciascuna delle 10 immagini**.

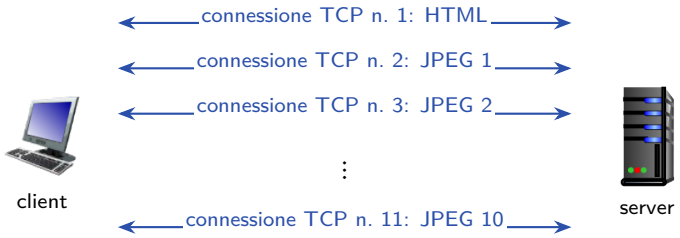


Figura 4.5 – Con connessioni non persistenti servono 11 connessioni TCP (e 11 coppie di socket) per una pagina di 11 oggetti.

Poiché la connessione non sopravvive tra un oggetto e l'altro, servono **11 connessioni TCP** per trasferire la pagina, e gli elementi possono arrivare in momenti diversi.

Connessioni in sequenza e in parallelo

Le connessioni possono essere aperte **in sequenza** o **in parallelo**. Nell'esempio, una volta ricevuto l'HTML e scoperto che servono 10 immagini, il browser può scaricarle tutte contemporaneamente.

- I browser moderni permettono di configurare il grado di parallelismo: di solito aprono **da 5 a 10 connessioni TCP parallele**, ciascuna con una sola transazione richiesta–risposta.
- Le connessioni parallele **accorciano il tempo di risposta** ma **aumentano il costo computazionale**.
- Anche con il parallelismo, aprire tante connessioni TCP ha un costo in tempo non trascurabile.

Il costo in RTT

È difficile stimare con precisione i tempi su Internet. Si può però misurare il costo di una richiesta in **RTT**.

Round-Trip Time (RTT)

L'**RTT** è il tempo impiegato da un **piccolo pacchetto** per andare dal client al server e **tornare** al client.

Il three-way handshake

Con un trasporto orientato alla connessione come TCP, prima di scambiare dati si esegue il **three-way handshake** (stretta di mano a tre vie):

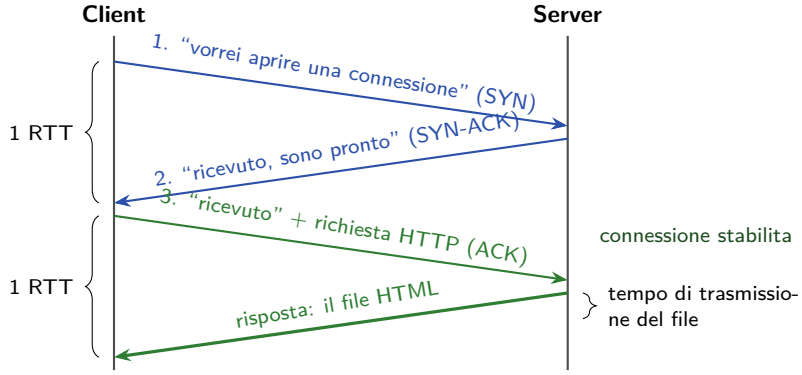


Figura 4.6 – Richiesta di un file HTML con connessione non persistente: circa 2 RTT più il tempo di trasmissione del file.

1. Il client invia un piccolo segmento TCP per segnalare che vuole aprire una connessione.
2. Il server conferma e risponde con un piccolo segmento TCP.
3. Il client conferma a sua volta.

Dal punto di vista degli RTT, i primi due passi dell'handshake richiedono **1 RTT**. Un handshake analogo avviene anche quando la connessione viene chiusa.

Il conto totale

- In HTTP il client invia la **richiesta** insieme al terzo passo dell'handshake (la conferma).
- Quando la richiesta arriva, il server invia il file HTML: la coppia richiesta–risposta costa **un altro RTT**.

Tempo di risposta con connessione non persistente

$$T_{\text{risposta}} \approx \underbrace{1\ \text{RTT}}_{\text{apertura TCP}} + \underbrace{1\ \text{RTT}}_{\text{richiesta e risposta}} + t_{\text{trasmissione del file}} = 2\ \text{RTT} + t_{\text{trasmissione}}$$

Esempio 4 — La stessa pagina, contata in quattro modi

La pagina con l'HTML e 10 immagini JPEG (11 oggetti), senza perdite e con tempi di trasmissione trascurabili, come nelle slide:

Modalità	Conto	RTT
Non persistente, un oggetto alla volta	2×11	**22**
Non persistente, 5 connessioni in parallelo	2 per l'HTML, poi due turni da 2 per le 10 immagini	**6**
Persistente, senza pipelining	1 per aprire, poi 1 per ogni oggetto	**12**
Persistente con pipelining	1 per aprire, 1 per l'HTML, 1 per le 10 immagini insieme	**3**

Con RTT = 50 ms si va da $22 \times 50 = 1{,}1$ s a $3 \times 50 = 0{,}15$ s. Stessa pagina, stesso server, stessa distanza: tutto ciò che sta fra 22 e 3 dipende solo da **come vengono gestite le connessioni**.

Connessioni persistenti e pipelining

Nelle connessioni **persistenti** il server **lascia aperta** la connessione TCP dopo aver inviato una risposta.

- Richieste e risposte successive tra lo stesso client e lo stesso server viaggiano sulla stessa connessione: l'intera pagina dell'esempio (HTML + 10 immagini) passa su **una sola** connessione TCP.
- Anche più pagine dello stesso server possono essere inviate sulla stessa connessione.
- L'handshake si paga **una volta sola**: ogni oggetto dopo il primo costa **1 RTT** invece di 2.
- Il server chiude la connessione quando resta inutilizzata per un tempo configurabile (*timeout*).

Pipelining

Con il **pipelining** le richieste di più oggetti vengono inviate **una dietro l'altra**, senza aspettare le risposte alle richieste precedenti. Da HTTP/2 richieste e risposte possono anche essere **intercalate** sulla stessa connessione, con un meccanismo di **priorità**. Le connessioni persistenti con pipelining sono oggi la modalità di default.

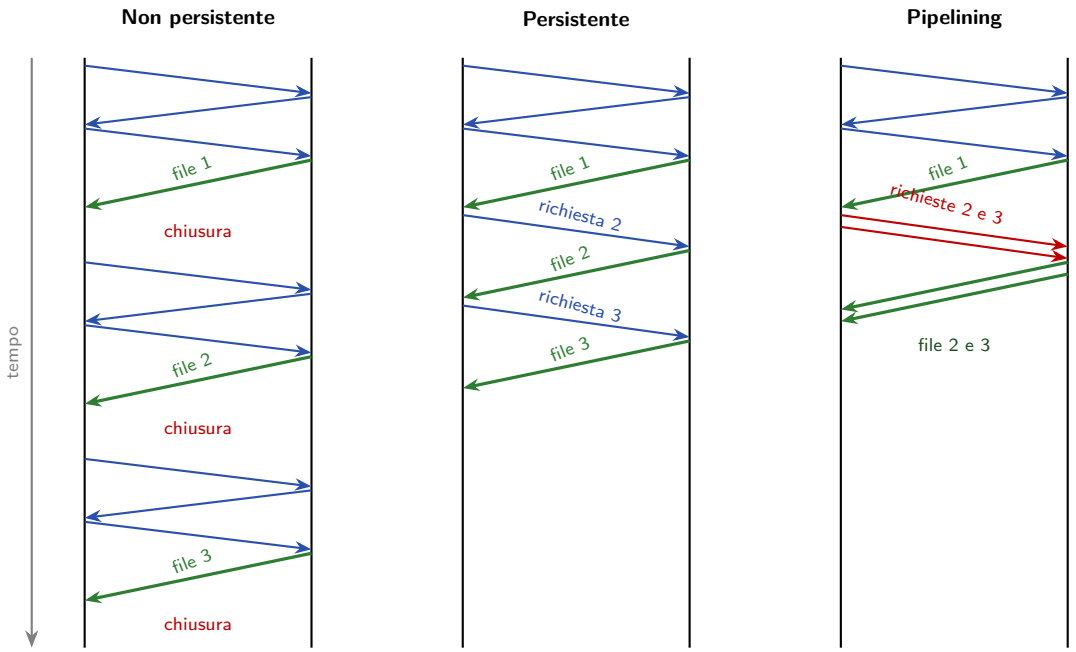


Figura 4.7 – Confronto indicativo degli RTT necessari per richiedere 3 file nelle tre modalità.

Confronto

Persistente	Non persistente
✔ più veloce, soprattutto con il pipelining	✔ semplice da implementare, in perfetta sintonia con la natura stateless di HTTP
✔ richiede meno risorse (soprattutto memoria)	✖ richiede più risorse: per ogni connessione vanno allocati buffer e variabili TCP su client e server
✖ più complessa da implementare, specie con il pipelining	✖ più lenta: 1–2 RTT in più per ogni richiesta
✖ la connessione può restare aperta anche se inutilizzata: il server la chiude dopo un *timeout*	✔ le connessioni non restano mai aperte inutilmente

Tabella 4.1 – Connessioni persistenti e non persistenti a confronto.

Lo scambio è quello di sempre: la connessione già aperta è veloce, ma è una risorsa tenuta occupata per qualcuno che potrebbe non tornare più.

La connessione che si chiude mentre la usi

Che cosa succede se il server chiude per timeout una connessione persistente **proprio mentre** il client la sta riusando?

- Il server chiude da solo una connessione inattiva, senza chiedere nulla al client.
- Una richiesta scritta in quello stesso istante si perde con la connessione: nessuna risposta, e nessun errore da parte del protocollo.
- Un client che riusa una connessione inattiva deve quindi essere pronto ad **aprirne una nuova** e a **reinviare** la richiesta.

Ripetere deve essere innocuo

Reinviare una richiesta è sicuro solo se ripeterla non fa danni. È uno dei motivi per cui ci si aspetta che una `GET` **non modifichi** nulla sul server: chiedere due volte la stessa pagina va bene, pagare due volte lo stesso ordine no.

HTTP/2 e HTTP/3

Il blocco in testa alla fila

Perché un oggetto grande può rallentare dieci oggetti piccoli? Prendiamo una pagina con un video pesante in alto e molti piccoli oggetti sotto, una sola connessione persistente e un collegamento lento lungo il percorso.

Head-of-line blocking

Il video occupa la connessione, e i piccoli oggetti, già pronti, aspettano dietro di lui: è il **blocco in testa alla fila** (*head-of-line blocking*, HOL).

I browser aggirano il problema aprendo **fino a sei connessioni parallele**. Così, però, si prendono anche una fetta più grande del collegamento più lento, perché TCP divide la banda in parti uguali fra le **connessioni**, non fra le pagine. Eliminare quelle connessioni parallele senza pagare il blocco è lo scopo di HTTP/2.

HTTP/2: frame intercalati su una sola connessione

- Un sotto-livello di **framing** spezza ogni messaggio in piccoli **frame**: l'intestazione in uno, il corpo in altri, tutti in codifica **binaria**.
- I frame di risposte diverse vengono **intercalati** sulla stessa connessione e riassemblati dall'altra parte.
- Metodi, codici di stato, URL e campi di intestazione **non cambiano**: cambia solo il modo in cui i dati sono formattati e trasportati.

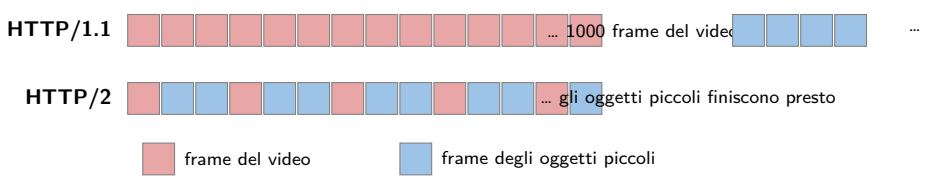


Figura 4.8 – Con HTTP/1.1 gli oggetti piccoli aspettano la fine del video; con HTTP/2 i frame si alternano.

Esempio 5 — 18 frame invece di 1016

Un video di 1000 frame e otto oggetti piccoli di 2 frame ciascuno. Con HTTP/1.1 gli oggetti piccoli arrivano dopo il video: l'ultimo è completo dopo 1000 + 16 = 1016 frame. Con HTTP/2, alternando un frame del video e quelli degli oggetti piccoli, tutti gli otto oggetti sono consegnati dopo appena **18** frame (i loro 16, più i 2 del video intercalati).

Priorità e server push

- Il client può dare a ogni richiesta un **peso** fra 1 e 256 e dichiarare da quale altro messaggio dipende.
- Il server invia per primi i frame delle risposte a priorità più alta: il foglio di stile prima dell'immagine in fondo alla pagina.
- Con il **server push** il server legge l'HTML di base, capisce quali oggetti serviranno e li invia **prima che vengano richiesti**: si risparmia un RTT per ogni oggetto "spinto", cioè la richiesta che il client avrebbe comunque mandato.

HTTP/3 e QUIC

QUIC

QUIC è un protocollo di trasporto realizzato **nel livello applicativo**, sopra il semplicissimo UDP. Fornisce ciò che serve a HTTP e che UDP non dà: affidabilità, multiplazione di più flussi, controllo di flusso per ciascun flusso. **HTTP/3** è HTTP sopra QUIC.

- Aprire una connessione richiede **meno RTT**, perché connessione e cifratura vengono negoziate **insieme** invece che una dopo l'altra.
- Un pacchetto perso blocca solo il **proprio** flusso: sparisce il blocco in testa alla fila che sopravviveva dentro TCP (in HTTP/2, una perdita ferma tutti i frame dietro di sé).
- Diversi meccanismi di HTTP/2 sono già dentro QUIC, quindi il protocollo sopra diventa più semplice.

Versione	Connessioni per pagina	Che cosa ha introdotto
HTTP/1.0 (1996)	una per oggetto	connessioni non persistenti
HTTP/1.1 (1997)	una, riusata	connessioni persistenti, pipelining
HTTP/2 (2015)	una, con frame intercalati	framing binario, priorità, server push
HTTP/3 (2022)	una, su QUIC sopra UDP	apertura in meno RTT, niente HOL fra flussi

Tabella 4.2 – Trent'anni di lavoro su un'unica domanda: quante connessioni, e per quanto tempo.

Laboratorio: il primo GET con Wireshark

Installare Wireshark

Su Debian, Ubuntu e derivate (rispondere *sì* quando si chiede se gli utenti non root possono catturare pacchetti):

```
$ sudo apt update && sudo apt install wireshark
$ sudo dpkg-reconfigure wireshark-common   # se si e' risposto no
$ sudo usermod -aG wireshark $USER         # poi uscire e rientrare
$ groups                                   # deve comparire wireshark
$ getcap /usr/bin/dumpcap
cap_net_raw,cap_net_admin=eip              # cio' che deve stampare
```

Se `getcap` non stampa nulla, bisogna assegnare a mano quelle due *capability* al programma, con `setcap`.

Catturare un GET

1. Avviare Wireshark, senza far partire la cattura.
2. Scrivere `http` nel filtro di visualizzazione, così da vedere solo i pacchetti HTTP.
3. Aspettare poco più di un minuto (vedremo perché), poi avviare la cattura.
4. Scaricare una pagina molto semplice: `$ lynx http://www.columbia.edu/~fdc/sample.html`
5. Fermare la cattura. Chiudere i dettagli Frame, Ethernet, IP e TCP, e aprire quello HTTP.

Si vedono **due** pacchetti HTTP: la `GET` inviata dal client e il `200 OK` di risposta, ciascuno dentro un segmento TCP, dentro un datagramma IP, dentro un frame Ethernet.

Esercizio 1 — Domande sulla cattura

Sulla **richiesta**: (a) quale versione di HTTP annuncia il client, e con quale risponde il server? (b) Quali lingue il client dichiara di accettare? (c) Qual è l'indirizzo IP della propria macchina, e quello del server?

Sulla **risposta**: (d) quale codice di stato ha restituito il server? (e) Quando è stata modificata l'ultima volta la pagina? (f) Quanti byte di contenuto sono stati inviati? (g) Nei byte grezzi del pacchetto ci sono righe di intestazione che l'elenco dei pacchetti non mostra?

Svolgimento

Le risposte vanno lette dalla **propria** cattura, non da qui. Dove guardare: la versione è nella prima riga di ciascun messaggio (riga di richiesta e riga di stato); le lingue nella riga `Accept-Language`; gli indirizzi nel dettaglio IP (*Source* e *Destination*); il codice nella riga di stato; la data di modifica in `Last-Modified`; i byte in `Content-Length`.

Per (e): la pagina può sembrare modificata meno di un minuto prima. Quel server aggiorna la data ogni minuto, così che tutti scarichino una copia nuova: è il motivo dell'attesa al passo 3. Si ripeta poi con `http://scottaaronson.com` e con un browser moderno al posto di `lynx`: che cosa cambia, e perché? (Un browser moderno usa quasi sempre HTTPS: si veda il laboratorio del capitolo successivo.)

Riepilogo

In breve

- Il Web è una collezione di risorse identificate da **URL** e accessibili con **HTTP**, un protocollo client-server.
- HTTP usa **TCP** (porta 80) e, con HTTP/3, anche **UDP** con QUIC.
- HTTP è **stateless**: il server non ricorda le richieste precedenti.
- Connessioni **non persistenti**: una connessione per oggetto, circa 2 RTT + trasmissione per ciascuno. Connessioni **persistenti**: l'handshake si paga una volta, poi 1 RTT per oggetto; con il **pipelining** le richieste partono senza aspettare le risposte.
- **HTTP/2** intercala i frame di più risposte su una connessione (niente blocco in testa alla fila fra oggetti), con priorità e server push; **HTTP/3** porta tutto su **QUIC**, sopra UDP.

HTTP: MESSAGGI, COOKIE E CACHING

Nel capitolo precedente abbiamo visto *come* HTTP usa le connessioni. Ricapitoliamo e passiamo a *che cosa* si scambiano client e server.

Ripasso: HTTP in cinque punti

- È uno dei protocolli più usati di Internet (applicazioni Web).
- È **client-server** e definisce **il formato dei messaggi** e **come vengono scambiati** (non come si implementano client e server).
- Usa messaggi di **richiesta** e di **risposta**.
- È **stateless**: client e server non ricordano le interazioni passate.
- Si appoggia a un trasporto affidabile e orientato alla connessione (soprattutto TCP, di recente anche UDP con QUIC), con connessioni persistenti o non persistenti.

IL FORMATO DEI MESSAGGI HTTP

HTTP ha due formati diversi per i due tipi di messaggio:

- la **richiesta** specifica il **comando** (metodo) che il server deve eseguire: dammi una pagina, compila questo modulo, …;
- la **risposta** riporta l'**esito** del comando (successo, errore, …) ed eventuali **dati** (per esempio il file richiesto).

Entrambi sono scritti in **testo ASCII** ordinario: si leggono facilmente anche a occhio.

Il formato della richiesta

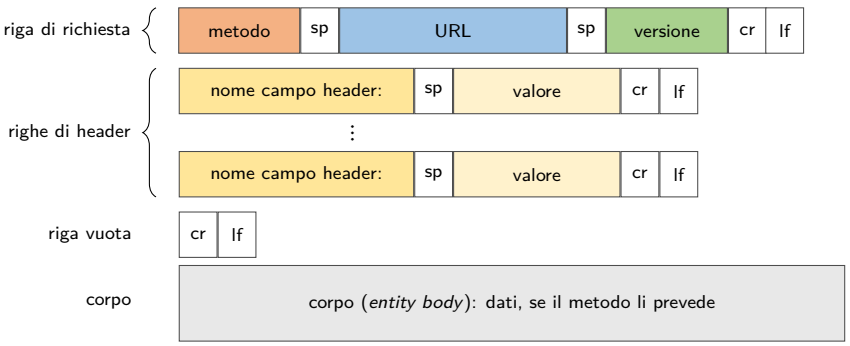


Figura 5.1 – Formato generale di un messaggio di richiesta HTTP.

Il messaggio di richiesta contiene:

- il **metodo**: il comando da far eseguire al server;
- l'**URL**: l'oggetto su cui si vuole operare;
- la **versione** di HTTP (per esempio `HTTP/1.1`);
- le **righe di header**: i parametri della richiesta, in numero e tipo non fissati; ogni riga ha il nome e il valore del parametro. Qui si può per esempio chiedere una connessione persistente o no; si possono usare anche header personalizzati;
- il **corpo** (*body*): dipende dal metodo e contiene gli eventuali dati associati al comando (per esempio il testo con cui compilare un modulo).

I separatori

I campi sono separati da caratteri speciali: `sp` è lo **spazio**; `cr` è il *carriage return* (`\r`); `lf` è il *line feed* (`\n`). Ogni riga termina con `\r\n`, e una **riga vuota** (`\r\n` da solo) separa gli header dal corpo. Dimenticarla, quando si scrive una richiesta a mano, è l'errore più comune.

I metodi HTTP

Metodo	Uso
`GET`	**recupera** un oggetto dal server. È il metodo più usato: ogni volta che si apre una pagina si fa una GET. Il corpo è **vuoto**.
`POST`	**invia informazioni** al server, per esempio il contenuto di un modulo. Il corpo contiene i dati.
`HEAD`	come GET, ma il server risponde **senza l'oggetto**, con i soli header. Utile per il debugging e per controllare se un oggetto esiste o è cambiato.
`PUT`	**carica** un oggetto in un certo percorso del server (strumenti di pubblicazione Web, applicazioni che devono caricare file).
`DELETE`	**cancella** un oggetto dal server.

Tabella 5.1 – I metodi HTTP principali.

I moduli non usano per forza POST

Una richiesta generata da un modulo HTML non usa necessariamente POST: spesso i moduli usano GET e inseriscono i dati dei campi **direttamente nell'URL**, come query (per esempio `...search?q=reti&lang=it`).

Esempio di richiesta

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

Richiesta HTTP

Un browser (Mozilla/5.0) che implementa HTTP/1.1 chiede l'oggetto `/somedir/page.html` al server `www.someschool.edu`. Ci sono cinque righe: la riga di richiesta (GET, risorsa, versione) e quattro righe di header; il corpo è vuoto.

- `Host: www.someschool.edu` indica l'host su cui si trova l'oggetto. Sembra ridondante (ci siamo già connessi a quell'host!), ma il destinatario della connessione non è sempre il server finale: il messaggio può essere inoltrato da un **proxy**, e allora l'indirizzo della connessione TCP è diverso da quello del server vero.
- `Connection: close` chiede una connessione **non persistente**: chiudi quando hai finito.
- `User-agent: Mozilla/5.0` indica il tipo di browser: il server può inviare versioni diverse dello stesso oggetto (stesso URL) a seconda del client.
- `Accept-language: fr` indica che l'utente preferisce, se esiste, la versione in francese.

Il formato della risposta

La risposta ha un formato analogo, ma al posto della riga di richiesta c'è una **riga di stato** che riporta l'esito del comando:

Figura 5.2 – Formato generale di un messaggio di risposta HTTP.

- la **versione** di HTTP usata dal server;
- il **codice di stato**: un numero che indica l'esito;
- la **frase**: una descrizione testuale dell'esito.

I codici di stato

Classe	Significato	Esempi
`1xx`	informativo	informazioni sulla richiesta in corso
`2xx`	successo	`200 OK`: richiesta riuscita, l'informazione è nella risposta
`3xx`	redirezione	`301 Moved Permanently`: l'oggetto è stato spostato in modo permanente; il nuovo URL è nell'header `Location`
`4xx`	errore del client	`400 Bad Request`: il server non ha capito la richiesta; `404 Not Found`: il documento non esiste su questo server
`5xx`	errore del server	`505 HTTP Version Not Supported`: il server non supporta la versione di HTTP richiesta

Tabella 5.2 – Le classi dei codici di stato HTTP.

Come ricordarle

La prima cifra dice di chi è la "colpa": **2** tutto bene, **3** vai altrove, **4** hai sbagliato tu (client), **5** ho sbagliato io (server).

Esempio di risposta

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html

(data data data data data ...)
```

Risposta HTTP

La riga di stato dice che il server usa HTTP/1.1 e che è tutto a posto: ha trovato l'oggetto e lo sta inviando. Il corpo contiene l'oggetto (`data data …`). Gli header:

- `Connection: close`: la connessione verrà chiusa dopo questo messaggio (non persistente);
- `Date`: data e ora in cui la risposta è stata creata e inviata;
- `Server`: il messaggio è stato generato da un server Apache (è l'analogo di `User-agent`);
- `Last-Modified`: data e ora in cui l'oggetto è stato creato o modificato l'ultima volta; fondamentale per il caching, perché le copie in cache possono essere vecchie;
- `Content-Length`: il numero di byte dell'oggetto;
- `Content-Type`: il tipo dell'oggetto, qui testo HTML. Il tipo è indicato ufficialmente da questo header, **non** dall'estensione del file.

Parlare HTTP a mano

In una richiesta non c'è nulla di magico: la si può scrivere a mano. Con `nc` (*netcat*) si apre una connessione TCP verso la porta 80 e si digitano le righe:

```
$ nc -C info.cern.ch 80          # apre una connessione TCP verso la porta 80
GET /hypertext/WWW/TheProject.html HTTP/1.1
Host: info.cern.ch
                                 # la riga vuota chiude la richiesta
HTTP/1.1 200 OK
...
```

L'opzione `-C` non è facoltativa

HTTP termina ogni riga con `CR LF`, mentre `nc` da solo invia soltanto `LF`: senza `-C` questo server risponde `400 Bad Request`.

Il server risponde a un programma che non è un browser perché non ha modo di accorgersene: vede soltanto quei byte. Sostituendo `GET` con `HEAD` si ottengono le sole intestazioni: gli stessi metadati, senza il traffico dell'oggetto.

I COOKIE

Un server HTTP senza stato è semplice, consuma poche risorse e regge migliaia di connessioni TCP simultanee. Ma la mancanza di stato è anche un grosso limite: molte funzioni del Web dipendono dal client (il carrello di Amazon, i suggerimenti di Netflix, …).

Cookie

Un **cookie** è un gettone digitale (un identificativo alfanumerico) creato dal server e consegnato al client, con cui il server **riconosce un client specifico** nelle richieste successive. I cookie possono avere attributi, come una data di scadenza.

Le quattro componenti

1. Una riga di header `Set-cookie` nel messaggio di **risposta** HTTP.
2. Una riga di header `Cookie` nel messaggio di **richiesta** HTTP.
3. Un **file** sul sistema del client, gestito dal browser.
4. Un **database** lato server (*back-end*).

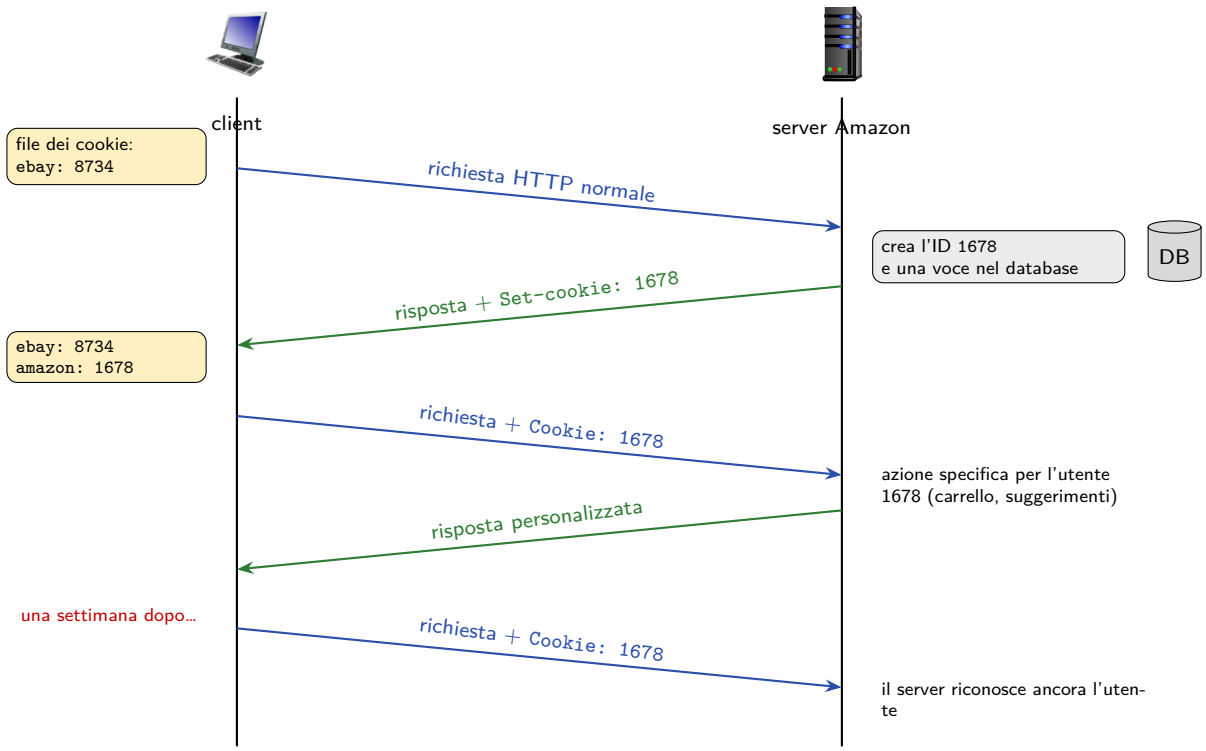


Figura 5.3 – Un client che usa già eBay contatta Amazon per la prima volta: nasce un cookie, che accompagnerà tutte le richieste successive.

Esempio 1 — Il cookie di Amazon passo per passo

1. Un host, che usa abitualmente eBay (e ha già un cookie per eBay), contatta Amazon per la **prima volta**: non ha alcun cookie per Amazon.
2. Il server di Amazon crea un **identificativo** (`1678`) e una voce corrispondente nel proprio database, e risponde includendo l'header `Set-cookie: 1678`.
3. Il browser aggiunge al file dei cookie una riga con il nome del server e l'identificativo.
4. Da quel momento ogni richiesta verso Amazon contiene `Cookie: 1678`: il server può tracciare l'attività dell'utente nel database, sapendo esattamente quali pagine ha visitato, in che ordine e quando.
5. Se l'utente torna una settimana dopo, il browser continua a inserire `Cookie: 1678`.

Se il file dei cookie viene cancellato

Che succede al carrello se si cancellano i dati del browser a metà di un acquisto? La richiesta successiva arriva **senza** la riga `Cookie`. Il server non ha modo di distinguerla da una prima visita, quindi assegna un **nuovo** identificativo: il carrello risulta vuoto, e quello vecchio resta nel database, ormai irraggiungibile.

- Il server non stava conservando la "sessione" dell'utente: conservava una **riga** di database, e l'utente portava con sé la **chiave** per raggiungerla.
- È anche il motivo per cui lo stesso account appare diverso sul telefono: un altro file, un'altra chiave.

HTTP resta stateless

HTTP non è diventato un protocollo con stato: lo stato è stato costruito **sopra** di esso, con due righe di intestazione e due archivi (il file dei cookie e il database), entrambi **agli estremi**, mai nel protocollo.

Usi e problemi dei cookie

I siti usano i cookie per:

- il **carrello**: la lista degli acquisti in corso durante la navigazione;
- la **registrazione**: associando al cookie nome, e-mail, indirizzo, carta di credito, così da non doverli reinserire ogni volta;
- i **suggerimenti**: prodotti consigliati in base alle pagine visitate.

Per fare tutto questo il server non ha mai avuto bisogno di sapere il **nome** dell'utente. Ma basta fornirlo una volta, al momento della registrazione, perché l'intera cronologia associata al numero acquisti un'identità.

Cookie e privacy

I cookie semplificano la vita, ma sono **controversi**: possono essere considerati un'invasione della privacy. Combinando cookie e dati forniti dall'utente, un sito può sapere moltissimo di una persona e, potenzialmente, vendere queste informazioni a terzi.

Il caching Web

Web cache e proxy

Una **Web cache** è un'entità di rete che soddisfa richieste HTTP **al posto** del server di origine. Ha un proprio disco, su cui conserva copie degli oggetti richiesti di recente. Le entità intermedie che lavorano per conto di altri si chiamano **proxy server**.

Un browser può essere configurato in modo che tutte le sue richieste passino prima dalla cache, per vedere se ha già una copia dell'oggetto.

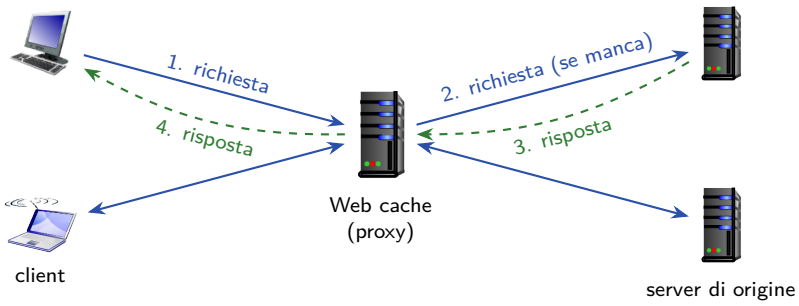


Figura 5.4 – La cache sta tra i client e i server di origine: è server verso i client e client verso i server.

Il funzionamento

Supponiamo che un browser chieda `http://www.someschool.edu/campus.gif` passando per una cache:

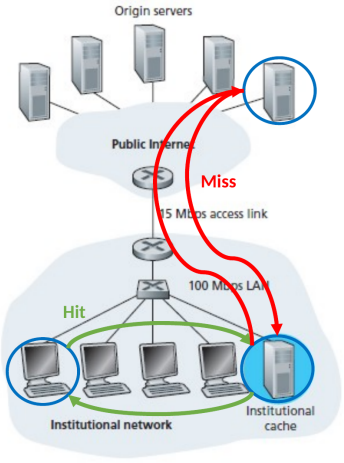
1. Il browser apre una connessione TCP con la **cache** e le invia la richiesta HTTP.
2. La cache controlla se ha una copia locale dell'oggetto. Se sì, la restituisce subito al browser in una risposta HTTP.
3. Se no, la cache apre una connessione TCP con il **server di origine** (`www.someschool.edu`) e gli chiede l'oggetto.
4. Ricevuto l'oggetto, la cache ne **salva una copia** e ne invia un'altra al browser.

La cache è client e server insieme

Una cache è contemporaneamente un **server** (quando fornisce oggetti ai browser) e un **client** (quando li chiede ai server di origine).

Perché usare le cache

1. Una cache può **ridurre molto il tempo di risposta**, soprattutto se tra il client e la cache c'è un collegamento veloce e tra la cache e Internet uno lento.
2. Una cache può **ridurre molto il traffico** di un'azienda o di un ente verso Internet, e quindi il costo della banda.



Le cache vengono spesso installate nella rete locale di un'azienda o di un ente.

Hit, miss e hit rate

Si ha un **hit** quando la cache fornisce un oggetto **senza contattare** il server di origine; un **miss** quando deve andarlo a prendere. La **hit rate** è la frazione di richieste soddisfatte dalla cache: tipicamente **tra 0,2 e 0,7**, e cresce con il numero di client che usano la cache.

Fino al 70% delle richieste, quindi, può essere servito localmente.

Figura 5.5 – Una cache istituzionale: il percorso rosso (miss) esce su Internet, quello verde (hit) resta nella rete locale.

Esempio 2 — Quanto vale una cache

Un'istituzione ha una rete locale a 100 Mbps e un collegamento di accesso a Internet a **15 Mbps**. L'oggetto medio è di **1 Mbit**, dall'istituzione partono **15 richieste al secondo**, e recuperare un oggetto dall'altra parte di Internet richiede in media **2 secondi**.

- Domanda sul collegamento di accesso: $15 \times 1 = 15$ Mbps, su un collegamento da esattamente 15 Mbps: l'**intensità di traffico** è 1.
- Un collegamento sfruttato al 100% della capacità non tiene il passo: la coda davanti a lui cresce senza limite e l'attesa diventa di **minuti** (il perché lo vedremo con il livello di trasporto e di rete).
- **Soluzione 1**: portare il collegamento a 100 Mbps. Funziona, e costa ogni mese.
- **Soluzione 2**: una cache con hit rate 0,4. Solo il 60% delle richieste esce: la domanda scende a $0{,}6 \times 15 = 9$ Mbps (intensità 0,6, ritardo di coda piccolo). Il tempo medio diventa circa $0{,}6 \times 2\ \text{s} + 0{,}4 \times (\text{pochi ms}) \approx 1{,}2$ s.

La cache costa meno dell'aggiornamento ed è anche più veloce. Per questo università, aziende e provider la installano per il **collegamento**, non per gli utenti.

Il GET condizionale

Il caching introduce un problema nuovo: la copia in cache potrebbe essere **vecchia**, se nel frattempo l'oggetto è cambiato sul server.

GET condizionale

Un **GET condizionale** è una richiesta HTTP con metodo GET e header `If-Modified-Since`. Il server invia l'oggetto **solo se** è stato modificato dopo la data indicata; altrimenti risponde `304 Not Modified`, senza corpo.

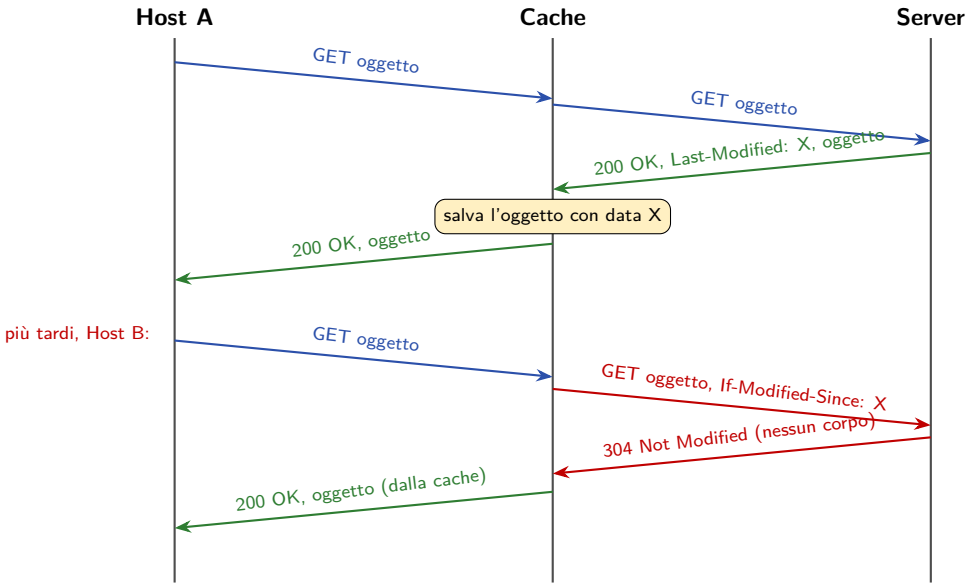


Figura 5.6 – La cache verifica con un GET condizionale che la propria copia sia ancora valida: se lo è, l'oggetto non viaggia di nuovo.

Se l'oggetto è aggiornato, alla cache basta una piccola risposta del server (niente oggetto da ritrasmettere) e il client riceve la copia locale.

Esempio 3 — Lo stesso scambio, nelle intestazioni

Che cosa la cache aveva memorizzato una settimana prima:

```
HTTP/1.1 200 OK
Last-Modified: Wed, 9 Sep 2015 09:23:24      # conservata con l'oggetto
```

Che cosa invia oggi, e che cosa torna indietro:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Wed, 9 Sep 2015 09:23:24  # lo stesso valore, restituito

HTTP/1.1 304 Not Modified
Date: Sat, 10 Oct 2015 15:39:29
                                             # corpo vuoto
```

L'immagine non è mai stata inviata: è tutto qui il risparmio, ed è per questo che `Last-Modified` compare in ogni risposta. Nessun protocollo nuovo: due righe di intestazione e un codice di stato.

Il caching nel browser

Anche i **browser** fanno caching in locale, con lo stesso principio: conservano gli oggetti già scaricati per non doverli richiedere al server. È una tecnica comunissima che migliora drasticamente le prestazioni. La copia locale però può non essere aggiornata, e questo genera errori piuttosto frequenti (la classica pagina che "non si aggiorna" finché non si svuota la cache).

Oltre il caching: i proxy istituzionali

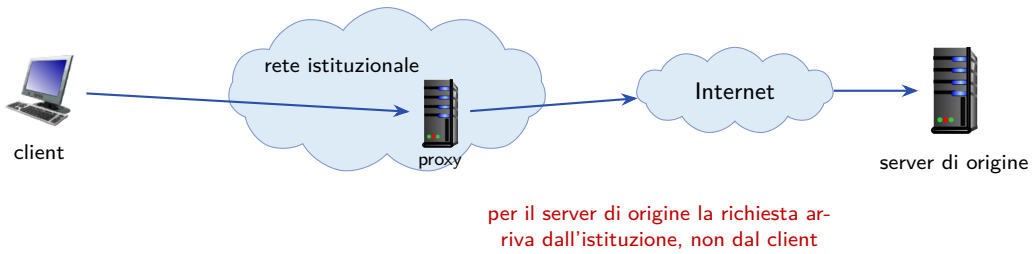


Figura 5.7 – Un proxy istituzionale: le richieste escono a nome dell'ente.

Oltre alle prestazioni, i proxy servono ad **accedere a servizi istituzionali**. La rete dell'Ateneo, per esempio, offre un proxy Web (`proxy.unina.it`): poiché le richieste vengono inoltrate dal proxy, per il server di origine **arrivano tutte dall'istituzione**. Così, da casa, si possono consultare riviste scientifiche abbonate dall'Università. Esistono anche molti proxy commerciali o gratuiti su Internet.

Il server di origine vede l'indirizzo del proxy, e ricava la vera destinazione dalla riga `Host`: ecco perché quella riga, che sembrava ridondante, serve.

Perché l'HTTP in chiaro non si vede più

Scrivendo oggi `http://` in un browser, è molto improbabile che dalla macchina esca davvero HTTP in chiaro.

- **HTTPS**: lo stesso HTTP, sopra **TLS**, sulla porta **443**. I messaggi sono identici, ma nessuno lungo il percorso può leggerli.
- **HSTS** (*HTTP Strict Transport Security*): una riga di intestazione con cui un sito dichiara che i browser non devono più usare HTTP in chiaro con lui, per mesi.
- I browser includono una lista precaricata (*preload list*) di questi siti e una modalità *HTTPS-first*: l'URL viene riscritto prima ancora di inviare un pacchetto.

Che cosa mostra il filtro `http` di Wireshark?

Niente. Il traffico c'è (TCP verso la porta 443), ma Wireshark vede **record TLS**, non richieste HTTP. Che cosa faccia TLS al loro interno lo vedremo nella parte sulla sicurezza.

Laboratorio: una risposta che non sta in un pacchetto

Si cattura una pagina abbastanza lunga da dover essere divisa in più segmenti TCP.

1. Svuotare la cache del browser, così che nulla venga servito localmente (in Firefox o Chrome `Ctrl+Shift+Canc`, oppure disattivarla negli strumenti per sviluppatori).
2. Avviare la cattura in Wireshark, ancora senza filtro.
3. Scaricare la pagina con un client che non trasforma la richiesta in HTTPS:

```
$ curl http://info.cern.ch/hypertext/WWW/TheProject.html > /dev/null
```

4. Fermare la cattura e scrivere `http` nel filtro per trovare i due messaggi HTTP.
5. Togliere di nuovo il filtro: la risposta cercata sta per lo più in pacchetti che **non** sono messaggi HTTP.

Esercizio 1 — Domande sulla cattura

(a) Quanti messaggi `GET` ha inviato il client, e quale numero di pacchetto contiene quello della pagina? (b) Quale pacchetto contiene il codice e la frase di stato della risposta? (c) Quali sono? (d) Quanti segmenti TCP con dati sono serviti per quell'unica risposta?

Svolgimento

Anche qui i numeri vanno letti dalla propria cattura. Per (d) si cercano, nella colonna *Info*, i pacchetti marcati *TCP segment of a reassembled PDU*: sono i pezzi della risposta, che Wireshark riassembla e mostra come un unico messaggio HTTP solo nell'ultimo segmento. Ripetendo con un browser al posto di `curl`, i messaggi HTTP spariscono: il browser è passato a HTTPS, e la pagina viaggia dentro record TLS.

Riepilogo

In breve

- Richieste e risposte HTTP sono testo ASCII: **riga di richiesta/stato**, **header**, **riga vuota**, **corpo**.
- Metodi principali: **GET**, **POST**, **HEAD**, **PUT**, **DELETE**.
- Codici di stato: 1xx informativi, 2xx successo, 3xx redirezione, 4xx errore del client, 5xx errore del server.
- I **cookie** (Set-cookie / Cookie + file sul client + database sul server) danno uno stato a un protocollo stateless, con implicazioni sulla privacy.
- Le **Web cache** (proxy) riducono tempi e traffico; il **GET condizionale** (`If-Modified-Since` / `304 Not Modified`) evita copie vecchie.

Posta elettronica e applicazioni peer-to-peer

La posta elettronica

La posta elettronica (*e-mail*) è una delle applicazioni più antiche e importanti di Internet. È realizzata in modo **client-server**, con due tipi di host:

- lo **user agent**: l'applicazione con cui l'utente gestisce la posta (Thunderbird, K-9 Mail, l'app Mail del telefono, …);
- il **mail server**: un server che conserva la posta e mantiene una **casella** (*mailbox*) per ciascun utente.

A differenza di HTTP, qui la comunicazione avviene **anche tra server**. Servono quindi due applicazioni (e due protocolli): una tra user agent e mail server, e una tra mail server e mail server.

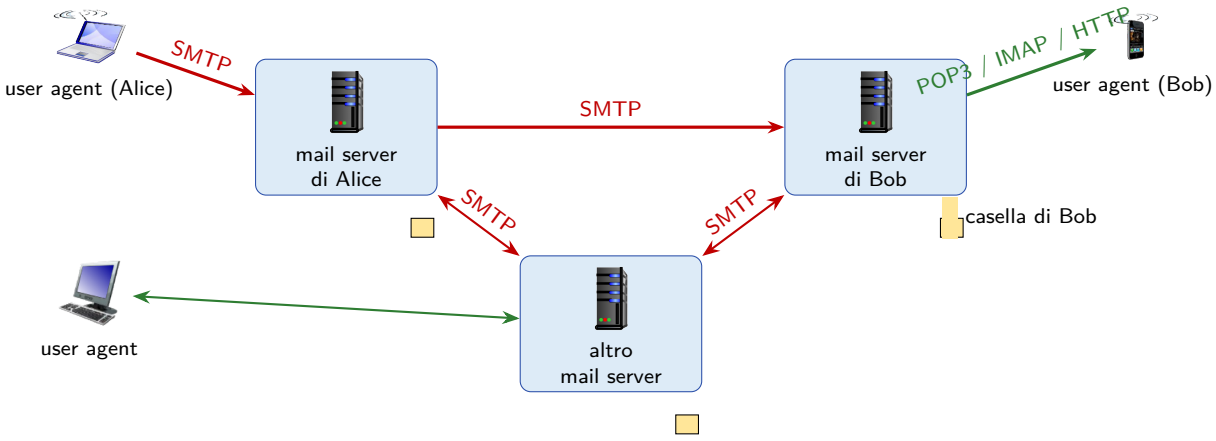


Figura 6.1 – Il sistema di posta: SMTP per inviare (dagli user agent ai server e tra server), protocolli di accesso per leggere la posta dalla propria casella.

Gli utenti non si scambiano la posta direttamente (come nella messaggistica istantanea): si preferisce passare per i mail server, più affidabili e specializzati.

SMTP

Il **Simple Mail Transfer Protocol** (SMTP) è il principale protocollo applicativo usato per **trasferire la posta tra mail server**. Ha un lato client (chi invia) e un lato server (chi riceve), entrambi eseguiti sui mail server, e usa il trasporto affidabile di **TCP** sulla **porta 25**.

Quando un nuovo messaggio arriva sul server, viene messo nella casella del destinatario, in attesa che lo user agent lo scarichi.

Il flusso completo di una mail

Alice invia una mail a Bob (`bob@someschool.edu`). Sono coinvolti quattro elementi: lo user agent di Alice, il mail server di Alice, il mail server di Bob, lo user agent di Bob.

Figura 6.2 – I sei passi dell'invio di una mail da Alice a Bob.

1. Alice apre il suo user agent, inserisce l'indirizzo di Bob, scrive il messaggio e chiede di inviarlo.
2. Lo user agent di Alice invia il messaggio al **suo** mail server, dove viene messo in una **coda di messaggi**.
3. Il lato client di SMTP, sul server di Alice, vede il messaggio in coda e **apre una connessione TCP** verso il server SMTP in esecuzione sul mail server di Bob.
4. Dopo un handshaking SMTP iniziale, il client SMTP invia il messaggio di Alice sulla connessione TCP.
5. Sul server di Bob il lato server di SMTP riceve il messaggio e lo mette nella **casella di Bob**.
6. Bob apre il proprio user agent e legge il messaggio quando vuole.

Nessun server intermedio

SMTP di norma **non** usa mail server intermedi: la connessione TCP va direttamente dal server di Alice a quello di Bob, anche se sono ai lati opposti del pianeta. Se il server di Bob è spento, il messaggio resta nella coda del server di Alice, che riprova più tardi.

Dettagli del protocollo

- Il client SMTP (sul server mittente) apre una connessione TCP verso la **porta 25** del server SMTP (sul server destinatario).
- Se il server non risponde, il client **riprova più tardi**.
- Stabilita la connessione, client e server fanno un **handshaking applicativo**: si presentano prima di trasferire informazioni. In questa fase il client indica l'indirizzo del **mittente** e quello del **destinatario**.
- Poi il client invia il messaggio, contando sul trasferimento affidabile di TCP per farlo arrivare senza errori.

Esempio 1 — Un dialogo SMTP

Le righe che iniziano con `S:` sono inviate dal server, quelle con `C:` dal client (da Kurose). Come HTTP, anche SMTP usa comandi testuali leggibili.

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

Cinque comandi (`HELO`, `MAIL FROM`, `RCPT TO`, `DATA`, `QUIT`), e a ciascuno una risposta numerica: la stessa idea dei codici di stato HTTP. Un punto da solo su una riga chiude il messaggio. Oggi al posto di `HELO` si usa `EHLO`, il saluto "esteso" che apre le estensioni del protocollo. La connessione è **persistente**: sulla stessa connessione possono seguire altri messaggi.

Il messaggio e il suo passato a 7 bit

Dentro `DATA` viaggia un messaggio che ha una **propria** intestazione, definita da uno standard diverso:

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Searching for the meaning of life.

(il corpo, dopo la riga vuota)
```

- Queste righe **non** sono i comandi SMTP, anche dove le parole coincidono: i comandi guidano la consegna, queste righe viaggiano **dentro** di essa (e sono quelle che il client di posta mostra).
- SMTP è più vecchio di HTTP e limita l'intero messaggio, corpo compreso, all'**ASCII a 7 bit**.
- Un'immagine viene quindi **codificata in caratteri** prima di partire e decodificata all'arrivo: per questo un allegato occupa più spazio del file originale.

Accedere alla posta

Usare dei server SMTP è più affidabile della consegna diretta da utente a utente:

- i server sono, per definizione, **sempre accesi** e visibili a tutti i dispositivi della rete (il portatile di Bob, invece, può essere spento);
- i server sono **certificati**;
- il server può **riprovare continuamente** a consegnare la posta se la consegna fallisce, un processo piuttosto oneroso da fare sul proprio dispositivo.

Esempio 2 — Perché il messaggio passa dal server di Alice

Perché non inviarlo direttamente dal computer di Alice al server di Bob? Perché il server di Bob potrebbe essere spento, e il portatile di Alice non resterà acceso a riprovare per tre giorni. Il server di Alice invece sì: tiene il messaggio nella coda in uscita e riprova più o meno **ogni mezz'ora**; dopo diversi giorni senza successo rinuncia e avvisa Alice, naturalmente con una mail.

D'altra parte, per **leggere** la posta dal proprio server serve un'altra applicazione client-server, con un protocollo diverso da SMTP (SMTP è un protocollo "push": serve a spingere la posta verso un server, non a tirarla giù). I più diffusi sono POP3, IMAP e HTTP.

POP3

POP3

Il **Post Office Protocol — Version 3** (POP3) è un protocollo di accesso alla posta estremamente semplice. Lo user agent (client) apre una connessione TCP verso il mail server sulla **porta 110**.

Stabilita la connessione, POP3 attraversa tre fasi:

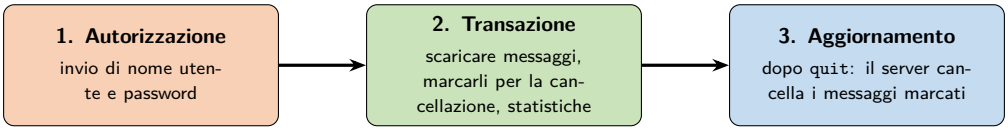


Figura 6.3 – Le tre fasi di una sessione POP3.

1. **Autorizzazione**: lo user agent invia nome utente e password per autenticare l'utente.
2. **Transazione**: lo user agent può recuperare messaggi, marcarli per la cancellazione, togliere le marcature e ottenere statistiche sulla casella.
3. **Aggiornamento**: quando il client dà il comando `quit`, la sessione termina e il server cancella i messaggi marcati.

IMAP

Con POP3 i messaggi si possono solo scaricare o cancellare: cercare, organizzare in cartelle e simili va fatto sulla macchina locale. Se si legge la posta da più dispositivi, questo è un problema.

IMAP

L'**Internet Mail Access Protocol** (IMAP, porta 143) permette al server di offrire funzioni aggiuntive, mantenendo la posta **sul server**:

- creare e gestire **cartelle** sul server;
- **cercare** nelle cartelle remote i messaggi che soddisfano certi criteri;
- scaricare solo **parti** di un messaggio (solo le intestazioni, solo gli allegati, …);
- collegare **più client** contemporaneamente alla stessa casella.

HTTP (webmail)

Sempre più utenti inviano e leggono la posta dal **browser**. L'accesso via Web fu introdotto da Hotmail a metà degli anni '90 ed è oggi l'approccio più diffuso (Google, Yahoo!, Libero, e quasi tutte le università e le aziende, UniNA compresa). In questo caso lo user agent è un normale browser, e l'utente comunica con la propria casella remota via **HTTP** invece che con POP3 o IMAP.

Tra server resta SMTP

HTTP sostituisce POP3/IMAP solo tra **user agent e server**. Tra **mail server** la posta continua a viaggiare con SMTP.

Protocollo	Tra	Porta TCP	Caratteristica
SMTP	server ↔ server, user agent → server	25	invio (push)
POP3	user agent ← server	110	scarica e (di solito) cancella
IMAP	user agent ↔ server	143	la posta resta sul server, cartelle, ricerca
HTTP	browser ↔ server	80/443	webmail

Tabella 6.1 – I protocolli della posta elettronica.

PEER-TO-PEER

A differenza del client-server, un'architettura P2P usa i server in modo minimo, o per nulla: coppie di host collegati in modo intermittente (i **peer**) comunicano direttamente. I peer non appartengono a un fornitore di servizi: sono computer e portatili controllati dagli utenti.

Distribuire un file: due modi

Un file, un server, e molti host che ne vogliono una copia.

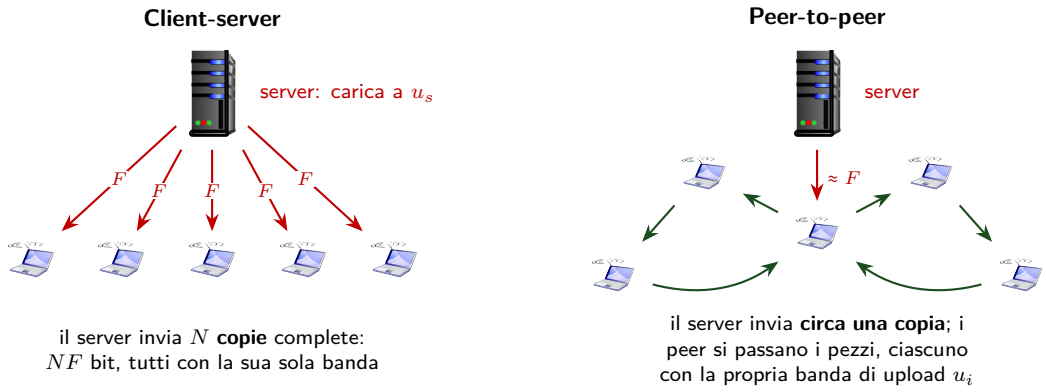


Figura 6.4 – A sinistra il server invia N copie; a destra ne invia circa una, e i peer si scambiano i pezzi tra loro.

Il ragionamento si può rendere quantitativo. Un file di F bit va distribuito a N peer; il server carica a velocità u_s, il peer i carica a u_i, e $d_{\min}$ è la velocità di download del peer più lento.

Tempo minimo di distribuzione

$$D_{\text{cs}} = \max\left\{\frac{NF}{u_s},\ \frac{F}{d_{\min}}\right\} \qquad D_{\text{P2P}} = \max\left\{\frac{F}{u_s},\ \frac{F}{d_{\min}},\ \frac{NF}{u_s + \sum_{i=1}^{N} u_i}\right\}$$

- **Client-server**: il server da solo deve inviare NF bit alla velocità u_s, e nessun client può finire prima di aver scaricato F bit alla propria velocità (il più lento a $d_{\min}$). Il tempo **cresce linearmente** con N, senza limite.
- **P2P**: il server deve inviare almeno una copia (F/u_s); il peer più lento non può scaricare più in fretta di $F/d_{\min}$; e i NF bit complessivi vengono caricati usando la banda del server **più quella di tutti i peer**. Ogni nuovo peer aggiunge un F al numeratore, ma anche il suo u_i al denominatore.

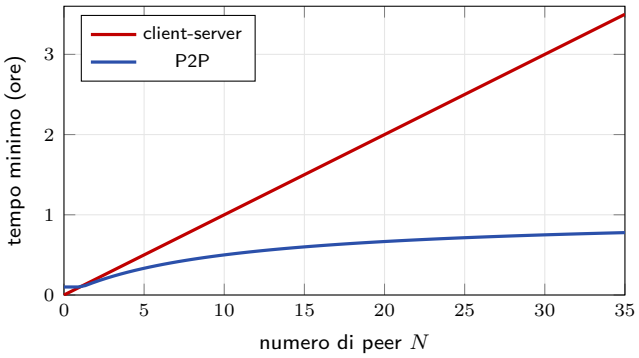


Figura 6.5 – Tempo minimo di distribuzione con $u_s = 10u$ (tutti i peer caricano a u), $F/u = 1$ ora e $d_{\min}$ abbondante (da Kurose): il client-server cresce come $N/10$, il P2P come $N/(10+N)$ e non supera mai 1 ora.

Mille volte i peer sono mille volte l'attesa nel client-server, e quasi nessuna attesa in più nel P2P: è la **auto-scalabilità** del P2P, e un esercizio numerico completo si trova nella parte degli esercizi.

Pro e contro del P2P

Nella condivisione di file il P2P **scala** di solito molto meglio del client-server. D'altra parte è più **complesso da implementare**, e avere tanti client in comunicazione diretta tra loro pone problemi di **sicurezza**.

BitTorrent

Un protocollo P2P molto diffuso per la distribuzione di file è **BitTorrent** (circa 150–170 milioni di utenti stimati nel 2023).

Torrent e chunk

Un **torrent** è l'insieme di tutti i peer che partecipano alla distribuzione di un certo file. I peer di un torrent si scambiano **chunk**, pezzi del file di uguale dimensione (tipicamente 256 kB).

Il ciclo di vita di un peer

Quando un peer entra in un torrent, senza avere ancora alcun chunk:

- comincia ad **accumulare chunk**;
- mentre scarica, **carica** anche chunk verso altri peer;
- quando ha l'intero file può andarsene (egoisticamente) oppure restare (altruisticamente) a continuare a caricare chunk per gli altri.

I peer possono lasciare il torrent in qualsiasi momento, anche con solo una parte dei chunk, e rientrare più tardi.

Il tracker

Tracker

Ogni torrent ha un nodo di infrastruttura, il **tracker**, che tiene traccia dei peer che partecipano al torrent. È, in sostanza, un **server**, in un'architettura che avrebbe dovuto farne a meno: un altro esempio di architettura ibrida.

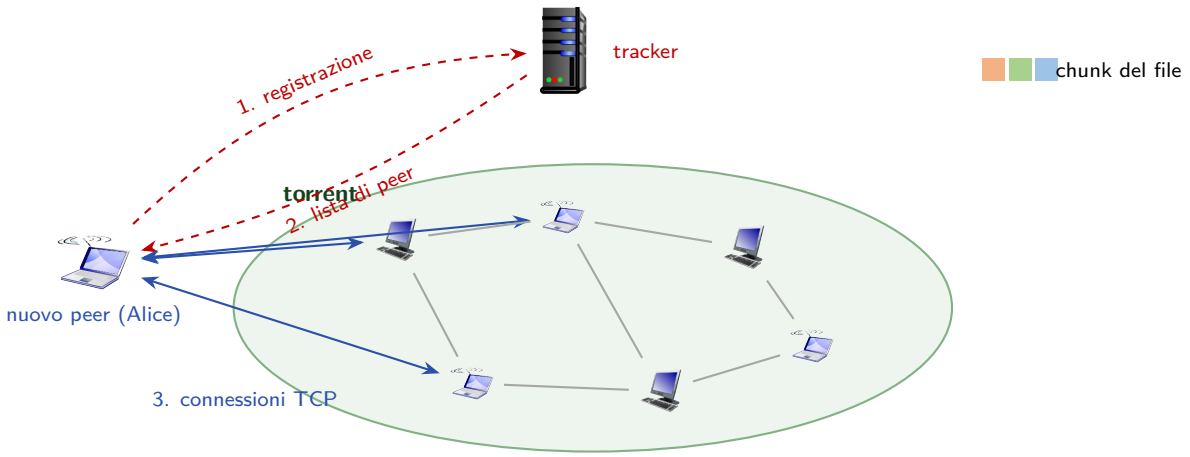


Figura 6.6 – Un nuovo peer si registra presso il tracker, riceve un sottoinsieme di peer e apre con loro connessioni TCP.

Quando un nuovo peer entra in un torrent:

- si **registra** presso il tracker e lo informa periodicamente del proprio stato;
- il tracker gli fornisce gli indirizzi IP di un **sottoinsieme casuale** di peer del torrent;
- avuta la lista, il nuovo peer prova ad aprire **connessioni TCP** concorrenti con tutti i peer della lista, che diventano i suoi **vicini** (*neighbors*);
- durante l'esecuzione alcuni vicini se ne vanno, mentre altri peer (fuori dalla lista iniziale) possono chiedere di connettersi.

In ogni istante ogni peer ha un sottoinsieme diverso dei chunk. Periodicamente un peer chiede a ogni vicino (sulle connessioni TCP) la lista dei chunk che possiede.

Le politiche del client

Rarest first

Chi **scarica** decide quali chunk chiedere, e a chi, secondo il principio **rarest first**: si danno priorità ai chunk che hanno **meno copie** tra i vicini. Così i chunk rari vengono ridistribuiti più rapidamente e il numero di copie di ciascun chunk nel torrent tende a equilibrarsi.

Chi **carica** decide a quali richieste rispondere secondo due principi intrecciati:

- **Scambio** (*trading*, detto anche *tit-for-tat*): si dà priorità ai **4 vicini migliori**, cioè quelli che in quel momento ci stanno inviando dati alla velocità più alta. Il controllo viene ripetuto ogni **10 secondi** e la lista dei 4 migliori aggiornata.
- **Selezione casuale** (*optimistic unchoking*): ogni **30 secondi** si sceglie anche un vicino **a caso** e gli si inviano chunk. Se lo scambio va bene (i due sono buoni partner), i due entrano nelle rispettive liste dei migliori. Questo permette anche ai peer con velocità di upload compatibili di trovarsi, e ai nuovi arrivati di ricevere i primi chunk.

Esempio 3 — Perché la selezione casuale

Alice ha appena iniziato e non ha nulla da offrire: nessuno la metterebbe tra i propri 4 migliori. Grazie alla selezione casuale, prima o poi un peer (Bob) le invia qualche chunk; Alice diventa così capace di ricambiare, magari entra tra i migliori di Bob, e Bob tra i suoi. Chi carica di più riceve di più: i "parassiti" che scaricano senza caricare vengono penalizzati.

Riepilogo

In breve

- La posta usa **user agent** e **mail server**; **SMTP** (TCP, porta 25) trasferisce i messaggi fino alla casella del destinatario, anche tra server.
- Per leggere la posta servono protocolli di accesso: **POP3** (semplice: autorizzazione, transazione, aggiornamento), **IMAP** (cartelle e ricerca sul server), **HTTP** (webmail).
- Nel **P2P** i peer sono client e server insieme: la distribuzione di file scala meglio.
- **BitTorrent**: torrent, chunk, **tracker**; **rarest first** per scaricare, **4 migliori ogni 10 s** + **uno casuale ogni 30 s** per caricare.

Il DNS

Il problema: nomi e indirizzi

Un host si può identificare in due modi: con un **nome** (*hostname*) o con un **indirizzo IP**. Le persone preferiscono i nomi, facili da ricordare; i dispositivi di rete preferiscono gli indirizzi IP, di lunghezza fissa e strutturati gerarchicamente.

Domain Name System (DNS)

Il **DNS** è un protocollo di livello applicativo che gestisce la **traduzione dei nomi degli host in indirizzi IP**. È un protocollo **client-server**: un client DNS chiede a un server DNS la traduzione di un certo nome. I server DNS sono spesso macchine UNIX con il software **BIND** (*Berkeley Internet Name Domain*) e usano tipicamente **UDP** sulla **porta 53**.

Il DNS è anche un esempio perfetto di **database distribuito**: nessun server contiene tutte le associazioni, ma l'insieme dei server sì.

Un'applicazione al servizio di altre applicazioni

Il DNS non è quasi mai usato direttamente dagli utenti: lo usano le altre applicazioni. Quando il browser deve aprire `http://www.unina.it/index.html`, prima chiede al DNS l'indirizzo di `www.unina.it`, poi apre la connessione TCP verso quell'indirizzo. Il ritardo introdotto dal DNS si somma quindi a quello di ogni navigazione, ed è uno dei motivi per cui usa UDP e fa largo uso di cache.

Perché non un repository centralizzato

La soluzione più semplice sarebbe un unico server con una grande tabella nome–indirizzo. Non funziona:

- **troppi host**: un'unica tabella con i nomi di tutta Internet sarebbe enorme e sempre da aggiornare;
- **distanza e ritardi**: un server unico sarebbe lontano dalla maggior parte degli utenti;
- **volume di traffico**: dovrebbe rispondere a tutte le interrogazioni del mondo;
- **single point of failure**: se si guasta, si ferma Internet.

Il DNS è quindi un sistema **distribuito** e **decentralizzato**:

- **gerarchico**;
- **basato sui domini**;
- realizzato tramite un **database distribuito** su molti server.

Resolver

Il **resolver** è il programma (di solito una funzione di libreria) che restituisce l'indirizzo IP associato a un nome. Interroga un **server DNS locale**, e tutti i messaggi sono pacchetti **UDP**.

Lo spazio dei nomi DNS

- La gerarchia dei nomi è organizzata dall'**ICANN** (*Internet Corporation for Assigned Names and Numbers*).
- Sotto la radice ci sono i **domini di primo livello** (*top-level domain*, TLD), alcune centinaia.
- Ogni dominio è diviso in **sottodomini**, e così via.

I domini di primo livello sono di due tipi, tutti gestiti da *registrar* nominati dall'ICANN:

- **generici**: `com`, `edu`, `org`, `net`, `aero`, …;
- **nazionali** (*country code*): `it`, `uk`, `jp`, `tv`, …

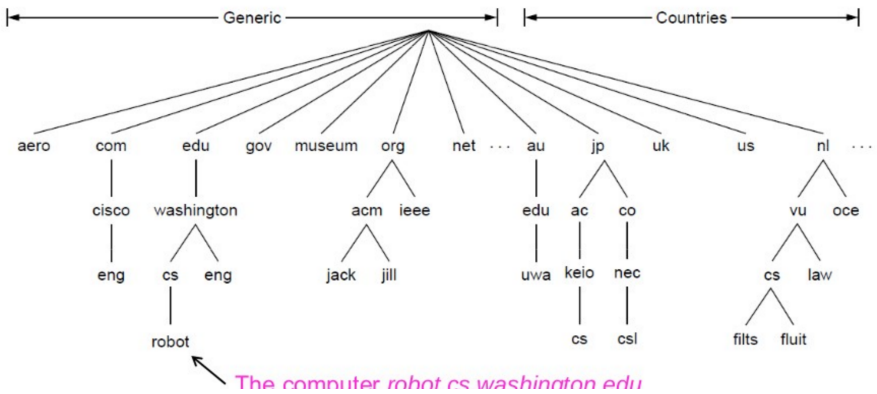


Figura 7.1 – Lo spazio dei nomi è un albero, dalla radice verso il basso; parti diverse sono delegate a organizzazioni diverse. In rosso il computer `robot.cs.washington.edu`.

Dominio	Uso previsto	Dal	Riservato?
com	commerciale	1985	no
edu	istituzioni educative	1985	sì
gov	governo	1985	sì
int	organizzazioni internazionali	1988	sì
mil	militare	1985	sì
net	fornitori di rete	1985	no
org	organizzazioni non profit	1985	no
aero, biz, info, name, pro, …	settori specifici	2001–2010	vari

Tabella 7.1 – Alcuni domini generici di primo livello, controllati dall'ICANN.

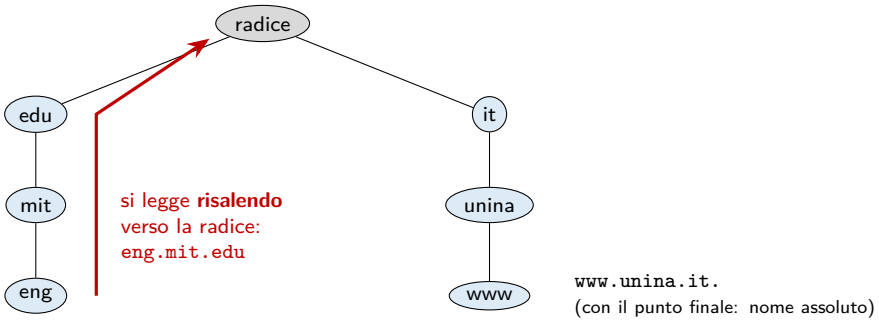


Figura 7.2 – Un nome di dominio è il percorso dal nodo verso la radice.

- I nomi sono **percorsi verso l'alto**, dal dominio alla radice: `eng.mit.edu`.
- I percorsi **assoluti** terminano con un punto (`eng.mit.edu.`); quelli **relativi** vanno interpretati rispetto a un dominio di partenza.
- Si può registrare lo stesso nome sotto più TLD (`ibm.com`, `ibm.us`).
- Il **cyber-squatting** è la pratica di comprare domini solo per rivenderli poi alle aziende interessate.
- Ogni dominio **controlla l'assegnazione** dei domini sotto di lui; creare un nuovo dominio richiede il **permesso del dominio padre**.
- La struttura dei nomi **non** segue confini fisici o geografici (`cs.washington.edu` e un server in un altro continente possono stare nello stesso dominio).

I RESOURCE RECORD

Il database distribuito del DNS è fatto di **resource record** (RR). Ogni record ha cinque campi:

- **NAME** (nome di dominio): il dominio a cui si riferisce il record. È la **chiave di ricerca** principale delle interrogazioni; di solito per ogni dominio esistono più record, su server diversi.
- **TTL** (*time to live*): la durata di validità del record, in secondi. Lunga per host stabili (86 400 = un giorno), breve per quelli volatili (60 = un minuto); il valore massimo è $2^{31} - 1$ secondi, circa 68 anni.
- **CLASS**: per i record di Internet è sempre **IN**; gli altri codici sono usati raramente.
- **TYPE**: il tipo di record (tabella seguente).
- **VALUE** (RDATA): il valore, che dipende dal tipo: un dominio, un numero o una stringa ASCII.

Tipo	Significato	Valore
SOA	*Start Of Authority*	parametri della zona: server principale, contatto dell'amministratore, …
A	*Address*	indirizzo IPv4 (32 bit) di un'interfaccia dell'host
AAAA	indirizzo IPv6	indirizzo IPv6 (128 bit)
MX	*Mail eXchange*	nome dell'host che accetta la posta per il dominio (con priorità)
NS	*Name Server*	nome di un server DNS per il dominio
CNAME	*Canonical NAME*	nome canonico di cui questo nome è un alias
PTR	*Pointer*	nome associato a un indirizzo (ricerca inversa)
SPF	*Sender Policy Framework*	regole su chi può inviare posta per il dominio
SRV	*Service*	host che offre un certo servizio
TXT	*Text*	testo ASCII descrittivo

Tabella 7.2 – I tipi principali di resource record.

Come "leggere" i record

- `(unina.it, dns.unina.it, IN, NS)`: `dns.unina.it` è un name server per il dominio `unina.it`; la ricerca prosegue interrogando lui.
- `(www.unina.it, 143.225.15.50, IN, A)`: l'indirizzo IPv4 di `www.unina.it`.
- `(www.ibm.com, servereast.backup2.ibm.com, IN, CNAME)`: `www.ibm.com` è un alias; la ricerca **prosegue** con il nome canonico.
- `(ibm.com, mail.bar.ibm.com, IN, MX)`: la posta per `@ibm.com` va consegnata a `mail.bar.ibm.com`.
- Con PTR, invece, la ricerca **non prosegue**: si restituisce semplicemente il nome.

Un host può avere **più interfacce**, e quindi più record A.

Esempio 1 — I record di un dominio: `cs.vu.nl`

```
; Authoritative data for cs.vu.nl
cs.vu.nl.       86400  IN  SOA    star boss (9527,7200,7200,241920,86400)
cs.vu.nl.       86400  IN  MX     1 zephyr
cs.vu.nl.       86400  IN  MX     2 top
cs.vu.nl.       86400  IN  NS     star                    <-- Name server

star            86400  IN  A      130.37.56.205
zephyr          86400  IN  A      130.37.20.10
top             86400  IN  A      130.37.20.11            <-- IP addresses of computers
www             86400  IN  CNAME  star.cs.vu.nl
ftp             86400  IN  CNAME  zephyr.cs.vu.nl

flits           86400  IN  A      130.37.16.112
flits           86400  IN  A      192.31.231.165
flits           86400  IN  MX     1 flits
flits           86400  IN  MX     2 zephyr
flits           86400  IN  MX     3 top

rowboat                IN  A      130.37.56.201
                       IN  MX     1 rowboat               <-- Mail gateways
                       IN  MX     2 zephyr

little-sister          IN  A      130.37.62.23

laserjet               IN  A      192.31.231.216
```

Nel database autorevole per `cs.vu.nl` si trovano: il record **SOA** della zona, i record **MX** (i server di posta, con la loro priorità: si prova prima quello con numero più basso), il record **NS** (il name server `star`), i record **A** con gli indirizzi dei computer (`star`, `zephyr`, `top`, …) e i **CNAME** che definiscono alias come `www` e `ftp`.

I NAME SERVER E LE ZONE

Un unico server DNS centralizzato non avrebbe senso. Lo spazio dei nomi è quindi diviso in **zone**.

Zona

Una **zona** è una porzione dello spazio dei nomi DNS, **senza sovrapposizioni** con le altre, i cui dati sono gestiti da uno o più **name server** (un *primario* e uno o più *secondari*). Come dividere il proprio dominio in zone lo decide l'amministratore.

Un dominio non è una zona

Una zona è la parte dell'albero che un'amministrazione serve davvero: un dominio, **meno** ciò che ha delegato ad altri. `unina.it` è una zona; un dipartimento che riceve un proprio server diventa una zona a sé ed esce dalla prima. Il dominio dice **come sono fatti i nomi**, la zona dice **chi risponde** per essi. E i confini sono amministrativi, non geografici: due host dello stesso dominio possono stare in continenti diversi.

- Ogni zona ha almeno **due** name server, su macchine separate: un **primario** e uno o più **secondari**. Entrambi sono **autorevoli**: le loro risposte sono il dato stesso, non la copia della risposta di qualcun altro.
- Il secondario copia la zona dal primario a intervalli regolari, e continua a rispondere quando il primario è irraggiungibile.
- Un'organizzazione può gestirli da sé o pagare un fornitore: la maggior parte delle università li gestisce da sé. Per esempio `a.nic.va` ha autorità su `vatican.va`: è lì che quei record vivono davvero.

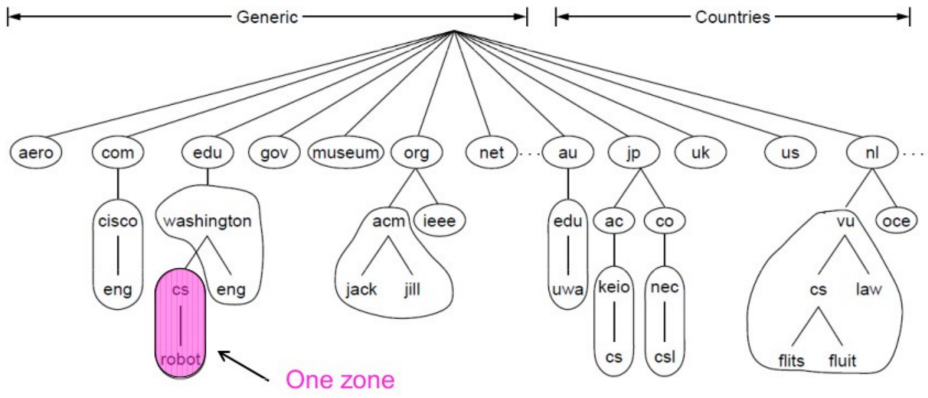
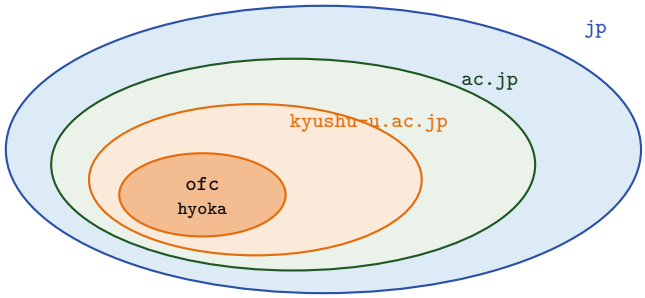


Figura 7.3 – Le zone (cerchiate) sono porzioni dello spazio dei nomi gestite da name server diversi.



Può darsi che un server per `kyushu-u.ac.jp` abbia anche i record A, con autorità, per i nodi della sotto-zona (in arancione scuro), oppure che la sotto-zona `ofc` sia delegata a un altro server, come nell'esempio con `nslookup` alla fine del capitolo.

Figura 7.4 – Domini annidati: ogni livello può gestire direttamente i nomi sottostanti o delegarli a una zona separata.

Server autorevole

Un name server è **autorevole** (*authoritative*) per una zona se contiene i record originali di quella zona. Le risposte che fornisce su quella zona sono **risposte autorevoli**; quelle ottenute da una cache sono **non autorevoli**.

La risoluzione dei nomi

La gerarchia dei server

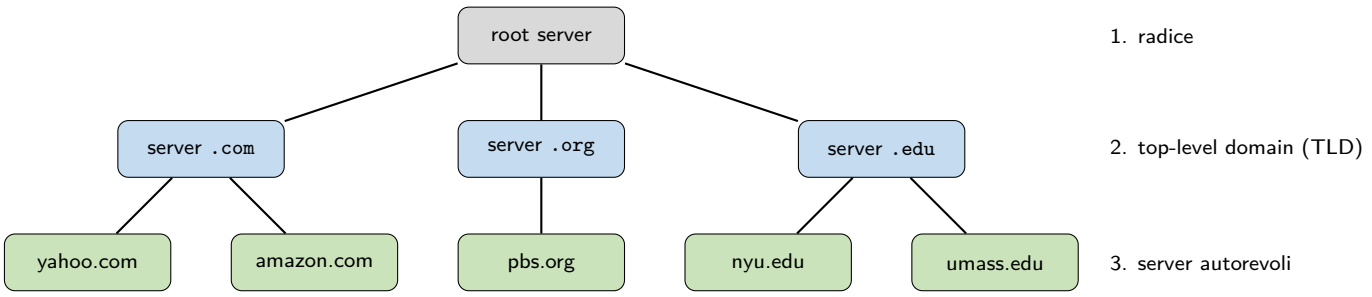


Figura 7.5 – Una parte della gerarchia dei server DNS.

- **Root server**: sono il punto di partenza quando non si ha nessuna informazione. Ci sono **13** "root server" logici, da `a.root-servers.net` a `m.root-servers.net`, ciascuno **altamente replicato** in centinaia di copie nel mondo. Sono molto carichi, quindi di solito restituiscono informazioni sulle zone inferiori (i server dei TLD), non sui singoli record.
- **Server TLD**: responsabili dei domini di primo livello (`com`, `it`, `jp`, …).
- **Server autorevoli**: quelli delle organizzazioni, che contengono i record dei loro host.

Il server DNS locale (resolver)

Server DNS locale

Il **server DNS locale** (o **DNS resolver**) non appartiene strettamente alla gerarchia: è gestito tipicamente dall'ISP. Quando un host si collega a un ISP, riceve gli indirizzi di uno o più server DNS locali, di solito "vicini". Le interrogazioni dell'host vanno al server locale, che fa da **proxy** e le inoltra alla gerarchia.

Anche Google offre server di questo tipo in tutto il mondo (*Google Public DNS*, agli indirizzi `8.8.8.8` e `8.8.4.4`). Su molti sistemi Linux il resolver locale risponde all'indirizzo `127.0.0.53`, porta 53: è un processo sulla macchina stessa che inoltra le richieste al server indicato dal router.

Query ricorsive e iterative

La risoluzione procede così:

1. l'host interroga il **server DNS locale**;
2. se il nome ricade nella zona di competenza del server locale, questo restituisce un record autorevole;
3. altrimenti il server locale gestisce la richiesta ed effettua interrogazioni remote:
   - se il server interrogato ha l'indirizzo, lo restituisce;
   - altrimenti restituisce il **name server di una zona inferiore** a cui chiedere;
   - il processo prosegue **iterativamente** finché l'indirizzo non viene trovato.

Query ricorsiva e iterativa

- In una **query ricorsiva** chi riceve la domanda si fa carico di **tutta** la risoluzione e restituisce la risposta completa (o un errore).
- In una **query iterativa** chi riceve la domanda risponde con ciò che sa: la risposta, oppure **il nome del prossimo server** da interrogare.

Tipicamente l'host fa una query **ricorsiva** al server locale, e il server locale fa query **iterative** verso la gerarchia. I server più carichi, come quelli vicini alla radice, **non** accettano query ricorsive.

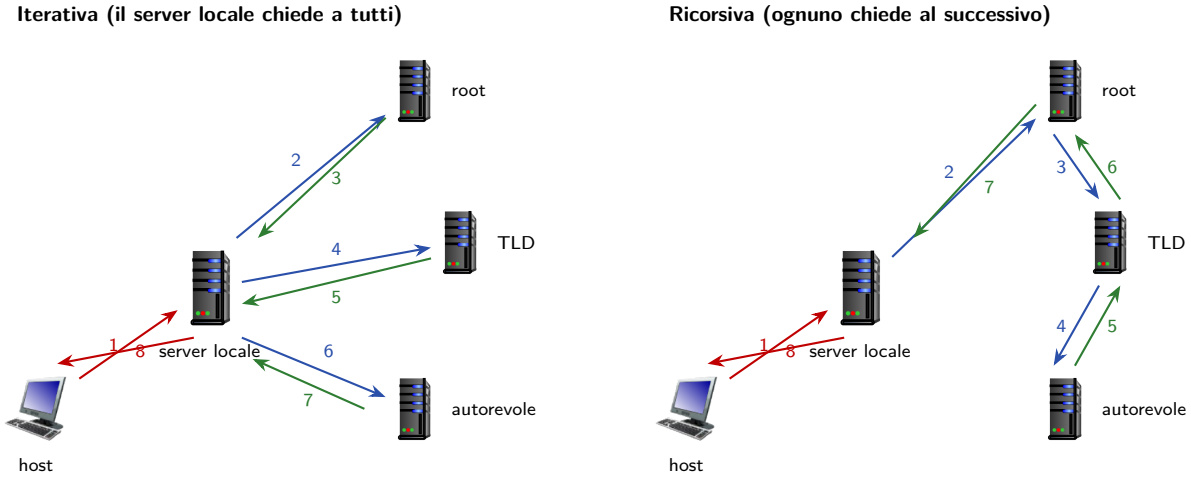


Figura 7.6 – Query iterativa e ricorsiva a confronto: nel primo caso il server locale conduce la ricerca, nel secondo il carico ricade sui server della gerarchia.

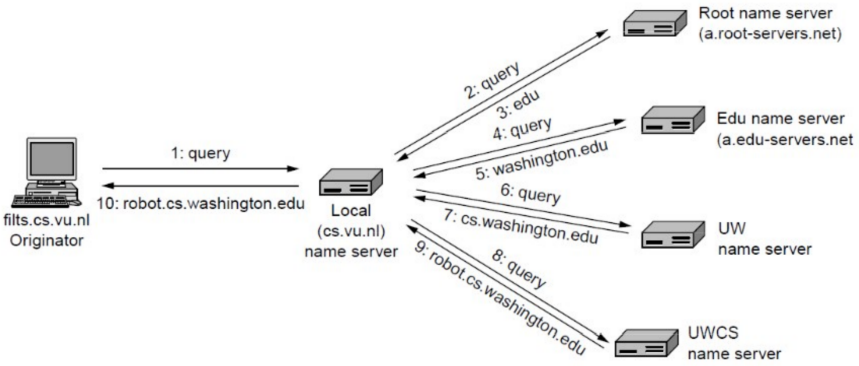


Figura 7.7 – L'host `flits.cs.vu.nl` risolve `robot.cs.washington.edu`: query ricorsiva (1, 10) verso il server locale, query iterative (2–9) verso root, `edu`, `washington.edu` e `cs.washington.edu`.

Il caching DNS

- I record possono essere **autorevoli** o **in cache** (*cached*).
- I record in cache **scadono**: è a questo che serve il TTL.
- Il caching migliora molto le prestazioni: **tutte le risposte** vengono memorizzate.
- Se il nome richiesto è in cache, il server locale risponde **subito**.
- Se arriva una richiesta per `cs.mit.edu` e il server conosce già l'indirizzo del server DNS di `mit.edu`, può interrogare direttamente quello, saltando root e TLD.
- I root server vengono interrogati solo quando non si ha **nessuna informazione** su un certo dominio.

Cache e TTL

Se un sito cambia indirizzo IP, per un po' alcuni utenti continueranno a raggiungere quello vecchio: i resolver hanno ancora in cache il record, finché il suo TTL non scade. Per questo prima di un trasloco si abbassa il TTL dei record interessati.

Esempio 2 — Quanto costa il TTL

`www.example.it` era `138.12.27.42` e viene spostato su una nuova macchina, `107.46.72.221`.

- La vecchia coppia non sta in un posto solo: **ogni** server che ha risolto quel nome negli ultimi giorni ne ha una copia.
- Nessuno va a correggerle: quelle copie continuano a rispondere con il vecchio indirizzo finché il loro TTL non scade.
- Un TTL **breve** propaga in fretta i cambiamenti ma costa più query; uno **lungo** risparmia query e ritarda i cambiamenti (86400 secondi, un giorno, per un record stabile; 60 per uno che si sposta).

Per questo il TTL si abbassa **giorni prima** di un trasloco programmato, e si rialza quando il nuovo indirizzo si è assestato.

I MESSAGGI DNS

Richieste e risposte DNS hanno lo stesso formato (utile per leggere le catture con Wireshark):

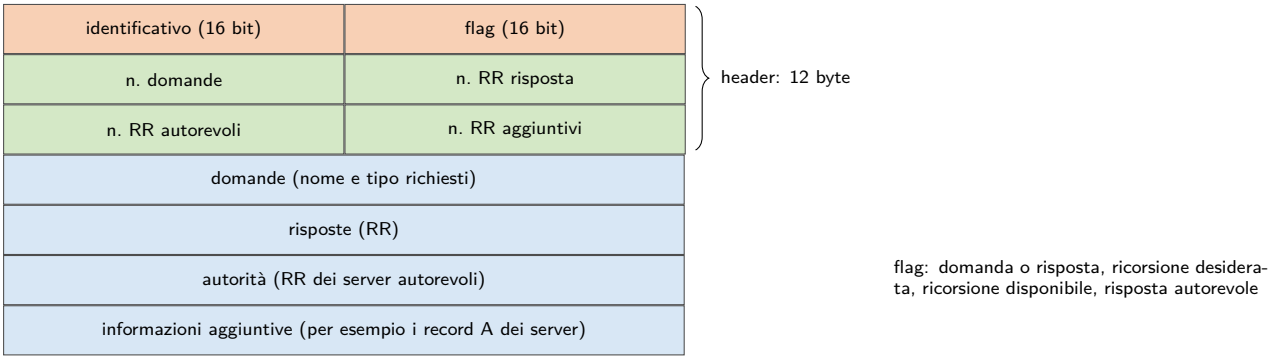


Figura 7.8 – Formato dei messaggi DNS: l'identificativo permette di associare la risposta alla domanda.

ALTRI SERVIZI DEL DNS

Aliasing

Il DNS offre un servizio di **alias**: a nomi complicati e lunghi (**canonici**) si associano nomi più semplici (**alias**).

- **Alias di host**: `relay1.west-coast.enterprise.com` può essere raggiunto come `www.enterprise.com` (record CNAME).
- **Alias di server di posta**: grazie ai record MX, l'indirizzo `bob@yahoo.com` funziona anche se il server di posta vero ha un nome complicato; e un'azienda può usare lo stesso nome per il sito Web e per la posta.
- Un'applicazione può comunque chiedere al DNS il **nome canonico** di un alias, oltre all'indirizzo IP.

Distribuzione del carico

Round-robin DNS

I siti molto frequentati (Google, Amazon, CNN, …) sono replicati su più server, ciascuno con un indirizzo IP diverso: al nome canonico è associato un **insieme di indirizzi**. Quando un client chiede quel nome, il server DNS restituisce tutti gli indirizzi ma **ne ruota l'ordine** a ogni risposta; poiché i client usano di solito il primo, il carico si distribuisce tra le repliche.

NSLOOKUP: INTERROGARE IL DNS A MANO

Con `nslookup` si possono fare a mano le interrogazioni che fa un server DNS, e anche la **ricerca inversa** (dal nome all'indirizzo). La sintassi è `nslookup [-type=TIPO] nome [server]`.

Esempio 3 — Record A: l'indirizzo di un nome

```
$ nslookup vatican.va
Server:    127.0.0.53
Address:   127.0.0.53#53

Non-authoritative answer:
Name:    vatican.va
Address: 185.152.70.106
```

Si recupera il record **A** di `vatican.va`. Il server interrogato è `127.0.0.53`, cioè il resolver locale sulla porta 53. La risposta è **non autorevole**: viene da una cache, non dal server che gestisce la zona. Il resolver locale ha condotto l'intera risoluzione con query iterative, mentre l'host gli ha fatto una query ricorsiva.

Esempio 4 — Record NS e SOA

```
$ nslookup -type=NS vatican.va
Non-authoritative answer:
vatican.va  nameserver = a.nic.va.
vatican.va  nameserver = b.nic.va.
vatican.va  nameserver = c.nic.va.
vatican.va  nameserver = osiris.namex.it.
vatican.va  nameserver = seth.namex.it.

$ nslookup -type=SOA vatican.va
Non-authoritative answer:
vatican.va
    origin = a.nic.va
    mail addr = dnsop.net.va
    serial = 2025062401
    refresh = 14400
    retry = 3600
    expire = 1209600
    minimum = 3600
```

La prima query elenca i **name server** del dominio. La seconda chiede il record **SOA**, che indica il server principale autorevole (`origin = a.nic.va`) e i parametri della zona. La riga "*Authoritative answers can be found from:*" senza elenco è normale con i resolver intermedi, che spesso non includono i record dei server autorevoli.

Esempio 5 — Interrogare direttamente il server autorevole

```
$ nslookup -type=A a.nic.va
Name:    a.nic.va
Address: 212.77.0.110

$ nslookup -type=A vatican.va 212.77.0.110
Server:  212.77.0.110
Address: 212.77.0.110#53
Non-authoritative answer:
Name:    vatican.va
Address: 185.152.70.106
```

Prima si trova l'indirizzo del server principale, poi lo si interroga direttamente (secondo argomento). In teoria la risposta dovrebbe essere autorevole, ma alcuni server non impostano il flag *Authoritative Answer* (per configurazione, inoltro o motivi di sicurezza).

La risoluzione iterativa fatta a mano

Proviamo a risolvere `hyoka.ofc.kyushu-u.ac.jp` **ignorando il server locale**, come farebbe un server DNS senza nulla in cache: si parte da un root server e si scende.

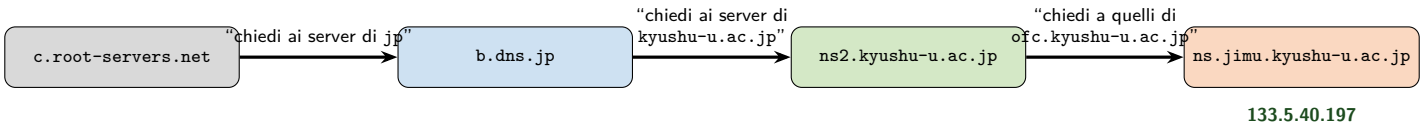


Figura 7.9 – Ogni server non conosce la risposta, ma sa a chi chiedere un livello più in basso.

```
$ nslookup -type=A hyoka.ofc.kyushu-u.ac.jp c.root-servers.net
*** Can't find hyoka.ofc.kyushu-u.ac.jp: No answer        # il root non sa nulla

$ nslookup -type=NS hyoka.ofc.kyushu-u.ac.jp c.root-servers.net
Authoritative answers can be found from:
jp  nameserver = a.dns.jp.   (... b.dns.jp ... h.dns.jp)  # ma conosce i server di jp
b.dns.jp  internet address = 202.12.30.131

$ nslookup -type=NS hyoka.ofc.kyushu-u.ac.jp b.dns.jp
Authoritative answers can be found from:
kyushu-u.ac.jp  nameserver = ns1.kyushu-u.ac.jp.          # jp conosce i server di kyushu-u
kyushu-u.ac.jp  nameserver = ns2.kyushu-u.ac.jp.
ns2.kyushu-u.ac.jp  internet address = 133.5.1.9

$ nslookup -type=NS hyoka.ofc.kyushu-u.ac.jp ns2.kyushu-u.ac.jp
Authoritative answers can be found from:
ofc.kyushu-u.ac.jp  nameserver = ns.jimu.kyushu-u.ac.jp.  # sotto-zona ofc
ns.jimu.kyushu-u.ac.jp  internet address = 133.5.40.1

$ nslookup -type=A hyoka.ofc.kyushu-u.ac.jp ns.jimu.kyushu-u.ac.jp
Name:    hyoka.ofc.kyushu-u.ac.jp
Address: 133.5.40.197                                    # finalmente il server autorevole
```

Da provare

Si possono interrogare i server in alto nella gerarchia (compresi i root server) con l'opzione `-norecurse`, e poi senza, confrontando i risultati. Altri esempi utili: `nslookup -type=MX mit.edu` (i server di posta) e `nslookup -type=NS mit.edu` (i server DNS con autorità).

DIG: LEGGERE IL MESSAGGIO DNS

Nelle lezioni si usa `dig`, che stampa il **messaggio DNS** quasi per intero (su Windows c'è `nslookup`, che ne mostra meno).

```
$ dig vatican.va
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 19917
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; QUESTION SECTION:
;vatican.va.                    IN      A
;; ANSWER SECTION:
vatican.va.             21559   IN      A       185.152.70.106
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
```

- La query è andata a `127.0.0.53`, il resolver locale di questa macchina, che l'ha inoltrata.
- Il numero prima di `IN` è il **TTL** rimasto, in secondi.
- Le sezioni stampate (*question*, *answer*, *authority*, *additional*) sono proprio quelle del formato dei messaggi DNS visto sopra.

I flag

La riga `flags` descrive come il server ha trattato la query.

Flag	Nome	Significato
`qr`	Query/Response	presente in ogni risposta
`aa`	Authoritative Answer	chi risponde è la fonte di verità della zona, non una cache
`tc`	Truncated	risposta troppo grande per UDP: riprovare su TCP
`rd`	Recursion Desired	il client ha chiesto la ricorsione
`ra`	Recursion Available	il server conferma di poter fare la ricorsione
`ad`	Authentic Data	la validazione DNSSEC è riuscita
`cd`	Checking Disabled	il client ha chiesto di saltare la validazione DNSSEC (`+cd`)

Tabella 7.3 – I flag più comuni nell'intestazione di una risposta di `dig`.

Nell'esempio di `vatican.va`: `qr`, è una risposta; `rd`, abbiamo chiesto la ricorsione; `ra`, il server la offre; niente `aa`, la risposta viene da una cache; niente `ad`, DNSSEC non è stato validato. Interrogando direttamente un server autorevole si vedono invece `flags: qr aa`, senza `rd` e `ra`, perché la ricorsione non è stata né chiesta né necessaria.

Chiedere un tipo

Il tipo va dopo il nome; senza tipo, `dig` chiede un indirizzo (A). Con `+noall +answer` si stampa solo la sezione delle risposte.

```
$ dig +noall +answer www.mit.edu A
www.mit.edu.               1070 IN  CNAME  www.mit.edu.edgekey.net.
www.mit.edu.edgekey.net.     51 IN  CNAME  e9566.dscb.akamaiedge.net.
e9566.dscb.akamaiedge.net.   20 IN  A      23.75.122.29
```

Due CNAME prima di un indirizzo: il nome pubblicato è un alias di un altro nome, e la macchina che alla fine risponde appartiene a un'altra azienda (una CDN). E si guardino i TTL: l'alias viene tenuto per minuti, l'indirizzo finale per secondi.

```
$ dig +noall +answer mit.edu MX
mit.edu.            59 IN MX 100 mit-edu.mail.protection.outlook.com.
$ dig +noall +answer mit-edu.mail.protection.outlook.com A
mit-edu.mail.protection.outlook.com. 10 IN A 52.101.60.185
mit-edu.mail.protection.outlook.com. 10 IN A 52.101.41.125
mit-edu.mail.protection.outlook.com. 10 IN A 52.101.9.26
mit-edu.mail.protection.outlook.com. 10 IN A 52.101.11.19
```

- Un record **MX** dà un **nome** e una preferenza (100), mai un indirizzo: serve sempre una seconda query.
- Quattro indirizzi per un nome: più server di posta equivalenti (distribuzione del carico), e la posta di questo dominio è gestita da un'altra azienda.

```
$ dig +noall +answer mit.edu NS
mit.edu.   840 IN NS use2.akam.net.
mit.edu.   840 IN NS eur5.akam.net.
mit.edu.   840 IN NS asia1.akam.net.        # otto in tutto, su tre continenti
$ dig +noall +answer +multiline mit.edu SOA
mit.edu.   1800 IN SOA use2.akam.net. network-request.mit.edu. (
                23589      ; serial
                900        ; refresh (15 minutes)
                900        ; retry (15 minutes)
                3600000    ; expire (5 weeks 6 days 16 hours)
                3600       ; minimum (1 hour)
                )
```

NS elenca tutti i server con autorità sulla zona; **SOA** nomina per primo l'unico da cui gli altri copiano. I suoi parametri sono l'orologio della zona: ogni quanto i secondari si riallineano (*refresh*), dopo quanto riprovano (*retry*), quando smettono di considerare valida la copia (*expire*) e il TTL minimo dei record.

Fare il server locale: partire dalla radice

Si può ignorare il server locale e percorrere la gerarchia a mano, come farebbe lui con le cache vuote. L'opzione `+norecurse` chiede esattamente **un passo**.

```
$ dig +norecurse @c.root-servers.net hyoka.ofc.kyushu-u.ac.jp A
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 8, ADDITIONAL: 16
;; AUTHORITY SECTION:
jp.                     172800   IN      NS        a.dns.jp.
jp.                     172800   IN      NS        b.dns.jp.     # otto in tutto
;; ADDITIONAL SECTION:
a.dns.jp.               172800   IN      A         203.119.1.1
b.dns.jp.               172800   IN      A         202.12.30.131
```

La radice non ha l'indirizzo dell'host: la sezione *answer* è vuota. Invece **delega**: i server di `jp` nella sezione *authority*, e i loro indirizzi in quella *additional* (altrimenti bisognerebbe prima risolvere i loro nomi).

```
$ dig +norecurse @b.dns.jp hyoka.ofc.kyushu-u.ac.jp A
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 3, ADDITIONAL: 3
kyushu-u.ac.jp.            86400 IN NS ns2.kyushu-u.ac.jp.
ns2.kyushu-u.ac.jp.        86400 IN A  133.5.1.9
$ dig +norecurse @ns2.kyushu-u.ac.jp hyoka.ofc.kyushu-u.ac.jp A
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 2, ADDITIONAL: 2
ofc.kyushu-u.ac.jp.         3600 IN NS ns.jimu.kyushu-u.ac.jp.
ns.jimu.kyushu-u.ac.jp.     3600 IN A  133.5.40.1
$ dig +norecurse @ns.jimu.kyushu-u.ac.jp hyoka.ofc.kyushu-u.ac.jp A
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
hyoka.ofc.kyushu-u.ac.jp.   3600 IN A  133.5.40.197
```

Radice → `jp` → `kyushu-u.ac.jp` → `ofc.kyushu-u.ac.jp`: a ogni passo la stessa domanda, e un rinvio al server successivo. Solo l'ultima risposta ha il flag `aa`, perché viene dalla zona stessa. `dig +trace` fa l'intero percorso con un solo comando.

Come un record entra nel database

Si è appena registrato `example.it`: per ora nessuno al mondo può raggiungerlo.

1. Si comunicano al **registrar** i nomi e gli indirizzi del proprio name server primario e secondario.
2. Il registrar inserisce nei **server del TLD** (`it`) un record **NS** per il dominio e un record **A** per ciascuno di quei server.
3. Nei propri server si inseriscono il record **A** del server Web e il record **MX** del server di posta.

Da quel momento la discesa dalla radice arriva fino al nuovo dominio, e un host dall'altra parte del mondo può trovare la macchina senza che nessuno gli abbia detto nulla.

La risposta che nessun server autorevole ha inviato

Abbiamo interrogato direttamente il server autorevole: perché la risposta **non** è autorevole? Perché chi ha risposto **non era** quel server. La risposta è tornata senza il flag `aa`, e quel flag lo imposta la macchina che possiede la zona.

- Una query DNS è un datagramma UDP verso la porta 53: niente connessione, niente autenticazione, niente firma.
- Qualunque apparato lungo il percorso può rispondere **per primo**, a nome del server interpellato, e il resolver non ha modo di accorgersene (alcune reti intercettano così tutto il traffico DNS).
- Il **cache poisoning** e il dirottamento verso il sito Web di qualcun altro sono lo stesso trucco, usato deliberatamente.

Le contromisure

DNSSEC firma i record, così che una risposta falsificata possa essere riconosciuta; i trasporti **cifrati** (DNS over TLS, DNS over HTTPS) nascondono la domanda stessa. Se ne parla nella parte sulla sicurezza.

Laboratorio: cinque domande a un dominio vero

Si sceglie un dominio (`vatican.va`, `unina.it`, uno proprio) e gli si chiedono cinque cose.

```
$ dig vatican.va                    # 1. l'indirizzo
$ dig vatican.va NS                 # 2. chi ha autorita'
$ dig vatican.va SOA                # 3. quale di loro e' il principale
$ dig a.nic.va A                    # 4. il suo indirizzo
$ dig @212.77.0.110 vatican.va A    # 5. chiederlo direttamente a lui
```

I passi 4 e 5 usano ciò che ha restituito il passo 3: prima il nome del server principale, poi l'indirizzo appena risolto per lui.

Esercizio 1 — Domande

(a) Quali delle cinque risposte arrivano senza il flag `aa`, e perché è normale per le prime tre? (b) Il server primario nominato nel record SOA compare fra i record NS? Dovrebbe? (c) Che TTL torna con la prima query, e a quanto scende ripetendola un minuto dopo? (d) Il passo 5 interroga il server che possiede la zona. La risposta è autorevole? Se non lo è, chi ha risposto? (e) Si provi una query MX sullo stesso dominio: quante query servono per arrivare a un indirizzo verso cui aprire una connessione?

Svolgimento

(a) Le prime tre passano dal resolver locale, che risponde dalla propria cache o da quella del server del provider: **nessuna** è autorevole, ed è normale. Anche la quarta, di solito, non lo è.

(b) Di norma sì: il primario è un server autorevole della zona, quindi compare anche fra gli NS. Non è però obbligatorio: alcune organizzazioni tengono un primario "nascosto" (*hidden primary*), da cui i secondari copiano la zona ma che non risponde al pubblico e quindi non viene elencato fra gli NS.

(c) La prima volta torna il TTL rimasto nella cache del resolver; un minuto dopo è **sceso di circa 60** secondi, perché si sta leggendo la stessa copia che invecchia. Quando arriva a zero, il resolver la richiede.

(d) Dovrebbe esserlo, con `aa`. Se manca, ha risposto qualcun altro al posto del server: un apparato lungo il percorso che intercetta le query DNS (si veda la sezione precedente).

(e) Almeno **due**: la query MX restituisce un nome, e serve una seconda query (A o AAAA) per ottenere l'indirizzo. Se il nome passa per dei CNAME, ne servono di più.

Riepilogo

In breve

- Il **DNS** traduce nomi in indirizzi IP: è un protocollo applicativo client-server su **UDP, porta 53**, realizzato come database **distribuito** e **gerarchico**.
- Lo spazio dei nomi è un albero (radice, TLD, sottodomini) diviso in **zone**, ciascuna con i propri name server **autorevoli**.
- I dati sono **resource record** (NAME, TTL, CLASS, TYPE, VALUE): A, AAAA, NS, CNAME, MX, SOA, PTR, …
- L'host fa una query **ricorsiva** al server locale; il server locale fa query **iterative** a root, TLD e server autorevoli.
- Il **caching** (con scadenza data dal TTL) rende il DNS veloce; il DNS offre anche **alias** e **distribuzione del carico**.
- `dig` stampa il messaggio DNS: flag (`aa`, `rd`, `ra`, …) e sezioni answer, authority e additional. Con `+norecurse` si percorre la gerarchia un passo alla volta.
- Le query viaggiano su UDP senza autenticazione: chiunque lungo il percorso può rispondere al posto del server; DNSSEC firma i record.

Parte III

Programmazione di rete

File descriptor, socket e un server TCP sequenziale

Finora ogni protocollo ci è arrivato come una **descrizione**: questi messaggi, in quest'ordine, con questi campi. Abbiamo letto richieste e risposte e detto che il client ne manda una e il server risponde, ma non ci siamo mai chiesti che cosa **fa** concretamente un programma perché un messaggio esca dalla macchina. Quel passo è una manciata di **chiamate di sistema**, e sono le stesse per tutti i protocolli visti.

Con questo capitolo comincia la parte dedicata alla **programmazione di rete**: le funzioni dei socket viste dal punto di vista di chi deve scrivere un server che funzioni davvero: che cosa sia un *file descriptor*, perché `read` e `write` possono trasferire **meno** byte di quelli chiesti, come si gestiscono gli errori, e come è fatto un server TCP che serve un client dopo l'altro.

Programmi, processi e protocolli

Un'applicazione di rete è di norma una **coppia di programmi**, un client e un server, su due end system diversi.

- Un **programma** è un file sul disco; un **processo** è quel file in esecuzione. A comunicare sono i **processi**: la rete non vede mai il primo.
- I due processi comunicano **leggendo e scrivendo sui socket**: nient'altro attraversa il confine tra i due.
- Il lavoro dello sviluppatore è scrivere entrambi i lati, oppure un solo lato conforme a ciò che l'altro si aspetta.

Se il protocollo è **aperto** (pubblicato in un RFC) chiunque può scrivere l'uno o l'altro lato e i due interopereranno: un browser di un produttore parla con il server di un altro. Se è **proprietario**, un solo team scrive entrambi i lati. Un protocollo aperto usa la propria **porta well-known**; un protocollo nostro deve stare **lontano** da quelle porte.

Tutto è un file: il descriptor

File descriptor

Quando un processo chiede al kernel di aprire qualcosa (un file, una pipe, un socket) riceve in cambio un **intero non negativo**: il **file descriptor**. È la «maniglia» del processo sull'oggetto aperto, ed è ciò che si passa a `read` e `write`. Il processo non vede mai altro.

Tre descriptor esistono già prima che il nostro codice parta: `0` è lo **standard input**, `1` lo **standard output**, `2` lo **standard error**. In `<unistd.h>` hanno i nomi `STDIN_FILENO`, `STDOUT_FILENO` e `STDERR_FILENO`. Il primo oggetto aperto dal programma riceve quindi, di norma, il descriptor `3`.

A che cosa punta un descriptor

L'intero è un **indice** in una tabella del processo. La riga della tabella punta a una voce condivisa, detta *open file*, che a sua volta punta all'oggetto vero e proprio.

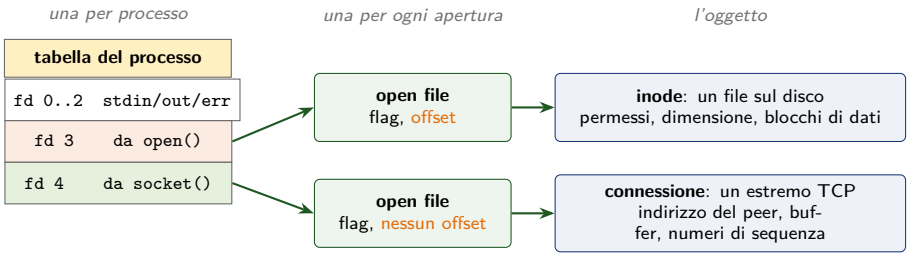


Figura 8.1 – Il descriptor è un indice: lo stesso tipo di intero può portare a un file sul disco o a una connessione TCP.

Un socket è uno di questi oggetti. È **lo stesso tipo di intero, ottenuto da una chiamata diversa** (`socket()` invece di `open()`): è questo il ponte che rende i socket simili ai file. La differenza si vede nella colonna di mezzo: un file ha una **posizione corrente** (l'offset), uno stream di rete no. Per questo `lseek()` funziona su `fd 3` e fallisce su `fd 4`: in un flusso non c'è una posizione verso cui spostarsi.

READ E WRITE

```c
#include <unistd.h>

ssize_t read (int fd,       void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
```

- `read` restituisce il numero di byte **effettivamente** letti, `0` alla fine dell'input, -1 in caso di errore.
- `write` restituisce il numero di byte **effettivamente** scritti, oppure -1 in caso di errore.

La parola chiave è **effettivamente**: nessuna delle due promette `count`.

read può restituire meno di quanto chiesto

Chiedo 1000 byte e ne ricevo 12: è un errore?

No. È il caso **normale** su tutto ciò che non è un file su disco: un terminale restituisce una riga, una pipe ciò che contiene, un socket **ciò che è arrivato finora**.

- I 1000 byte del mittente possono essere ancora in viaggio, divisi in più pacchetti che arrivano in momenti diversi.
- Un segnale può interrompere la chiamata e accorciare la lettura.
- **Conseguenza**: su un socket una sola `read` non basta quasi mai. Si **cicla** finché non si ha tutto ciò che serve.
- Un ritorno di `0` è diverso: non è una lettura corta, significa che **l'altro lato ha chiuso**.

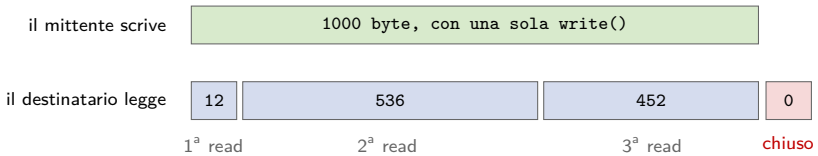


Figura 8.2 – Una sola `write` di 1000 byte può arrivare in tre `read` diverse (le quantità sono un esempio). Solo lo 0 finale dice che non arriverà più nulla.

write può scrivere meno di quanto chiesto

Se `write` restituisce 600 su 1000, **400 byte non sono stati inviati**: il buffer di invio del kernel era pieno, e `write` ha preso quello che poteva. Bisogna **riprovare con il resto**, ripartendo dall'offset indicato dal valore restituito.

```c
/* scrive tutti gli n byte, riprovando dopo le scritture parziali */
int write_all(int fd, const char *p, size_t n) {
    while (n > 0) {
        ssize_t w = write(fd, p, n);
        if (w < 0) return -1;      /* errore vero */
        p += w;                    /* si riparte da dove ci si era fermati */
        n -= w;
    }
    return 0;
}
```

Con i messaggi brevi la scrittura parziale non capita quasi mai: è proprio per questo che la si scopre tardi, quando fa più male.

Scrivere su un socket chiuso

Scrivere su un socket che il peer ha già chiuso è peggio di una scrittura parziale: il processo riceve un **segnale** (`SIGPIPE`) e **termina senza alcun messaggio**. Ci torneremo quando il codice diventerà un server che deve sopravvivere ai propri client.

Quando una chiamata fallisce: -1 ed errno

Quasi tutte le chiamate di sistema segnalano il fallimento nello stesso modo, restituendo -1, e lasciano il **motivo** in una variabile globale, `errno`.

```c
#include <errno.h>

int fd = open("data.txt", O_RDONLY);
if (fd < 0) {                  /* -1: qualcosa e' andato storto */
    perror("open");            /* stampa  open: No such file or directory */
    exit(EXIT_FAILURE);
}
```

`errno` contiene il codice; `perror` stampa la nostra stringa, i due punti e il messaggio corrispondente a quel codice. Da qui in poi il controllo sarà scritto su una riga sola, ma **non va mai omesso**.

Il socket

Socket

Un **socket** è un estremo di un canale di comunicazione bidirezionale tra due processi, identificato da un descriptor.

- È stato introdotto in **BSD 4.2** nel 1983 ed è ancora lo standard *de facto*: Unix, macOS e Windows (con nomi leggermente diversi).
- I due processi possono stare sulla stessa macchina o su due macchine ai capi del mondo: le chiamate non cambiano.
- Non è legato a TCP/IP: la stessa API serve altre famiglie di protocolli, ed è per questo che le chiamate hanno un aspetto così generico.
- Essendo un descriptor, accetta `read`, `write` e `close`: le chiamate che già conosciamo.

Il socket è il confine fra il nostro codice e il sistema operativo: sopra c'è l'applicazione, che scriviamo noi; sotto, trasporto e rete sono implementati dal kernel, che li offre come una scatola nera attraverso l'**API dei socket** (i *socket di Berkeley*, o socket BSD/POSIX). Esistono librerie e *middleware* che li avvolgono, ma sotto c'è sempre questa API.

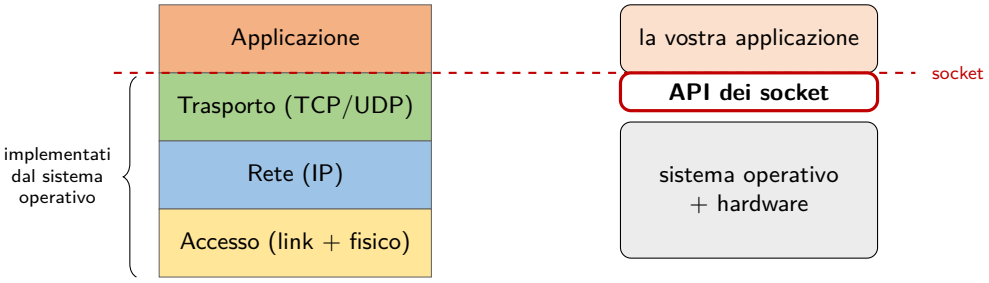


Figura 8.3 – Dove sono implementati gli strati: l'applicazione è nostra, trasporto e rete stanno nel sistema operativo e si usano attraverso l'API dei socket.

Dominio, tipo, protocollo

I tre argomenti di `socket()` scelgono che cosa significa l'API generica.

Dominio: quale famiglia	
`AF_INET`	IPv4
`AF_INET6`	IPv6
`AF_LOCAL`	due processi sulla stessa macchina, attraverso il file system
Tipo: quale genere di canale	
`SOCK_STREAM`	un flusso di byte: connesso, ordinato, affidabile
`SOCK_DGRAM`	messaggi indipendenti: nessuna connessione, nessuna garanzia
`SOCK_RAW`	il pacchetto costruito a mano, sotto il livello di trasporto

Tabella 8.1 – I primi due argomenti di `socket()`.

Il terzo argomento sceglie un protocollo dentro la famiglia, e `0` significa «quello usuale»: **TCP** per uno stream, **UDP** per i datagrammi.

```c
#include <sys/socket.h>

int fd = socket(AF_INET, SOCK_STREAM, 0);    /* IPv4, TCP */
if (fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
```

`socket()` restituisce un descriptor, esattamente come `open()`. L'estremo esiste ma è **anonimo**: non ha ancora un indirizzo e non è collegato a niente. Tutto ciò che segue (`bind`, `connect`, `listen`, `accept`) prende questo descriptor come primo argomento.

Le quattro fasi, con e senza connessione

Client e server, con o senza connessione, **creano** il socket, si **scambiano messaggi** e lo **distruggono**. Solo il lato orientato alla connessione ha una **fase di mezzo**, la connessione: tutto ciò che cambia nel codice tra UDP e TCP deriva da quell'unica fase.

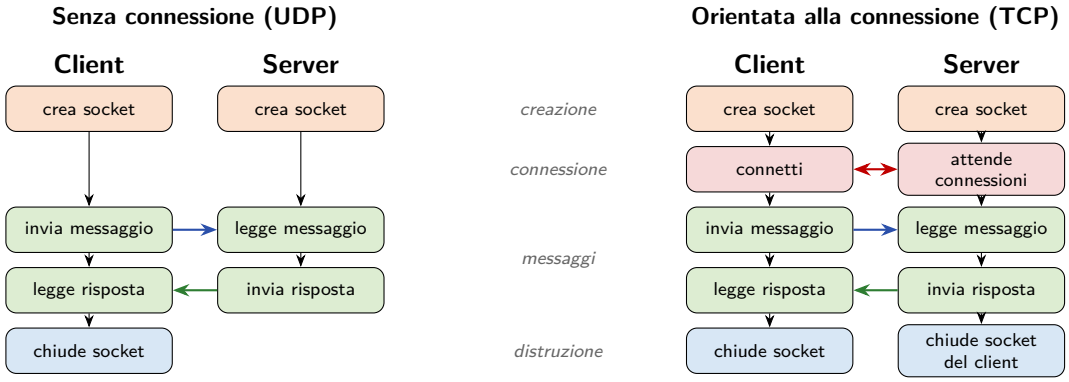


Figura 8.4 – Le fasi di una comunicazione senza connessione e con connessione.

Un indirizzo per l'estremo

```
struct sockaddr {              /* quella generica: e' cio' che prendono le chiamate */
    sa_family_t sa_family;
    char        sa_data[14];   /* 14 byte, il cui significato dipende dalla famiglia */
};

struct sockaddr_in {           /* quella IPv4: e' quella che si riempie */
    short          sin_family; /* AF_INET                               */
    unsigned short sin_port;   /* porta     \                           */
    struct in_addr sin_addr;   /* indirizzo  > gli stessi 14 byte       */
    char           sin_zero[8];/* zeri      /                           */
};
```

Ogni famiglia ha la propria struttura, ma le chiamate prendono un puntatore a quella **generica**: da qui il cast `(struct sockaddr *)` che si vede ovunque. Per riempirla servono `htons` e `inet_pton`, perché la rete scrive i numeri a modo suo: l'ordine dei byte è l'argomento del prossimo capitolo.

LE CHIAMATE LATO SERVER

bind(): un indirizzo locale

```
int bind(int fd, const struct sockaddr *addr, socklen_t len);
```

- Associa il socket a un indirizzo e una porta **locali**: il punto in cui il processo sarà raggiungibile.
- **Server**: obbligatoria. Un client non può bussare a una porta il cui numero nessuno ha concordato prima.
- **Client**: facoltativa, e di norma omessa: il kernel assegna una porta libera al momento della connessione.
- Restituisce `0` o -1. L'errore che si incontra in pratica è **address already in use**, quando la porta è ancora occupata.

listen() e accept()

```
int listen(int fd, int backlog);
int accept(int fd, struct sockaddr *addr, socklen_t *len);
```

- `listen` **non aspetta nulla**: rende il socket **passivo** e apre dietro di lui una **coda**. `backlog` è quante connessioni completate possono attendere prima che il kernel cominci a rifiutarle o ignorarle.
- Le connessioni si completano e **si accodano da sole**; `accept` estrae la prima, una chiamata per ogni client.
- `accept` si blocca finché non ce n'è una, poi restituisce un **descriptor nuovo**, non un codice di ritorno.
- Attraverso `addr` dice anche **chi** si è connesso; se non interessa si passa `NULL`.
- `len` entra come dimensione della nostra struttura ed esce come dimensione effettivamente usata: per questo è un puntatore.

Il server ha due tipi di socket

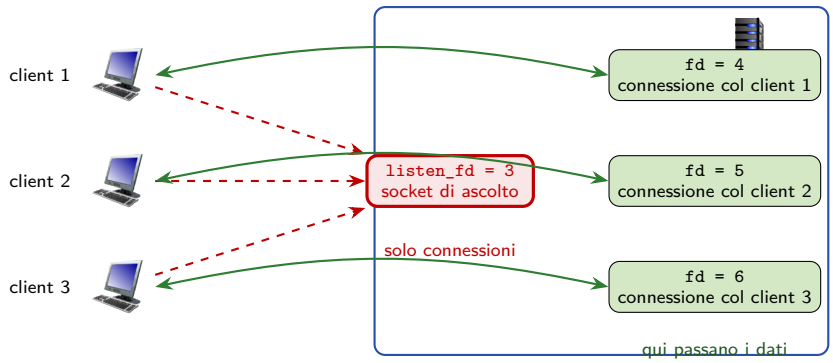


Figura 8.5 – Un server con tre client tiene **quattro** descriptor: uno di ascolto e uno per ogni connessione.

- Il **socket di ascolto** si crea una volta sola, si lega alla porta nota e **non trasporta mai dati**.
- Ogni `accept` restituisce un **socket di connessione**, dedicato a un solo client e inutile per qualunque altro.
- I dati passano solo sul secondo tipo; il primo continua ad ascoltare mentre la conversazione avviene altrove.
- Il three-way handshake che crea la connessione avviene nel livello di trasporto ed è **invisibile** a entrambi i programmi.

LE CHIAMATE LATO CLIENT

```
int connect(int fd, const struct sockaddr *addr, socklen_t len);
```

- Il client non ha bisogno di `bind`, `listen` e `accept`: una sola chiamata fa tutto.
- `connect` ritorna solo quando la connessione è **stabilita**, oppure quando è fallita. In mezzo si svolge l'handshake, e il processo semplicemente aspetta.
- Dopo il ritorno, lo stesso descriptor è un canale bidirezionale verso il socket di connessione del server.

Leggere e scrivere su un socket connesso

```
n = write(fd, "hello\n", 6);        /* nessun indirizzo: e' gia' fissato */
n = read (fd, buf, sizeof buf);
```

Una volta connesso, il descriptor si comporta come qualunque altro: le chiamate di inizio capitolo funzionano senza modifiche, e non si passa alcuna destinazione perché la coppia di indirizzi è stata fissata dalla connessione. `send` e `recv` sono le versioni specifiche dei socket: stessi argomenti più un campo `flags`, che per ora non ci serve. Tutte restituiscono **quanto è stato effettivamente trasferito**: i due casi visti prima ora sono sul percorso critico.

close(), e come il peer capisce che ce ne siamo andati

- `close` dice che **questo processo** ha finito con il descriptor, e lo rilascia.
- Su una connessione avvia anche la chiusura: la `read` successiva del peer restituisce `0`.
- Quello `0` è l'**unico** «fine messaggio» che l'API dà gratis, e costa la connessione.
- La chiusura è **simmetrica**: la connessione è davvero finita quando entrambi i lati hanno chiuso.

Quale descriptor si sta chiudendo?

Un server che chiude il **socket di ascolto** invece del socket di connessione serve un client e poi non ne accetta più nessuno. Con quattro descriptor aperti, chiudere quello sbagliato è l'errore classico.

IL CLIENT E IL SERVER, PER INTERO

```c
int main(void) {
    char buf[256];
    struct sockaddr_in srv;
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
    memset(&srv, 0, sizeof srv);
    srv.sin_family = AF_INET;
    srv.sin_port   = htons(5000);
    inet_pton(AF_INET, "127.0.0.1", &srv.sin_addr);   /* non INADDR_ANY */
    if (connect(fd, (struct sockaddr *) &srv, sizeof srv) < 0) {
        perror("connect"); exit(EXIT_FAILURE); }
    write(fd, "hello\n", 6);
    ssize_t n = read(fd, buf, sizeof buf - 1);         /* spazio per il '\0' */
    if (n > 0) { buf[n] = '\0'; printf("%s", buf); }
    close(fd);
    return 0;
}
```

client.c

Quattro chiamate fanno il lavoro: `socket`, `connect`, `write`, `read`. Il resto è riempire un indirizzo. `inet_pton` («presentation to network») converte `"127.0.0.1"` nel formato binario della rete; nel codice più vecchio si trova `inet_addr`, che non distingue un errore dall'indirizzo `255.255.255.255`.

```c
struct sockaddr_in me;
int yes = 1;
int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
if (listen_fd < 0) { perror("socket"); exit(EXIT_FAILURE); }
setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
memset(&me, 0, sizeof me);
me.sin_family      = AF_INET;
me.sin_port        = htons(5000);
me.sin_addr.s_addr = htonl(INADDR_ANY);    /* qualunque interfaccia locale */
if (bind(listen_fd, (struct sockaddr *) &me, sizeof me) < 0) {
    perror("bind"); exit(EXIT_FAILURE); }
if (listen(listen_fd, 16) < 0) {
    perror("listen"); exit(EXIT_FAILURE); }
```

server.c: il socket di ascolto

`SO_REUSEADDR` permette di **riavviare subito** il server, invece di aspettare che la vecchia porta venga rilasciata (la connessione chiusa resta per qualche minuto nello stato `TIME_WAIT`, e senza questa opzione `bind` fallirebbe con *address already in use*).

```c
static void serve(int fd) {             /* un'intera conversazione */
    char buf[256];
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0)
        if (write(fd, buf, n) != n) break;   /* scrittura parziale: si rinuncia */
    if (n < 0) perror("read");
}

for (;;) {
    struct sockaddr_in cli;
    socklen_t len = sizeof cli;             /* socklen_t, non int */
    int fd = accept(listen_fd, (struct sockaddr *) &cli, &len);
    if (fd < 0) { perror("accept"); continue; }
    serve(fd);
    close(fd);                              /* il socket del client, non il nostro */
}
```

server.c: il ciclo di accept

- `serve` è un server *echo*: rimanda indietro ciò che riceve, finché `read` non restituisce `0` (il client ha chiuso) o -1.
- Se `accept` fallisce il server **non termina**: stampa l'errore e torna ad aspettare.
- Dopo la conversazione si chiude `fd`, il socket del client, e si torna ad `accept`: il socket di ascolto resta aperto per sempre.

Che cosa si potrebbe migliorare

In `serve` una scrittura parziale fa abbandonare il client. È una scelta prudente per un esempio, ma con la funzione `write_all` vista prima il server potrebbe semplicemente completare l'invio.

Provarli sulla stessa macchina

```
$ ./server &
$ ss -tln | grep 5000
LISTEN 0 16 0.0.0.0:5000 0.0.0.0:*
$ nc 127.0.0.1 5000          # non e' il nostro codice
hello
hello
$ ss -tn | grep 5000
ESTAB 0 0 127.0.0.1:47122 127.0.0.1:5000
```

Il server, provato con nc prima di scrivere il client

- `ss -tln` mostra i socket TCP in ascolto (`LISTEN`), con la coda da 16 e l'indirizzo `0.0.0.0`, cioè `INADDR_ANY`.
- `nc` (*netcat*) è un client TCP generico: quello che scriviamo viene inviato al server, e quello che arriva viene stampato. La prima riga `hello` l'abbiamo scritta noi, la seconda è l'eco del server.
- `ss -tn` mostra le connessioni stabilite (`ESTAB`): la porta 47122 è quella scelta dal kernel per il client, che non ha chiamato `bind`.

Client e server comunicano attraverso `127.0.0.1` (*localhost*): nulla raggiunge il cavo. Provare il server con `nc` **prima** di scrivere il client è un'ottima abitudine: se qualcosa non va, si sa già quale dei due programmi è sbagliato.

Un socket trasporta byte, non tipi

Abbiamo scritto `write(fd, &x, sizeof x)` e compila. Sull'altra macchina sarà comunque sbagliato, per tre motivi.

- **Dimensione**: `int` non ha la stessa larghezza ovunque; sulla rete sono sicuri solo i tipi a larghezza fissa come `uint32_t`.
- **Ordine**: le due macchine possono memorizzare i byte di un numero in ordini opposti (argomento del prossimo capitolo).
- **Confini**: nel flusso nulla dice dove finisce un valore o un messaggio. Questo problema spetta a noi.

La risposta più semplice a tutti e tre è inviare **testo**: è ciò che hanno scelto HTTP, SMTP e il formato delle query DNS che vediamo nei comandi.

Codificare un messaggio come testo

```c
char msg[128] = "";                  /* invio: numeri -> una riga */
for (int i = 0; i < n; i++) {
    char field[16];
    snprintf(field, sizeof field, "%d ", a[i]);
    strncat(msg, field, sizeof msg - strlen(msg) - 1);
}
write(fd, msg, strlen(msg));

char *tok = strtok(line, " ");       /* ricezione: riga -> numeri */
while (tok != NULL) {
    vals[k++] = atoi(tok);
    tok = strtok(NULL, " ");
}
```

Con `a = {7, 42, -3}` si invia la stringa `"7 42 -3 "`, e il destinatario ricostruisce gli stessi tre numeri qualunque sia la sua architettura. Entrambi i lati devono però concordare sul **separatore** e sull'**ordine dei campi**: **quell'accordo è il protocollo applicativo**.

Dove finisce un messaggio?

Il client invia due righe. Quante read servono al server?

Non si può sapere. Può essere una sola chiamata che restituisce entrambe le righe, oppure tre chiamate che restituiscono pezzi della prima. TCP è un **flusso di byte** e conserva l'ordine, nient'altro.

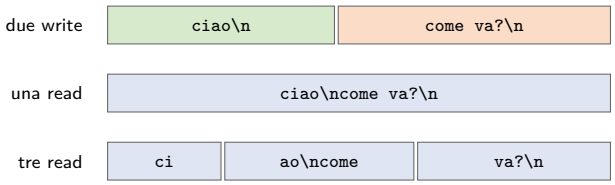


Figura 8.6 – Gli stessi byte possono arrivare raggruppati in modi diversi: i confini tra le `write` non viaggiano.

- Le nostre chiamate `write` **non sono messaggi**: i confini tra una e l'altra non vengono trasmessi.
- Il destinatario quindi non può sapere che un messaggio è completo, a meno che non glielo dica il **protocollo**.
- I modi sono due, e ogni protocollo ne sceglie uno: un **delimitatore** da cercare, oppure una **lunghezza** annunciata prima.
- HTTP li usa entrambi: una riga vuota chiude l'intestazione, poi `Content-Length` conta i byte del corpo.

Un client alla volta

Due client si connettono a un secondo di distanza. Che cosa vede il secondo?

La sua `connect` riesce **subito**, perché la connessione la completa il kernel, e poi **nulla**, finché il server non ha finito di servire il primo.

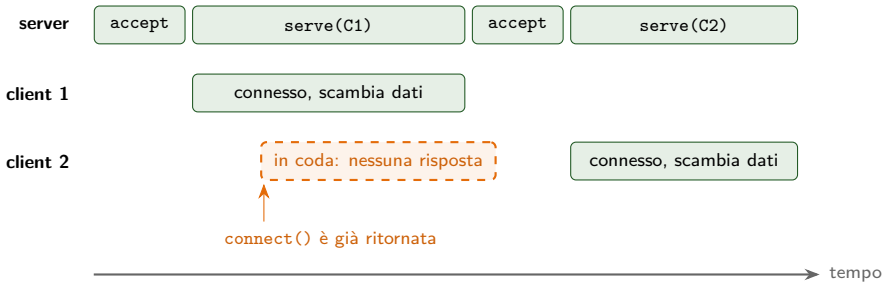


Figura 8.7 – Con un server sequenziale il secondo client resta in coda per tutta la conversazione del primo.

Il server **non è lento**: è occupato dentro `serve`. Un client che non invia mai nulla blocca tutti gli altri. Provando il server di questo capitolo con due `nc`, con il primo tenuto aperto per due secondi, il secondo ottiene l'eco solo dopo circa due secondi e mezzo, quando il primo ha chiuso.

Un server che serve un client alla volta è **corretto** e **inutilizzabile**: da qui partiranno le prossime lezioni.

Il progetto: una chat in nove versioni

Per il resto del corso si scrive **un solo programma**, invece di un esercizio nuovo ogni settimana: un **servizio di chat a righe**.

- Ogni versione mantiene funzionante la precedente e ne rimuove **un** limite.
- Prima la **concorrenza**: processi, thread, un solo processo che osserva molte connessioni insieme, un pool di worker.
- Poi il **protocollo**: delimitazione vera dei messaggi e codici di errore, e una versione su datagrammi che deve sopravvivere alle perdite.
- Infine lo si **osserva dall'esterno**: i pacchetti che genera, e quanto costa sotto carico.

Ogni limite si scopre **eseguendo** la versione precedente, non perché qualcuno lo dice.

Esercizio 1 — Iterazione 1: una chat TCP sequenziale

Scrivere:

- un **server** (`chatd`) che accetta un client alla volta e, per ogni riga che riceve, rimanda indietro la stessa riga preceduta da un contatore;
- un **client** (`chat`) che legge righe dal terminale e stampa ciò che torna indietro.

Le righe sono separate da `\n`, e una riga è completa solo quando lo si è visto: **ciclare sulla read**. Il client si ferma alla fine dell'input e chiude; il server nota lo `0` e torna ad `accept`. I nomi `chatd` e `chat` restano gli stessi per tutte e nove le versioni. Provare su `127.0.0.1`, prima il server con `nc`, poi con il proprio client. Infine aprire un secondo client e rispondere: che cosa vede, e che cosa mostra `ss -tn` mentre aspetta?

Svolgimento

Il punto delicato è proprio la riga: una `read` può restituire mezza riga, o due righe insieme. Il server tiene quindi un buffer con la riga in costruzione, e risponde solo quando incontra `\n`.

```c
static void serve(int fd) {
    char buf[512], line[512];
    size_t len = 0;                    /* byte della riga in costruzione */
    int counter = 0;
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0) {
        for (ssize_t i = 0; i < n; i++) {
            if (len < sizeof line - 1) line[len++] = buf[i];
            if (buf[i] == '\n') {      /* riga completa: si risponde */
                char out[600];
                int k = snprintf(out, sizeof out, "%d: %.*s",
                                 ++counter, (int) len, line);
                if (write_all(fd, out, k) < 0) return;
                len = 0;
            }
        }
    }
    if (n < 0) perror("read");
}
```

chatd.c: la conversazione

Il resto di `chatd.c` è identico al server del capitolo: socket di ascolto, `SO_REUSEADDR`, `bind`, `listen` e il ciclo di `accept`. Il client legge dal terminale con `fgets` e, per la risposta, legge dal socket finché non vede un `\n`:

```c
/* legge finche' non ha visto un '\n'; 0 se il server ha chiuso */
static ssize_t read_line(int fd, char *buf, size_t size) {
    size_t len = 0;
    while (len < size - 1) {
        ssize_t n = read(fd, buf + len, 1);
        if (n <= 0) return n;
        if (buf[len++] == '\n') break;
    }
    buf[len] = '\0';
    return len;
}

/* ... socket() e connect() come nel client del capitolo ... */
while (fgets(line, sizeof line, stdin) != NULL) {   /* fine input: Ctrl-D */
    if (write_all(fd, line, strlen(line)) < 0) { perror("write"); break; }
    if (read_line(fd, reply, sizeof reply) <= 0) break;
    fputs(reply, stdout);
}
close(fd);
```

chat.c: il ciclo principale

```
$ ./chatd &
$ printf 'ciao\ncome va?\n' | ./chat
1: ciao
2: come va?
```

Una prova

Il contatore riparte da 1 a ogni nuovo client, perché è una variabile locale di `serve`. Anche inviando una riga spezzata in due `write` (per esempio `"meta' "` e poi `"riga\n"`), il server risponde una volta sola, con `1: meta' riga`: il buffer ha fatto il suo lavoro.

Il secondo client, mentre il primo è collegato, vede la `connect` riuscire e poi nessuna risposta. In `ss -tn` la sua connessione compare già come `ESTAB`: il kernel l'ha completata e messa in coda, ma nessuno ha ancora chiamato `accept` per lui.

Leggere un byte alla volta

`read_line` legge un byte per chiamata: è semplice e corretto, ma fa una chiamata di sistema per ogni carattere. Va bene per il client di una chat; un server molto carico userebbe invece un buffer, come fa `serve`.

Riepilogo

In breve

- Un **descriptor** è la maniglia del processo su qualcosa che il kernel ha aperto, e un socket è uno di questi.
- `read` e `write` restituiscono **quanto è stato effettivamente trasferito**: su un socket, raramente ciò che si è chiesto. Si cicla; lo `0` di `read` significa che l'altro lato ha chiuso.
- Le chiamate falliscono con -1 e lasciano il motivo in `errno`; `perror` lo stampa.
- Server: `socket`, `bind`, `listen`, `accept`. Client: `socket`, `connect`.
- Il server tiene un **socket di ascolto** che non trasporta mai dati e un **socket di connessione** per ogni client.
- Un flusso trasporta **byte**: i confini dei messaggi non esistono, a meno che il protocollo non li crei con un delimitatore o una lunghezza.
- Un server che serve un client alla volta è corretto e inutilizzabile: è da qui che si parte.

Indirizzi, ordine dei byte e struttura di client e server

Nel capitolo precedente client e server funzionavano, ma tre cose nel loro codice erano state date senza spiegazione:

- `5000`: chi può scegliere una porta, e quali sono libere?
- `htons`: perché un numero va convertito prima di entrare in una struttura?
- `127.0.0.1`: e come si scrive un client per una macchina di cui conosciamo solo il **nome**?

Qui rispondiamo a tutte e tre, e alla fine scriviamo un client che esce dall'aula: una richiesta HTTP composta a mano contro un vero server web.

La forma dei due programmi

Le chiamate sono quelle che già conosciamo; quello che lo schema aggiunge sono i **due cicli** del server.

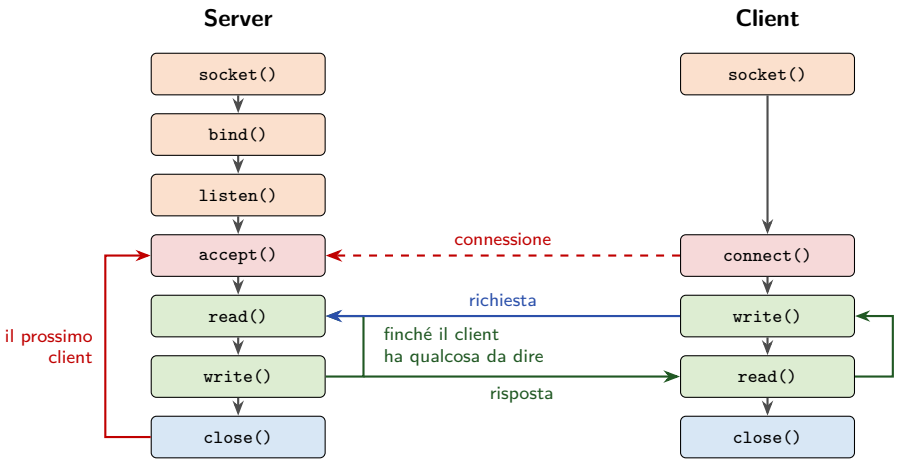


Figura 9.1 – Il ciclo interno è una **conversazione**: leggere e rispondere finché il client ha qualcosa da dire. Quello esterno è la **vita del server**: dopo `close`, di nuovo ad `accept` per il prossimo client.

Tutto quello che manca sta negli **argomenti** di queste chiamate: gli indirizzi.

Un indirizzo è un IP e una porta

Estremo di una comunicazione

Un estremo (*endpoint*) di una comunicazione è identificato da un **indirizzo IP** di 32 bit, che sceglie l'host, e da un **numero di porta** di 16 bit, che sceglie un programma su quell'host.

- L'indirizzo porta i byte fino alla macchina; senza la porta il kernel non saprebbe a chi consegnarli.
- Per questo uno stesso host può far girare insieme un server web, un server di posta e il nostro server di chat.
- Sedici bit vogliono dire da 0 a 65535, e il numero viene scritto nell'header di ogni pacchetto.
- I due numeri insieme sono tutto ciò che prendono `bind` e `connect`: non si negozia nient'altro.

Quali porte si possono usare

Intervallo	Nome	Chi le usa
0–1023	*well known*	server dei servizi standard; solo `root`
1024–49151	*registered*	assegnate su richiesta alle applicazioni
49152–65535	*dynamic*	distribuite dal kernel, mai assegnate

Tabella 9.1 – I tre intervalli dei numeri di porta.

- L'unico confine che conta per noi è **1024**: sotto quel numero il nostro server non parte, a meno di essere `root`.
- Linux distribuisce le sue porte **effimere** (quelle dei client) fra **32768 e 60999**: non è l'intervallo *dynamic* della tabella, ed è un parametro configurabile.
- Porte *well known* già incontrate: 22 `ssh`, 25 `smtp`, 53 `dns`, 80 `http`, 443 `https`.
- Per i nostri server si sceglie un numero sopra 1024, e ci si aspetta l'errore *address already in use* se un vicino ha scelto lo stesso.

Una connessione sono quattro numeri, non due

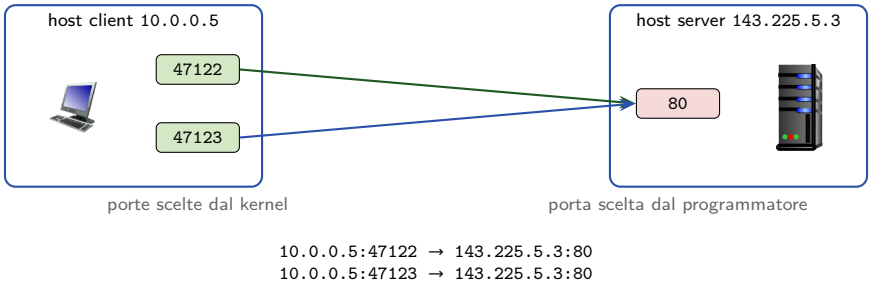


Figura 9.2 – Due connessioni, una sola porta del server: sono i **quattro numeri** insieme a distinguerle.

Il client non sceglie mai la propria porta e non dice mai il proprio indirizzo: li riempie il kernel. Il server li scopre dalla connessione, ed è per questo che `accept` può dirci chi ha chiamato.

```
$ ss -ltn                          # socket TCP in ascolto, in forma numerica
State   Recv-Q  Send-Q   Local Address:Port   Peer Address:Port
LISTEN  0       16       0.0.0.0:5000         0.0.0.0:*

$ ss -tn                           # due client, stessa porta del server
ESTAB   0       0        127.0.0.1:5000       127.0.0.1:47122
ESTAB   0       0        127.0.0.1:5000       127.0.0.1:47123
```

I quattro numeri visti con ss

- `0.0.0.0:5000` è il socket di ascolto: non ha un peer, perché non trasporta mai dati.
- Le due righe `ESTAB` differiscono solo nell'ultimo numero, e basta.
- `netstat -tan` stampa la stessa cosa: è il comando più vecchio che si trova nei libri.

L'ORDINE DEI BYTE

Lo stesso numero, due disposizioni in memoria

L'intero 513 occupa quattro byte, e le due famiglie di processori li scrivono in ordine opposto.

Indirizzo	Big endian	Little endian
0	00000000	00000001
1	00000000	00000010
2	00000010	00000000
3	00000001	00000000

Tabella 9.2 – $513 = 2 \cdot 256 + 1$: nel big endian il byte più significativo viene per primo, nel little endian l'ultimo.

- **Big endian** mette per primo il byte più significativo; **little endian** il meno significativo. Nessuno dei due è sbagliato.
- La nostra macchina è quasi certamente little endian; i protocolli di Internet sono **big endian**.
- Quindi la conversione non è facoltativa, e non è qualcosa che il compilatore possa fare al posto nostro.

L'ordine di rete e quattro funzioni

Network byte order

L'ordine dei byte usato negli header dei protocolli di Internet è il **big endian**, detto per questo *network byte order*. Ogni numero che finisce in un header va scritto in quest'ordine.

```c
#include <arpa/inet.h>

uint16_t htons(uint16_t v);   /* host    -> network, 16 bit: una porta */
uint32_t htonl(uint32_t v);   /* host    -> network, 32 bit: un IPv4   */
uint16_t ntohs(uint16_t v);   /* network -> host,    16 bit            */
uint32_t ntohl(uint32_t v);   /* network -> host,    32 bit            */
```

- I nomi si leggono: `h` *host*, `n` *network*, `s` *short* (16 bit), `l` *long* (32 bit).
- Su una macchina big endian non fanno nulla, ed è proprio questo il punto: lo stesso sorgente si compila e funziona ovunque.
- Si converte quando il valore **entra** nella struttura e si riconverte quando ne **esce**. Mai due volte.

Che cosa costa un htons dimenticato

Il server fa `bind` sulla porta 5000 **senza** `htons`; il client si connette a `htons(5000)`. Non si incontrano: 5000 = `0x1388`, e letto al contrario diventa `0x8813` = 34835. Il server è in ascolto su una porta che nessuno ha chiesto, e il client riceve *connection refused*: l'errore sembra un problema di rete, e non lo è.
Peggio ancora il caso simmetrico: se entrambi i lati dimenticano la conversione, sbagliano allo stesso modo e **tutto funziona**. Continuerà a funzionare finché uno dei due non girerà su una macchina big endian, o finché non si controlla con `ss`.

Quali campi viaggiano davvero

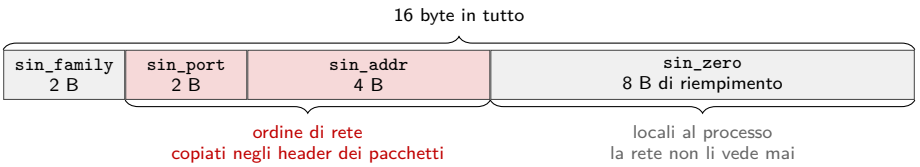


Figura 9.3 – La struttura `sockaddr_in`: si converte ciò che finisce in un header, si lascia stare ciò che il kernel legge localmente.

È tutta la regola: si converte quello che viene **copiato in un header**, si lascia com'è quello che il kernel legge **localmente**. `sin_family` non si converte mai, e `sin_zero` esiste soltanto per arrivare a sedici byte.

Anche i nostri dati

L'ordine dei byte non è una proprietà degli indirizzi: è una proprietà di **ogni numero che mettiamo sul filo**.

```c
uint32_t v = htonl(12345);              /* mittente: converte, poi invia */
write(fd, &v, sizeof v);

uint32_t net;                           /* destinatario: tutti e quattro i byte */
size_t got = 0;
while (got < sizeof net) {
    ssize_t n = read(fd, (char *) &net + got, sizeof net - got);
    if (n <= 0) { /* il peer se n'e' andato, o errore */ break; }
    got += (size_t) n;
}
uint32_t back = ntohl(net);
```

Inviare e ricevere un intero di 32 bit

Il ciclo serve perché quattro byte possono arrivare in due pezzi: è la *short read* del capitolo precedente.

RIEMPIRE GLI INDIRIZZI

Da una stringa puntata a quattro byte

```c
struct sockaddr_in a;
char txt[INET_ADDRSTRLEN];

if (inet_pton(AF_INET, "143.225.5.3", &a.sin_addr) != 1) {     /* 1 = ok */
    fprintf(stderr, "indirizzo non valido\n"); exit(EXIT_FAILURE);
}
inet_ntop(AF_INET, &a.sin_addr, txt, sizeof txt);               /* e ritorno */
```

- `inet_pton` scrive direttamente in `sin_addr`, già in ordine di rete: niente `htonl` dopo.
- Restituisce 1 se va tutto bene, 0 se la stringa è malformata, -1 se la famiglia non è supportata.
- `inet_ntop` fa il percorso inverso e vuole un buffer da noi. Le vecchie `inet_addr` e `inet_ntoa` sono da evitare.

L'indirizzo del server e quello del client

```
struct sockaddr_in me;                      /* server: il proprio indirizzo */
memset(&me, 0, sizeof me);
me.sin_family      = AF_INET;
me.sin_port        = htons(5000);           /* scelta da noi */
me.sin_addr.s_addr = htonl(INADDR_ANY);     /* qualunque interfaccia locale */
bind(fd, (struct sockaddr *) &me, sizeof me);

struct sockaddr_in srv;                     /* client: l'altro */
memset(&srv, 0, sizeof srv);
srv.sin_family = AF_INET;
srv.sin_port   = htons(5000);               /* deve coincidere con quella del server */
inet_pton(AF_INET, "143.225.5.3", &srv.sin_addr);
connect(fd, (struct sockaddr *) &srv, sizeof srv);   /* nessuna bind */
```

La stessa struttura, due significati

Stessa struttura, significato opposto: **`bind` dice dove sono io, `connect` dice dove sei tu**.

INADDR_ANY o una sola interfaccia?

Il nostro server funziona con `nc` sulla nostra macchina ed è irraggiungibile dal portatile accanto. Perché? Perché è stato legato a `127.0.0.1`. Quell'indirizzo esiste su ogni macchina e non esce da nessuna: i pacchetti che arrivano da fuori entrano da un'altra interfaccia, e lì nessuno è in ascolto.

- Un host ha più indirizzi (loopback, cavo, wireless), e `bind` ci lega a **uno** di essi.
- `INADDR_ANY`, stampato come `0.0.0.0`, significa **tutte** le interfacce di questa macchina, presenti e future.
- È la scelta normale per un server; l'eccezione voluta è un servizio che non deve essere raggiungibile da fuori.

Riavviare il server: le opzioni dei socket

```
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);

struct timeval tv = { .tv_sec = 5, .tv_usec = 0 };
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
```

- Le opzioni stanno su **livelli**: `SOL_SOCKET` qui, `IPPROTO_TCP` e `IPPROTO_IP` più in basso.
- `SO_REUSEADDR` permette a `bind` di prendere una porta che l'esecuzione precedente ha chiuso ma che il kernel sta ancora tenendo occupata. Quell'attesa è il rilascio ordinato della vecchia connessione: vedremo che cosa protegge quando apriremo TCP.
- `SO_RCVTIMEO` fa sì che una `read` bloccante rinunci dopo cinque secondi, restituendo -1 con `errno` impostato.

Lo stesso codice su IPv6

```
struct sockaddr_in6 me6;                    /* dentro, un indirizzo di 16 byte */
memset(&me6, 0, sizeof me6);
me6.sin6_family = AF_INET6;
me6.sin6_port   = htons(5000);              /* htons, esattamente come prima */
me6.sin6_addr   = in6addr_any;              /* il jolly, scritto :: */
bind(fd6, (struct sockaddr *) &me6, sizeof me6);
```

Struttura diversa, famiglia diversa, **stesse sei chiamate**: l'API è stata progettata perché cambi solo questo. Il socket va creato con `AF_INET6`, e il cast a `struct sockaddr *` è il motivo per cui quel tipo generico esiste. Scrivere a mano entrambe le versioni è noioso ed evitabile: è l'argomento della prossima sezione.

Dal nome all'indirizzo

Tutto quello visto finora voleva **numeri**, e su Internet gli host si conoscono per **nome**.

- L'API dei socket non può aiutarci: sta al livello di trasporto, e i nomi sono un servizio del livello applicativo.
- Quindi, prima di connettersi, si fa un altro scambio sulla rete (una **query DNS**) e se ne usa la risposta come indirizzo.
- Il nostro client è dunque **client due volte**: prima del resolver, poi del server che volevamo davvero.

getaddrinfo: da un nome a indirizzi pronti

```c
struct addrinfo hints, *res;
memset(&hints, 0, sizeof hints);
hints.ai_family   = AF_UNSPEC;       /* IPv4 o IPv6, quello che c'e' */
hints.ai_socktype = SOCK_STREAM;     /* TCP: niente risposte per datagrammi */

int rc = getaddrinfo("www.unina.it", "80", &hints, &res);
if (rc != 0) {
    fprintf(stderr, "%s\n", gai_strerror(rc)); exit(EXIT_FAILURE);
}
```

Risolvere nome e servizio insieme

- Risolve insieme il **nome** e il **servizio**: `"80"` si potrebbe scrivere `"http"`.
- Restituisce 0 oppure un **codice d'errore suo**, che si stampa con `gai_strerror`: non `errno`, non `perror`.
- Per un server si passa `NULL` come nome e `hints.ai_flags = AI_PASSIVE`, e si ottiene il jolly.
- Il codice più vecchio chiama `gethostbyname`: solo IPv4, non rientrante, fuori dallo standard. Bisogna saperla riconoscere, non scriverla.

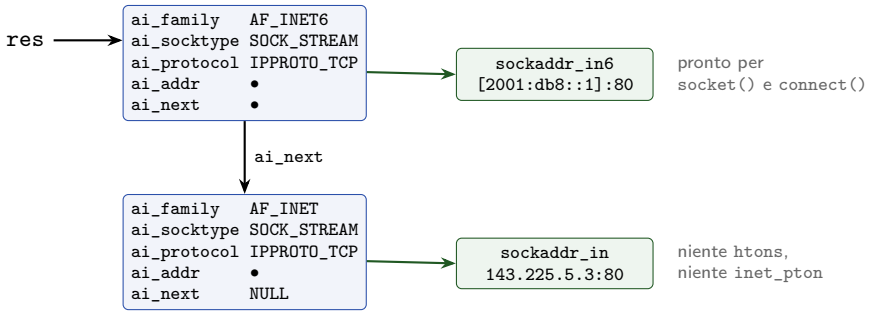


Figura 9.4 – `getaddrinfo` restituisce una **lista**, un nodo per ogni modo di raggiungere quel nome. I primi tre campi sono i tre argomenti di `socket`, e `ai_addr` va bene così com'è per `connect`.

Provare la lista finché uno funziona

```c
int fd = -1;
for (struct addrinfo *p = res; p != NULL; p = p->ai_next) {
    fd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
    if (fd < 0) continue;                                  /* famiglia non disponibile */
    if (connect(fd, p->ai_addr, p->ai_addrlen) == 0) break;  /* fatto */
    close(fd);
    fd = -1;
}
freeaddrinfo(res);                                         /* la lista e' allocata con malloc */
if (fd < 0) { fprintf(stderr, "impossibile connettersi\n"); exit(EXIT_FAILURE); }
```

connect_any: il ciclo da scrivere una volta e tenere

Questo ciclo è uguale in ogni client, ed è ciò che fa funzionare un programma su IPv4 e su IPv6 **senza sapere quale dei due ha ottenuto**.

Un nome, più indirizzi

Il nostro client funziona per una settimana e poi smette, senza che sia cambiata una riga. Che cosa è cambiato? Il nome si è risolto in un **indirizzo diverso**. Un sito molto frequentato risponde con più indirizzi, in un ordine che cambia a ogni query, e quello che avevamo memorizzato è ora una macchina in sostituzione. È il bilanciamento del carico visto nel capitolo sul DNS. Quindi: si risolve **al momento di connettersi**, non una volta all'avvio, e non si dà per scontato che il primo indirizzo risponda. Il ciclo qui sopra è esattamente questa disciplina: provare, fallire, passare al successivo.

Le stesse ricerche dalla shell

```
$ getent hosts www.unina.it      # cio' che restituirebbe getaddrinfo
143.225.5.3     www.unina.it
$ dig +short www.unina.it        # chiede direttamente al DNS
143.225.5.3
$ ip addr show                   # quali indirizzi ha questo host
```

- `getent` passa per lo stesso resolver usato dal nostro programma, quindi vede anche `/etc/hosts`.
- `dig` parla con il name server: per questo i due possono essere in disaccordo, ed è una differenza che prima o poi ci farà perdere tempo.
- Il vecchio nome di `ip` è `ifconfig`: lo si trova nei libri, e molti sistemi non lo installano più.

HTTP SCRITTO A MANO

```
$ nc -C www.unina.it 80
HEAD /chi-siamo/cenni-storici HTTP/1.1          <- scritto da noi
Host: www.unina.it
Connection: close
                                                <- riga vuota: richiesta finita
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Connection: close
```

Una richiesta HEAD digitata con nc

- `nc` apre un socket e vi copia il terminale: è un client generico per qualunque protocollo testuale (`-C` manda le righe terminate da `\r\n`).
- Il server non lo distingue da un browser, perché sul filo non c'è nient'altro da distinguere.
- `telnet host porta` fa la stessa cosa ed è la versione che si trova nel materiale più vecchio.

Ora scriviamo la stessa richiesta in un programma. Servono esattamente tre cose, e le abbiamo appena digitate tutte e tre: il **nome dell'host**, che diventa un indirizzo; la **porta**, 80 per un server web; il **percorso** dell'oggetto voluto. Nient'altro: il browser aggiunge comodità, non protocollo.

```c
struct addrinfo hints, *res;
memset(&hints, 0, sizeof hints);
hints.ai_family   = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;

int rc = getaddrinfo("www.unina.it", "http", &hints, &res);
if (rc != 0) { fprintf(stderr, "%s\n", gai_strerror(rc)); exit(EXIT_FAILURE); }

int fd = connect_any(res);       /* il ciclo della sezione precedente */
freeaddrinfo(res);
if (fd < 0) { fprintf(stderr, "impossibile connettersi\n"); exit(EXIT_FAILURE); }

const char *req = "HEAD /chi-siamo/cenni-storici HTTP/1.1\r\n"
                  "Host: www.unina.it\r\n"
                  "Connection: close\r\n\r\n";      /* la riga vuota la chiude */

if (write(fd, req, strlen(req)) < 0) { perror("write"); exit(EXIT_FAILURE); }

char buf[4096];
ssize_t n;
while ((n = read(fd, buf, sizeof buf)) > 0)         /* finche' il server chiude */
    fwrite(buf, 1, (size_t) n, stdout);
close(fd);
```

head.c: risolvere, connettersi, chiedere, leggere

Nessun indirizzo copiato a mano e nessun byte convertito: tutta la prima metà del capitolo ora sta dentro `getaddrinfo`. `HEAD` chiede soltanto gli header; ogni riga finisce con `\r\n`, e la riga vuota dice al server che la richiesta è completa.

Quello che abbiamo scritto è un payload

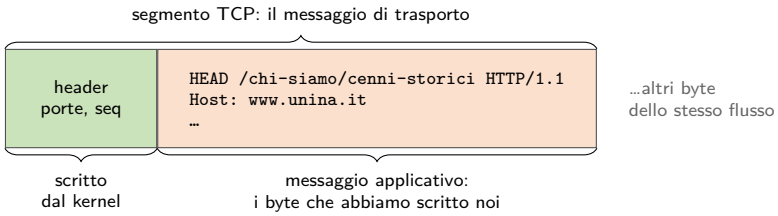


Figura 9.5 – Il livello di trasporto trasporta quei byte **senza leggerli**: per lui il payload è trasparente.

Abbiamo costruito un messaggio applicativo con `strlen` e `write`, e il livello di trasporto lo ha portato senza guardarci dentro. Scrivere la richiesta a mano è ciò che rende concreta la parola *payload* incontrata parlando di incapsulamento.

Una read non è la risposta

Togliendo il ciclo e stampando ciò che restituisce una sola `read`, si ottiene un **prefisso** della risposta, di lunghezza imprevedibile: quello che era arrivato fino a quel momento. La risposta è un flusso, e la sua fine non è un evento di cui il programma viene avvisato.

- Il ciclo sopra si ferma sullo 0, e solo perché `Connection: close` fa chiudere il server alla fine.
- Senza quell'header la connessione resta aperta, il ciclo non finisce, e il programma resta appeso dopo aver stampato tutto.
- Per questo HTTP dice quanto è lungo il corpo: `Content-Length`, oppure *chunk* che annunciano ciascuno la propria dimensione.
- Ogni protocollo deve rispondere a questa domanda, e anche il nostro dovrà farlo.

Esercizio 1 — Una somma sulla rete

Modificare client e server del capitolo precedente perché il client invii **due numeri** e il server risponda con la loro **somma**, in un ciclo finché non lo si interrompe. I numeri viaggiano in binario, come interi di 32 bit.

Svolgimento

Ogni numero va convertito con `htonl` prima di partire e con `ntohl` all'arrivo, e va letto per intero: la funzione `read_full` è il ciclo di questo capitolo, che si ferma solo con tutti i byte o con la chiusura del peer.

```c
static int read_full(int fd, void *buf, size_t n) {   /* 1 = ok, 0 = peer chiuso */
    size_t got = 0;
    while (got < n) {
        ssize_t r = read(fd, (char *) buf + got, n - got);
        if (r <= 0) return 0;
        got += (size_t) r;
    }
    return 1;
}

static void serve(int fd) {
    uint32_t in[2];
    while (read_full(fd, in, sizeof in)) {            /* 8 byte: due numeri */
        uint32_t sum = htonl(ntohl(in[0]) + ntohl(in[1]));
        if (write(fd, &sum, sizeof sum) != sizeof sum) break;
    }
}
```

Lato server: la conversazione

Il client fa l'inverso: legge due numeri da tastiera, scrive `htonl(a)` e `htonl(b)` con una sola `write` di 8 byte, legge 4 byte con `read_full` e stampa `ntohl` del risultato. Dimenticando le conversioni su entrambi i lati il programma funziona lo stesso fra due macchine little endian: è il caso simmetrico dell'avvertimento sull'`htons`.

Esercizio 2 — Progetto: la chat sa dove connettersi

La chat dell'iterazione 1 ha `127.0.0.1` e `5000` scritti nel sorgente. Fare in modo che il client prenda **host e porta** da `argv` e risolva l'host con `getaddrinfo`, e che il server prenda la propria porta allo stesso modo. Provare con il server su una macchina e il client su un'altra. Con le due macchine collegate, che aspetto hanno i quattro numeri in `ss -tn` da ciascun lato?

Svolgimento

Il client passa `argv[1]` e `argv[2]` a `getaddrinfo` e usa `connect_any`; il server chiede l'indirizzo passivo, così resta su `INADDR_ANY` e risponde sull'interfaccia che l'altra macchina riesce a raggiungere.

```c
int listening_socket(const char *port) {
    struct addrinfo hints = {0}, *res;
    hints.ai_family   = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags    = AI_PASSIVE;                /* NULL + AI_PASSIVE = il jolly */
    int rc = getaddrinfo(NULL, port, &hints, &res);
    if (rc != 0) { fprintf(stderr, "%s\n", gai_strerror(rc)); exit(EXIT_FAILURE); }

    int fd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
    int yes = 1;
    setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
    if (bind(fd, res->ai_addr, res->ai_addrlen) < 0) { perror("bind"); exit(EXIT_FAILURE); }
    freeaddrinfo(res);
    if (listen(fd, 16) < 0) { perror("listen"); exit(EXIT_FAILURE); }
    return fd;
}
```

chatd.c: il socket di ascolto su una porta data

Da qui in avanti ogni versione del progetto si avvia così: `./chatd 5000` e `./chat host 5000`. In `ss -tn` le due righe sono **speculari**: sul server l'indirizzo locale è quello della sua scheda con la porta 5000 e il peer è il client con una porta effimera; sul client gli stessi quattro numeri compaiono con locale e peer scambiati.

RIEPILOGO

In breve

- Un estremo è un **indirizzo e una porta**; una connessione sono **quattro numeri**, ed è così che due client restano distinti.
- Le porte sotto 1024 non sono nostre; quella del client la sceglie il kernel da un intervallo effimero.
- I numeri che finiscono negli header vanno in **ordine di rete**: `htons` per le porte, `htonl` per gli indirizzi, e lo stesso per i nostri dati.
- `bind` dice dove sono io, `connect` dove sei tu; `INADDR_ANY` significa tutte le interfacce dell'host.
- `getaddrinfo` trasforma un nome e un servizio in una **lista** di indirizzi pronti, IPv4 e IPv6 allo stesso modo; si prova la lista finché un indirizzo risponde.
- Un messaggio applicativo viaggia come **payload** di un messaggio di trasporto, e nessuno strato inferiore lo legge.

Processi: fork, ereditarietà dei descriptor e segnali

Il server della chat è **corretto e inutilizzabile**: mentre è dentro `serve`, nessun altro viene letto.

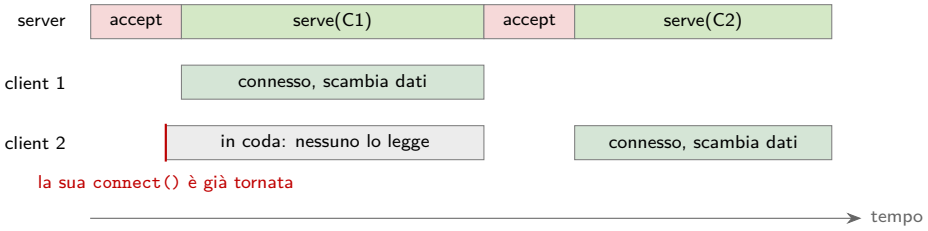


Figura 10.1 – Con un server sequenziale il secondo client è connesso ma nessuno lo legge, finché il primo non se ne va.

La via d'uscita è far eseguire `serve` da **qualcun altro**, un secondo processo, mentre il server torna ad `accept`. Questo capitolo presenta lo strumento (`fork`), ciò che il figlio eredita (soprattutto i descriptor), come si raccolgono i figli morti, e il meccanismo con cui il kernel interrompe un processo per dirgli che è successo qualcosa: i **segnali**.

Un processo, un PID, un genitore

Processo

Un **processo** è un programma in esecuzione: ha la propria memoria, la propria tabella dei descriptor e il proprio posto nelle code dello scheduler.

- Ogni processo ha un **pid**, un piccolo intero non negativo, e un **ppid**, il pid di chi l'ha creato.
- `getpid()` e `getppid()` li restituiscono e non possono fallire; entrambi hanno tipo `pid_t`.
- La relazione di parentela fa dei processi di una macchina un **albero**, la cui radice è il primo processo avviato dal kernel, il pid 1.
- Un pid è unico finché il processo vive, e viene **riusato** dopo: identifica un processo, non un programma.

FORK: UNA CHIAMATA, DUE RITORNI

Un processo la chiama, **due processi** ne ritornano.

```c
#include <unistd.h>

pid_t fork(void);

pid_t pid = fork();
if (pid < 0) { perror("fork"); exit(EXIT_FAILURE); }
if (pid == 0) {
    /* figlio: qui fork ha restituito 0 */
} else {
    /* genitore: qui fork ha restituito il pid del figlio */
}
```

La forma di ogni fork

Il figlio non riceve da `fork` il proprio pid: se lo chiede con `getpid`. Il genitore sì, perché gli servirà per aspettarlo.

Che cosa eredita il figlio

Il figlio nasce come un **duplicato** del genitore, nello stesso punto dello stesso codice.

- **Memoria**: una copia privata di tutto (globali, locali, heap). Il kernel la copia in modo pigro (*copy-on-write*), ma nessuno dei due può accorgersene.
- **Descriptor**: ogni descriptor aperto viene duplicato, e le due copie si riferiscono allo **stesso file aperto**. È la cosa importante.
- **Segnali**: il figlio mantiene gli handler e la maschera del genitore, finché non installa i propri o non sostituisce il programma.
- **Non ereditati**: il pid, il ppid, e gli eventuali figli che il genitore stava aspettando: quelli sono del genitore, non della famiglia.

```c
int glob = 1;                          /* una copia ora, due dopo la fork */

int main(void) {
    int local = 10;
    pid_t pid = fork();
    if (pid == 0) { glob++; local++; }  /* il figlio, e solo lui */
    else            sleep(1);           /* lascia stampare prima il figlio */
    printf("pid=%d glob=%d local=%d\n", getpid(), glob, local);
    return 0;
}
```

fork1.c: due copie delle stesse variabili

```
pid=7183 glob=2 local=11        <- figlio
pid=7182 glob=1 local=10        <- genitore
```

Nulla dice chi dei due va per primo: qui decide la `sleep`, e una `sleep` **non** è un meccanismo di sincronizzazione.

La riga stampata due volte

Se il programma stampa una riga terminata da `\n` **prima** della `fork`, quante volte compare? Sul terminale una; rediretto su un file **due**, senza che un carattere del programma sia cambiato.

```
$ ./fork1 > out.txt ; cat out.txt
before the fork
pid=7189 glob=2 local=11
before the fork
pid=7188 glob=1 local=10
```

Verso un terminale `stdout` viene svuotato a ogni a-capo; verso un file solo quando il buffer si riempie o quando il programma termina. La riga è quindi ancora nel buffer quando `fork` copia la memoria, buffer compreso, e entrambi i processi la scrivono più tardi. Un `fflush(stdout)` prima della `fork` l'avrebbe evitato.

I DESCRIPTOR DOPO LA FORK

Per capire che cosa viene condiviso bisogna ricordare che i livelli sono **tre**: una **tabella dei descriptor** per processo, una **voce di file aperto** per ogni apertura (lì stanno i flag e l'offset, non nel processo), e un oggetto per ogni file. La `fork` tocca soltanto il primo livello.

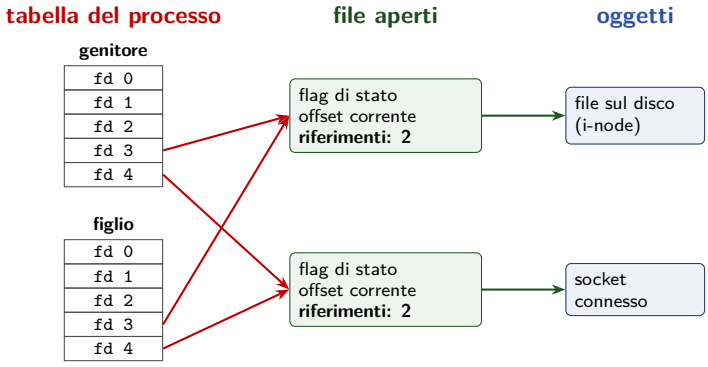


Figura 10.2 – Dopo la `fork`: la tabella è copiata, le voci no. Il `fd 3` del genitore e il `fd 3` del figlio sono due frecce verso **un solo** file aperto, con un solo offset e un contatore di riferimenti che ora vale due.

- Due processi **non imparentati** che aprono lo stesso file ottengono invece **una voce ciascuno**: due offset, e ognuno sovrascrive l'altro.
- La condivisione quindi non nasce dal fatto che l'oggetto è lo stesso: nasce dal fatto che il descriptor è stato **ereditato**.
- `dup` crea un secondo descriptor sulla stessa voce **dentro un solo processo**: la stessa duplicazione, senza un secondo processo.

Anche il socket connesso viene ereditato

Un socket è uno di quegli oggetti, quindi qui non c'è nulla di nuovo, se non quanto costa.

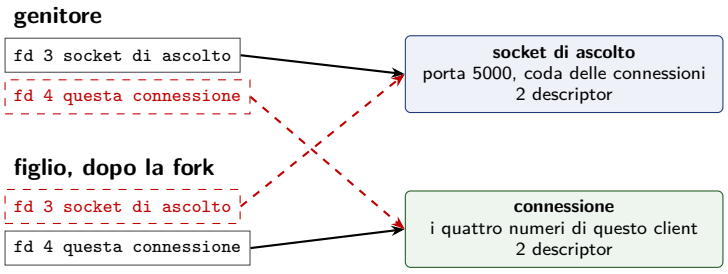


Figura 10.3 – In rosso tratteggiato i descriptor che ciascun processo deve chiudere. Ogni oggetto resta vivo finché non se ne va il suo ultimo descriptor: né il figlio né il genitore, da soli, possono chiudere la connessione.

Chi chiude che cosa

```c
for (;;) {
    int fd = accept(listen_fd, NULL, NULL);
    if (fd < 0) { perror("accept"); continue; }

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); close(fd); continue; }

    if (pid == 0) {                  /* il figlio serve questo client */
        close(listen_fd);            /* non accettera' mai nulla */
        serve(fd);
        close(fd);
        _exit(0);                    /* non exit(): i buffer non sono suoi */
    }
    close(fd);                       /* il genitore non parla su questa connessione */
}
```

Il ciclo di accept che fa fork

Due `close` che sembrano ridondanti e non lo sono: ciascun lato rinuncia al descriptor che non usa.

Il genitore che dimentica di chiudere

Il figlio ha finito ed è uscito. Perché il client sta ancora aspettando? Perché la connessione **non è chiusa**: il genitore ha ancora un descriptor su di essa, quindi il contatore non arriva mai a zero e nessun FIN viene inviato.

- La `read` del client non restituisce 0: resta bloccata su una connessione su cui nessuno scriverà più.
- Intanto il genitore accumula un descriptor per ogni client servito, e prima o poi `accept` fallisce con `EMFILE`.
- Il caso speculare è altrettanto grave: un figlio che tiene `listen_fd` tiene occupata la porta anche dopo che il server se n'è andato.

LA VITA DI UN PROCESSO

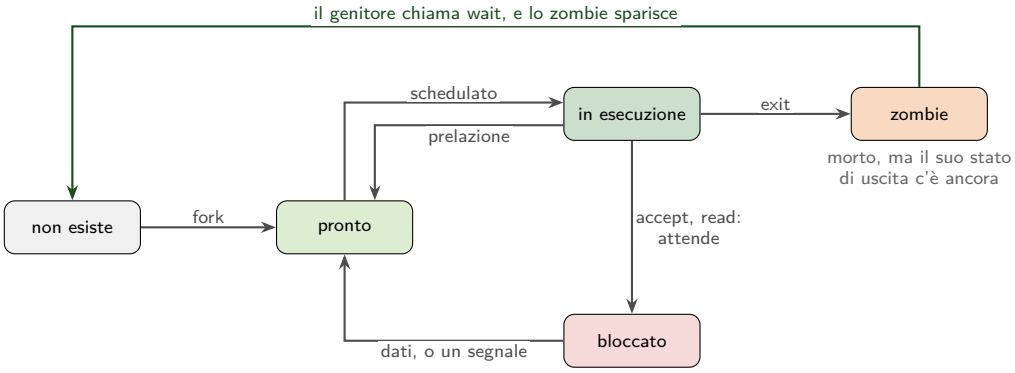


Figura 10.4 – Un server passa quasi tutto il tempo **bloccato**. Il passo interessante è l'ultimo: un figlio morto non sparisce finché qualcuno non chiede come è morto.

Tre modi di finire, e lo stato

- **return da main**: esattamente come chiamare `exit` con quel valore.
- **`exit(n)`**: esegue gli handler registrati con `atexit`, svuota gli stream standard, poi chiama `_exit`.
- **`_exit(n)`**: dritto al kernel, senza svuotare nulla. Per questo lo usa un figlio creato con `fork`: i buffer che svuoterebbe sono una copia di quelli del genitore.
- **In modo anomalo**: un segnale non gestito, oppure `abort`, che ne invia uno a se stesso.

In qualunque modo finisca, il kernel chiude i descriptor e libera la memoria, e conserva per il genitore uno **stato di terminazione**.

wait e waitpid

```c
#include <sys/wait.h>

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);

int status;
wait(&status);                      /* un figlio qualsiasi; blocca finche' uno termina */
waitpid(pid, &status, 0);           /* quel figlio, e nessun altro */
waitpid(-1, &status, WNOHANG);      /* un figlio qualsiasi, senza mai bloccare */
```

- Entrambe restituiscono il pid del figlio raccolto, oppure -1 con `ECHILD` se non ci sono più figli da aspettare.
- Con `WNOHANG` il valore è 0 quando esistono figli ma nessuno è terminato: niente da raccogliere, si va avanti.

Domanda	Poi
`WIFEXITED(status)`	`WEXITSTATUS` dà il valore passato a `exit`
`WIFSIGNALED(status)`	`WTERMSIG` dà il segnale che l'ha ucciso
`WIFSTOPPED(status)`	`WSTOPSIG` dà il segnale che l'ha fermato

Tabella 10.1 – Leggere lo stato di terminazione.

- `status` non è un numero da stampare: è una parola che il kernel impacchetta, e solo queste macro la leggono.
- Un figlio che ha chiamato `exit(7)` lascia uno `status` che **non** vale 7: solo `WEXITSTATUS` dice 7.
- Del valore sopravvivono solo gli 8 bit bassi, quindi i codici di uscita stanno fra 0 e 255.

Gli zombie

Un processo terminato ma non ancora aspettato viene tenuto dal kernel: la sua memoria non c'è più, il suo stato sì.

```
$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 4821  3310 S+   ./server
 4822  4821 Z+   [server] <defunct>
```

- Costa un posto nella tabella dei processi, e un server che fa una `fork` per client esaurisce i posti in un pomeriggio.
- Non lo si può uccidere: è già morto. Sparisce quando il genitore lo aspetta, o quando muore il genitore.
- Un **orfano** è il caso opposto: il genitore è morto per primo, il pid 1 lo ha adottato, e il pid 1 aspetta sempre.

Un server non può stare in wait()

Mettendo `wait(NULL)` dopo la `fork`, nel ciclo di `accept`, che cosa si è costruito? Di nuovo il server sequenziale, con un processo in più per client: il genitore torna ad `accept` solo quando il figlio ha finito di servire.

- Il genitore deve tornare ad `accept` **subito**, e comunque raccogliere figli terminati in momenti imprevedibili.
- Interrogare con `WNOHANG` a ogni giro funziona, ma si sveglia solo quando per caso si connette un nuovo client.
- Manca un modo con cui il kernel possa dire **adesso**, nel mezzo di qualunque cosa il processo stia facendo.

Segnale

Un **segnale** interrompe un processo per dirgli che è successo qualcosa. Porta un numero e nient'altro: niente dati, niente mittente, niente coda.

- **Dal terminale**: `Ctrl-C` invia `SIGINT` a tutti i processi del gruppo in primo piano, figli compresi.
- **Dal kernel**: un riferimento di memoria non valido, una divisione per zero, un figlio terminato, una scrittura su una connessione chiusa.
- **Da un altro processo**: `kill`, che invia un segnale ed è un nome infelice, visto che la maggior parte dei segnali non uccide nulla.

Arriva **fra due istruzioni** del programma, e non siamo noi a scegliere quali.

Segnale	Inviato quando	Default
`SIGCHLD`	un figlio è terminato o si è fermato	ignorato
`SIGPIPE`	si scrive su una connessione che il peer ha chiuso	termina
`SIGINT`	`Ctrl-C` sul terminale	termina
`SIGTERM`	`kill` senza indicare un segnale	termina
`SIGQUIT`	`Ctrl-\` sul terminale	termina, con core
`SIGKILL`	qualcuno lo vuole fuori, subito	termina

Tabella 10.2 – I segnali che un server incontra.

Due di questi sono l'intera ragione per cui un server con `fork` ha bisogno dei segnali: `SIGCHLD`, che per default è ignorato e ci lascia gli zombie, e `SIGPIPE`, che uccide il server la prima volta che un client chiude a metà di una risposta. Su quest'ultimo torneremo parlando di robustezza.

Ignorare, catturare, o farsi uccidere

Per ogni segnale un processo ha una **disposizione**, e la disposizione viene ereditata attraverso la `fork`.

- **Default**: quello che dice il segnale; per la maggior parte, terminare il processo, a volte con un file core.
- **Ignorare** (`SIG_IGN`): il segnale viene consegnato e buttato via. Per `SIGCHLD` questo ha un significato speciale, come vedremo.
- **Catturare**: viene eseguita una funzione nostra, l'*handler*, e poi il programma riprende dov'era stato interrotto.
- **Nessuna delle due**: `SIGKILL` e `SIGSTOP` non si possono né catturare né ignorare: esistono perché un processo si possa sempre fermare.

```c
#include <signal.h>

static void on_child(int sig) { /* ... */ }

struct sigaction sa;
sa.sa_handler = on_child;        /* oppure SIG_IGN, oppure SIG_DFL */
sigemptyset(&sa.sa_mask);        /* nient'altro bloccato mentre gira */
sa.sa_flags   = 0;
if (sigaction(SIGCHLD, &sa, NULL) < 0) { perror("sigaction"); }
```

Installare un handler con sigaction

- L'handler riceve il numero del segnale, quindi una funzione può servire più segnali; va installata una volta per ciascuno.
- Il codice vecchio scrive `signal(SIGCHLD, on_child)`: è più corto, ma il suo comportamento esatto cambia da sistema a sistema. Si usa `sigaction`. Solo per **ignorare** un segnale `signal(sig, SIG_IGN)` va ancora bene: lo incontreremo per `SIGPIPE`.

Che cosa può fare un handler

L'handler gira **dentro** il programma, fra due delle sue istruzioni, e magari dentro una funzione di libreria a metà del proprio lavoro.

- Può chiamare soltanto funzioni **async-signal-safe**: `write` sì, `printf` e `malloc` no, per quanto spesso le si veda lì.
- Deve salvare e ripristinare `errno`: il codice interrotto stava probabilmente per leggerlo.
- L'unica variabile che dovrebbe condividere con il programma è una `volatile sig_atomic_t`, letta nel ciclo principale.
- Va tenuto corto: il suo compito è **registrare** che qualcosa è successo, non gestirlo.

Raccogliere i figli con SIGCHLD

```c
static void reap(int sig) {
    int saved = errno;                          /* l'handler deve restituirlo */
    while (waitpid(-1, NULL, WNOHANG) > 0)
        ;                                       /* ogni figlio che e' terminato */
    errno = saved;
}
```

L'handler che raccoglie i figli

- Installato una volta prima del ciclo di `accept`, raccoglie i figli nel momento in cui muoiono, e il genitore non si blocca mai.
- Alternativa più semplice: impostare `SIGCHLD` a `SIG_IGN`, e il sistema non crea affatto zombie, al prezzo di non sapere mai come è terminato un figlio.
- Mentre l'handler gira, l'`accept` in cui il genitore era bloccato restituisce -1 con `EINTR`. La gestione di questo caso fa parte del rendere robusto il server.

Tre figli muoiono nello stesso istante

Tre client si disconnettono insieme: quante volte gira l'handler? Una, forse due; mai tre su cui si possa contare. I segnali standard **non sono accodati**: «in attesa» è un flag, non un numero.

- Mentre l'handler gira gli altri `SIGCHLD` sono bloccati, e si fondono nell'unico che resta in attesa.
- Un handler che chiama `waitpid` una volta sola lascia quindi zombie dietro di sé, e la perdita è abbastanza lenta da sopravvivere ai test.
- Da qui il ciclo: si raccoglie finché `waitpid` dice che non c'è altro, non finché se n'è raccolto uno.

Inviare segnali: kill

Un segnale si invia per numero, dalla shell o da un programma, a un processo o a un intero gruppo.

```
$ kill -TERM 4821       # stessa cosa di: kill 4821
$ kill -INT 4821
$ kill -l               # nomi e numeri
```

```c
int kill(pid_t pid, int sig);

kill(pid, SIGTERM);     /* un processo */
kill(0,   SIGTERM);     /* tutto il mio gruppo */
kill(pid, 0);           /* non invia nulla: esiste? */
```

Si possono segnalare solo i propri processi. E poiché i pid vengono riusati, `kill(pid, 0)` risponde «esiste un processo con quel pid», che non è esattamente la domanda che ci si era posti.

```c
static volatile sig_atomic_t running = 1;          /* l'unica variabile condivisa */

static void stop(int sig) { running = 0; }         /* registra, non agisce */

while (running) {
    int fd = accept(listen_fd, NULL, NULL);
    if (fd < 0) {
        if (errno == EINTR) continue;               /* e' arrivato un segnale: si ricontrolla */
        perror("accept"); break;
    }
    /* ... fork, serve, close ... */
}
close(listen_fd);                                   /* salutare come si deve */
```

Un arresto ordinato

Senza questo il server muore dentro `accept` e lascia i figli in esecuzione, con le connessioni ancora aperte.

EXEC: IL PROCESSO DIVENTA UN ALTRO PROGRAMMA

- `exec` non crea nulla: **sostituisce** il programma in esecuzione nel processo che la chiama.
- Testo, dati, heap e stack vengono buttati e ricostruiti dall'eseguibile; l'esecuzione parte dal suo `main`.
- Il pid, il ppid e i descriptor aperti **restano**: è lo stesso processo, che indossa un altro programma.
- Se riesce **non ritorna**. Il codice dopo la chiamata viene eseguito solo se la chiamata è fallita.

Funzione	Programma	Argomenti	Ambiente
`execl`	percorso	lista	ereditato
`execlp`	nome, cercato nel `PATH`	lista	ereditato
`execle`	percorso	lista	dato
`execv`	percorso	vettore	ereditato
`execvp`	nome, cercato nel `PATH`	vettore	ereditato
`execve`	percorso	vettore	dato

Tabella 10.3 – Sei nomi per una sola chiamata di sistema (`execve`): `p` cerca nel `PATH`, `l` argomenti come lista, `v` come vettore, `e` ambiente passato esplicitamente.

```c
pid_t pid = fork();
if (pid < 0) { perror("fork"); return -1; }

if (pid == 0) {                         /* il figlio, e solo per un istante */
    execlp("gzip", "gzip", "-c", path, (char *) NULL);
    perror("execlp");                   /* raggiunto solo se exec e' fallita */
    _exit(127);
}

int status;
if (waitpid(pid, &status, 0) < 0) perror("waitpid");
if (WIFEXITED(status) && WEXITSTATUS(status) == 0) { /* ha funzionato */ }
```

fork, exec, wait

Il nome del programma compare **due volte**: una per trovare il file, una perché diventa l'`argv[0]` del nuovo programma. Con il vettore: `char *argv[] = {"gzip", "-c", "log", NULL}; execvp("gzip", argv);`.

I descriptor sopravvivono a exec

Il server esegue un programma esterno: che cosa riceve quel programma? **Ogni** descriptor che il server aveva aperto, socket di ascolto compreso, e può tenere occupata la porta molto dopo che il server se n'è andato. Questa ereditarietà è ciò che rende possibili le redirezioni, quindi è una funzionalità; diventa un bug solo quando la si dimentica. I descriptor che non devono viaggiare si marcano *close-on-exec*: vengono chiusi da `exec`, e solo da lei.

```
fcntl(listen_fd, F_SETFD, FD_CLOEXEC);      /* oppure SOCK_CLOEXEC alla creazione */
```

ESERCIZI

Esercizio 1 — Uno zombie e un processo che ascolta i segnali

Scrivere due programmi brevi.

(a) Un programma che crea un figlio che termina subito, mentre il genitore dorme. Trovare entrambi con `ps`, poi aggiungere la `wait` e guardare lo zombie sparire.

(b) Un programma che non termina mai da solo e stampa una riga ogni volta che riceve `SIGINT` o `SIGUSR1`. Provarlo con `kill -USR1`, con `Ctrl-C` e infine con `kill -KILL`.

Perché `Ctrl-C` raggiunge il programma anche se non abbiamo mai detto al terminale il suo pid?

Svolgimento

(a) Il figlio esce con `_exit(0)`; il genitore dorme 30 secondi. In quei 30 secondi `ps -o pid,ppid,stat,cmd` mostra il figlio in stato `Z` (*defunct*), con ppid uguale al pid del genitore. Aggiungendo `waitpid(pid, NULL, 0)` prima della `sleep` lo zombie non compare più.

(b) L'handler deve solo registrare; la stampa avviene nel ciclo principale.

```c
static volatile sig_atomic_t last = 0;

static void note(int sig) { last = sig; }       /* registra soltanto */

int main(void) {
    struct sigaction sa;
    sa.sa_handler = note;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT,  &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);
    printf("pid %d\n", getpid());
    for (;;) {
        pause();                                /* dorme fino al prossimo segnale */
        if (last == SIGINT)  printf("ricevuto SIGINT\n");
        if (last == SIGUSR1) printf("ricevuto SIGUSR1\n");
        last = 0;
    }
}
```

waiter.c

`kill -USR1` e `Ctrl-C` stampano una riga ciascuno; `kill -KILL` lo termina, perché `SIGKILL` non si può catturare.

`Ctrl-C` non viene mandato a un pid: il driver del terminale lo trasforma in un `SIGINT` per tutto il **gruppo di processi in primo piano** di quel terminale, e la shell, avviando il programma, ve lo ha messo. Per questo arriva anche ai figli.

Riepilogo

In breve

- `fork` ritorna due volte: il figlio riceve 0 e una copia privata della memoria, e nessuno dei due sa chi andrà per primo.
- I descriptor sono **duplicati**, non copiati: due processi puntano a un solo file aperto, una sola connessione, un solo contatore di riferimenti.
- Quindi ciascun lato chiude ciò che non usa, e una connessione finisce solo quando si chiude il suo ultimo descriptor.
- Un figlio morto resta **zombie** finché qualcuno non lo aspetta, e il server non può permettersi di stare in `wait`.
- `SIGCHLD` è il modo in cui il kernel dice «adesso»; i segnali non sono accodati, quindi l'handler raccoglie in un ciclo.
- `exec` tiene il processo e ne sostituisce il programma, descriptor compresi, a meno di dire altrimenti.

I THREAD

Il server sa già tenere più client insieme: una `fork` per connessione, un figlio ciascuno. Ma quando un client scrive una riga, come arriva agli altri due? **Non arriva**. Alla `fork` ogni figlio ha ricevuto una copia privata della memoria, quindi non esiste «la lista dei client connessi».

- Si potrebbe mandare la riga da un processo all'altro, ma sarebbe un secondo programma, con un suo protocollo, che non abbiamo scritto.
- Ogni connessione costa inoltre uno spazio di indirizzamento, le sue tabelle delle pagine e una nuova voce nelle tabelle dello scheduler.
- Quello che vogliamo è un altro **flusso di controllo dentro lo stesso processo**, che condivida tutto per impostazione predefinita.

UN PROCESSO, PIÙ FLUSSI DI CONTROLLO

Thread

Un **thread** è un flusso di controllo dentro un processo: ha il proprio stack e i propri registri, e tutto il resto in comune con gli altri thread dello stesso processo.

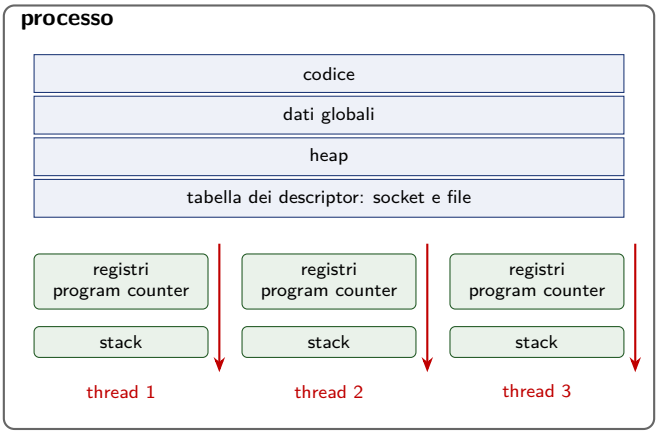


Figura 11.1 – Il processo non è più ciò che gira: è il **contenitore**. Lo scheduler esegue un thread, e fino a oggi ogni processo ne aveva uno solo.

Che cosa è privato e che cosa è condiviso

Privato di ogni thread	Condiviso da tutti i thread del processo
thread id, registri, program counter	il codice, e ogni variabile globale
il suo stack, quindi le sue variabili locali	lo heap: tutto ciò che ha restituito `malloc`
la maschera dei segnali	la tabella dei descriptor: socket e file
`errno`	la disposizione di ogni segnale (gli handler)
la priorità di scheduling	pid, ppid, directory di lavoro, credenziali

Tabella 11.1 – La colonna di sinistra è corta di proposito: un thread possiede lo stato di **una computazione**, nient'altro.

In particolare, un descriptor aperto da un thread è **`fd 4` per tutti**.

Figura 11.2 – Una sola tabella dei descriptor, quindi una sola `close` chiude una connessione; e lo stato della chat è una **variabile**, non un messaggio che qualcuno deve inviare.

Che cosa compra la condivisione, e che cosa costa

- **Niente comunicazione fra processi**: la lista dei client connessi è una variabile; un thread la scrive, gli altri la leggono, e niente viene trasmesso.
- **Più economico**: creare un thread non copia alcuno spazio di indirizzamento, e passare da un thread all'altro non cambia le tabelle delle pagine.
- **Blocco indipendente**: un thread fermo in `read` blocca solo se stesso, purché il kernel sappia che esiste.
- **Il prezzo**: niente li separa. Un thread che corrompe una struttura condivisa la corrompe per tutti.
- E un thread che dereferenzia un puntatore nullo porta giù l'intero server, client compresi; un processo figlio portava giù soltanto se stesso.

Un server con `fork` è isolato per costruzione; uno con i thread no, ed è di questo che parla la seconda metà del capitolo.

Concorrenza non è parallelismo

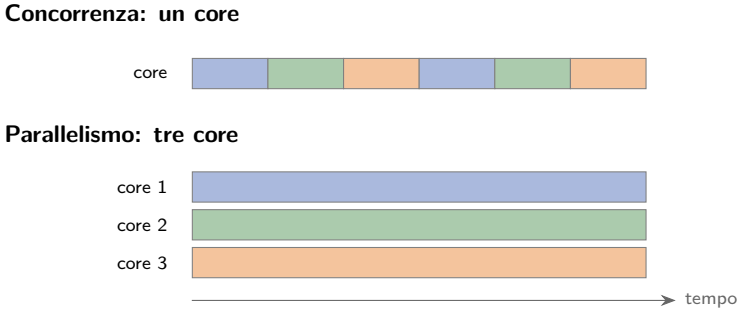


Figura 11.3 – **Concorrenza**: più compiti in corso insieme, alternati. **Parallelismo**: più compiti eseguiti nello stesso istante.

A un server serve la prima: i suoi thread passano la vita ad **aspettare la rete**, non a calcolare. La seconda è un regalo che l'hardware può fare.

Dove sono conosciuti i thread

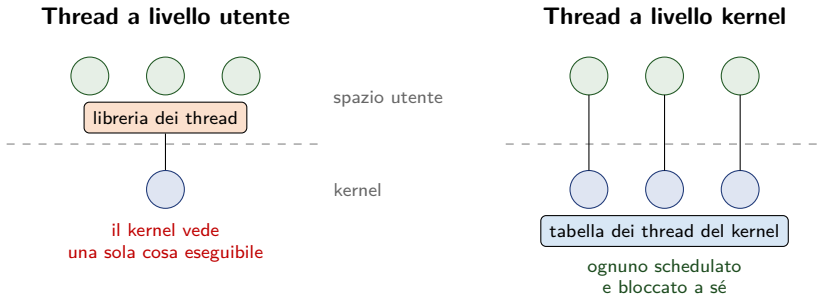


Figura 11.4 – A sinistra la libreria fa tutto nello spazio utente: cambiare thread è una chiamata di funzione, ma il kernel vede una sola cosa eseguibile. A destra il kernel ha una propria tabella: schedula ogni thread e blocca ciascuno separatamente.

- **Molti a uno**: tutti i thread utente su un solo thread del kernel. Cambiare è economico, ma una chiamata bloccante li ferma tutti, e due core non vengono mai usati.
- **Uno a uno**: un thread del kernel per ogni thread utente: blocco vero, parallelismo vero, e un oggetto del kernel per ciascuno. È **Linux**, e quindi il nostro caso.
- **Molti a molti**: m thread utente schedulati su n thread del kernel. È la risposta generale, abbastanza complicata da far tornare indietro i sistemi che l'avevano provata.

Nel nostro server ogni thread viene quindi bloccato e schedulato per conto suo, e ogni thread è un oggetto del kernel: per questo non se ne possono avere un milione.

Creare un thread

```c
#include <pthread.h>

int pthread_create(pthread_t *tid, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
```

- `tid`: un **output**, l'identità del nuovo thread, di tipo opaco `pthread_t`. Un thread chiede la propria con `pthread_self()`.
- `attr`: gli attributi, o `NULL` per quelli predefiniti, che vanno bene finché non serve un thread *detached*.
- `start_routine`: dove comincia il nuovo thread. Non prosegue da qui: **chiama** quella funzione.
- `arg`: l'unico argomento che quella funzione riceve.

Si compila con `gcc -pthread`: collega la libreria e imposta i flag che gli header si aspettano.

Gli errori non passano per errno

Tutte le chiamate viste finora restituivano -1 e impostavano `errno`. Le funzioni `pthread` **restituiscono il codice d'errore** e lasciano stare `errno`: si stampa con `strerror(err)`, non con `perror`, che stamperebbe l'ultimo fallimento non correlato. Il motivo è storico (una globale sarebbe ambigua con i thread) e oggi `errno` è in effetti per thread. `pthread_create` fallisce davvero: `EAGAIN` quando il sistema non concede un altro thread, e un server deve sopravvivere.

```c
int err = pthread_create(&tid, NULL, worker, arg);
if (err != 0)
    fprintf(stderr, "pthread_create: %s\n", strerror(err));   /* non perror */
```

La funzione di partenza e il suo unico argomento

```c
struct job { int fd; unsigned long n; };     /* cio' che serve a un thread */

static void *worker(void *arg) {
    struct job *j = arg;                     /* il void * torna com'era partito */
    serve(j->fd, j->n);
    close(j->fd);
    free(j);                                 /* e' di questo thread, quindi lo libera lui */
    return NULL;                             /* come pthread_exit(NULL) */
}

struct job *j = malloc(sizeof *j);
j->fd = fd; j->n = ++served;
pthread_create(&tid, NULL, worker, j);       /* un puntatore, e qualunque cosa dietro */
```

Un argomento solo: una struct per passarne di piu'

Più di un valore vuol dire una struttura, e quella struttura deve **esistere ancora** quando il thread la legge.

L'argomento non dev'essere una variabile riusata

```c
for (;;) {
    int fd = accept(listen_fd, NULL, NULL);
    pthread_create(&tid, NULL, worker, &fd);    /* una sola variabile, per tutti */
}
```

Due client si connettono a un millisecondo di distanza: quale descriptor serve ciascun thread? Quello che c'è in `fd` **quando guarda**. La seconda `accept` può averlo già sovrascritto: due thread sulla stessa connessione, e una connessione che nessuno serve. Si dà a ogni thread **un oggetto suo**: una `malloc` per connessione, liberata dal thread che la riceve. Nei test funziona, perché nei test i client arrivano uno alla volta.

Due thread e un main

```c
static void *say(void *arg) {
    printf("%s: tid %lu\n", (char *) arg, (unsigned long) pthread_self());
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, say, "first");
    pthread_create(&t2, NULL, say, "second");
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

hello.c

Nulla dice quale dei due stampi per primo, ed eseguirlo due volte può dare risposte diverse. `pthread_t` è opaco: stamparlo è una comodità di questo sistema, non una cosa portabile.

Quando main ritorna, muoiono tutti

Togliendo le due `join`, che cosa stampa il programma? Spesso **niente**. Il `return` da `main` è una `exit`, e `exit` termina il processo, compresi i thread che stavano per stampare.

- `exit` chiamata da **qualunque** thread termina tutti i thread: non esiste una sua versione che termini solo il chiamante.
- O si aspettano con `join` i thread avviati, oppure, se `main` non ha altro da fare, lo si chiude con `pthread_exit(NULL)`: il processo vive allora fino alla fine del suo ultimo thread.
- Una `sleep` in `main` non è un modo di aspettare un thread: è una scommessa su quanto dura il lavoro.

LA FINE DI UN THREAD

- **return** dalla funzione di partenza: il valore restituito è il valore di uscita del thread, ed è il modo normale.
- `pthread_exit(valore)`: la stessa cosa da qualunque punto, anche da una funzione chiamata dieci livelli più in basso.
- **Cancellato**: un altro thread ne ha chiesto la terminazione; il valore di uscita è allora `PTHREAD_CANCELED`.
- **Non sono modi di finire un thread**: `exit`, `_exit`, o un segnale la cui azione è terminare: quelli chiudono l'intero processo.

In qualunque modo finisca, il valore lasciato deve **sopravvivere al thread**: il suo stack non lo fa.

pthread_join

```c
int pthread_join(pthread_t tid, void **retval);

void *status;
pthread_join(tid, &status);      /* blocca finche' quel thread termina */
pthread_join(tid, NULL);         /* il valore di uscita non ci interessa */
```

- Aspetta **un thread con un nome**: non esiste `pthread_join(-1)`, un «chiunque finisca per primo» come con `wait`.
- Se quel thread è già terminato ritorna subito, esattamente come aspettare un figlio già zombie.
- Ed è l'analogia da tenere: un thread terminato e non aspettato occupa ancora le sue risorse.

```c
1 static void *local(void *arg) {
2     struct pair p = { 1, 2 };            /* sullo stack di questo thread */
3     return &p;                           /* che fra un attimo non c'e' piu' */
4 }
5 static void *heap(void *arg) {
6     struct pair *p = malloc(sizeof *p);
7     p->a = 5; p->b = 6;
8     return p;                            /* chi fa la join lo legge, poi lo libera */
9 }
```

Il valore deve sopravvivere al thread

```
thread 1: a=1 b=2
master  : a=1075156600 b=1077281896      <- lo stack nel frattempo riusato
thread 3: a=5 b=6
master  : a=5 b=6
```

Una globale o lo heap; mai lo stack. Il compilatore non lo impedisce, e la prima esecuzione spesso stampa i numeri giusti.

Thread detached

Un thread che nessuno aspetterà mai dovrebbe dirlo, e allora rilascia tutto nel momento in cui termina.

```
pthread_detach(tid);                    /* da un altro thread, oppure... */
pthread_detach(pthread_self());         /* ...dal thread stesso, per prima cosa */

pthread_attr_t attr;                    /* oppure deciso alla creazione */
pthread_attr_init(&attr);
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
pthread_create(&tid, &attr, worker, j);
pthread_attr_destroy(&attr);
```

- Per default i thread sono *joinable*: un server che se ne dimentica perde, per ogni client servito, la contabilità di un thread.
- *Detached* è in pratica irreversibile: da lì in poi `pthread_join` su quel thread è un errore, e il suo valore di uscita è perso.

UN THREAD PER CONNESSIONE

```
for (;;) {
    int *fd = malloc(sizeof *fd);          /* uno per connessione, liberato dal thread */
    *fd = accept(listen_fd, NULL, NULL);
    if (*fd < 0) { perror("accept"); free(fd); continue; }

    pthread_t tid;
    int err = pthread_create(&tid, NULL, worker, fd);
    if (err != 0) {                        /* finiti i thread: si rifiuta questo client */
        fprintf(stderr, "pthread_create: %s\n", strerror(err));
        close(*fd); free(fd); continue;
    }
    pthread_detach(tid);                   /* nessuno la aspettera' mai */
}
```

Il ciclo di accept, in sintesi

Si noti che cosa manca: **nessuna `close`** nel thread che accetta. Il descriptor è uno solo, non due, quindi chi lo chiude chiude la connessione.

```
$ gcc -pthread -o chatd chatd.c
$ ./chatd 5000 &
$ ps -o pid,lwp,nlwp,stat,comm -L -p $(pgrep chatd)
  PID   LWP NLWP STAT COMMAND
 5310  5310    4 Sl   chatd       <- il thread principale, in accept
 5310  5311    4 Sl   chatd
 5310  5312    4 Sl   chatd       <- uno per client connesso
 5310  5313    4 Sl   chatd
```

I thread visti da fuori

Un pid, quattro LWP (*lightweight process*): il kernel li schedula uno per uno, e `top -H` li mostra consumare CPU separatamente. `kill` invece si rivolge ancora al processo, perché un segnale si manda a un pid.

Processi	Thread	E la differenza
`fork`	`pthread_create`	il figlio riparte da qui, il thread parte in una funzione
`exit`	`pthread_exit`	`exit` in un thread termina il processo
`waitpid`	`pthread_join`	niente «uno qualsiasi»: si nomina il thread
`kill`	`pthread_kill`	la disposizione è una per processo
`getpid`	`pthread_self`	l'id è opaco, solo confrontabile
zombie	un thread mai aspettato	stessa perdita, stessa cura

Tabella 11.2 – Processi e thread, fianco a fianco. I nomi si corrispondono, ed è una buona mappa, salvo che i processi sono isolati e i thread no.

Il prezzo della condivisione

Il contatore che perde gli incrementi

```c
static int count = 0;                    /* una variabile, tutti i thread */

static void *bump(void *arg) {
    for (int i = 0; i < 100000; i++)
        count++;
    return NULL;
}
/* main: ne crea otto, li aspetta tutti e otto, stampa count */
```

bump.c

```
$ ./bump ; ./bump ; ./bump
count = 431027      <- atteso 800000
count = 800000      <- a volte giusto: ancora peggio
count = 512663
```

Nessun thread sbaglia e nessuna riga di C è sbagliata. E non è una questione di due core: su un core solo gli stessi incrementi si perdono, perché un thread può essere interrotto **nel mezzo** di un incremento.

Thread A	Thread B	`count`
load → 7		7
	load → 7	7
add, store 8		8
	add, store 8	8

Tabella 11.3 – `count++` è un carattere solo ma tre operazioni macchina: load, add, store. Due incrementi, uno perso.

Ogni volta che due thread possono toccare la stessa variabile e almeno uno la scrive, il risultato dipende dai tempi: è una **race condition**, e serve un modo per vietare quella sovrapposizione. È l'argomento del prossimo capitolo.

Non tutte le funzioni possono essere chiamate da due thread

Le nostre globali non sono l'unico stato condiviso: alcune funzioni di libreria tengono il proprio, in un buffer statico che non si vede.

- `strtok` ricorda dove si era fermata: due thread che analizzano due righe si calpestano la posizione.
- `gethostbyname`, `ctime`, `inet_ntoa` restituiscono un puntatore a **un solo** buffer, sovrascritto dal chiamante successivo.
- Alcune hanno un gemello **rientrante** che prende il buffer da noi: `strtok_r`, `rand_r`. Si preferiscono loro, e si preferisce `getaddrinfo`, progettata così fin dall'inizio.
- `errno` non è fra queste: è per thread, quindi il fallimento di un altro non compare mai nel nostro.

La pagina del manuale dice *MT-Safe* oppure no, e vale la pena guardarla prima di mettere una funzione dentro un worker.

Thread e segnali

- La **disposizione** è una per processo: una `sigaction`, e vale per tutti i thread. Non esiste un handler per thread.
- La **maschera** è una per thread: ciascuno decide che cosa bloccare, con `pthread_sigmask`.
- Un segnale inviato al processo viene consegnato a **un** thread qualsiasi che non lo blocchi, e non si può sapere quale.
- `pthread_kill(tid, sig)` nomina un thread; ma se la disposizione è terminare, muore comunque tutto il processo.

Per questo un server con i thread di solito blocca i segnali ovunque e lascia che sia un thread solo ad aspettarli; e ignora `SIGPIPE` con `signal(SIGPIPE, SIG_IGN)`, così la scrittura su una connessione chiusa restituisce `EPIPE` invece di uccidere il server.

Un thread chiama fork

Uno di otto thread chiama `fork`: quanti thread ha il figlio? **Uno**, quello che l'ha chiamata. Gli altri sette non esistono lì, ma tutto quello che avevano fatto alla memoria sì.

- Quello che avevano in mano resta com'era: una struttura aggiornata a metà resta a metà, e un lock resta chiuso per sempre, perché il suo proprietario non c'è a rilasciarlo.
- Il figlio di un processo con più thread può quindi fare solo le cose sicure: in pratica, una `exec`, subito.
- `exec` non ne risente: sostituisce il programma, e il nuovo parte con il solo thread che l'ha chiamata.

Cancellare un thread, e perché un server lo fa di rado

```
pthread_cancel(tid);              /* chiede la terminazione; non aspetta */
pthread_join(tid, &status);       /* status == PTHREAD_CANCELED */
```

- Per default la cancellazione è *differita*: ha effetto a un **punto di cancellazione** (`read`, `write`, `accept`, `sleep`), quindi un thread occupato a calcolare non se ne accorge per un po'.
- Il thread si ferma dove si trova: il suo buffer, il suo descriptor e qualunque cosa avesse bloccato restano com'erano.
- Per questo un server **chiede** invece di ordinare: un flag `volatile sig_atomic_t` controllato dai worker fra una richiesta e l'altra, oppure semplicemente chiude il descriptor e lascia che la loro `read` ritorni.

Quanto costa un thread, e quanti se ne possono avere

- Crearne uno costa all'incirca **un ordine di grandezza meno** di una `fork`: nessuno spazio di indirizzamento da duplicare, nessuna tabella delle pagine da copiare.
- Passare fra due thread dello stesso processo non cambia lo spazio di indirizzamento, quindi le cache e il TLB sopravvivono.
- Ma ogni thread riserva uno stack: su Linux 8 MB di spazio di indirizzamento per default, e se ne può chiedere meno alla creazione.
- E ognuno è un oggetto del kernel con il suo stack del kernel: migliaia sono normali, centinaia di migliaia no.

Un thread per client va bene per una chat con un'aula collegata, non per diecimila connessioni: lì subentrano un **insieme fisso di worker** e un thread che osserva **molte connessioni insieme**.

Esercizio 1 — Contatori e thread

Scrivere due programmi brevi.

(a) Prendere n dalla riga di comando, creare n thread che dormono ciascuno un numero casuale di secondi, poi incrementano un contatore globale e lo stampano; aspettarli tutti.

(b) Lo stesso, senza `sleep`, con ogni thread che incrementa 100 000 volte.

Eseguire ciascuno venti volte e contare le risposte. Ci sono race condition nel primo programma? E perché il secondo le mostra mentre il primo le nasconde?

Svolgimento

```c
static int counter = 0;

static void *work(void *arg) {
    unsigned seed = (unsigned) (uintptr_t) arg;   /* un seme diverso per thread */
    sleep(rand_r(&seed) % 3 + 1);
    counter++;                                    /* la stessa race del capitolo */
    printf("counter = %d\n", counter);
    return NULL;
}

int main(int argc, char *argv[]) {
    int n = atoi(argv[1]);
    pthread_t *t = malloc(n * sizeof *t);
    for (int i = 0; i < n; i++)
        pthread_create(&t[i], NULL, work, (void *) (uintptr_t) (i + 1));
    for (int i = 0; i < n; i++)
        pthread_join(t[i], NULL);
    free(t);
    return 0;
}
```

threads.c: la versione (a)

La versione (b) toglie la `sleep` e mette `counter++` in un ciclo di 100 000 iterazioni, stampando solo alla fine in `main`. Si usa `rand_r` e non `rand`, che tiene il proprio stato in una globale.

Sì, la race c'è **anche** nel primo: `counter++` non è indivisibile, e due thread che si svegliano insieme possono perdere un incremento. Ma i thread si svegliano a secondi di distanza, e ognuno esegue un solo incremento, che dura nanosecondi: la probabilità che due di essi si sovrappongano proprio in quelle tre istruzioni è minima, quindi il risultato è quasi sempre giusto. Nel secondo otto thread eseguono centinaia di migliaia di incrementi negli stessi millisecondi, e le sovrapposizioni diventano inevitabili. La race non dipende da quanto spesso si vede: un programma che «funziona sempre» nei test può averla.

Riepilogo

In breve

- Un thread possiede il proprio stack, i propri registri e la propria maschera dei segnali; tutto il resto (globali, heap, descriptor) appartiene al processo.
- Quindi i thread condividono lo stato **leggendo una variabile**, che è proprio ciò che un server di chat con i processi non poteva fare.
- `pthread_create` prende una funzione e un argomento, e quell'argomento deve appartenere al thread a cui viene dato.
- Le chiamate `pthread` restituiscono un codice d'errore, non -1; e un thread terminato va aspettato con `join` o reso *detached*, altrimenti perde risorse.
- `exit` e il ritorno di `main` terminano tutti i thread; solo `pthread_exit` e `return` ne terminano uno.
- E il prezzo della condivisione è `count++`: codice corretto, risultato sbagliato, ogni volta che due thread scrivono la stessa cosa.

Sincronizzazione

I thread condividono globali, heap e descriptor: è per questo che un server di chat può tenere la lista dei client in una variabile. Ed è anche il motivo per cui `count++` ha stampato 431027 invece di 800000.

- Il problema non è mai stato l'incremento: è che una **sequenza di operazioni su uno stato condiviso** può essere interrotta a metà, e niente in C la rende indivisibile.
- Bisogna quindi poter dire: **da qui a qui, un thread alla volta**.
- E serve l'altra metà, che il contatore non mostrava: un thread che non ha niente da fare deve poter **aspettare** che un altro abbia fatto qualcosa.

Due strumenti per i due problemi (il **mutex** e la **variabile di condizione**), un terzo che conta (il **semaforo**) e una struttura che li mette insieme (il **buffer limitato**).

Lo stato condiviso di un server

- La **tabella dei client connessi**: un thread aggiunge una voce all'`accept`, un altro la scorre per inoltrare una riga, un terzo toglie una voce quando un client se ne va.
- I **contatori** che si stampano in una riga di stato: byte serviti, client visti, errori.
- Una **cache** o una tabella delle stanze: letta da tutti, scritta ogni tanto.
- La **coda del lavoro** da fare, che costruiremo alla fine del capitolo.

Ognuna di queste ha un **invariante** (`n` è il numero di voci valide, ogni voce sotto `n` è un descriptor aperto), ed è proprio un invariante che un aggiornamento interrotto a metà rompe.

Il mutex

Mutex

Un **mutex** (*mutual exclusion*) è un lucchetto con due stati, aperto e chiuso, tenuto da **al più un thread alla volta**. Vive in una variabile di tipo `pthread_mutex_t`.

- Il thread che lo chiude ne diventa il **proprietario**, e solo il proprietario può aprirlo: un mutex non si passa di mano in mano.
- Un thread che lo trova chiuso **dorme** finché non si libera: non gira a vuoto, e il kernel dà il processore a qualcun altro.
- Ogni thread che tocca i dati deve chiudere **lo stesso** mutex. La protezione è cooperativa: niente la impone.

Il mutex non sa nulla di ciò che protegge: quell'associazione esiste solo nel codice che scriviamo.

```c
#include <pthread.h>

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;     /* statico: non serve altro */

int pthread_mutex_init(pthread_mutex_t *m, const pthread_mutexattr_t *attr);
int pthread_mutex_destroy(pthread_mutex_t *m);     /* nessuno deve tenerlo */

int pthread_mutex_lock(pthread_mutex_t *m);        /* aspetta che sia libero */
int pthread_mutex_trylock(pthread_mutex_t *m);     /* EBUSY invece di aspettare */
int pthread_mutex_unlock(pthread_mutex_t *m);
```

L'API del mutex

- `init` serve per un mutex che non è globale: uno dentro una struttura allocata con `malloc`. `attr` a `NULL` significa i valori predefiniti, che vanno bene.
- Restituiscono un codice d'errore, come ogni chiamata `pthread`, e non -1 con `errno`.
- `trylock` non è la versione educata di `lock`: si usa solo quando c'è qualcos'altro da fare nel frattempo.

```c
static int count = 0;
static pthread_mutex_t count_mtx = PTHREAD_MUTEX_INITIALIZER;

static void *bump(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&count_mtx);
        count++;                          /* tutto il load-add-store, da solo */
        pthread_mutex_unlock(&count_mtx);
    }
    return NULL;
}
```

Il contatore, corretto

Ora `count` vale 800000 a ogni esecuzione: la risposta non dipende più dai tempi. Gli otto thread non possono più sovrapporsi su quella riga, e il programma è più lento, perché ora fanno la fila. È questo lo scambio.

Un mutex protegge dati, non righe di codice

- Ogni accesso alla variabile prende lo stesso mutex, **letture comprese**: un thread che legge una struttura mentre un altro la sta aggiornando legge qualcosa che non è mai stato vero.
- Due mutex sulla stessa variabile non proteggono niente: due thread ne prendono uno ciascuno ed entrano entrambi.
- Se la struttura viene modificata in un solo punto, basta un mutex attorno a quella **sezione critica**. Se la si tocca da più punti, il mutex appartiene alla struttura e viaggia con lei.
- Bisogna dire ad alta voce che cosa protegge il mutex, e scriverlo accanto alla sua dichiarazione: è l'unico posto in cui quell'associazione resta registrata.

Niente controlla nulla di tutto questo: `count++` senza lock compila benissimo.

```c
struct clients {
    int fd[MAX];
    int n;                        /* protetto da mtx, insieme a fd[] */
    pthread_mutex_t mtx;          /* il lock viaggia con i dati */
};

struct clients *clients_new(void) {
    struct clients *c = malloc(sizeof *c);
    if (c == NULL) return NULL;
    c->n = 0;
    pthread_mutex_init(&c->mtx, NULL);   /* allocato con malloc: niente inizializzatore statico */
    return c;
}
```

Il lock dentro la struttura che protegge

Chi riceve il puntatore riceve il lock con esso, e non c'è modo di raggiungere i dati senza vederlo. Un lock per struttura, non un lock per tutto il server.

```c
struct session { int fd; int refs; pthread_mutex_t mtx; };

void session_hold(struct session *s) {          /* un thread in piu' la usa */
    pthread_mutex_lock(&s->mtx);
    s->refs++;
    pthread_mutex_unlock(&s->mtx);
}

void session_release(struct session *s) {
    pthread_mutex_lock(&s->mtx);
    if (--s->refs > 0) { pthread_mutex_unlock(&s->mtx); return; }
    pthread_mutex_unlock(&s->mtx);              /* l'ultimo a uscire: nessuno la raggiunge */
    pthread_mutex_destroy(&s->mtx);
    free(s);
}
```

Un oggetto condiviso con un contatore di riferimenti

Il contatore è stato condiviso come qualunque altro, quindi vuole il lock; e l'oggetto viene liberato da chi lo porta a **zero**, non dal thread che l'ha creato.

Per quanto tempo tenere un lock

- **Chiudere tardi e aprire presto**: quello che sta fra i due gira un thread alla volta, ed è la parte del server che non scala.
- **Mai attraverso una chiamata bloccante**. Una `write` verso un client lento può durare secondi; tenendo il lock mentre succede, ogni altro thread aspetta quel client. Si copia ciò che serve, si apre, poi si invia.
- Ma senza andare all'estremo opposto: chiudere, leggere un valore, aprire e poi agire. A quel punto il valore può essere falso. Ciò che deve essere indivisibile è **la decisione insieme all'azione**.
- Due strutture indipendenti hanno due mutex, non uno: i thread che lavorano su cose diverse non devono fare la fila l'uno dietro l'altro.

In una riga: lo si tiene per poche istruzioni sulla memoria, **mai durante l'I/O**.

Due lock, due thread, nessun progresso

Il thread A inoltra una riga e ha bisogno del log; il thread B scrive il log e ha bisogno della tabella dei client. Ciascuno ha già preso un lock. Nessuno dei due andrà più avanti: ognuno tiene ciò che l'altro aspetta, e un mutex lo rilascia solo il suo proprietario. È un **deadlock**.

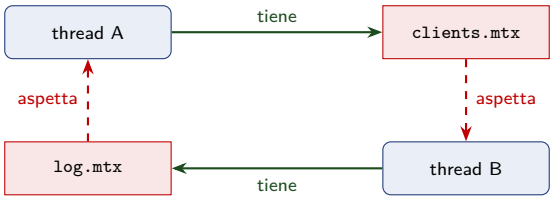


Figura 12.1 – Un ciclo di attese: A aspetta `log.mtx`, tenuto da B, che aspetta `clients.mtx`, tenuto da A.

La cura è fissare un **ordine** sui lock (prima la tabella, poi il log) e far sì che ogni thread li prenda in quell'ordine: il ciclo non può formarsi. Dove l'ordine non si conosce in anticipo, si usa `trylock`, e su `EBUSY` si rilascia tutto ciò che si tiene e si ricomincia.

Tipi di mutex, e mutex fra processi

- **Default** (`PTHREAD_MUTEX_NORMAL`): chiuderlo due volte dallo stesso thread è un deadlock: il thread aspetta se stesso.
- `PTHREAD_MUTEX_ERRORCHECK` restituisce invece `EDEADLK`, e rifiuta un `unlock` da un thread che non ne è proprietario. Utile mentre si fa il debug.
- `PTHREAD_MUTEX_RECURSIVE` conta, e si apre all'`unlock` corrispondente. Averne bisogno di solito significa che una funzione che ha chiuso il lock ne chiama un'altra che lo chiude di nuovo: è la struttura da correggere, non il mutex.

```c
pthread_mutexattr_t a;
pthread_mutexattr_init(&a);
pthread_mutexattr_settype(&a, PTHREAD_MUTEX_ERRORCHECK);
pthread_mutex_init(&m, &a);
```

Un mutex in memoria ordinaria non significa nulla per un figlio dopo la `fork`: il figlio ne ha una copia. Per condividerne uno fra processi deve stare in **memoria condivisa**, con `PTHREAD_PROCESS_SHARED` impostato negli attributi.

LE VARIABILI DI CONDIZIONE

Un mutex risponde a una sola domanda: *nessun altro mentre ci sono io*. Non risponde all'altra, *aspetta che ci sia qualcosa da fare*, che è ciò di cui ha bisogno un worker con la coda vuota.

```c
for (;;) {
    pthread_mutex_lock(&m);
    if (queue->count > 0) break;      /* trovato lavoro: si esce tenendo m */
    pthread_mutex_unlock(&m);         /* ancora niente: si lascia entrare qualcun altro e si richiede */
}
```

Aspettare con il solo mutex: attesa attiva

- Funziona, e brucia un processore a porre una domanda la cui risposta è quasi sempre no.
- Dormendo fra un tentativo e l'altro si smette di bruciarlo, ma si risponde in ritardo: non esiste un intervallo giusto.
- Manca un modo di **dormire finché qualcuno non dice che lo stato è cambiato**: le variabili di condizione di Hoare.

Variabile di condizione

Una **variabile di condizione** è una coda di thread che aspettano che qualcosa diventi vero sullo stato condiviso, con due operazioni: `wait`, che mette in coda, e `signal`, che ne fa uscire qualcuno.

- Non contiene alcun valore e non ha memoria: un `signal` quando nessuno aspetta **va perso**. Per questo la condizione vera e propria vive nello stato condiviso (`count > 0`), mai nella variabile.
- Ed è per questo che va sempre insieme al mutex che protegge quello stato.

```c
pthread_cond_t c = PTHREAD_COND_INITIALIZER;                /* oppure, in una struttura: */
int pthread_cond_init(pthread_cond_t *c, const pthread_condattr_t *attr);
int pthread_cond_destroy(pthread_cond_t *c);                /* nessuno deve aspettarci sopra */
```

Che cosa fa pthread_cond_wait con il mutex

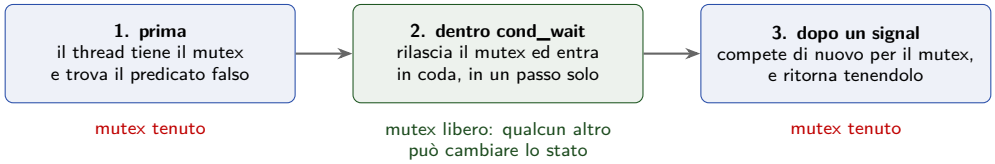


Figura 12.2 – Solo nella fase centrale un altro thread può rendere vero il predicato.

- Il mutex va tenuto quando la si chiama, perché si è appena letto lo stato condiviso per trovare la condizione falsa.
- Rilascia il mutex ed entra in coda come **un unico passo indivisibile**: nessun `signal` può infilarsi in mezzo, ed è proprio quello che andrebbe perso se si aprisse il mutex da soli prima.
- Quando ritorna il mutex è di nuovo tenuto: il chiamante è dov'era, e può guardare lo stato ancora una volta.

Chi aspetta la condizione	Chi la rende vera
`pthread_mutex_lock(&m)`	`pthread_mutex_lock(&m)`
`while (condizione falsa)`	cambia lo stato condiviso
`  pthread_cond_wait(&c, &m)`	`pthread_cond_signal(&c)`
usa lo stato	`pthread_mutex_unlock(&m)`
`pthread_mutex_unlock(&m)`	

Tabella 12.1 – I due lati dello schema. Entrambi tengono il mutex: chi cambia lo stato perché nessuno lo legga cambiato a metà, chi aspetta perché il controllo e il sonno non si possano separare. Rendere vera una condizione non blocca mai.

Perché la wait sta dentro un while

`pthread_cond_wait` è appena tornata: la condizione è vera? **Non necessariamente**. Per questo la chiamata sta dentro un `while`, mai dentro un `if`.

- Fra il `signal` e la riacquisizione del mutex un altro thread può aver preso l'elemento: ci è stato detto *c'è qualcosa*, non *c'è qualcosa per te*.
- Un `broadcast` sveglia dieci thread per un solo elemento, e nove trovano di nuovo la condizione falsa.
- E POSIX ammette i **risvegli spuri**: la chiamata può tornare senza che nessuno abbia segnalato.

Il `while` costa un controllo in più e rende i tre casi uno solo: si guarda lo stato, e se non è quello che serve si aspetta di nuovo.

signal o broadcast

- `pthread_cond_signal` sveglia **esattamente uno** dei thread in attesa, scelto in modo non deterministico. Se nessuno aspetta non fa nulla, e non resta traccia di niente.
- `pthread_cond_broadcast` li sveglia **tutti**. Escono comunque uno alla volta, perché ciascuno deve riprendere il mutex.
- Si usa `signal` quando uno qualsiasi dei thread in attesa può gestire ciò che è cambiato: è arrivato un elemento, basta un worker.
- Si usa `broadcast` quando chi aspetta non è intercambiabile, o quando ciò che è cambiato riguarda tutti (il server si sta chiudendo).

`broadcast` dove bastava `signal` è solo più lento; `signal` dove serviva `broadcast` è un thread che non si sveglia più.

Un produttore, due consumatori

```c
static struct { int a, b; } shared;
static pthread_mutex_t m        = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t  nonempty = PTHREAD_COND_INITIALIZER;
static const struct timespec pause = { 0, 200000000 };   /* 200 ms */

static void *consumer_one(void *arg) {
    for (;;) {
        pthread_mutex_lock(&m);
        while (shared.a == 0)                       /* ancora niente da prendere */
            pthread_cond_wait(&nonempty, &m);
        printf("consumer 1: a = %d\n", shared.a);
        shared.a--;
        pthread_mutex_unlock(&m);
        nanosleep(&pause, NULL);                    /* fuori dal lock, di proposito */
    }
}
```

Il consumatore

```c
static void *producer(void *arg) {
    for (int j = 0; j < 10; j++) {
        pthread_mutex_lock(&m);
        shared.a++; shared.b++;
        printf("producer: a = %d b = %d\n", shared.a, shared.b);
        pthread_cond_signal(&nonempty);            /* uno dei due consumatori */
        pthread_mutex_unlock(&m);
        nanosleep(&pause, NULL);
    }
    return NULL;
}
```

Il produttore

Il secondo consumatore è lo stesso ciclo sulla stessa variabile di condizione, e prende `a` e `b` insieme: due thread nella stessa coda. Nell'esecuzione `a` non supera mai 1: i consumatori lo svuotano alla stessa velocità con cui si riempie. Quale dei due riceve l'elemento è una scelta di `signal`, non nostra.

Aspettare con una scadenza

```c
int pthread_cond_timedwait(pthread_cond_t *c, pthread_mutex_t *m,
                           const struct timespec *abstime);   /* un istante, non una durata */

struct timespec deadline;
clock_gettime(CLOCK_REALTIME, &deadline);
deadline.tv_sec += 2;

pthread_mutex_lock(&m);
int rc = 0;
while (!ready && rc == 0)
    rc = pthread_cond_timedwait(&c, &m, &deadline);          /* 0, oppure ETIMEDOUT */
pthread_mutex_unlock(&m);
```

La scadenza è **assoluta** perché rientrare nel ciclo non faccia ripartire l'orologio. Per default si misura su `CLOCK_REALTIME`, che salta quando qualcuno regola l'orologio: `pthread_condattr_setclock(&attr, CLOCK_MONOTONIC)` è ciò che la rende un timeout vero.

IL BUFFER LIMITATO

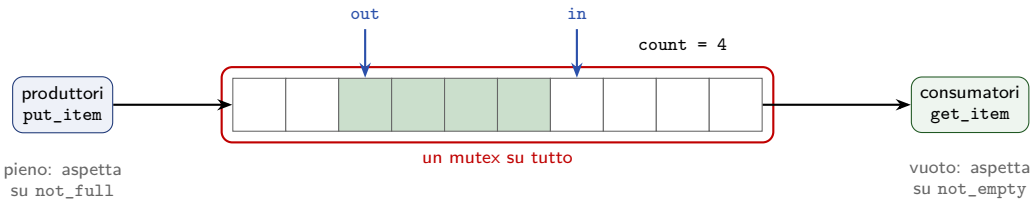


Figura 12.3 – n posti, alcuni produttori, alcuni consumatori. Se è pieno un produttore aspetta; se è vuoto aspetta un consumatore.

Servono uno stato condiviso (l'array, `in`, `out`, `count`), **un** mutex su tutto, e **due** variabili di condizione, perché le cose da aspettare sono due. È lo stesso oggetto che più avanti starà fra un ciclo di `accept` e un insieme fisso di worker.

```c
#define BUF_SIZE 10

typedef struct {
    int buf[BUF_SIZE];
    int in, out, count;          /* 0 <= count <= BUF_SIZE; in e out modulo BUF_SIZE */
    pthread_mutex_t mtx;
    pthread_cond_t not_full;     /* qui aspettano i produttori */
    pthread_cond_t not_empty;    /* qui aspettano i consumatori */
} bbuff_t;

void bbuff_init(bbuff_t *b) {
    b->in = b->out = b->count = 0;
    pthread_mutex_init(&b->mtx, NULL);
    pthread_cond_init(&b->not_full, NULL);
    pthread_cond_init(&b->not_empty, NULL);
}
```

La struttura condivisa

```c
void put_item(bbuff_t *b, int item) {
    pthread_mutex_lock(&b->mtx);
    while (b->count == BUF_SIZE)                    /* pieno: si aspetta posto */
        pthread_cond_wait(&b->not_full, &b->mtx);
    b->buf[b->in] = item;
    b->in = (b->in + 1) % BUF_SIZE;
    b->count++;
    pthread_cond_signal(&b->not_empty);             /* adesso c'e' qualcosa */
    pthread_mutex_unlock(&b->mtx);
}

int get_item(bbuff_t *b) {
    pthread_mutex_lock(&b->mtx);
    while (b->count == 0)                           /* vuoto: si aspetta un elemento */
        pthread_cond_wait(&b->not_empty, &b->mtx);
    int item = b->buf[b->out];
    b->out = (b->out + 1) % BUF_SIZE;
    b->count--;
    pthread_cond_signal(&b->not_full);              /* di nuovo un posto libero */
    pthread_mutex_unlock(&b->mtx);
    return item;
}
```

put_item e get_item

Ciascuna funzione aspetta su una condizione e segnala l'altra. Queste due funzioni sono **tutta l'interfaccia**: chi le chiama non vede mai il mutex, e quindi non può dimenticarlo.

Perché due variabili di condizione e non una

Una variabile sola per entrambe le condizioni. Il buffer è pieno, i produttori ci aspettano sopra, e un consumatore prende un elemento e segnala: chi si sveglia? Il thread che sceglie l'implementazione, e può essere **un altro consumatore**, che trova il buffer vuoto e torna a dormire. Il risveglio è sprecato, i produttori aspettano ancora, e c'è posto.

- Niente viene memorizzato: un segnale consegnato a un thread che non può usarlo è semplicemente perso.
- O una variabile per condizione, come abbiamo fatto, oppure una sola e sempre `broadcast`, svegliando tutti ogni volta per trovare l'unico che può procedere.

Da qui la regola: **una variabile di condizione per ogni cosa che i thread aspettano**.

```c
for (int i = 0; i < N_PROD; i++) pthread_create(&prod[i], NULL, producer, bb);
for (int i = 0; i < N_CONS; i++) pthread_create(&cons[i], NULL, consumer, bb);

for (int i = 0; i < N_PROD; i++)
    pthread_join(prod[i], NULL);          /* nessuno mettera' altro */

for (int i = 0; i < N_CONS; i++)
    put_item(bb, KILL_PILL);              /* un elemento velenoso per ogni consumatore */

for (int i = 0; i < N_CONS; i++)
    pthread_join(cons[i], NULL);          /* ciascuno ritorna quando legge il suo */
```

La pillola avvelenata

Un consumatore addormentato in `cond_wait` non si può avvisare di fermarsi: cancellarlo lascerebbe il mutex chiuso. Lo si ferma allora nel modo in cui sta già ascoltando: **attraverso il buffer**.

I SEMAFORI

Semaforo

Un **semaforo** è un intero non negativo con due operazioni atomiche: `wait` lo decrementa, bloccando finché vale zero, e `signal` lo incrementa. Dijkstra le chiamava **P** e **V**.

- Un semaforo **contatore**: il valore è il numero di **permessi** ancora disponibili, cioè quante copie di una risorsa sono libere.
- Un semaforo **binario**, zero o uno, si comporta come un mutex con una differenza: **non ha proprietario**, quindi qualunque thread può restituire un permesso.
- Ed è questo che lo rende anche un modo di dire *è successo*: parte da zero, un thread aspetta, un altro fa `post`.
- Il conteggio è tenuto per noi, quindi, a differenza di una variabile di condizione, non c'è un nostro predicato da controllare, e niente `while`.

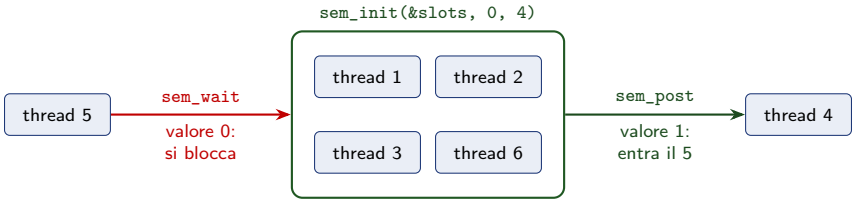


Figura 12.4 – Il valore non è un flag né un lock: è **quanti possono stare ancora dentro**. Il 4 esce e restituisce il suo permesso, il 5 lo prende. E siccome nessuno possiede un permesso, un thread può restituirne uno che non ha mai preso: è tutta la differenza da un mutex.

```c
#include <semaphore.h>

sem_t s;
int sem_init(sem_t *s, int pshared, unsigned value);  /* pshared 0: thread di questo processo */
int sem_wait(sem_t *s);       /* prende un permesso; blocca finche' il valore e' 0 */
int sem_trywait(sem_t *s);    /* EAGAIN invece di bloccare */
int sem_post(sem_t *s);       /* ne restituisce uno, e sveglia chi aspetta */
int sem_destroy(sem_t *s);
```

I semafori POSIX

- Sono chiamate Unix, non `pthread`: restituiscono -1 e impostano `errno`.
- `sem_wait` restituisce `EINTR` se un handler di segnale gira mentre aspetta, quindi va messa in un ciclo.
- `sem_post` è l'unica chiamata di sincronizzazione che si può usare dentro un handler di segnale.
- Su macOS `sem_init` è deprecata e non funziona: si usa il container del corso.

```c
sem_t empty, full, lock;          /* inizializzati a BUF_SIZE, 0 e 1 */

void put_item(int item) {
    sem_wait(&empty);             /* un posto libero, o si aspetta */
    sem_wait(&lock);
    buf[in] = item; in = (in + 1) % BUF_SIZE;
    sem_post(&lock);
    sem_post(&full);              /* un elemento in piu' da prendere */
}

int get_item(void) {
    sem_wait(&full);
    sem_wait(&lock);
    int item = buf[out]; out = (out + 1) % BUF_SIZE;
    sem_post(&lock);
    sem_post(&empty);
    return item;
}
```

Lo stesso buffer con i semafori

Niente `while` e niente `count`: `empty` e `full` **sono** il conteggio. `lock` serve ancora: i due semafori contano gli elementi, non proteggono `in` e `out`.

Semaforo o variabile di condizione

- Se ciò che si aspetta è *è disponibile un'unità di qualcosa*, un semaforo lo dice in una chiamata e tiene il conteggio per noi.
- Se è qualunque altro predicato (*due elementi sono pronti*, *questo client entra nella tabella*, *il server si sta chiudendo*), lo strumento è la variabile di condizione: il predicato è nostro e può essere qualsiasi cosa.
- L'ordine delle due `sem_wait` in `put_item` non è libero: prendendo `lock` prima di `empty`, un buffer pieno va in deadlock. È di nuovo la regola dell'ordine dei lock.
- Per uno stato che tutti leggono e quasi nessuno scrive (la tabella dei client connessi) POSIX ha `pthread_rwlock_t`: molti lettori insieme, uno scrittore da solo.

Tre strumenti, una domanda ciascuno: **chi può stare dentro** (mutex), **che cosa sto aspettando** (variabile di condizione), **quanti insieme** (semaforo).

Limitare la concorrenza con un semaforo contatore

```c
static sem_t slots;                    /* quanti client serviamo insieme */

static void *worker(void *arg) {
    int fd = *(int *) arg; free(arg);
    serve(fd);
    close(fd);
    sem_post(&slots);                  /* il permesso torna indietro */
    return NULL;
}

sem_init(&slots, 0, 64);
for (;;) {
    int fd = accept(listen_fd, NULL, NULL);
    sem_wait(&slots);                  /* 64 gia' dentro: il ciclo aspetta qui */
    int *p = malloc(sizeof *p); *p = fd;
    pthread_create(&tid, NULL, worker, p);
}
```

Al piu' 64 client serviti insieme

Senza, il server crea un thread per qualunque cosa arrivi. Con, il numero di thread è limitato e gli altri aspettano nella coda di `listen`.

Una globale privata di ogni thread

```c
static pthread_key_t ctx_key;

static void ctx_free(void *p) { free(p); }       /* gira quando un thread termina */

pthread_key_create(&ctx_key, ctx_free);          /* una volta, prima dei thread */

pthread_setspecific(ctx_key, ctx);               /* in ogni worker: la mia connessione */
struct ctx *c = pthread_getspecific(ctx_key);    /* ovunque piu' sotto, senza passarlo */

_Thread_local int depth;                         /* C11: la stessa idea, variabile semplice */
```

Una chiave, un valore per thread, `NULL` ovunque finché qualcuno non lo imposta; il distruttore gira una volta per thread, alla sua fine. Serve per un contesto che altrimenti si dovrebbe passare attraverso ogni funzione, non come scusa per non passare un argomento.

Esercizio 1 — Produttori e consumatori

Prendere n e m dalla riga di comando e avviare n produttori e m consumatori su un array condiviso di 100 interi. Ogni produttore aspetta un numero casuale di secondi fra 1 e 10, poi inserisce da 1 a 5 numeri casuali; se trova l'array pieno, salta il turno. Ogni consumatore aspetta finché c'è qualcosa, poi stampa il proprio thread id e il valore preso. Provare con `./prodcons 3 2` e poi con 1 produttore e 8 consumatori. Quale delle due attese è una `cond_wait`, e quale non è affatto un'attesa? E che cosa va storto se le due variabili di condizione diventano una?

Svolgimento

Il produttore **non aspetta** il posto: se l'array è pieno rinuncia al turno, quindi gli basta il mutex. Solo il consumatore aspetta, su `not_empty`.

```c
#define CAP 100
static int arr[CAP], count = 0;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;

static void *producer(void *arg) {
    unsigned seed = (unsigned) (uintptr_t) arg;
    for (;;) {
        sleep(rand_r(&seed) % 10 + 1);
        int k = rand_r(&seed) % 5 + 1;
        pthread_mutex_lock(&m);
        if (count + k <= CAP) {                      /* c'e' posto per tutti e k */
            for (int i = 0; i < k; i++) arr[count++] = rand_r(&seed) % 1000;
            pthread_cond_broadcast(&not_empty);      /* k elementi: piu' consumatori */
        }                                            /* altrimenti salta il turno */
        pthread_mutex_unlock(&m);
    }
}

static void *consumer(void *arg) {
    for (;;) {
        pthread_mutex_lock(&m);
        while (count == 0)
            pthread_cond_wait(&not_empty, &m);
        int v = arr[--count];
        pthread_mutex_unlock(&m);
        printf("%lu: %d\n", (unsigned long) pthread_self(), v);   /* fuori dal lock */
    }
}
```

prodcons.c: le due funzioni dei thread

Si usa `broadcast` perché un produttore può aggiungere fino a cinque elementi, e con `signal` se ne sveglierebbe un consumatore solo. Con 1 produttore e 8 consumatori quasi tutti i consumatori passano il tempo in `cond_wait`: l'output è a raffiche, una ogni volta che il produttore si sveglia.

Nella versione completa, in cui anche il produttore aspetta su `not_full`, unire le due variabili in una porta al caso del riquadro: un `signal` destinato a un produttore può svegliare un consumatore, che trova la condizione falsa e torna a dormire, e il segnale è perso. Il programma può restare fermo con posto libero e produttori in attesa.

Riepilogo

In breve

- Un **mutex** risponde «uno alla volta»: ogni accesso prende lo stesso, letture comprese, tenuto per poche istruzioni e mai durante l'I/O.
- Due lock presi in due ordini sono un **deadlock**; un ordine unico per tutto il programma lo elimina.
- Una **variabile di condizione** serve ad aspettare un predicato nostro: `wait` dentro un `while`, sempre tenendo il mutex, e `signal` dopo aver cambiato lo stato.
- `signal` sveglia uno, `broadcast` tutti; un `signal` che nessuno aspetta va perso, per questo la condizione vive nello stato condiviso.
- Il **buffer limitato** è un mutex e due condizioni dietro due funzioni: la coda che starà fra un ciclo di `accept` e i suoi worker.
- Un **semaforo** conta permessi: lo stesso buffer senza predicato, e il modo di limitare quanti client un server serve insieme.

Server concorrenti: processi e thread

Abbiamo un server che funziona e che non si può usare: serve un client, e tutti gli altri aspettano che quello se ne vada.

- Nel frattempo abbiamo smontato i tre meccanismi che lo risolvono: un **secondo processo**, un **secondo thread**, e un **mutex** per rendere sicura la condivisione.
- Nessuno dei tre ha mai toccato un socket: la `fork` stampava una riga due volte, i thread incrementavano un contatore.
- Ora entrano nel ciclo di `accept`, e lo stesso servizio si scrive due volte: una con i processi, una con i thread.
- Le due versioni differiscono in una cosa sola, che poi decide tutto il resto: se il codice che serve un client **può vedere gli altri**.

Che cosa rende concorrente un server

- Il kernel accetta già le connessioni mentre siamo occupati: restano nella **coda di listen**. Quello che non può fare è servirle.
- La coda quindi compra **spazio, non tempo**. Un client che ci sta dentro aspetta esattamente quanto prima, e quando la coda è piena il successivo viene rifiutato.
- E il server, quasi sempre, non sta calcolando: un server di chat passa la vita bloccato in `read`.

Server concorrente

Un server è **concorrente** quando il tempo che un client passa a pensare non diventa l'attesa di un altro: il secondo viene servito mentre il primo tace.

Ci sono tre modi di arrivarci: **un processo per client**, **un thread per client**, e **un solo flusso di controllo che osserva molte connessioni**. In quest'ordine: i primi due in questo capitolo, il terzo nel prossimo.

Un processo per client

Nel meccanismo non c'è nulla di nuovo: dopo `accept` il server fa `fork`, e il figlio eredita **entrambi** i socket, quello di ascolto e quello connesso. È la figura dei descriptor ereditati del capitolo sui processi: ciascuno dei due chiude quello che non usa, e ogni oggetto vive finché non se ne va il suo ultimo descriptor. Cambia solo ciò che va preparato **prima** del ciclo.

```c
static void reap(int sig) {                 /* installato una volta, prima di ogni fork */
    int saved = errno;
    while (waitpid(-1, NULL, WNOHANG) > 0)
        ;                                   /* tutti i figli che hanno finito */
    errno = saved;
}

int main(int argc, char *argv[]) {
    struct sigaction sa;
    int listen_fd = listening_socket(argc > 1 ? argv[1] : "5000");

    sa.sa_handler = reap;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGCHLD, &sa, NULL);          /* altrimenti i figli si accumulano come zombie */
    signal(SIGPIPE, SIG_IGN);               /* un client che se ne va non deve ucciderci */
```

Il server concorrente: che cosa cambia prima del ciclo

Due righe, ed entrambe riguardano **qualcun altro che finisce**: un figlio che è morto, e un client che ha chiuso mentre gli stavamo scrivendo.

```c
for (;;) {
    int fd = accept(listen_fd, NULL, NULL);
    if (fd < 0) {
        if (errno == EINTR) continue;   /* e' morto un figlio mentre aspettavamo */
        perror("accept"); continue;
    }
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); close(fd); continue; }   /* si rifiuta solo questo */

    if (pid == 0) {                     /* il figlio: solo questa connessione */
        close(listen_fd);
        serve(fd);
        close(fd);
        _exit(0);                       /* non exit(): i buffer non sono suoi */
    }
    close(fd);                          /* il genitore torna subito in accept */
}
```

Il ciclo di accept che fa fork

- L'`EINTR` è una conseguenza diretta dell'handler: il `SIGCHLD` arriva mentre il genitore è bloccato in `accept`, e la chiamata ritorna con -1. Non è un errore: si ricomincia.
- Una `fork` che fallisce (troppi processi, poca memoria) rifiuta **questo** client, non ferma il server.

```c
static unsigned long served = 0;            /* quanti messaggi abbiamo inoltrato */

static void serve(int fd) {                 /* un'intera conversazione */
    char buf[256], head[32];
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0) {
        int h = snprintf(head, sizeof head, "[%lu] ", ++served);
        if (write(fd, head, h) != h) break;
        if (write(fd, buf, n) != n) break;  /* quello che e' arrivato: non ancora una riga */
    }
    if (n < 0) perror("read");
}
```

Il figlio: il servizio non è cambiato

La concorrenza sta **tutta nel ciclo**, non nel servizio: questa funzione è quella del server sequenziale, intatta. Teniamo d'occhio `served`.

Venti client insieme

```
$ ./chatd 5000 &
$ for i in $(seq 1 20); do sleep 60 | nc 127.0.0.1 5000 & done
$ ps --ppid $(pgrep -o chatd) -o pid,stat,cmd | head -4
    PID STAT CMD
   7413 S    ./chatd 5000       <- un figlio per ogni client connesso
   7414 S    ./chatd 5000
   7415 S    ./chatd 5000
$ ss -Htn state established '( sport = :5000 )' | wc -l
20
```

Venti client, venti figli

`pgrep -o` sceglie il processo più vecchio con quel nome, cioè il genitore; `--ppid` elenca i suoi figli. Venti client, venti figli, e ciascuno servito. Allora scriviamo una riga in uno di loro: chi la vede?

Nessuno, tranne chi l'ha mandata

Il client 1 invia una riga, e altri diciannove sono connessi. Che cosa vedono? **Niente, mai**. La riga è stata letta da un figlio, e quel figlio non ha alcun descriptor verso le altre connessioni: il genitore le ha chiuse una per una, e i fratelli sono stati creati prima che la sua esistesse.

- Per lo stesso motivo `served` è inutile: ogni client si sente dire che il suo è il messaggio numero uno, perché ogni figlio ha ricevuto la propria copia al momento della `fork`.
- Abbiamo costruito venti server di eco indipendenti che per caso condividono una porta: va benissimo per un server web, è inutile per una chat.

Gli spazi di indirizzi separati sono esattamente ciò che abbiamo chiesto. Se i processi devono parlarsi, **deve portarlo il kernel**.

FAR PARLARE I PROCESSI

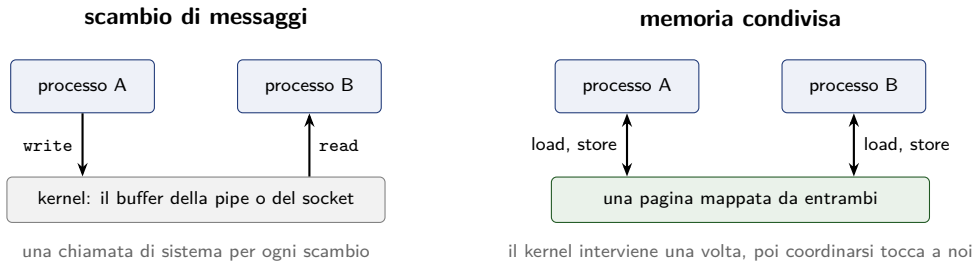


Figura 13.1 – I due modi in cui due processi si parlano. A sinistra ogni dato passa per il kernel, che ne garantisce anche l'ordine; a destra il kernel prepara la pagina e poi si fa da parte.

- **Scambio di messaggi**: ogni scambio è una chiamata di sistema, il kernel tiene i dati, e l'ordine di arrivo è una sua garanzia.
- **Memoria condivisa**: il kernel viene interpellato una volta sola, poi i due processi leggono e scrivono memoria ordinaria, e il coordinamento è **interamente loro**.
- La seconda è più veloce, e ci riporta a `count++`: la corsa critica è la stessa che hanno i thread.

La pipe

Pipe

`pipe(fd)` restituisce **due descriptor su un unico buffer del kernel**: `fd[0]` per leggere e `fd[1]` per scrivere. Dopo una `fork` entrambi i processi li hanno entrambi.

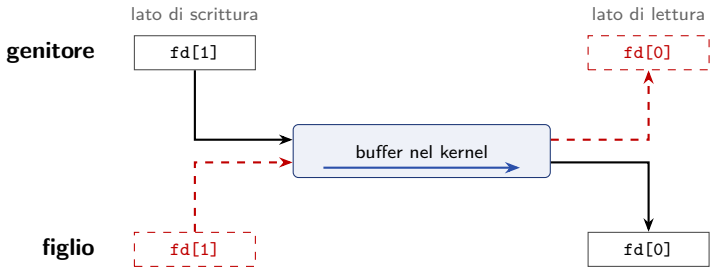


Figura 13.2 – Una pipe dopo la fork, quando il genitore scrive e il figlio legge. In rosso tratteggiato i due descriptor da chiudere: ciascun processo rinuncia al lato che non usa.

È **a senso unico** ed è **fra parenti**: l'unico modo di entrarci è ereditare un descriptor che esiste già.

```c
int fd[2];
pipe(fd);                           /* controllo d'errore omesso */
if (fork() == 0) {
    close(fd[1]);                   /* chi legge rinuncia al lato di scrittura */
    n = read(fd[0], buf, sizeof buf);
    write(STDOUT_FILENO, buf, n);
    _exit(0);
}
close(fd[0]);                       /* chi scrive rinuncia al lato di lettura */
write(fd[1], "hello\n", 6);
close(fd[1]);                       /* ora, e solo ora, il figlio vede la fine del file */
```

Chiudere i lati che non si usano

La read che non restituisce mai zero

Il genitore scrive una riga, chiude `fd[1]` e aspetta. Il figlio cicla sulla `read` e stampa. Non finisce mai. Perché? Perché il figlio **non ha chiuso il proprio** `fd[1]`. Un lato di scrittura è ancora aperto (il suo), quindi la pipe non arriva mai alla fine del file, la `read` si blocca, e il genitore aspetta un figlio che aspetta se stesso.

- La fine del file su una pipe significa **nessuno scrittore rimasto**, e il conto include la copia del lettore stesso.
- È la stessa regola del socket connesso: l'oggetto muore quando se ne va il suo ultimo descriptor, chiunque lo tenga.
- Il caso speculare è rumoroso invece che silenzioso: chi scrive in una pipe il cui lettore se n'è andato riceve `SIGPIPE`.

	L'altro lato è aperto	**L'altro lato è chiuso**
`read`, dati disponibili	restituisce quel che c'è	restituisce quel che c'è
`read`, pipe vuota	si blocca	restituisce 0
`write`, c'è posto	copia e ritorna	`SIGPIPE`, `EPIPE`
`write`, buffer pieno	si blocca	`SIGPIPE`, `EPIPE`

Tabella 13.1 – Leggere e scrivere una pipe. Un lettore vede la fine solo quando i dati sono finiti *e* non resta nessuno scrittore.

- Una `write` fino a `PIPE_BUF` byte (4096 su Linux) è **atomica**: più scrittori non possono mescolarsi al suo interno.
- È la soglia dell'atomicità, non la dimensione della pipe: il buffer contiene 64 KB, e chi scrive si blocca quando è pieno.

I figli che riferiscono al genitore

Con le pipe la chat multiprocesso si può fare: ogni figlio passa al genitore le righe che legge, e il genitore le ridistribuisce.

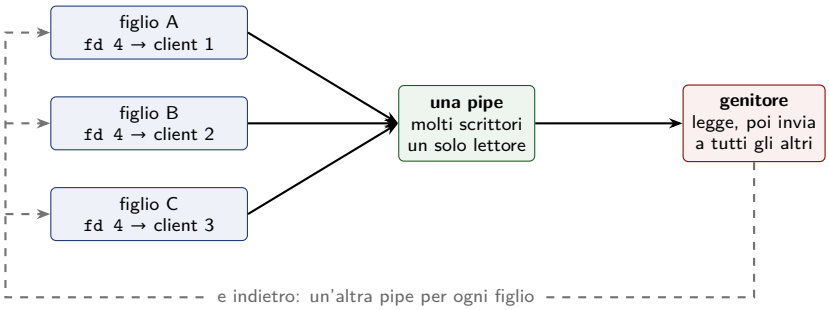


Figura 13.3 – Ogni riga attraversa il kernel due volte ed è copiata una volta per destinatario, e il genitore deve stare in un ciclo tutto suo. Con i thread la stessa cosa è un `for` su un array.

Una pipe verso l'alto basta, perché le scritture sotto `PIPE_BUF` non si mescolano; il ritorno verso il basso è la metà costosa. E il genitore, che deve leggere dalla pipe e insieme accettare nuove connessioni, si trova già con il problema del prossimo capitolo: aspettare su più descriptor alla volta.

dup e dup2

```c
int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

Due prototipi

- Entrambe creano un **secondo descriptor sullo stesso oggetto**: stesso offset, stessa connessione, un unico contatore di riferimenti.
- `dup` prende il numero libero più basso; `dup2` prende il numero che indichiamo, chiudendolo prima se era aperto.
- Ed è proprio questo il punto: si può scegliere di atterrare su **0 o 1**.

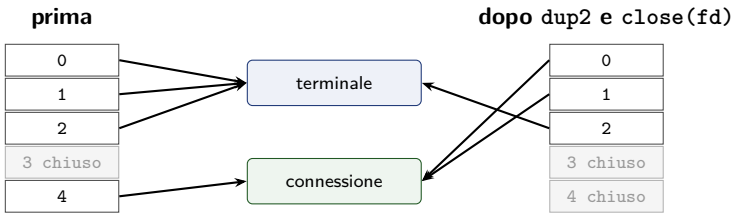


Figura 13.4 – La tabella dei descriptor del figlio. Dopo i due `dup2` lo standard input e lo standard output sono la connessione; lo standard error resta sul terminale del server, dove finiscono i messaggi diagnostici.

```c
if (fork() == 0) {
    close(listen_fd);
    dup2(fd, STDIN_FILENO);              /* la connessione diventa lo standard input */
    dup2(fd, STDOUT_FILENO);             /* ... e lo standard output */
    if (fd > STDERR_FILENO) close(fd);   /* bastano i due duplicati */
    execl("/usr/local/bin/quoted", "quoted", (char *) NULL);
    perror("execl");                     /* si arriva qui solo se exec e' fallita */
    _exit(127);
}
close(fd);
```

Un programma che non sa di essere in rete

Il programma dall'altra parte di `execl` legge con `scanf`, scrive con `printf` e non scopre mai da dove arriva il suo input. È così che funziona un servizio avviato da **inetd**, ed è così che un server web esegue un programma **CGI**.

Il buffer di stdout dopo dup2

Su un terminale `printf` svuota il buffer a ogni `\n`; su un socket no, perché lo standard output non è più un terminale e la libreria C passa al buffer pieno. Il programma che risponde una riga e poi aspetta la successiva deve chiamare `fflush(stdout)`, oppure `setvbuf(stdout, NULL, _IOLBF, 0)` all'inizio: altrimenti il client non riceve nulla finché il buffer non si riempie.

Parentela, un nome, un indirizzo

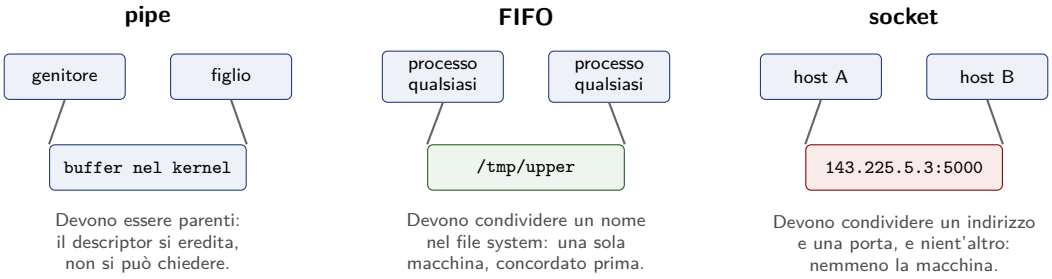


Figura 13.5 – Tre modi in cui due processi si trovano, e a ogni passo devono avere meno cose in comune.

È la stessa forma cliente-servente tre volte: uno aspetta in un posto convenuto, l'altro ci va. Cambia solo che cosa rende il posto raggiungibile.

Una FIFO: server e client

FIFO

Una **FIFO** (o *named pipe*) è una pipe con un **nome nel file system**. Si crea con `mkfifo`, si apre con `open` come un file qualsiasi, e da lì in poi si comporta esattamente come una pipe.

```c
/* server: crea il nome, poi aspetta su di esso */
mkfifo("/tmp/upper", 0666);                 /* se esiste gia' non e' un errore per noi */
int fd = open("/tmp/upper", O_RDONLY);      /* si blocca finche' uno scrittore non apre */
while ((n = read(fd, buf, sizeof buf - 1)) > 0) {
    buf[n] = '\0';
    for (int i = 0; i < n; i++) buf[i] = toupper((unsigned char) buf[i]);
    printf("%s\n", buf);
}

/* client: gli basta lo stesso nome */
int fd = open("/tmp/upper", O_WRONLY);      /* si blocca finche' un lettore non apre */
write(fd, argv[1], strlen(argv[1]));
close(fd);
```

Un server FIFO e il suo client

```
$ ./upper &
$ ls -l /tmp/upper
prw-r--r-- 1 utente utente 0 ott 19 10:02 /tmp/upper      <- p: una pipe con nome
$ ./shout ciao
CIAO
```

Il nome nel file system

Due programmi, uno che aspetta e uno che chiede, e nessuna rete: il nome nel file system fa ciò che più avanti faranno un indirizzo e una porta, e solo `unlink` lo toglie di mezzo.

- Le due `open` si aspettano a vicenda: è il primo esempio di **appuntamento** fra due processi senza parentela.
- Al primo client che chiude, la `read` del server restituisce 0 e il server finisce. Per servirne altri lo si riapre in un ciclo, oppure lo si apre anche in scrittura (`O_RDWR`): con uno scrittore sempre presente, la fine del file non arriva mai.

```c
unsigned long *served = mmap(NULL, sizeof *served,
                             PROT_READ | PROT_WRITE,
                             MAP_SHARED | MAP_ANONYMOUS, -1, 0);
if (served == MAP_FAILED) { perror("mmap"); exit(EXIT_FAILURE); }
*served = 0;

/* ogni figlio creato da qui in poi vede questo stesso contatore */
```

Un contatore che tutti i figli vedono

- `MAP_ANONYMOUS` significa nessun file dietro: una pagina di memoria che i figli ereditano **condivisa** invece di copiarla.
- `MAP_PRIVATE` al suo posto restituisce esattamente ciò che la `fork` dava già: una copia ciascuno.
- E `++*served` da due processi è la corsa critica che già conosciamo. Un mutex lì deve **vivere in questa pagina** ed essere creato con `PTHREAD_PROCESS_SHARED`.

```c
struct shared { pthread_mutex_t m; unsigned long served; };

struct shared *sh = mmap(NULL, sizeof *sh, PROT_READ | PROT_WRITE,
                         MAP_SHARED | MAP_ANONYMOUS, -1, 0);
pthread_mutexattr_t a;
pthread_mutexattr_init(&a);
pthread_mutexattr_setpshared(&a, PTHREAD_PROCESS_SHARED);
pthread_mutex_init(&sh->m, &a);           /* prima di ogni fork */
sh->served = 0;

/* in un figlio */
pthread_mutex_lock(&sh->m);
unsigned long me = ++sh->served;
pthread_mutex_unlock(&sh->m);
```

Il mutex nella pagina condivisa

Un problema che con i thread non c'era: un figlio può **morire con il lock in mano**, e allora nessuno lo riaprirà più. Un mutex creato anche con `PTHREAD_MUTEX_ROBUST` lo segnala al successivo che lo prende, con `EOWNERDEAD`, invece di bloccarlo per sempre.

Un thread per client

Stesso server, stesse chiamate, **un solo spazio di indirizzi**: i descriptor stanno in un'unica tabella e lo stato della chat è una variabile.

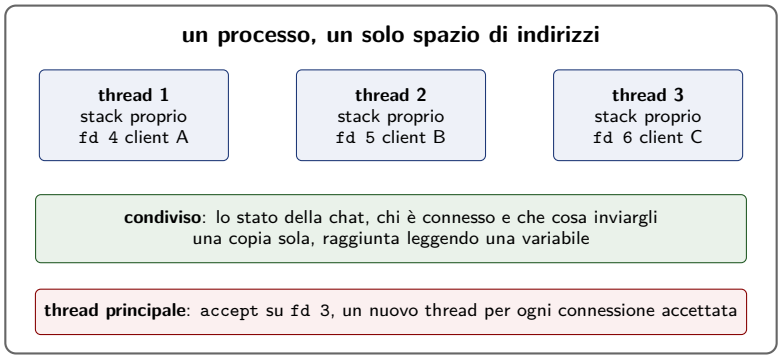


Figura 13.6 – Il server di chat con un thread per client. Ogni thread vede tutti i descriptor, quindi può scrivere a tutti i client: è ciò che mancava alla versione con i processi.

```c
int main(int argc, char *argv[]) {
    int listen_fd = listening_socket(argc > 1 ? argv[1] : "5000");
    signal(SIGPIPE, SIG_IGN);
    for (int i = 0; i < MAXC; i++) clients[i] = -1;

    for (;;) {
        int *fd = malloc(sizeof *fd);              /* l'argomento tutto suo del thread */
        if (fd == NULL) { sleep(1); continue; }
        *fd = accept(listen_fd, NULL, NULL);
        if (*fd < 0) { if (errno != EINTR) perror("accept"); free(fd); continue; }

        pthread_t tid;
        int err = pthread_create(&tid, NULL, worker, fd);
        if (err) { fprintf(stderr, "pthread_create: %s\n", strerror(err));
                   close(*fd); free(fd); continue; }
        pthread_detach(tid);                       /* nessuno lo aspettera' mai */
    }
}
```

Il ciclo di accept che avvia un thread

Lo stato che i thread condividono

```c
#define MAXC 64
static int clients[MAXC];                  /* socket connessi, -1 = posto libero */
static pthread_mutex_t roster = PTHREAD_MUTEX_INITIALIZER;

static int add_client(int fd) {
    int slot = -1;
    pthread_mutex_lock(&roster);
    for (int i = 0; i < MAXC; i++)
        if (clients[i] == -1) { clients[i] = fd; slot = i; break; }
    pthread_mutex_unlock(&roster);
    return slot;                           /* -1: la chat e' piena, si rifiuta */
}

static void del_client(int slot) {
    pthread_mutex_lock(&roster); clients[slot] = -1; pthread_mutex_unlock(&roster);
}
```

La lista dei client e il suo mutex

```c
static void *worker(void *arg) {
    int fd = *(int *) arg;
    free(arg);                             /* l'argomento era nostro: finito */

    int slot = add_client(fd);
    if (slot < 0) { write(fd, "chat is full\n", 13); close(fd); return NULL; }

    char buf[256];
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0)
        broadcast(slot, buf, n);           /* a tutti tranne me */

    del_client(slot);
    close(fd);
    return NULL;
}
```

Il worker: registrarsi, servire, cancellarsi

```c
static void broadcast(int from, const char *msg, size_t len) {
    pthread_mutex_lock(&roster);
    for (int i = 0; i < MAXC; i++)
        if (clients[i] != -1 && i != from)
            write(clients[i], msg, len);    /* EPIPE: se ne accorgera' il thread di quel client */
    pthread_mutex_unlock(&roster);
}
```

Inviare a tutti gli altri

- Questa è **tutta la differenza** dalla versione multiprocesso: un `for` su una variabile che ogni thread può vedere.
- Il mutex non è facoltativo: un altro thread potrebbe aggiungersi a `clients` mentre questo ciclo la legge.
- Ed è tenuto **durante una write**, che è proprio l'unica cosa che avevamo detto di non fare mai.

Il client lento che congela la chat

Un client smette di leggere: la sua finestra si riempie, e si riempie anche il buffer del suo socket. Che cosa succede agli altri diciannove? La `write` verso quel client **si blocca**, con il mutex `roster` in mano. Ogni altro thread si ferma al suo `broadcast` successivo: un solo client silenzioso ha fermato l'intera chat.

Rilasciare il lock prima di scrivere non lo risolve: fra la copia della lista e la `write` quel posto può essere chiuso, e il suo numero di descriptor riusato da un nuovo client.

Passo	Thread A (broadcast)	Thread B	Thread principale
1	copia la lista: posto 3 = `fd 7` rilascia il lock		
2		il client 3 se ne va: `del_client(3)`, `close(7)`	
3			`accept` restituisce di nuovo 7
4	`write(7, ...)`: al nuovo arrivato		

Tabella 13.2 – Perché non basta copiare la lista e scrivere senza lock. Il kernel riassegna sempre il descriptor libero più basso, quindi il riuso non è un caso raro: la riga va alla connessione sbagliata, di rado e mai mentre si guarda.

Nessuna delle due strade è la risposta. Un server vero **non si blocca mai in un worker**: ogni client ha una **coda in uscita**, e qualcuno la svuota quando il socket è pronto. È l'argomento del prossimo capitolo.

PROCESSI O THREAD, PER UN SERVER

	Un processo per client	Un thread per client
stato condiviso	deve portarlo il kernel	una variabile
il crash di un client	uccide il suo figlio	uccide il server
un descriptor dimenticato	se ne va con il figlio	resta per sempre
costo di avvio	uno spazio di indirizzi	uno stack
`close` su un socket	chiude solo quella copia	lo chiude per tutti
debugging	un flusso, molti processi	un processo, molti flussi

Tabella 13.3 – I processi pagano per essere isolati; i thread pagano per condividere. Una chat vuole condividere, un server che esegue codice non fidato vuole l'isolamento.

Un worker dereferenzia un puntatore nullo

Un bug nel codice che serve un client: quanto costa? **Con i processi**: muore la connessione di quel client, il genitore raccoglie un figlio terminato da un segnale, e diciannove conversazioni continuano. **Con i thread**: il processo riceve `SIGSEGV`, e ogni connessione muore con lui.

- La stessa asimmetria vale per un buffer perso, uno heap corrotto, un descriptor che nessuno chiude: con i processi tutto finisce a `_exit`.
- È per questo che la scelta si fa ancora in entrambi i modi: Apache offre un modello a processi e uno a thread, e quello a processi si sceglie quando il codice non è fidato.

Quanto costa «uno per client»

- Entrambe le versioni pagano **per connessione**, al momento della connessione: uno spazio di indirizzi o uno stack, creati e distrutti per un client che magari dice una parola.
- Entrambe mantengono quel costo finché il client resta, che parli o no; e un client di chat tace quasi sempre.
- Diecimila connessioni inattive significano diecimila stack e diecimila oggetti del kernel, per non fare nulla.
- E una raffica di connessioni brevi passa più tempo a creare e distruggere che a servire.

Entrambi i limiti indicano la stessa direzione: smettere di creare **un flusso di controllo per client**. Ne serve uno che osserva molte connessioni e si accorge di quali sono pronte, più un insieme fisso di worker tenuti occupati da una coda.

Esercizio 1 — Una pipeline fatta a mano

Scrivere un programma che esegua l'equivalente di `ls | wc -l` usando soltanto `pipe`, `fork`, `dup2`, `execlp` e `wait`. Poi togliere dal genitore la `close(fd[1])`: che cosa succede, e perché?

Svolgimento

Due figli, uno per programma: il primo mette il lato di scrittura sul proprio standard output, il secondo il lato di lettura sul proprio standard input. Poi ciascuno chiude **entrambi** gli originali, perché i duplicati bastano.

```c
int fd[2];
if (pipe(fd) < 0) { perror("pipe"); exit(EXIT_FAILURE); }

if (fork() == 0) {                      /* primo figlio: ls */
    dup2(fd[1], STDOUT_FILENO);
    close(fd[0]); close(fd[1]);
    execlp("ls", "ls", (char *) NULL);
    perror("execlp"); _exit(127);
}
if (fork() == 0) {                      /* secondo figlio: wc -l */
    dup2(fd[0], STDIN_FILENO);
    close(fd[0]); close(fd[1]);
    execlp("wc", "wc", "-l", (char *) NULL);
    perror("execlp"); _exit(127);
}
close(fd[0]); close(fd[1]);             /* il genitore non usa la pipe */
while (wait(NULL) > 0)
    ;
```

pipeline.c

Senza la `close(fd[1])` nel genitore, `ls` stampa, esce, e `wc` **non termina mai**: un lato di scrittura è ancora aperto (quello del genitore), quindi la sua `read` non restituisce mai 0. E il genitore, fermo in `wait`, aspetta un figlio che aspetta lui. È il caso del riquadro sulla `read` che non restituisce mai zero, con tre processi invece di due. La shell fa esattamente queste chiamate per ogni `|`.

Esercizio 2 — Iterazione 2: una chat multiprocesso

Si riparte dal server sequenziale: corretto, e inutilizzabile appena si connette un secondo client, quello lasciato ad aspettare in `ss -tn`.

- **Che cosa cambia**: solo il ciclo di `accept`. La `fork` dopo `accept`, le due `close`, `_exit` nel figlio, e un handler di `SIGCHLD` installato prima del primo client.
- **Che cosa no**: `serve()` resta intatta. Se ci si ritrova a modificarla, si sta risolvendo il problema sbagliato.

Connettere venti client e poi ucciderne uno. Dimostrare entrambe le cose: venti figli, e nessuno zombie. E che cosa dice il contatore al ventesimo client?

Svolgimento

Il `main` è quello del capitolo: `reap` installato con `sigaction`, `SIGPIPE` ignorato, e il ciclo con la `fork`. La prova si fa da un secondo terminale:

```
$ ./chatd 5000 &
$ for i in $(seq 1 20); do sleep 60 | nc 127.0.0.1 5000 & done
$ ps --ppid $(pgrep -o chatd) --no-headers | wc -l
20
$ kill %2                                   <- termina uno dei client
$ ps --ppid $(pgrep -o chatd) --no-headers | wc -l
19
$ ps --ppid $(pgrep -o chatd) -o stat --no-headers | grep -c Z
0
```

Venti figli, nessuno zombie

Quando il client muore, la `read` del suo figlio restituisce 0, il figlio esce, e il `SIGCHLD` lo fa raccogliere subito dal genitore: i figli scendono a diciannove e nessuno resta `Z`. Togliendo la `sigaction`, lo stesso comando mostrerebbe un `Z` per ogni client che se n'è andato.

Il ventesimo client si sente rispondere `1:` alla sua prima riga, come tutti gli altri. Nell'iterazione 1 il contatore era locale a `serve`; ma anche spostandolo in una globale non cambierebbe nulla, perché ogni figlio ne riceve una copia al momento della `fork`. Per un contatore comune servirebbe la pagina `mmap` con il suo mutex.

Esercizio 3 — Iterazione 3: la stessa chat con i thread

Si riparte dall'iterazione 2, che serve tutti e non avvisa nessuno: una riga scritta da un client arriva solo al figlio che l'ha letta.

- **Che cosa cambia**: `pthread_create` e `pthread_detach` nel ciclo, una `malloc` per connessione per l'argomento, e il worker del capitolo.
- **Che cosa è nuovo**: la lista dei client e il suo mutex, perché una riga di un client arrivi a tutti gli altri. È la prima versione che è davvero una chat.

Poi misurare la memoria per client connesso, con 50 client, in entrambe le versioni. Quale costa di più per client, e quale si sceglierebbe comunque se `serve()` eseguisse codice scritto da altri?

Svolgimento

Il worker del capitolo inoltra quello che `read` restituisce, che non è ancora una riga. Si recupera il buffer di riga dell'iterazione 1, e `broadcast` parte solo a riga completa:

```c
static void *worker(void *arg) {
    int fd = *(int *) arg;
    free(arg);
    int slot = add_client(fd);
    if (slot < 0) { write(fd, "chat is full\n", 13); close(fd); return NULL; }

    char buf[512], line[512];
    size_t len = 0;
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0)
        for (ssize_t i = 0; i < n; i++) {
            if (len < sizeof line - 1) line[len++] = buf[i];
            if (buf[i] == '\n') { broadcast(slot, line, len); len = 0; }
        }
    del_client(slot);
    close(fd);
    return NULL;
}
```

chatd.c: il worker inoltra righe intere

```
$ for i in $(seq 1 50); do sleep 300 | nc 127.0.0.1 5000 & done
$ ps -o pid,nlwp,rss,cmd -p $(pgrep -o chatd)          <- iterazione 3: un processo, 51 thread
$ ps -o rss= --ppid $(pgrep -o chatd) | paste -sd+ | bc  <- iterazione 2: somma dei 50 figli
```

La misura

Con i thread la memoria residente cresce di poco per client: ogni stack riserva 8 MB di spazio virtuale, ma diventano memoria vera solo le poche pagine che il thread tocca. Con i processi ogni figlio ha la propria tabella delle pagine, le proprie strutture nel kernel e le proprie pagine di stack e di dati; il codice e le librerie restano condivise, ed è per questo che **sommare l'RSS dei figli sovrastima**: conta le pagine condivise una volta per figlio. La misura onesta è il PSS, in `/proc/PID/smaps_rollup`, che divide ogni pagina condivisa fra chi la usa. Anche così, il processo costa più del thread.

Ma se `serve()` eseguisse codice di altri si sceglierebbero comunque i **processi**: un bug in quel codice ucciderebbe un figlio solo, invece dell'intero server.

Riepilogo

In breve

- Un **server concorrente** è lo stesso servizio in un ciclo diverso: `fork` o `pthread_create` subito dopo `accept`, e il genitore o il thread principale di nuovo in `accept`.
- Con i processi il prezzo si paga subito: i figli non si vedono, quindi ogni cosa condivisa attraversa il kernel (una pipe, o una pagina di `mmap` con un lock dentro).
- Una **pipe** unisce parenti, una **FIFO** chiunque abbia il nome, un **socket** chiunque abbia l'indirizzo: la stessa forma cliente-servente tre volte.
- `dup2` mette una connessione sui descriptor 0 e 1, e un programma che non sa nulla di reti diventa un servizio di rete.
- Con i thread la condivisione è gratuita e la sicurezza no: la lista dei client vuole un mutex, e tenerlo durante una `write` consegna l'intera chat al client più lento.
- Isolamento contro condivisione, e un costo che nessuno dei due evita: **un flusso di controllo per connessione**.

Multiplexing dell'I/O, server a eventi e pool di thread

Due versioni della stessa chat (un processo per client, un thread per client) e un costo che nessuna delle due evita.

- Entrambe pagano al momento della connessione e continuano a pagare: uno spazio di indirizzi o uno stack, per un client che tace quasi sempre.
- E abbiamo lasciato una questione aperta: una `write` verso un client lento congela tutta la chat, perché il lock della lista viene tenuto durante la scrittura.
- Qui il flusso di controllo smette di essere uno per client. **Uno solo osserva molte connessioni** e serve quelle pronte; un **insieme fisso di worker** è tenuto occupato da una coda.
- Poi l'altra direzione: un datagramma, molti destinatari, e nessuno che tiene una lista.

Aspettare su più di un descriptor

- `read` aspetta su un descriptor. Mentre aspetta lì, tutto il resto che il programma potrebbe vedere gli è invisibile.
- È l'unica ragione per cui avevamo un flusso di controllo per connessione: il processo, o il thread, esisteva **per potersi bloccare**.
- `select` capovolge la cosa: ci si blocca una volta sola, **su un insieme di descriptor**, e si torna appena uno qualsiasi può essere servito senza bloccarsi.
- Che cosa compra: nessun thread per socket, nessun ciclo di polling che brucia il processore, e un solo posto in cui vive lo stato di ogni client.
- Che cosa non compra: il **parallelismo**. Un flusso di controllo fa ancora una cosa alla volta, su un core.

Multiplexing dell'I/O

Si parla di **multiplexing dell'I/O** quando un solo flusso di controllo attende **contemporaneamente** su più descriptor, e il kernel gli dice quali sono pronti. In POSIX la chiamata è `select`; `poll` ed `epoll` sono varianti della stessa idea.

```c
#include <sys/select.h>

int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
           struct timeval *timeout);
```

select

- Si blocca finché almeno un descriptor degli insiemi può essere usato, oppure scade il timeout, oppure arriva un segnale.
- Restituisce -1 ed `errno` in caso di errore, 0 se il timeout è scaduto senza nulla di pronto, altrimenti **quanti descriptor sono pronti**, contati su tutti e tre gli insiemi.
- `nfds` non è un conteggio: è il **descriptor più alto più uno**, perché il kernel scandisce l'intervallo da 0 a `nfds-1`.

Gli insiemi di descriptor

Un `fd_set` è una **maschera di bit**: un bit per ogni numero di descriptor, e quattro macro per toccarla.

Macro	Che cosa fa
`FD_ZERO(fd_set *set)`	svuota l'insieme: si comincia sempre da qui
`FD_SET(int fd, fd_set *set)`	mette `fd` nell'insieme
`FD_CLR(int fd, fd_set *set)`	toglie `fd` dall'insieme
`FD_ISSET(int fd, fd_set *set)`	`fd` è nell'insieme? Si usa dopo la chiamata

Tabella 14.1 – Le quattro macro sugli insiemi di descriptor.

- `readfds` per i descriptor da cui si vuole leggere, `writefds` per quelli su cui si vuole scrivere; `NULL` significa non osservare nulla di quel tipo.
- La maschera ha dimensione fissa: `FD_SETSIZE` vale 1024 su Linux, e un descriptor uguale o superiore non può proprio entrare in un insieme.

Quando un descriptor è pronto

Pronto in lettura: una `read` tornerebbe subito	**Pronto in scrittura**: una `write` non si bloccherebbe
sono arrivati byte nel buffer di ricezione	c'è posto nel buffer di invio (`SO_SNDLOWAT`, 2048 byte per default) e il socket è connesso
il peer ha chiuso il suo lato di scrittura, quindi `read` restituisce 0	una `connect` non bloccante è terminata, in un modo o nell'altro
è un socket di ascolto e c'è una connessione in attesa	il peer ha chiuso il suo lato di lettura: la `write` torna con `EPIPE`
c'è un errore in sospeso sul socket	c'è un errore in sospeso sul socket

Tabella 14.2 – Le condizioni che rendono pronto un socket.

Pronto significa **non si bloccherà**, non *riuscirà*: un errore è un'ottima ragione per essere pronti. `exceptfds` è quasi sempre `NULL`: su Linux l'unica cosa che segnala sono i dati fuori banda.

Il timeout

```c
struct timeval {
    time_t      tv_sec;     /* secondi */
    suseconds_t tv_usec;    /* microsecondi, non millisecondi */
};

struct timeval tv   = { .tv_sec = 2, .tv_usec = 500000 };    /* 2,5 secondi */
struct timeval zero = { .tv_sec = 0, .tv_usec = 0 };

select(nfds, &rset, NULL, NULL, NULL);      /* per sempre */
select(nfds, &rset, NULL, NULL, &zero);     /* polling: torna subito */
select(nfds, &rset, NULL, NULL, &tv);       /* al massimo 2,5 s */
```

Tre modi di aspettare

Su Linux `select` riscrive in quella struttura il tempo che **non** ha usato: dentro un ciclo va riempita di nuovo prima di ogni chiamata, altrimenti l'attesa si riduce fino a niente.

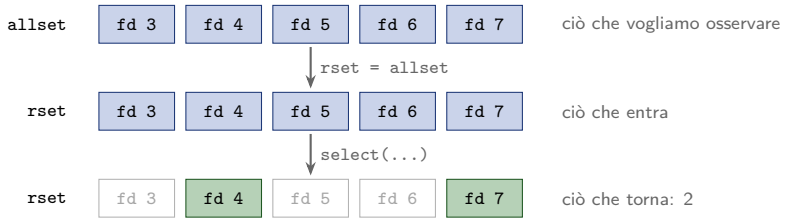


Figura 14.1 – Gli insiemi sono argomenti *e* risultati. I tre descriptor non pronti sono stati azzerati: un insieme costruito una volta, fuori dal ciclo, aspetta la seconda volta solo quelli pronti la prima, e dopo ancora niente.

L'insieme costruito una volta sola

Si costruisce l'insieme una volta, prima del ciclo, e si chiama `select` nel ciclo. Che cosa aspetta la seconda chiamata? **Solo i descriptor che erano pronti la prima volta**. `select` sovrascrive l'insieme con il sottoinsieme pronto e azzera tutto il resto. La cura è tenere un insieme *principale* e passarne una **copia** a ogni giro, oppure ricostruirlo da capo ogni volta.

Un client che ascolta due descriptor

Il client che non sente il server

Il client legge una riga, la invia e legge la risposta. Il server però parla anche di sua iniziativa: una riga ogni dieci secondi. Quando la vede l'utente? **Quando scrive qualcosa**. Il client è fermo nella `read` sulla tastiera, e le righe del server si accumulano nel buffer del socket finché non torna.

- Non è un bug del server: il client può essere bloccato in uno solo dei due posti, e ha scelto quello sbagliato.
- Un secondo thread lo risolverebbe: di nuovo uno per descriptor, con uno schermo condiviso su cui scrivere.

Per due descriptor basta **una chiamata**.

```c
static void chat_client(int fd) {
    char buf[256];
    fd_set rset;
    int nfds = (fd > STDIN_FILENO ? fd : STDIN_FILENO) + 1;

    for (;;) {
        FD_ZERO(&rset);                          /* ricostruito a ogni giro */
        FD_SET(STDIN_FILENO, &rset);
        FD_SET(fd, &rset);

        if (select(nfds, &rset, NULL, NULL, NULL) < 0) {
            if (errno == EINTR) continue;        /* un segnale, non un errore */
            perror("select"); return;
        }
        if (FD_ISSET(STDIN_FILENO, &rset)) {     /* l'utente ha scritto */
            ssize_t n = read(STDIN_FILENO, buf, sizeof buf);
            if (n <= 0) return;                  /* fine dell'input: abbiamo finito */
            if (write(fd, buf, n) != n) { perror("write"); return; }
        }
        if (FD_ISSET(fd, &rset)) {               /* ha parlato il server */
            ssize_t n = read(fd, buf, sizeof buf);
            if (n == 0) { fprintf(stderr, "server closed\n"); return; }
            if (n < 0) { perror("read"); return; }
            write(STDOUT_FILENO, buf, n);
        }
    }
}
```

Osservare la tastiera e il socket

- Due descriptor, un'attesa: il client ora è bloccato **in entrambi i posti insieme**, ed è tutto il punto.
- Due `if`, non un `if/else`: `select` può segnalarli entrambi, e il secondo resterebbe in attesa fino alla prossima riga digitata.

UN SOLO FLUSSO DI CONTROLLO, MOLTE CONNESSIONI

Lo stesso server, con il ciclo rovesciato: non aspetta più un client, aspetta **quello che parla per primo**.

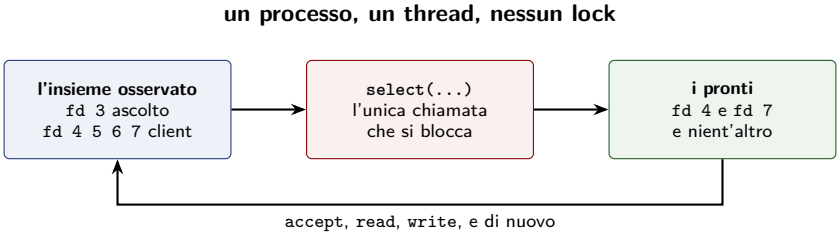


Figura 14.2 – Ogni client è un posto in un array e un bit in un insieme. Non c'è nulla da proteggere con un lock, perché nient'altro è in esecuzione; e nulla sulla via del ritorno può aspettare, perché tutti aspetterebbero con lui.

Server guidato dagli eventi

Un server **guidato dagli eventi** (*event-driven*) ha un solo ciclo, il **ciclo degli eventi**: si blocca in `select`, poi esegue per ogni descriptor pronto un pezzo di lavoro breve e che non si blocca mai, e torna in `select`. Lo stato di ogni client sta in strutture dati, non sullo stack di un thread.

```c
#define MAXC 64
static int clients[MAXC];                   /* socket connessi, -1 = posto libero */

int main(int argc, char *argv[]) {
    int listen_fd = listening_socket(argc > 1 ? argv[1] : "5000");
    fd_set allset, rset;                    /* l'insieme principale, e la copia */
    int maxfd = listen_fd;

    for (int i = 0; i < MAXC; i++) clients[i] = -1;
    FD_ZERO(&allset);
    FD_SET(listen_fd, &allset);             /* la porta e' un descriptor come gli altri */

    for (;;) {
        rset = allset;                      /* assegnamento di struct: una copia */
        int ready = select(maxfd + 1, &rset, NULL, NULL, NULL);
        if (ready < 0) { if (errno == EINTR) continue; perror("select"); break; }
```

Lo stato del server

La stessa lista di client della versione con i thread, **senza il mutex**: non c'è nient'altro in esecuzione.

```c
if (FD_ISSET(listen_fd, &rset)) {
    int fd = accept(listen_fd, NULL, NULL);
    if (fd < 0) { perror("accept"); continue; }

    int slot = -1;
    for (int i = 0; i < MAXC; i++)
        if (clients[i] == -1) { clients[i] = fd; slot = i; break; }

    if (slot < 0) { write(fd, "chat is full\n", 13); close(fd); }
    else {
        FD_SET(fd, &allset);          /* osservato dal prossimo giro */
        if (fd > maxfd) maxfd = fd;
    }
    if (--ready == 0) continue;       /* nient'altro era pronto */
}
```

Arriva una connessione sul socket di ascolto

Un socket di ascolto leggibile significa una connessione in attesa, quindi questa `accept` ritorna subito.

```
for (int i = 0; i < MAXC && ready > 0; i++) {
    int fd = clients[i];
    if (fd == -1 || !FD_ISSET(fd, &rset)) continue;
    --ready;

    char buf[256];
    ssize_t n = read(fd, buf, sizeof buf);
    if (n > 0) {
        broadcast(i, buf, n);        /* tutti tranne i */
    } else {                         /* 0: se n'e' andato. <0: si e' rotto */
        FD_CLR(fd, &allset);
        close(fd);
        clients[i] = -1;
    }
}
```

Servire le connessioni pronte

- `broadcast` è il `for` della versione con i thread, meno il mutex.
- `maxfd` non si abbassa quando un client se ne va: costa una scansione un po' più lunga, e l'alternativa è una ricerca a ogni uscita.
- `ready` serve a fermarsi presto: quando sono stati serviti tanti descriptor quanti ne ha contati `select`, il resto dell'array non va guardato.

In questo ciclo nulla può bloccarsi

Mezza riga, un byte al secondo

Un client invia mezza riga, un byte alla volta, un byte al secondo. La `read` ha restituito sei byte e la riga non è finita. E adesso? Si prendono i sei byte, si mettono **nel buffer di quel client** e si torna in `select`. Aspettare il resto vorrebbe dire aspettare un client con il tempo di tutti gli altri.

- Ogni client quindi ha bisogno del proprio **buffer di ingresso**: il ciclo tiene lo stato che un thread teneva sul suo stack.
- E *pronto* non è una promessa: fra la `select` e l'`accept` il client può mandare un reset, e l'`accept` si blocca, con tutto il server dentro.
- Da qui i descriptor **non bloccanti**: `EAGAIN` invece di un'attesa, e il ciclo va avanti.

```
int flags = fcntl(fd, F_GETFL, 0);
fcntl(fd, F_SETFL, flags | O_NONBLOCK);      /* da ora: mai un'attesa */

ssize_t n = read(fd, buf, sizeof buf);
if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK))
    ;                                        /* niente per ora: si torna in select */

n = recv(fd, buf, sizeof buf, MSG_DONTWAIT); /* lo stesso, per una sola chiamata */
```

Un descriptor non bloccante

Il client lento, finalmente

```c
static struct { char buf[4096]; size_t len; } out[MAXC];   /* byte in attesa, per client */
static fd_set wall;                          /* chi ha ancora qualcosa da inviare */

static void queue(int i, const char *msg, size_t n) {      /* non si blocca mai */
    if (out[i].len + n > sizeof out[i].buf) { drop(i); return; }  /* troppo lento: via */
    memcpy(out[i].buf + out[i].len, msg, n);
    out[i].len += n;
    FD_SET(clients[i], &wall);               /* avvisami quando puo' ricevere byte */
}

/* e nel ciclo, per ogni i che select ha segnalato scrivibile */
ssize_t w = write(clients[i], out[i].buf, out[i].len);
if (w > 0) { out[i].len -= w; memmove(out[i].buf, out[i].buf + w, out[i].len); }
if (out[i].len == 0) FD_CLR(clients[i], &wall);            /* niente da inviare: basta chiedere */
```

Una coda in uscita per client

Ora `select` riceve anche un insieme di scrittura. `broadcast` non scrive più: **accoda**. Il client lento riempie il proprio buffer e viene scartato quando trabocca, e nessun altro lo aspetta.

Perché un descriptor entra in wall solo se ha qualcosa

Un socket connesso è scrivibile quasi sempre, perché il suo buffer di invio è quasi sempre vuoto. Se tutti i client stessero sempre nell'insieme di scrittura, `select` tornerebbe subito a ogni giro: il server girerebbe al 100% di CPU senza fare nulla. Un descriptor entra nell'insieme di scrittura quando la sua coda si riempie e ne esce quando si svuota.

Quanto costa select

- Gli insiemi sono copiati nel kernel a ogni chiamata, e il kernel percorre tutto l'intervallo da 0 a `nfds-1`: il costo segue il **descriptor più alto**, non il numero di quelli pronti.
- Al ritorno si percorre il proprio array per trovare chi è pronto: un secondo passaggio lineare su client che per lo più non hanno detto niente.
- E `FD_SETSIZE` vale 1024: un server con diecimila connessioni non può nemmeno nominare i propri descriptor in un insieme.
- `poll` prende un array invece di una maschera: nessun limite fisso, ma ancora lineare. `epoll` su Linux e `kqueue` sui BSD **registrano i descriptor una volta sola** e restituiscono solo quelli pronti.

```c
int ep = epoll_create1(0);
struct epoll_event ev = { .events = EPOLLIN, .data.fd = listen_fd };
epoll_ctl(ep, EPOLL_CTL_ADD, listen_fd, &ev);          /* registrato una volta */

struct epoll_event ready[64];
for (;;) {
    int n = epoll_wait(ep, ready, 64, -1);             /* -1: per sempre */
    for (int k = 0; k < n; k++) {                      /* solo i pronti, nessuna scansione */
        int fd = ready[k].data.fd;
        if (fd == listen_fd) { /* accept, poi EPOLL_CTL_ADD del nuovo fd */ }
        else                 { /* read, oppure EPOLL_CTL_DEL e close */ }
    }
}
```

Lo stesso ciclo con epoll

La forma del ciclo non cambia: cambia la chiamata nel mezzo. Tutto quello che vale per `select` vale ancora.

Un insieme fisso di worker

Il ciclo degli eventi ha eliminato il flusso di controllo per client e ne ha lasciato uno per tutti. Un **pool** ne tiene diversi, ma in numero fisso, creati prima che arrivi il primo client.

Pool di thread

Un **pool di thread** è un insieme di n thread creati all'avvio, che prendono il lavoro da una **coda limitata** e, finito un lavoro, tornano a prenderne un altro. Chi produce il lavoro lo accoda e basta.

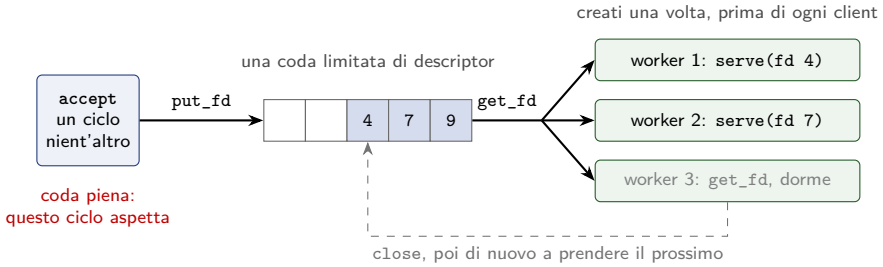


Figura 14.3 – Una connessione costa un inserimento in coda e un risveglio: quando arriva un client non si crea niente, e niente si distrugge.

```c
#define QSIZE 128

typedef struct {
    int fd[QSIZE];                          /* descriptor invece di interi */
    int in, out, count;
    pthread_mutex_t mtx;
    pthread_cond_t not_full, not_empty;     /* le stesse due attese di prima */
} queue_t;

void put_fd(queue_t *q, int fd);            /* si blocca finche' la coda e' piena */
int  get_fd(queue_t *q);                    /* si blocca finche' e' vuota */
```

La coda è il buffer limitato, con dentro descriptor

È il buffer produttore-consumatore, intatto: un mutex sullo stato, una variabile di condizione per ogni cosa che qualcuno aspetta. Quello che lo attraversa ora è una connessione.

```c
int main(int argc, char *argv[]) {
    int listen_fd = listening_socket(argc > 1 ? argv[1] : "5000");
    queue_t q; queue_init(&q);
    pthread_t tid;

    for (int i = 0; i < NWORKERS; i++)      /* una volta, prima di ogni client */
        if (pthread_create(&tid, NULL, worker, &q) == 0) pthread_detach(tid);

    for (;;) {
        int fd = accept(listen_fd, NULL, NULL);
        if (fd < 0) { if (errno == EINTR) continue; perror("accept"); continue; }
        put_fd(&q, fd);                     /* l'unica cosa che fa il ciclo */
    }
}
```

Il ciclo di accept è il produttore

Niente `fork`, niente `pthread_create` nel ciclo, niente `malloc` per un argomento: una connessione costa un inserimento in coda.

```c
static void *worker(void *arg) {
    queue_t *q = arg;

    for (;;) {
        int fd = get_fd(q);                 /* dorme qui finche' non c'e' niente */
        if (fd == STOP) break;              /* la pillola avvelenata: una per worker */

        serve(fd);                          /* la funzione del server sequenziale */
        close(fd);
    }
    return NULL;
}
```

Il worker è il consumatore

Il thread c'era prima del client e c'è dopo: per connessione non si crea né si distrugge niente. E `serve()` è passata ormai per tre cicli diversi senza essere toccata.

Quanti worker, e che cosa succede quando la coda si riempie

- Il numero non è «quanti sono i client»: è **quanti possono lavorare insieme**. Circa quanti sono i core per un lavoro che calcola, di più quando i worker passano il tempo bloccati in `read`.
- La coda è limitata **di proposito**. Quando si riempie, `put_fd` blocca il ciclo di `accept`; le connessioni si accumulano allora nella coda di listen, e quando si riempie anche quella il kernel le rifiuta.
- Una coda illimitata sposta solo il guasto più in profondità: accetta tutti e non risponde a nessuno, e il client ha rinunciato molto prima che arrivi il suo turno.

Contropressione

La catena coda piena → `accept` ferma → coda di listen piena → connessioni rifiutate è la **contropressione** (*back pressure*): un server che non riesce a stare al passo lo dice, alla porta, invece di allocare memoria finché non muore.

Il numero da misurare quindi non è il throughput, ma **quanto aspetta una richiesta** prima che un worker la prenda.

Tutti e sedici i worker parlano con qualcuno

Sedici worker, sedici client di chat connessi e silenziosi. Si connette il diciassettesimo: quando viene servito? **Mai**. Ogni worker è dentro `serve()`, bloccato in `read` sul proprio client, e `serve()` ritorna solo quando quel client se ne va. La coda non è piena: semplicemente non viene mai letta.

- Un pool il cui lavoro è una connessione serve tanti client quanti worker ha. È la forma giusta quando il lavoro **finisce**: una richiesta HTTP, una query, un file da inviare.
- Per connessioni che durano, o si tiene un flusso per client limitandone il numero (il semaforo contatore già visto), oppure il lavoro diventa **il messaggio**.
- Il che richiede qualcuno che si accorga che un messaggio è arrivato: il ciclo di prima. Uno che osserva, un pool che lavora.

	Uno per client	Un ciclo	Pool fisso
costo di un nuovo client	uno stack	un posto e un bit	un inserimento in coda
quanti insieme	migliaia	decine di migliaia	quanti sono i worker
una chiamata bloccante	ferma quel client	ferma tutti	occupa un worker
stato condiviso	vuole un mutex	non vuole niente	vuole un mutex
usa più core	sì	no	sì

Tabella 14.3 – Le tre forme di server a confronto.

Non restano alternative a lungo: un server reale esegue **un ciclo per core**, ciascuno con un pool alle spalle per il lavoro che non deve fermare il ciclo.

UN MITTENTE, MOLTI DESTINATARI

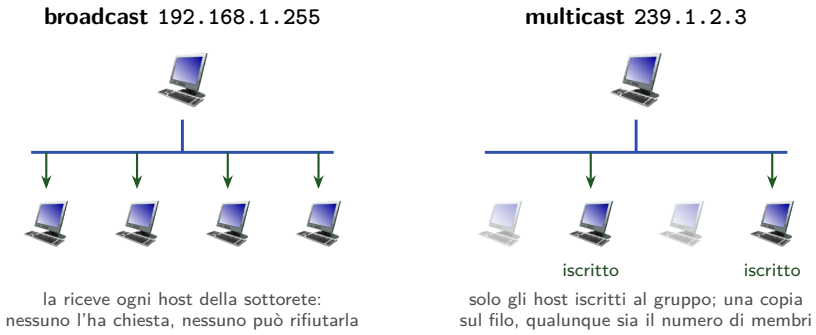


Figura 14.4 – Entrambi sono datagrammi: non esiste una connessione con molti peer, quindi niente TCP. E solo IPv4: IPv6 ha eliminato il broadcast e ne affida i compiti al multicast.

```c
/* il mittente */
int fd = socket(AF_INET, SOCK_DGRAM, 0);
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_BROADCAST, &yes, sizeof yes);   /* spento per default */
to.sin_family = AF_INET;
to.sin_port   = htons(5000);
inet_pton(AF_INET, "192.168.1.255", &to.sin_addr);  /* il broadcast di questa sottorete */
sendto(fd, msg, len, 0, (struct sockaddr *) &to, sizeof to);

/* il ricevente: un normale server UDP */
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
me.sin_family      = AF_INET;
me.sin_addr.s_addr = htonl(INADDR_ANY);             /* non l'indirizzo di broadcast */
me.sin_port        = htons(5000);                   /* la stessa porta del mittente */
bind(fd, (struct sockaddr *) &me, sizeof me);
recvfrom(fd, buf, sizeof buf, 0, (struct sockaddr *) &from, &flen);
```

Broadcast: i due estremi

`SO_BROADCAST` esiste perché nessuno inondi una LAN sbagliando a scrivere un indirizzo: bisogna dire che lo si voleva davvero. Senza, la `sendto` verso un indirizzo di broadcast fallisce con `EACCES`.

Il broadcast che non arriva

Il programma gira, `sendto` restituisce il numero di byte inviati, e nessuno riceve niente. Dove è finito? **Fuori dal socket, e non oltre**. `sendto` riferisce ciò che ha lasciato la nostra macchina: con UDP è tutto ciò che può promettere.

- `255.255.255.255` è il **broadcast limitato**: non lascia mai l'interfaccia da cui è inviato. I router non inoltrano affatto il broadcast: si ferma alla sottorete, per progetto.
- Il broadcast di sottorete deve essere **quello di quella sottorete**: va letto, non indovinato.
- Una macchina virtuale in modalità NAT sta su una rete privata tutta sua, e quel NAT non inoltra il broadcast. In modalità bridge prende un indirizzo sulla LAN vera e l'aula lo riceve, se il driver del Wi-Fi permette il bridging.

```
$ ip addr show ens33 | grep inet
    inet 192.168.1.23/24 brd 192.168.1.255 scope global ens33     <- brd: quello giusto
```

L'indirizzo di broadcast, letto e non indovinato

Gli indirizzi multicast

Gruppo multicast

Un **gruppo multicast** è un indirizzo IPv4 fra `224.0.0.0` e `239.255.255.255`. Nomina **una conversazione, non un host**: nessuno lo possiede, e qualsiasi host può iscriversi.

Intervallo	Uso
`224.0.0.0/24`	link-local: mai inoltrato; qui vivono i protocolli di routing
`224.0.1.0 -- 238.255.255.255`	ambito globale: richiede router configurati per questo
`239.0.0.0/8`	ambito amministrativo: locale a un'organizzazione

Tabella 14.4 – Gli ambiti degli indirizzi multicast IPv4.

Per qualsiasi cosa nostra si usa `239.x.y.z`. L'altro limite a quanto lontano viaggia un datagramma è il suo **TTL**, che per il multicast il mittente sceglie esplicitamente.

Entrare in un gruppo

```c
int fd = socket(AF_INET, SOCK_DGRAM, 0);
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &yes, sizeof yes);

me.sin_family      = AF_INET;
me.sin_addr.s_addr = htonl(INADDR_ANY);
me.sin_port        = htons(5000);
bind(fd, (struct sockaddr *) &me, sizeof me);          /* la porta, non il gruppo */

struct ip_mreq mreq;
inet_pton(AF_INET, "239.1.2.3", &mreq.imr_multiaddr); /* quale gruppo */
mreq.imr_interface.s_addr = htonl(INADDR_ANY);         /* su quale interfaccia */
setsockopt(fd, IPPROTO_IP, IP_ADD_MEMBERSHIP, &mreq, sizeof mreq);
```

Iscriversi a 239.1.2.3 sulla porta 5000

- `bind` sceglie la porta, l'iscrizione sceglie il gruppo: senza di essa il kernel butta via i datagrammi anche quando arrivano sul filo.
- `SO_REUSEADDR` riusa una porta lasciata da un socket precedente; `SO_REUSEPORT` permette a più socket vivi di tenere la stessa porta nello stesso momento, ed è ciò che serve per eseguire due peer sulla stessa macchina.

Inviare a un gruppo

```c
int fd = socket(AF_INET, SOCK_DGRAM, 0);              /* un normale socket UDP */

unsigned char ttl = 1;                                /* 1: questa sottorete e non oltre */
setsockopt(fd, IPPROTO_IP, IP_MULTICAST_TTL, &ttl, sizeof ttl);

unsigned char loop = 1;                               /* anche ai membri su questo host */
setsockopt(fd, IPPROTO_IP, IP_MULTICAST_LOOP, &loop, sizeof loop);

to.sin_family = AF_INET;
to.sin_port   = htons(5000);
inet_pton(AF_INET, "239.1.2.3", &to.sin_addr);
sendto(fd, msg, len, 0, (struct sockaddr *) &to, sizeof to);
```

Il mittente

Il mittente non si iscrive a niente, non si connette a niente e non sa chi sta ascoltando: questo codice è identico che il gruppo abbia zero membri o duecento. TTL 1 e loop attivo sono anche i valori predefiniti; qui sono scritti per mostrarli.

Chi rende selettivo il multicast

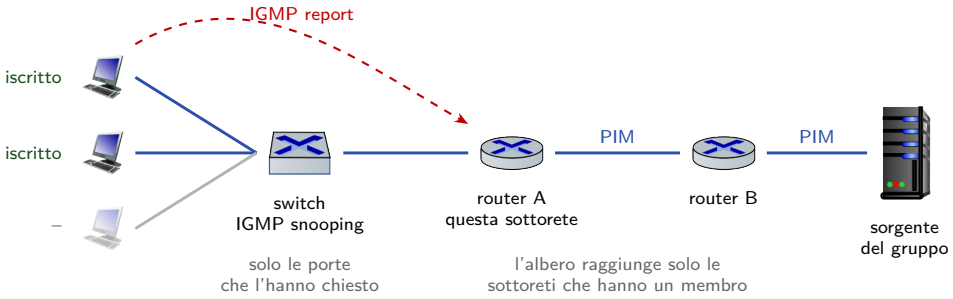


Figura 14.5 – I tre meccanismi che portano un datagramma multicast solo dove serve.

- **IGMP**: l'iscrizione non è solo locale. Il kernel annuncia il gruppo al router della sottorete, e lo ripete quando gli viene chiesto.
- **PIM**: i router costruiscono un albero fra sottoreti, così che una sottorete senza membri non riceva niente.
- **IGMP snooping**: lo switch inoltra il gruppo solo sulle porte che l'hanno chiesto; senza, lo inonda su tutta la LAN e il filtro lo fa ciascun kernel.
- Nessuno dei tre è un controllo di accesso: qualsiasi host lungo il percorso può iscriversi e leggere tutto. Se deve restare privato, **si cifra il contenuto**.

Esercizio 1 — fork, pipe e select

Un genitore e un figlio osservano entrambi la tastiera con `select`; il genitore inoltra al figlio, attraverso una pipe, ciò che legge. Il figlio osserva sia la tastiera sia la pipe e stampa da dove arriva ogni riga. Scrivere qualche stringa e spiegare che cosa si vede: chi riceve la riga scritta, e si può dire in anticipo?

Svolgimento

```c
int p[2]; char buf[256]; ssize_t n;
pipe(p);
if (fork() == 0) {                          /* figlio: tastiera e pipe */
    close(p[1]);
    for (;;) {
        fd_set r; FD_ZERO(&r); FD_SET(STDIN_FILENO, &r); FD_SET(p[0], &r);
        if (select(p[0] + 1, &r, NULL, NULL, NULL) < 0) continue;
        if (FD_ISSET(STDIN_FILENO, &r) && (n = read(STDIN_FILENO, buf, sizeof buf)) > 0)
            printf("figlio, dalla tastiera: %.*s", (int) n, buf);
        if (FD_ISSET(p[0], &r) && (n = read(p[0], buf, sizeof buf)) > 0)
            printf("figlio, dal genitore:   %.*s", (int) n, buf);
    }
}
close(p[0]);                                /* genitore: tastiera, poi pipe */
for (;;) {
    fd_set r; FD_ZERO(&r); FD_SET(STDIN_FILENO, &r);
    if (select(STDIN_FILENO + 1, &r, NULL, NULL, NULL) < 0) continue;
    if ((n = read(STDIN_FILENO, buf, sizeof buf)) <= 0) break;
    write(p[1], buf, n);
}
```

fork_pipe_select.c

I due processi condividono lo **stesso** standard input, ereditato con la `fork`. Quando si preme Invio, `select` sveglia **entrambi**: per tutti e due la tastiera è pronta. Ma la riga è una sola, e la prende chi arriva prima alla `read`, cosa che decide lo scheduler. Non si può dire in anticipo.

L'altro processo, svegliato per niente, chiama `read` e **si blocca** fino alla riga successiva. Se è il figlio, la riga che il genitore gli ha appena inoltrato resta nella pipe finché non si scrive ancora qualcosa: le righe *dal genitore* compaiono in ritardo e fuori ordine. È il «pronto non è una promessa» del capitolo, con due processi: fra la `select` e la `read` qualcun altro ha preso i dati. Mettere lo standard input in `O_NONBLOCK` lo trasforma in un `EAGAIN`; ma il flag sta nella descrizione del file aperto, quindi vale per entrambi i processi, e anche per la shell dopo che il programma è finito.

Esercizio 2 — Un peer multicast

Scrivere un peer: un solo socket UDP, iscritto a `239.1.2.3:5000`, con `select` sullo standard input e sul socket. Ciò che si scrive va al gruppo, ciò che arriva va sullo schermo. Nessun server: ogni peer è entrambe le cose. Provarne tre sulla stessa macchina (`./peer 239.1.2.3 5000`), poi su due macchine. Perché si vedono i propri messaggi, e quali sono i due modi per non vederli più?

Svolgimento

```c
/* socket, SO_REUSEADDR, SO_REUSEPORT, bind sulla porta e IP_ADD_MEMBERSHIP come nel capitolo */
struct sockaddr_in to = me;                  /* stessa porta ... */
inet_pton(AF_INET, argv[1], &to.sin_addr);   /* ... ma l'indirizzo del gruppo */

for (;;) {
    fd_set r; FD_ZERO(&r); FD_SET(STDIN_FILENO, &r); FD_SET(fd, &r);
    if (select(fd + 1, &r, NULL, NULL, NULL) < 0) { if (errno == EINTR) continue; break; }

    if (FD_ISSET(STDIN_FILENO, &r)) {        /* da tastiera al gruppo */
        ssize_t n = read(STDIN_FILENO, buf, sizeof buf);
        if (n <= 0) break;
        sendto(fd, buf, n, 0, (struct sockaddr *) &to, sizeof to);
    }
    if (FD_ISSET(fd, &r)) {                  /* dal gruppo allo schermo */
        struct sockaddr_in from; socklen_t fl = sizeof from;
        char ip[INET_ADDRSTRLEN];
        ssize_t n = recvfrom(fd, buf, sizeof buf, 0, (struct sockaddr *) &from, &fl);
        inet_ntop(AF_INET, &from.sin_addr, ip, sizeof ip);
        printf("[%s:%d] %.*s", ip, ntohs(from.sin_port), (int) n, buf);
    }
}
```

peer.c: il cuore

Ci si vede perché `IP_MULTICAST_LOOP` vale 1 per default e il nostro socket è iscritto al gruppo: la copia rimandata indietro sull'host arriva a tutti i socket iscritti, compreso il mittente. I due modi:

- **Spegnere il loop** con `IP_MULTICAST_LOOP` a 0. Ma la copia locale è una sola per tutto l'host: spegnendola, non sentono più i nostri messaggi nemmeno gli altri peer sulla stessa macchina.
- **Riconoscere i propri messaggi** e scartarli. L'indirizzo del mittente non basta, perché sulla stessa macchina tutti i peer hanno lo stesso IP e la stessa porta 5000: serve un identificatore nel messaggio, per esempio il pid e il nome dell'host in testa a ogni riga.

Con due macchine, l'indirizzo fra parentesi quadre distingue i peer remoti; se non arriva niente, il colpevole tipico è l'interfaccia scelta da `INADDR_ANY` per l'iscrizione, o una VM in NAT.

Esercizio 3 — Iterazione 4: una chat guidata dagli eventi

Si riparte dalla chat con i thread: funziona, e un client che smette di leggere congela tutti, perché il `broadcast` si blocca con il lock della lista in mano.

- **Che cosa cambia**: un thread solo. Niente `pthread_create`, niente mutex: la lista è un array qualsiasi, e `accept` avviene quando il socket di ascolto è leggibile.
- **Che cosa è nuovo**: niente può bloccarsi. I descriptor diventano non bloccanti, e ogni client riceve una coda in uscita che entra nell'insieme di scrittura solo quando contiene qualcosa.

Misurare la memoria con 200 client connessi, contro la versione con i thread. Poi sospendere un client con Ctrl-Z: che cosa succede agli altri 199? E che cosa fa la memoria del server mentre la coda di quel client cresce?

Svolgimento

Ogni client diventa una struttura con le due metà dello stato che prima stavano sullo stack del suo thread:

```c
struct client {
    int    fd;                              /* -1 = posto libero */
    char   in[512];   size_t inlen;         /* la riga in costruzione */
    char   out[4096]; size_t outlen;        /* byte che aspettano di partire */
};
static struct client cl[MAXC];
static fd_set allset, wall;
```

chatd.c: lo stato di un client

```c
for (;;) {
    fd_set rset = allset, wset = wall;
    if (select(maxfd + 1, &rset, &wset, NULL, NULL) < 0) { if (errno == EINTR) continue; break; }

    if (FD_ISSET(listen_fd, &rset)) accept_one();    /* nuovo client, O_NONBLOCK */
    for (int i = 0; i < MAXC; i++) {
        if (cl[i].fd != -1 && FD_ISSET(cl[i].fd, &rset)) on_readable(i);
        if (cl[i].fd != -1 && FD_ISSET(cl[i].fd, &wset)) on_writable(i);
    }
}
```

chatd.c: il ciclo

`on_readable` legge quello che c'è, lo aggiunge a `in` e, per ogni `\n` trovato, chiama `queue(j, ...)` per ogni altro client j; su 0 o su un errore diverso da `EAGAIN` chiama `drop(i)`. `on_writable` è il pezzo «client lento» del capitolo. `drop` toglie il descriptor da entrambi gli insiemi, lo chiude e libera il posto. Il secondo `if` ricontrolla `fd`, perché il primo può aver appena scartato quel client.

Con un client sospeso, gli altri 199 **continuano a chattare**. Le righe per lui riempiono prima i buffer del kernel (la sua finestra di ricezione e il nostro buffer di invio, centinaia di KB in tutto), poi la sua coda di 4096 byte; a quel punto `queue` lo scarta. La memoria del server **non cresce**: le code sono di dimensione fissa. Con una coda che si allarga con `realloc` crescerebbe senza limite, un messaggio alla volta, ed è proprio ciò che il limite evita.

Quanto alla memoria: con il ciclo gli stati di 200 client sono 200 strutture da circa 4,6 KB in un solo processo; la versione con i thread ha in più 200 stack, 200 thread nel kernel, e i cambi di contesto fra di loro.

Esercizio 4 — Iterazione 5: un pool dietro il ciclo

Si riparte dall'iterazione 4: un solo flusso di controllo per tutti. Un client che fa lavorare il server (una ricerca, un file letto dal disco) ferma ogni altra conversazione per tutto il tempo che serve.

- **Che cosa cambia**: il ciclo smette di servire. Quando un descriptor è leggibile, esce dall'insieme osservato e va nella coda; un worker lo legge, fa il lavoro, accoda la risposta e restituisce il descriptor.
- **Perché non un pool di connessioni**: perché una connessione di chat non finisce mai, e sedici worker servirebbero sedici client per sempre. Il lavoro è il messaggio.

Misurare, con otto worker e un client che chiede qualcosa di lento, il ritardo visto dagli altri. Perché il descriptor deve uscire dall'insieme osservato mentre lo tiene un worker? E dopo quanti worker la risposta smette di migliorare?

Svolgimento

Il ciclo e i worker si passano il descriptor in due direzioni: verso i worker con la coda limitata, all'indietro con una **pipe** che il ciclo osserva insieme ai socket. È l'unico modo di svegliare un ciclo fermo in `select`.

```c
/* nel ciclo: un client leggibile va a un worker */
if (FD_ISSET(fd, &rset)) {
    FD_CLR(fd, &allset);                    /* finche' lo tiene un worker, select non lo guarda */
    put_fd(&jobs, fd);
}

/* il worker */
for (;;) {
    int fd = get_fd(&jobs);
    handle(fd);                             /* legge, lavora, accoda la risposta */
    write(done[1], &fd, sizeof fd);         /* lo restituisce: 4 byte, scrittura atomica */
}

/* nel ciclo: done[0] e' nell'insieme osservato */
if (FD_ISSET(done[0], &rset)) {
    int fd;
    read(done[0], &fd, sizeof fd);
    FD_SET(fd, &allset);                    /* di nuovo osservato */
}
```

chatd.c: andata e ritorno di un descriptor

Ora le code dei client sono toccate dal ciclo e dai worker insieme, quindi tornano un mutex (uno per client basta).

Il descriptor esce dall'insieme perché, finché il worker non ha letto, il socket resta **leggibile**: `select` tornerebbe subito a ogni giro, facendo girare il ciclo a vuoto, e il ciclo potrebbe consegnare lo stesso descriptor a un secondo worker. Due worker sulla stessa connessione si dividerebbero le righe, e le risposte arriverebbero mescolate.

Con un solo client lento, il ritardo degli altri crolla già con **due** worker: uno resta occupato con la richiesta lenta, l'altro smaltisce tutte le veloci. In generale si smette di migliorare quando i worker sono tanti quante le richieste lente contemporanee più uno; e se il lavoro calcola invece di aspettare il disco, quando sono tanti quanti i core.

Riepilogo

In breve

- `select` si blocca su un insieme invece che su un descriptor, e restituisce quelli che si possono servire senza aspettare: un flusso di controllo per tutte le connessioni.
- Gli insiemi sono argomenti e risultati, quindi l'insieme principale si copia a ogni giro; e niente, sulla via da `select` a `select`, può bloccarsi.
- È questo che finalmente risponde al client lento: una **coda in uscita** per connessione, svuotata quando il socket dice di essere scrivibile.
- Un **pool** paga i suoi thread una volta e li tiene: il ciclo di `accept` produce, i worker consumano, e una coda limitata trasforma il sovraccarico in rifiuto invece che in esaurimento.
- Un pool serve tante connessioni lunghe quanti worker ha: quindi il lavoro è un messaggio, e qualcun altro si accorge che un messaggio è arrivato.
- Il **broadcast** raggiunge ogni host della sottorete e si ferma lì; il **multicast** raggiunge chi si è iscritto a un gruppo, su UDP, con una copia sul filo e nessuna idea di chi ascolta.

Protocolli applicativi, delimitazione e affidabilità su UDP

Cinque versioni della stessa chat, e ognuna ha cambiato **chi esegue il codice**: un processo, un thread, un ciclo, un pool.

- Ciò che viaggia sulla connessione non è mai cambiato: una riga di testo, senza nome, senza lunghezza e senza un modo per essere sbagliata.
- Il server non può rifiutare niente, perché non ha modo di rispondere a niente se non con altro testo.
- E tutto funziona perché sotto c'è TCP: i byte arrivano, in ordine, esattamente una volta, oppure la connessione muore.
- Qui il protocollo diventa un oggetto **da progettare**: tipi di messaggio, confini, codici di errore.
- Poi togliamo TCP, mettiamo sotto i datagrammi, e scriviamo a mano quello che faceva lui per noi.

Le quattro domande, rivolte al nostro protocollo

Un protocollo applicativo fissa i **tipi** di messaggio, la loro **sintassi**, la **semantica** dei campi e le **regole** dello scambio: è la definizione della prima parte, e ne abbiamo già usati tre (HTTP, SMTP, DNS). Rivolte alla nostra chat, le quattro domande diventano:

- **Tipi**: un client che entra, invia, esce; un server che consegna, conferma, rifiuta. Il nostro ne ha uno solo, e senza nome.
- **Sintassi**: dove comincia e dove finisce ogni messaggio, e come sono separati i campi al suo interno. È la domanda difficile.
- **Semantica**: che cosa significa un nome, che cosa significa un corpo vuoto, che cosa fa il server con un messaggio per qualcuno che se n'è andato.
- **Regole**: chi parla per primo, che cosa si può inviare prima di essere entrati, e che fine fa un messaggio che arriva fuori turno.

Testo o byte

Testo	Binario
lo si scrive a mano con `nc`	serve un programma per inviare qualsiasi cosa
lunghezza variabile: un numero è da 1 a 10 cifre	larghezza fissa: un numero è sempre 4 byte
leggerlo è scandire e convertire	leggerlo è copiare e invertire i byte
HTTP, SMTP, WHOIS	DNS, TLS, tutto ciò che sta sotto il trasporto

Tabella 15.1 – Le due codifiche a confronto.

- Il testo costa byte e analisi; il binario costa **ogni sessione di debug** che si farà mai su di esso.
- Nessuno dei due risponde alla domanda di dove finisce un messaggio: è una decisione separata, e viene subito dopo.
- Un compromesso comune: un'**intestazione binaria** che dice quanto è lungo il messaggio, e un corpo che è quello che si vuole.

Codificare un messaggio come testo, per bene

La codifica a testo del primo capitolo funzionava, ma aveva tre difetti che un server non può permettersi.

```c
char msg[128];
size_t off = 0;
for (int i = 0; i < n; i++) {                          /* codifica */
    int k = snprintf(msg + off, sizeof msg - off, "%d ", a[i]);
    if (k < 0 || (size_t) k >= sizeof msg - off) return -1;   /* non ci starebbe */
    off += (size_t) k;
}
msg[off++] = '\n';                     /* il delimitatore fa parte del protocollo */
send_all(fd, msg, off);

char *save, *tok = strtok_r(line, " ", &save);          /* decodifica */
for (i = 0; tok != NULL && i < MAX_VALUES; i++) {
    char *end;
    errno = 0;
    long v = strtol(tok, &end, 10);
    if (errno != 0 || *end != '\0') return -1;           /* "abc" non e' zero: e' un errore */
    vals[i] = (int) v;
    tok = strtok_r(NULL, " ", &save);
}
```

Codificare e decodificare una riga di numeri

- `snprintf` con il controllo del valore restituito non tronca in silenzio, come faceva `strncat`.
- `strtok` tiene il suo stato in una variabile statica: due thread che analizzano due messaggi si distruggono a vicenda la scansione. `strtok_r` lo tiene in `save`, che è nostra.
- `atoi` non può fallire: `"abc"` vale zero. Per rifiutare un input sbagliato serve `strtol`.

Codificare un messaggio come byte

```c
uint32_t n_net = htonl(n);                          /* mittente: prima il conteggio */
send_all(fd, &n_net, sizeof n_net);
for (uint32_t i = 0; i < n; i++) {
    uint32_t v_net = htonl(vals[i]);
    send_all(fd, &v_net, sizeof v_net);
}

uint32_t n_net, n;                                  /* destinatario: decide il conteggio */
if (recv_all(fd, &n_net, sizeof n_net) != sizeof n_net) return -1;
n = ntohl(n_net);
if (n > MAX_VALUES) return -1;          /* mai allocare quello che chiede il peer */
for (uint32_t i = 0; i < n; i++) { /* ... */ }
```

Prima il conteggio, poi i valori

- `uint32_t` da `<stdint.h>`, non `int`: la larghezza di `int` è una proprietà del compilatore, non del protocollo.
- L'ordine di rete vale anche per i nostri dati, non solo per le porte e gli indirizzi nelle intestazioni.
- Il conteggio inviato per primo **è la delimitazione**: il destinatario sa quanto segue prima di leggerlo.

Dove finisce un messaggio

La domanda si è già presentata due volte: con due righe arrivate in una sola `read`, e con una risposta HTTP che non finiva mai. Le risposte possibili sono soltanto tre.

Delimitazione dei messaggi

La **delimitazione** (*framing*) è la regola con cui il destinatario trova i confini dei messaggi in un flusso di byte. Le regole possibili sono tre:

- un **delimitatore**: si cercano i suoi byte nei dati che arrivano;
- una **lunghezza davanti**: si legge un'intestazione di dimensione fissa, poi esattamente quei byte;
- la **chiusura della connessione**: la fine del flusso è la fine del messaggio.

- Il delimitatore è comodo da scrivere a mano, ma il contenuto non deve mai contenerlo, oppure va protetto con un carattere di escape.
- La lunghezza davanti non richiede né scansione né escape, e il contenuto binario è gratuito.
- La chiusura funziona **una volta**, e solo per l'ultimo messaggio.
- HTTP usa la prima per l'intestazione e la seconda per il corpo; WHOIS usa la terza; DNS su TCP usa la seconda.

Il nostro protocollo deve sceglierne una, e il destinatario deve poterla applicare **senza indovinare**.

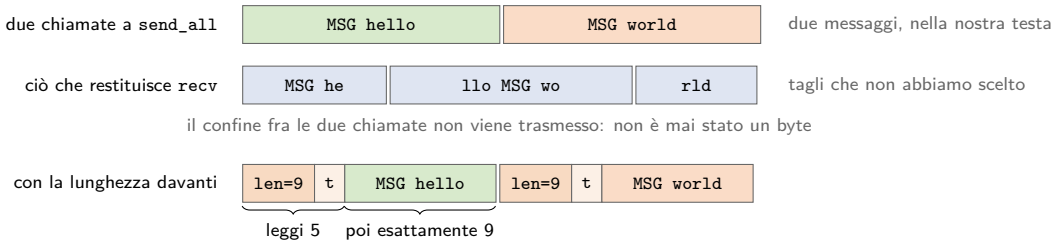


Figura 15.1 – Il flusso è lo stesso nelle due metà. Cambia che nella seconda i confini sono byte inviati da noi, quindi il destinatario li può trovare.

Leggere e scrivere esattamente n byte

```c
ssize_t recv_all(int fd, void *buf, size_t n) {
    size_t got = 0;
    char *p = buf;
    while (got < n) {
        ssize_t r = recv(fd, p + got, n - got, 0);
        if (r > 0) { got += (size_t) r; continue; }
        if (r == 0) return (ssize_t) got;   /* il peer ha chiuso: corta, non un errore */
        if (errno == EINTR) continue;       /* e' girato un handler: nemmeno questo */
        return -1;
    }
    return (ssize_t) n;
}
```

recv_all

```c
ssize_t send_all(int fd, const void *buf, size_t n) {
    size_t sent = 0;
    const char *p = buf;
    while (sent < n) {
        ssize_t w = send(fd, p + sent, n - sent, MSG_NOSIGNAL);
        if (w > 0) { sent += (size_t) w; continue; }
        if (errno == EINTR) continue;
        return -1;                          /* il peer non c'e' piu', o peggio */
    }
    return (ssize_t) n;
}
```

send_all

- Una lunghezza davanti è inutile senza `recv_all`: `recv` può restituire meno byte di quanti se ne chiedono, e di solito lo fa.
- `send` non trasmette: **copia** nel buffer di invio del socket, nel kernel, la cui dimensione è `SO_SNDBUF`. Se quel buffer è pieno di dati non ancora confermati, copia meno di quanto chiesto e dice quanto.
- Su un socket bloccante può anche aspettare; su uno non bloccante restituisce `EAGAIN`, che il ciclo degli eventi conosce già.
- Queste funzioni hanno dei nomi: `readn`, `writen` e `readline` di *UNIX Network Programming*. Si trovano in ogni codice di rete, e non sono nella libreria C.

Leggere un frame

```c
int read_frame(int fd, uint8_t *type, char *body, size_t max) {
    uint8_t h[5];               /* mai inviare una struct: il padding e' del compilatore */
    if (recv_all(fd, h, sizeof h) != (ssize_t) sizeof h) return -1;

    uint32_t len;
    memcpy(&len, h, 4);
    len = ntohl(len);
    *type = h[4];

    if (len > max) return -2;               /* si rifiuta prima di allocare */
    if (recv_all(fd, body, len) != (ssize_t) len) return -1;
    return (int) len;
}
```

read_frame: due letture, sempre

- Due letture, sempre: l'intestazione ha dimensione fissa, quindi la prima è uguale ogni volta; la seconda è quella che dice l'intestazione.
- Il formato sul filo è una **sequenza di byte**, non una struct: padding, allineamento e ordine dei campi cambiano da una macchina all'altra.

La lunghezza a cui si è creduto

Un client annuncia un corpo di quattro gigabyte e poi non invia niente. Che cosa fa il server? Chiama `malloc` con un numero scelto dal peer, e poi aspetta dentro `recv_all` byte che non arriveranno mai, tenendo occupata la memoria, e un worker, per tutto il tempo che il client vuole.

- Ogni campo che arriva dalla rete è un **input**, e una lunghezza è il più pericoloso di tutti.
- Quindi il protocollo dichiara un **massimo**, e il controllo avviene prima dell'allocazione e prima della lettura.
- Al client che si ferma risponde l'altro strumento del capitolo: un **timeout di lettura** sul socket.

IL PROTOCOLLO DELLA CHAT, MESSO PER ISCRITTO

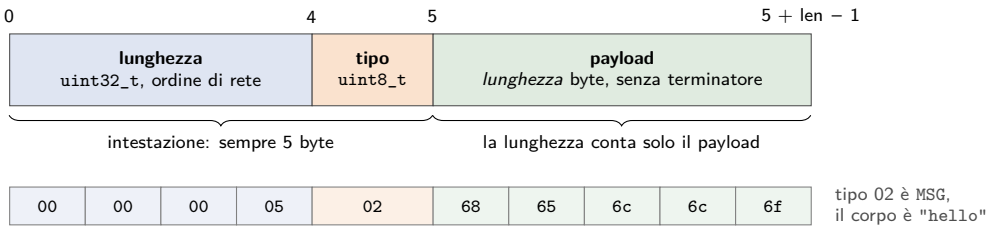


Figura 15.2 – Il formato sul filo della chat. Sotto, il frame completo di un messaggio `hello`: dieci byte.

Tipo	Nome	Chi lo invia	Corpo
01	`JOIN`	client	il nome scelto
02	`MSG`	client	una riga
03	`QUIT`	client	vuoto
10	`OK`	server	un testo facoltativo
11	`ERR`	server	codice di 2 byte in ordine di rete, poi il motivo in testo
12	`DELIVER`	server	un byte di lunghezza del nome, il nome, poi la riga

Tabella 15.2 – I tipi di messaggio: una numerazione possibile. Dentro il corpo di `DELIVER` c'è di nuovo una delimitazione, con una lunghezza davanti al nome.

Saper dire di no

Un protocollo che non può rifiutare ha un solo modo di essere in disaccordo con un client: riattaccare. È per questo che HTTP risponde con un **numero** e un **motivo**.

- Il codice è per il **programma**: decide se riprovare, lasciar perdere, o chiedere qualcosa all'utente.
- La stringa è per la **persona**: nessuno fa il debug di un protocollo cercando `ERR 7` nel nostro sorgente.
- I codici si raggruppano: ciò che ha sbagliato il client, ciò che il server non ha potuto fare, ciò che è temporaneo e vale la pena riprovare.
- Un errore è una **risposta normale**: la connessione resta aperta, e il messaggio successivo viene servito come sempre.
- La chiusura è per ciò a cui non si può proprio rispondere: un'intestazione che non si analizza, una lunghezza che si rifiuta, un peer che ha smesso di parlare.

Codice	Significato	Che cosa fa chi lo riceve
`400`	messaggio non valido o tipo sconosciuto	correggere il client: riprovare uguale è inutile
`403`	`MSG` prima di `JOIN`	entrare, poi riprovare
`409`	nome già in uso	chiedere un altro nome all'utente
`413`	corpo oltre il massimo dichiarato	il server chiude dopo averlo inviato
`503`	chat piena	temporaneo: riprovare più tardi

Tabella 15.3 – Un esempio di codici, raggruppati come in HTTP: 4xx sono errori del client, 5xx del server.

Un server che sopravvive ai propri client

Una chiamata bloccante interrotta da un segnale

```c
for (;;) {
    int fd = accept(listen_fd, NULL, NULL);
    if (fd >= 0) break;
    if (errno == EINTR) continue;    /* e' girato un handler: niente e' andato storto */
    perror("accept");
    continue;
}
```

EINTR non è un errore

- Qualsiasi chiamata bloccante (`accept`, `read`, `recv`, `select`) può restituire -1 con `EINTR` quando un handler viene eseguito mentre aspetta.
- Nel nostro server ce n'è uno: l'handler che raccoglie i figli. Ogni `SIGCHLD` può interrompere l'`accept`.
- `EINTR` non è un errore, e un `perror` su di esso riempie il log di righe che significano soltanto «è uscito un figlio».
- È per questo che `recv_all` e `send_all` lo controllano prima di restituire -1.

```c
struct sigaction sa;
memset(&sa, 0, sizeof sa);
sa.sa_handler = reap;
sa.sa_flags   = SA_RESTART;          /* riavvia cio' che si puo' riavviare */
sigemptyset(&sa.sa_mask);
sigaction(SIGCHLD, &sa, NULL);
```

SA_RESTART

- Solo `sigaction` può chiederlo: un motivo in più per cui `signal` è la chiamata che si trova nel codice vecchio e non si scrive.
- Con esso il kernel rientra da solo in `accept`, `read` e `write`: il ramo `EINTR` semplicemente non scatta mai.
- `select`, `poll` e `nanosleep` non vengono **mai** riavviate, con o senza: riavviarle farebbe ripartire anche il loro timeout, e l'attesa non finirebbe più.
- Quindi il ramo `EINTR` resta comunque nel codice. Costa una riga ed è l'unica risposta portabile.

La write che uccide il server

Un client sparisce e gli si scrive lo stesso

Che cosa succede al server? **Muore**. La disposizione predefinita di `SIGPIPE` termina il processo, e basta la prima scrittura dopo la chiusura del peer: un client che chiude una finestra butta giù la chat di tutti.

```c
signal(SIGPIPE, SIG_IGN);                    /* una volta, prima del primo client */
ssize_t w = send(fd, p, len, MSG_NOSIGNAL);  /* oppure per ogni chiamata */
if (w < 0 && errno == EPIPE) { close(fd); drop_client(fd); }
```

Due modi di sopravvivere

- Ignorarlo trasforma il segnale in un valore che si può controllare; con il flag dentro `send_all` è coperto tutto il programma.
- Dal lato della lettura lo si scopre diversamente, e non è un errore: `recv` restituisce 0.

Flag	Effetto
`0`	esattamente ciò che fanno `read` e `write`
`MSG_WAITALL`	aspetta che ci siano tutti i byte richiesti
`MSG_PEEK`	legge senza togliere dalla coda di ricezione
`MSG_DONTWAIT`	solo questa chiamata è non bloccante
`MSG_NOSIGNAL`	niente `SIGPIPE`: segnala `EPIPE`

Tabella 15.4 – I flag di `send` e `recv`.

- `MSG_DONTWAIT` è la versione per singola chiamata di ciò che il ciclo degli eventi imposta una volta sul descriptor con `O_NONBLOCK`.
- `MSG_WAITALL` non sostituisce `recv_all`: torna corta lo stesso quando il peer chiude, quando arriva un segnale, o quando il socket va in errore.

Arrendersi in tempo

```c
struct timeval tv = { .tv_sec = 2, .tv_usec = 0 };
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);

ssize_t n = recv(fd, buf, sizeof buf, 0);
if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK))
    ;   /* due secondi e non e' arrivato niente: ritrasmettere, o scartare il client */
```

Un timeout di lettura

- L'opzione appartiene al descriptor, quindi ogni lettura su di esso si arrende dopo lo stesso intervallo; `SO_SNDTIMEO` è la gemella per le scritture.
- `recv_all` ora ha un terzo esito, e chi la chiama deve saperli distinguere: n byte, peer andato, tempo scaduto.
- È questo che risponde al client che invia un'intestazione e si ferma, ed è il timer della ritrasmissione alla fine del capitolo.
- Il controllo d'inattività del kernel, `SO_KEEPALIVE`, comincia a sondare dopo circa due ore: una chat che deve accorgersi di un peer morto invia un proprio *heartbeat*.

Un protocollo che si può scrivere a mano

```
$ nc 127.0.0.1 5000
JOIN alice
OK welcome, alice
MSG hello everybody
HELLO everybody                 <- un errore di battitura, apposta
ERR 400 unknown message type
```

La versione a testo dello stesso protocollo, con nc

- `nc` apre un socket e lo collega al terminale: ciò che si scrive esce, ciò che arriva si stampa. È un **client generico**.
- Un protocollo a testo quindi si può provare prima che esista il suo client, e smontare a mano quando il client si comporta male.
- È per questo che SMTP, HTTP e WHOIS si possono digitare, ed è esattamente quello che costa un protocollo binario.
- Attenzione alla fine riga: molti protocolli prescrivono `\r\n`, e `nc` invia solo `\n`; `nc -C` invia `\r\n`.

COMUNICARE SENZA CONNESSIONE

UDP

Con UDP si inviano **messaggi indipendenti**, i **datagrammi**, da un estremo a un altro, senza connessione e senza garanzia che arrivino.

- Nessun handshake e nessun rilascio: il primo datagramma è il primo pacchetto sul filo, e non c'è stato da creare né da distruggere.
- Quindi non c'è niente da accettare: **un solo socket** serve tutti i client del server, e l'indirizzo del mittente arriva con ogni messaggio.
- Un datagramma arriva intero o non arriva: **i confini sono conservati**, ed è la delimitazione di questo capitolo, gratis.
- E nient'altro è promesso: né la consegna, né l'ordine, né che arrivi una volta sola.
- Ne vale la pena quando lo scambio è una richiesta e una risposta, o quando un dato in ritardo è un dato inutile: voce, video, un gioco.

```
1 ssize_t sendto(int fd, const void *buf, size_t len, int flags,
2                const struct sockaddr *to, socklen_t tolen);
3
4 ssize_t recvfrom(int fd, void *buf, size_t len, int flags,
5                  struct sockaddr *from, socklen_t *fromlen);
```

sendto e recvfrom

- Ogni invio porta con sé la destinazione: non c'è una connessione che la tenga per noi.
- `recvfrom` riempie chi l'ha inviato, e quell'indirizzo è l'unico modo che ha il server per rispondere.
- Il valore restituito è la lunghezza di **un** datagramma: mai un pezzo di uno, mai due attaccati.
- Un errore significa un errore **locale** (troppo grande, nessuna rotta), e mai che il datagramma non è arrivato. Questo ancora non lo sa nessuno.
- `fromlen` è valore-risultato: lo si imposta alla dimensione della nostra struttura, e il kernel riscrive la dimensione che ha usato.

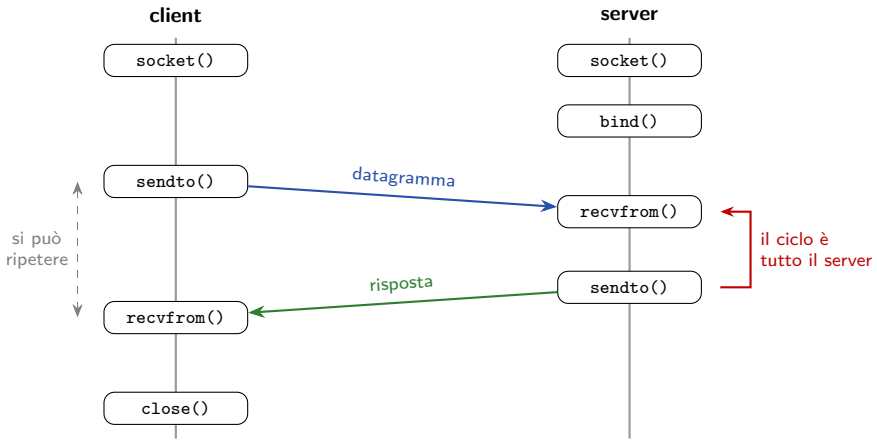


Figura 15.3 – La forma di un server e di un client UDP. Quattro chiamate invece di sette: niente `listen`, niente `accept`, nessun secondo socket.

Un server di eco UDP

```c
int fd = socket(AF_INET, SOCK_DGRAM, 0);
struct sockaddr_in addr = {0};
addr.sin_family      = AF_INET;
addr.sin_port        = htons(5000);
addr.sin_addr.s_addr = htonl(INADDR_ANY);
bind(fd, (struct sockaddr *) &addr, sizeof addr);   /* il client deve trovarci */

for (;;) {
    struct sockaddr_in from;
    socklen_t flen = sizeof from;           /* valore-risultato: va reimpostato ogni volta */
    ssize_t n = recvfrom(fd, buf, sizeof buf, 0,
                         (struct sockaddr *) &from, &flen);
    if (n < 0) { if (errno == EINTR) continue; perror("recvfrom"); break; }
    sendto(fd, buf, (size_t) n, 0, (struct sockaddr *) &from, flen);
}
```

Il server: un socket, un ciclo

Non c'è un descriptor per client, quindi non c'è niente da chiudere e niente da dare a un thread: lo stato per client, se serve, è nostro da tenere, in una tabella indicizzata da indirizzo e porta.

```c
int fd = socket(AF_INET, SOCK_DGRAM, 0);     /* niente bind: la porta la sceglie il kernel */
struct sockaddr_in srv = {0};
srv.sin_family = AF_INET;
srv.sin_port   = htons(5000);
inet_pton(AF_INET, "127.0.0.1", &srv.sin_addr);

sendto(fd, "hello", 5, 0, (struct sockaddr *) &srv, sizeof srv);

ssize_t n = recvfrom(fd, buf, sizeof buf - 1, 0, NULL, NULL);
if (n >= 0) { buf[n] = '\0'; printf("%s\n", buf); }
```

Il client

- Niente `connect`, niente handshake: la prima `sendto` è il primo pacchetto, e fissa anche la **porta effimera** del client.
- Quella porta è l'indirizzo a cui risponde il server, ed esiste solo perché la `sendto` è avvenuta.
- `NULL` come mittente significa accettare una risposta **da chiunque**, non solo dal server a cui si è scritto.

Una sendto, un datagramma

- Una chiamata invia un messaggio e una `recvfrom` ne riceve uno: due invii non si fondono mai, e un invio non si spezza mai.
- Se il buffer è più piccolo del datagramma, il resto viene **scartato** e non lo si viene a sapere, a meno di chiederlo con `MSG_TRUNC`, che restituisce la lunghezza vera.
- Il massimo è circa 64 KB, ma è il limite del formato, non un buon consiglio: un datagramma troppo grande per i collegamenti lungo il percorso viene spezzato più in basso, e un pezzo perso li perde tutti (lo vedremo al livello di rete).
- In pratica lo si tiene piccolo (qualche centinaio di byte per una riga di chat) e viaggia in un pezzo solo.
- I datagrammi corrotti vengono scartati da un checksum sottostante: ciò che il programma vede è **silenzio**, mai spazzatura.

connect su un socket a datagrammi

```c
connect(fd, (struct sockaddr *) &srv, sizeof srv);   /* non viene trasmesso niente */

send(fd, msg, len, 0);                      /* la destinazione e' registrata */
ssize_t n = recv(fd, buf, sizeof buf, 0);   /* e solo quel peer viene consegnato */
```

Un peer registrato

- Su un socket a datagrammi `connect` **registra soltanto** il peer: non parte nessun pacchetto, non si negozia niente, nessuno viene avvisato.
- Dopo si possono usare `send`, `recv`, `write` e `read`, e i datagrammi da altri indirizzi vengono scartati: la risposta economica al punto precedente.
- Ed è l'unico modo di sapere che **nessuno ascolta** sulla porta remota: senza, il messaggio che lo dice viene buttato, perché il kernel non sa a quale socket darlo. Con `connect`, la `recv` successiva restituisce `ECONNREFUSED`.
- Chi invia quel messaggio, e che cos'altro può dire, è una domanda per il livello di rete (è ICMP).

AFFIDABILITÀ SCRITTA A MANO

Che cosa fa la rete ai nostri datagrammi

- **Persi**: una coda si riempie in un router, o il buffer del destinatario è pieno. Il datagramma viene scartato e nessuno viene avvisato, né il mittente né il destinatario.
- **Duplicati**: una ritrasmissione da qualche parte più in basso, o un ciclo di instradamento che si risolve, e lo stesso messaggio arriva due volte.
- **Riordinati**: due datagrammi prendono strade diverse, e il secondo arriva per primo.
- **Corrotti**: presi dal checksum e scartati, quindi per il programma sono una perdita in più.
- L'API non riferisce niente di tutto questo: `sendto` ha successo appena il datagramma parte, e `recvfrom` continua ad aspettare.

TCP faceva questo lavoro da sempre. Se lo si vuole qui, **lo si scrive**.

Che cosa bisogna aggiungere

- Una **conferma** (*acknowledgement*): l'unico modo di sapere che un messaggio è arrivato è che l'altro lato lo dica. Niente più in basso ce lo dirà.
- Un **timeout**: l'unico modo di accorgersi di una perdita è che la conferma non arrivi entro un tempo scelto da noi.
- Una **ritrasmissione**: quando il timer scade, si invia di nuovo. È tutto il recupero.
- Un **numero di sequenza**: perché una ritrasmissione che si rivela non necessaria arriva come una seconda copia.

Stop-and-wait

Con un solo messaggio in volo alla volta il protocollo si chiama **stop-and-wait**: si invia, si aspetta la conferma, solo allora si invia il successivo. È semplice, corretto, e lento quanto **un tempo di andata e ritorno per messaggio**.

Ritroveremo tutti e quattro come meccanismo generale quando ci chiederemo come si costruisce un trasferimento affidabile su un canale inaffidabile.

La ritrasmissione che nessuno aveva chiesto

Il messaggio è arrivato, e lo si ritrasmette lo stesso. Perché? Perché anche la conferma è un datagramma, e può perdersi. Il mittente vede la stessa cosa nei due casi, cioè niente, e il destinatario riceve una copia che non può distinguere da un messaggio nuovo.

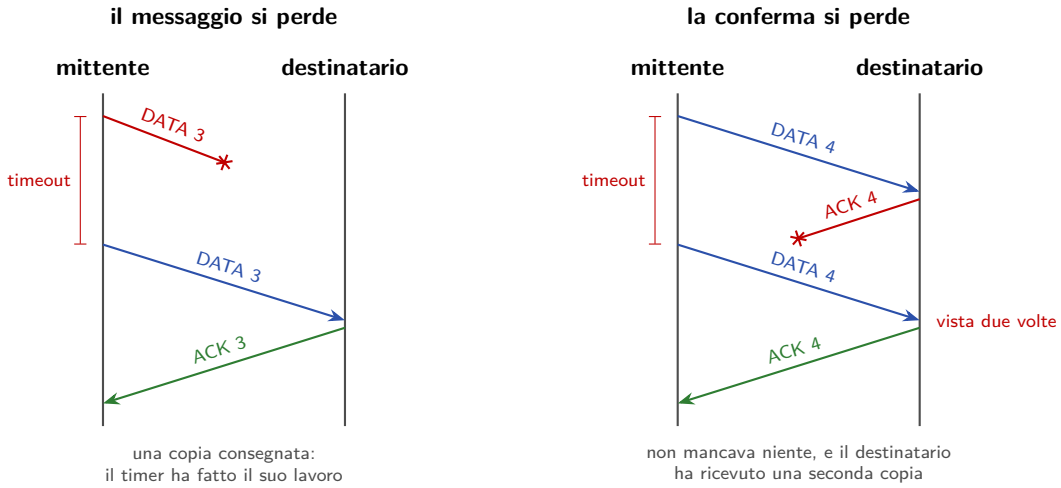


Figura 15.4 – Dal lato del mittente i due casi sono identici: nessuna conferma prima del timeout. La stessa copia doppia compare senza perdere niente se il timer è più corto dell'andata e ritorno: come sceglierlo è una domanda per TCP.

Numeri di sequenza: scartare, ma confermare di nuovo

- Ogni messaggio porta un numero, e ogni conferma porta il numero del messaggio a cui si riferisce.
- Il destinatario tiene l'ultimo numero accettato: uno già visto è un **duplicato**, e il contenuto si butta.
- Ma va **confermato di nuovo**. Il mittente sta ritrasmettendo perché non ha sentito niente, e il silenzio lo farebbe ritrasmettere finché non si arrende.
- Una conferma senza numero è peggio di nessuna: ferma il timer di qualunque messaggio si trovi in volo in quel momento.
- Con un solo messaggio in volo basta un singolo **bit alternato**; ma un contatore costa quattro byte ed è molto più facile da leggere in un dump.

```c
for (int tries = 0; tries < MAX_TRIES; tries++) {
    send_frame(fd, &srv, seq, body, len);               /* una sendto */

    for (;;) {                                          /* si aspetta la risposta giusta */
        ssize_t n = recvfrom(fd, ack, sizeof ack, 0, NULL, NULL);
        if (n < 0 && errno == EINTR) continue;          /* un segnale: si continua ad aspettare */
        if (n < 0 && errno == EAGAIN) break;            /* il timer e' scaduto */
        if (n < 0) return -1;                           /* un errore vero */
        if (ack_seq(ack) == seq) return 0;              /* confermato: fatto */
        /* la conferma di un messaggio vecchio: la si ignora e si aspetta ancora */
    }
}
return -1;                  /* MAX_TRIES datagrammi, nessuna risposta: si avvisa l'utente */
```

Il ciclo di ritrasmissione

Il timer è `SO_RCVTIMEO` su questo socket. Un segnale non deve consumare un tentativo, e arrendersi fa parte del protocollo: ritrasmettere per sempre non è affidabilità, è un programma bloccato.

Il timer riparte a ogni recvfrom

Con `SO_RCVTIMEO` il tempo si misura per ogni chiamata, non per il messaggio: una conferma vecchia che arriva a metà attesa fa ripartire il conto da zero. Per un timeout esatto si annota l'istante di invio con `clock_gettime` e si aspetta, con `select`, solo il tempo che resta.

ESERCIZI

Esercizio 1 — Il protocollo applicativo più piccolo che ci sia

Scrivere un client `whois`: risolvere `whois.iana.org` con `getaddrinfo`, connettersi alla porta 43, inviare il nome di dominio seguito da `\r\n`, stampare tutto ciò che torna indietro, chiudere. Poi rifarlo a mano con `nc -C whois.iana.org 43`. Quale delle tre delimitazioni usa WHOIS, e che cosa dice al ciclo che la risposta è finita? Poi inviare un secondo nome sulla stessa connessione: che cosa succede, e perché era prevedibile?

Svolgimento

```c
int main(int argc, char *argv[]) {
    if (argc != 2) { fprintf(stderr, "uso: %s dominio\n", argv[0]); return 1; }
    struct addrinfo hints = {0}, *res;
    hints.ai_family   = AF_UNSPEC;                /* IPv4 o IPv6, quello che c'e' */
    hints.ai_socktype = SOCK_STREAM;
    int rc = getaddrinfo("whois.iana.org", "43", &hints, &res);
    if (rc != 0) { fprintf(stderr, "%s\n", gai_strerror(rc)); return 1; }
    int fd = connect_any(res);                    /* il ciclo del capitolo sugli indirizzi */
    freeaddrinfo(res);
    if (fd < 0) { fprintf(stderr, "impossibile connettersi\n"); return 1; }

    char q[256];
    int k = snprintf(q, sizeof q, "%s\r\n", argv[1]);
    if (k < 0 || k >= (int) sizeof q || send_all(fd, q, k) < 0) return 1;

    char buf[4096];
    ssize_t n;
    while ((n = recv(fd, buf, sizeof buf, 0)) > 0)   /* fino alla chiusura del server */
        fwrite(buf, 1, (size_t) n, stdout);
    close(fd);
    return n < 0;
}
```

whois.c

WHOIS usa la terza delimitazione, la **chiusura della connessione**: la richiesta finisce con `\r\n`, la risposta finisce quando `recv` restituisce 0. Il ciclo non conta e non cerca niente: legge finché il server non chiude.

Un secondo nome sulla stessa connessione non riceve risposta. Il server ha risposto al primo e ha chiuso: la `send` del secondo può anche riuscire (copia nel buffer), ma il server risponde con un reset, e la lettura successiva restituisce 0 o `ECONNRESET`. Era prevedibile: la chiusura delimita **un solo** messaggio, quindi un protocollo che la usa può servire una sola richiesta per connessione.

Dal 2025 i domini generici (`.com`, `.org`, …) pubblicano i loro dati attraverso RDAP, JSON su HTTPS. WHOIS sopravvive per molti altri, e per questo esercizio.

Esercizio 2 — Iterazione 6: un protocollo con frame ed errori

Si riparte dalla chat con il pool. Serve tutti, e un messaggio è ancora una riga di testo: nessun tipo, nessuna lunghezza, e nessun modo per il server di rifiutarne uno.

- **Che cosa cambia**: il formato sul filo (quattro byte di lunghezza, uno di tipo, poi il corpo), con `send_all` e `read_frame` al posto di ogni `read` e `write` nude.
- **Che cosa è nuovo**: risposte di errore con un codice, una lunghezza massima dichiarata che si rifiuta prima di allocare, e un timeout di lettura per il client che si ferma a metà di un frame.
- **Che cosa non cambia**: il pool, la coda e la lista dei client. Se ci si ritrova a modificarli, si sta risolvendo il problema sbagliato.

Svolgimento

Il frame si costruisce in un buffer solo e parte con una `send_all`: due chiamate separate per intestazione e corpo funzionerebbero, ma produrrebbero due segmenti per messaggio.

```c
enum { T_JOIN = 0x01, T_MSG = 0x02, T_QUIT = 0x03, T_OK = 0x10, T_ERR = 0x11, T_DELIVER = 0x12 };
#define MAX_BODY 1024

int write_frame(int fd, uint8_t type, const void *body, uint32_t len) {
    uint8_t f[5 + MAX_BODY];
    if (len > MAX_BODY) return -1;
    uint32_t n = htonl(len);
    memcpy(f, &n, 4);                       /* byte per byte: mai una struct */
    f[4] = type;
    memcpy(f + 5, body, len);
    return send_all(fd, f, 5 + len) < 0 ? -1 : 0;
}

int send_err(int fd, uint16_t code, const char *why) {
    uint8_t b[128];
    uint16_t c = htons(code);
    size_t k = strlen(why) > sizeof b - 2 ? sizeof b - 2 : strlen(why);
    memcpy(b, &c, 2);
    memcpy(b + 2, why, k);
    return write_frame(fd, T_ERR, b, (uint32_t) (2 + k));
}
```

chat_proto.c: scrivere un frame

```c
static void handle(int i) {
    uint8_t type;
    char body[MAX_BODY];
    int len = read_frame(cl[i].fd, &type, body, sizeof body);
    if (len == -2) { send_err(cl[i].fd, 413, "frame too long"); drop(i); return; }
    if (len < 0)   { drop(i); return; }    /* chiuso, intestazione rotta o tempo scaduto */

    switch (type) {
    case T_JOIN: join(i, body, len);  break;          /* OK, oppure ERR 409 */
    case T_MSG:
        if (!cl[i].joined) { send_err(cl[i].fd, 403, "join first"); break; }
        deliver_all(i, body, len);                    /* DELIVER a tutti gli altri */
        break;
    case T_QUIT: drop(i); break;
    default:     send_err(cl[i].fd, 400, "unknown message type");
    }
}
```

chatd.c: il worker legge un frame e risponde

Per il worker il descriptor è **bloccante** con `SO_RCVTIMEO` di qualche secondo: `read_frame` legge l'intero frame, e un client che si ferma a metà occupa un worker al massimo per quel tempo, poi viene scartato. Le risposte agli altri client restano accodate come nell'iterazione 4, scritte con `MSG_DONTWAIT`.

```
$ printf "\x00\x00\x00\x05\x02hello" | nc 127.0.0.1 5000 | xxd
$ printf "\xff\xff\xff\xff\x02"      | nc 127.0.0.1 5000 | xxd
```

Due frame scritti a mano

Il primo è un `MSG` valido ma arriva prima di un `JOIN`: torna un `ERR 403`, e la connessione resta aperta. Il secondo annuncia quattro gigabyte: il server risponde `ERR 413` e chiude subito, **senza allocare niente e senza aspettare** il corpo.

Esercizio 3 — Iterazione 7: la stessa chat su datagrammi

Si riparte dall'iterazione 6, che è corretta perché sotto c'è TCP. Stesso protocollo, stessi tipi di messaggio, un socket di tipo `SOCK_DGRAM`.

- **Che cosa cambia**: niente `accept` e nessun descriptor per client. La lista dei client è una tabella di indirizzi, e un client esiste perché da lui è arrivato un datagramma.
- **Che cosa si ottiene gratis**: la delimitazione. Un messaggio è un datagramma.
- **Che cosa è nuovo**: un numero di sequenza e una conferma per ogni messaggio, `SO_RCVTIMEO` come timer, e un duplicato scartato e confermato di nuovo.

Scartare di proposito un datagramma su cinque e contare le copie che il destinatario vede prima del controllo della sequenza. Poi portare il timer a 50 ms su un percorso con 80 ms di andata e ritorno: quante copie arrivano, e quante perdite riferisce il mittente?

Svolgimento

Ogni datagramma porta davanti al tipo un numero di sequenza; la lunghezza non serve più, perché la dà `recvfrom`. Una conferma è un datagramma di tipo `ACK` con lo stesso numero.

```c
struct peer { struct sockaddr_in addr; uint32_t last_seq; int joined; char name[32]; };
static struct peer peers[MAXC];              /* la lista: indirizzi, non descriptor */

/* per ogni datagramma arrivato da 'from', con numero 'seq' */
struct peer *c = find_or_add(&from);         /* confronta indirizzo e porta */
send_ack(fd, &from, seq);                    /* si conferma sempre, anche un duplicato */
if (seq == c->last_seq) return;              /* gia' visto: il contenuto si butta */
c->last_seq = seq;
dispatch(c, type, body, len);                /* JOIN, MSG, QUIT come nell'iterazione 6 */
```

chatd.c: il lato che riceve

Il mittente usa il ciclo di ritrasmissione del capitolo. Perdite e ritardi si ottengono senza toccare il codice, con il modulo `netem` del kernel sull'interfaccia di loopback:

```
$ sudo tc qdisc add dev lo root netem loss 20%         <- un datagramma su cinque
$ sudo tc qdisc change dev lo root netem delay 40ms    <- 40 ms per verso: 80 ms di andata e ritorno
$ sudo tc qdisc del dev lo root                        <- di nuovo la rete di prima
```

Una rete peggiore, su richiesta

Con un datagramma su cinque perso (sia i dati sia le conferme), un tentativo va a buon fine con probabilità $0{,}8 \cdot 0{,}8 = 0{,}64$: servono in media $1/0{,}64 \approx 1{,}56$ invii per messaggio, e di questi ne arrivano $0{,}8 \cdot 1{,}56 = 1{,}25$. Il destinatario vede quindi circa **un quarto di copie in più**: sono le conferme perse, e il controllo della sequenza le scarta tutte.

Con il timer a 50 ms e 80 ms di andata e ritorno, senza perdite, ogni messaggio scade **sempre** una volta: la conferma del primo invio arriva a 80 ms, dopo la ritrasmissione partita a 50. Arrivano **due copie** di ogni messaggio, e un mittente che conta i timeout come perdite ne riferisce una per messaggio, anche se non si è perso niente. La seconda conferma arriva quando il mittente è già passato al messaggio successivo, ed è proprio la «conferma di un messaggio vecchio» che il ciclo ignora. Il timer va scelto più lungo dell'andata e ritorno, e misurandola: è ciò che fa TCP.

Riepilogo

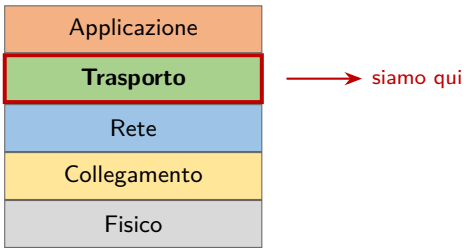
In breve

- Un protocollo fissa tipi di messaggio, sintassi, significato e regole; la più difficile delle quattro è **dove finisce un messaggio**.
- Tre risposte, e solo tre: un **delimitatore**, una **lunghezza davanti**, o la **chiusura** della connessione.
- Una lunghezza davanti non vale niente senza cicli che leggono e scrivono esattamente n byte, e senza un tetto a ciò a cui si crede.
- Un server che sopravvive ai propri client ignora `SIGPIPE`, riprova su `EINTR`, e si arrende in tempo.
- UDP conserva i confini dei messaggi e non promette nient'altro: una `sendto` è un datagramma, e può essere perso, duplicato o sorpassato.
- L'affidabilità sopra di esso sono quattro cose da scrivere: **conferma**, **timeout**, **ritrasmissione**, **numero di sequenza**.

Parte IV

Il livello di trasporto

Il livello di trasporto e UDP



Dall'applicazione al trasporto

Livello di trasporto

Un protocollo di trasporto fornisce una **comunicazione logica tra processi applicativi** in esecuzione su host diversi. Dal punto di vista dell'applicazione, gli host che eseguono i processi sembrano **direttamente collegati**, come se fossero sulla stessa macchina, anche se si trovano ai lati opposti del pianeta.

Il livello di trasporto trasforma i messaggi applicativi in uno o più pacchetti di trasporto, detti **segmenti**. Il Kurose usa questo nome per entrambi i protocolli; le RFC parlano di **datagrammi** per UDP, e così faremo anche noi:

- se i messaggi vengono spezzati in pezzi più piccoli, a ciascun pezzo viene aggiunto un **header di trasporto**;
- i segmenti vengono passati uno alla volta al livello di rete, che incapsula ciascuno in un datagramma IP e fa un tentativo *best effort* di consegnarlo;
- i router lavorano sui campi del datagramma IP: ciò che ha aggiunto il trasporto **non spetta a loro** leggerlo.

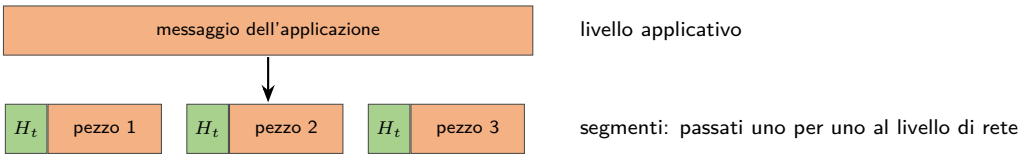


Figura 16.1 – Il trasporto spezza il messaggio in segmenti, ciascuno con il proprio header.

L'analogia delle due case

Immaginate due case, una a Napoli e una a Milano, ciascuna con una dozzina di cugini che si scrivono lettere. In ogni casa c'è un cugino (Anna a Napoli, Bruno a Milano) che raccoglie la posta di tutti, la consegna al servizio postale e distribuisce quella in arrivo ai destinatari.

- le **case** sono gli *host*; i **cugini** sono i *processi*;
- le **lettere** sono i *messaggi applicativi*;
- il **servizio postale** è il *livello di rete*: consegna da casa a casa;
- **Anna e Bruno** sono il *livello di trasporto*: consegnano da cugino a cugino, ma lavorano solo *dentro* le case.

La distinzione fondamentale

A livello di **trasporto** la comunicazione è tra **processi**; a livello di **rete** è tra **host**. Il trasporto, dal punto di vista logico, è punto-punto e il suo software gira **sugli host**; la rete deve instradare i pacchetti attraverso la topologia e il suo software gira soprattutto **sui nodi della rete**. Per questo lo strato di rete non può occuparsi fino in fondo dell'affidabilità: si preoccupa di attraversare il grafo.

Le quattro responsabilità del livello di trasporto

1. **Consegna processo-a-processo**: i messaggi vengono consegnati ai processi, ovunque essi si trovino. È diversa dalla consegna *host-a-host*, che è responsabilità del protocollo IP.
2. **Controllo di integrità**: campi per la rilevazione degli errori nell'header dei segmenti.
3. **Trasferimento dati affidabile**: i dati arrivano dal processo mittente al processo destinatario **corretti** e **in ordine**.
4. **Controllo di congestione e di flusso**: evita che una connessione sommerga di traffico i dispositivi di rete (collegamenti, router) o il destinatario. Migliora le prestazioni ed è molto utile a tutta Internet.

Servizio	UDP	TCP
Consegna processo-a-processo (porte, multiplexing)	✔	✔
Controllo di integrità (checksum)	✔	✔
Trasferimento affidabile	✖	✔
Controllo di flusso e di congestione	✖	✔

Tabella 16.1 – UDP, più veloce e semplice, offre solo i primi due servizi; TCP li offre tutti e quattro.

Cosa nessuno dei due garantisce

Né TCP né UDP garantiscono **ritardi massimi** o **banda minima**: il livello di rete di Internet (IP) è *best effort*, e il trasporto non può inventarsi garanzie che sotto non esistono.

La consegna processo-a-processo

A livello applicativo molti processi possono usare la rete nello stesso momento (musica in streaming, un download, la navigazione), e un processo può essere collegato a più processi remoti contemporaneamente (un proxy, un mail server). Per gestire tutto questo si usano i **socket**: invece di consegnare il messaggio direttamente a un processo, lo si consegna a un socket, un punto di accesso da cui il processo lo preleva. Così il numero dei processi è in parte svincolato dal numero delle connessioni: il server concorrente della parte sulla programmazione teneva **un socket per ogni client connesso**, e un solo processo con dieci socket parla con dieci peer.

Poiché sull'host ricevente ci sono molti socket, servono:

- un **identificatore univoco** per ogni socket (la **porta**), da scrivere nei segmenti;
- un meccanismo di **multiplexing** e **demultiplexing**.

Multiplexing e demultiplexing

- **Multiplexing** (lato mittente): raccogliere i dati dai diversi socket, creare i segmenti e assegnare a ciascuno l'identificatore giusto (le porte).
- **Demultiplexing** (lato ricevente): consegnare ogni segmento ricevuto al **socket giusto**, in base all'identificatore contenuto nell'header.

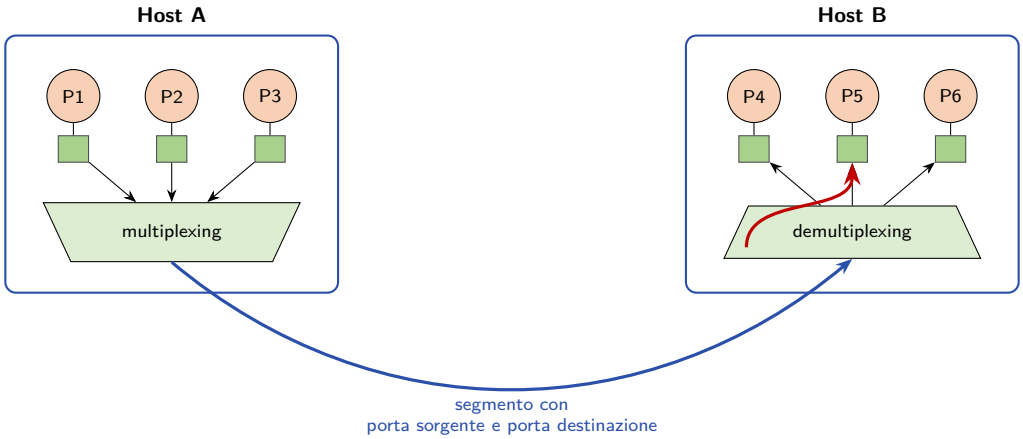


Figura 16.2 – Il mittente fonde i dati di più socket nel flusso verso la rete; il ricevente li smista al socket indicato dalla porta di destinazione.

Il problema di smistare messaggi verso più destinazioni è comunissimo: multiplexing e demultiplexing esistono anche in altri livelli.

Le porte

Porta

Gli identificatori dei socket si chiamano **porta sorgente** e **porta di destinazione** (o semplicemente **porte**). Una porta è un numero a **16 bit**, da 0 a 65535, e ogni header di trasporto ne porta due nei suoi primi quattro byte.

- La porta di **destinazione** è la chiave del demultiplexing: è ciò che legge l'host ricevente per scegliere il socket.
- La porta **sorgente** è l'indirizzo a cui andrà la risposta: il mittente annuncia dove bussare per rispondergli.
- Una porta appartiene a un **socket**, non a un processo: un processo può averne molte, e un socket di ascolto conserva la propria mentre i figli lavorano.

- Le porte da **0 a 1023** sono le **porte ben note** (*well-known ports*): riservate ai protocolli applicativi noti, e aprirle richiede di solito privilegi di amministratore.
- Da 1024 a 49151 ci sono le porte **registrate** presso la IANA per applicazioni specifiche; da 49152 a 65535 le porte **dinamiche** (o effimere), che il sistema operativo assegna ai client.
- L'elenco aggiornato è mantenuto dalla **IANA** (`http://www.iana.org`). Quando si sviluppa una nuova applicazione, le porte vanno scelte di conseguenza.

Porta	Uso
20, 21	FTP: trasferimento dati e comandi di controllo
22	SSH (Secure Shell)
23	Telnet: login remoto, testo non cifrato
25	SMTP: invio della posta
53	DNS
80	HTTP
110	POP3: accesso alla posta
119	NNTP (news)
123	NTP (sincronizzazione dell'orologio)
143	IMAP: accesso alla posta
161	SNMP: gestione della rete
443	HTTPS (HTTP su TLS/SSL)

Tabella 16.2 – Alcune porte ben note.

nmap

Per controllare l'uso delle porte su una macchina Linux si può usare **nmap** (*Network Mapper*), un programma nato per mappare host e servizi di una rete. Permette di sondare le porte di un host per scoprire quelle **aperte** (su cui un'applicazione è in ascolto) ed eventualmente il servizio associato.

```
$ sudo nmap [indirizzo]              # scansione delle porte di un host
$ sudo nmap -sV [indirizzo]          # prova a identificare i servizi in ascolto
$ sudo nmap --top-ports N [indirizzo]  # solo le N porte piu' usate
```

```
$ nmap --top-ports 6 scanme.nmap.org
Nmap scan report for scanme.nmap.org (45.33.32.156)
PORT    STATE    SERVICE
21/tcp  filtered ftp
22/tcp  open     ssh
23/tcp  filtered telnet
25/tcp  filtered smtp
80/tcp  open     http
110/tcp closed   pop3
```

Quali porte sono aperte

- `open`: qualcosa ha fatto `bind` sulla porta ed è in ascolto.
- `closed`: l'host ha risposto, e lì non c'è nessuno.
- `filtered`: non è tornata alcuna risposta, perché un firewall ha inghiottito la sonda; lo stato è **sconosciuto**.
- `nmap -sV` va oltre e chiede alla porta che servizio sia, che è anche la prima mossa di chi cerca un modo per entrare.

`scanme.nmap.org` è un host messo a disposizione dagli autori di nmap proprio per queste prove.

Solo sulle proprie macchine

Scansionare le porte di host altrui senza autorizzazione è considerato un comportamento ostile (è il primo passo di molti attacchi, e i sistemi di rilevazione delle intrusioni lo segnalano). Usate nmap solo sulle vostre macchine o in laboratorio.

Multiplexing e demultiplexing in azione

Un processo client sull'host A, con porta 46428, vuole mandare un messaggio a un processo server sull'host B, in ascolto sulla porta 19157.

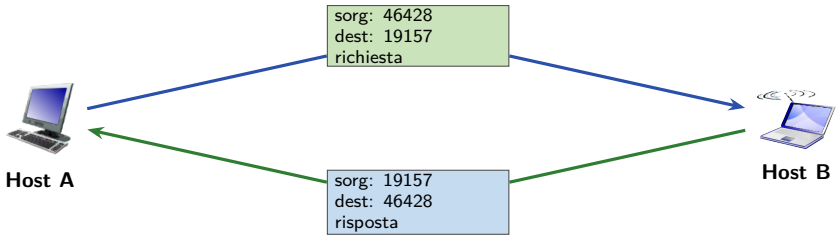


Figura 16.3 – Nella risposta le porte si scambiano: la porta sorgente del segmento di andata diventa la porta di destinazione del segmento di ritorno. È l'unico indirizzo di ritorno che il client abbia mai inviato.

- **Multiplexing**: il trasporto di A crea un segmento con i dati, la porta sorgente (46428) e la porta di destinazione (19157), e lo passa al livello di rete, che lo incapsula in un datagramma IP e fa del suo meglio (*best effort*) per consegnarlo a B.
- **Demultiplexing**: se il segmento arriva a B, il trasporto di B lo decapsula ed esamina la porta di destinazione (19157) per consegnarlo al socket giusto.

Che cosa compilava recvfrom

È questo che `recvfrom` riempiva per noi: l'indirizzo del client che il server di eco passava tale e quale a `sendto` è l'indirizzo e la porta che il client non ha mai dovuto comunicare, perché viaggiavano nell'header.

In UDP

- Il **client** crea il socket con `socket(AF_INET, SOCK_DGRAM, 0)`. Di solito non lo associa a una porta: è il sistema operativo a sceglierne una libera (per esempio 46428).
- La porta e l'indirizzo di **destinazione** si impostano nella `sockaddr_in` passata a `sendto()`: `servaddr.sin_port = htons(19157)`.
- Il **server** crea il socket e lo associa alla porta 19157 con `bind()`. Porta e indirizzo del client li ricava da `recvfrom()`, e li usa per rispondere, chiunque sia il client.

Identificazione di un socket UDP

Un socket UDP è identificato da **due** elementi: **indirizzo IP di destinazione** e **porta di destinazione**. Segmenti con indirizzi o porte sorgente diversi, ma stessa destinazione, finiscono nello **stesso socket**: lo stesso socket può rispondere a molti client diversi.

- È per questo che il server di eco UDP serviva tutti con un solo socket, senza mai accettare niente.
- E un client che sparisce non rompe nulla: non c'era alcuno stato su di lui da rompere.
- Il prezzo è che il socket non sa dire chi sta parlando, a meno che `recvfrom` non consegni ogni volta il mittente.

Nessuno ascolta su quella porta

Il client invia un datagramma alla porta 9999 di un host dove nessuno ha fatto `bind` su di essa. Che cosa torna indietro? **Niente dal livello di trasporto**: non ha un socket a cui consegnare, quindi scarta il datagramma e chiede al livello di rete di segnalare il fallimento.

- La segnalazione è un messaggio **ICMP**, inviato dall'host e non da un processo: lo leggeremo al livello di rete.
- Se il socket del client era connesso (`connect` su UDP), il kernel abbina la segnalazione e la chiamata successiva fallisce con `ECONNREFUSED`.
- Se non lo era, non c'è nessuno a cui dare l'errore, e `recvfrom` semplicemente aspetta: il timeout non era un dettaglio, era l'unico modo di accorgersene.

In TCP

- Il server ha un **socket di benvenuto** che attende richieste di connessione su una porta (19157).
- Il client crea un socket e invia un segmento di **richiesta di connessione**: un segmento TCP con la porta di destinazione 19157 e un bit speciale dell'header acceso (**SYN = 1**).
- Il server riceve la richiesta, trova il processo in attesa sulla porta 19157, e `accept()` crea un **nuovo socket**, riempiendo la struttura del client con porte e indirizzi del segmento arrivato.
- Tutti i segmenti successivi con **quella** porta sorgente, **quell'**indirizzo sorgente, **quella** porta di destinazione e **quell'**indirizzo di destinazione vengono consegnati al nuovo socket.

Identificazione di un socket TCP

Un socket TCP è identificato da **quattro** elementi: **indirizzo IP sorgente**, **porta sorgente**, **indirizzo IP di destinazione**, **porta di destinazione**.

La richiesta di connessione viene smistata **sulla sola porta di destinazione**, perché la connessione che chiede non esiste ancora; la porta effimera del client viaggia nel campo sorgente, ed è così che il server la impara.

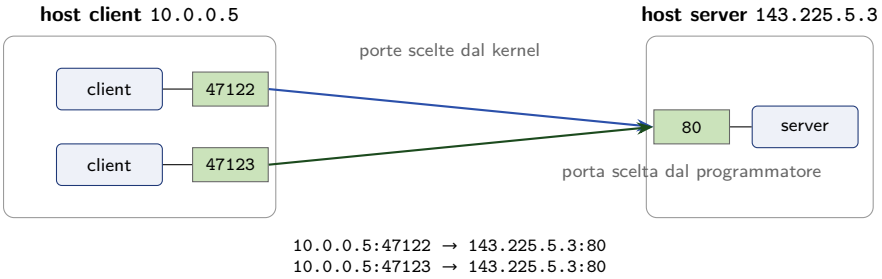


Figura 16.4 – Due connessioni, una sola porta del server: sono i quattro numeri insieme a distinguerle. `accept` crea il secondo socket e il trasporto registra i quattro valori letti nel segmento arrivato; ogni segmento successivo con gli stessi quattro valori va a quel socket, ed è per questo che quello di ascolto può continuare ad ascoltare.

Grazie all'handshake iniziale, la connessione TCP "lega" i processi ai due lati: il socket ha una sola sorgente e una sola destinazione, e se uno dei due lati si interrompe il socket viene chiuso da entrambe le parti (cosa che in UDP non succede).

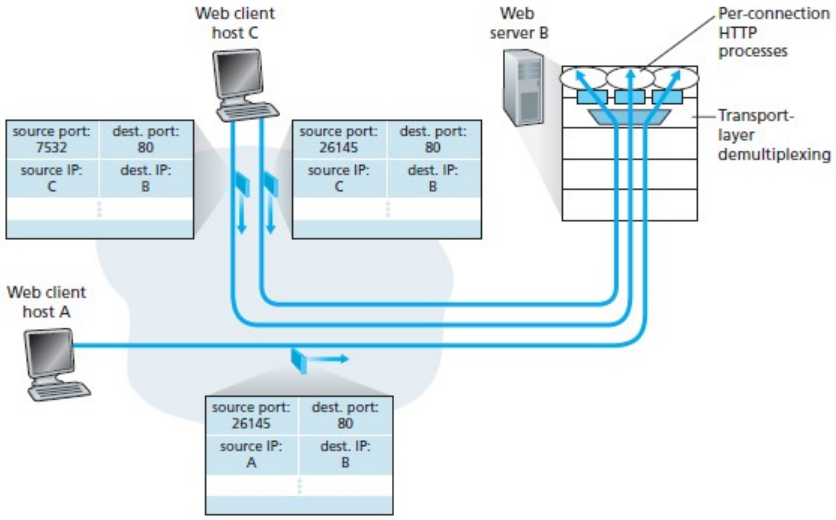


Figura 16.5 – Due client (A e C) aprono tre connessioni HTTP verso il server B: i segmenti sono smistati usando la quaterna completa.

- L'host C apre **due** sessioni HTTP verso il server B e l'host A ne apre **una**. A, C e B hanno indirizzi IP distinti.
- C assegna alle sue due connessioni porte sorgente diverse (26145 e 7532).
- A, che non sa nulla di C, potrebbe scegliere anch'esso la porta 26145: nessun problema, perché le due connessioni hanno **indirizzi IP sorgente diversi**.
- Il server apre un nuovo processo (o thread) per ogni connessione, con il suo socket dedicato. È l'approccio tipico, ma aprire troppi processi o thread può peggiorare le prestazioni del server.

IP sorgente	porta sorgente	IP destinazione	porta dest.	socket
C	26145	B	80	1
C	7532	B	80	2
A	26145	B	80	3

Tabella 16.3 – Tre connessioni verso la stessa porta 80 del server: le quaterne sono tutte diverse.

UDP

UDP (User Datagram Protocol)

UDP è il protocollo di trasporto **minimale**: aggiunge a IP "solo l'essenziale", cioè
- la **consegna processo-a-processo** (porte, multiplexing e demultiplexing);
- il **controllo degli errori** (checksum).

È **senza connessione** (nessun handshake prima di inviare), **non garantisce la consegna** e **non ha controllo di flusso né di congestione**.

Perché usare UDP invece di TCP

- **Controllo a livello applicativo**: UDP è più diretto, e l'applicazione controlla meglio *cosa* e *quando* viene inviato. Utile nelle applicazioni in tempo reale, dove velocità di invio e ritardi contano più di qualche dato perso.
- **Nessun ritardo di connessione**: niente handshake, quindi niente attesa prima di trasmettere. Ottimo per il DNS, che non deve aggiungere ritardi a ogni navigazione.
- **Nessuno stato della connessione**: niente buffer, parametri di congestione, numeri di sequenza e di ack da mantenere. Un server UDP può gestire molti più client attivi di uno TCP.
- **Header piccolo**: TCP aggiunge 20 byte di header a ogni segmento, UDP solo **8**.

Esempio 1 — Il DNS su UDP

- **Livello applicativo**: l'host che vuole interrogare un server DNS costruisce un messaggio di richiesta e lo passa a UDP.
- **Livello di trasporto**: senza alcun handshake, UDP aggiunge l'header e passa il segmento al livello di rete.
- **Livello di rete**: il segmento UDP viene incapsulato in un datagramma IP e inviato al server DNS.

Il client poi attende la risposta. Se non arriva (magari la rete ha perso la domanda o la risposta), può **rinviare** la richiesta, **chiedere a un altro** name server, oppure **avvisare** l'applicazione che la risposta non è arrivata. L'affidabilità, se serve, la gestisce l'applicazione.

Chi usa cosa

Servizio	Protocollo applicativo	Trasporto
Posta elettronica	SMTP	TCP
Terminale remoto	Telnet, SSH	TCP
Web	HTTP	TCP (o UDP con HTTP/3)
Trasferimento file	FTP	TCP
Traduzione dei nomi	DNS	UDP
Gestione dei dispositivi di rete	SNMP	UDP
Streaming multimediale	proprietario	UDP o TCP
Telefonia su Internet	proprietario	UDP o TCP

Tabella 16.4 – Applicazioni e protocolli di trasporto.

- Chi ha bisogno di **affidabilità** (posta, terminale remoto, Web) usa TCP, oppure protocolli affidabili costruiti su UDP come QUIC: una mail non può arrivare a pezzi.
- **SNMP** usa UDP perché le applicazioni di gestione devono funzionare proprio quando la rete è in difficoltà, e il trasferimento affidabile con tutti i suoi controlli sarebbe difficile.
- Le applicazioni **multimediali** (telefonia, videoconferenza, streaming) usano spesso UDP: tollerano qualche perdita e reagiscono male al controllo di congestione di TCP.

Il lato oscuro di UDP

Usare UDP per le applicazioni multimediali è **controverso**, proprio perché non c'è controllo di congestione. Se tutti inviassero egoisticamente video ad alta velocità senza alcun controllo, i router trabocherebbero, facendo perdere ancora più pacchetti UDP. E le perdite elevate spingerebbero i mittenti TCP, che invece si controllano, a ridurre drasticamente la loro velocità.

Il formato del segmento UDP

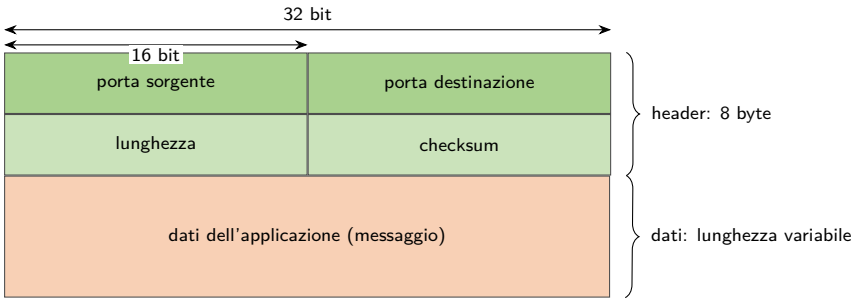


Figura 16.6 – Il segmento (datagramma) UDP.

L'header UDP è lungo **8 byte** (64 bit) e ha quattro campi da 16 bit:

- **porta sorgente**: la porta del processo mittente;
- **porta di destinazione**: la porta del processo destinatario;
- **lunghezza**: la lunghezza, in byte, dell'intero datagramma (header + dati);
- **checksum**: usata dal destinatario per controllare che il messaggio sia integro.

Seguono i **dati**, cioè il messaggio del livello applicativo.

Il checksum UDP

Checksum

Il **checksum** UDP è un campo di 16 bit per la rilevazione degli errori.

- **Mittente**: somma tutte le parole da 16 bit del datagramma, riportando ogni bit di overflow (*wraparound*) nella somma, e calcola il **complemento a 1** del risultato (inverte tutti i bit). Il risultato va nel campo checksum.
- **Destinatario**: somma tutte le parole da 16 bit ricevute, **checksum compresa**. Se non ci sono errori il risultato è `1111111111111111` (sedici 1). Se anche un solo bit è 0, il pacchetto contiene errori.

Esempio 2 — Calcolo del checksum su tre parole

	`0110011001100000`	(prima parola)
+	`0101010101010101`	(seconda parola)
	`1011101110110101`	(somma delle prime due)
+	`1000111100001100`	(terza parola)
	`10100101011000001`	(c'è un bit di overflow)
+	`0000000000000001`	(wraparound: si somma il riporto)
	`0100101011000010`	(somma)
checksum =	`1011010100111101`	(complemento a 1: si invertono i bit)

Al destinatario: `0100101011000010` + `1011010100111101` = `1111111111111111`, quindi nessun errore rilevato.

Perché il checksum, se c'è già il livello di collegamento?

Molti protocolli di collegamento controllano già gli errori, ma non è garantito che lo facciano **tutti** i collegamenti del cammino, e un errore può anche nascere mentre un segmento è nella memoria di un router. Poiché il trasporto deve funzionare da un estremo all'altro, controllare l'integrità **end-to-end** è necessario (principio *end-to-end*). Per rendere il controllo più robusto, il checksum viene calcolato anche su alcuni campi dell'header IP (gli indirizzi, il cosiddetto *pseudo-header*).

Che cosa il checksum non vede

Due parole da 16 bit del segmento arrivano una al posto dell'altra. Il checksum se ne accorge? **No**. La somma non dipende dall'ordine, quindi è la stessa, e il controllo passa su un segmento sbagliato.

- Non nota nemmeno due errori che si compensano: una parola troppo grande esattamente quanto un'altra è troppo piccola.
- È un test economico che coglie il caso comune, non una garanzia che i bit siano giusti. Dove gli errori sono attesi invece che eccezionali (su un cavo, nell'aria) serve un controllo più forte, che costruiremo al livello di collegamento.
- E il checksum **rileva** ma non **corregge**: cosa fare di un datagramma danneggiato UDP non lo dice; di solito lo scarta, oppure lo passa all'applicazione con un avviso.

Sotto il trasporto: ritardi, code e perdite

UDP consegna ciò che la rete gli porta, e la rete non porta tutto. Per capire che cosa dovrà recuperare il trasporto affidabile bisogna guardare che cosa succede a un pacchetto in ogni nodo che attraversa.

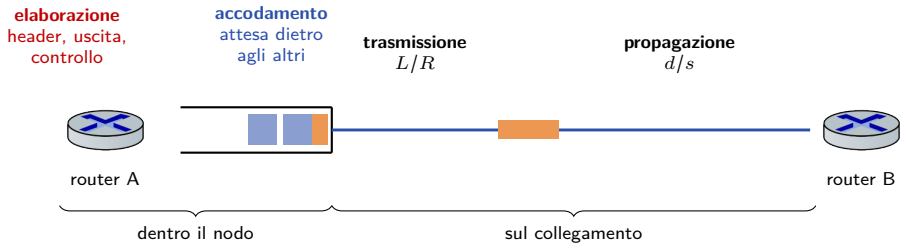


Figura 16.7 – I quattro ritardi di ogni nodo. I primi due avvengono dentro il nodo, gli ultimi due sul collegamento; solo l'accodamento cambia da un pacchetto all'altro.

Ritardo nodale

Il ritardo che un pacchetto subisce in un nodo è la somma di quattro termini:

$$d_{\text{nodo}} = d_{\text{elab}} + d_{\text{coda}} + d_{\text{trasm}} + d_{\text{prop}}$$

- **elaborazione**: leggere l'header, scegliere il collegamento di uscita, controllare i bit; pochi microsecondi su un router moderno;
- **accodamento**: aspettare dietro a ciò che è già in coda per quel collegamento;
- **trasmissione**: spingere L bit in un collegamento di rate R, cioè L/R, il termine già noto, ora uno dei quattro;
- **propagazione**: percorrere la distanza d alla velocità s del mezzo, poco meno di quella della luce, cioè d/s.

Trasmissione non è propagazione

- La trasmissione dipende dal pacchetto e dal rate, e per nulla da quanto è lontano l'altro capo.
- La propagazione dipende dalla distanza, e per nulla dal pacchetto o dal rate.
- Fra due router di un campus la propagazione è di un paio di microsecondi. Attraverso un satellite geostazionario è di centinaia di millisecondi, e domina tutto il resto.

L'analogia del casello

Dieci auto escono da un casello, una ogni dodici secondi: servono due minuti per farle passare tutte (è la **trasmissione**), e un'ora per percorrere i cento chilometri fino al casello successivo (è la **propagazione**). Con un casello più veloce i due minuti si accorciano, ma l'ora resta un'ora. E se la strada fosse cortissima, le prime auto arriverebbero mentre le ultime sono ancora in fila: allo stesso modo, i primi bit di un pacchetto possono già arrivare mentre gli ultimi aspettano di partire.

Esempio 3 — Quale dei due conta

Un pacchetto di 1500 byte ($L = 12\,000$ bit) su un collegamento da $R = 100$ Mb/s lungo $d = 1000$ km, con $s = 2 \cdot 10^8$ m/s:

$$d_{\text{trasm}} = \frac{12\,000}{10^8} = 120\ \mu\text{s} \qquad d_{\text{prop}} = \frac{10^6}{2 \cdot 10^8} = 5\ \text{ms}$$

Su questa distanza la propagazione è quaranta volte la trasmissione: un collegamento più veloce non avvicina i due capi.

Intensità di traffico, e da dove vengono le perdite

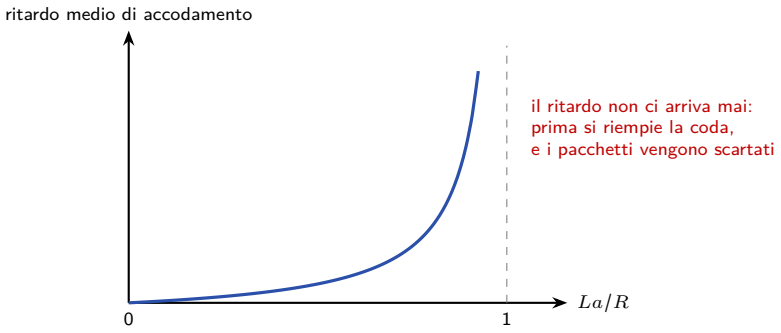


Figura 16.8 – Il ritardo medio di accodamento al crescere dell'intensità di traffico: resta piccolo finché il rapporto è basso, e sale ripidamente avvicinandosi a 1.

Intensità di traffico

Se arrivano in media a pacchetti al secondo, ciascuno di L bit, su un collegamento che ne smaltisce R bit al secondo, l'**intensità di traffico** è il rapporto La/R.

- Sopra 1 la coda cresce senza limite. La regola dell'ingegneria del traffico è restarne sotto.
- Sotto 1 il ritardo dipende da quanto sono raggruppati gli arrivi, e sale ripidamente man mano che il rapporto si avvicina a 1.
- E una coda reale è **finita**: un pacchetto che arriva a coda piena viene scartato, in silenzio. È lì che erano finiti i messaggi che la chat su UDP non ha mai ricevuto.

Ritardo da un capo all'altro e throughput

- I ritardi dei nodi si sommano: su N collegamenti, senza traffico, $N\,(d_{\text{elab}} + d_{\text{trasm}} + d_{\text{prop}})$, e sopra a questo gli accodamenti.
- Il **throughput**, come definito nella prima parte, è F bit ricevuti in T secondi. Da un capo all'altro è il rate del collegamento più lento, il **collo di bottiglia**, e aggiungere capacità altrove non cambia niente.
- Né il ritardo né il throughput sono garantiti da qualcosa: non da IP, non da UDP, e non da TCP. Le applicazioni che ne hanno bisogno sono costruite per farne a meno, ed è l'unico motivo per cui funzionano.

Riepilogo

In breve

- Il trasporto offre comunicazione logica **tra processi**; la rete, **tra host**.
- Quattro servizi: consegna processo-a-processo, integrità, affidabilità, controllo di flusso e congestione; UDP fa i primi due, TCP tutti.
- Le **porte** (16 bit) identificano i socket; multiplexing al mittente, demultiplexing al destinatario.
- Socket UDP: identificato da (IP dest, porta dest). Socket TCP: dalla **quaterna** completa.
- **UDP**: senza connessione, header di 8 byte (porte, lunghezza, checksum), nessuna garanzia; usato da DNS, SNMP, applicazioni multimediali.
- Il **checksum** è il complemento a 1 della somma a 16 bit con wraparound: rileva e non corregge, e non vede errori che si compensano o parole scambiate.
- Sotto, ogni nodo aggiunge elaborazione, accodamento, trasmissione e propagazione; le code introducono ritardo e, quando si riempiono, perdite. Renderle affidabili è il prossimo problema.

Il trasferimento dati affidabile

Il problema

Siamo in stazione ad aspettare il treno numero 6, previsto al binario 5. Il treno cambia binario e arriverà al 9: la stazione lo comunica con un messaggio, "*Il treno 6 arriverà al binario 9*". Cosa succede se la comunicazione è inaffidabile?

- le parole possono **perdersi**: "*Il treno %&! arriverà al binario 9*";
- possono essere **alterate**: "*Il treno 7 arriverà al binario 9*";
- possono essere **scambiate**: "*Il treno 9 arriverà al binario 6*";
- possono essere **duplicate**: "*Il treno 66 arriverà al binario 9*".

Nel primo caso ci si accorge che il messaggio è sbagliato; negli altri si rischia di prendere il treno sbagliato o di perdere quello giusto. Nel terzo, in particolare, ogni parola è arrivata: non è arrivato il significato.

A volte ci si accorge che il messaggio è rotto, a volte si prende il treno sbagliato: e **accorgersene non è la stessa cosa che rimediare**. Il controllo degli errori (per esempio con il checksum) permette al destinatario almeno di capire se un messaggio è corrotto, ma non basta. Il trasferimento affidabile è uno dei problemi centrali delle reti, e non viene affrontato solo dal trasporto, ma anche dal livello di collegamento e spesso da quello applicativo.

Canale affidabile

In un canale affidabile sono garantite tre cose:

1. nessun bit trasmesso viene **corrotto** (da 0 a 1 o viceversa);
2. nessun bit viene **perso** o **duplicato**;
3. tutti i bit arrivano **nell'ordine** in cui sono stati inviati.

Si parte da un canale inaffidabile

In generale si deve assumere che il livello di rete sottostante sia **inaffidabile**. È un'ipotesi realistica: TCP è un protocollo affidabile costruito sopra IP, che è inaffidabile, e tutto il lavoro si fa **ai due estremi**, a partire da un canale punto-punto inaffidabile. Studieremo il problema complicando il canale un passo alla volta: prima solo bit corrotti, poi anche pacchetti persi.

È esattamente ciò che abbiamo già fatto a mano nell'ultima versione della chat, su UDP: conferme, timeout, ritrasmissioni e numeri di sequenza. Qui gli stessi quattro meccanismi tornano come meccanismo **generale**, costruito un modello di canale alla volta.

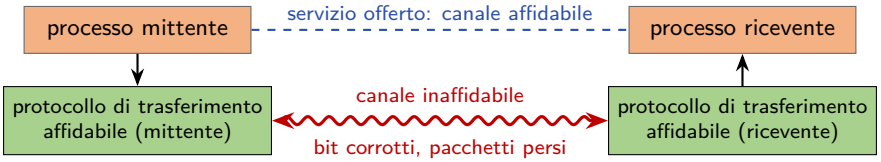


Figura 17.1 – Il protocollo di trasferimento affidabile offre all'applicazione un canale affidabile, costruendolo sopra un canale che non lo è.

Stop-and-wait con corruzione dei pacchetti

Supponiamo che il canale possa solo **corrompere** i bit, senza perdere pacchetti. La corruzione è un problema tipico, dovuto soprattutto alle componenti fisiche: un pacchetto viene trasmesso, propagato, memorizzato in buffer molte volte durante il viaggio.

Stop-and-wait

Nel protocollo **stop-and-wait** il mittente invia un messaggio e **aspetta una conferma** prima di inviarne un altro:
- un **ACK** (*acknowledgment* positivo) significa che il messaggio è arrivato integro;
- un **NAK** (*acknowledgment* negativo) significa che c'è stato un errore e il messaggio va ripetuto.

I protocolli basati sulla ritrasmissione si chiamano **ARQ** (*Automatic Repeat reQuest*).

Un protocollo ARQ ha bisogno di tre cose, e nessuna è facoltativa:

- **rilevazione degli errori**: bit in più che permettono al ricevente di accorgersi che il messaggio è cambiato (il checksum già visto);
- **riscontro dal ricevente**: il mittente è a migliaia di chilometri e non vede niente, l'unico modo di sapere è sentirselo dire. In linea di principio basta un bit: un valore per sì e uno per no;
- **ritrasmissione**: ciò che è arrivato con errori viene inviato di nuovo; non c'è altra riparazione.

E, come si vedrà subito, nemmeno tutte e tre insieme bastano.

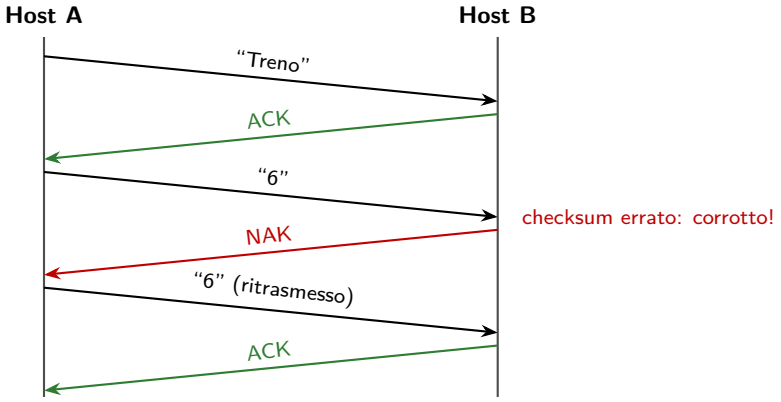


Figura 17.2 – Stop-and-wait con conferme positive e negative.

I due problemi dello stop-and-wait ingenuo

- Che succede se è **l'ACK o il NAK a essere corrotto**? Il mittente non sa cosa sia successo al suo messaggio.
- Se il mittente, nel dubbio, ritrasmette, come fa B a sapere se il messaggio è una **ripetizione** o un **messaggio nuovo**?

Numero di sequenza

Si aggiunge all'header dei pacchetti un campo, il **numero di sequenza**, che indica l'ordine dei pacchetti. Il ricevente può così riconoscere e scartare i duplicati.

In stop-and-wait c'è **un solo messaggio alla volta** sulla linea: non serve numerare l'intera sequenza, basta distinguere il pacchetto attuale dal precedente. È sufficiente **un bit** ($s = 0$ oppure $s = 1$).

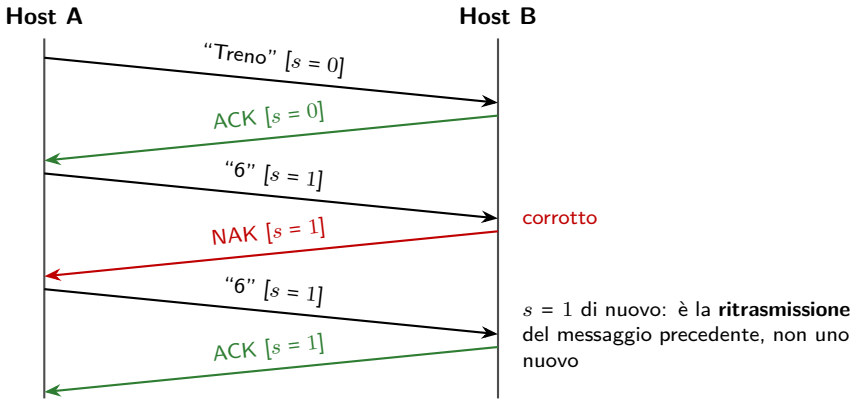


Figura 17.3 – Con un bit di sequenza il ricevente distingue un messaggio nuovo da una ritrasmissione.

Con gli ACK numerati il NAK non serve più

- Un NAK dice «rimanda». Una conferma che porta un numero dice «ho tutto fino a qui», che dice di più.
- Se il mittente riceve due volte la conferma del messaggio precedente, non si è mosso niente: è un NAK con un altro nome.
- Il ricevente ha quindi una sola risposta per entrambi i casi, messaggio danneggiato o duplicato: sempre un **ACK con un numero**.
- Un tipo di pacchetto in meno, e un ricevente che non deve mai spiegare che cosa è andato storto, solo che cosa ha.
- Quegli **ACK duplicati** torneranno: sono il modo in cui TCP si accorge di una perdita senza aspettare un timer.

Stop-and-wait con perdita dei pacchetti

Ora il canale può anche **perdere** pacchetti. È realistico: i dispositivi di rete possono avere i buffer pieni quando il traffico è intenso.

Con lo stop-and-wait visto finora c'è un problema evidente: il **blocco**. Se un messaggio si perde, il mittente aspetta per sempre un ACK che non arriverà, e nessuno invia più nulla. Succede sia se si perde il messaggio sia se si perde l'ACK: in entrambi i casi A non riceve lo stimolo per inviare di nuovo.

Timeout

Il mittente fa partire un **timer** a ogni invio. Se il timer scade (**timeout**) prima che arrivi la conferma, il mittente **ritrasmette** il messaggio. Il meccanismo funziona sia quando si perde il pacchetto sia quando si perde l'ACK; un approccio simile si usa nel DNS su UDP.

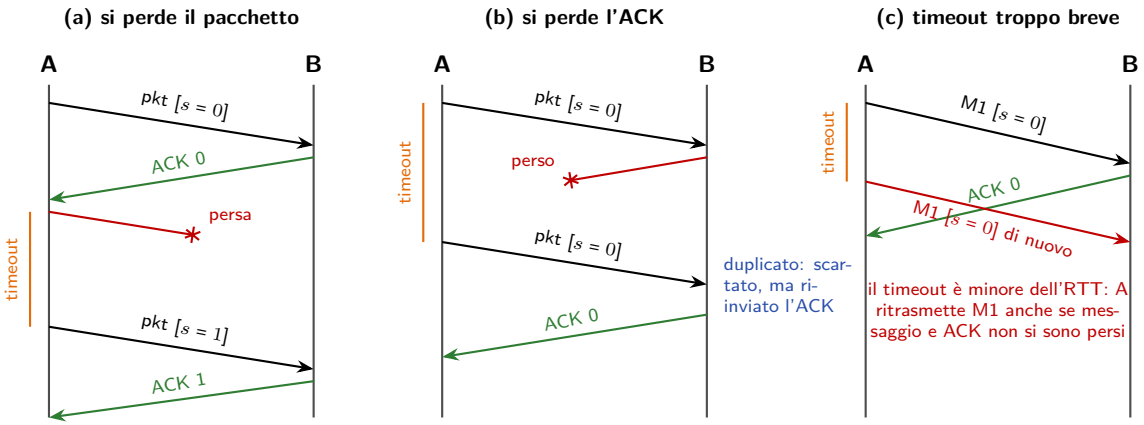


Figura 17.4 – Il timeout salva lo stop-and-wait dal blocco. In caso di duplicato il ricevente scarta il pacchetto ma deve rimandare l'ACK.

La stima del timeout

La difficoltà è scegliere un timeout adatto, e questo dipende dall'**RTT**:

- un timeout **troppo lungo** fa pagare ogni perdita con una lunga attesa a vuoto: il protocollo resta corretto, ma l'utente se n'è andato;
- un timeout **troppo breve** scatta su un messaggio ancora in viaggio, e le copie si accumulano su un canale che non sta perdendo niente (caso (c) della figura);
- il minimo non è una costante: almeno un'andata e ritorno, più il tempo che il ricevente impiega a rispondere;
- e il caso peggiore su un percorso reale non si può nemmeno stimare, perché le code fanno variare il ritardo da un pacchetto all'altro.

Il valore si sceglie quindi perché una perdita sia **probabile**, non certa: il protocollo è costruito sapendo che il timer a volte sbaglierà. Un timeout ragionevole è **un po' più lungo dell'RTT**.

Un timeout più corto dell'RTT

Il timer è più corto dell'andata e ritorno. Che cosa può fare di giusto il mittente? **Niente**. La conferma non può arrivare prima che il timer scatti, quindi ogni messaggio viene ritrasmesso, su un canale che non perde niente, e il ricevente riceve due copie di tutto. È il timer da 50 ms sul percorso da 80 ms dell'ultima chat: il numero di sequenza al ricevente è l'unica ragione per cui il programma funziona ancora.

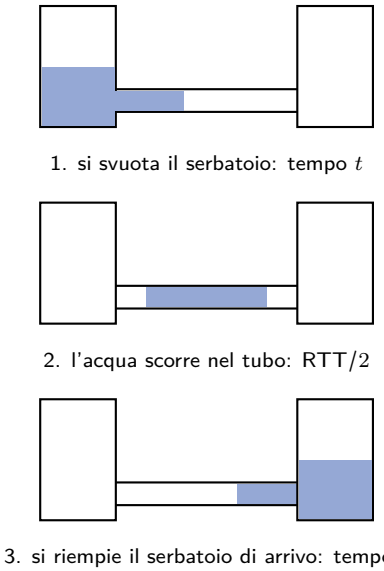
Che cosa fa il canale	Che cosa deve aggiungere il protocollo
non succede niente di male	niente: si costruisce il pacchetto e lo si invia
i bit possono essere corrotti	checksum, conferme, ritrasmissione
anche la conferma	un numero di sequenza, perché una copia sia riconoscibile
i pacchetti possono perdersi	un timer al mittente, e la ritrasmissione che fa partire

Tabella 17.1 – Dal canale perfetto a quello reale. L'ultima riga è il **protocollo ad alternanza di bit**: checksum, numero di sequenza di un bit, conferme numerate, un timer. È corretto su ogni canale descritto finora.

Il timeout salva lo stop-and-wait dal blocco, ma le **prestazioni** restano pessime, come vediamo ora.

Il modello "ad acqua"

Per ragionare sui tempi si può immaginare la trasmissione come acqua che passa da un serbatoio a un altro attraverso un tubo.



- t = tempo per "versare" tutti i bit nel collegamento (tempo di trasmissione, L/R);
- RTT/2 = tempo di propagazione fino al destinatario;
- l'ultima goccia arriva dopo RTT/2 + t dall'inizio.

Tempo totale = RTT/2 + t

Figura 17.5 – Il trasferimento come flusso d'acqua: mentre il serbatoio si svuota l'acqua già si muove nel tubo, e l'ultima goccia arriva dopo RTT/2 + t.

Le prestazioni dello stop-and-wait

Consideriamo il caso ideale di due host sulle coste opposte degli Stati Uniti. L'RTT alla velocità della luce è di circa **30 ms** (0,03 s). Supponiamo:

- un canale con **velocità di trasmissione** R = 1 Gb/s = 10^9 bit/s;
- pacchetti di **dimensione** L = 1000 byte = 8000 bit.

Il tempo per immettere il pacchetto nel canale è:

$$t = \frac{L}{R} = \frac{8000 \text{ bit}}{10^9 \text{ bit/s}} = 8 \cdot 10^{-6} \text{ s} = 8 \ \mu\text{s}$$

In stop-and-wait:

- il mittente comincia a trasmettere al tempo t_0;
- l'ultimo bit entra nel canale a t_0 + 0,000008 s;
- il pacchetto impiega 0,015 s per arrivare;
- l'ultimo bit arriva a t = RTT/2 + L/R = 0,015008 s;
- supponendo gli ACK piccolissimi (tempo di trasmissione trascurabile), l'ACK torna al mittente a t = RTT+L/R = 0,030008 s.

Utilizzo del mittente

L'**utilizzo** è la frazione di tempo in cui il mittente è effettivamente occupato a trasmettere:

$$U_{\text{mittente}} = \frac{L/R}{\text{RTT} + L/R} = \frac{0{,}000008}{0{,}030008} \approx 0{,}00027$$

Su 0,030008 s di tempo complessivo, il mittente ha trasmesso solo lo 0,027% del tempo: è rimasto **in attesa il 99,973% del tempo**. Un collegamento da 1 Gb/s usato così trasmette in media circa 267 kb/s!

Niente è rotto e niente si è perso: il protocollo invia semplicemente **un messaggio per andata e ritorno**. Il limite quindi non si sposta comprando un collegamento più veloce: si sposta cambiando il protocollo.

Esempio 1 — Una chat stop-and-wait

Un messaggio, una conferma, poi il successivo. Su un percorso con 100 ms di andata e ritorno, messaggi da 1 KB ($L \approx 8000$ bit) e un collegamento da 100 Mb/s:

$$\frac{L}{R} = 80\ \mu\text{s} \qquad U = \frac{0{,}00008}{0{,}10008} \approx 0{,}0008$$

e la chat muove circa 80 kb/s. Su loopback e sulla rete del laboratorio l'andata e ritorno è una frazione di millisecondo, e non lo si vede mai. Dieci messaggi al secondo vanno benissimo per una chat; non sono un modo di spostare un file, e non sono ciò che fa TCP.

Il pipelining

Pipelining

Con il **pipelining** il mittente può inviare **più pacchetti** senza aspettare le conferme.

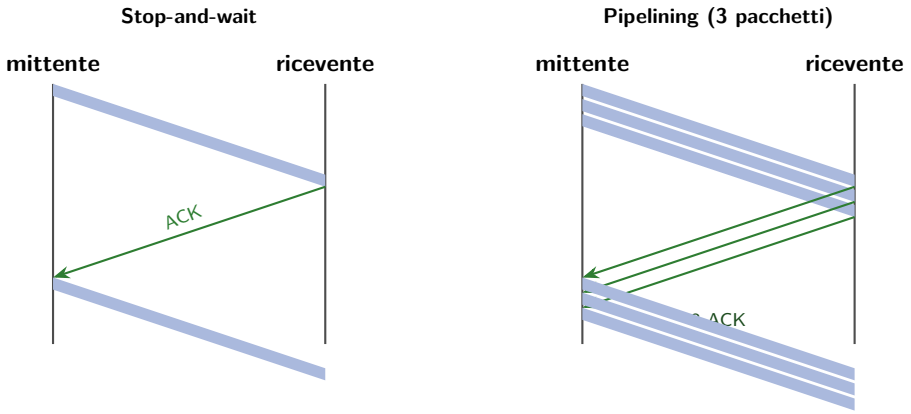


Figura 17.6 – Nello stesso tempo il pipelining trasmette più pacchetti: l'utilizzo cresce (quasi) in proporzione.

Con 3 pacchetti invece di uno: l'ultimo bit dell'ultimo pacchetto entra nel canale a $t_0 + 0{,}000024$ s, arriva a 0,015024 s e l'ACK torna a 0,030024 s. Il mittente ha lavorato tre volte tanto:

$$U = \frac{3 \cdot L/R}{\text{RTT} + L/R} = \frac{0{,}000024}{0{,}030008} \approx 0{,}0008$$

cioè ha aspettato il 99,92% del tempo invece del 99,973%, e il throughput sale a circa 800 kb/s. Il guadagno cresce con il numero di pacchetti inviati in parallelo, finché il limite torna a essere il canale invece del protocollo.

Il pipelining ha però delle conseguenze:

- l'intervallo dei **numeri di sequenza** deve crescere: ogni pacchetto in viaggio (escluse le ritrasmissioni) deve avere un numero univoco, quindi un solo bit alternato non basta più;
- servono **buffer al mittente**: tutto ciò che è stato inviato e non ancora confermato va conservato, perché potrebbe dover ripartire;
- e **buffer al ricevente**: pacchetti corretti possono ora arrivare con un buco davanti, e con loro qualcosa bisogna fare.

Quest'ultima decisione è **tutta la differenza** fra i due protocolli fondamentali basati sul pipelining: **Go-Back-N** li butta via, **Selective Repeat** li tiene.

Go-Back-N

Go-Back-N (GBN)

Nel protocollo **Go-Back-N** (detto anche protocollo a *finestra scorrevole*) il mittente può trasmettere più pacchetti senza aspettare le conferme, ma può avere **al massimo N pacchetti non confermati** contemporaneamente.

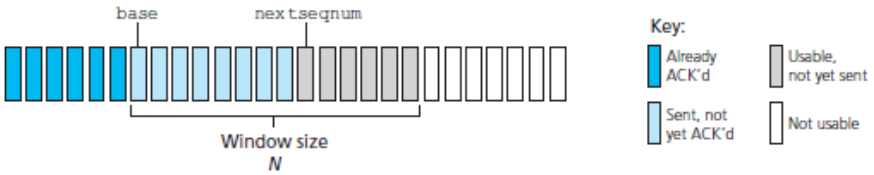


Figura 17.7 – La finestra del mittente in Go-Back-N: blu i pacchetti già confermati, azzurri quelli inviati e non confermati, bianchi quelli utilizzabili, grigi quelli non ancora utilizzabili.

Il mittente mantiene due indici:

- `base`: il numero di sequenza del **più vecchio pacchetto non confermato**;
- `nextseqnum`: il più piccolo numero di sequenza **non ancora usato** (il prossimo da inviare).

Intervallo	Stato dei pacchetti
$[0,\ \texttt{base}-1]$	trasmessi e confermati
$[\texttt{base},\ \texttt{nextseqnum}-1]$	inviati ma non ancora confermati
$[\texttt{nextseqnum},\ \texttt{base}+N-1]$	possono essere inviati
$[\texttt{base}+N,\ +\infty)$	non utilizzabili finché non arriva una nuova conferma

Tabella 17.2 – Le zone della finestra di Go-Back-N (N è la dimensione della finestra).

Quando la finestra è piena il mittente **rifiuta** i dati e li restituisce all'applicazione: un livello di trasporto che dice «non ora» all'applicazione è una novità, ed è da qui che nascerà il controllo di flusso.

Quando arriva la conferma di `base`, la finestra **scorre** in avanti. In GBN gli ACK sono **cumulativi**: "ACK n" significa "ho ricevuto correttamente tutto fino a n compreso".

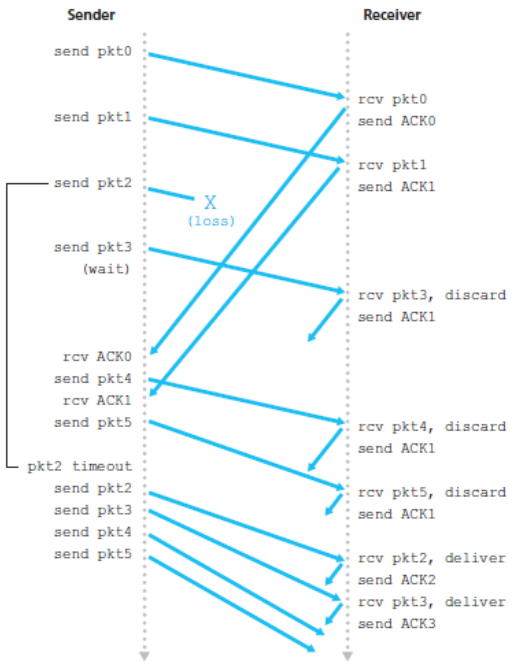
- Una conferma persa quindi si ripara da sola: ACK 5 rende inutili ACK 3 e ACK 4, e la finestra salta in avanti.
- C'è un **unico timer**, e appartiene al pacchetto più vecchio non confermato, non a ciascun pacchetto.
- Allo scadere il mittente **ritrasmette tutti i pacchetti inviati e non confermati**: da qui il nome "torna indietro di N".
- Quando arriva una conferma il timer riparte se resta qualcosa in sospeso, e si ferma se non resta niente.

Il ricevente in GBN

Il ricevente di GBN è molto semplice:

- **scarta i pacchetti fuori ordine**, anche se sono integri;
- consegna all'applicazione solo i pacchetti **in ordine**, e conferma l'ultimo ricevuto in ordine;
- i pacchetti scartati (non confermati) verranno prima o poi ritrasmessi dal mittente.

Si buttano via anche pacchetti buoni: un pacchetto perfetto che arriva in anticipo viene scartato esattamente come se fosse danneggiato. Ma la complessità è tutta da una parte, di proposito: il mittente mantiene una finestra, il ricevente un intero, `expectedseqnum`, il numero del prossimo pacchetto che è disposto a consegnare.



Finestra di dimensione $N = 4$; il pacchetto 2 si perde.

- Il ricevente riceve 0 e 1 e li conferma.
- Arrivano 3, 4 e 5: sono **fuori ordine** (manca il 2), quindi vengono **scartati**, e per ciascuno il ricevente rimanda ACK 1 (l'ultimo ricevuto in ordine).
- Al mittente scade il timer del pacchetto 2: ritrasmette 2, 3, 4 e 5.

Figura 17.8 – Go-Back-N in azione: poiché si perde pkt2, anche pkt3, pkt4 e pkt5 vanno scartati e ritrasmessi, pur essendo arrivati integri.

Una reazione a catena

Lo svantaggio di scartare pacchetti ricevuti correttamente è che bisogna **rispedirli**. È una reazione a catena: se si perdono pacchetti per problemi di rete, molti pacchetti buoni ma fuori ordine vengono buttati, costringendo il mittente a ritrasmetterli; e le ritrasmissioni stesse possono perdersi o danneggiarsi, richiedendo ancora altre ritrasmissioni.

Selective Repeat

Anche se GBN è molto meglio dello stop-and-wait, con finestre grandi un solo errore può causare la ritrasmissione di moltissimi pacchetti, per lo più inutilmente.

Selective Repeat (SR)

Nel protocollo **Selective Repeat** il mittente **ritrasmette solo i pacchetti che probabilmente sono andati persi o corrotti**:

- come in GBN, si usa una finestra di dimensione N per limitare i pacchetti non confermati;
- a differenza di GBN, ogni pacchetto è **confermato singolarmente** (ACK non cumulativi) e ha il **proprio timer**;
- il ricevente **conserva in un buffer** e conferma anche i pacchetti fuori ordine, e li consegna all'applicazione quando i "buchi" sono stati colmati.

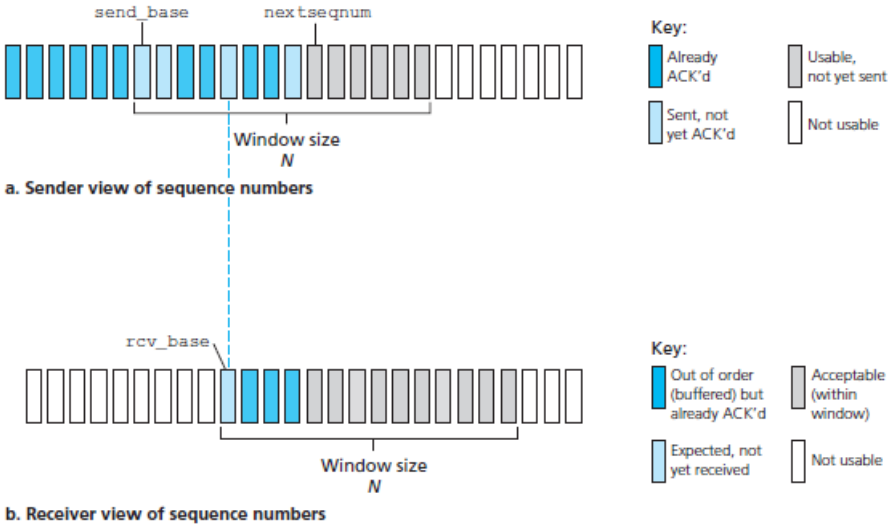
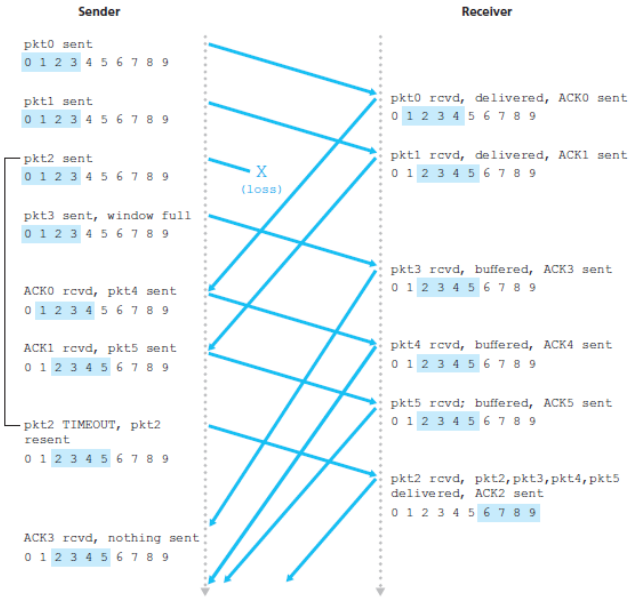


Figura 17.9 – Selective Repeat: finestra del mittente (sopra) e del ricevente (sotto). Il ricevente accetta e conserva i pacchetti che cadono nella propria finestra, anche se fuori ordine.



Stesso scenario di prima: finestra $N = 4$, si perde il pacchetto 2.

- Il ricevente conferma 0 e 1 e li consegna.
- Riceve 3, 4 e 5 fuori ordine: li **mette nel buffer** e li conferma singolarmente (ACK 3, ACK 4, ACK 5).
- Scade il timer del solo pacchetto 2, che viene ritrasmesso.
- Quando arriva pkt2, il ricevente consegna in blocco 2, 3, 4 e 5 all'applicazione.

Non si rispediscono pkt3, pkt4 e pkt5, evitando altre possibili perdite o errori.

Figura 17.10 – Selective Repeat in azione.

Mittente	**Ricevente**
Dati dall'alto: se il numero cade nella finestra, si costruisce il pacchetto e lo si invia.	Un pacchetto dentro la finestra: lo si conferma, e lo si mette nel buffer se è nuovo.
Un timer per pacchetto. Allo scadere si rimanda quel pacchetto e nessun altro.	Se è la base della finestra: lo si consegna, insieme a tutti i pacchetti del buffer consecutivi a esso.
ACK n: si segna n come ricevuto; se n era la base, si scorre fino al prossimo non confermato.	Un pacchetto appena sotto la finestra: lo si conferma di nuovo, e lo si scarta.

Tabella 17.3 – Che cosa fa ciascun lato in Selective Repeat. Sono i timer a renderlo costoso: N, dove Go-Back-N ne ha uno.

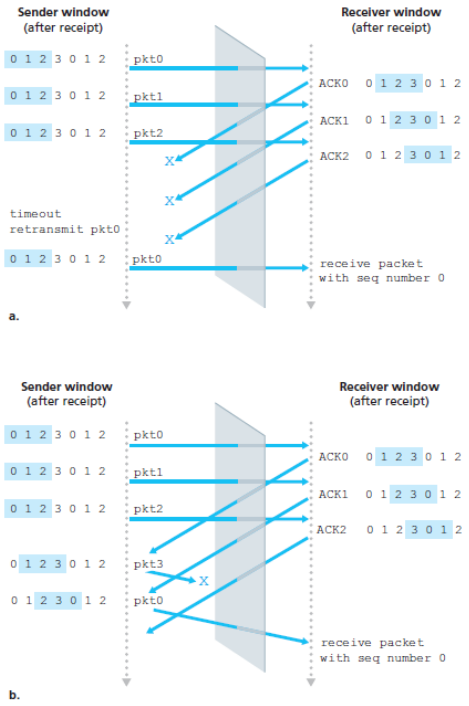
Confermare ciò che sta già sotto la finestra

Il ricevente ha quel pacchetto, consegnato da tempo. Perché rispondere? **Perché il mittente non lo sa**. Se la conferma del più vecchio pacchetto non confermato si è persa, il mittente continuerà a ritrasmetterlo, e la sua finestra non avanzerà mai.

- I due lati non condividono la visione di ciò che è arrivato, e in Selective Repeat questo scarto è permanente, non temporaneo.
- Il silenzio non è mai una risposta accettabile a una ritrasmissione: è indistinguibile da una seconda perdita.

Il vincolo sui numeri di sequenza

In SR la dimensione della finestra è legata al numero di sequenza, che è **finito** (il campo nell'header ha un numero limitato di bit, e i numeri vengono riusati ciclicamente).



Numeri di sequenza da 0 a 3 (quattro valori) e finestra di dimensione 3. I pacchetti 0, 1, 2 arrivano correttamente e i loro ACK partono (ma non sono ancora arrivati). La finestra del ricevente avanza su $[3, 0, 1]$, anche se non è certo che gli ACK siano arrivati.

- **Caso (a)**: gli ACK dei primi tre pacchetti si perdono e il mittente, allo scadere dei timer, ritrasmette il pacchetto 0 (vecchio). Il ricevente lo accetta come **nuovo** pacchetto 0!
- **Caso (b)**: gli ACK arrivano, il mittente invia i nuovi 3 e 0, ma il 3 si perde. Il ricevente riceve uno 0 che stavolta è davvero nuovo.

Il ricevente **non può distinguere** i due casi: ai suoi occhi sono identici.

Figura 17.11 – Il dilemma di Selective Repeat con finestra troppo grande.

Vincolo della finestra

Per evitare l'ambiguità, lo spazio dei numeri di sequenza deve essere **almeno il doppio** della dimensione della finestra:

$$\text{numero di valori di sequenza} \geq 2N \quad \Longleftrightarrow \quad N \leq \frac{\text{numero di valori}}{2}$$

Nell'esempio, con 4 valori la finestra può essere al massimo 2.

Un'ipotesi fatta fin dall'inizio

- Ogni modello di canale finora poteva corrompere e perdere, ma **mai riordinare**: ciò che usciva, usciva nell'ordine in cui era entrato.
- È vero per un singolo cavo. Non è vero per una rete, dove due pacchetti possono seguire strade diverse e arrivare nell'ordine sbagliato.
- Allora una vecchia copia del numero di sequenza x può riemergere molto dopo che x è stato ritirato, ed essere accettata come se fosse attuale.
- La difesa è il **tempo**: un numero non si riusa finché il mittente non è sicuro che nessuna vecchia copia possa essere ancora viva là fuori.
- Internet assume una **durata massima di un pacchetto** di circa tre minuti, ed è per questo che TCP non può semplicemente cominciare a contare da zero.

Riepilogo

	Stop-and-wait	**Go-Back-N**	**Selective Repeat**
Pacchetti in viaggio	1	fino a N	fino a N
Numeri di sequenza	1 bit ($s = 0/1$)	range più ampio	almeno $2N$ valori
ACK	per ogni pacchetto	cumulativi	individuali
Timer	uno	uno (pacchetto più vecchio)	uno per pacchetto
Fuori ordine al ricevente	—	scartati	bufferizzati
Ritrasmissione	il pacchetto	tutti i non confermati	solo quelli persi
Complessità	minima	bassa	più alta

Tabella 17.4 – I protocolli di trasferimento affidabile a confronto.

Meccanismo	**A che cosa serve**
Checksum	rileva errori sui bit dentro un pacchetto
Timer	scatta quando non è tornato niente, e la risposta è sempre la stessa: rimandare; i duplicati sono il prezzo
Numero di sequenza	un buco dice che qualcosa si è perso; un valore ripetuto dice che è una copia
Conferma (ACK)	dice al mittente che cosa ha il ricevente: un pacchetto alla volta, o tutto fino a un numero
Conferma negativa	dice che un pacchetto non è arrivato bene; inutile quando le conferme portano un numero
Finestra, pipelining	limita ciò che può essere in sospeso, e ricompra l'utilizzo che lo stop-and-wait butta via

Tabella 17.5 – I meccanismi, in una tabella.

In breve

- Un canale affidabile garantisce bit non corrotti, niente perdite o duplicati, ordine corretto.
- Contro la corruzione: **checksum**, **ACK/NAK** e **numeri di sequenza** (ARQ).
- Contro le perdite: **timeout** e ritrasmissione; il timeout deve essere un po' più lungo dell'RTT.
- Lo stop-and-wait ha un **utilizzo** bassissimo: $U = \frac{L/R}{\mathrm{RTT}+L/R}$.
- Il **pipelining** invia più pacchetti senza aspettare: **Go-Back-N** (ACK cumulativi, scarta i fuori ordine, ritrasmette tutto) e **Selective Repeat** (ACK individuali, bufferizza, ritrasmette solo i persi, sequenze $\geq 2N$).
- Ora viene il protocollo che non è né l'uno né l'altro: conferme cumulative come in Go-Back-N, buffer come in Selective Repeat. È TCP.

TCP: SEGMENTI, NUMERI DI SEQUENZA E RITRASMISSIONE

TCP A CONFRONTO CON UDP

Tutti i meccanismi sono pronti: checksum, numeri di sequenza, conferme, timer, una finestra. Ora ricevono un header. TCP conferma in modo cumulativo, come Go-Back-N, e tiene ciò che arriva in anticipo, come Selective Repeat; e una decisione attraversa tutto il capitolo: **i numeri contano byte, non pacchetti**.

Rispetto a UDP, TCP è:

- **orientato alla connessione**: prima della trasmissione c'è una fase di *handshake* che assicura che i due processi (mittente e ricevente) siano "collegati";
- **affidabile**: implementa rilevazione degli errori, ritrasmissioni, conferme, timer e numeri di sequenza;
- **attento al flusso e alla congestione**: regola la velocità di trasmissione in base alle capacità del ricevente (flusso) e allo stato della rete (congestione).

Proprietà della connessione TCP

- **Full-duplex**: se l'host A è connesso con B, anche B è connesso con A, e i dati possono scorrere contemporaneamente nelle due direzioni.
- **Punto-punto**: un solo mittente e un solo ricevente. Non è possibile trasferire dati da un mittente a molti riceventi con una sola connessione (UDP, invece, permette il *multicasting*).
- **Orientato al flusso di byte** (*stream*): più segmenti TCP possono far parte di un unico flusso ordinato di dati, mentre i datagrammi UDP sono sostanzialmente indipendenti tra loro. L'applicazione consegna byte, non messaggi, e dove cadono i confini dei segmenti lo decide TCP.

Una connessione senza niente in mezzo

Connessione logica

I due moduli di trasporto si comportano come se un canale affidabile li unisse direttamente. Sotto, i segmenti attraversano una rete che sa solo instradare.

- **Non è un circuito**: non si riserva capacità, e dentro la rete non si mette da parte niente per la connessione.
- Buffer, variabili e timer esistono **nei due sistemi terminali** e da nessun'altra parte.
- I router vedono datagrammi, non connessioni: inoltrano il segmento senza leggere un byte del suo header.
- È per questo che l'affidabilità si costruisce qui: il livello di rete è occupato a decidere *dove* va un pacchetto, non *se* è arrivato.

I BUFFER TCP

In una connessione TCP l'invio e la ricezione si basano sui **buffer**. I buffer permettono di disaccoppiare, almeno in parte, i tempi di trasmissione dai ritardi dell'applicazione, dai ritardi del sistema operativo nel multiplexing e demultiplexing, e dalle oscillazioni delle prestazioni della rete. Inoltre la velocità di trasmissione è limitata: i messaggi lunghi vengono spezzati in segmenti, che si accumulano nei buffer prima di essere spediti (lato mittente) o riassemblati (lato ricevente).

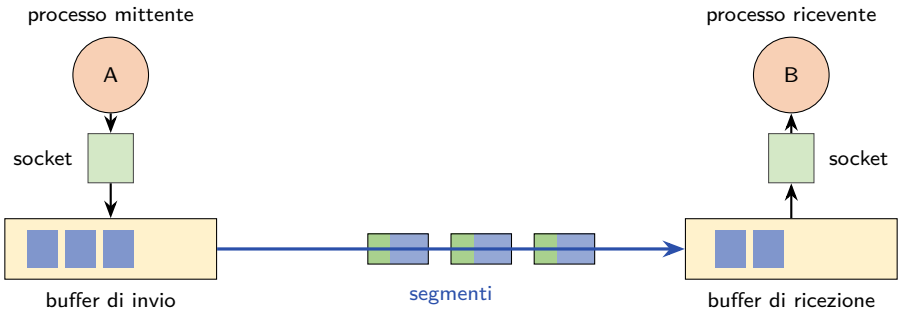


Figura 18.1 – Ogni lato di una connessione TCP ha un buffer di invio e uno di ricezione.

- **In invio**: `write` non trasmette; **copia** i byte nel buffer di invio, e TCP li spedisce da lì quando gli conviene, creando i segmenti e passandoli al livello di rete.
- **In ricezione**: `read` non riceve; prende ciò che è già nel buffer di ricezione, messo lì da segmenti arrivati prima.
- I due buffer assorbono la differenza fra quanto velocemente scrive l'applicazione, quanto velocemente si muove la rete e quanto velocemente legge l'altro lato. Ogni estremo ne ha uno di ciascun tipo, perché ogni estremo sia invia sia riceve.

Ecco perché, nella parte sulla programmazione, una `write` di 1000 byte poteva arrivare in tre `read`: le due chiamate non toccano la rete, toccano due buffer diversi.

LA MAXIMUM SEGMENT SIZE (MSS)

MSS e MTU

La quantità di dati applicativi in un segmento è limitata dalla **Maximum Segment Size** (MSS):

$$\text{MSS} + 40 = \text{MTU}$$

dove la **Maximum Transmission Unit** (MTU) è la lunghezza massima di un frame accettata dal livello di collegamento (in Ethernet **1500 byte**), e 40 è la somma tipica degli header TCP e IP (20 + 20 byte).

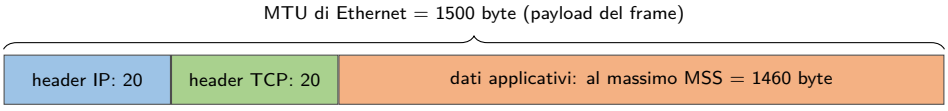


Figura 18.2 – Con MTU di 1500 byte e header da 20 + 20 byte, la MSS tipica è di 1460 byte.

La MSS è la quantità di **dati** in un segmento, non la dimensione del segmento. E un'immagine da tre megabyte non è un messaggio: sono duemila segmenti, e l'applicazione non viene mai a sapere dove sono caduti i tagli.

Il segmento TCP

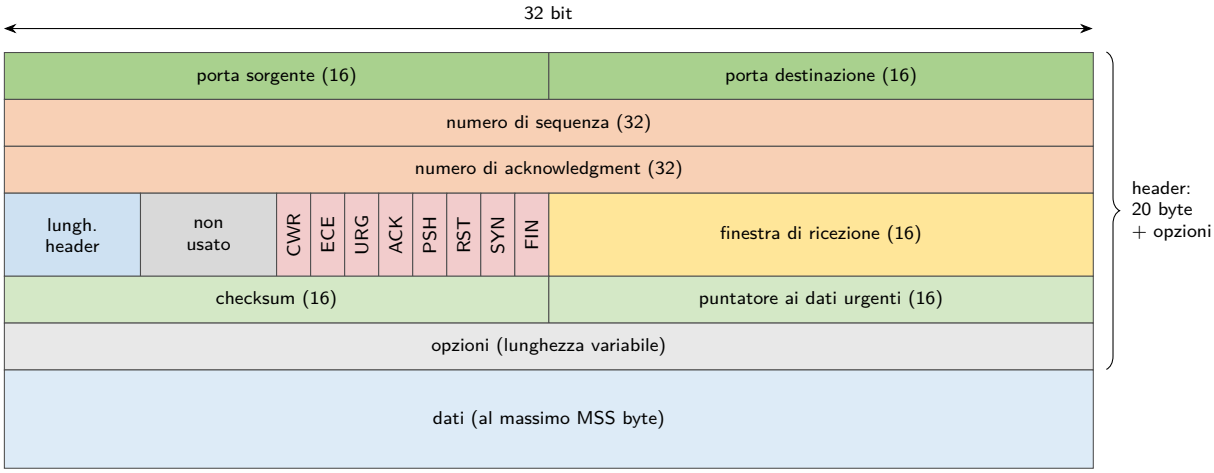


Figura 18.3 – La struttura del segmento TCP.

Il segmento TCP è fatto di un header e di un campo dati, che contiene al massimo MSS byte di dati applicativi. I campi dell'header:

- **porta sorgente** e **porta destinazione** (16 bit ciascuna): per multiplexing e demultiplexing;
- **numero di sequenza** (32 bit) e **numero di acknowledgment** (32 bit): per il trasferimento affidabile;
- **finestra di ricezione** (*receive window*, 16 bit): il numero di byte che il ricevente è disposto ad accettare (serve al controllo di flusso);
- **lunghezza dell'header** (4 bit): il numero di parole da 32 bit dell'header. L'header TCP ha lunghezza variabile per via delle opzioni (tipicamente 20 byte senza opzioni);
- **opzioni** (lunghezza variabile): per processi facoltativi come la negoziazione della MSS o i timestamp;
- **flag**:
  - **ACK**: il segmento contiene una conferma (il campo acknowledgment è valido);
  - **RST**, **SYN**, **FIN**: apertura e chiusura della connessione;
  - **CWR** e **ECE**: notifica esplicita della congestione (ECN), in cui è la rete a dire ai due estremi che è congestionata, invece di limitarsi a scartare;
  - **PSH**: chiede al ricevente di passare subito i dati all'applicazione;
  - **URG**: il segmento contiene dati "urgenti", e il puntatore dice dove finiscono.

  Otto flag, con quattro bit inutilizzati accanto; gli ultimi due (PSH e URG) in pratica non si usano;
- **checksum** (16 bit): controllo di integrità, come in UDP;
- **puntatore ai dati urgenti** (16 bit): la posizione dell'ultimo byte dei dati urgenti (raro).

Numeri di sequenza e di acknowledgment

I numeri di sequenza non servono solo al trasferimento affidabile, ma anche a gestire la **segmentazione**: un flusso di dati grande va spezzato in "pezzi" in base alla MSS, e ogni pezzo va in un segmento. Numeri di sequenza e di acknowledgment sono strettamente legati.

TCP è un protocollo di trasporto generico: non sa nulla dei dati che trasporta. Per TCP i dati sono solo un **flusso ordinato di byte**, e i numeri contano **byte**, non segmenti.

Numero di sequenza e numero di acknowledgment

- Il **numero di sequenza** di un segmento è la posizione, nel flusso di byte, del **primo byte di dati** contenuto nel segmento.
- Il **numero di acknowledgment** è il numero di sequenza del **prossimo byte atteso** dal ricevente. Gli ACK di TCP sono **cumulativi**: ACK n conferma tutti i byte fino a $n-1$.

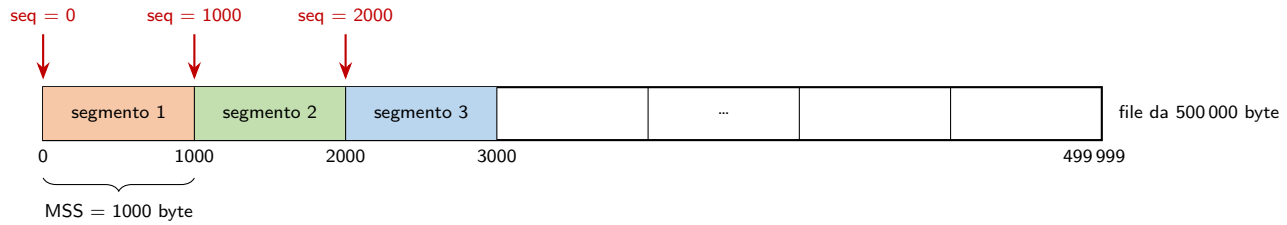


Figura 18.4 – Un file di 500 000 byte con MSS di 1000 byte: 500 segmenti, con numeri di sequenza 0, 1000, 2000, …

Esempio 1 — Un file di 500 kB

Un flusso di 500 000 byte con MSS di 1000 byte diventa **500 segmenti**: il primo ha numero di sequenza 0, il secondo 1000, il terzo 2000, e così via. Trascurando per semplicità lo scambio iniziale (nella realtà il primo numero di sequenza è scelto a caso), dall'host A all'host B:

- B riceve il primo segmento, con i byte da 0 a 999;
- B conferma con numero di acknowledgment **1000**: il prossimo byte che si aspetta;
- B riceve il secondo segmento, byte da 1000 a 1999, e conferma con ACK **2000**; e così via.

ACK n non conferma il byte n

L'errore più comune: ACK = 1000 **non** significa "ho ricevuto il byte 1000", ma "ho ricevuto tutto fino al 999, mandami dal 1000 in poi". Il numero esiste anche prima di qualsiasi dato: appena la connessione è aperta, B aspetta già un byte preciso.

Conferme cumulative, e un segmento in anticipo

Esempio 2 — Un buco nel flusso

A ha ricevuto da B i byte 0–535, e poi un segmento con i byte 900–1000. I byte 536–899 non sono mai arrivati. A conferma **536**.

- Una conferma dice «tutto ciò che sta sotto questo numero è arrivato», e **niente** su ciò che sta sopra.
- Quindi una conferma copre un numero qualsiasi di segmenti, e perderne una non costa niente purché ne passi una successiva.
- Non c'è modo di riferire i cento byte che A sta tenendo: l'header ha un numero solo, e il buco viene prima.
- Quel numero unico è ciò che rende economico il mittente, e costoso un segmento mancante.

E i byte 900–1000, arrivati in anticipo: tenerli o buttarli? **Le RFC non lo decidono**: entrambi i comportamenti sono corretti, e la scelta è lasciata a chi scrive l'implementazione. Buttandoli il ricevente resta semplice e il mittente paga rimandandoli; tenendoli i byte aspettano che il buco si riempia, e la rete trasporta ogni byte una volta. **Tutte le implementazioni in uso li tengono**, ed è per questo che una perdita costa una ritrasmissione e non una finestra intera.

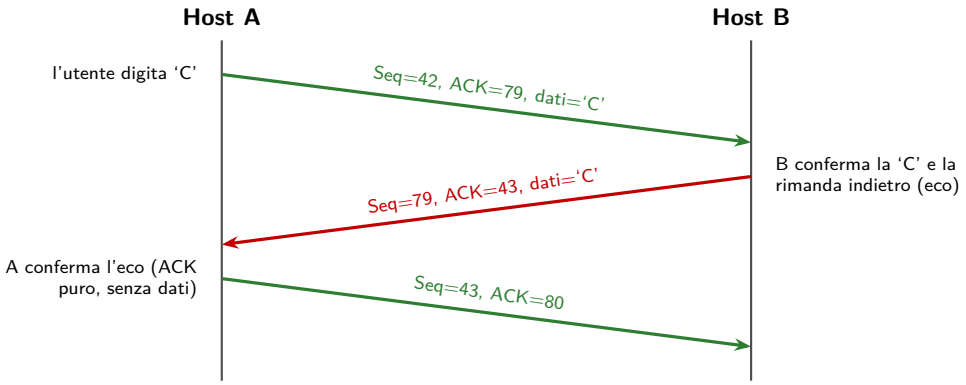
Il caso full-duplex

TCP è full-duplex: se A e B comunicano ci sono **due flussi** di dati da rendere affidabili, da A a B e da B ad A. Si usano numeri di sequenza e di acknowledgment **contemporaneamente**:

- i segmenti inviati da A hanno numeri di **sequenza** relativi al flusso A→B e numeri di **acknowledgment** relativi al flusso B→A;
- i segmenti inviati da B hanno numeri di sequenza relativi al flusso B→A e numeri di acknowledgment relativi al flusso A→B.

Un segmento può quindi portare **insieme dati e conferma** (si dice che l'ACK viaggia "a cavalluccio", *piggybacking*), con il flag ACK a 1.

Esempio 3 — L'eco di un carattere (Telnet)



Due messaggi da un solo byte: A invia la 'C', che B rimanda indietro. In verde le informazioni sul flusso A→B, in rosso quelle sul flusso B→A.

1. **Da A a B**: il numero di sequenza è la posizione del byte trasmesso (42); l'ACK è il prossimo byte atteso sul flusso B→A (79).
2. **Da B ad A**: l'ACK è il prossimo byte atteso sul flusso A→B, cioè 42 + 1 byte = 43; il numero di sequenza è la posizione del byte che B trasmette (79), proprio quello che A aspettava.
3. **Da A a B**: un ACK puro, con numero di sequenza 43 (come atteso da B) e ACK 79 + 1 = 80.

In ogni scambio vale: **ACK ricevuto = Seq inviato + byte inviati**, e **Seq inviato = ACK ricevuto**. I due flussi partono da 42 e 79, concordati all'apertura della connessione, e non da zero.

Anche un segmento senza dati ha un numero di sequenza

Il terzo segmento non porta niente. Che cosa ci fa `Seq=43`? Il campo è nell'header che lo si usi o no, quindi dice dove comincerebbe A se avesse qualcosa da inviare. **Non consuma alcun byte**: il prossimo segmento di A porterà di nuovo 43.

- Un ACK puro non fa avanzare il flusso, e ciò che B aspetta non si muove: sempre il byte 43.
- Quindi nessuno lo conferma: non c'è niente da confermare, e per lui non parte nessun timer.
- Se si perde non si ritrasmette niente: la prossima conferma di A porterà la stessa informazione, o di più.

Da dove parte la numerazione

In tutti gli esempi il primo byte era numerato 0. In una connessione reale **non lo è mai**.

- Ciascun lato sceglie a caso il proprio **numero di sequenza iniziale** (ISN) e lo comunica all'altro durante l'apertura della connessione.
- Il motivo è quello visto con il riuso dei numeri di sequenza: un segmento di una connessione precedente può essere ancora vivo nella rete.
- Stessi due host, stesse due porte, numerazione da zero ogni volta: una vecchia copia verrebbe accettata come dato attuale.
- Un inizio casuale non lo rende impossibile, lo rende **improbabile**, ed è il massimo che il protocollo può fare.

Il mittente TCP in tre eventi

Evento	Che cosa fa il mittente
Arrivano dati dall'applicazione	Costruisce un segmento con numero di sequenza `NextSeqNum`, avvia il timer se non è già in corsa, lo passa a IP e fa avanzare `NextSeqNum` del numero di byte inviati.
Il timer scade	Ritrasmette il segmento non confermato con il numero di sequenza più piccolo, e riavvia il timer.
Arriva una conferma y	Se $y >$ `SendBase`, porta `SendBase` a y, e riavvia il timer se resta qualcosa di non confermato.

Tabella 18.1 – Due variabili contengono l'intero mittente: `SendBase`, il byte più vecchio non ancora confermato, e `NextSeqNum`, il prossimo che partirà.

Un timer, non uno per segmento

- Un timer per ogni segmento in sospeso è il progetto naturale, ed è ciò che chiede Selective Repeat.
- TCP ne tiene **uno**, legato al segmento più vecchio non confermato, quello a cui punta `SendBase`.
- Quando scatta riparte **solo quel segmento**, per quanti altri ce ne siano in volo senza risposta: un mittente con dieci segmenti in sospeso ne ritrasmette uno, e lascia che le conferme cumulative sistemino il resto.
- Il motivo è il costo: una scadenza da seguire invece di una per segmento, su una macchina che può avere migliaia di connessioni.

Il timeout di ritrasmissione

Il trasferimento affidabile di TCP usa ampiamente i timer. Nella trasmissione TCP con pipelining l'approccio di base è ritrasmettere un segmento **solo se il suo ACK non arriva prima del timeout**. La stima del timeout è quindi critica:

- **troppo lungo**: ogni perdita si paga con una lunga attesa a vuoto, e il trasferimento si ferma finché il timer corre;
- **troppo breve**: segmenti soltanto lenti vengono dichiarati persi, e la rete trasporta copie che nessuno aveva chiesto. Il caso estremo lo conosciamo: con un timeout più corto dell'andata e ritorno si ritrasmette tutto, senza perdere niente.

Il timeout deve quindi essere più grande dell'RTT, e l'RTT non è una costante, e nessuno dice al mittente quanto vale.

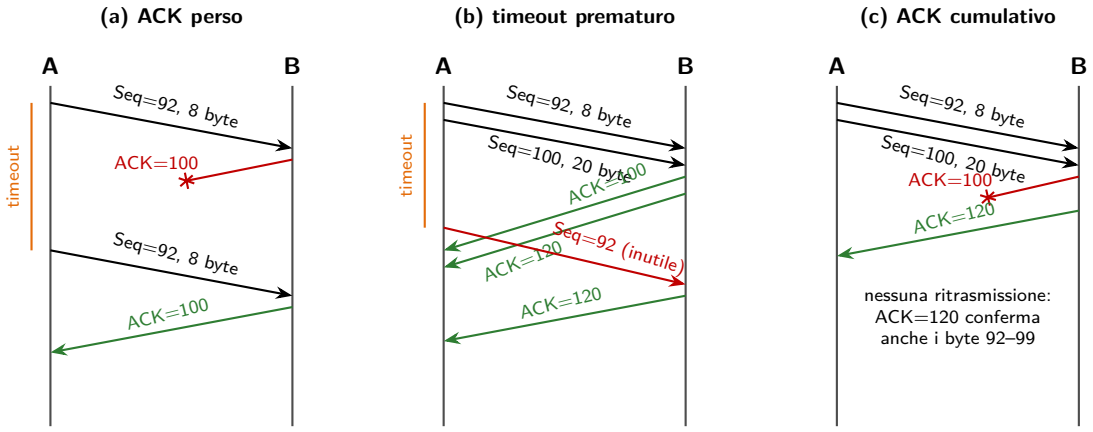


Figura 18.5 – Tre scenari di ritrasmissione TCP. Nel terzo una conferma si è persa e nulla è stato inviato due volte: ACK=120 dice che tutto ciò che sta sotto 120 è arrivato, copre entrambi i segmenti, e `SendBase` salta subito a 120 fermando il timer.

Raddoppiare il timeout

- Quando il timer scade, TCP ritrasmette e imposta il timeout successivo al **doppio** dell'intervallo appena trascorso: tre quarti di secondo diventano 1,5, poi 3, poi 6. Le attese crescono esponenzialmente finché non torna niente.
- Nel frattempo la stima non si usa: torna appena arriva una conferma e il timer riparte normalmente.
- Un timeout di solito significa congestione, e ritrasmettere più forte in una rete congestionata la peggiora. Quindi il mittente **si ritira**: è una forma rudimentale di controllo di congestione, e la stessa idea tornerà quando le stazioni collidono su un collegamento condiviso.

LA STIMA DELL'RTT

Per fissare il timeout serve una stima del **round-trip time**: il timeout (quanto aspettare l'ACK di un segmento) deve essere maggiore dell'RTT, altrimenti si ritrasmette inutilmente.

SampleRTT

Il tempo fra la consegna di un segmento a IP e l'arrivo di una conferma che lo copre. Una misura, per un segmento.

- TCP misura un segmento alla volta, non tutti: più o meno un nuovo campione per andata e ritorno.
- I campioni si muovono, e di molto: le code dei router si riempiono e si svuotano, e l'host all'altro capo ha un carico suo.
- Un singolo campione è quindi una cattiva stima dell'andata e ritorno, e una base pessima per una scadenza.
- E i segmenti ritrasmessi non vengono **mai** campionati.

A quale trasmissione risponde quella conferma?

Un segmento parte, il timer scatta, riparte, e poi arriva una conferma. Porta un numero di byte, non un istante: è identica sia che risponda alla prima copia sia alla seconda, e il mittente **non può saperlo**.

- Misurando dalla prima trasmissione, il campione è troppo lungo se la conferma risponde in realtà alla seconda.
- Misurando dalla seconda, è troppo corto se la prima copia era arrivata e la sua conferma era solo lenta.
- Un campione corto accorcia il timeout, che produce più ritrasmissioni premature, che producono più campioni corti.

La regola (di Karn) taglia il ciclo prima che cominci: **mai campionare un segmento ritrasmesso**.

EstimatedRTT

Poiché i campioni oscillano (congestione, carico del ricevente, …), la stima è una **media mobile esponenziale pesata** (EWMA, *Exponential Weighted Moving Average*):

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

con α tipicamente $0{,}125 = 1/8$.

- Ogni nuovo campione tira la stima di un ottavo verso di sé: una misura strana non può trascinarla lontano.
- Il peso di un campione vecchio si riduce di $7/8$ a ogni aggiornamento, esponenzialmente, da cui il nome.
- Quindi la media **dimentica**: ciò che faceva il percorso un minuto fa ora non conta quasi niente, ed è proprio lo scopo.
- Non si conserva alcuna storia: una variabile, e due operazioni per conferma.

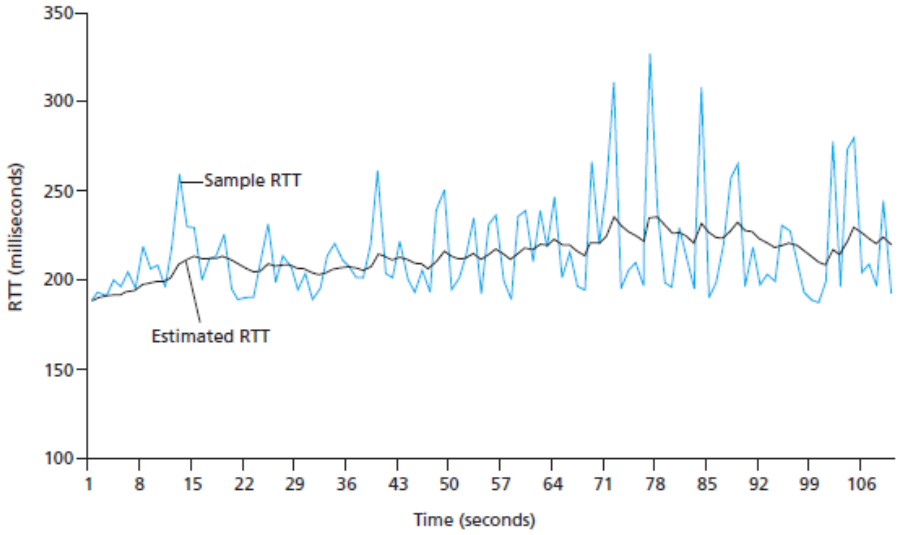


Figura 18.6 – Campioni di RTT (azzurro) e stima EWMA (nero) in una connessione reale tra Stati Uniti e Francia.

DevRTT

Si stima anche la **variabilità** dell'RTT, come media mobile della differenza tra campione e stima:

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\,\text{SampleRTT} - \text{EstimatedRTT}\,|$$

con β tipicamente $0{,}25 = 1/4$.

Il timeout

TimeoutInterval

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

La stima fissa il **minimo**: sotto, le conferme arriverebbero dopo che il timer è già scattato. La deviazione fornisce il **margine**, ampio quando il percorso è irregolare, stretto quando è stabile. Così il timeout segue un collegamento che comincia a comportarsi male senza che nessuno debba scegliere un numero. Prima del primo campione non c'è niente da cui calcolare, quindi il valore iniziale è di **1 secondo**; e dopo una ritrasmissione la formula si mette da parte e l'intervallo raddoppia, finché non torna qualcosa.

Esempio 4 — Aggiornare stima e timeout

Siano EstimatedRTT = 100 ms, DevRTT = 10 ms, e arrivi un campione SampleRTT = 140 ms.

$$\begin{aligned} \text{EstimatedRTT} &= 0{,}875 \cdot 100 + 0{,}125 \cdot 140 = 87{,}5 + 17{,}5 = 105 \text{ ms} \\ \text{DevRTT} &= 0{,}75 \cdot 10 + 0{,}25 \cdot |140 - 105| = 7{,}5 + 8{,}75 = 16{,}25 \text{ ms} \\ \text{TimeoutInterval} &= 105 + 4 \cdot 16{,}25 = 170 \text{ ms} \end{aligned}$$

Due dettagli pratici

- Nello standard (RFC 6298) `DevRTT` viene aggiornata *prima* di `EstimatedRTT`, usando il valore vecchio della stima; molti testi, come le trasparenze, le presentano nell'ordine inverso. Negli esercizi seguite l'ordine indicato dal testo.
- Quando scade un timeout, TCP ritrasmette e **raddoppia** il timeout, invece di ricalcolarlo: se c'è congestione, non ha senso ritrasmettere sempre più in fretta.

FAST RETRANSMIT

La ritrasmissione dopo il timeout funziona, ma il timeout può essere relativamente lungo. Prima di vedere come accorciarlo, bisogna sapere quando il ricevente invia una conferma.

Che cosa arriva	Che cosa fa il ricevente
Segmento in ordine, e tutto ciò che lo precede è già confermato	Aspetta fino a 500 ms il successivo; se non arriva, conferma.
Segmento in ordine, con un segmento precedente ancora da confermare	Conferma entrambi insieme, con un numero solo.
Segmento fuori ordine, con numero più alto dell'atteso: un buco	Conferma **subito**, ripetendo il numero del fondo del buco.
Un segmento che riempie il buco	Conferma subito, fino alla fine di ciò che ora è contiguo.

Tabella 18.2 – Quando il ricevente TCP invia una conferma. La prima riga è la **conferma ritardata** (*delayed ACK*): due segmenti, una risposta. La terza è da dove vengono i duplicati.

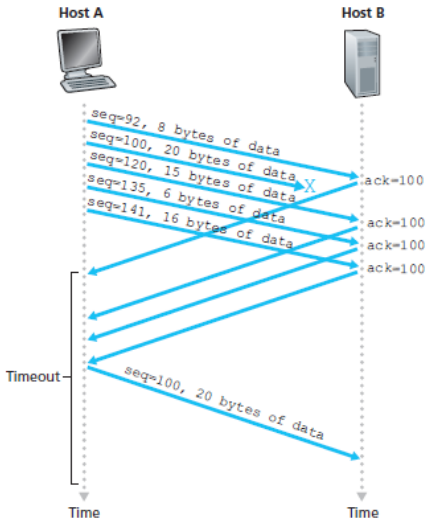
Le conferme duplicate

- Una conferma duplicata ripete un numero che il mittente aveva già: presa da sola non porta alcuna informazione nuova.
- Ma è stata inviata per una ragione: al ricevente è arrivato qualcosa, e non era ciò che serviva al buco.
- Ogni duplicato è quindi la prova che i segmenti dopo quello mancante stanno passando: il percorso funziona, un segmento no.
- Un mittente che qui aspetta il suo timer sta aspettando una scadenza di cui ha già la risposta.
- TCP non ha conferme negative: un numero ripetuto è la cosa più vicina, e non costa un nuovo tipo di pacchetto.

Fast retransmit

Con la **ritrasmissione veloce** il ricevente segnala al mittente che un segmento potrebbe essere perso inviando **ACK duplicati**.

- Se il ricevente riceve un segmento con numero di sequenza **più alto** di quello atteso, significa che probabilmente un segmento è andato perso.
- Il ricevente rimanda allora il vecchio ACK (**ACK duplicato**), con il numero del segmento atteso.
- Se il mittente riceve **3 ACK duplicati**, considera perso il segmento e lo ritrasmette subito, **molto prima** della scadenza del timeout.



- Si perde il segmento con numero di sequenza 100.
- Ogni volta che B riceve un segmento fuori ordine, rimanda un ACK duplicato (ACK=100).
- Al terzo duplicato A considera perso il segmento 100 e lo rispedisce (fast retransmit), senza aspettare il timer.
- Quando B riceve finalmente il segmento mancante, invia l'ACK del prossimo segmento atteso secondo la propria politica (tipo Go-Back-N o Selective Repeat).

Figura 18.7 – Ritrasmissione veloce dopo tre ACK duplicati.

Perché proprio tre duplicati?

Perché non ritrasmettere al primo ACK duplicato? Perché un ACK "sbagliato" può arrivare anche per motivi diversi da una perdita. Per esempio, se due segmenti arrivano **scambiati di ordine**, il ricevente invia un ACK duplicato (per il primo segmento) prima di accorgersi dello scambio. Aspettare tre duplicati evita ritrasmissioni inutili dovute al semplice riordinamento: tre di fila sono difficili da spiegare così, perché vuol dire che tre segmenti successivi sono passati mentre quello no. Il numero è un compromesso fissato dalle misure più che derivato: tre è il valore su cui si è assestato lo standard.

Go-Back-N o Selective Repeat?

- **Come Go-Back-N**: conferme cumulative, nessuna risposta individuale per ciò che arriva in anticipo, un timer e due variabili.
- **Diversamente da Go-Back-N**: un timeout ritrasmette un segmento, non tutto ciò che è in sospeso, e nemmeno quello se la conferma successiva lo copre.
- **Come Selective Repeat**: il ricevente tiene in un buffer ciò che arriva in anticipo invece di buttarlo.

TCP quindi **non è nessuno dei due**: conferme cumulative sopra un ricevente che conserva, cioè la metà utile di ciascuno. Un'opzione, la **conferma selettiva** (*SACK*), permette al ricevente di nominare i blocchi che sta tenendo, e porta TCP il resto della strada.

RIEPILOGO

In breve

- TCP è orientato alla connessione, affidabile, full-duplex, punto-punto e orientato al flusso di byte, con buffer di invio e ricezione.
- MSS + 40 = MTU (in Ethernet MSS tipica 1460 byte).
- Header TCP: porte, **Seq**, **ACK**, lunghezza, flag (SYN, ACK, FIN, RST, …), finestra di ricezione, checksum, opzioni.
- Seq = numero del primo byte del segmento; ACK = prossimo byte atteso (cumulativo).
- EstimatedRTT = $0{,}875 \cdot \text{Est} + 0{,}125 \cdot \text{Sample}$; DevRTT = $0{,}75 \cdot \text{Dev} + 0{,}25 \cdot |\text{Sample} - \text{Est}|$; Timeout = $\text{Est} + 4 \cdot \text{Dev}$.
- Un solo timer, sul segmento più vecchio non confermato; dopo un timeout l'intervallo raddoppia, e i segmenti ritrasmessi non si campionano.
- **Fast retransmit**: 3 ACK duplicati $\Rightarrow$ ritrasmissione immediata.
- Resta aperto: come i due estremi si accordano sui numeri iniziali, e a che cosa serve la finestra di ricezione nell'header.

TCP: LA GESTIONE DELLA CONNESSIONE

IL PROBLEMA

Poiché TCP è orientato alla connessione, i due host devono mettersi d'accordo sia quando la connessione si **apre** sia quando si **chiude**. Ma i messaggi possono perdersi o danneggiarsi, e trovare un accordo è tutt'altro che banale: se un messaggio si perde, un lato può ritenere la connessione aperta (o chiusa) mentre l'altro no.

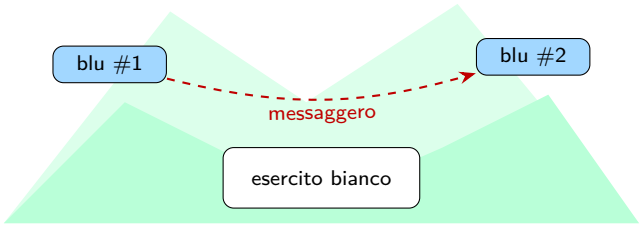
Per evitare, o meglio per **mitigare**, il problema TCP usa una procedura detta **three-way handshake** (stretta di mano a tre vie): le richieste di apertura e di chiusura devono essere **confermate** prima che la connessione sia considerata stabilita o rilasciata. Niente conta finché non è stato confermato, e l'apertura è costruita su questa regola.

Delle cose che fa il trasporto, questa è quella che il nostro codice ha chiesto fin dall'inizio: `connect`, `accept`, `close`. E non serve niente di nuovo nell'header: bastano tre bit di flag mai usati finora.

- **SYN**: questo segmento apre una connessione. È acceso nei primi due segmenti di una connessione e mai più.
- **FIN**: il mittente non ha più dati. Chiude **una direzione**, e si invia una volta per direzione.
- **RST**: qui non c'è nessuna connessione. Butta via il segmento e non rimandarlo.

I campi di sequenza e di acknowledgment portano l'accordo; questi tre bit dicono di che tipo di accordo si tratta.

IL PROBLEMA DEI DUE ESERCITI



Il bianco è più forte di ciascun esercito blu, ma più debole dei due insieme: i blu vincono solo se attaccano **nello stesso momento**. Per accordarsi possono mandare messaggeri attraverso la valle, dove possono essere catturati.

Figura 19.1 – Il problema dei due eserciti: comunicare attraverso un canale inaffidabile.

Un esercito bianco è accampato in una valle; sulle due colline ai lati ci sono due eserciti blu. Il bianco è più grande di ciascuno dei due eserciti blu, ma insieme i blu sono più numerosi: vincono **solo se attaccano contemporaneamente**. Per sincronizzarsi devono mandare messaggeri attraverso la valle, dove possono essere catturati (comunicazione **inaffidabile**). Esiste un protocollo che permetta ai blu di vincere?

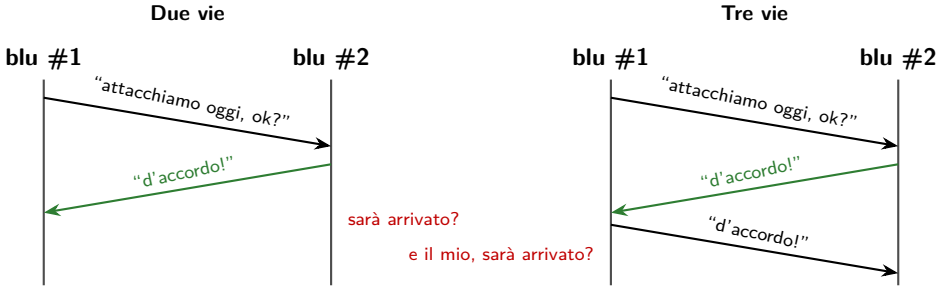


Figura 19.2 – Aggiungere conferme sposta solo il dubbio sull'ultimo messaggio.

- Il comandante del blu #1 manda: "*Propongo di attaccare oggi, va bene?*". Il messaggio arriva, il blu #2 è d'accordo e la sua risposta torna sana e salva. L'attacco ci sarà? Probabilmente no: il comandante #2 **non sa** se la sua risposta è arrivata, e se non fosse arrivata il #1 non attaccherebbe.
- Proviamo con tre vie: il #1 conferma la risposta. Se nessun messaggio si perde, il #2 riceve la conferma, ma ora è il #1 a esitare: non sa se la sua conferma è arrivata.
- Con quattro vie non cambia nulla. Si può **dimostrare** che **non esiste alcun protocollo** che risolva il problema con certezza.

Imperfetto ma adeguato

Il three-way handshake **non è perfetto**, ma nella pratica è di solito **più che adeguato**: le situazioni residue vengono gestite con timer e ritrasmissioni. L'incertezza non riguarda il messaggio: riguarda che cosa sa l'altro.

Che cosa è impossibile

Che entrambi gli estremi siano **certi, nello stesso istante**, che l'altro sia pronto. Nessun numero di messaggi ci riesce, su un canale che perde messaggi.

- Ciò di cui ha bisogno TCP è più debole: ciascun estremo ha imparato il numero iniziale dell'altro, e sa che il proprio è stato ricevuto.
- Tre segmenti danno esattamente questo, e nessuno va creduto sulla parola: ciascuno è confermato dal successivo.
- Il dubbio che resta è quello del mittente sul suo ultimo messaggio, e lo risolve il primo segmento di dati.
- Dove quel dubbio costa qualcosa è alla chiusura, e ci torneremo con i timer.

L'APERTURA DELLA CONNESSIONE

1. La `connect` del client invia una richiesta di connessione: un segmento **SYN**, senza dati.
2. Il server risponde con un SYN proprio che conferma anche quello del client: un solo segmento, **SYN-ACK** (o SYNACK).
3. Il client invia la conferma finale: un segmento **ACK**, che può già portare dati dell'applicazione.

Un'intera andata e ritorno, più il tempo che il server impiega a rispondere, prima del primo byte della richiesta.

I dettagli

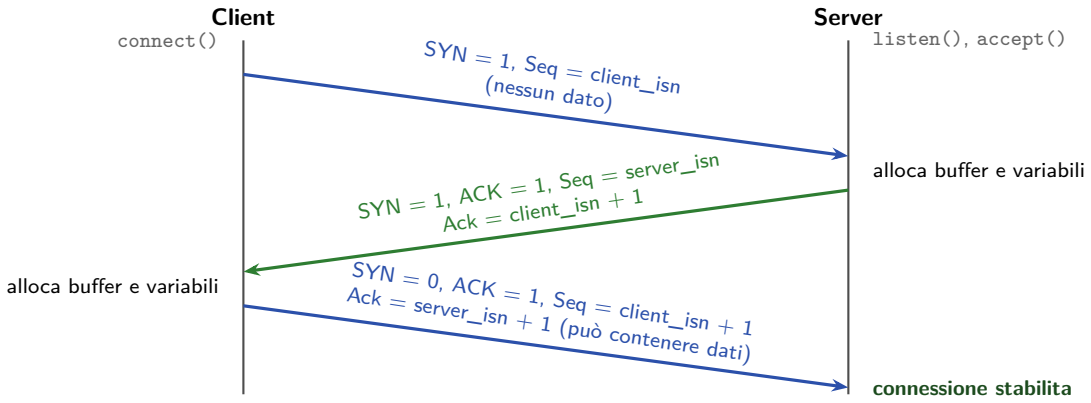


Figura 19.3 – Il three-way handshake nel dettaglio.

1. Il client invia un segmento **SYN** che ha:
   - **nessun dato** applicativo;
   - il bit **SYN a 1**;
   - come numero di sequenza un **numero iniziale casuale** (`client_isn`, *initial sequence number*).
2. Quando il SYN arriva (si spera), il server **alloca buffer e variabili** TCP e risponde con un segmento **SYNACK** che ha:
   - nessun dato applicativo;
   - i bit **SYN e ACK a 1**;
   - numero di acknowledgment `client_isn + 1`;
   - come numero di sequenza un proprio numero iniziale casuale (`server_isn`).
3. Quando il SYNACK arriva (si spera), anche il client alloca buffer e variabili e invia un segmento **ACK** finale che ha:
   - eventualmente **dati** applicativi (per esempio la richiesta HTTP);
   - il bit **SYN a 0** (la connessione ora è stabilita) e il bit **ACK a 1**;
   - numero di acknowledgment `server_isn + 1`.

Perché il SYN "consuma" un numero di sequenza

Il SYN non porta dati, eppure l'ACK di risposta vale `client_isn + 1`: per convenzione SYN (e FIN) contano come un byte, così possono essere confermati come qualsiasi altro dato.

Perché il numero iniziale è estratto a caso

- Niente obbliga una connessione a partire da zero, e partire da lì sarebbe la scelta peggiore possibile.
- Un segmento di una connessione precedente fra gli stessi quattro numeri può essere ancora in giro nella rete. Con un punto di partenza nuovo cade fuori dalla finestra della nuova connessione, e viene scartato invece di essere accettato come dato.
- Un attaccante che indovina i numeri può iniettare segmenti in una connessione che non vede: renderli casuali è una difesa.
- E il +1 che torna indietro è una piccola prova a sé: solo chi ha ricevuto il SYN poteva scriverlo.

Diagramma degli stati (apertura)

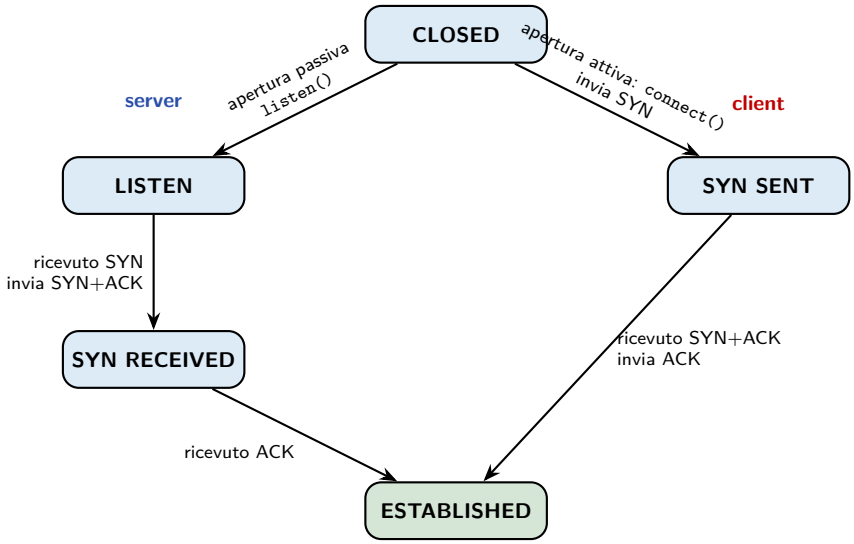


Figura 19.4 – Diagramma degli stati semplificato per l'apertura della connessione. Due percorsi dentro la stessa macchina: il server aspetta (apertura passiva) e risponde a un SYN con un SYNACK; il client chiede (apertura attiva) e invia il SYN. Si incontrano in ESTABLISHED, il server un segmento dopo il client.

Gli stessi stati, visti dal nostro codice

- `listen` è la transizione in LISTEN. Da quell'istante il kernel risponde ai SYN **da solo**, senza alcun processo coinvolto.
- `connect` invia il SYN e si blocca; ritorna quando è arrivato il SYNACK ed è partito il terzo segmento.
- `accept` **non partecipa affatto** all'handshake: consegna una connessione già stabilita. Un server può quindi essere lento a chiamare `accept` senza che alcun client se ne accorga: la connessione è aperta, e i dati aspettano in un buffer.
- L'handshake lo fanno i due kernel. Il programma ne legge il risultato, e il suo unico resoconto è un descriptor o un errore.

Le due code di listen

Un socket di ascolto ha una coda delle connessioni **mezze aperte** (SYN ricevuto, terzo segmento non ancora) e una coda di quelle **completate** che aspettano `accept`.

- Un SYN in arrivo crea una voce nella prima coda; l'ACK finale la sposta nella seconda.
- `accept` toglie una voce dalla seconda coda: il *backlog* passato a `listen` è la lunghezza di quella coda.
- Se si riempie, le nuove connessioni vengono scartate o rifiutate, e il client vede un ritardo senza mai saperne il motivo.
- La prima coda è più piccola, più vecchia e sguarnita: è quella a cui mira un attaccante.

Quanto costa l'handshake

- Prima che parta il primo byte di una richiesta, un'intera andata e ritorno è già stata spesa per mettersi d'accordo a parlare. Su una rete locale non è niente; attraverso un oceano è un decimo di secondo, per connessione.
- È esattamente ciò che comprano le **connessioni persistenti** del livello applicativo: si paga l'handshake una volta, e si riusa ciò che ha costruito.
- Costa anche al server: buffer e variabili vengono impegnati al primo segmento, prima che gli sia stato chiesto niente.

Nessuno risponde, oppure qualcuno rifiuta

Il client chiama `connect` e il SYN non riceve risposta. Quanto aspetta? **Decine di secondi**: TCP ritrasmette il SYN qualche volta con un intervallo crescente, poi abbandona il tentativo e `connect` fallisce.

- Un rifiuto è molto più veloce: se l'host è acceso ma nessuno ha fatto `bind` su quella porta, risponde con un segmento con **RST**.
- Il programma vede due errori diversi: un timeout, o l'`ECONNREFUSED` ottenuto la prima volta che si è avviato il client prima del server.
- Per UDP quella segnalazione arrivava dal livello di rete; qui la risposta è un segmento di trasporto, dal TCP del peer.

Il SYN flood e i SYN cookie

E se il terzo segmento non arriva mai? Il server ha già impegnato buffer e variabili, e tiene la connessione mezza aperta per un minuto o più prima di recuperarla.

- Inviando abbastanza SYN, la coda delle connessioni mezze aperte si riempie di voci che non si completeranno mai, e i client legittimi vengono rifiutati: si è esaurita una **coda**, non la banda, e l'attacco richiede pochissimo traffico.
- Gli indirizzi sorgente sono falsificati, quindi i SYNACK vanno a host che non hanno chiesto niente, e l'attaccante non deve mai rispondere.
- Il **SYN flood** è stato fra i primi attacchi di *denial of service* documentati, ed è ancora fra i più comuni.

SYN cookie

La difesa capovolge il problema: **rispondere al SYN senza ricordarlo**. `server_isn` non è casuale ma calcolato: un hash dei due indirizzi, delle due porte e di un segreto noto solo al server.

- Non si tiene alcuno stato, quindi un SYN falsificato costa al server una risposta e nient'altro.
- Un client legittimo restituisce quel numero più uno. Il server ricalcola l'hash, confronta, e solo allora crea la connessione e il suo socket.
- La coda non si riempie mai, perché fino al terzo segmento non c'è niente dentro. Oggi lo fanno quasi tutti i sistemi operativi.

Che cosa torna	Che cosa significa	Stato della porta
SYNACK	qualcosa ascolta lì	*open*
RST	l'host è acceso, quella porta non è servita	*closed*
niente	non è mai arrivato: qualcosa l'ha scartato	*filtered*

Tabella 19.1 – Le tre risposte a un SYN. Un SYN per porta, e le tre risposte mappano una macchina dall'esterno senza mai completare una connessione: è ciò che fa `nmap`, ed è il significato degli stati visti con le porte.

La chiusura della connessione

La chiusura (*teardown*) può essere avviata da **uno qualsiasi** dei due host. Supponiamo che a chiudere sia il client:

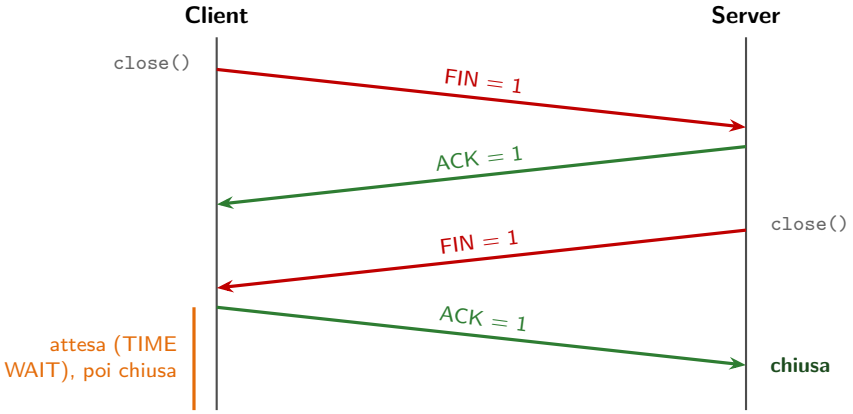


Figura 19.5 – La chiusura della connessione: ACK e FIN del server possono viaggiare nello stesso segmento o in due segmenti separati.

1. Il client invia un segmento speciale di chiusura: un segmento **FIN**, con il bit FIN a 1.
2. Quando lo riceve, il server invia una conferma e, quando anche lui ha finito, un proprio segmento di chiusura, con ACK a 1 e FIN a 1. ACK e FIN possono viaggiare nello stesso segmento o in due separati: nel mezzo il server può ancora inviare dati (la connessione è chiusa solo in una direzione).
3. Infine il client conferma il FIN del server.

Mezza chiusura: una direzione alla volta

- Un FIN dice «non ho più dati da inviare». Non dice «questa connessione è finita»: chiude **una direzione**.
- Dopo aver inviato il suo FIN il client può ancora ricevere; dopo averlo confermato il server può ancora inviare, per tutto il tempo che vuole.
- I due FIN sono quindi indipendenti, e il rilascio normale è di **quattro** segmenti: FIN, ACK, poi FIN, ACK nell'altro verso. Si riducono a tre solo quando il peer non ha più niente da inviare e chiude subito: allora il suo ACK e il suo FIN viaggiano insieme.
- `close` invia il FIN e rinuncia al descriptor. `shutdown(fd, SHUT_WR)` lo invia e continua a leggere: è il modo in cui un client dice «questa era tutta la richiesta».

E dal lato del programma, il FIN che arriva è proprio lo `0` restituito da `recv`: un'istruzione a chiudere, non un errore da ignorare.

Diagramma degli stati (chiusura)

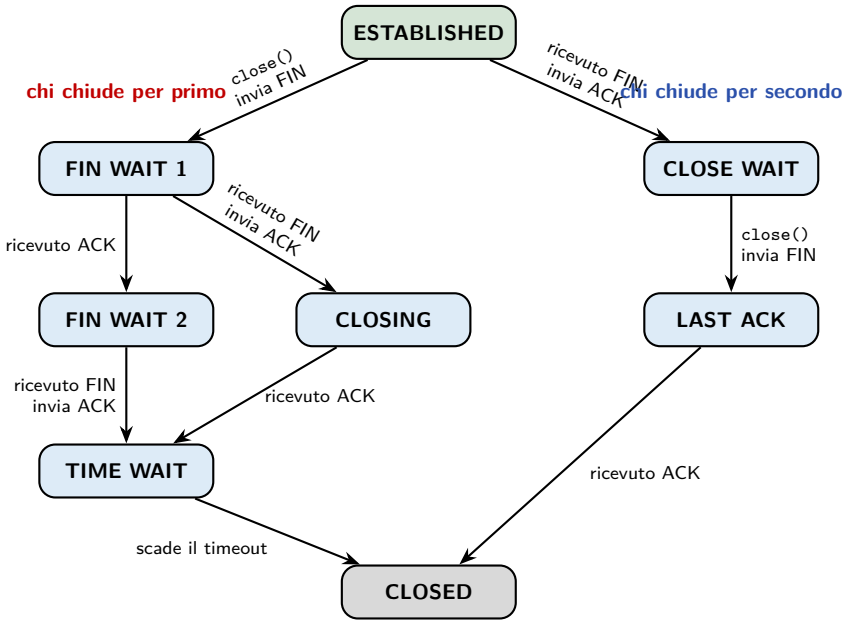


Figura 19.6 – Diagramma degli stati semplificato per il rilascio della connessione. Chi chiude per primo scende a sinistra; chi risponde scende a destra, e resta in CLOSE WAIT finché la sua applicazione non chiude. CLOSING corrisponde al caso raro in cui i due lati inviano il FIN contemporaneamente e nessuno dei due è secondo.

Il descriptor che nessuno ha chiuso

La `recv` del server restituisce 0 e ci si dimentica di chiudere il descriptor. Che cosa si rompe? All'inizio **niente**. La connessione resta in CLOSE WAIT per sempre: il FIN del peer è stato confermato, ma il nostro non parte mai.

- Anche il peer resta in attesa. Il rilascio non finisce su nessuno dei due lati, e nessuno dei due programmi viene avvisato.
- Qualche migliaio di client dopo il server ha finito i descriptor e smette di accettare: il guasto compare lontano dalla sua causa.
- `ss -tn state close-wait` elenca esattamente quelle connessioni. Una colonna di CLOSE WAIT è **sempre un bug del programma**, mai della rete.

TIME WAIT

- Chi chiude per primo non passa subito a CLOSED: aspetta, di solito da trenta secondi a due minuti.
- **Primo motivo**: se la sua ultima conferma si perde, il peer rimanderà il suo FIN, e qualcuno deve essere lì a rispondere.
- **Secondo motivo**: segmenti di questa connessione possono essere ancora in rete, e i suoi quattro numeri potrebbero essere assegnati a una connessione nuova.
- L'attesa è circa il doppio del tempo massimo per cui un segmento può sopravvivere, così tutto ciò che appartiene alla vecchia connessione è morto prima.
- È il prezzo di chiudere per primi, e lo paga chi chiude per primo, che non è sempre il client.

Address already in use

Si interrompe il server, lo si riavvia, e `bind` fallisce. Non c'è nient'altro in esecuzione. Perché? Perché interromperlo lo ha reso **l'estremo che ha chiuso per primo**. Le sue connessioni sono in TIME WAIT, ancora legate alla porta locale, e `bind` non prende una porta che una connessione viva sta usando.

- `SO_REUSEADDR`, la riga scritta in ogni server fin dal primo, dice di fare `bind` lo stesso.
- È sicuro: la vecchia connessione conserva i suoi quattro numeri, e nessun suo segmento può essere consegnato al nuovo socket.
- Quando chiude prima il client, che è il caso abituale, il server non entra mai in TIME WAIT e il problema non compare.

Come TCP si salva dalla perdita di pacchetti

Poiché il three-way handshake non è perfetto, TCP si affida ai **timer** per chiudere comunque le connessioni o per ripetere le richieste. Alcuni esempi durante il rilascio:

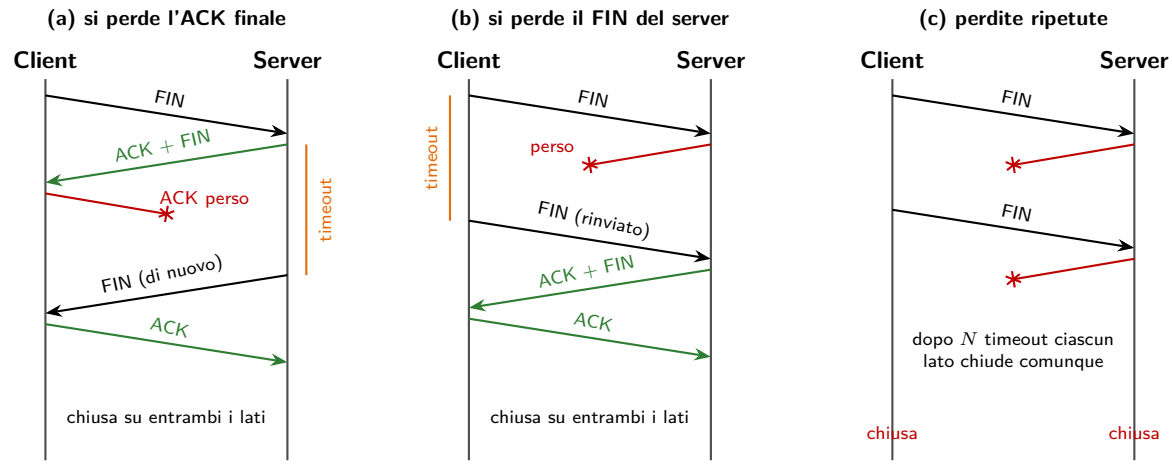


Figura 19.7 – Timer e ritrasmissioni permettono di chiudere la connessione anche quando si perdono segmenti; dopo un certo numero di tentativi falliti la connessione viene chiusa d'ufficio.

- **Si perde l'ultima conferma**: il client la invia e si considera chiuso; il server resta ad aspettarla. Il suo timer scade e rimanda il FIN (a cui risponde il client in TIME WAIT), oppure, finiti i tentativi, rilascia comunque la connessione. Non si danneggia niente: entrambi finiscono chiusi, solo non nello stesso istante, e nessuno dei due lo saprà mai. È il dubbio dei due eserciti, e qui costa un ritardo da una parte.
- **Si perde la risposta al FIN**: il client non ha saputo ancora niente. Il suo timer scade e rimanda il FIN, esattamente come ritrasmetterebbe un segmento di dati; il server risponde con lo stesso segmento di prima. La ritrasmissione alla chiusura non è un meccanismo nuovo: un timer, e una copia tenuta finché non è confermata.
- **Le perdite continuano**: dopo un numero limitato di tentativi il client smette e chiude da solo, e il timer del server chiude il suo lato. La connessione finisce senza che nessuno dei due abbia confermato niente all'altro, che è proprio il punto di partenza dei due eserciti. Nessun protocollo lo elimina: ciò che aggiunge TCP è un timer, un limite ai tentativi, e le risorse rilasciate in ogni caso.

RST: chiudere senza chiedere

Un FIN termina una connessione in ordine. Un **RST** (*reset*) la termina **ora**, e ciò che era ancora nei buffer viene scartato.

- Viene inviato ogni volta che arriva un segmento per una connessione che non esiste: nessuno in ascolto su quella porta, o un peer che si è riavviato. Nel caso di UDP, invece, a una porta chiusa si risponde con un messaggio ICMP "port unreachable" (lo vedremo con traceroute).
- Una macchina che va in crash non invia niente. Il suo peer tiene una connessione che all'altro capo non esiste più, e lo scopre solo quando scrive.
- Allora torna l'RST, e il programma lo apprende come `ECONNRESET` su una lettura, oppure `EPIPE` e `SIGPIPE` su una scrittura.
- La sonda d'inattività del kernel parte dopo circa due ore: per questo un programma che deve accorgersi di un peer morto invia un proprio *heartbeat*.

In laboratorio: una connessione vista da vicino

Una macchina, due finestre. `nc` dà un estremo TCP senza scrivere un programma: in ascolto da una parte, in connessione dall'altra.

```
$ nc -l 5001                        # server: in ascolto sulla porta 5001
Maradona
sfogliatella

$ ss -tln 'sport = :5001'           # qualcuno ascolta?
$ nc localhost 5001                 # client: connettersi e scrivere
Maradona
sfogliatella
```

Due terminali, una connessione

Ciò che si scrive sul client compare sul server. Nell'altro verso viaggia solo una conferma, un segmento senza alcun dato. Per catturare il traffico in Wireshark si usa il filtro `tcp && tcp.port==5001 && ip`:

- tre clausole, una per livello della domanda: il protocollo di trasporto, la porta, il protocollo di rete sotto;
- quella centrale basterebbe da sola, perché nient'altro sulla macchina usa la 5001: le altre due rendono esplicita l'intenzione;
- tutto avviene su `127.0.0.1`: i due estremi sono lo stesso host, e nemmeno un pacchetto raggiunge un cavo. L'interfaccia su cui catturare è quindi quella di **loopback**; puntando lo sniffer su quella reale non si vede niente.

```
$ tshark -i lo -f "tcp port 5001"
 1  0.000000 41230 -> 5001  [SYN]       Seq=0  Win=65495 Len=0
 2  0.000018  5001 -> 41230 [SYN, ACK]  Seq=0  Ack=1     Len=0
 3  0.000027 41230 -> 5001  [ACK]       Seq=1  Ack=1     Len=0
 4  6.213004 41230 -> 5001  [PSH, ACK]  Seq=1  Ack=1     Len=9
 5  6.213021  5001 -> 41230 [ACK]       Seq=1  Ack=10    Len=0
 6  9.402771 41230 -> 5001  [PSH, ACK]  Seq=10 Ack=1     Len=13
 7  9.402788  5001 -> 41230 [ACK]       Seq=1  Ack=23    Len=0
 8 19.884113 41230 -> 5001  [FIN, ACK]  Seq=23 Ack=1     Len=0
```

Che cosa mostra la cattura

- Tre segmenti prima che si prema un tasto: l'handshake.
- Poi un segmento per ogni riga scritta, di nove e tredici byte (c'è dentro il carattere di a capo), ciascuno con una conferma pura che torna indietro: `Ack=10` è 1 + 9, `Ack=23` è 10 + 13.
- L'ultima riga è il Ctrl-C, che chiude e fa partire il FIN.
- I numeri di sequenza partono da 0 perché Wireshark e `tshark` mostrano numeri **relativi**: sottraggono l'ISN casuale per renderli leggibili. Quelli veri sono nel dettaglio del pacchetto.

```
$ nc -l 12338 > real2.doc           # server: scrive quanto arriva
$ nc 127.0.0.1 12338 < real.doc     # client: invia un file
^C
```

La stessa connessione, con un file

- Le due ridirezioni fanno tutto: `nc` invia il proprio standard input e stampa ciò che riceve. La copia avviene su TCP.
- Ciò che viaggia è un flusso di byte senza struttura: niente sul filo dice dove finisce il file.
- `nc` non si ferma quando l'input finisce, quindi il trasferimento si chiude a mano, ed è il Ctrl-C a inviare il FIN. Qui il FIN è il segnale di fine file: un protocollo che deve inviare due file su una connessione deve dirlo da sé (è la delimitazione per chiusura della parte sulla programmazione).

RIEPILOGO

In breve

- Accordarsi su un canale inaffidabile con certezza è impossibile (**problema dei due eserciti**): il three-way handshake è imperfetto ma adeguato.
- **Apertura**: SYN (Seq = client_isn), SYNACK (Seq = server_isn, Ack = client_isn+1), ACK (Ack = server_isn+1, può contenere dati). Numeri iniziali casuali.
- Stati di apertura: CLOSED, LISTEN / SYN SENT, SYN RECEIVED, ESTABLISHED.
- **Chiusura**: FIN, ACK, FIN, ACK (ACK e FIN possono essere uniti); stati FIN WAIT 1/2, CLOSE WAIT, LAST ACK, CLOSING, TIME WAIT, CLOSED.
- `listen` crea due code (mezze aperte e completate); `accept` non partecipa all'handshake. I SYN cookie difendono dal SYN flood senza tenere stato.
- Un FIN chiude una direzione; lo 0 di `recv` è il FIN del peer. Un CLOSE WAIT che non finisce è un bug nostro; TIME WAIT protegge l'ultima conferma e i quattro numeri, ed è il motivo di `SO_REUSEADDR`.
- I **timer** risolvono le perdite residue; RST rifiuta o interrompe le connessioni.
- Resta aperto: la finestra di ricezione nell'header, e che cosa fa un mittente quando il limite non è il ricevente ma la rete.

Controllo di flusso e controllo di congestione

Il problema comune: i buffer che traboccano

C'è un campo dell'header che non abbiamo mai usato, la finestra di ricezione, e un limite che non sta in nessun header.

- La finestra di ricezione dice quanto spazio resta nel buffer all'altro capo: un limite fissato dal **ricevente**.
- L'altro limite lo fissa la **rete**, che non annuncia niente e va indovinato da ciò che torna indietro.
- Entrambi finiscono nella stessa azione, inviare di meno, per ragioni che non hanno niente a che vedere l'una con l'altra. L'affidabilità ci ha detto che cosa fare dopo una perdita; niente ancora dice come non causarla.

Oltre all'affidabilità, TCP offre rispetto a UDP il **controllo di flusso** e il **controllo di congestione**. Entrambi nascono dallo stesso fatto: la perdita di pacchetti su Internet raramente è un cavo rotto, e quasi sempre è un **buffer pieno**.

- Il **buffer di ricezione** del destinatario trabocca se l'applicazione non legge i dati abbastanza in fretta ⇒ problema di **flusso**.
- I **buffer dei dispositivi di rete** (i router) traboccano se i nodi sono congestionati ⇒ problema di **congestione**.

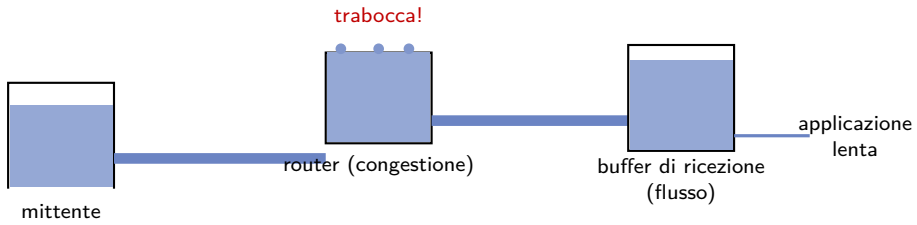


Figura 20.1 – L'analogia dell'acqua: i buffer sono secchi intermedi che traboccano se entra più acqua di quanta ne esca.

Se i pacchetti si perdono, gli host devono affidarsi a ritrasmissioni e timeout, che curano il sintomo e aggiungono traffico a una rete che ne aveva già troppo. Meglio evitare di arrivarci.

Il controllo di flusso

Quando TCP riceve byte corretti e in sequenza, li mette nel **buffer di ricezione**. L'applicazione li legge da lì, ma non necessariamente nell'istante in cui arrivano (può essere occupata). Se l'applicazione è lenta a leggere, il mittente può facilmente far traboccare il buffer inviando troppi dati troppo in fretta.

Controllo di flusso

Il **controllo di flusso** di TCP è un servizio di **adeguamento della velocità** (*speed-matching*): cerca di adeguare (ridurre) il ritmo di invio del mittente al ritmo con cui l'**applicazione ricevente** legge i dati. Si basa sulla **finestra di ricezione** (*receive window*, rwnd), il campo dell'header TCP con cui il ricevente comunica al mittente quanto spazio libero gli resta nel buffer.

- Un mittente veloce e un lettore lento fanno traboccare il buffer con una rete che ha fatto tutto giusto.
- Anche il controllo di congestione frena il mittente, per una ragione del tutto diversa: i due vengono confusi di continuo.
- UDP non ha niente di tutto questo: il buffer davanti al socket si riempie e i datagrammi vengono scartati senza una parola.

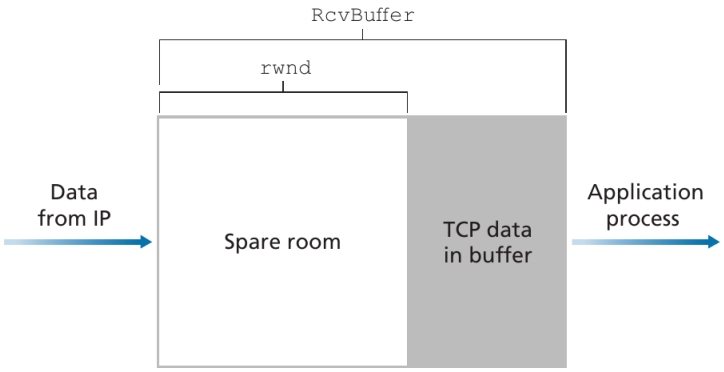


Figura 20.2 – Il buffer di ricezione: la parte grigia contiene dati non ancora letti dall'applicazione, la parte bianca (rwnd) è lo spazio libero.

Le formule

Supponiamo per semplicità che il ricevente scarti i segmenti fuori ordine, così che tutti i dati nel buffer siano in ordine. Sul **ricevente** (host B) si definiscono:

- `RcvBuffer`: la dimensione del buffer di ricezione (in byte);
- `LastByteRead`: l'ultimo byte del flusso letto dall'applicazione;
- `LastByteRcvd`: l'ultimo byte del flusso arrivato e messo nel buffer.

Finestra di ricezione

$$\text{rwnd} = \text{RcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$$

cioè lo spazio totale meno i byte ricevuti ma non ancora letti.

Sul **mittente** (host A) si tengono `LastByteSent` (l'ultimo byte inviato) e `LastByteAcked` (l'ultimo byte confermato dal ricevente). Conoscendo la finestra comunicata da B, il mittente garantisce in ogni istante:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

cioè i byte "in volo" (inviati ma non confermati) non superano mai lo spazio libero del ricevente, quindi il buffer non può traboccare. rwnd cambia ogni volta che uno dei due numeri si muove, quindi va annunciata **di continuo**: il ricevente la scrive nel campo finestra di ogni segmento che rimanda indietro. E la connessione è full-duplex: ogni direzione ha la propria finestra, e ogni estremo ne annuncia una.

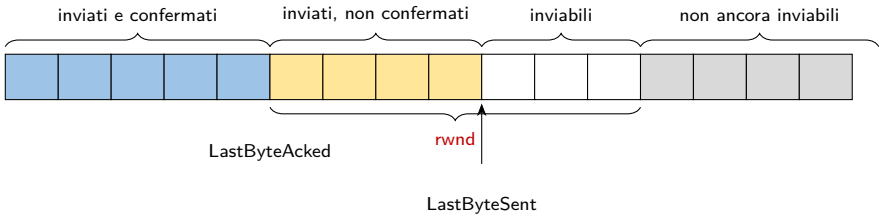


Figura 20.3 – Lato mittente: il flusso di byte e la finestra consentita dal ricevente.

Esempio 1 — Calcolare la finestra

B ha un buffer da 4000 byte. Ha ricevuto fino al byte 3000 (`LastByteRcvd` = 3000) e l'applicazione ha letto fino al byte 1000 (`LastByteRead` = 1000). Allora:

$$\text{rwnd} = 4000 - (3000 - 1000) = 2000 \text{ byte}$$

B scrive 2000 nel campo finestra dei segmenti che invia ad A. Se A ha già inviato fino al byte 3500 e ha ricevuto conferma fino al 3000, ha 500 byte in volo e può inviarne ancora al massimo 2000 − 500 = 1500.

Finestra a zero

Se il buffer di B si riempie, B comunica rwnd = 0 e A si ferma. Ma se B non ha dati da inviare, non manderà più segmenti, e A non saprebbe mai che si è liberato spazio! Per evitare il blocco, quando la finestra è zero il mittente continua a inviare periodicamente segmenti di **un byte** (*zero window probe*): le risposte di B contengono il valore aggiornato della finestra. Sono le **sonde di finestra**, e si vedono in una cattura: un trasferimento fermo non è un trasferimento silenzioso.

Sedici bit non bastano più

- Il campo contiene al massimo 65 535: la finestra più grande che un ricevente possa annunciare è 64 KB.
- Una finestra è anche un **limite di velocità**: al massimo una finestra per andata e ritorno, quindi al massimo finestra/RTT. 64 KB su un percorso da 100 ms sono circa 5 Mb/s, qualunque cosa valga il collegamento sotto.
- Il **prodotto banda-ritardo** dice quanto deve stare in volo per riempire il tubo: 1 Gb/s sullo stesso percorso sono 12,5 MB in volo, duecento volte il campo.
- Per questo l'opzione **window scale** porta un numero di bit di cui spostare il valore, concordato nei due segmenti SYN e mai più. Il numero nel campo non è la finestra: va spostato, e una cattura che ha perso l'handshake non può sapere di quanto.

Il problema della congestione

Il problema della congestione è simile a quello del flusso, ma riguarda l'**infrastruttura di rete** invece del ricevente.

- Il controllo di flusso è un accordo fra i due estremi: la rete non ne fa parte e non vi compare.
- Ma il percorso è condiviso: un mittente può stare ben dentro la finestra del ricevente e immettere comunque più di quanto il percorso porti.
- E **nessuno lo dice**: i router non mandano lamentele, e nessuno dei due estremi vede la coda che si sta riempiendo.

Due domande: che cosa costa davvero la congestione, e come fa un mittente ad accorgersene? Alla prima rispondono tre scenari, e la risposta peggiora ogni volta.

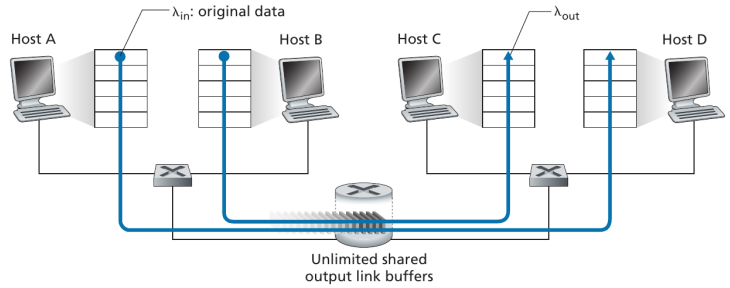


Figure 3.43 ♦ Congestion scenario 1: Two connections sharing a single hop with infinite buffers

Figura 20.4 – Due connessioni condividono un solo router con buffer infinito e un collegamento in uscita di capacità R.

Due host (A e B) hanno ciascuno una connessione che attraversa lo stesso router e lo stesso collegamento in uscita di capacità R. Il router ha dei buffer che conservano i pacchetti quando la velocità di arrivo supera la capacità del collegamento. Entrambe le applicazioni inviano dati alla stessa velocità media di λ_{in} byte/s.

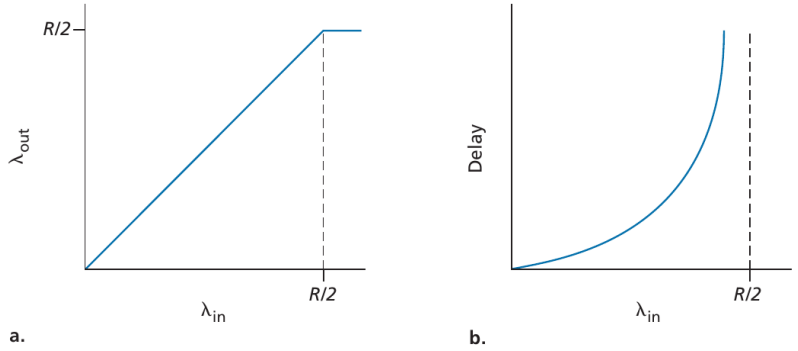


Figure 3.44 ♦ Congestion scenario 1: Throughput and delay as a function of host sending rate

Figura 20.5 – (a) Throughput per connessione in funzione della velocità di invio: non supera $R/2$. (b) Ritardo: cresce senza limite avvicinandosi a $R/2$.

Qui il trasporto non fa niente: nessuna ritrasmissione, nessun controllo di flusso o di congestione. I router sono store-and-forward, quindi hanno buffer, e in questo primo scenario i buffer sono infiniti: non si scarta mai niente.

- Se $\lambda_{in} < R/2$, tutto ciò che il mittente invia arriva al ricevente con un ritardo finito.
- Se $\lambda_{in} = R/2$, il collegamento lavora a piena capacità (R), e i pacchetti in eccesso finiscono nel buffer.
- Finché si supera la capacità, i pacchetti si accumulano nel buffer in attesa del loro turno.
- Anche un buffer **infinito** non basta: i pacchetti non vengono persi, ma il **ritardo** cresce continuamente; se si supera la capacità per sempre, il ritardo tende all'infinito, e i pacchetti sono come persi.

Qui non si perde niente: le code da sole, senza alcuna perdita, rendono già inutilizzabile un collegamento vicino alla capacità.

Scenario 2: buffer finito e ritrasmissioni

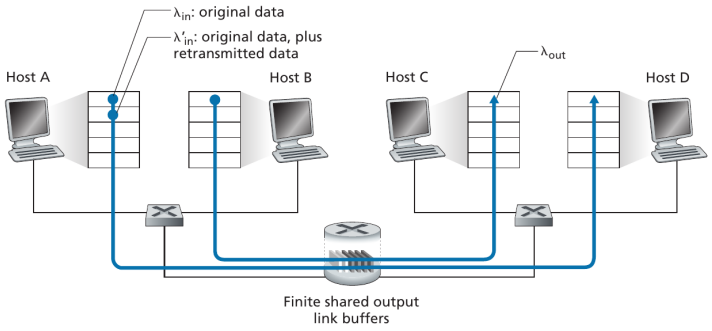


Figure 3.45 ♦ Scenario 2: Two hosts (with retransmissions) and a router with finite buffers

Figura 20.6 – Lo stesso scenario con un buffer finito: i pacchetti in eccesso vengono scartati e i mittenti ritrasmettono.

Poiché il buffer è finito, i pacchetti accumulati prima o poi vengono **scartati**, e gli host devono **ritrasmetterli**: altro traffico che si aggiunge a quello già eccessivo. Ora le velocità sono tre invece di una:

- λ_{in}: i dati **originali**, ciò che l'applicazione consegna al socket;
- λ'_{in}: il **carico offerto**, dati originali più ritrasmissioni, cioè ciò che vede davvero il collegamento;
- λ_{out}: ciò che viene consegnato all'altro capo.

Nel primo scenario erano lo stesso numero. Ora non lo sono più, e le differenze fra loro sono il costo.

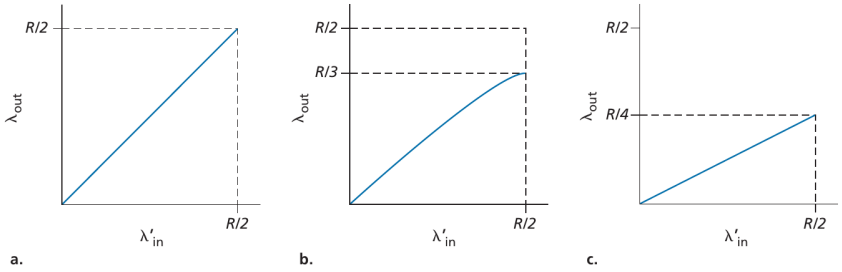


Figura 20.7 – (a) Ritrasmissioni solo dei pacchetti davvero persi: si arriva a $R/2$. (b) Parte della capacità va in ritrasmissioni: il throughput utile scende (per esempio a $R/3$). (c) Con timeout prematuri si ritrasmettono anche pacchetti non persi: il throughput utile scende ancora (per esempio a $R/4$).

- (a) Un mittente che sapesse magicamente quando c'è posto nel buffer: non si perde niente, e ciò che si offre viene consegnato.
- (b) Ritrasmettendo solo ciò che si è davvero perso: con un carico offerto di $R/2$ solo $R/3$ sono dati utili, il resto del collegamento trasporta seconde copie.
- (c) Con timer che scattano troppo presto ogni pacchetto viene inoltrato in media due volte, e la parte utile scende a $R/4$. ($R/3$ e $R/4$ sono valori d'esempio: ciò che mostrano è il degrado, non una costante.)

Scenario 3: la perdita al secondo salto

- Ora i mittenti sono quattro, su percorsi di due salti che si sovrappongono: ogni connessione compete con una diversa a ciascun router.
- Il traffico che ha già attraversato il primo router deve lottare per un posto nel buffer del secondo.
- Al crescere del carico offerto il throughput di una connessione non si livella: **scende**, e al limite va a zero.
- Il costo peggiore: quando un pacchetto viene scartato al secondo salto, tutta la capacità spesa per portarlo fin lì è sprecata. Il primo router avrebbe fatto meglio a restare fermo.

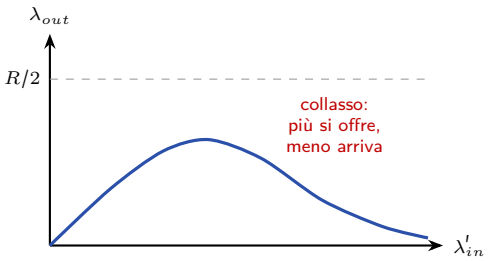


Figura 20.8 – Il throughput di una connessione nello scenario a più salti: oltre un certo carico offerto cala fino a zero.

Collasso da congestione

Il **collasso da congestione** è lo stato in cui la rete è piena di pacchetti che verranno scartati prima di arrivare: lavora al massimo e consegna quasi niente. È successo a Internet reale, alla fine degli anni Ottanta, e tutto ciò che segue in questo capitolo è stato scritto in risposta.

I costi della congestione

- **Ritardi** di accodamento elevati quando la velocità di arrivo si avvicina alla capacità.
- Il mittente deve **ritrasmettere** i pacchetti scartati, sprecando capacità.
- Timeout prematuri causano ritrasmissioni di pacchetti **non persi**: altro spreco.
- Un pacchetto scartato a metà cammino ha sprecato la capacità di tutti i collegamenti già attraversati.

I DUE APPROCCI AL CONTROLLO DI CONGESTIONE

1. **Controllo end-to-end**: è l'approccio standard di TCP. La congestione è **dedotta** dai sistemi terminali osservando il comportamento della rete (perdite, ritardi), senza alcun aiuto dal livello di rete: è ciò che impone IP, che non offre alcun riscontro. La perdita di un segmento (timeout o 3 ACK duplicati) è presa come segnale di congestione, e TCP riduce la velocità di invio.
2. **Controllo assistito dalla rete**: approccio più recente e facoltativo, in cui trasporto e rete collaborano. I **router** forniscono un riscontro esplicito sullo stato di congestione: il mittente viene informato invece di dover indovinare. Il messaggio può tornare direttamente al mittente (un *choke packet*), oppure proseguire come un segno su un pacchetto, che il ricevente poi rimanda indietro. Il secondo modo costa un'intera andata e ritorno, ed è quello che Internet ha adottato.

ECN: due bit e due flag

- Due bit nell'header IP la portano: un valore dice che entrambi gli estremi sanno gestire ECN, un altro dice «questo router è congestionato».
- Il router congestionato **non scarta niente**: marca un datagramma che è già in viaggio verso il ricevente.
- Il TCP ricevente gira il segno e accende **ECE** in una conferma diretta al mittente.
- Il mittente dimezza la finestra come se si fosse perso un segmento, e accende **CWR** nel segmento successivo per dire che l'ha fatto.
- Il punto: il segnale può arrivare **prima** della perdita, quindi il costo degli scenari precedenti non si paga mai. Funziona solo se i due estremi e i router in mezzo sono tutti d'accordo a usarlo.

Il controllo di congestione di TCP

TCP usa soprattutto il controllo **end-to-end**: ogni mittente adatta la propria velocità alla congestione che percepisce. I problemi da risolvere sono tre:

1. **Regolazione del ritmo**: come fa il mittente a limitare la velocità a cui immette traffico nella connessione?
2. **Rilevazione della congestione**: come fa ad accorgersi che c'è congestione tra sé e la destinazione?
3. **Aggiustamento del ritmo**: con quale algoritmo cambia la velocità in funzione della congestione percepita?

E il vincolo dietro a tutte e tre: i mittenti **non sono coordinati**. Ognuno agisce su informazioni locali, in modo asincrono.

1. Regolazione del ritmo: la congestion window

Nel controllo di flusso il ritmo è regolato dallo spazio del ricevente (LastByteSent − LastByteAcked ≤ rwnd). Nel controllo di congestione il mittente tiene anche una **finestra di congestione** (*congestion window*, cwnd).

Congestion window

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min\{\text{cwnd},\ \text{rwnd}\}$$

Il ritmo di invio viene regolato **aumentando o diminuendo cwnd**: più grande è la finestra, più byte si possono avere in volo, più veloce è la trasmissione.

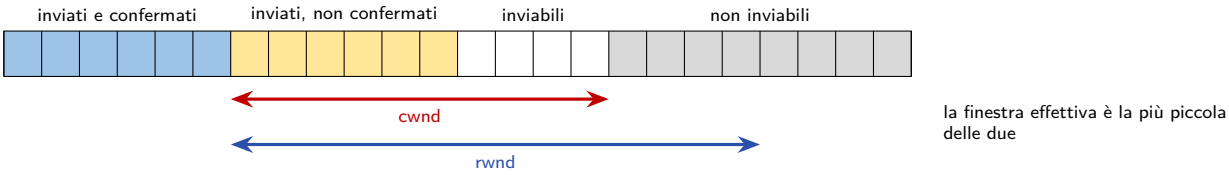


Figura 20.9 – Congestion window e receive window: comanda la più restrittiva.

cwnd non sta in nessun header: il ricevente non la conosce mai. Approssimativamente, all'inizio di ogni andata e ritorno il mittente può immettere cwnd byte, e alla fine sono stati confermati; la velocità di invio è quindi circa

$$\text{velocità} \approx \frac{\text{cwnd}}{\text{RTT}}\ \text{byte/s}$$

e cambiare la finestra è l'unico modo che il mittente ha per cambiarla.

2. Rilevazione della congestione: i loss event

Loss event

Il mittente TCP percepisce la congestione attraverso gli **eventi di perdita** (*loss event*):
- la scadenza di un **timeout**;
- la ricezione di **3 ACK duplicati**.

Entrambi indicano che un pacchetto precedente non è arrivato, probabilmente per il traboccare di un buffer in rete.

- Un evento di perdita sono in tutto quattro conferme per lo stesso byte (l'originale e tre duplicati), oppure un timeout. Entrambi significano un datagramma scartato, e un datagramma scartato significa quasi sempre un buffer pieno in un router: il mittente lo legge come un'affermazione sul **percorso**, non sul cavo o sul ricevente.
- Se **non** ci sono eventi di perdita, cioè gli ACK arrivano come previsto, TCP assume che la rete non sia congestionata e **aumenta** cwnd. Gli ACK che arrivano fanno da "orologio": più arrivano in fretta, più in fretta cresce la finestra (TCP è *self-clocking*). Un percorso che restituisce le conferme lentamente apre quindi lentamente la finestra, senza che nessuno lo decida; e due connessioni con andate e ritorni diversi attraverso lo stesso collo di bottiglia non crescono alla stessa velocità.
- Se ci sono eventi di perdita, cwnd va **diminuita**.
- Si noti che cosa **non** si usa: nessuna misura del ritardo, e nessun messaggio da alcun router. Solo i due eventi.

3. Aggiustamento del ritmo: l'algoritmo di Jacobson

L'aggiustamento avviene tramite il **sondaggio della banda** (*bandwidth probing*): non c'è una velocità giusta da calcolare. Il mittente aumenta finché fa male, arretra, e ricomincia ad aumentare: il ritmo cresce finché gli ACK arrivano correttamente (si "tasta" la rete), e quando si rileva congestione (eventi di perdita) viene ridotto. Il processo si ripete all'infinito.

TCP usa l'algoritmo di controllo di congestione di **Jacobson** (1988), con tre fasi:

1. **Slow start**: si parte da 1 MSS e la velocità cresce **esponenzialmente**.
2. **Additive increase** (o *congestion avoidance*): la velocità cresce **linearmente**.
3. **Fast recovery** (raccomandata): dopo 3 ACK duplicati si **dimezza** la velocità invece di ripartire da capo, e si prosegue con l'incremento lineare.

Le prime due sono obbligatorie; la terza è raccomandata, ed è ciò che separa le due versioni di TCP che tutti citano.

LE FASI DELL'ALGORITMO

Slow start

Slow start

All'inizio della connessione cwnd = 1 MSS. Per ogni ACK ricevuto cwnd aumenta di 1 MSS: così la finestra **raddoppia a ogni RTT** (1, 2, 4, 8, …). Nonostante il nome, la crescita è esponenziale: "lento" è solo il punto di partenza.

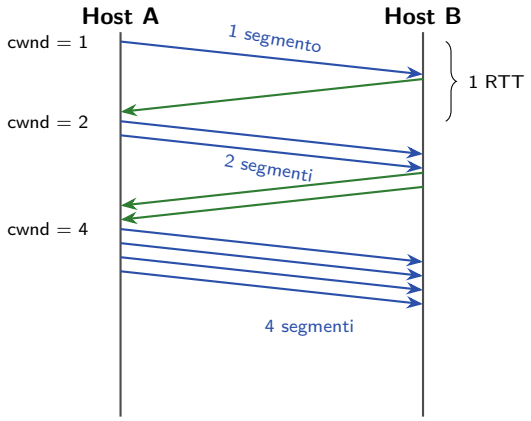


Figura 20.10 – Slow start: ogni ACK fa crescere cwnd di 1 MSS, quindi la finestra raddoppia a ogni RTT.

Con una MSS di 500 byte e un percorso da 200 ms, 1 MSS sono circa 20 kb/s: troppo poco, e il mittente non ha idea di quanto. La crescita è lineare nelle conferme ed esponenziale nel tempo: ciò che è lento è il punto di partenza, non la crescita. La crescita esponenziale termina in esattamente tre modi:

- si verifica un **timeout**: si pone ssthresh = cwnd/2 (la *slow start threshold*, soglia di slow start), cwnd = 1 MSS, e si ricomincia lo slow start;
- cwnd raggiunge **ssthresh**: si passa in **congestion avoidance**, perché continuare a raddoppiare sarebbe imprudente;
- arrivano **3 ACK duplicati**: si esegue la ritrasmissione veloce e si entra in **fast recovery**.

ssthresh è la memoria di dove ha fatto male l'ultima volta. Parte grande, 64 KB, perché all'inizio non c'è alcuna memoria; ogni evento di perdita la riscrive, sempre come metà della finestra in uso quando la perdita è stata rilevata.

Congestion avoidance

Congestion avoidance (additive increase)

Invece di raddoppiare, cwnd cresce di **1 MSS per ogni RTT** (in pratica, di MSS $\cdot \frac{\text{MSS}}{\text{cwnd}}$ per ogni ACK ricevuto). La crescita è **lineare**.

All'ingresso cwnd è circa la metà di quanto era al momento della perdita: il guaio è vicino, e la prudenza è voluta. Verifica: con cwnd = 10 MSS arrivano dieci conferme in un'andata e ritorno, e ognuna aggiunge un decimo: un MSS in tutto. Più grande è la finestra, più piccolo è ogni passo, ed è questo che mantiene la crescita lineare nel tempo invece che esponenziale.

- Se scade un **timeout**: come nello slow start, ssthresh = cwnd/2, cwnd = 1 MSS, e si torna in slow start.
- Se arrivano **3 ACK duplicati**: la rete riesce ancora a consegnare qualche segmento (gli ACK arrivano), quindi la reazione è meno drastica: ssthresh = cwnd/2, cwnd = ssthresh + 3 MSS (i tre segmenti che hanno generato i duplicati sono usciti dalla rete), e si entra in **fast recovery**.

Due perdite, due reazioni

Entrambi sono eventi di perdita. Perché un timeout riporta cwnd a 1 MSS, mentre tre conferme duplicate la dimezzano soltanto? Perché tre duplicati sono la **prova che la rete sta ancora consegnando**: i segmenti dopo quello mancante sono passati, e qualcosa è tornato a dirlo. Un timeout è silenzio, e il silenzio può significare qualsiasi cosa. Per questo la reazione al timeout è quella drastica, e quella ai tre duplicati tiene metà della finestra e prosegue da lì.

Fast recovery

Fast recovery

In **fast recovery** cwnd aumenta di 1 MSS per ogni ulteriore ACK duplicato ricevuto per il segmento mancante. Quando arriva l'ACK del segmento perso, si pone cwnd = ssthresh e si torna in **congestion avoidance**. Se invece scade un timeout, si abbandona tutto: si torna in slow start con cwnd = 1 MSS.

Il +3 non è decorativo: tre duplicati significano che tre segmenti hanno lasciato la rete e sono arrivati. Ogni duplicato successivo aggiunge un altro MSS per la stessa ragione: il tubo si sta svuotando mentre il mittente aspetta.

Tahoe e Reno

La prima versione di TCP con controllo di congestione, **TCP Tahoe**, non aveva la fast recovery: dopo *qualsiasi* evento di perdita ripartiva da cwnd = 1 MSS. **TCP Reno** ha introdotto la fast recovery, che dopo 3 ACK duplicati dimezza la finestra invece di azzerarla.

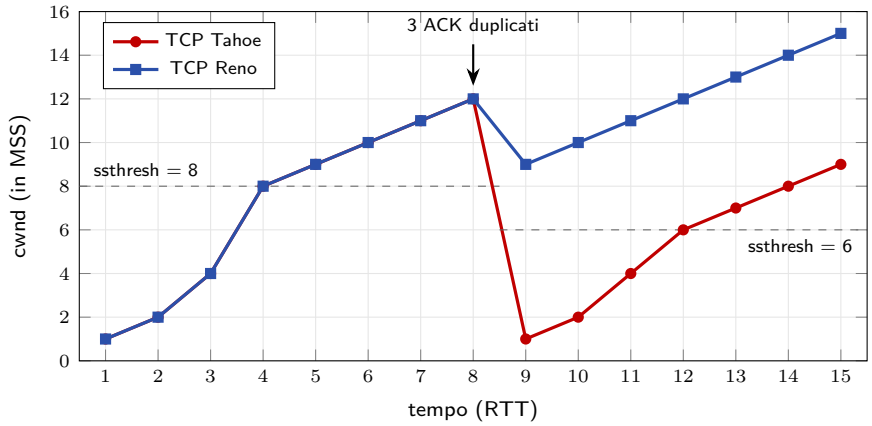


Figura 20.11 – Evoluzione di cwnd. Slow start fino a ssthresh = 8, poi crescita lineare. All'RTT 8 (cwnd = 12) arrivano 3 ACK duplicati: ssthresh diventa 6. Tahoe riparte da 1 MSS; Reno riparte da 6 + 3 = 9 MSS e prosegue linearmente.

LA MACCHINA A STATI DEL CONTROLLO DI CONGESTIONE

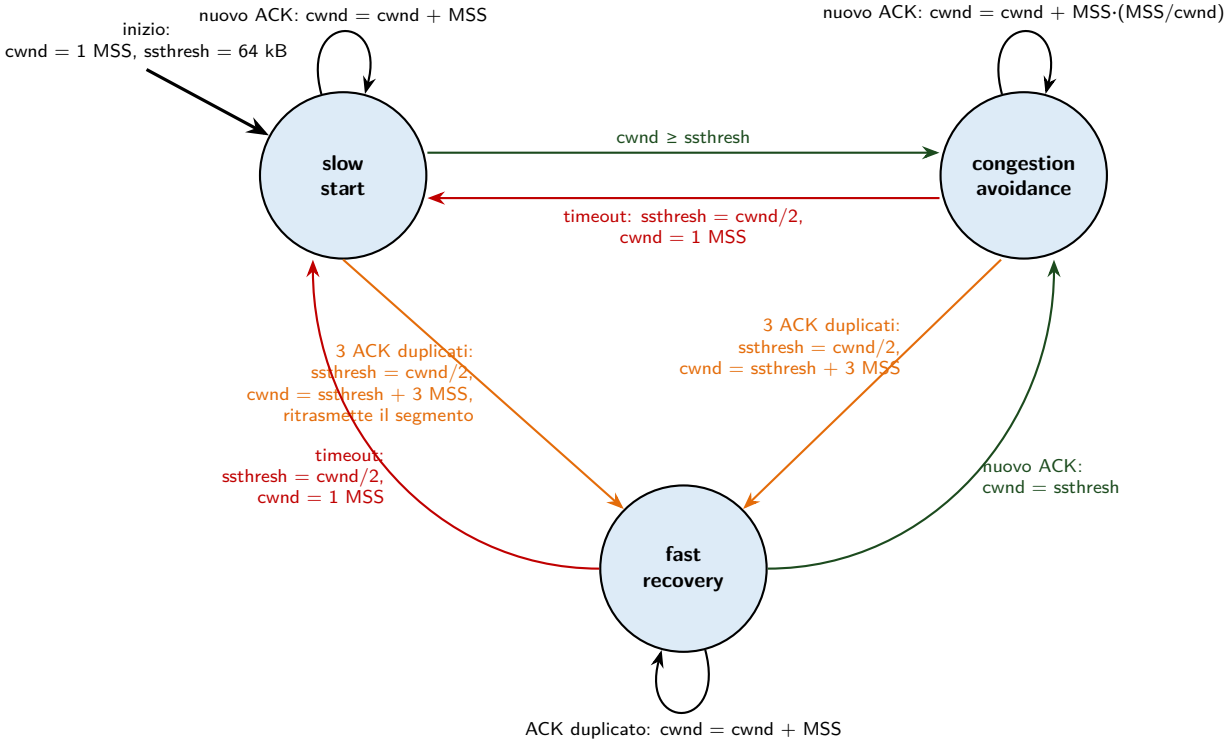


Figura 20.12 – Macchina a stati semplificata del controllo di congestione di TCP Reno.

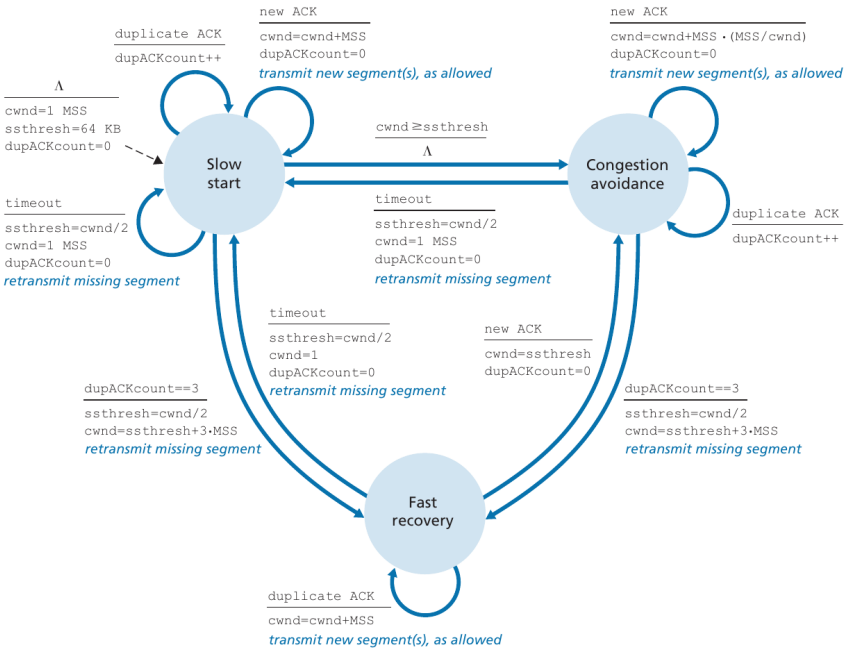


Figura 20.13 – La macchina a stati completa (Kurose), come mostrata a lezione.

Il quadro completo delle transizioni

Stato	Evento	Azione	Nuovo stato
Slow start	nuovo ACK	cwnd += MSS	slow start (o CA se cwnd ≥ ssthresh)
Slow start	3 ACK duplicati	ssthresh = cwnd/2; cwnd = ssthresh + 3 MSS; ritrasmissione	fast recovery
Slow start	timeout	ssthresh = cwnd/2; cwnd = 1 MSS; ritrasmissione	slow start
Congestion avoid.	nuovo ACK	cwnd += MSS·MSS/cwnd	congestion avoidance
Congestion avoid.	3 ACK duplicati	ssthresh = cwnd/2; cwnd = ssthresh + 3 MSS; ritrasmissione	fast recovery
Congestion avoid.	timeout	ssthresh = cwnd/2; cwnd = 1 MSS; ritrasmissione	slow start
Fast recovery	ACK duplicato	cwnd += MSS	fast recovery
Fast recovery	nuovo ACK	cwnd = ssthresh	congestion avoidance
Fast recovery	timeout	ssthresh = cwnd/2; cwnd = 1 MSS	slow start

Tabella 20.1 – Tutte le transizioni del controllo di congestione di TCP Reno.

AIMD e il dente di sega

Ignorando lo slow start iniziale e supponendo le perdite segnalate da 3 ACK duplicati, TCP aumenta cwnd di 1 MSS per RTT e la dimezza a ogni perdita: è l'approccio **AIMD** (*Additive Increase, Multiplicative Decrease*). Il grafico di cwnd nel tempo ha la tipica forma a **dente di sega**. Se W è la finestra a cui avviene la perdita, il throughput medio di una connessione è circa $\frac{0{,}75 \cdot W}{\text{RTT}}$.

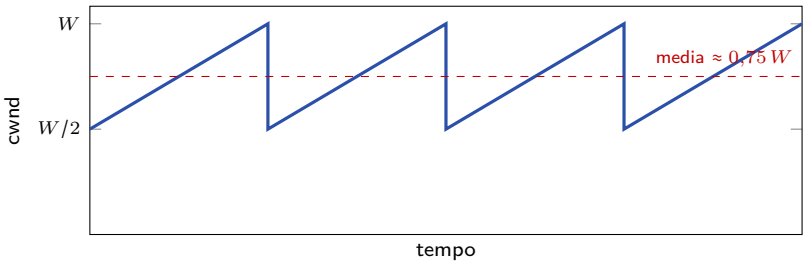


Figura 20.14 – Il dente di sega di AIMD: crescita lineare, dimezzamento a ogni perdita.

Che cosa esegue davvero la nostra macchina

- Reno classico è il **modello**, non il codice: è ciò che vale la pena capire, e non è ciò che esegue il kernel.
- **CUBIC**: stesso slow start, stessa fast recovery, congestion avoidance diversa: la finestra segue una funzione cubica del tempo trascorso dall'ultima perdita. Risale in fretta fino alla finestra che aveva prima della perdita, poi sonda con cautela attorno a essa. È il default di Linux.
- Basati sul ritardo: **Vegas** e **BBR** osservano l'andata e ritorno che cresce e rallentano **prima** che si perda qualcosa. L'idea: tenere il tubo appena pieno, ma non di più; un tubo pieno è lavoro utile, una coda piena è solo ritardo.
- Per la cattura di fine capitolo: il dente di sega che si troverà probabilmente non sarà quello di Reno, e ora si sa perché.

Equità

Equità

Con K connessioni attraverso un collo di bottiglia di rate R, il meccanismo è **equo** se ciascuna finisce con circa R/K.

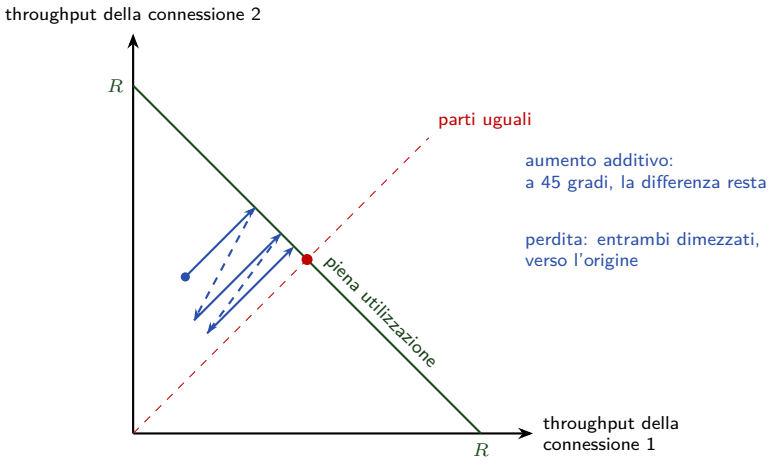


Figura 20.15 – Due connessioni con la stessa MSS e la stessa andata e ritorno. Ogni ciclo di aumento e dimezzamento avvicina la coppia alla bisettrice, e la coppia converge sulla parte uguale da qualunque punto parta.

- Finché non si perde niente, entrambe aggiungono un MSS per andata e ritorno. Aggiungere la stessa quantità a entrambe non cambia la differenza fra loro: la coppia si muove lungo una retta a 45 gradi.
- Una perdita dimezza entrambe, e dimezzare due numeri diversi li avvicina: in proporzione, il più grande perde di più.
- La ragione è che la diminuzione è moltiplicativa e l'aumento no: l'equità è una conseguenza di quell'asimmetria.

E perché l'equità si rompe facilmente

- L'argomento assumeva andate e ritorni uguali. Un percorso più corto apre la finestra più in fretta e si prende più collegamento, gratis.
- UDP non fa niente di tutto questo: non ha finestra, né eventi di perdita, né ragioni per rallentare. Continua a inviare. La chat su datagrammi della parte sulla programmazione ritrasmette ciò che si è perso e non riduce mai la velocità: è esattamente il flusso iniquo di questa figura.
- Nove applicazioni con una connessione ciascuna e una decima con undici: la decima si prende più di metà del collegamento (11/20), tutto legalmente.

L'equità di TCP è una **convenzione fra mittenti**, non una regola che la rete impone.

CHE COSA C'È SOTTO LA NOSTRA WRITE

`write` non trasmette: lo avevamo detto quando sono comparsi i due buffer, lasciando aperto il resto. Ecco il resto.

- Copia byte nel buffer di invio e ritorna. TCP li toglie più tardi, secondo il proprio calendario.
- Quanti ne possono partire adesso: $\min\{\text{cwnd}, \text{rwnd}\}$, meno ciò che è già in volo. Il buffer si svuota quindi alla velocità che il peer e il percorso consentono, e l'applicazione non vede mai nessuno dei due numeri.
- Una `write` bloccante ritorna quando c'è posto, quindi **si blocca quando il buffer è pieno**, cioè quando entrambi i limiti sono stati raggiunti.
- Una scrittura corta è lo stesso evento a metà: il buffer ha preso parte di ciò che si offriva e si è riempito. Ecco perché si cicla. Sui messaggi brevi nessuna delle due cose capita mai, ed è per questo che le si scopre tardi.

Il client che ha smesso di leggere

Un client della chat smette di chiamare `read`: non è morto, è solo occupato. Che cosa arriva al server? Il suo buffer di ricezione si riempie, quindi annuncia rwnd = 0. Il buffer di invio del server si riempie dietro a lui, e la `write` del worker **si blocca**, tenendo il lock che teneva.

- Niente è rotto e niente è perso: è il controllo di flusso che funziona esattamente come progettato, visto dal lato sbagliato.
- Sul filo si vede: una finestra che si restringe fino a zero, poi le sonde da un byte, poi un aggiornamento della finestra quando il client torna a leggere.
- Il bug non è mai stato il client lento: era bloccarsi tenendo il lock della lista, che ha trasformato un client silenzioso in una chat congelata.

```c
ssize_t n = write(fd, out[i].buf, out[i].len);

if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
    FD_SET(fd, &wall);          /* buffer di invio pieno: min(cwnd,rwnd) e' esaurito */
    return;                     /* si torna quando si svuota; nessuno aspetta */
}
if (n < 0 && errno == EINTR) return;    /* un segnale, non uno stallo */
if (n < 0) { drop(i); return; }

out[i].len -= n;                /* anche una scrittura corta arriva qui, ed e' normale */
memmove(out[i].buf, out[i].buf + n, out[i].len);
```

EAGAIN, e ciò che non dice

`EAGAIN` dice che il buffer di invio è pieno **ora**, e niente sul perché. rwnd viaggia in ogni segmento e la si può leggere in una cattura; cwnd non lascia mai il kernel del mittente. Due cause, un sintomo, e solo una delle due visibile da fuori.

Analizzare il traffico TCP con Wireshark

La connessione con `nc` sull'interfaccia di loopback si trova nel capitolo sulla gestione della connessione; qui si guarda un trasferimento vero, abbastanza lungo da far vedere le finestre al lavoro.

Esercizio 1 — Uno scaricamento reale da dieci megabyte

Si avvia **prima** la cattura, sull'interfaccia davvero in uso, e poi si scarica un file di prova:

```
$ wget http://speedtest.tele2.net/10MB.zip
```

Un lungo trasferimento in una sola direzione è la cosa più facile da leggere: un flusso solo, nessun intreccio, e una dimensione nota. Clic destro su un segmento, *Follow* → *TCP Stream*: Wireshark scrive il filtro `tcp.stream eq N` e tutto il resto sparisce.

(a) Quali quattro numeri sta riassumendo quel filtro, e quale dei quattro non ha scelto nessuno?

(b) `wget` ha preso la porta del server dall'URL. Si sarebbe potuta scegliere anche la propria porta locale?

(c) In *Statistics* → *TCP Stream Graphs* → *Time/Sequence (tcptrace)*, nella direzione che porta i dati: dove finisce la curvatura iniziale? In circa quante andate e ritorno, e che RTT dà al percorso?

(d) Guardare il fattore di window scale nei due SYN: qual è la finestra più grande che ciascun lato potrebbe annunciare?

(e) Trovare una ritrasmissione. È stata preceduta da tre conferme duplicate o da un silenzio? Quale delle due reazioni dovrebbe seguire?

(f) Catturare di nuovo con `wget --limit-rate=200k`. Quale delle due finestre si esaurisce questa volta, e quale mai?

Svolgimento

(a) Indirizzo e porta del client, indirizzo e porta del server. La porta del client è **effimera**: l'ha scelta il kernel al momento della `connect`, e nessuno l'ha decisa.

(b) Sì, ma non con `wget`: un programma può chiamare `bind` sul socket del client prima di `connect` (con `nc` c'è l'opzione `-p`). Di solito non conviene: una porta fissa entra in conflitto con le connessioni precedenti ancora in TIME WAIT.

(c) Nel grafico il tempo è in orizzontale e il numero di sequenza in verticale, quindi la pendenza è il throughput. La linea bassa sono i dati inviati, quella alta il massimo consentito dal ricevente (conferma più finestra); la distanza verticale fra le due è ciò che è in volo. I primi istanti curvano verso l'alto, poi la curva si raddrizza: la curvatura è la finestra che si apre (slow start), il tratto dritto è il regime. Contando i «gradini» della curvatura (uno per andata e ritorno) e dividendo il tempo che occupano si ottiene una stima dell'RTT, da confrontare con il tempo fra SYN e SYNACK.

(d) Il campo è di 16 bit, quindi al massimo $65\,535 \cdot 2^s$ byte, dove s è il fattore annunciato nel SYN di quel lato (per esempio con $s = 7$, circa 8 MB).

(e) Il filtro `tcp.analysis.duplicate_ack` mostra se prima della ritrasmissione ci sono stati tre duplicati: allora è una *fast retransmission* e la finestra si dimezza; se invece c'è stato un silenzio è un timeout, e la finestra riparte da 1 MSS.

(f) Con il limite di velocità è `wget` a leggere piano: il buffer di ricezione si riempie e compaiono `tcp.analysis.zero_window` e `window_update`. Si esaurisce **rwnd**; cwnd non viene mai raggiunta, perché il percorso potrebbe portare molto di più di 200 KB/s.

La finestra che si vede, e quella che no

- `tcp.analysis.bytes_in_flight`: byte non confermati, pacchetto per pacchetto;
- `tcp.analysis.duplicate_ack`: il sintomo di una perdita;
- `tcp.analysis.window_update`: il ricevente che si riapre;
- `tcp.analysis.zero_window`: il ricevente che non ha più posto.

Si combinano con `&&` nella barra dei filtri. `bytes_in_flight` è un calcolo di Wireshark su ciò che ha visto: coincide con cwnd solo quando il mittente ha sempre dati e rwnd non è la finestra più piccola; altrimenti è un limite inferiore. Nella cattura: lo slow start è la curva che piega verso l'alto, con pochi segmenti per andata e ritorno; la congestion avoidance una retta, con un ritmo costante di segmenti; la fast recovery conferme duplicate, una ritrasmissione, un gradino.

RIEPILOGO

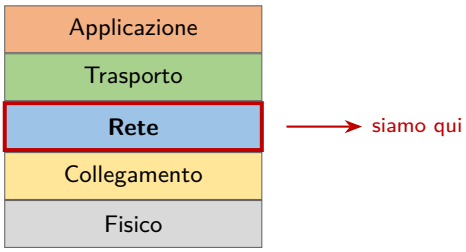
In breve

- **Flusso**: proteggere il ricevente. rwnd = RcvBuffer − (LastByteRcvd − LastByteRead); il mittente mantiene LastByteSent − LastByteAcked ≤ rwnd.
- **Congestione**: proteggere la rete. Buffer infiniti ⇒ ritardi infiniti; buffer finiti ⇒ perdite e ritrasmissioni che sprecano capacità.
- TCP usa il controllo **end-to-end**: LastByteSent − LastByteAcked ≤ min{cwnd, rwnd}.
- **Loss event**: timeout o 3 ACK duplicati.
- **Slow start** (cwnd raddoppia per RTT fino a ssthresh), **congestion avoidance** (+1 MSS per RTT), **fast recovery** (dopo 3 duplicati: ssthresh = cwnd/2, cwnd = ssthresh + 3); timeout ⇒ cwnd = 1 MSS.
- La congestione costa ritardo prima che perdite, e un pacchetto scartato tardi spreca ogni collegamento che lo ha già portato. ECN segnala la congestione prima della perdita.
- AIMD rende TCP equo fra connessioni con lo stesso RTT; UDP e le connessioni parallele rompono l'equità, che è una convenzione.
- La nostra `write` si blocca, o restituisce `EAGAIN`, quando min{cwnd, rwnd} non ha più niente da dare. Prossimo argomento: i router, delle cui code abbiamo parlato fin qui.

Parte V

Il livello di rete

Il livello di rete e i router



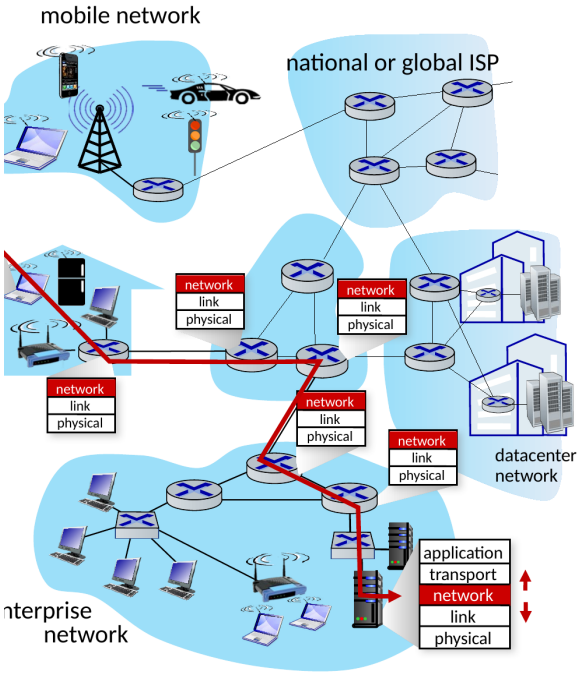
Dal trasporto alla rete

Per tre capitoli i mittenti hanno reagito a qualcosa che non potevano vedere: un timeout, tre conferme duplicate, una finestra che si chiude, tutto in risposta a una perdita. E la perdita stessa era una curva, il ritardo di accodamento che sale quando l'intensità di traffico si avvicina a uno, e poi pacchetti scartati. Nessuno ha mai aperto il nodo dove questo succede: ora lo apriamo. Sotto il trasporto non ci sono più processi; questo livello sposta datagrammi da host a host, e un suo pezzo gira in ogni router del percorso. Le domande sono due: come fa un datagramma che arriva da un collegamento a uscire da quello giusto, e chi decide quale pacchetto parte per primo.

Il compito del livello di rete è permettere a host remoti di comunicare:

- **sull'host mittente**: prendere i segmenti dal livello di trasporto, incapsulare ciascuno in un **datagramma** e inviare i datagrammi nella rete;
- **sull'host ricevente**: ricevere i datagrammi dalla rete, estrarne i segmenti di trasporto e consegnarli al livello di trasporto.

Fra i due non vale niente di ciò che abbiamo studiato finora: nessuna connessione, nessuna conferma, nessuna finestra. A questo livello lavorano alcuni dei protocolli più importanti delle reti (IP, DHCP, NAT, ICMP, i protocolli di routing) e i dispositivi che fanno di una rete una rete: i **router**. Il trasporto gira solo ai bordi; questo livello gira **ovunque**, nei due host e in ogni router in mezzo, ed è per questo l'unico punto in cui tutta Internet deve essere d'accordo.



All'interno della rete ci sono nodi, i **router**, che inoltrano i datagrammi ai nodi adiacenti fino all'host di destinazione.

- Nelle figure i router hanno una **pila troncata** (rete, collegamento, fisico): non eseguono applicazioni né protocolli di trasporto.
- L'atto di un datagramma che viene reindirizzato da un router si chiama **hop** (salto).

Figura 21.1 – Solo gli host hanno la pila completa; i router si fermano al livello di rete.

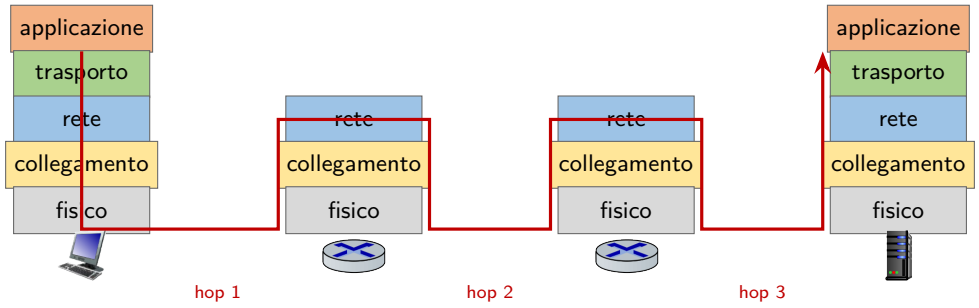


Figura 21.2 – Il datagramma sale fino al livello di rete in ogni router, che decide dove inoltrarlo, e riscende.

Che servizi offre il livello di rete

Il livello di rete è responsabile della **consegna host-a-host**. In teoria, insieme alla consegna potrebbe offrire servizi come:

- **consegna garantita**: un pacchetto inviato arriverà prima o poi a destinazione;
- **consegna garantita con ritardo limitato**: arriverà entro un certo tempo (per esempio 100 ms);
- **consegna in ordine**: i pacchetti arrivano nell'ordine di invio;
- **banda minima garantita**: se il mittente non supera una certa velocità (per esempio 1 Mb/s), tutti i pacchetti arrivano;
- **sicurezza**: cifratura e decifratura di tutti i datagrammi.

In realtà, **nessuno** di questi servizi è offerto in generale dalle reti.

Servizio best effort

Il livello di rete di Internet offre un solo servizio, detto **best effort** ("del proprio meglio"): la rete **fa del suo meglio** per consegnare un pacchetto dalla sorgente alla destinazione, ma non garantisce né che arrivi, né che arrivi in ordine, né ritardi massimi, né banda minima.

Suona come un eufemismo per «nessun servizio»: una rete che non consegnasse niente soddisferebbe la definizione. Altre architetture facevano di più: **ATM** garantiva consegna in ordine, ritardo limitato e banda minima, e Internet non le ha seguite. Nonostante la sua semplicità, il modello best effort, insieme a una buona disponibilità di banda e ad applicazioni che si adattano, si è dimostrato **adeguato**: Netflix, le videochiamate, le conferenze in tempo reale funzionano tutte così. E tutto ciò che di affidabile abbiamo scritto finora è costruito sopra di esso, dai due estremi, partendo da niente.

I ROUTER NELLA RETE

I router sono nodi speciali con **più collegamenti in ingresso e in uscita**. Offrono due funzioni di rete fondamentali.

Forwarding e routing

- **Forwarding** (inoltro): quando un pacchetto arriva su un collegamento di ingresso, il router lo sposta sul **collegamento di uscita** appropriato. È un'azione **locale** al router.
- **Routing** (instradamento): decidere il **percorso** che i pacchetti devono seguire dalla sorgente alla destinazione. È un processo che riguarda **tutta la rete**; gli algoritmi che calcolano i percorsi sono gli **algoritmi di routing**.

Durante il forwarding un router può anche:

1. **bloccare** un pacchetto (se proviene da un host noto come malevolo o è diretto a un host vietato);
2. **scartare** un pacchetto scaduto;
3. **duplicare** un pacchetto e inviarlo su più collegamenti;
4. **modificare** un pacchetto (per esempio per segnalare congestione).

L'analogia del viaggio in auto

Il **routing** è pianificare il viaggio da Napoli a Milano, scegliendo le autostrade da percorrere. Il **forwarding** è quello che si fa a ogni svincolo: guardare il cartello e imboccare l'uscita giusta. Il primo richiede una visione d'insieme e si fa una volta; il secondo è rapido, locale e si ripete a ogni incrocio.

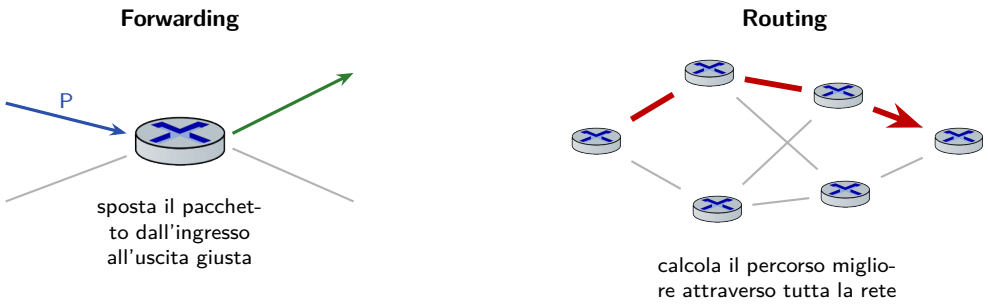
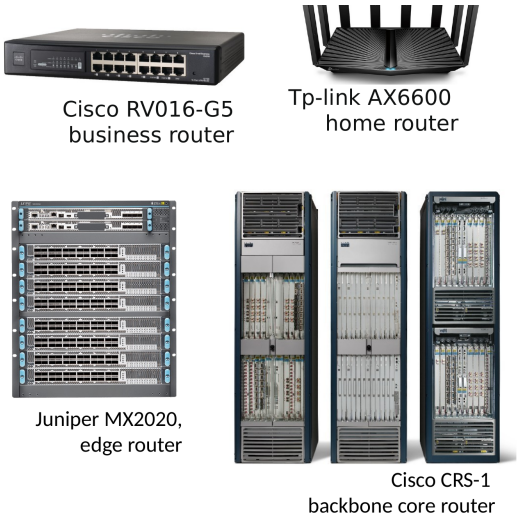


Figura 21.3 – Forwarding (locale) e routing (globale).

Tipi di router



I router possono essere molto diversi a seconda di uso e tecnologia: per casa o per ufficio, con o senza fili. Una distinzione importante è quella tra:

- **edge router** (router di bordo): distribuiscono pacchetti **tra reti diverse**, per esempio collegando una rete con l'ISP. È il tipo più comune;
- **core router** (router di nucleo): distribuiscono pacchetti **all'interno di una stessa rete**. Sono usati anche sulla dorsale (*backbone*) di Internet, dove devono smaltire trasferimenti di dati enormi.

Figura 21.4 – Router per uso domestico e aziendale (in alto), un edge router e un core router da dorsale (in basso).

Data plane e control plane

I due piani di un router

- Il **data plane** (piano dei dati, o di forwarding) comprende l'azione locale di trasferire un pacchetto da un'interfaccia di ingresso a quella di uscita appropriata. È un'operazione **veloce** (pochi nanosecondi), di solito realizzata in **hardware**.
- Il **control plane** (piano di controllo) comprende il processo, esteso a tutta la rete, che determina i percorsi da un estremo all'altro. È più **lento** (secondi), e spesso realizzato in **software**.

Due funzioni su scale di tempo distanti un miliardo di volte: è per questo che hanno nomi separati. Fra le due sta una sola struttura dati, la **forwarding table** (tabella di inoltro): il control plane la scrive, il data plane la legge una volta per pacchetto. Associa gli indirizzi di destinazione ai collegamenti di uscita: il router inoltra un pacchetto esaminando uno o più campi del suo header, e il valore trovato nella tabella indica l'interfaccia su cui inoltrarlo. Conseguenza: un router inoltra **senza conoscere il percorso**; sa solo quale dei suoi collegamenti viene dopo.

La tabella non contiene i singoli indirizzi

Una forwarding table **non** elenca gli indirizzi dei singoli host (sarebbero miliardi): associa alle porte di uscita **gruppi di indirizzi**, cioè intervalli individuati da un prefisso (le *subnet*).

Centralizzato o decentralizzato

Poiché prima di arrivare a destinazione si attraversano molti router, il contenuto di una forwarding table va determinato raccogliendo informazioni da router diversi. Lo si fa con una o più **tabelle di routing** (control plane), da cui si ricava la forwarding table (data plane). Ci sono due modi:

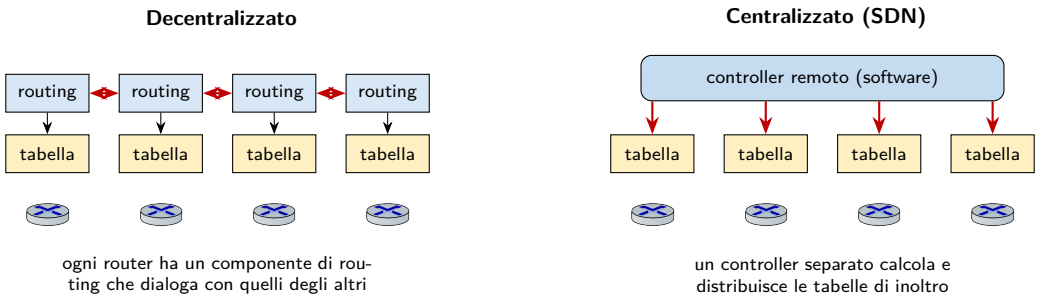


Figura 21.5 – Control plane decentralizzato e centralizzato.

- **Decentralizzato** (distribuito): ogni router ha un componente di routing che comunica con i componenti di routing degli altri router. È stato a lungo l'approccio dominante.
- **Centralizzato**: un **controller remoto**, fisicamente separato dai router, calcola le tabelle e le distribuisce. Può trovarsi in un data center remoto, affidabile e ridondato, gestito dall'ISP o da terzi.

Software-Defined Networking

L'approccio centralizzato è alla base del **Software-Defined Networking** (SDN): la rete è "definita dal software" perché il controller che calcola le tabelle e dialoga con i router è un software centralizzato, talvolta anche open source. Si noti che cosa non cambia: il data plane delle due figure è identico, cambia solo l'autore della tabella.

LE COMPONENTI DI UN ROUTER

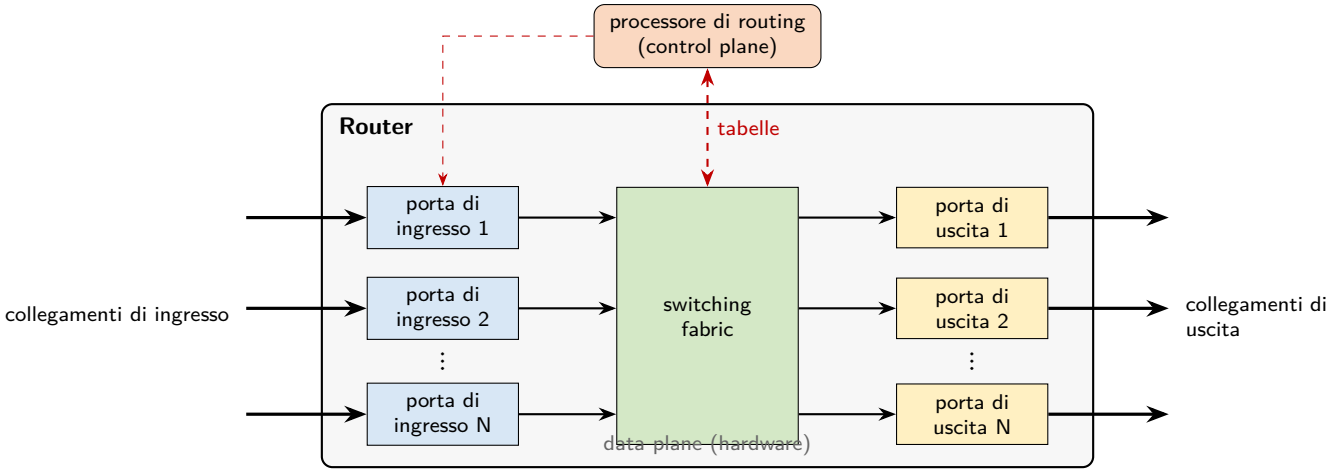


Figura 21.6 – Architettura di un router.

- **Porte di ingresso** (da non confondere con le porte del trasporto, i numeri a 16 bit: queste sono interfacce fisiche) collegate ai collegamenti in ingresso. Il loro compito è:
  - terminare il collegamento fisico e fare il lavoro di livello di collegamento necessario a parlare con l'altro capo;
  - consultare la forwarding table (*lookup*) e preparare la switching fabric verso la porta di uscita scelta;
  - inoltrare al processore di routing i pacchetti di controllo (per esempio quelli con le informazioni di routing);
  - controllare versione, checksum e *time-to-live* del datagramma, riscrivere gli ultimi due, e aggiornare i contatori di gestione.

Il numero di porte va da decine a centinaia: il Juniper MX2020 arriva a 800 porte da 100 Gb/s in un solo chassis. È la scala a distinguere un router di casa da uno di dorsale, non la funzione.

- **Switching fabric** (matrice di commutazione): collega le porte di ingresso a quelle di uscita.
- **Porte di uscita**: conservano i pacchetti ricevuti dalla switching fabric e li trasmettono sul collegamento in uscita. Le porte possono essere bidirezionali (ingresso e uscita accoppiati).
- **Processore di routing**: esegue i protocolli di routing, mantiene informazioni sullo stato dei collegamenti e calcola e aggiorna la forwarding table.

Perché il lookup non può essere software

Un collegamento in ingresso da 100 Gb/s e un datagramma da 64 byte: la porta ha **5,12 ns** prima che possa arrivare il successivo. Le schede di linea portano più porte insieme, e allora tutta la catena deve girare N volte più in fretta. Nessun software ci si avvicina: porte di ingresso, porte di uscita e fabric sono hardware, progettato dal costruttore o comprato come silicio. Il processore di routing sta all'altro capo della scala, su millisecondi e secondi, e può essere una normale CPU con normale codice. È la forma di tutto il livello: la parte veloce è stupida e ripetitiva, quella intelligente è lenta e rara.

Il forwarding e la longest prefix matching

La forwarding table associa **indirizzi IP** a **porte di uscita**, così che i pacchetti possano essere inoltrati sul collegamento giusto verso il nodo successivo (dove, eventualmente, verranno inoltrati di nuovo).

Un indirizzo IP è un numero di **4 byte** (32 bit), che di solito si scrive in notazione decimale ma che si può vedere in binario:

decimale puntata	`192.168.1.1`
binario	`11000000 10101000 00000001 00000001`

Quando un pacchetto arriva su una porta di ingresso si esegue un **lookup**. Ogni porta ha una **copia** della forwarding table, ricevuta dal processore di routing su un bus dedicato (per esempio PCI), per evitare il collo di bottiglia di interrogare il processore per ogni pacchetto. Più indirizzi possono essere associati alla stessa porta.

Una riga per indirizzo è fuori discussione

Che cosa c'è di sbagliato nell'elencare semplicemente gli indirizzi? Una riga per ogni destinazione possibile sono 2^{32} righe, più di quattro miliardi, in ogni router, da cercare in nanosecondi; e andrebbero riscritte ogni volta che un host compare o sparisce da qualche parte. Quindi una riga non nomina un indirizzo: nomina un **gruppo di indirizzi che condividono i primi bit**, una subnet.

Ogni riga fissa i primi bit e lascia liberi gli altri: la parte fissa nomina il gruppo, quella libera un host al suo interno. La prima riga della tabella seguente fissa 21 bit; gli 11 liberi danno $2^{11} = 2048$ indirizzi, da tutti zeri a tutti uni. Una riga è quindi un **intervallo**, e quattro righe sostituiscono quattro miliardi. L'ultima, *altrimenti*, è il **default**: ciò che non corrisponde a niente deve comunque uscire da qualche parte.

Prefisso degli indirizzi	Intervallo di indirizzi corrispondente	Interfaccia
`11001000 00010111 00010*** ********`	da `...00010000 00000000` a `...00010111 11111111`	0
`11001000 00010111 00011000 ********`	da `...00011000 00000000` a `...00011000 11111111`	1
`11001000 00010111 00011*** ********`	da `...00011000 00000000` a `...00011111 11111111`	2
altrimenti		3

Tabella 21.1 – Una forwarding table a 4 porte: ogni riga è un prefisso, cioè un intervallo di indirizzi (i primi 16 bit sono sempre `11001000 00010111`).

Righe che si sovrappongono

Le righe non sono necessariamente mutuamente esclusive. Per esempio l'indirizzo `11001000 00010111 00011000 10101010` corrisponde sia alla riga dell'interfaccia 1 sia a quella dell'interfaccia 2. A quale porta va?

Longest prefix matching

Se un indirizzo corrisponde a più righe della forwarding table, il router lo inoltra verso la porta associata alla riga con il **prefisso più lungo** (*longest prefix matching rule*).

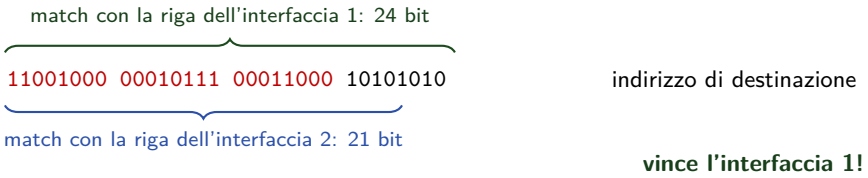


Figura 21.7 – Longest prefix matching: tra le righe che corrispondono si sceglie quella con il prefisso più lungo.

Esempio 1 — Dove va il pacchetto?

Con la tabella precedente:

- `11001000 00010111 00010110 10100001`: corrisponde solo al prefisso `...00010` ⇒ interfaccia **0**;
- `11001000 00010111 00011000 10101010`: corrisponde a `...00011000` (24 bit) e a `...00011` (21 bit) ⇒ interfaccia **1**;
- `11001000 00010111 00011001 00000001`: corrisponde solo a `...00011` ⇒ interfaccia **2**;
- `11001000 00010111 00100000 00000001`: nessun prefisso ⇒ interfaccia **3**.

Il confronto avviene solo sui bit del prefisso e si ferma dove il prefisso finisce; i bit che avanzano dicono quale host dentro il gruppo, e al router non servono. È il senso di tutta l'organizzazione: un router in un altro continente ha bisogno di conoscere il gruppo, mai la macchina.

Esempio 2 — Ritagliare un blocco da un blocco

Due righe di una forwarding table:

Prefisso	Lunghezza	Porta
`11001000 00010111 00011000 ********`	24 bit	1 (verso l'Europa)
`11001000 00010111 00011000 01******`	26 bit	2 (verso le Americhe)

Arriva un pacchetto per `11001000 00010111 00011000 01001101`: corrisponde a entrambe, e la corrispondenza a 26 bit è la più lunga, quindi vince la porta 2. Le due righe descrivono lo stesso blocco da 24 bit; la seconda ne prende un quarto (due bit in più) e lo manda altrove. Senza la regola le due righe si contraddirebbero; con la regola dicono «tutto questo blocco di là, **tranne** questo pezzo». È così che si sposta un cliente senza rinumerare nessuno.

Il lookup in nanosecondi

Come problema è semplice: trovare la riga corrispondente più lunga. A velocità di gigabit va risolto in nanosecondi, quindi non solo in hardware, ma con qualcosa di meglio di una scansione lineare. Il vincolo che plasma il progetto è il tempo di accesso alla memoria: DRAM integrata nel chip, SRAM più veloce come cache davanti. Oppure la **TCAM** (*ternary content-addressable memory*): si presentano i 32 bit alla memoria e restituisce la riga in tempo praticamente costante; i dispositivi grandi ne contengono dell'ordine di un milione.

Determinata la porta di uscita, il pacchetto entra nella switching fabric; se altre porte stanno usando la fabric, può dover attendere in coda.

Match-plus-action

L'operazione in due passi "cerca l'indirizzo di destinazione" (*match*) e "inoltra" (*action*) si chiama **match-plus-action** ed è eseguita in molti dispositivi di rete:

- **switch**: azione simile a quella dei router (ma su indirizzi di livello 2);
- **firewall**: l'azione è filtrare, cioè bloccare, certi pacchetti in arrivo;
- **NAT** (*Network Address Translator*): l'azione è riscrivere indirizzo e porta di certi pacchetti prima di inoltrarli.

Tre dispositivi che il corso incontra separatamente, a fare una sola cosa in tre costumi: vale la pena riconoscerli quando compaiono.

LA SWITCHING FABRIC

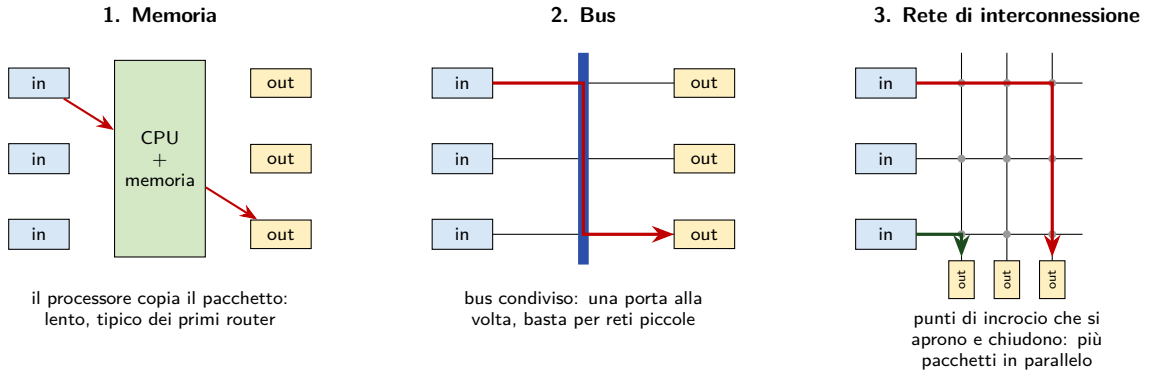


Figura 21.8 – I tre modi di realizzare la commutazione.

1. **Commutazione tramite memoria**: le porte sono trattate come dispositivi di I/O; un pacchetto in arrivo interrompe il processore, viene copiato in memoria, cercato nella tabella e copiato nel buffer della porta di uscita. Il pacchetto attraversa la memoria **due volte**: se la memoria regge B pacchetti al secondo, il router ne inoltra meno di $B/2$. E due pacchetti non possono essere inoltrati insieme nemmeno verso porte diverse, perché sul bus condiviso passa una lettura o una scrittura alla volta. Era il modo dei primi router, che erano normali computer; i router moderni a memoria condivisa fanno lookup e scrittura sulla scheda di linea, non in una CPU centrale.
2. **Commutazione tramite bus**: la porta di ingresso trasferisce il pacchetto direttamente alla porta di uscita su un bus condiviso, senza intervento del processore. Gli antepone un'etichetta interna che nomina la porta di uscita: tutte le porte vedono il pacchetto, solo quella nominata lo tiene, e toglie l'etichetta. Un pacchetto alla volta sul bus: la velocità di commutazione del router è quella del bus. Spesso basta (un noto router aziendale di questo tipo commuta su un bus da 32 Gb/s): semplice, economico, e con un tetto rigido.
3. **Commutazione tramite rete di interconnessione** (*crossbar*): $2N$ bus collegano N ingressi a N uscite, con un punto di incrocio a ogni intersezione che si apre e si chiude. Per portare un pacchetto da A a Y il controllore chiude l'incrocio AY, e solo il bus Y lo raccoglie; così A→Y e B→X avvengono **insieme**, cosa che le altre due non possono fare. È **non bloccante**: un pacchetto non viene mai fermato sulla strada verso una porta di uscita che nessun altro sta usando. Ma due pacchetti per la stessa porta si contendono ancora, e uno aspetta in ingresso.

LE CODE SULLE PORTE

Poiché la commutazione richiede tempo, porte di ingresso e di uscita hanno **code** (buffer) in cui i pacchetti aspettano, come le auto a un semaforo. L'entità delle code non è fissa: dipende dal carico di traffico, dalla velocità della switching fabric e da quella delle linee. In generale i pacchetti possono arrivare o partire più velocemente o più lentamente di quanto avvenga la commutazione, e accumularsi in un buffer di ingresso o di uscita, che può **traboccare**.

Esempio 3 — Dove si formano le code

Un router ha N porte di ingresso e N di uscita; tutte le linee hanno velocità R_{line} pacchetti al secondo e tutte le porte ricevono pacchetti contemporaneamente. Nel caso peggiore **tutti** i pacchetti vanno alla **stessa** porta di uscita. Detta R_{switch} la velocità della switching fabric:

- se $R_{switch} \approx N \cdot R_{line}$: la fabric smaltisce tutto in tempo, le code in **ingresso** sono trascurabili, ma quella in **uscita** cresce, perché arrivano N volte più pacchetti di quanti il collegamento ne trasmetta;
- se $R_{line} < R_{switch} < N \cdot R_{line}$: si formano code **su entrambe** le porte (l'ingresso aspetta la fabric, l'uscita aspetta il collegamento);
- se $R_{switch} \approx R_{line}$: la coda in uscita è trascurabile, ma quelle in **ingresso** sono consistenti.

Letti insieme, i tre casi dicono che nessuna scelta di fabric elimina l'accodamento: decide solo **dove** avviene.

Head-of-the-line blocking

La fabric è non bloccante. Perché allora un pacchetto si ferma? Perché una coda di ingresso è servita **in ordine**: il pacchetto in testa aspetta una porta di uscita occupata, e tutto ciò che sta dietro aspetta con lui, compreso un pacchetto la cui porta sarebbe libera. È il *blocco in testa alla coda* (HOL blocking): da solo fa crescere la coda di ingresso senza limite già quando il ritmo di arrivo raggiunge circa il **58%** della capacità della linea. È uno dei motivi per cui le code in ingresso vanno evitate.

Quando non c'è più posto

- Anche con una fabric N volte più veloce delle linee, N pacchetti possono raggiungere una porta di uscita nel tempo che serve a inviarne uno: le code in uscita si formano comunque.
- Quando la memoria è finita le scelte sono due: scartare il pacchetto che arriva (**drop-tail**), o fare posto togliendo pacchetti già in coda. Un pacchetto lascia la coda solo quando è stato trasmesso per intero: fino ad allora occupa il suo buffer.
- Può anche convenire scartare, o **marcare**, prima che il buffer sia pieno, come avvertimento al mittente: sono le politiche di **gestione attiva della coda** (AQM), di cui la più nota è **RED**. E marcare invece di scartare è proprio ciò a cui servivano i due bit ECN nell'header: è questa la macchina che li accende.

Quanto buffer basta

- La regola pratica per anni: buffer = RTT medio × capacità del collegamento. Un collegamento da 10 Gb/s con 250 ms vuole 2,5 Gbit di buffer.
- Quella cifra viene dall'analisi di pochi flussi TCP. Con N flussi numerosi e indipendenti il requisito scende a $\mathrm{RTT} \cdot C/\sqrt{N}$: nel nucleo, dove migliaia di flussi condividono un collegamento, è una riduzione enorme.
- E di più non è meglio: un buffer più grande assorbe raffiche più grandi e allunga il ritardo di accodamento di tutto ciò che contiene. Dieci volte il buffer per evitare perdite può significare dieci volte il ritardo da un capo all'altro, e un'andata e ritorno più lunga rende ogni mittente su quel percorso più lento a reagire.

Esempio 4 — La coda che non si svuota mai

Un router di casa impiega 20 ms a inviare un pacchetto, l'andata e ritorno è di 200 ms, e a $t = 0$ una raffica di 25 pacchetti arriva in coda; c'è un solo flusso, nessun altro traffico.

- A $t = 200$ ms arriva la prima conferma, proprio mentre parte il ventunesimo pacchetto, e libera un nuovo pacchetto nella coda.
- A $t = 220$ ms arriva la successiva, e un altro pacchetto entra in coda: uno dentro per ognuno fuori, finché dura il trasferimento.
- La coda si assesta a cinque pacchetti e ci resta. Il tubo è pieno, il throughput è esattamente giusto, e il ritardo è costante, grande, e causato soltanto dal buffer: $5 \times 20 = 100$ ms in più a ogni andata e ritorno.

Bufferbloat

Il **bufferbloat** è il ritardo permanente causato da buffer troppo grandi che restano pieni. Nessuno si comporta male: il mittente fa ciò per cui è stato progettato, e a essere sbagliato è il buffer.

LA SCHEDULAZIONE DEI PACCHETTI

È normale che più pacchetti, anche da porte di ingresso diverse, debbano uscire dalla stessa porta. L'accesso dei pacchetti in coda al collegamento di uscita va quindi **schedulato**. I tre approcci più noti sono:

- **FIFO** (*First-In-First-Out*, o FCFS, *First-Come-First-Served*): basato sull'ordine di arrivo;
- **a priorità** (*priority queuing*): basato sull'importanza dei pacchetti;
- **round-robin**: i pacchetti sono divisi in classi, servite a turno.

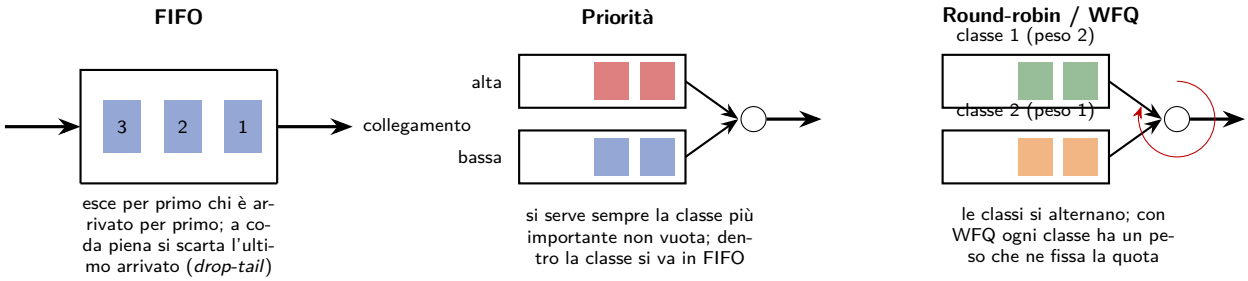


Figura 21.9 – Tre politiche di schedulazione della coda di uscita.

In ogni caso un pacchetto viene tolto solo quando è stato trasmesso per intero: la scelta si fa alla fine di ogni trasmissione, non di continuo.

FIFO (First-Come-First-Served)

- Se il collegamento di uscita è occupato, i pacchetti in arrivo alla porta di uscita vengono **bufferizzati**.
- Se non c'è spazio sufficiente, serve una politica di scarto: la più comune è scartare il pacchetto appena arrivato (**drop-tail**); in approcci più sofisticati si possono rimuovere anche pacchetti già in coda per fare posto ai nuovi.
- Un pacchetto viene tolto dalla coda solo quando è stato **completamente trasmesso**.
- I pacchetti vengono trasmessi nello stesso ordine in cui sono arrivati: nessun pacchetto può essere favorito, e nessuno ritardato a vantaggio di un altro. È il default proprio per questo: non c'è niente da configurare e niente da sbagliare.

Priority queuing

- All'arrivo in coda i pacchetti vengono **classificati** in classi di priorità (per esempio in base alle porte TCP/UDP).
- Un operatore può configurare la coda perché alcuni pacchetti (informazioni di gestione della rete, voce su IP in tempo reale, …) abbiano priorità sul traffico degli utenti o non in tempo reale.
- Ogni classe ha la propria coda: si trasmettono per primi i pacchetti della classe **più prioritaria non vuota**; dentro una stessa classe si procede in FIFO.
- È **non preemptive**: una trasmissione già cominciata non viene mai interrotta, e un pacchetto ad alta priorità che arriva a metà aspetta comunque che finisca.

Round-robin queuing

- I pacchetti sono ancora divisi in classi, ma le classi vengono **alternate** invece che scelte per priorità.
- È **work-conserving**: non lascia mai il collegamento inattivo, e una classe con la coda vuota viene saltata subito.
- Un'implementazione comune è il **Weighted Fair Queuing** (WFQ): i pacchetti vengono classificati e accodati nella propria area di attesa, e a ogni classe i è associato un **peso** w_i. Una classe con pacchetti da inviare riceve una quota $w_i / \sum_j w_j$ del servizio, quindi su un collegamento di rate R ha garantito almeno

$$R \cdot \frac{w_i}{\sum_j w_j}$$

qualunque cosa facciano le altre; la priorità stretta, invece, alla classe bassa non garantisce niente.

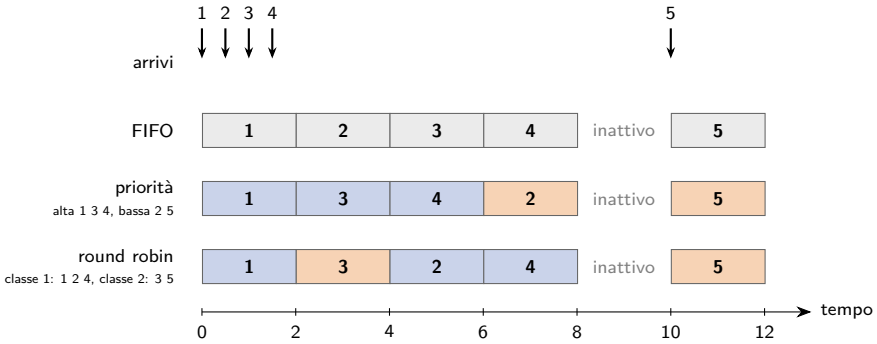


Figura 21.10 – Stessi arrivi, tre ordini di partenza (ogni trasmissione dura 2 unità). Con la priorità 3 e 4 superano 2, che era arrivato prima di entrambi e parte ultimo dei quattro. Con il round robin 3 parte presto perché alla sua classe spettava un turno, mentre 4 non supera 2, che è della stessa classe. Il collegamento resta inattivo prima del 5 in tutti e tre: uno scheduler decide un ordine, non crea mai capacità.

Chi decide che cos'è una classe

Il meccanismo è neutro. Che cosa conta come classe lo decide chi gestisce la rete, su qualsiasi campo dell'header: una porta ben nota (la gestione prima della posta), o un indirizzo sorgente (il traffico di un'azienda che ha pagato prima di quello di una che non l'ha fatto). Lo stesso match-plus-action può anche bloccare il traffico di una certa azienda, o di un certo paese. I limiti qui sono quindi **legali** più che tecnici, e cambiano da paese a paese: è il tema della **neutralità della rete**. Le regole statunitensi del 2015 vietavano blocco, rallentamento e priorità a pagamento, e nel 2017 sono state sostituite da obblighi di trasparenza; nell'Unione Europea il regolamento 2015/2120 vieta blocchi e rallentamenti discriminatori. La questione torna, dal lato del denaro, quando ci si chiede perché due reti accettino di trasportare l'una il traffico dell'altra.

Riepilogo

In breve

- Il livello di rete consegna datagrammi **da host a host**, con servizio **best effort**.
- I router fanno **forwarding** (locale, data plane, hardware, nanosecondi) e **routing** (globale, control plane, software, secondi); il control plane può essere distribuito o centralizzato (SDN).
- Un router ha porte di ingresso, switching fabric (memoria, bus, crossbar), porte di uscita e processore di routing.
- La forwarding table associa **prefissi** a interfacce; vince il **prefisso più lungo**.
- Le code si formano dovunque un rate incontra un rate più lento, in ingresso, in uscita o in entrambi: è lì che nasce la perdita che i mittenti deducono. Buffer troppo grandi non la eliminano, la trasformano in ritardo (bufferbloat).
- La schedulazione può essere FIFO, a priorità (non preemptive) o round-robin (WFQ, con una quota garantita per classe). Resta da vedere da dove vengono gli indirizzi dentro quei prefissi.

Indirizzamento IP e il protocollo IPv4

Il protocollo IP

Avevamo lasciato la forwarding table sistemata: una riga è un prefisso, e quando più righe corrispondono vince la più lunga. Non abbiamo detto **da dove vengono i prefissi**, cioè chi decide che quattro miliardi di indirizzi siano raggruppati così. Niente nell'hardware lo impone: gli indirizzi potrebbero essere distribuiti uno alla volta, nell'ordine in cui si accendono le macchine. Ma allora nessun router potrebbe tenere una tabella: tutto l'edificio poggia sul fatto che gli indirizzi vengono assegnati **a blocchi**.

Ricapitolando: il livello di rete deve consegnare i datagrammi **da host a host**, incapsulando e decapsulando i segmenti, identificando gli host con un **indirizzo** e inoltrando i messaggi verso la destinazione; nella rete i **router**, con le loro porte, la switching fabric e il processore di controllo che crea le tabelle, inoltrano i datagrammi ai nodi adiacenti.

Internet Protocol (IP)

L'**Internet Protocol** è il protocollo di livello di rete che assicura la consegna host-a-host. Oggi ne esistono due versioni:

- **IPv4**, la più usata e diffusa;
- **IPv6**, più recente, proposta per sostituire IPv4.

La funzione più importante di IP è **identificare gli host** in una rete: l'**indirizzamento IP** è il processo con cui si assegnano gli indirizzi IP ai dispositivi. È cruciale ed è molto complesso su Internet, dove ci sono miliardi di host da indirizzare.

IPv4 trasporta ancora quasi tutto il traffico; IPv6 è stato progettato per sostituirlo, e ci arriveremo quando finiranno gli indirizzi. Il servizio è il best effort già descritto: ciò che IP aggiunge è una cosa sola, **i nomi degli estremi**, e da quella dipende ogni altra funzione del livello. È difficile per una ragione banale: lo schema deve nominare miliardi di interfacce e restare cercabile in nanosecondi.

Gli indirizzi IPv4

Gli indirizzi IPv4 si scrivono di solito in **notazione decimale puntata** (*dotted-decimal*): ogni byte dell'indirizzo è scritto in decimale e separato dagli altri da un punto.

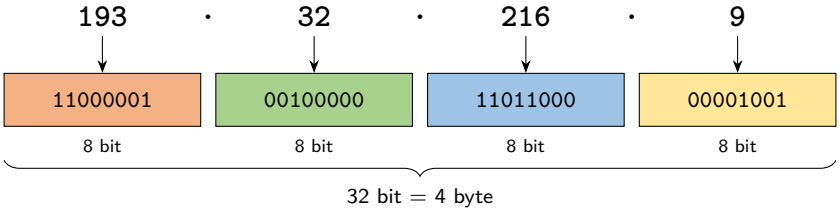


Figura 22.1 – L'indirizzo 193.32.216.9 in notazione decimale puntata e in binario: ogni numero va da 0 a 255.

- Ogni dispositivo sull'Internet globale deve avere (in qualche modo) un indirizzo IP **univoco**.
- Poiché un indirizzo è lungo 32 bit, ci sono in tutto 2^{32} indirizzi possibili, circa **4 miliardi**: un numero che nel 1981 sembrava inesauribile.
- La forma puntata è una comodità per noi. Il protocollo non la usa mai: ogni operazione sugli indirizzi è un'operazione sui bit.

L'indirizzo appartiene all'interfaccia

Host e dispositivi di rete sono collegati da collegamenti, cablati o senza fili. Un host ha di solito un solo collegamento verso la rete, mentre un router ne ha molti.

Interfaccia

Il confine tra un host (o un router) e un collegamento fisico si chiama **interfaccia**. Poiché ogni host e ogni router può inviare e ricevere datagrammi, IP richiede che **ogni interfaccia abbia il proprio indirizzo IP**. Tecnicamente, quindi, un indirizzo IP è associato all'**interfaccia**, non all'host o al router che la contiene.

Un router con tre interfacce ha tre indirizzi IP, e non devono assomigliarsi; un portatile con scheda cablata e Wi-Fi può averne due. Sembra pedanteria finché non si contano le subnet di una topologia reale, dove è proprio ciò che fa tornare il conto.

Il subnetting

Assegnare gli indirizzi alle interfacce non è banale. Assegnarli a caso sarebbe insensato: gli indirizzi non darebbero alcuna indicazione di dove si trovano gli host, e i router avrebbero bisogno di forwarding table enormi.

L'indirizzamento assomiglia a quello della telefonia tradizionale (prefisso internazionale, prefisso della città, numero): le reti sono divise **gerarchicamente** in sottoreti (**subnet**), con prefissi diversi.

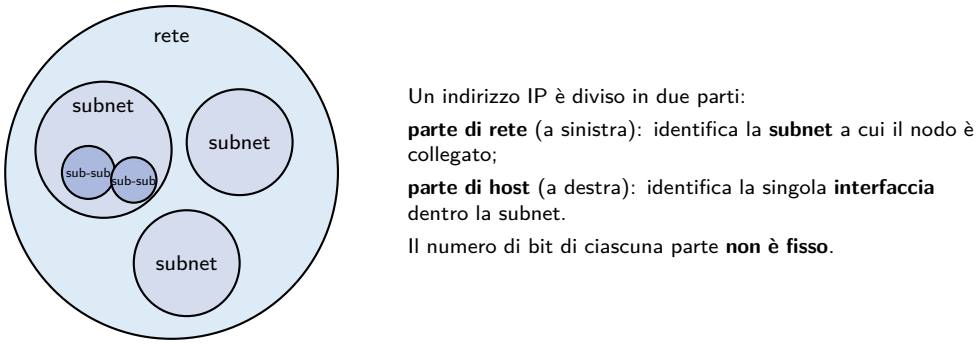


Figura 22.2 – Reti divise gerarchicamente in subnet.

La subnet mask

Subnet mask

Per distinguere la parte di subnet dalla parte di host, a un indirizzo IP è associata una **subnet mask** (maschera di sottorete): un numero di 32 bit che ha **1** nei bit che appartengono alla parte di subnet e **0** in quelli della parte di host.

	Decimale puntata	Binario
Indirizzo IP	193.32.216.9	11000001 00100000 11011000 00001001
Subnet mask	255.255.255.0	11111111 11111111 11111111 00000000
Indirizzo della subnet (AND)	193.32.216.0	11000001 00100000 11011000 00000000

Tabella 22.1 – Un indirizzo con la sua maschera: in blu la parte di subnet, in rosso la parte di host.

Notazione con lo slash

Un altro modo comune di rappresentare le maschere è la **notazione con lo slash**: `193.32.216.0/24` indica l'indirizzo della subnet, dove `/24` dice che i **24 bit più a sinistra** dell'indirizzo sono dedicati alla subnet (maschera 255.255.255.0).

Esempio 1 — Tutti gli indirizzi di una subnet

La subnet `193.32.216.0/24` contiene tutti gli indirizzi che hanno i primi 24 bit uguali:

- il più piccolo: `11000001 00100000 11011000 00000000` = `193.32.216.0`;
- il più grande: `11000001 00100000 11011000 11111111` = `193.32.216.255`.

Sono $2^{32-24} = 2^8 = 256$ indirizzi. Due sono riservati: quello con la parte di host tutta a 0 indica la **rete stessa** (193.32.216.0), quello con la parte di host tutta a 1 è l'indirizzo di **broadcast** (193.32.216.255). Restano **254** indirizzi assegnabili alle interfacce.

Quanti host in una subnet /x

Una subnet `/x` contiene 2^{32-x} indirizzi, di cui $2^{32-x} - 2$ assegnabili agli host. Esempi: /24 → 254 host; /25 → 126; /26 → 62; /30 → 2 (usata per i collegamenti punto-punto tra router); /16 → 65 534.

Quali maschere sono valide

Una subnet mask identifica un **prefisso**: gli 1 devono quindi essere tutti **a sinistra** e **contigui**.

Decimale puntata	Binario	Valida?
255.255.255.0	`11111111 11111111 11111111 00000000`	✔ /24
255.255.128.0	`11111111 11111111 10000000 00000000`	✔ /17
255.224.0.0	`11111111 11100000 00000000 00000000`	✔ /11
255.255.10.0	`11111111 11111111 00001010 00000000`	✖ 1 non contigui
63.255.255.0	`00111111 11111111 11111111 00000000`	✖ non parte da sinistra

Tabella 22.2 – Maschere valide e non valide.

I valori possibili di un byte della maschera

Poiché gli 1 sono contigui, ogni byte di una maschera può valere solo: 0, 128, 192, 224, 240, 248, 252, 254, 255.

La contiguità è ciò che rende il blocco un **intervallo**: fissati i primi x bit, quelli liberi vanno da tutti zeri a tutti uni. Con un buco nella maschera gli indirizzi che corrispondono sarebbero sparsi nello spazio, e nessun confronto dei bit iniziali li troverebbe mai.

Verificare se due host sono nella stessa subnet

Per capire se due host sono nella stessa subnet basta controllare se i loro **prefissi** sono uguali, cioè se **IP1 AND maschera** = **IP2 AND maschera**.

IP 1	Maschera	IP 2	Esito
231.23.11.117	255.255.255.0	231.23.11.9	stessa subnet
10.54.32.1	255.255.128.0	10.54.60.203	stessa subnet
110.32.100.10	255.224.0.0	110.64.100.11	subnet diverse
121.50.79.4	255.0.0.0	121.10.36.240	stessa subnet
121.50.79.4	255.0.0.0	120.50.79.5	subnet diverse

Tabella 22.3 – Esempi di confronto tra indirizzi.

Esempio 2 — Due casi svolti in binario

10.54.32.1 e 10.54.60.203 con maschera /17. Il terzo byte della maschera è 128 = `10000000`: conta solo il suo primo bit.

- 32 = `00100000`, 60 = `00111100`: il primo bit è 0 in entrambi, e i primi due byte coincidono (10.54) ⇒ **stessa subnet** (10.54.0.0/17).

110.32.100.10 e 110.64.100.11 con maschera /11. Il secondo byte della maschera è 224 = `11100000`: contano i primi 3 bit.

- 32 = `00100000`, 64 = `01000000`: i primi tre bit sono diversi ⇒ **subnet diverse**.

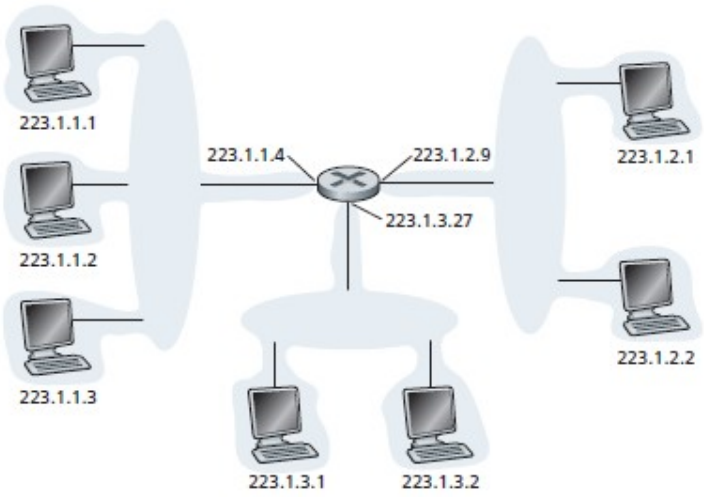
Le cifre decimali non sono una guida: nel terzo caso il primo byte è uguale, eppure 32 e 64 differiscono già nei tre bit coperti dalla maschera /11. Host di subnet diverse possono comunque comunicare, attraverso un router, ed è lì che gli amministratori di rete possono decidere di trattare diversamente i messaggi che entrano o escono da una subnet (per esempio bloccandoli o filtrandoli).

ROUTER E SUBNET

Il dispositivo che collega subnet diverse è il **router**. Per definizione un router ha più porte fisiche, che possono corrispondere a interfacce di rete diverse con indirizzi IP diversi. I router stanno spesso sul **bordo** di più subnet e gestiscono i messaggi in entrata e in uscita: si dice che fanno da **gateway**. Grazie a questa posizione privilegiata offrono spesso altri servizi oltre al routing:

- **IP masquerading** (NAT);
- **assegnazione dinamica degli indirizzi** (DHCP);
- **protezione** (firewall).

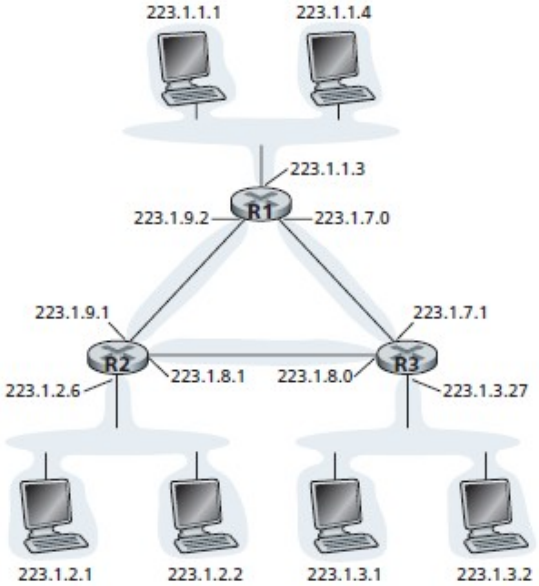
Esempio 3 — Tre subnet e un router



Un router con tre interfacce collega sette host, divisi in tre subnet (sinistra, destra, sotto), ciascuna collegata a un'interfaccia del router.

- La subnet di sinistra ha indirizzo `223.1.1.0/24`: tutte le interfacce hanno indirizzi della forma `223.1.1.xxx` — `223.1.1.1`, `223.1.1.2`, `223.1.1.3` (host) e `223.1.1.4` (router).
- Le altre due subnet sono `223.1.2.0/24` e `223.1.3.0/24`.

Esempio 4 — Tre router e sei subnet



Tre router sono collegati tra loro da collegamenti diretti punto-punto. Ogni router ha tre interfacce: una per ciascun collegamento punto-punto e una per il collegamento broadcast verso una coppia di host. Le subnet sono **sei**:

- `223.1.1.0/24` (R1–host), `223.1.2.0/24` (R2–host), `223.1.3.0/24` (R3–host);
- `223.1.9.0/24` (R1–R2), `223.1.8.0/24` (R2–R3), `223.1.7.0/24` (R3–R1).

Sei subnet, non tre: è per questo che i blocchi finiscono prima di quanto suggerisca il numero degli host. Le organizzazioni medio-grandi (aziende, università) hanno tipicamente molte subnet interconnesse.

Subnet

Si staccano tutte le interfacce dai loro host e router: ogni **isola** che resta è una subnet. Una subnet non è un cavo e non è un dispositivo: è ciò che è raggiungibile **senza attraversare un router**, e anche un collegamento punto-punto fra due router è una subnet, con due interfacce e nessun host.

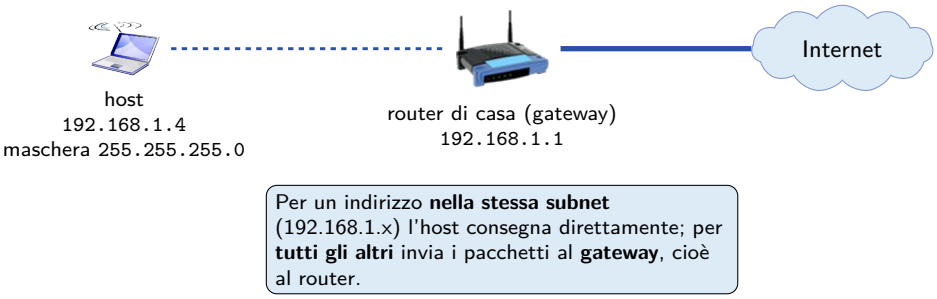


Figura 22.3 – La configurazione tipica di un host di casa: indirizzo, maschera e gateway predefinito.

Anche il router di casa ha **due** interfacce, quindi due indirizzi, uno in ciascuna delle due subnet che unisce: uno interno (`192.168.1.1`, nella subnet della casa) e uno esterno, nella subnet del fornitore. `192.168.0.0/16` è riservato all'uso privato e **non** è globalmente unico: gli stessi indirizzi sono in milioni di case. Come un solo indirizzo pubblico serva tutti i dispositivi dietro di lui è compito della traduzione degli indirizzi (NAT), che vedremo a breve.

Leggere la propria configurazione

```
$ ip -4 addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    inet 217.149.127.10/26 brd 217.149.127.63 scope global eth0
       valid_lft forever preferred_lft forever
```

ip addr: le interfacce e la loro configurazione

- `ip addr show` elenca le interfacce con la loro configurazione; il vecchio comando era `ifconfig`, l'equivalente su Windows è `ipconfig`.
- La parte interessante è il prefisso: `/26` sono 26 uni, cioè la maschera `255.255.255.192`, quindi restano 6 bit. La subnet è `217.149.127.0/26`, da `.0` a `.63`: sessantaquattro indirizzi e non di più.
- `brd 217.149.127.63` è l'indirizzo di broadcast, cioè l'ultimo del blocco: dei 64 indirizzi del /26 se ne possono assegnare **62**.
- `mtu 1500` è il payload più grande, in frame fino a 1518 byte: servirà fra poco.

I due indirizzi di ogni blocco che non sono nostri

L'indirizzo più basso del blocco (bit di host tutti a zero) nomina la subnet stessa e non viene mai dato a un'interfaccia; il più alto (bit di host tutti a uno) è il **broadcast**, consegnato a ogni interfaccia della subnet. Più il blocco è piccolo, più pesano. `255.255.255.255` è invece il **broadcast limitato**: ogni interfaccia della mia stessa subnet, e i router non lo inoltrano.

L'indirizzamento su Internet

Dividere le grandi reti in reti più piccole è particolarmente importante su Internet, dove vanno collegati miliardi di dispositivi (e ancora più interfacce). Gli indirizzi vanno assegnati con cura per evitare che:

- le forwarding table dei router diventino enormi;
- interfacce diverse abbiano lo stesso indirizzo;
- si esauriscano gli indirizzi.

L'approccio è dare alle organizzazioni (ISP, aziende, enti) delle subnet, in due modi: l'indirizzamento **classful** (vecchio, non più usato in pratica) e quello **classless** (attuale).

Indirizzamento classful

Classe	Formato	Primo byte	Esempio	Indirizzi per rete
A	a.b.c.d/8	0–127	10.X.X.X	oltre 16 milioni
B	a.b.c.d/16	128–191	150.10.X.X	65 536
C	a.b.c.d/24	192–223	200.10.10.X	256 (254 host)

Tabella 22.4 – Le classi di indirizzi (le classi D, 224–239, ed E, 240–255, erano riservate a multicast e usi sperimentali).

Esempio 5 — Lo spreco del classful

Fino agli anni Novanta il confine poteva cadere solo a 8, 16 o 24 bit: uno, due o tre byte interi, e niente in mezzo. Un'organizzazione che ha bisogno di 300 indirizzi non può usare una classe C (troppo piccola): riceve una classe B, per esempio `241.115.0.0/16`, con tutti i suoi 65 536 indirizzi. Ne usa 300, e ne **spreca oltre 65 000** che nessun altro potrà mai usare. Lo spazio delle classi B si è svuotato in fretta: la distanza fra ciò di cui un'organizzazione ha bisogno e ciò che le si può dare era strutturale.

Indirizzamento classless (CIDR)

CIDR

L'approccio moderno, più flessibile, si chiama **Classless InterDomain Routing** (CIDR, pronunciato come "*cider*"). A un'organizzazione si può assegnare un indirizzo di rete della forma `a.b.c.d/x`, con x **qualsiasi**.

- La parte di rete dell'indirizzo si chiama **prefisso**.
- L'insieme di indirizzi riservati all'organizzazione si chiama **blocco**.

Esempio 6 — 300 indirizzi con CIDR

Servono 300 indirizzi: bastano 9 bit di host ($2^9 = 512 \geq 300$), quindi un prefisso di $32 - 9 = 23$ bit. L'organizzazione riceve per esempio `241.115.2.0/23`: 512 indirizzi, con soli 212 sprecati (invece di oltre 65 000).

Niente altro cambia: la maschera è ancora uni seguiti da zeri, il blocco è ancora un intervallo; è sparito solo il vincolo su x. E i blocchi vengono assegnati in modo **contiguo**, il che conta più degli indirizzi risparmiati.

Lo stesso indirizzo sta in più blocchi

A quale blocco appartiene `10.10.10.15`? A `10.10.0.0/16`, a `10.10.10.0/24` e a `10.10.10.0/28`, tutti insieme. Un indirizzo da solo non nomina un blocco: la **lunghezza del prefisso** è la metà mancante dell'affermazione.

- I blocchi sono **annidati**, uno dentro l'altro, non affiancati; e un prefisso più corto significa un blocco più grande.
- Quindi un router può benissimo trovare due righe che corrispondono, e sa già che cosa fare: tiene il prefisso più lungo.

Subnet interne a un blocco

Dentro un blocco CIDR si possono creare **subnet interne** all'organizzazione. Il blocco `241.115.0.0/16`, per esempio, si può dividere in: `241.115.1.0/24` (subnet 1), `241.115.2.0/24` (subnet 2), `241.115.3.0/24` (subnet 3), e così via.

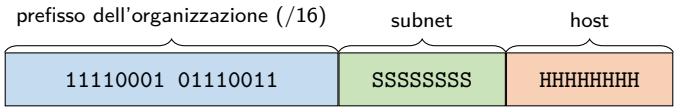


Figura 22.4 – Un blocco /16 diviso internamente in subnet /24. Dentro il campus i router usano tutti e 24 i bit; fuori ne bastano 16, perché i bit sotto il prefisso appartengono all'organizzazione e i router esterni non li guardano mai.

Esempio 7 — Dividere un blocco in sottoblocchi uguali

Un ISP ha il blocco `200.23.16.0/20` e vuole dividerlo tra 8 organizzazioni. Servono $\log_2 8 = 3$ bit in più: i sottoblocchi sono /23, ciascuno da $2^9 = 512$ indirizzi. Il terzo byte vale 16 = `0001 0000`: i 4 bit evidenziati sono del prefisso /20, i 3 successivi numerano l'organizzazione.

Organizzazione	Terzo byte	Blocco
0	0001 0000	200.23.16.0/23
1	0001 0010	200.23.18.0/23
2	0001 0100	200.23.20.0/23
...	...	...
7	0001 1110	200.23.30.0/23

L'AGGREGAZIONE DEGLI INDIRIZZI

L'indirizzamento basato sui prefissi è utilissimo ai dispositivi che collegano prefissi diversi: un router può ricordare (cioè scrivere nella forwarding table) solo i **prefissi**. Quando riceve un messaggio con un certo prefisso lo inoltra verso un router più specifico, e così via.

Aggregazione degli indirizzi

L'**aggregazione degli indirizzi** (*address aggregation* o *route summarization*) consiste nell'annunciare al resto della rete un **unico prefisso** che riassume molti blocchi più piccoli.

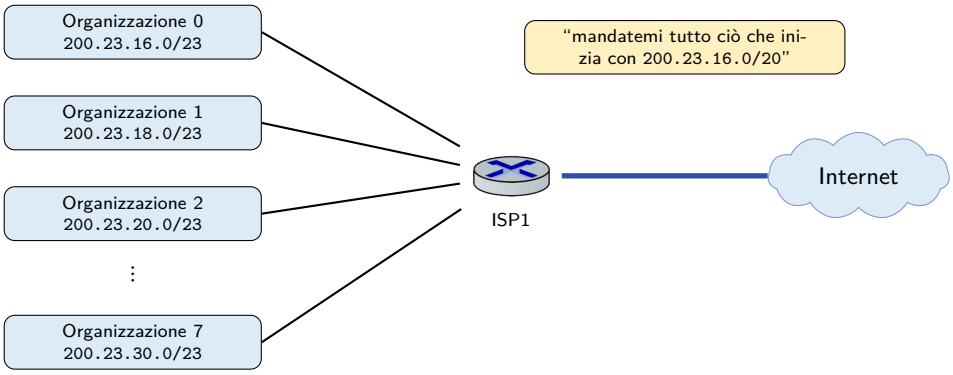


Figura 22.5 – ISP1 annuncia un solo prefisso /20 per le otto organizzazioni: il resto di Internet ha bisogno di una sola riga.

Che succede se l'Organizzazione 1 cambia fornitore e passa a ISP2, portandosi dietro il proprio blocco?

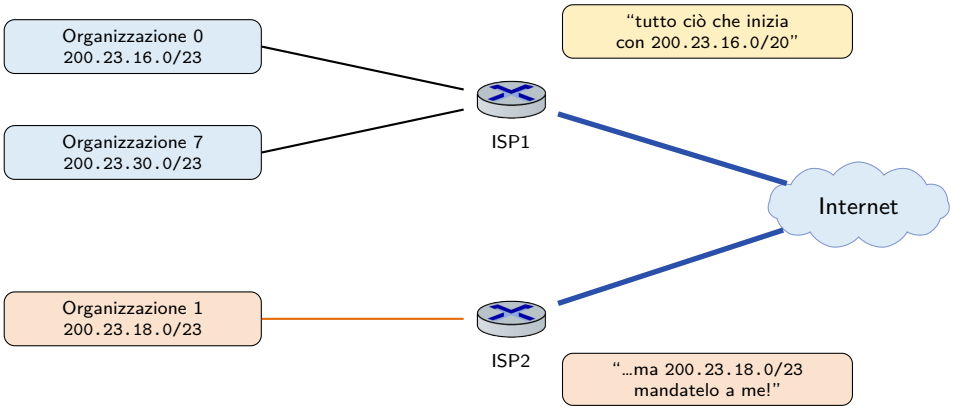


Figura 22.6 – ISP2 annuncia il prefisso più specifico: grazie alla longest prefix matching i pacchetti per l'Organizzazione 1 vanno a ISP2, tutti gli altri del /20 a ISP1.

- Un indirizzo dell'Organizzazione 1 corrisponde sia al /20 di ISP1 sia al /23 di ISP2; per la **longest prefix matching** vince il /23: nessuna ambiguità.
- Blocchi e sottoblocchi possono quindi essere assegnati a linee diverse senza ambiguità, con grande flessibilità.
- L'aggregazione è tanto più efficace quanto più i blocchi restano **raggruppati** (clusterizzati): ogni eccezione è una riga in più nelle tabelle.

Come ottenere un blocco di indirizzi

Per ottenere un blocco di indirizzi per le subnet di un'organizzazione ci sono due strade:

1. **Contattare un ISP**, che fornisce indirizzi presi da un blocco più grande che gli è già stato assegnato. Per esempio l'ISP può avere il blocco `200.23.16.0/20` e dividerlo in sottoblocchi di dimensione variabile secondo le proprie politiche.
2. **Rivolgersi all'ICANN** (*Internet Corporation for Assigned Names and Numbers*), l'autorità globale no-profit responsabile dello spazio degli indirizzi IP, che delega a cinque **registri regionali**: RIPE per l'Europa, poi ARIN, APNIC, LACNIC e AFRINIC. L'ICANN gestisce anche i root server DNS, assegna i nomi di dominio e risolve le controversie sui nomi: entrambi gli spazi di nomi che Internet usa rispondono a una sola autorità.

La prima strada è il modo in cui il raggruppamento avviene in pratica. Resta l'ultimo passo: dare i singoli indirizzi alle interfacce dentro il blocco. Quelli dei router si configurano a mano; quelli degli host li distribuisce automaticamente un protocollo apposito (DHCP), con il resto dei meccanismi *plug-and-play*.

Il datagramma IPv4

Il pacchetto del livello di rete di Internet si chiama **datagramma** (come in UDP).

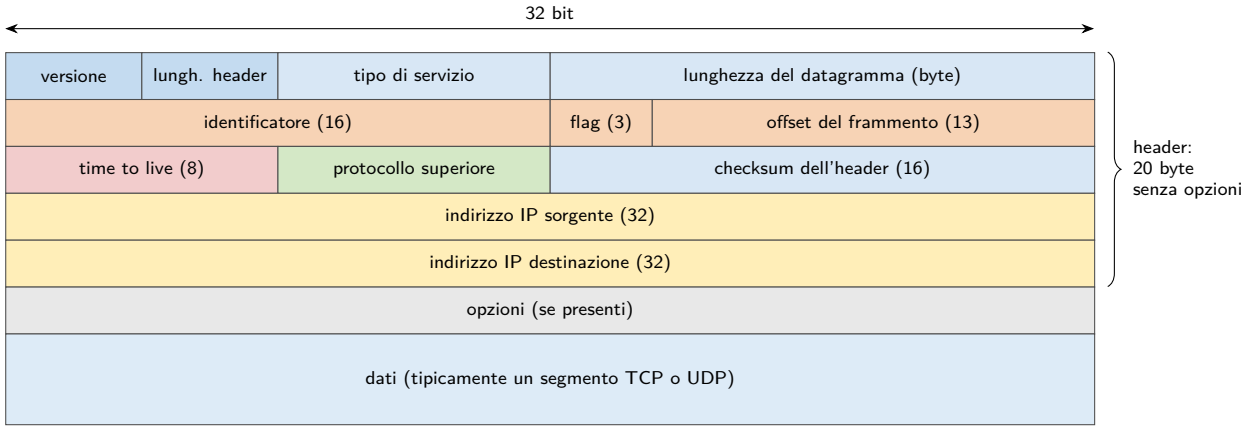


Figura 22.7 – Il datagramma IPv4.

- **Versione** (4 bit): la versione di IP (4 o 6); il formato dipende dalla versione.
- **Lunghezza dell'header** (4 bit): la dimensione dell'header, non fissa per via delle opzioni; serve a sapere dove iniziano i dati. Senza opzioni l'header è di **20 byte**.
- **Tipo di servizio** (TOS, 8 bit): classi di traffico (per esempio tempo reale o no), definite dagli amministratori dei router; due di questi bit portano la notifica esplicita di congestione (ECN) già vista.
- **Lunghezza del datagramma** (16 bit): lunghezza in byte di header + dati. I datagrammi raramente superano i 1500 byte, sui 65 535 possibili.
- **Identificatore** (16 bit): numero progressivo che identifica il datagramma (usato nella frammentazione).
- **Flag e offset del frammento** (3 + 13 bit): proprietà e posizione del frammento (usati nella frammentazione).
- **Time-to-live** (TTL, 8 bit): impedisce che i datagrammi circolino per sempre. Ogni router che elabora il datagramma lo **decrementa di 1**; se arriva a 0, il router **scarta** il datagramma.
- **Protocollo superiore** (8 bit): il protocollo di trasporto a cui consegnare i dati (per esempio **6** per TCP, **17** per UDP; l'elenco completo è sul sito della IANA). È la colla fra rete e trasporto, come il numero di porta lo è fra trasporto e applicazione.
- **Checksum dell'header** (16 bit): complemento a 1 della somma a 16 bit dell'header. I router lo controllano e di solito scartano i datagrammi errati. Va **ricalcolato a ogni router**, perché TTL e opzioni possono cambiare.
- **Indirizzi IP sorgente e destinazione** (32 + 32 bit).
- **Opzioni** (lunghezza variabile): estendono l'header con funzionalità aggiuntive. Aggiungono complessità (la dimensione non è nota a priori) e in IPv6 non esistono più.
- **Dati** (lunghezza variabile): il payload, tipicamente un segmento TCP o UDP.

Quanto pesano gli header

Un segmento TCP dentro un datagramma IP porta almeno 40 byte di header (20 TCP + 20 IP), più l'header del livello applicativo. Per messaggi piccoli l'overhead è significativo.

Due checksum sugli stessi byte

Perché un segmento TCP viaggia sotto due checksum? Perché coprono cose **diverse**: IP controlla solo il proprio header, TCP e UDP l'intero segmento, payload compreso. Un indirizzo di destinazione corrotto lo prende il primo; un bit girato nei dati no, e serve il secondo. E i due livelli non sono saldati: IP trasporta cose che non sono TCP, e TCP potrebbe in principio girare su qualcos'altro. Nessuno dei due livelli può assumere che il lavoro lo faccia l'altro: è la forma generale dell'argomento end-to-end.

La frammentazione IPv4

Il primo problema dei datagrammi IP è che possono dover essere **frammentati** a causa del livello di collegamento: protocolli di collegamento diversi trasportano pacchetti di dimensioni diverse (i frame Ethernet, per esempio, portano al massimo 1500 byte di dati).

MTU

I datagrammi IP vengono incapsulati in frame di collegamento per essere trasportati da un nodo all'altro. La quantità massima di dati che un frame può trasportare è la **Maximum Transmission Unit** (MTU): 1500 byte per Ethernet, 2272 per 802.11. I limiti vengono dall'hardware, dai campi degli header, e dal desiderio che un singolo pacchetto non occupi il canale troppo a lungo.

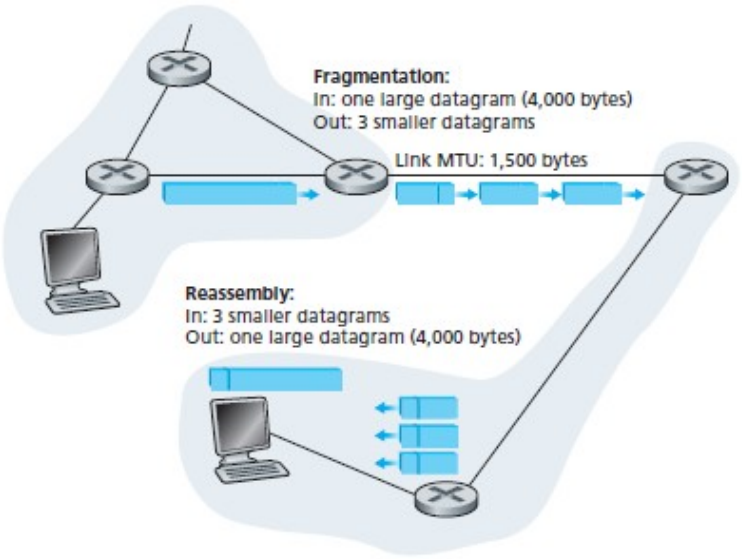


Figura 22.8 – Un router frammenta un datagramma da 4000 byte per un collegamento con MTU di 1500 byte; la destinazione riassembla i tre frammenti.

Un router che collega due collegamenti con MTU diverse può ricevere dal collegamento in ingresso un datagramma che non entra in quello in uscita. Deve allora dividere il payload in due o più datagrammi più piccoli, i **frammenti**, che vengono incapsulati in frame e inoltrati.

Come funziona la frammentazione

Quando un router frammenta un datagramma:

- copia nei nuovi frammenti lo stesso **identificatore**, lo stesso indirizzo sorgente e lo stesso indirizzo di destinazione;
- imposta il campo **offset** di ciascun frammento con numeri progressivi (la posizione dei suoi dati nel datagramma originale, misurata in multipli di 8 byte);
- imposta il **flag** ("more fragments") a 1 in tutti i frammenti tranne l'ultimo, dove vale 0: così si capisce che i frammenti sono finiti.

Esempio 8 — 4000 byte su un collegamento con MTU di 1500

Un datagramma di 4000 byte (20 di header + 3980 di dati) con identificatore 777 deve attraversare un collegamento con MTU di 1500 byte. Ogni frammento può portare al massimo 1500 − 20 = 1480 byte di dati (multiplo di 8).

Frammento	Byte di dati	Identificatore	Offset	Flag
1	1480 (byte 0–1479)	777	0/8 = 0	1
2	1480 (byte 1480–2959)	777	1480/8 = 185	1
3	1020 (byte 2960–3979)	777	2960/8 = 370	0

Il terzo frammento è lungo 1020 + 20 = 1040 byte in tutto. La somma dei dati è 1480 + 1480 + 1020 = 3980 byte.

Il riassemblaggio avviene negli host

I frammenti vanno riassemblati prima di arrivare al trasporto, perché TCP e UDP si aspettano segmenti completi. I progettisti di IPv4 ritennero che riassemblare nei router avrebbe complicato il protocollo e rallentato i router: in IPv4 il **riassemblaggio è eseguito dagli end system**.

- Se l'host riceve più datagrammi con gli stessi indirizzi e lo stesso identificatore, capisce che il datagramma originale è stato frammentato.
- Deve ricostruire il datagramma originale dai frammenti, usando offset e flag. Se un frammento si perde, si scarta l'intero datagramma.

Un frammento perso perde il datagramma

Partono tre frammenti, uno viene scartato lungo la strada. Che cosa arriva? **Niente**. I due arrivati non possono essere consegnati in alto, perché il trasporto si aspetta un segmento intero, e vengono buttati allo scadere del timer di riassemblaggio.

- L'uno per cento di frammenti persi diventa circa il tre per cento di datagrammi persi, e il mittente ritrasmette tutto, che viene frammentato di nuovo, sullo stesso collegamento, con le stesse probabilità.
- Si aggiungono un header per pezzo e il lavoro imposto al ricevente.

La frammentazione è quindi un meccanismo che funziona e che si preferirebbe non usare mai.

Non frammentare: la scoperta dell'MTU di percorso

- I mittenti moderni evitano la frammentazione invece di affidarvisi, e l'header ha un bit apposta: **DF**, *don't fragment*.
- Con DF acceso, un router che non può inoltrare il datagramma lo scarta e restituisce un errore alla sorgente, indicando l'MTU che avrebbe potuto accettare.
- La sorgente riduce i pacchetti e riprova; un collegamento più stretto più avanti fa partire un altro errore, e si riduce di nuovo.
- L'**MTU di percorso** (*path MTU*) è la più piccola MTU lungo il percorso: costa alcune andate e ritorno prima che arrivi qualsiasi dato, e cambia quando cambia la rotta.
- IPv6 ha eliminato del tutto la frammentazione nei router: un datagramma troppo grande viene scartato e la sorgente viene avvisata. Quei messaggi di errore sono un protocollo a sé di questo livello (ICMP), che leggeremo quando arriveremo alla diagnosi di una rete.

Riepilogo

In breve

- Un indirizzo IPv4 è di 32 bit, scritto in notazione decimale puntata; appartiene a un'**interfaccia**.
- Indirizzo = **prefisso** (subnet) + parte di host; la **subnet mask** (o /x) dice quanti bit sono di prefisso. Due host sono nella stessa subnet se IP AND maschera coincide.
- Una subnet /x ha 2^{32-x} indirizzi e $2^{32-x} - 2$ host.
- Dal **classful** (A, B, C, sprechi) al **CIDR** (a.b.c.d/x qualsiasi); **aggregazione** dei prefissi con longest prefix matching per le eccezioni.
- Header IPv4: 20 byte + opzioni; campi chiave TTL, protocollo, checksum, indirizzi, identificatore/flag/offset.
- **Frammentazione**: se il datagramma supera l'MTU, i router lo dividono; gli host lo riassemblano. Un frammento perso perde tutto il datagramma, quindi oggi la si evita con DF e la scoperta dell'MTU di percorso.

ICMP e la diagnosi di una rete

Avevamo lasciato il livello di rete con una promessa: un router che non riesce a consegnare un datagramma lo scarta e **avvisa la sorgente**. Quell'avviso non è una cortesia di IP: è un protocollo di questo livello a pieno titolo, e qui lo leggiamo.

- Con lui tornano tre questioni rimaste aperte: chi dice che nessuno ascolta su una porta, che cosa legge davvero uno scanner di porte, e quale messaggio indica l'MTU che sarebbe passata.
- Poi i due programmi che tutti usano senza sapere che cosa inviano, `ping` e `traceroute`, e un **metodo** per scoprire dove una rete è rotta.
- Il metodo è il punto. I comandi sono dieci minuti; sapere quale domanda fare dopo è il resto del capitolo.

Parlare della rete, non attraverso la rete

ICMP

L'**Internet Control Message Protocol** (ICMP) è il protocollo con cui host e router si scambiano informazioni di livello di rete **sulla rete stessa**: segnalazioni di errore (destinazione irraggiungibile, TTL scaduto) e richieste di verifica (*echo*).

- Dentro non viaggia niente di nostro: nessuna applicazione lo chiede, e nessuna applicazione ne è il mittente.
- Il suo output lo leggiamo da anni: un *destination network unreachable* nel browser è nato come messaggio ICMP di un router che non trovava un percorso.
- Esiste perché IP non promette niente e, lasciato a sé, non riferisce niente: un datagramma che muore muore in silenzio.
- E non tutto ciò che porta è un errore: due dei suoi messaggi sono una domanda e la sua risposta, e su quelle due si regge `ping`.

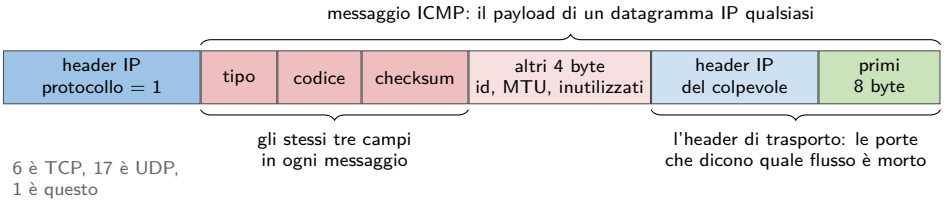


Figura 23.1 – Un messaggio di errore ICMP dentro un datagramma IP. Il ricevente smista sul numero di protocollo esattamente come farebbe per un segmento; non ci sono porte, perché sopra non c'è niente da indirizzare: il messaggio è per la macchina, non per un processo.

L'errore torna con la sua causa allegata

- Ogni messaggio comincia allo stesso modo: un tipo, un codice dentro quel tipo, un checksum, e quattro byte il cui uso dipende dal tipo.
- Poi qualcosa di insolito: una **copia dell'header del datagramma che l'ha causato**, più i suoi primi **8 byte**.
- Perché otto: perché sono i byte dell'header di trasporto che portano i due numeri di porta (oppure, per un echo, l'identificatore e il numero di sequenza).
- Un host può avere centinaia di conversazioni aperte. Senza quegli otto byte saprebbe che qualcosa è fallito, e nient'altro.

Un errore ICMP quindi si può **leggere**: dice che cosa è andato storto, e a quale dei nostri pacchetti.

Tipo	Codice	Significato
0	0	echo reply (risposta a ping)
3	0	destinazione irraggiungibile: rete
3	1	destinazione irraggiungibile: host
3	2	destinazione irraggiungibile: protocollo
3	3	destinazione irraggiungibile: **porta**
3	4	frammentazione necessaria ma vietata (DF)
3	13	comunicazione vietata dall'amministratore
4	0	source quench (controllo di congestione, in disuso)
8	0	echo request (richiesta di ping)
9	0	annuncio di un router
10	0	scoperta dei router
11	0	**TTL scaduto** (Wireshark: *time-to-live exceeded*)
12	0	header IP errato

Tabella 23.1 – I tipi di messaggi ICMP. Quattro bastano per tutto il capitolo: 8 e 0 sono ping, 11 e 3/3 sono traceroute.

Quando non si invia alcun messaggio

- Un errore ICMP non causa mai un altro errore ICMP: altrimenti un datagramma malformato potrebbe far litigare due router per sempre.
- Non si invia niente per un datagramma diretto a un indirizzo di broadcast o multicast: un pacchetto sbagliato, mille risposte.
- Per un datagramma frammentato riceve un messaggio solo il primo frammento, l'unico che porta gli otto byte che vale la pena citare.
- I router limitano il ritmo di ciò che generano, quindi gli errori sono la prima cosa scartata proprio quando sono più numerosi. E un firewall può togliere il resto per scelta.

Un messaggio mancante non significa niente

Ogni altra cosa in questo capitolo dipende da questa frase. Che un errore non sia arrivato non dice che il pacchetto sia passato: può non essere mai stato generato, essere stato limitato, o filtrato.

ping

ping invia a un host messaggi ICMP **echo request** (tipo 8), il cui unico contenuto sono alcuni byte a scelta; la destinazione rimanda un **echo reply** (tipo 0) con quegli stessi byte copiati invariati. È tutto il test. Per ogni coppia ping mostra l'**RTT**, e alla fine le statistiche sulle perdite.

- E la destinazione risponde **da sola**: il server di echo è dentro il sistema operativo, non è un processo, non c'è niente da installare né niente in ascolto.
- Per questo risponde anche una macchina che non esegue alcun servizio, ed è per questo che una risposta dimostra molto meno di quanto si pensi: un datagramma ha raggiunto quell'indirizzo, e lì qualcosa era disposto a rispondere.

```
$ ping iana.org
PING iana.org (192.0.43.8) 56(84) bytes of data.
64 bytes from 192.0.43.8: icmp_seq=1 ttl=51 time=98.2 ms
64 bytes from 192.0.43.8: icmp_seq=2 ttl=51 time=98.0 ms
64 bytes from 192.0.43.8: icmp_seq=3 ttl=51 time=99.4 ms

--- iana.org ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
rtt min/avg/max/mdev = 98.0/98.5/99.4/0.60 ms
```

Leggere un ping

- `56(84)`: 56 byte scelti da noi, più 8 di header ICMP e 20 di header IP. Non si aggiunge altro: non c'è un header di trasporto.
- `ttl=51` è ciò che restava all'arrivo, non ciò che è stato inviato: era partito da 64, quindi la risposta ha attraversato circa tredici router.
- `icmp_seq` conta le sonde, e un buco nella numerazione è l'unico modo in cui una perdita diventa visibile.

Un'applicazione senza livello di trasporto

ping è un programma come gli altri. Quale protocollo di trasporto usa? **Nessuno**. Costruisce da sé il messaggio ICMP e lo consegna direttamente a IP; la risposta torna nello stesso modo. ICMP in entrambe le direzioni: in un ping non c'è alcun segmento, alcuna porta, alcuna connessione.

```c
int fd = socket(AF_INET, SOCK_RAW,   IPPROTO_ICMP);   /* il modo classico */
int fd = socket(AF_INET, SOCK_DGRAM, IPPROTO_ICMP);   /* senza privilegi, su Linux */
```

I due modi di aprire un socket ICMP

Un **raw socket** richiede privilegi, ed è per questo che ping veniva installato *setuid*; oggi Linux offre la seconda forma agli utenti normali.

L'identificatore che fa il lavoro della porta

Due ping girano insieme, verso due host diversi. Entrambe le risposte arrivano come ICMP, senza alcuna porta. Come trova ciascuna il proprio programma? La richiesta porta un **identificatore** e un **numero di sequenza** in quei quattro byte, e la risposta li ricopia entrambi: l'identificatore dice quale ping, il numero di sequenza quale sonda. La chiave che smista il traffico esiste, solo che sta dentro il messaggio invece che in un header proprio; ed è anche ciò che permette a ping di abbinare ogni risposta alla richiesta che ha cronometrato. Un protocollo senza porte ha comunque bisogno di demultiplexing: qualcuno lo paga sempre.

Un silenzio non è una risposta

Si fa ping a un host e non torna niente. È spento? **Non ne segue nulla**. La richiesta può essere stata scartata, la risposta può essere stata scartata, l'host può essere configurato per non rispondere, e un firewall lungo la strada può inghiottire l'una o l'altra direzione.

- Molte reti filtrano ICMP per scelta, quindi nessuna risposta è il comportamento normale di macchine perfettamente sane.
- Il contrario è peggio: un host che risponde può avere tutti i servizi morti. L'echo lo risponde il kernel, e non dimostra niente di ciò che sta sopra.

Va letto come un bit: una risposta significa che il percorso funziona in entrambe le direzioni, adesso. L'assenza significa che non si è imparato niente, che è anch'esso un risultato.

Dove va il tempo

```
$ ping -c3 192.168.1.1                # il gateway, a un salto
rtt min/avg/max/mdev = 0.9/2.4/5.1/1.7 ms

$ ping -c3 iana.org                   # un altro continente
rtt min/avg/max/mdev = 98.0/98.5/99.4/0.60 ms
```

Vicino e lontano

- L'RTT è la somma dei quattro ritardi già noti (elaborazione, accodamento, trasmissione, propagazione) su ogni salto di entrambe le direzioni.
- Lontano, il minimo è quasi pura **propagazione**: un pavimento fissato dalla distanza, che nessun aggiornamento abbasserà.
- Vicino, il minimo è trascurabile e tutto ciò che si vede è **accodamento**: stesso comando, grandezza del tutto diversa.
- Per questo `mdev` conta più della media: 100 ms stabili sono un percorso lungo, 20 ms instabili sono un percorso congestionato.

TRACEROUTE

Il campo si chiama *time to live*, e niente in lui misura il tempo: ogni router lo decrementa di uno. A zero il datagramma viene scartato, e il router che l'ha scartato rimanda **TTL scaduto**, tipo 11. Quel messaggio porta il nome del suo mittente: un datagramma che muore è un datagramma che riferisce **dove** è morto. Il campo era stato messo lì per fermare i cicli di instradamento; si è rivelato uno strumento di misura, e **traceroute** è tutta l'idea: uccidere di proposito i propri pacchetti, un salto più in là ogni volta.

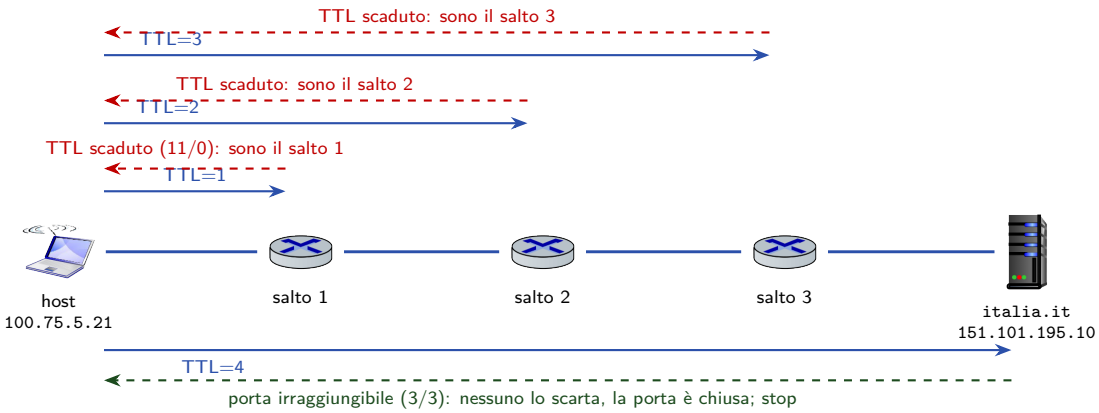


Figura 23.2 – Una sonda per salto. Ogni sonda è un normale datagramma diretto alla destinazione finale: cambia solo il TTL, e solo di uno. La sorgente cronometra ciascuna, quindi ogni riga dell'output è un indirizzo e un tempo di andata e ritorno per quel salto; tre sonde per TTL, quindi tre tempi per riga.

Come sa di essere arrivato

- Prima o poi il TTL è abbastanza grande e la sonda raggiunge la destinazione, che non ha ragione di scartarla.
- Per questo le sonde sono costruite **per essere rifiutate**: ciascuna porta un segmento UDP diretto a una porta molto alta dove nessuno ascolta (Linux parte da 33434 e conta in su).
- La destinazione risponde *port unreachable*, tipo 3 codice 3, e quel messaggio è il segnale di smettere.
- Due messaggi, due mittenti: i router dicono *TTL scaduto*, la destinazione dice *porta irraggiungibile*. Il primo disegna il percorso, il secondo lo chiude. La porta improbabile non è un trucco giocato alla destinazione: è l'unico modo di rendere l'arrivo distinguibile dal transito.

```
$ traceroute italia.it
traceroute to italia.it (151.101.195.10), 30 hops max, 60 byte packets
 1 _gateway (100.75.0.1) 1.785 ms 1.726 ms 2.633 ms
 2 ru-unina-l2-rl1-na01.na01.garr.net (193.206.130.9) 3.907 ms 3.869 ms 3.831 ms
 3 212.189.220.46 31.333 ms 212.189.220.218 4.657 ms 212.189.220.46 11.247 ms
 4 212.189.220.41 17.597 ms 18.875 ms 212.189.220.43 9.881 ms
 5 rs1-bo01-re1-rm02.rm02.garr.net (185.191.181.78) 27.630 ms 27.593 ms 27.556 ms
 6 * * *
```

Leggere un traceroute

- Il salto 1 è il proprio gateway; il salto 2 nomina già la rete che si sta attraversando (GARR, la rete della ricerca italiana). I nomi valgono più degli indirizzi.
- Il salto 3 risponde da due indirizzi diversi: le tre sonde non hanno preso lo stesso percorso. È **bilanciamento del carico**, non un errore, e lo stesso salto mostra 31 ms e poi 4 ms.
- I tempi non sono monotoni lungo la colonna, perché ciascuno è una misura nuova con il proprio accodamento.

Gli asterischi

La traccia arriva fino al salto 30 senza mai arrivare. I pacchetti sono arrivati? **Molto probabilmente sì**. La destinazione sta dietro un filtro che scarta le sonde in silenzio, quindi il *port unreachable* che fermerebbe il comando non arriva mai, e il comando continua a contare fino al limite.

- Un asterisco è nessuna risposta entro il timeout: un filtro, un limite di ritmo, o un router che semplicemente non genera ICMP.
- Asterischi in mezzo, con risposte dopo, dicono solo che quel salto resta zitto: il percorso è intatto.
- Né la direzione inversa deve coincidere: ciò che si misura è un'andata e ritorno, e la strada del ritorno può non essere quella dell'andata.
- Un traduttore di indirizzi a un confine è un altro motivo per cui una traccia smette di essere informativa.

Due varianti della stessa idea

- **Sonde UDP**: il default su Linux, quello appena descritto; la traccia finisce su un *port unreachable*.
- **Sonde echo**: `traceroute -I`, ed è ciò che fa `tracert` su Windows: echo request con TTL crescente, e la traccia finisce su un echo reply.
- Il meccanismo è identico: cambiano solo la sonda e la condizione di arresto, e i router in mezzo rispondono allo stesso modo.
- Conta perché le due vengono filtrate in modo diverso: un percorso che inghiotte UDP verso porte strane può rispondere all'echo, e uno che blocca tutto ICMP non risponde a nessuna delle due. Quando una traccia sembra sbagliata, lanciare l'altra variante è il secondo parere più economico che ci sia.

Quale processo riceve la risposta?

Il messaggio del router è indirizzato al nostro host e non porta porte. Quale dei nostri programmi lo riceve? **Nessuno**, per la via ordinaria. traceroute legge ICMP da un raw socket, oppure chiede al kernel di riferire gli errori sul socket con cui ha inviato la sonda, e riconosce la propria sonda dagli **otto byte citati**, che sono le porte che ha scelto. È lì che la regola della citazione si guadagna il posto: senza, una risposta non potrebbe essere abbinata a una sonda. Quindi traceroute invia UDP e comunque non usa alcun servizio di trasporto: il segmento è un contenitore scelto perché nessuno ascolti. Due programmi, la stessa forma: entrambi vivono al livello di rete e prendono in prestito l'API dei socket per arrivarci.

Il nome risolto prima della prima sonda

```
Domain Name System (query)
  Flags: 0x0100 Standard query
    .... ...1 .... .... = Recursion desired: Do query recursively
  Queries
    italia.it: type AAAA, class IN
```

Il primo pacchetto della cattura, in Wireshark

- Il primo pacchetto di una cattura non è una sonda: è una query DNS, perché al comando si è dato un nome. È acceso *recursion desired*: l'host chiede al resolver di fare tutto il lavoro e restituire una risposta.
- E succede di nuovo al contrario, una volta per salto, per trasformare ogni indirizzo in un nome: è per questo che `traceroute -n` è più veloce.
- Quindi una traccia rotta può essere un servizio dei nomi rotto, e la cattura distingue i due casi nei primi due pacchetti.

TRACEROUTE E LA FRAMMENTAZIONE IP

Con il traceroute di Linux si può indicare, come secondo argomento, la **lunghezza totale** del pacchetto sonda (non il payload; il default è 60 byte):

```
$ traceroute italia.it          # pacchetti piccoli (dimensione di default)
$ traceroute italia.it 3000     # pacchetti da 3000 byte: verranno frammentati
```

La dimensione comprende la parte dati più gli header UDP e IP che la incapsulano (20 byte di header IP, 8 di header UDP, il resto è payload). Se supera l'MTU del percorso la sonda viene **frammentata**. La dimensione riguarda le sonde, non i messaggi ICMP ricevuti, che contengono solo una parte del pacchetto che ha generato l'errore.

	Dati	Nel campo	Wireshark stampa	MF
frammento 1	1480	0	0	1
frammento 2	1480	185	1480	1
frammento 3	20	370	2960	0

Tabella 23.2 – Una sonda da 3000 byte su un collegamento Ethernet. L'header conta l'offset in unità di 8 byte; Wireshark lo moltiplica per 8 prima di stamparlo. La sua stessa visualizzazione dei bit lo conferma: `0 0000 1011 1001` è 185, etichettato 1480. Tutti e tre hanno lo stesso identificatore e lo stesso TTL: la frammentazione avviene sotto la sonda e non la disturba.

Destinazione irraggiungibile, e chi lo dice

Codice	Significa	Inviato da
0	nessuna rotta verso quella rete	un router lungo la strada
1	la rete c'è, l'host no	l'ultimo router
2	nessuno gestisce quel numero di protocollo	la destinazione
3	nessuno ha fatto `bind` su quella porta	la destinazione
4	troppo grande, e la frammentazione è vietata	un router lungo la strada
13	vietato dall'amministratore	un firewall

Tabella 23.3 – I codici del tipo 3. La colonna utile è quella del mittente: i codici 0, 1 e 4 vengono dal mezzo del percorso, 2 e 3 dimostrano che si è arrivati. Il codice 3 è quello che incontrano i nostri programmi, ed è quello che chiude una traccia.

Frammentazione necessaria, e il buco nero

Il datagramma è troppo grande per il collegamento successivo e porta *don't fragment*. Il router lo scarta e avvisa, e l'avviso viene filtrato sulla via del ritorno. Che cosa vede il mittente? **Niente**. Ritrasmette per sempre lo stesso datagramma troppo grande. L'handshake è riuscito, i messaggi piccoli passano, quelli grandi spariscono, e la connessione si blocca a metà di un trasferimento.

- Il tipo 3 codice 4 porta, nei suoi quattro byte liberi, l'MTU che sarebbe passata: la scoperta dell'MTU di percorso è costruita su quel numero.
- Bloccare tutto ICMP rompe quindi un meccanismo da cui TCP dipende, e il sintomo indica ovunque tranne che il firewall. Nasconde ben poco (un host si trova in altri modi) e costa esattamente i messaggi che servono quando qualcosa si rompe.

I messaggi caduti in disuso

- **Source quench**: controllo di congestione via ICMP, un router che chiede a un host di rallentare. Non si usa più: quel compito è passato al livello di trasporto, e ai due bit ECN nell'header IP.
- **Redirect**: un router che dice a un host che ha scelto il primo salto sbagliato. Oggi di solito si ignora, perché uno falsificato dirotta il traffico.
- **Scoperta dei router**: un host che chiede chi sono i router. Sopravvive, ma l'indirizzo arriva di solito per un'altra via (DHCP).
- La versione nuova del protocollo (ICMPv6) riorganizza tutto questo e aggiunge *packet too big*, che lì non è facoltativo: i router non frammentano più, quindi il messaggio è l'unico modo di scoprirlo.

Una scala di domande

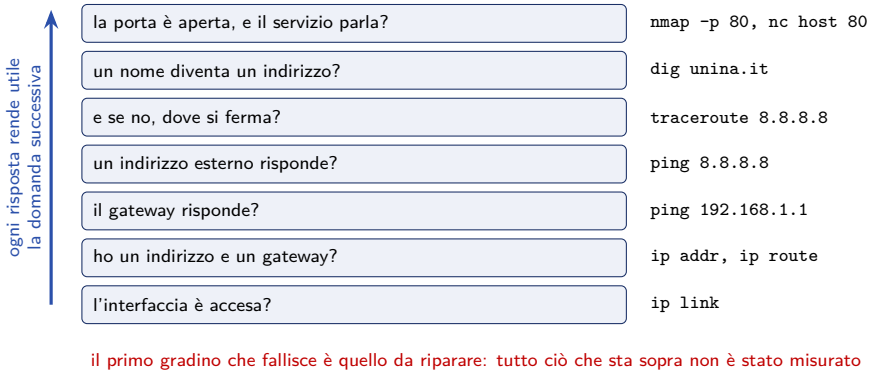


Figura 23.3 – Una scala di domande per diagnosticare una rete, dal basso verso l'alto.

- Si comincia dal basso anche quando si è sicuri: le due domande più economiche sono quelle che falliscono più spesso.
- Ciò che ICMP aggiunge è che ai gradini di mezzo si possa rispondere: senza, tutto ciò che sta fra noi e il servizio è un unico silenzio indiviso.

Comando	A quale domanda risponde	Dove guarda
`ip link, ip addr`	sono connesso, e chi sono	questa macchina
`ip route`	dove va per primo il mio traffico	questa macchina
`ping`	un datagramma arriva e torna?	il percorso
`traceroute`	fin dove arriva prima di fermarsi	il percorso
`dig`	il nome diventa un indirizzo?	il servizio dei nomi
`ss -tn`	ci sono i socket che mi aspetto?	questa macchina
`nmap`	chi è acceso, quali porte rispondono	l'altro capo
`nc`	il servizio parla davvero?	l'altro capo
`tcpdump`	che cosa ha lasciato davvero la macchina	ogni livello

Tabella 23.4 – La cassetta degli attrezzi. Nessuno risponde a una domanda che quello sopra ha lasciato aperta: è l'ordine lo strumento, i comandi sono solo comandi.

Chi c'è su questa rete

```
$ sudo nmap -sn 192.168.1.0/24
Nmap scan report for _gateway (192.168.1.1)
Host is up (0.0024s latency).
MAC Address: 10:BD:18:E5:74:80 (Cisco Systems)
Nmap scan report for 192.168.1.17
Host is up (0.0031s latency).
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.35 seconds
```

nmap -sn: chi esiste, senza toccare le porte

- `-sn` chiede chi esiste e si ferma lì: nessuna porta viene sondata.
- Sulla propria subnet gli indirizzi MAC tradiscono il metodo: non sta facendo ping a nessuno, sta chiedendo al livello di collegamento chi possiede ogni indirizzo (ARP). Nient'altro potrebbe restituire un MAC.
- Fuori dalla subnet quell'opzione non c'è più, e la scansione ripiega su echo request più una sonda a una o due porte comuni. Lo stesso comando risponde a una domanda diversa da dentro e da fuori una rete.

Risultato	Che cosa torna	Chi l'ha inviato
open	SYNACK	il servizio stesso
closed	RST	il TCP remoto: l'host è acceso
filtered	niente, oppure ICMP 3/13	qualcosa lungo la strada

Tabella 23.5 – Aperta, chiusa, filtrata, e chi lo dice. La terza riga è dove compare questo livello: un firewall può essere educato e dire *administratively prohibited*, o non dire niente.

Su UDP non c'è RST: una porta chiusa risponde *port unreachable*, una filtrata non risponde niente, quindi lo scanner deve aspettare un timeout per distinguerle. E il limite di ritmo sugli errori ICMP rende una scansione UDP lenta di proposito: è il protocollo che si difende.

DNS, VPN e spoofing: un caso di studio

Durante un'esercitazione si è provato a interrogare direttamente un root server, collegati attraverso una **VPN** (una rete privata virtuale che cifra tutto il traffico verso un server remoto).

```
$ dig +norecurse @<un root server> iana.org A        # chiedere direttamente alla radice
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 6, ADDITIONAL: 13
org.        172800 IN NS a0.org.afilias-nst.info.     <- atteso: un rinvio, nessun indirizzo

$ dig +norecurse @<un root server> iana.org A        # stesso comando, dietro una VPN
iana.org.      340 IN A     192.0.43.8                <- una risposta che non poteva dare
```

Ciò che una cattura non può vedere

Qualcosa non torna. Un root server **non** ha un indirizzo per quel nome: dovrebbe restituire i server della zona `org`. Dietro un certo fornitore di VPN, ha restituito l'indirizzo.

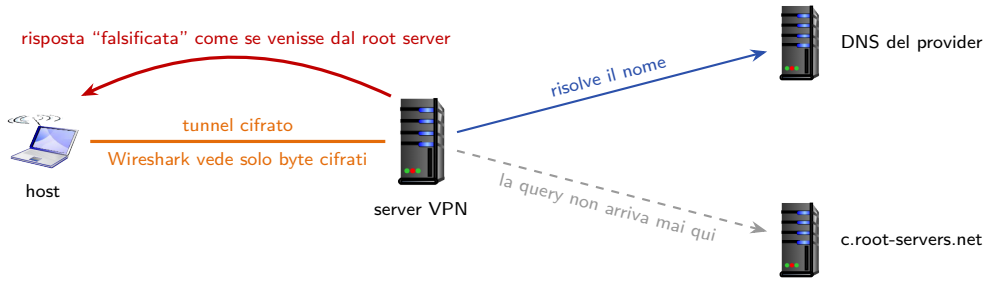


Figura 23.4 – Il server VPN intercetta la query DNS, la risolve da sé e risponde fingendo di essere il server interrogato.

- Wireshark non ha mostrato niente: il traffico verso la VPN è cifrato, e la cattura non aveva pacchetti da mostrare.
- La spiegazione è che il server VPN ha risolto il nome da sé e ha restituito la risposta come se venisse dal server interrogato, facendo **spoofing** dei datagrammi UDP/IP. Con un altro fornitore il rinvio è tornato come atteso.
- Lo strumento non ha mentito: ha riferito ciò che è arrivato. Ciò che non può riferire è chi l'ha inviato davvero; ma l'header sì: un root server **non accende mai `ra`** (e in un rinvio nemmeno `aa`). Non serve una VPN per vederlo: molte connessioni domestiche intercettano sempre la porta 53.

La lezione

Il campo "indirizzo sorgente" di un datagramma IP **non è una prova** di chi l'abbia davvero inviato: chi controlla un punto del percorso può falsificarlo. È esattamente il problema dell'**IP spoofing** che affronteremo nella parte sulla sicurezza, e il motivo per cui servono meccanismi di autenticazione.

La nostra chat vista da un livello più in basso

```
$ ./chatd 5000 &
$ sudo tcpdump -i lo -n -X 'port 5000'       # oppure il filtro ip.addr==... in Wireshark
```

- Tutto ciò che la chat invia è un segmento dentro un datagramma: protocollo 6 per la versione connessa, 17 per quella su datagrammi.
- Su loopback il TTL parte a 64 e arriva a 64: nessuno l'ha instradato. Attraverso il laboratorio arriva a 63, e quello che manca è il router.
- Nel payload si legge la nostra delimitazione: quattro byte di lunghezza, uno di tipo, poi il corpo. È il primo header del corso scritto da noi. Niente di ciò che sta sotto sa che cosa significano quei cinque byte: il formato è un accordo fra i nostri due programmi e nessun altro.

Il messaggio che la chat riceve già

Si avvia il client su datagrammi senza alcun server: nessuno ha fatto `bind` sulla porta 5000. Che cosa succede sul filo? L'host a cui si è scritto risponde *port unreachable*, tipo 3 codice 3: proprio il messaggio che chiude un traceroute. Se il programma lo viene mai a sapere dipende da una chiamata: senza `connect` il kernel non sa quale socket riguardi il messaggio, e lo butta; il client va in timeout e ritrasmette per sempre.

```
connect(fd, (struct sockaddr *) &srv, sizeof srv);   /* registra il peer */
sendto(fd, msg, len, 0, NULL, 0);                    /* ritorna bene: e' partito */

ssize_t n = recv(fd, buf, sizeof buf, 0);
if (n < 0 && errno == ECONNREFUSED)
    ;                                                /* l'ICMP e' tornato, ed e' stato tenuto */
```

Con connect, l'ICMP diventa un errno

errno	Che cosa è arrivato	Che cosa se ne deduce
`ECONNREFUSED` (TCP)	un RST	host acceso, niente su quella porta
`ECONNREFUSED` (UDP)	ICMP 3/3	host acceso, nessuno in ascolto lì
`EHOSTUNREACH`	ICMP 3/1	un router non è riuscito a consegnare
`ENETUNREACH`	ICMP 3/0	nessuna rotta verso quella rete
`EACCES`	ICMP 3/13	un firewall l'ha rifiutato, e l'ha detto
`ETIMEDOUT`	niente del tutto	non si è imparato niente

Tabella 23.6 – Che cosa dice `connect`, e chi gliel'ha detto. Quattro dei sei sono un messaggio ICMP tradotto in un intero: il livello di rete che riferisce attraverso una chiamata dei socket. Il primo è un segmento TCP, e solo l'ultimo è niente; ed è proprio l'ultimo che viene riferito come «il server è giù», che è l'unico a non dire nulla del genere.

Esercizio 1 — Catturare un traceroute

Si avvia la cattura, si lancia `traceroute italia.it` e lo si lascia finire. Filtro di visualizzazione: `ip.src==<mio indirizzo> && ip.dst==<il server> && udp`. Si seleziona la prima sonda e si apre la sezione *Internet Protocol*.

(a) Qual è l'indirizzo dell'host, e che cosa c'è nel campo TTL di questa prima sonda?

(b) Quale protocollo superiore dichiara l'header, e con quale numero?

(c) Quanti byte di header IP ci sono, e quanti di payload? Dire come si è ricavato il secondo: non è un campo.

(d) Questo datagramma è stato frammentato? Indicare i campi guardati, e che cosa avrebbero detto se lo fosse stato.

Poi si guarda l'intera sequenza di sonde, e ciò che è tornato con `ip.dst==<mio indirizzo> && icmp`.

(e) Fra le sonde, quali campi dell'header restano costanti e quali cambiano? Per ciascuno costante, perché deve esserlo?

(f) Descrivere l'andamento del campo *Identification* delle sonde.

(g) Nei datagrammi inviati dai router, quale protocollo superiore è dichiarato?

(h) I loro valori di *Identification* sono legati a quelli delle sonde? Dovrebbero esserlo?

(i) I TTL dei messaggi che tornano sono simili fra loro? Da dove è partito ciascuno?

Svolgimento

(a) Il campo *Source* è l'indirizzo dell'host (sulla rete di ateneo per esempio `100.75.5.21`); il TTL della prima sonda vale **1**.

(b) **17**, UDP: le sonde sono segmenti UDP verso porte alte.

(c) Il campo *Header Length* vale 5 parole da 32 bit, cioè **20 byte**; il payload si ottiene come *Total Length* meno l'header: con le sonde di default da 60 byte, **40 byte** (8 di header UDP e 32 di dati).

(d) No: *More Fragments* vale 0 e *Fragment Offset* vale 0. In un frammento uno dei due sarebbe diverso da zero (MF a 1 in tutti tranne l'ultimo, offset diverso da 0 in tutti tranne il primo).

(e) Restano costanti versione, lunghezza dell'header, protocollo e i due indirizzi: la sonda deve andare sempre alla stessa destinazione e la risposta tornare allo stesso host, e il tipo di sonda non cambia. Cambiano il **TTL** (cresce di uno ogni tre sonde), l'**identificatore** e quindi il **checksum dell'header**, ricalcolato perché TTL e identificatore cambiano; nel segmento UDP cambia anche la porta di destinazione.

(f) Cresce di uno a ogni sonda: è il contatore dell'host che numera i propri datagrammi.

(g) **1**, ICMP.

(h) No, e non devono esserlo: ogni router numera i datagrammi che genera con il proprio contatore. Il legame con la sonda sta altrove, negli otto byte citati.

(i) No: ogni router parte dal TTL iniziale del proprio sistema (spesso 255, oppure 64), e il valore che arriva è quello iniziale meno i salti del ritorno; i router più lontani arrivano con valori più bassi.

Esercizio 2 — Tremila byte, sulla propria macchina

Si ripete con `traceroute italia.it 3000` e il filtro `ip.src==<mio indirizzo> && ip.dst==<il server>`.

(a) Trovare il primo frammento della prima sonda. In quanti datagrammi si sono trasformati i 3000 byte?

(b) Quali campi dell'header dicono che si tratta di un frammento?

(c) Come si distingue un primo frammento da quelli successivi, e l'ultimo da quelli precedenti?

(d) Quanti byte porta ciascuno, header compreso? Verificare che tornino.

(e) Confrontare i bit del campo offset con il numero che Wireshark stampa accanto. Quale dei due sta nel datagramma?

Svolgimento

(a) In **tre**: l'MTU Ethernet è 1500 byte.

(b) Il flag *More Fragments* e il campo *Fragment Offset*.

(c) Il primo ha offset 0 e MF = 1; quelli in mezzo offset > 0 e MF = 1; l'ultimo offset > 0 e MF = 0.

(d) 1500, 1500 e 40 byte. Tolti i tre header da 20 byte restano 1480 + 1480 + 20 = 2980 byte, cioè i 3000 della sonda meno il suo header originale.

(e) Nel datagramma c'è il numero in unità di 8 byte: per il secondo frammento `0 0000 1011 1001` = 185. Wireshark stampa $185 \times 8 = 1480$, la posizione in byte.

Esercizio 3 — E il ping

Si cattura `ping -c5 iana.org` con il solo filtro `icmp`.

(a) Individuare gli echo request inviati e le risposte corrispondenti. Quali indirizzi sono sorgente e destinazione in ciascuna direzione?

(b) In un echo request: qual è il TTL nell'header IP, il tipo nell'header ICMP, l'identificatore e il numero di sequenza?

(c) Misurare il tempo fra una richiesta e la sua risposta, e confrontarlo con quello stampato da ping per quel numero di sequenza.

(d) Ripetere verso un host che non risponde. Che cosa contiene la cattura, e che cosa permette di concludere?

Svolgimento

(a) Le richieste vanno dal proprio indirizzo a `192.0.43.8`; le risposte fanno il percorso inverso.

(b) TTL 64 (il default di Linux), tipo **8**; l'identificatore è lo stesso in tutte e cinque le richieste (identifica questo ping), il numero di sequenza va da 1 a 5. Le risposte, di tipo 0, ricopiano entrambi.

(c) I due tempi coincidono quasi esattamente: ping misura lo stesso intervallo, dal lato dell'applicazione, con in più il poco che serve a passare dal kernel.

(d) Solo echo request, nessuna risposta. Non si può concludere che l'host sia spento: è il «silenzio che non è una risposta» del capitolo.

Riepilogo

In breve

- **ICMP** è il modo in cui host e router parlano della rete: sta sopra IP, viaggia dentro IP (protocollo 1), e non ha porte.
- Un messaggio porta un tipo, un codice, e l'header più i primi otto byte del datagramma che l'ha causato: un errore nomina la propria causa.
- **ping** sono i tipi 8 e 0, e risponde a una sola domanda: un datagramma ha raggiunto quell'indirizzo e qualcosa ha risposto.
- **traceroute** uccide di proposito le proprie sonde: TTL scaduto (11) da ogni router, porta irraggiungibile (3/3) dalla destinazione, che è come sa di doversi fermare.
- Un silenzio non è una risposta: gli errori sono limitati, filtrati, e in diversi casi mai generati. Bloccare tutto ICMP rompe la scoperta dell'MTU di percorso.
- La diagnosi è un ordine di domande, dal basso verso l'alto. Prossimo argomento: come un host ottiene il proprio indirizzo, e che cosa succede quando un'intera rete si nasconde dietro uno solo.

DHCP, NAT e IPv6

L'assegnazione degli indirizzi IP

Ora sappiamo scoprire dove una rete è rotta. Non ci siamo mai chiesti da dove venga l'indirizzo di questa macchina: nessuno l'ha scritto. È stato **chiesto**, concesso per un tempo limitato, e domani può essere un altro. L'altra domanda è quella in cui le tracce continuavano a inciampare: un'intera rete che vive dietro un solo indirizzo pubblico. Il protocollo che distribuisce gli indirizzi, la traduzione che fa servire una casa da un indirizzo, e il successore di IPv4 stanno insieme: esistono tutti e tre perché trentadue bit non bastavano, e non bastano da trent'anni.

Un blocco di indirizzi non è una configurazione: qualcuno deve ancora dare un indirizzo a ciascuna interfaccia. Un amministratore di sistema può farlo in due modi:

- **manualmente**: scrivendo indirizzo, maschera e gateway nella macchina (indirizzo *statico*). È così che si configurano le interfacce dei router;
- **automaticamente**: è la rete ad assegnare un indirizzo libero a chiunque arrivi. È così che si configurano gli host.

Fare gli host a mano significa tenere una lista di quali indirizzi sono liberi e non sbagliarla mai; e non sopravvive al movimento: un portatile portato da un'aula a una biblioteca a un'aula magna ha bisogno di un indirizzo nuovo in ciascun posto. Il modo automatico è un protocollo, e gira su ogni macchina di quest'aula: il **Dynamic Host Configuration Protocol** (DHCP).

DHCP

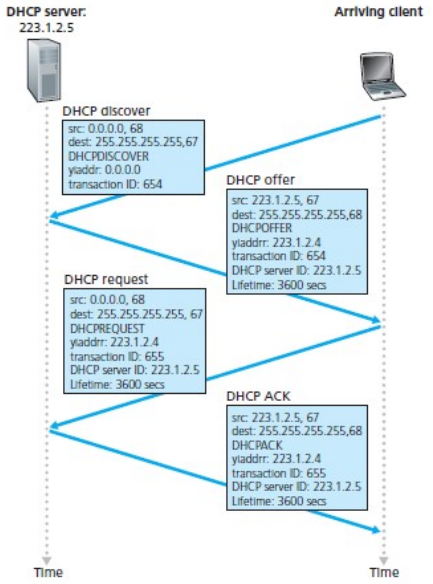
Il **DHCP** assegna automaticamente a un host che si collega alla rete le informazioni necessarie per usarla: oltre all'**indirizzo IP**, la **subnet mask**, l'indirizzo del **gateway predefinito** (per uscire dalla rete) e quello del **server DNS locale**. È *plug-and-play* (o *zeroconf*), ed è quello che si usa nelle nostre case.

Un indirizzo da solo non porta una macchina da nessuna parte. Le quattro cose arrivano insieme: l'indirizzo e la maschera (quali destinazioni sono sulla mia subnet e quali no), il router del primo salto (dove deve andare tutto ciò che non è sulla mia subnet) e il resolver DNS locale (senza il quale ogni nome che l'utente scrive è irrisolvibile).

Il funzionamento del DHCP

Il DHCP è un protocollo **client-server**: un host appena arrivato (client) contatta il server DHCP per ricevere la configurazione di rete. Il servizio può essere offerto da un computer o, in una casa quasi sempre, dal router stesso. Gira su UDP, porta **67** al server e **68** al client: formalmente è un protocollo di **livello applicativo**. Vale un momento di riflessione: ciò che configura il livello di rete sta in cima alla pila e lo usa come qualsiasi altra applicazione.

Riprendiamo l'esempio delle tre subnet collegate da un router, e supponiamo che la subnet di destra (`223.1.2.0/24`) abbia un server DHCP: quando un nuovo host entra nella rete, "contratta" il proprio indirizzo con il server. L'aggiunta di un host avviene in quattro passi (ricordati spesso con l'acronimo **DORA**):



1. **Discover** — ricerca del server: il nuovo host invia in **broadcast** un messaggio *DHCP discover* (UDP, porta **67**) per trovare un server DHCP.
   - IP destinazione: `255.255.255.255` (broadcast);
   - IP sorgente: `0.0.0.0` ("questo host", che non ha ancora un indirizzo);
   - un identificativo di transazione casuale.
2. **Offer** — offerta: poiché possono esserci più server, ogni server che riceve il discover risponde con un messaggio in broadcast (porta **68**) che offre una possibile configurazione (campo `yiaddr`, *Your Internet ADDRess*). IP destinazione `255.255.255.255`, IP sorgente quello del server (`223.1.2.5`).
3. **Request** — richiesta: il client sceglie un'offerta tra quelle ricevute e la accetta "facendo l'eco" del messaggio di offerta.
4. **ACK** — conferma: il server conferma la transazione con un messaggio finale.

Figura 24.1 – I quattro messaggi del DHCP (da Kurose).

Un **identificativo di transazione** lega insieme i quattro messaggi: nessuno di loro è indirizzato a qualcuno in particolare.

Indirizzare un host che non ha un indirizzo

Il client non ha un indirizzo. Che cosa va nel campo sorgente, e come può mai tornargli una risposta? Sorgente `0.0.0.0`, «questo host»; destinazione `255.255.255.255`, «tutti qui». Anche la risposta è in broadcast: nessuno dei due può ancora indirizzare l'altro, quindi gridano entrambi.

- A consegnare davvero quei due datagrammi è il livello sotto, che ha indirizzi suoi e un broadcast suo. Ogni interfaccia ne porta uno, fissato in fabbrica: non è un indirizzo IP, e non l'ha mai configurato nessuno.
- Un broadcast di subnet non viene inoltrato dai router, quindi tutto lo scambio è confinato a una subnet per costruzione: i quattro messaggi funzionano solo se il server sta sulla stessa subnet del client.

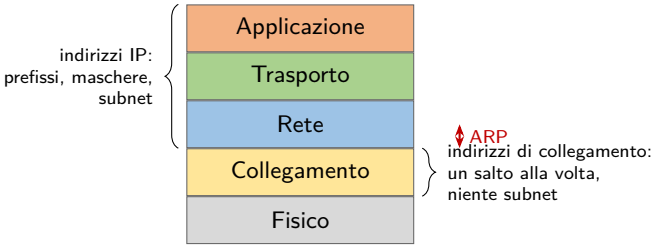


Figura 24.2 – Quale indirizzo appartiene a quale livello. Fra i due sta ARP: dato un indirizzo IP sulla propria subnet, produce l'indirizzo di collegamento da mettere sul frame. Lo smonteremo al livello di collegamento.

Il DHCP relay agent

- Nell'esempio tutto funziona perché il server DHCP e l'host sono nella **stessa subnet**: i broadcast non attraversano i router.
- Se il server esiste ma sta in un'altra subnet, serve un **DHCP relay agent** che inoltri i messaggi DHCP (può farlo un router).
- Configurando il router come relay agent, anche gli host delle subnet `223.1.1.0/24` e `223.1.3.0/24` vengono serviti dal server DHCP della subnet `223.1.2.0/24`. Il relay raccoglie il discover e lo inoltra al server come normale datagramma unicast, aggiungendo l'indirizzo dell'interfaccia da cui è arrivata la richiesta: così il server sa da quale pool prendere l'indirizzo.
- Più server possono anche rispondere allo stesso discover: il client prende un'offerta e la nomina nella richiesta; le altre vengono rilasciate.

Leggere che cosa ha dato il lease

```
$ ip addr show wlp9s0
2: wlp9s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP
    link/ether 34:e6:ad:f0:d5:11                        <- l'indirizzo del livello sotto
    inet 100.103.0.64/16 brd 100.103.255.255 scope global dynamic wlp9s0
       valid_lft 84532sec preferred_lft 84532sec
    inet6 fe80::47c4:ec38:2dc3:5d70/64 scope link
$ ip route
default via 100.103.0.1 dev wlp9s0 proto dhcp metric 600
100.103.0.0/16 dev wlp9s0 proto kernel scope link src 100.103.0.64
```

La configurazione ricevuta dal DHCP

- `dynamic` e `valid_lft` lo dicono chiaramente: questo indirizzo è in prestito, e il conto alla rovescia è partito.
- Nemmeno la rotta di default è stata scritta a mano: `proto dhcp` dice chi l'ha messa lì.
- Tre delle quattro cose che servono a un host sono su questo schermo; la quarta, il resolver, è finita nel file `/etc/resolv.conf`.

Un lease è temporaneo, e la connessione anche

- Un lease dura ore o giorni. Un client che vuole tenere il proprio indirizzo lo rinnova prima che il tempo scada; se non lo fa, l'indirizzo torna nel pool e viene offerto a qualcun altro.
- Se si sposta la macchina su un'altra subnet riceve un indirizzo diverso, che è l'unica cosa corretta da fare lì. Ma una connessione è nominata da quattro valori, e uno è appena cambiato: **ogni connessione aperta in quel momento è morta**.
- Niente a questo livello lo ripara. Tenere un indirizzo cambiando rete è un meccanismo separato, costruito per le reti cellulari.

Il problema della visibilità

Su Internet ci sono miliardi di dispositivi che dovrebbero avere un indirizzo IP univoco per essere raggiungibili da chiunque. Ma è davvero necessario che **tutti** i dispositivi siano visibili a tutti? È una buona idea che la mia stampante sia raggiungibile (e usabile) da chiunque su Internet? E una casa con quindici dispositivi non riceve quindici indirizzi pubblici.

Avere tutti i dispositivi raggiungibili dall'esterno delle reti locali pone problemi evidenti:

- gli indirizzi IP sono **finiti** (e gli IPv4 praticamente esauriti);
- se i dispositivi sono collegati localmente, spesso è irragionevole o indesiderabile esporli tutti su Internet;
- gli amministratori locali dovrebbero conoscere la struttura dei blocchi di indirizzi soprastanti per assegnare indirizzi liberi, cosa che spesso non dipende da loro (per esempio nelle reti di casa).

Network Address Translation (NAT)

Il servizio di **NAT** permette di **rimappare** gli indirizzi IP dei pacchetti di una rete in indirizzi diversi (*network masquerading*). Si usa una **tabella di traduzione** che associa molte coppie indirizzo/porta locali (della LAN) a una coppia indirizzo/porta globale (della WAN). Il servizio può essere offerto dai router.

La rete locale mascherata si chiama **rete privata** (o *realm* con indirizzi privati): gli indirizzi privati sono validi **solo all'interno** della rete privata. Gli stessi indirizzi sono in uso in questo momento in un milione di altre case, e due di loro non si incontreranno mai. Da fuori la scatola non è affatto un router: è una macchina con un indirizzo, da cui l'altro nome, *masquerading*.

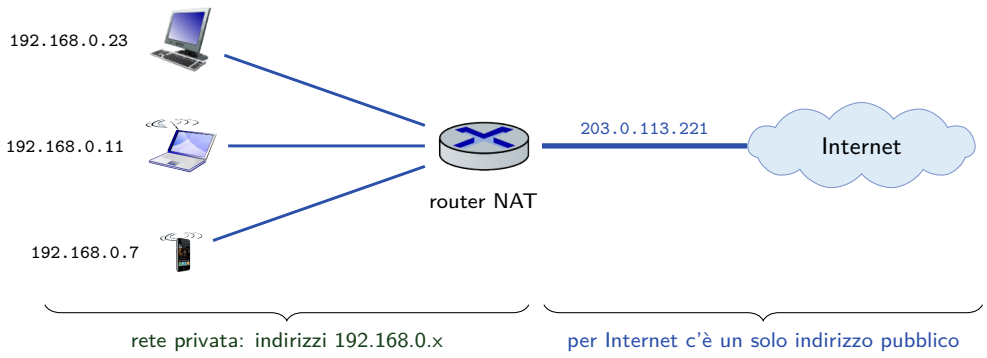


Figura 24.3 – Con il NAT tutti i dispositivi di casa escono verso Internet con lo stesso indirizzo pubblico del router.

Un esempio di NAT

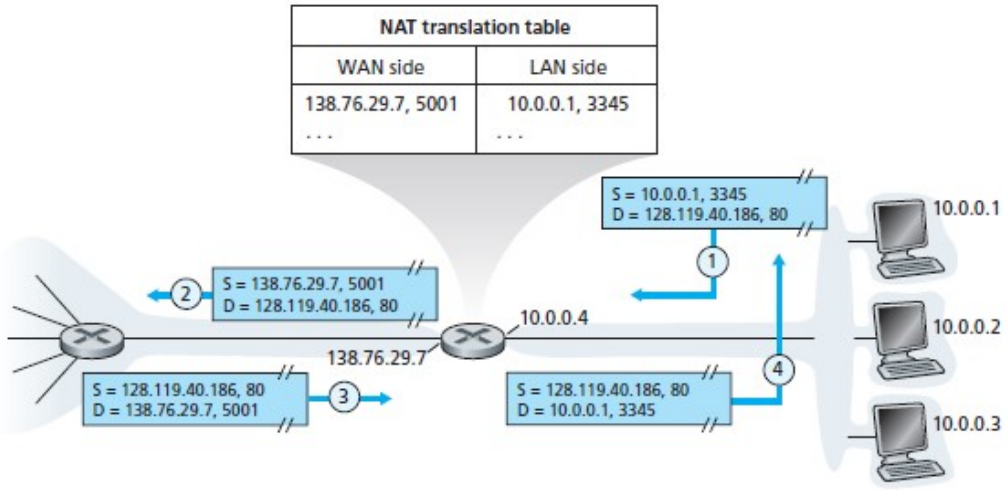


Figura 24.4 – Un router NAT tra una rete privata 10.0.0.0/24 e Internet (da Kurose).

Le quattro interfacce della rete locale hanno indirizzi della subnet `10.0.0.0/24`; lato Internet il router ha un solo indirizzo, `138.76.29.7`. Il router si comporta da "passamessaggi" che sovrascrive gli indirizzi secondo la tabella:

1. L'host `10.0.0.1` invia un datagramma verso il server Web `128.119.40.186`, porta 80, usando la porta sorgente 3345.
2. Il router NAT riceve il datagramma, sceglie una porta libera (**5001**), **sostituisce** indirizzo e porta sorgente con `138.76.29.7, 5001` e aggiunge la riga alla tabella di traduzione.
3. Il server risponde a `138.76.29.7, 5001`: per lui, la richiesta è arrivata dal router.
4. Il router riceve la risposta, cerca `138.76.29.7, 5001` nella tabella e **riscrive** destinazione e porta con `10.0.0.1, 3345`, poi la inoltra nella LAN.

Lato WAN	Lato LAN
138.76.29.7, 5001	10.0.0.1, 3345
138.76.29.7, 5002	10.0.0.2, 3345
…	…

Tabella 24.1 – La tabella di traduzione NAT: la seconda riga mostra perché serve una porta nuova.

Perché il NAT cambia anche la porta

Gli host non sanno nulla del traffico degli altri host: due host della LAN potrebbero scegliere **la stessa porta** sorgente (3345) nello stesso momento. Il NAT non può quindi riusare la porta iniziale, e ne assegna una nuova. Poiché la porta è di 16 bit, un NAT può gestire **oltre 60 000 connessioni** contemporanee con un solo indirizzo IP pubblico; ed è anche il tetto, perché a quel confine finisce lo spazio delle porte, non l'indirizzo. E si noti che cosa fa il router per arrivarci: legge e riscrive un campo del livello di trasporto. Il server Web non lo scopre mai: risponde all'indirizzo e alla porta da cui è stato interpellato.

Il limite del NAT

Nessuno può cominciare da fuori

Un host su Internet invia il primo datagramma a `10.0.0.1`. Che cosa ne fa il router? **Niente di utile**. Quella riga della tabella esiste solo perché la conversazione è stata avviata da dentro: senza riga, niente dice a quale delle macchine dietro il router sia destinato il datagramma.

- L'**inoltro delle porte** (*port forwarding*) è una riga scritta a mano: tutto ciò che arriva su questa porta va a quella macchina. È la risposta per un servizio.
- La risposta generale è una famiglia di tecniche detta **NAT traversal**, in cui una terza macchina fuori aiuta due macchine interne a trovarsi. Ogni applicazione peer-to-peer deve farci i conti.
- E anche il server della chat: sulla propria LAN risponde, da fuori non esiste.

Nemmeno il proprio indirizzo è nostro

`100.103.0.64`, dalla configurazione letta poco fa, è uno degli indirizzi privati? Non sta in nessuno dei tre blocchi, e non è nemmeno un nostro indirizzo pubblico: appartiene a `100.64.0.0/10`, lo **spazio di indirizzi condiviso**, che i fornitori usano per numerare i propri clienti dietro un traduttore loro (*carrier-grade NAT*).

- Allora le traduzioni sono due in fila, una in casa e una dal fornitore: niente fra quella macchina e Internet è suo.
- Il primo salto di una traccia da lì ha un indirizzo dello stesso blocco: un router del fornitore, non di Internet.
- Conseguenza: configurare il router di casa non basta per essere raggiungibili da fuori, perché il traduttore esterno non è nostro. È così che appare l'esaurimento degli indirizzi visto da un portatile.

Che cosa costa la traduzione

- **Le porte indirizzano processi**, non host: qui lo stesso campo serve a distinguere gli host, che è un lavoro diverso fatto con gli stessi bit.
- **I router sono dispositivi di livello di rete**, in teoria: questo legge e riscrive header di trasporto.
- E **tiene stato**: si perde la tabella e muore ogni conversazione che la attraversa. Un router che si limita a inoltrare non ha niente da perdere.
- L'argomento end-to-end dice che due host devono poter parlare senza che niente in mezzo alteri ciò che hanno detto; questo lo altera.
- E ha funzionato abbastanza bene da rimandare la vera risposta di venticinque anni: ci arriviamo alla fine del capitolo.

Gli indirizzi IP riservati

Perché gli amministratori locali possano organizzare una rete privata senza interferenze, per convenzione alcuni blocchi di indirizzi sono **riservati**: ISP e ICANN non possono assegnarli a nessuna organizzazione.

Blocco	Intervallo	Uso
10.0.0.0/8	10.0.0.0 – 10.255.255.255	reti private
172.16.0.0/12	172.16.0.0 – 172.31.255.255	reti private
192.168.0.0/16	192.168.0.0 – 192.168.255.255	reti private (tipiche delle case)
127.0.0.0/8	127.0.0.0 – 127.255.255.255	"questa macchina" (*loopback*), di solito 127.0.0.1
100.64.0.0/10	100.64.0.0 – 100.127.255.255	spazio condiviso dei fornitori (*carrier-grade NAT*)
0.0.0.0	—	rete corrente; per estensione, un indirizzo con la parte di host a 0 indica la rete
255.255.255.255	—	broadcast; per estensione, un indirizzo con la parte di host tutta a 1 è il broadcast della subnet

Tabella 24.2 – Indirizzi IPv4 riservati.

La riserva è necessaria perché gli indirizzi privati potrebbero coincidere con indirizzi pubblici, rendendo ambiguo l'instradamento dentro la rete. Nessun registro li assegna mai: sono tenuti da parte proprio perché ogni rete privata possa usarli nello stesso momento. Un router di Internet pubblica scarta un datagramma diretto a uno di loro: là fuori non c'è un posto simile a cui mandarlo. Per sapere chi è acceso in una subnet privata si usano `ping` e `nmap -sn`, visti nel capitolo precedente.

Esempio completo: configurare una LAN

Vogliamo creare una nuova rete locale (LAN) dentro una rete più grande preesistente (WAN). L'amministratore della WAN ci fornisce:

- l'indirizzo della WAN: `150.100.0.0/16`;
- l'indirizzo del gateway che porta all'esterno (WAN/Internet): `150.100.50.1`;
- un indirizzo libero per la nostra rete: `150.100.50.10`.

Useremo un router con NAT e decidiamo che la nuova LAN avrà indirizzo `192.168.1.0/24` (riservato alle reti private).

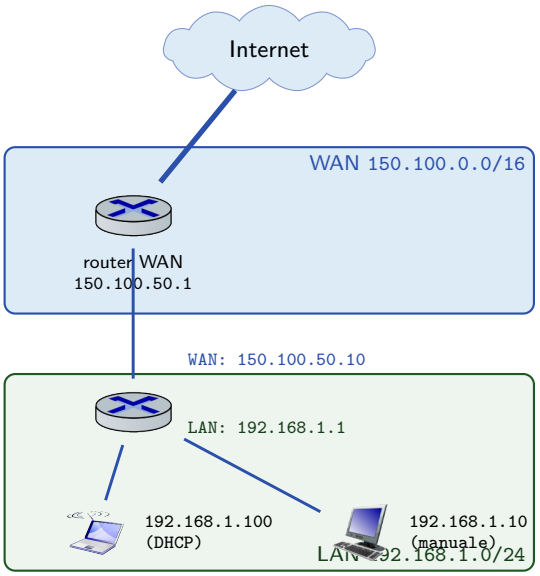


Figura 24.5 – La LAN da configurare: il router locale ha un'interfaccia nella WAN e una nella LAN.

1. Il lato WAN del router

Il router locale ha **due interfacce**, una verso la WAN e una verso la LAN. Sul lato WAN vanno impostati:

- l'indirizzo del router nella WAN: `150.100.50.10`;
- la subnet mask della WAN: `255.255.0.0`;
- il gateway: `150.100.50.1`;
- uno o più server DNS.

Vista dalla rete in cui entra, il router è solo un altro host: un'interfaccia, un indirizzo, e i quattro campi sono le quattro cose di cui ogni host ha bisogno. È **statico** perché l'amministratore ci ha dato un indirizzo fisso; se la WAN avesse un server DHCP, questa scatola sarebbe un client da un lato e un server dall'altro.

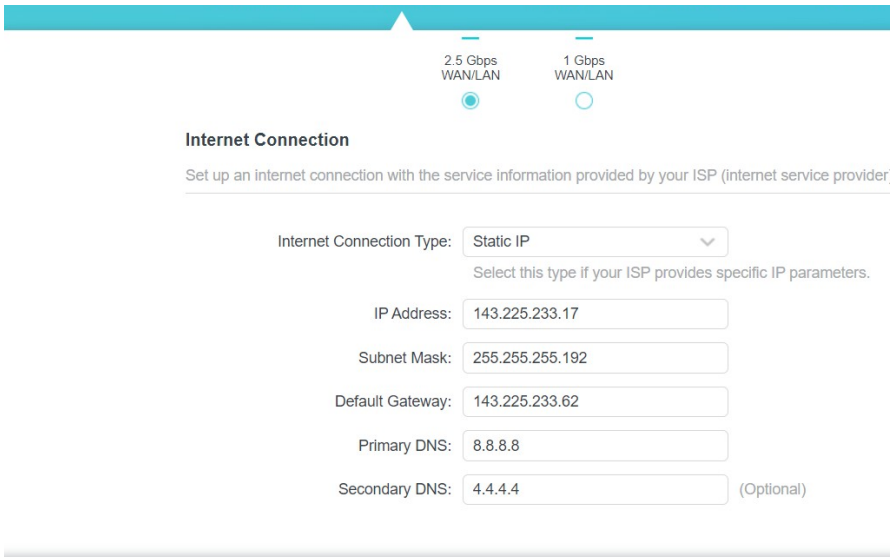


Figura 24.6 – La pagina di configurazione "Internet" (lato WAN) di un router domestico: indirizzo statico, maschera, gateway e DNS (valori di un esempio reale, diversi da quelli del testo).

2. Il lato LAN del router

- All'interfaccia LAN del router si dà un indirizzo della nuova rete: poiché la LAN è `192.168.1.0/24`, al router si assegna `192.168.1.1/24` (è consuetudine dare al router l'indirizzo `.1`).
- Se il router offre anche il DHCP, si specificano:
  - il **pool di indirizzi** per gli host DHCP, per esempio da `192.168.1.100` a `192.168.1.250`;
  - il **lease time**: la durata dell'assegnazione, dopo la quale l'indirizzo può essere riusato (per esempio 120 minuti: lease brevi riciclano gli indirizzi in fretta, lunghi sopravvivono a una macchina spenta);
  - gateway e DNS da comunicare agli host: e il gateway che annuncia è se stesso.

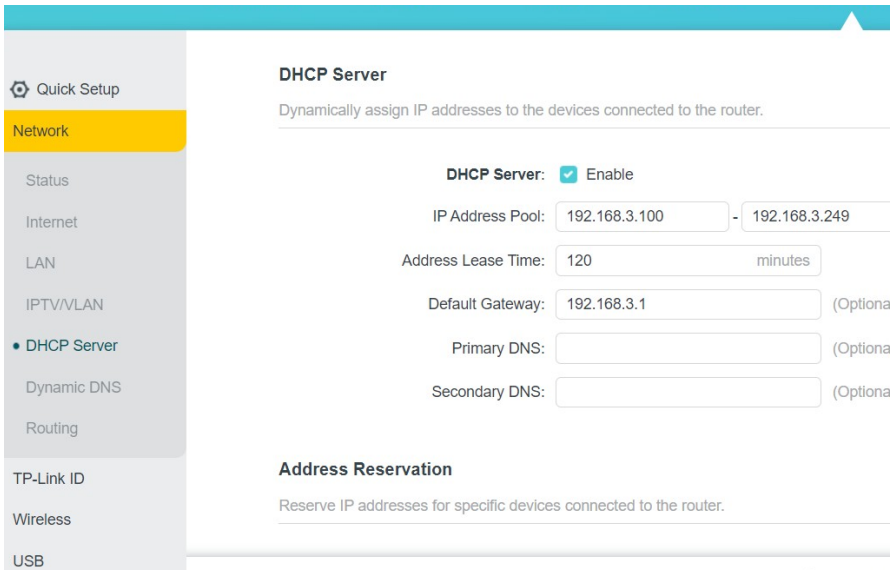
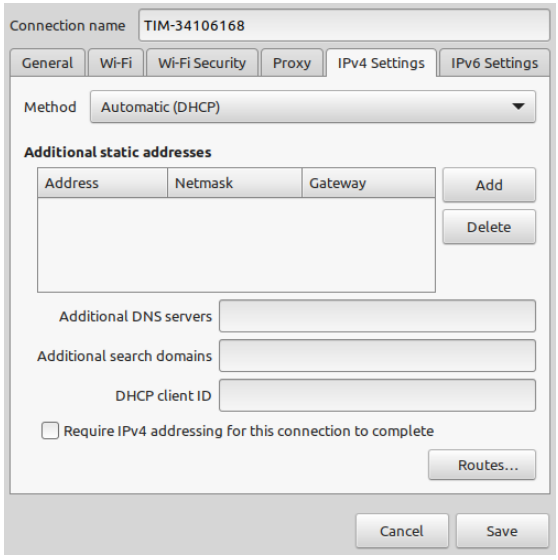


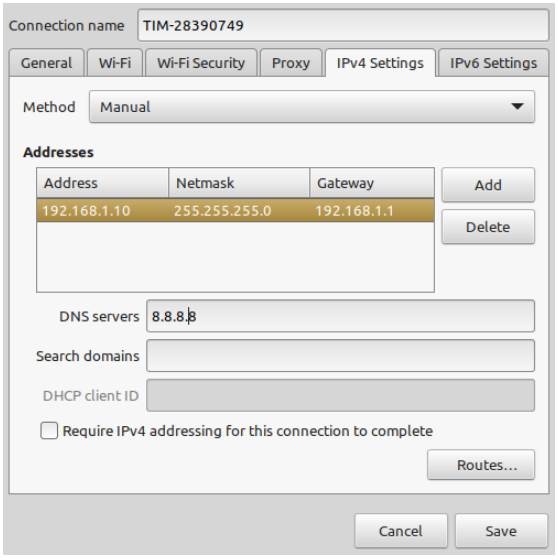
Figura 24.7 – La configurazione del server DHCP del router: pool di indirizzi, lease time, gateway e DNS.

3. Gli host

Su un nuovo host (per esempio con Linux Mint) si può scegliere come configurare la connessione:



(a) automatica (DHCP)



(b) manuale

Figura 24.8 – Configurazione della connessione su un host.

- **Automatica (DHCP)**: l'host riceve dal DHCP un indirizzo tra `192.168.1.100` e `192.168.1.250`.
- **Manuale**: bisogna conoscere la configurazione della rete e gli indirizzi disponibili, e indicare l'**indirizzo dell'host** (statico, **fuori dal pool** del DHCP, per esempio `192.168.1.10`), la **subnet mask**, il **gateway** e uno o più **DNS**.

Gli host collegati inviano i loro pacchetti al router locale, senza sapere nulla della WAN esterna; il router locale è configurato per inoltrarli al router della WAN, e da lì eventualmente a Internet.

Indirizzi manuali fuori dal pool

Un indirizzo assegnato a mano deve stare **fuori** dall'intervallo del DHCP: `.10` sta sotto il pool, ed è voluto. Altrimenti prima o poi il DHCP lo offrirà a una macchina appena arrivata, e due interfacce della stessa subnet risponderanno allo stesso indirizzo (conflitto). Dopo la configurazione, niente distingue i due host: uno ha chiesto, l'altro è stato detto da qualcuno che sapeva. Tutto ciò che l'host invia va a `192.168.1.1`, che lo traduce e lo inoltra a `150.100.50.1`; l'host non sa nulla né dell'uno né dell'altro.

IPv6

All'inizio degli anni '90 l'IETF (organizzazione nata nel 1986) avviò lo sviluppo di un successore di IPv4, per affrontare la **carenza di indirizzi** a 32 bit. IPv6 fu progettato per avere molti più indirizzi, ma fu anche l'occasione per aggiornare altri aspetti di IPv4 sulla base dell'esperienza accumulata.

- Due stime di chi lo stava misurando: gli indirizzi sarebbero finiti nel 2008, o nel 2018.
- **Febbraio 2011**: l'ultimo blocco non assegnato ha lasciato il pool centrale verso un registro regionale. Da allora i registri razionano.
- La traduzione è gran parte del motivo per cui la scadenza continuava a spostarsi: non ha creato indirizzi, ha fatto servire più macchine da ciascuno.

E IPv5?

Il numero di versione 5 è stato speso per un protocollo sperimentale di streaming, *Internet Stream Protocol* (ST, poi ST-2), che è stato abbandonato e non è mai stato pensato come successore di IPv4. Al successore è toccato il 6.

I vantaggi di IPv6

- **Indirizzi molto più grandi**: da 32 a **128 bit**, cioè $2^{128} \approx 3{,}4 \cdot 10^{38}$ indirizzi. Il mondo non resterà senza indirizzi: si potrebbe dare un indirizzo a ogni granello di sabbia del pianeta.
- **Header di lunghezza fissa**, 40 byte: alcuni campi di IPv4 sono stati eliminati o resi facoltativi, e i router elaborano i pacchetti più velocemente.
- **Etichettatura dei flussi** (*flow labeling*): 20 bit che nominano un flusso di datagrammi che vogliono lo stesso trattamento (un video, per esempio, piuttosto che un trasferimento di posta). Che cosa sia esattamente un flusso è stato lasciato aperto dalla specifica, ed è rimasto aperto: il campo c'è per quando sarà deciso.
- **Anycast**: un tipo di indirizzo del tutto nuovo, «consegna questo a uno qualsiasi di un gruppo»: il mirror più vicino, il resolver più vicino.

Con $3{,}4 \cdot 10^{38}$ indirizzi nessuno progetterà mai più uno schema per risparmiarli.

Come si scrive un indirizzo IPv6

Sedici byte in decimale puntato sarebbero illeggibili: un indirizzo IPv6 si scrive come **8 gruppi di 4 cifre esadecimali** separati da due punti, per esempio `8000:0000:0000:0000:0123:4567:89AB:CDEF`.

- Gli **zeri iniziali** di un gruppo si possono omettere: `0123` diventa `123`.
- Una sequenza di gruppi tutti a zero si può sostituire con `::`, **una sola volta** per indirizzo: due volte e la lunghezza sarebbe ambigua. L'esempio diventa `8000::123:4567:89AB:CDEF`.
- Un indirizzo IPv4 si può scrivere dopo una coppia di due punti, come in `::192.31.20.46`.
- L'indirizzo di loopback è `::1`; la riga `fe80::` nella configurazione letta prima è un indirizzo di questo tipo (*link-local*): quella macchina aveva un indirizzo IPv6 da sempre.

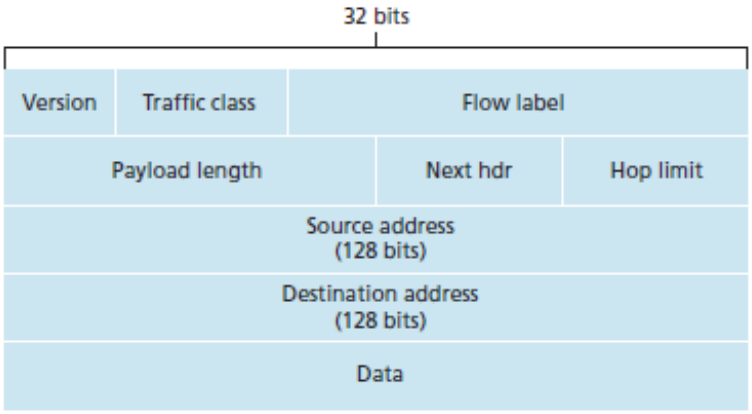


Figura 24.9 – Il datagramma IPv6: header fisso di 40 byte.

- **Version** (4 bit): la versione, cioè 6.
- **Traffic class** (8 bit): come il TOS di IPv4, serve a dare priorità ai datagrammi (per esempio voce su IP rispetto a SMTP).
- **Flow label** (20 bit): identifica un flusso di datagrammi.
- **Payload length** (16 bit): numero di byte del payload (esclusi i 40 byte di header).
- **Next header** (8 bit): il protocollo superiore a cui consegnare i dati (TCP, UDP, …), come il campo protocollo di IPv4.
- **Hop limit** (8 bit): il time-to-live, decrementato a ogni inoltro.
- **Indirizzi sorgente e destinazione** (128 + 128 bit).
- **Data**: il payload.

Nella prima parola versione, classe di traffico e flow label (quale protocollo, quale priorità, quale flusso); nella seconda lunghezza del payload, next header e hop limit (quanto segue, che cos'è, quanto lontano può ancora andare); poi i due indirizzi, sedici byte ciascuno: trentadue dei quaranta byte sono indirizzi, e nessun campo misura un header di dimensione fissa.

IPv4	IPv6	
time to live	hop limit	lo stesso contatore, chiamato onestamente
protocol	next header	gli stessi valori, e un compito in più
type of service	traffic class	lo stesso compito
total length	payload length	non conta più l'header

Tabella 24.3 – Stesso compito, nome nuovo. Il cambio non è cosmetico: il time to live era specificato in secondi e usato da tutti come conteggio dei salti, e il nuovo nome dice che cos'è. *Next header* nomina il protocollo di trasporto oppure punta al primo di una catena di header facoltativi; *payload length* conta ciò che segue l'header, quindi un payload può essere di 65 535 byte pieni.

Cosa manca rispetto a IPv4

- **Campi della frammentazione**: IPv6 non permette frammentazione e riassemblaggio nei router intermedi, ma solo negli host. Se un router riceve un datagramma IPv6 troppo grande da inoltrare, lo **scarta** e rimanda al mittente un messaggio di errore "*Packet Too Big*"; il mittente rinvia i dati con datagrammi più piccoli.
- **Checksum dell'header**: richiede tempo in ogni router ed è ridondante, perché i protocolli di collegamento di solito controllano già l'intero pacchetto (e TCP/UDP controllano i propri dati).
- **Opzioni**: non fanno più parte dell'header standard; alcune si possono indicare nel campo *next header* (accanto agli identificativi di TCP/UDP), come header di estensione, dove un router a cui non interessano li salta.

Tutto ciò che è stato tolto è stato tolto per una sola ragione: era lavoro fatto a ogni salto, da ogni router, su ogni datagramma.

Dalla IPv4 alla IPv6

C'è un grosso problema: IPv6 **non è retrocompatibile**. I sistemi che capiscono solo IPv4 non sanno gestire i datagrammi IPv6. Come può Internet adottare IPv6?

- **Flag day**: a una certa data e ora tutte le macchine di Internet vengono spente e aggiornate da IPv4 a IPv6. Si fece una volta, per il passaggio a TCP, su una rete di qualche centinaio di macchine, e fu doloroso già allora: con miliardi di dispositivi e nessuno che li governi tutti, l'idea non è nemmeno discutibile.
- **Doppio stack**: ciò che si fa invece è far convivere i due protocolli. La maggior parte delle macchine oggi li esegue entrambi e sceglie per destinazione, ed è per questo che la configurazione letta sopra aveva un indirizzo di ciascun tipo.
- **Tunneling**: dove un tratto del percorso parla solo IPv4, i datagrammi vanno trasportati attraverso in qualche modo. Due router IPv6 con solo IPv4 in mezzo: l'intero datagramma IPv6 diventa il payload di un datagramma IPv4 indirizzato all'altro capo.

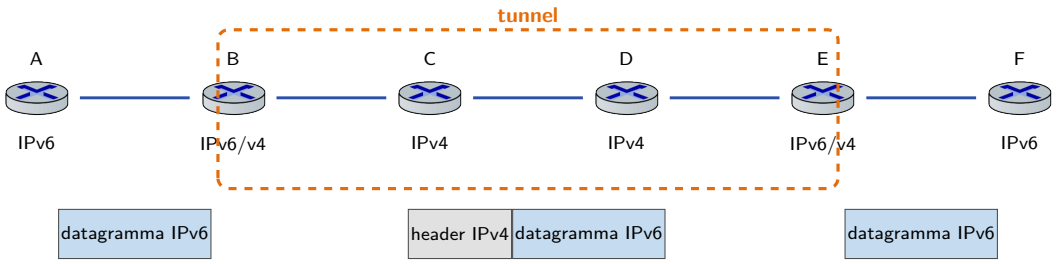


Figura 24.10 – Tunneling: B incapsula il datagramma IPv6 in un datagramma IPv4 per attraversare C e D; E lo estrae. I router in mezzo inoltrano un normale datagramma IPv4 e non scoprono mai che cosa trasportano; il numero **41** nel campo protocollo dell'header IPv4 dice all'altro capo che cosa c'è dentro, esattamente come 6 dice TCP e 17 UDP. Niente viene tradotto: ciò che esce è ciò che è entrato.

Perché questo livello è quello difficile da cambiare

- L'adozione è stata lenta per un decennio, poi ha cominciato a muoversi: nel 2020 circa un quarto dei client che raggiungevano Google lo faceva su IPv6. Le reti mobili hanno spinto più di tutti: un operatore che numera milioni di telefoni non ha altre opzioni.
- La lezione generale: il livello di rete è l'unica cosa su cui ogni macchina di Internet deve essere d'accordo, e l'accordo è la parte costosa. Il livello applicativo cambia di continuo (HTTP è stato riscritto due volte nel frattempo) perché ogni volta devono essere d'accordo solo due estremi.
- Venticinque anni, per un protocollo il cui unico cambiamento serio è la larghezza di un indirizzo.

Riepilogo

In breve

- Il **DHCP** assegna automaticamente indirizzo, maschera, gateway e DNS con quattro messaggi: discover, offer, request, ACK (broadcast, porte UDP 67/68); oltre le subnet serve un relay agent. Tutto arriva in prestito (lease), e cambiando subnet le connessioni aperte muoiono con il vecchio indirizzo.
- Il **NAT** fa uscire una rete privata con un solo indirizzo pubblico, riscrivendo indirizzi e porte secondo una tabella; funziona solo per conversazioni avviate da dentro (le altre richiedono port forwarding o NAT traversal), mette stato in mezzo alla rete, ed è ciò che ha fatto durare IPv4 così a lungo.
- Blocchi **riservati**: 10/8, 172.16/12, 192.168/16 (privati), 127/8 (loopback), 0.0.0.0, 255.255.255.255.
- **IPv6**: indirizzi a 128 bit, header fisso di 40 byte, flow label; niente frammentazione nei router, niente checksum, niente opzioni nell'header; transizione con doppio stack e **tunneling** (protocollo 41), senza una data di fine.
- Resta da chiedersi da dove vengono le righe di una forwarding table: è la prossima domanda.

Il piano di controllo: routing, sistemi autonomi e BGP

Introduzione al routing

Per quattro capitoli la forwarding table è stata data: una riga è un prefisso, la riga nomina un'interfaccia di uscita, vince la corrispondenza più lunga. Nessuno ha detto **chi scrive le righe**. È il piano di controllo, e ne abbiamo già nominato le due organizzazioni possibili: un algoritmo che gira in ogni router e le cui copie si parlano, oppure un controller che calcola tutte le tabelle e le distribuisce. Qui apriamo la scatola: che cosa calcola un tale algoritmo, e come router che vedono solo il proprio vicinato finiscono per accordarsi su percorsi da un capo all'altro. Due famiglie di algoritmi, entrambe in uso, e poi la ragione per cui nessuna delle due gira su Internet nel suo insieme.

Ricordiamo che, alla ricezione di un pacchetto, un router può:

- inoltrarlo così com'è a un vicino adatto;
- inoltrarne una versione modificata o sostituita (frammentazione, NAT, …);
- scartarlo (per esempio se è scaduto).

Ci sono router molto semplici, usati dagli amministratori locali per collegare LAN e WAN, e router usati dagli ISP per far arrivare i pacchetti al dispositivo giusto in tutta Internet. I router hanno forwarding table che stabiliscono la strategia di inoltro **locale** (nel vicinato del router), create in modo centralizzato o decentralizzato. Ma all'avvio un router **non sa nulla** del resto della rete.

Algoritmo di routing

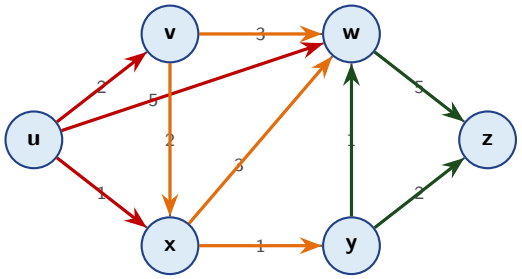
Per decidere dove inoltrare i pacchetti, i router usano **algoritmi di routing**, che determinano **percorsi buoni** (sequenze di router) dai mittenti ai destinatari. Di solito un percorso "buono" è quello a **costo minimo** (il più corto, il più veloce, …); nel mondo reale contano anche questioni di *policy* (regole che dicono, per esempio, se un'organizzazione può scambiare traffico con un'altra).

Il problema di trovare percorsi buoni su una rete (o su un grafo) è cruciale per le reti di calcolatori, ma è rilevantissimo anche altrove: videogiochi, guida autonoma, robotica, navigatori. Ciò che è nostro è il vincolo: **nessun nodo ha la mappa**, e la mappa cambia mentre ci si calcola sopra.

Il flooding

Flooding

Nell'approccio più semplice, il **flooding** (inondazione), ogni router inoltra ogni pacchetto in arrivo su **tutti** i collegamenti tranne quello da cui è arrivato.



1° giro, 2° giro, 3° giro: il pacchetto si moltiplica e raggiunge tutti, esplorando ogni cammino.

Figura 25.1 – Flooding a partire da u.

- Nessuna tabella, nessun calcolo, nessun accordo fra router. Tutti i cammini vengono esplorati, compreso quello ottimo: se un cammino esiste, il pacchetto **arriverà**.
- I pacchetti che non vanno da nessuna parte raggiungono prima o poi il limite di hop e vengono scartati: lo stesso campo che fa scadere una traccia un router più in là ogni volta.
- È **robustissimo** grazie alla ridondanza: se alcuni router sono spenti, i cammini alternativi vengono comunque usati.

Il flooding si usa in situazioni particolari in cui serve ridondanza (diffusione di messaggi a tutti, scenari militari), ma è impraticabile in reti grandi e ad alte prestazioni come Internet, dove ogni pacchetto diventa molti, e i molti diventano di più: serve un algoritmo che decida **un** collegamento di uscita.

Formalizzazione del problema

Grafo e costi

La rete si rappresenta come un **grafo** $G = (N, E)$ dove:
- N è l'insieme dei **nodi** (i router);
- $E \subseteq N \times N$ è l'insieme degli **archi** (i collegamenti fisici), coppie di nodi.

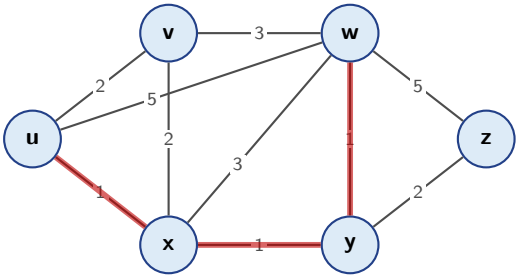
Ogni arco ha un **costo**, che può riflettere la lunghezza fisica del collegamento, la sua velocità o il suo costo economico. Formalmente una funzione c tale che, per ogni coppia di nodi (x, y):
- se $(x, y) \in E$, $c(x, y)$ è il costo dell'arco;
- se $(x, y) \notin E$, $c(x, y) = \infty$;
- il grafo è **non orientato**: se $(x, y) \in E$ allora anche $(y, x) \in E$ e $c(x, y) = c(y, x)$.

Cammino e cammino minimo

Un **cammino** è una sequenza di nodi $(x_1, x_2, \ldots, x_p)$ tale che nodi adiacenti sono collegati: $(x_1, x_2), \ldots, (x_{p-1}, x_p) \in E$. Il suo **costo** è la somma dei costi degli archi:

$$c(\pi) = \sum_{i=1}^{p-1} c(x_i, x_{i+1})$$

Tra tutti i cammini che collegano due nodi u e z, il **cammino migliore** (a costo minimo) è quello di costo minimo.



Un grafo di 6 nodi (da Kurose), usato in tutto il capitolo.

Esercizio: qual è il cammino migliore tra u e w?

Soluzione: u–x–y–w, di costo 1 + 1 + 1 = 3 (il collegamento diretto u–w costa 5, e u–v–w costa 5).

Figura 25.2 – Il grafo di esempio; evidenziato il cammino minimo da u a w.

- Gli algoritmi che trovano un cammino ottimo tra due nodi si chiamano algoritmi **di cammino a costo minimo** (*least-cost path*; *shortest path* se tutti gli archi hanno lo stesso costo).
- Nelle reti interessa trovare il cammino ottimo non per una coppia, ma per **tutte** le coppie di nodi.
- Un algoritmo di routing **converge** quando ha trovato il cammino minimo per tutte le coppie di nodi.
- Da dove vengono i costi è un problema di qualcun altro: per noi sono dati. Gli algoritmi si estendono a collegamenti orientati con un costo diverso per verso.

Nelle reti si usano tipicamente due famiglie di algoritmi, che differiscono in ciò che un nodo **sa**, non in ciò che calcola:

- **globali**: all'algoritmo si danno tutta la topologia e tutti i costi prima che cominci. Poiché gli serve lo stato di ogni collegamento, sono gli algoritmi **link state**;
- **decentralizzati**: un nodo parte conoscendo solo il costo dei propri collegamenti, e ottiene tutto il resto parlando con i vicini, più e più volte. Sono gli algoritmi **distance vector**.

Attenzione: «globale» non significa calcolato in un solo posto; un algoritmo link state di solito gira in ogni router, ciascuno con la stessa mappa.

L'algoritmo Distance Vector

Distance Vector (DV)

L'algoritmo **Distance Vector** è un approccio **decentralizzato**: sfrutta la conoscenza **locale** della rete combinata con la comunicazione tra nodi. Si basa sull'algoritmo di **Bellman-Ford**. Ogni nodo riceve informazioni da uno o più vicini diretti, fa un calcolo, e ridistribuisce i risultati ai vicini. È **iterativo** (lo scambio continua finché non c'è più niente di nuovo da dire, e si ferma da solo: nessun segnale annuncia la fine) e **asincrono**: non richiede che tutti i nodi siano coordinati, e agisce quando le notizie gli arrivano, il che lo rende usabile su una rete reale dove i router si riavviano.

L'equazione di Bellman

Sia $d_x(y)$ il costo del cammino migliore dal nodo x al nodo y. Dall'equazione di Bellman (Bellman-Ford):

Equazione di Bellman-Ford

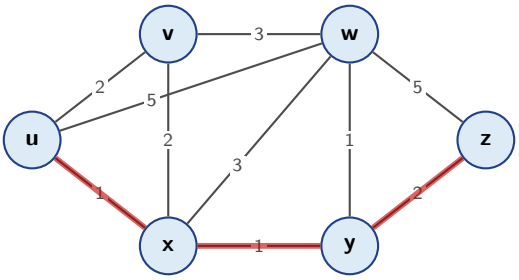
$$d_x(y) = \min_v \{\, c(x,v) + d_v(y) \,\}$$

dove il minimo è preso su tutti i **vicini** v di x.

È intuitiva: sul cammino migliore da x a y deve esserci un vicino v di x, e il costo di quel cammino è il costo per raggiungere v più il costo del cammino migliore da v a y.

- Se troviamo questo v, basta scriverlo nella forwarding table: tutti i pacchetti diretti a y vanno inoltrati a v. La parte utile non è il numero, è il **vicino vincente** v^* che realizza il minimo: il nodo non memorizza mai il cammino, solo il primo salto.
- Per farlo, x deve avere una stima di $d_v(y)$ per tutti i suoi vicini: il **distance vector**, esattamente ciò che i vicini continuano a inviargli.
- Nell'equazione niente riguarda la topologia oltre x: è tutto il punto, e più avanti sarà anche tutto il problema.

Esempio 1 — L'equazione sul grafo di esempio



Il cammino migliore da u a z è u→x→y→z. Per tutti i nodi lungo questo cammino l'equazione di Bellman deve valere. Il nodo u ha tre vicini, w, v e x:

- per x: $c(u,x) = 1$ e $d_x(z) = 3$, quindi $1 + 3 = 4$;
- per v: $c(u,v) = 2$ e $d_v(z) = 5$, quindi $2 + 5 = 7$;
- per w: $c(u,w) = 5$ e $d_w(z) = 3$, quindi $5 + 3 = 8$.

$$d_u(z) = \min\{1 + 3,\ 2 + 5,\ 5 + 3\} = 4$$

Il nodo **x** minimizza l'espressione: è il prossimo nodo del cammino ottimo, e nella forwarding table di u la destinazione z va associata al collegamento verso x. Ripetendo il ragionamento su x si vede che y minimizza la sua espressione, e così via.

Lo pseudocodice

```
DistanceVector(x):
    // inizializzazione
    for all nodes v:
        if v is a neighbor of x then  D_x(v) = c(x,v)
        else                          D_x(v) = ∞
    for all neighbors w and destinations y:
        D_w(y) = ?                    // ancora sconosciuti
    send initial D_x(·) to all neighbors

    // fase online
    repeat
        wait for (a link cost change) or (an update D_w(·) from a neighbor w)
        for all nodes y:
            D_x(y) = min_v { c(x,v) + D_v(y) }  // equazione di Bellman
        if D_x(y) changed for any destination y then
            send updated D_x(·) to all neighbors
    until false   // per sempre
```

Distance Vector, eseguito da ogni nodo x

- La struttura $D_w(v)$ conserva i distance vector dei nodi w verso le destinazioni v.
- **Inizializzazione**: sono note solo le distanze verso i vicini.
- **Fase online**: si ricevono le novità dai vicini, si aggiornano i distance vector con l'equazione di Bellman, e i cambiamenti locali vengono comunicati ai nodi adiacenti. Niente è periodico: un nodo aspetta, e parla solo quando uno dei suoi numeri si è mosso. Quando nessuno parla, la rete è convergente.

Esempio 2 — Come si propaga una buona notizia

Seguiamo il cammino da u a z:

1. All'avvio u conosce solo i suoi vicini: il costo verso z è infinito, $D_u(z) = \infty$.
2. Il nodo y si "sveglia" e scopre un cammino verso z: il collegamento diretto y–z di costo 2, $D_y(z) = c(y, z) = 2$. Comunica la buona notizia a x.
3. Il nodo x scopre che il cammino migliore verso z passa per y, con costo $D_x(z) = c(x, y) + D_y(z) = 1 + 2 = 3$, e inoltra la notizia a u.
4. Il nodo u riceve la notizia: ora esiste un cammino verso z di costo finito, attraverso x: $D_u(z) = c(u, x) + D_x(z) = 1 + 3 = 4$.

Lo stesso processo avviene per tutti i nodi e tutti i cammini. Nessuno ha calcolato il cammino u, x, y, z: ogni nodo ha aggiunto il costo di un collegamento a un numero che gli era stato detto; e u ha lasciato il proprio collegamento diretto verso w, che costa 5, per un cammino attraverso x che costa 3.

Esempio 3 — Le tabelle di un nodo (dalla lavagna)

Il nodo x mantiene il proprio vettore e quelli ricevuti dai vicini. Quando riceve da y il vettore aggiornato, ricalcola ogni destinazione: per esempio $D_x(z) = \min\{c(x, y) + D_y(z),\ c(x, w) + D_w(z),\ \dots\}$. Poi u, ricevuto il nuovo vettore di x, ricalcola $D_u(w) = c(u, x) + D_x(w) = 1 + 2 = 3$ e $D_u(z) = c(u, x) + D_x(z) = 1 + 3 = 4$.

vettore di u dopo la convergenza	u	v	w	x	y	z
costo	0	2	3	1	2	4
prossimo hop	—	v	x	x	x	x

Complessità, pro e contro

Il traffico generato è piccolo e locale: un nodo parla solo ai vicini, e solo quando uno dei suoi cammini minimi è davvero cambiato. Non c'è un numero fisso di giri: quanto tempo serve alla convergenza dipende dalla topologia e da dove è avvenuto il cambiamento, perché le notizie devono camminare, un salto per scambio. Nel caso peggiore la complessità è **quadratica**.

- ✔ **Asincrono**: i router possono adattarsi ai costi aggiornati in qualsiasi momento, senza coordinarsi.
- ✖ **Convergenza lenta**: gli aggiornamenti si propagano lentamente, perché ogni nodo deve accorgersi del cambiamento e avvisare i vicini.
- ✖ **Count-to-infinity**: le buone notizie viaggiano veloci e quelle cattive lente. Un cammino più economico viene adottato in un paio di scambi; un collegamento che peggiora richiede molto di più, e nel frattempo ci possono essere cicli.

Il problema del count-to-infinity

Il problema nasce proprio dalla natura "locale" di DV, che è anche la sua forza: il vettore locale $D_x(y)$ si calcola a partire dai vettori degli altri nodi $D_v(y)$, ma non c'è alcuna indicazione su **quali nodi** stanno sui cammini scelti dai vicini. Posso essere sicuro di non essere io stesso sul "cammino migliore" dei miei vicini?

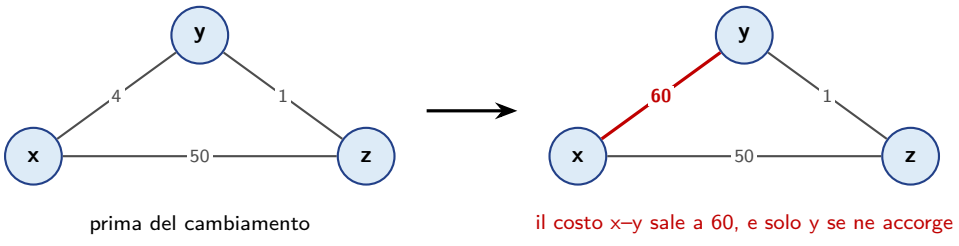


Figura 25.3 – Lo scenario del count-to-infinity.

Situazione iniziale ($c(x,y) = 4$): $D_y(x) = 4$, $D_y(z) = 1$, $D_z(y) = 1$, $D_z(x) = 1 + D_y(x) = 5$ (z raggiunge x passando per y), $D_x(y) = 4$, $D_x(z) = 4 + D_y(z) = 5$.

Ora il costo di x–y sale a 60 e **solo y** se ne accorge:

Passo	**Chi aggiorna**	$D_y(x)$	$D_z(x)$
0	situazione iniziale	4	5
1	y: $\min\{60,\ 1 + D_z(x)\} = 1 + 5$	**6**	5
2	z: $\min\{50,\ 1 + D_y(x)\} = 1 + 6$	6	**7**
3	y: $\min\{60,\ 1 + D_z(x)\} = 1 + 7$	**8**	7
4	z: $\min\{50,\ 1 + 8\}$	8	**9**
…	…	…	…
	y: $\min\{60,\ 1 + 49\}$	**50**	49
	z: $\min\{50,\ 1 + 50\} = 50$ (diretto)	50	**50**
	y: $\min\{60,\ 1 + 50\}$	**51**	50

Tabella 25.1 – y crede di poter raggiungere x passando per z, ma z raggiungeva x passando per y: i due vettori si rincorrono.

1. y rileva l'aumento e aggiorna il proprio vettore verso x: $D_y(x) = \min\{60,\ 1 + D_z(x)\} = 6$. Ma il valore $D_z(x) = 5$ che sta usando era calcolato **passando per y stesso**, sul vecchio collegamento da 4!
2. Il vettore $D_y(x)$ è usato da z per calcolare il cammino verso x, quindi anche $D_z(x)$ va aggiornato: 7.
3. Ma $D_z(x)$ è a sua volta usato da y, che deve aggiornarsi di nuovo: 8. E così via.

È un **ciclo**: i due vettori si aggiornano continuamente, crescendo di poco a ogni scambio, finché non si raggiunge una configurazione stabile (qui dopo più di 40 scambi, quando il cammino diretto z–x da 50 diventa conveniente). Nel frattempo i pacchetti per x rimbalzano tra y e z. Se il collegamento si interrompesse del tutto ($c = \infty$), il conteggio salirebbe davvero verso l'infinito.

Come si sarebbe evitato

Il problema non ci sarebbe stato se anche x si fosse accorto del cambiamento e lo avesse comunicato. Se x è malfunzionante o in ritardo (e i costi alti dei suoi collegamenti potrebbero proprio indicarlo), la convergenza richiede tempo. Se invece anche x rileva il cambiamento, i costi convergono subito:

$$
\begin{aligned}
D_y(x) &= \min\{c(y,x),\ c(y,z) + D_z(x)\} = c(y,z) + D_z(x) = 1 + 50 = 51\\
D_y(z) &= \min\{c(y,z),\ c(y,x) + D_x(z)\} = c(y,z) = 1\\
D_x(y) &= \min\{c(x,y),\ c(x,z) + D_z(y)\} = c(x,z) + D_z(y) = 50 + 1 = 51\\
D_x(z) &= \min\{c(x,z),\ c(x,y) + D_y(z)\} = c(x,z) = 50\\
D_z(x) &= \min\{c(z,x),\ c(z,y) + D_y(x)\} = c(z,x) = 50\\
D_z(y) &= \min\{c(z,y),\ c(z,x) + D_x(y)\} = c(z,y) = 1
\end{aligned}
$$

Poisoned reverse

La difesa standard è una bugia. Con il **poisoned reverse** (inversione avvelenata), se z raggiunge x passando per y, allora z dice a y che la propria distanza da x è **infinita**, anche se sa benissimo che è 5. Finché y crede che z non abbia un cammino verso x, y non proverà mai a raggiungere x attraverso z, che è proprio la mossa che ha fatto partire il conteggio. Così, quando il collegamento sale a 60, y tiene il collegamento diretto a 60 e lo dice; z passa al proprio collegamento da 50 e ora dice la verità; y si assesta a 51. Tre scambi invece di quarantaquattro. Non è però una cura: un ciclo di tre o più nodi non viene rilevato in questo modo, e il problema generale resta.

L'algoritmo Link State

Link State (LS)

L'algoritmo **Link State** è un approccio **globale** (nel senso della conoscenza): ogni nodo conosce tutto il grafo, tutti i nodi, tutti i collegamenti, tutti i costi, prima di cominciare a calcolare. Si basa sull'algoritmo di **Dijkstra** e **non soffre del count-to-infinity**: nessuna stima viene costruita sulla stima di qualcun altro, e ogni nodo lavora su fatti, non su dicerie. La conoscenza completa si ottiene facendo inviare a ogni nodo, in broadcast, dei *link-state packet* con gli identificativi e i costi dei propri collegamenti.

Che la mappa sia assemblata dentro ogni router o dentro un unico controller non cambia niente del calcolo: la versione con il controller è proprio il piano di controllo centralizzato già disegnato.

La versione di base calcola i cammini minimi **da un nodo di partenza a tutte le destinazioni**, e va quindi eseguita su ogni nodo.

La fase di scoperta

Prima di calcolare i cammini, ogni router deve scoprire i nodi vicini e condividere l'informazione con gli altri, in quattro passi:

1. **Scoperta dei vicini**: il router invia uno speciale messaggio "hello" su tutti i collegamenti; gli altri router rispondono con il proprio identificativo.
2. **Impostazione dei costi**: il router fissa la distanza o il costo verso ogni vicino, che può dipendere dai costi reali (tempo, banda, …) o essere impostato dall'amministratore.
3. **Costruzione del pacchetto**: il router crea un pacchetto che riassume gli identificativi dei vicini e i relativi costi.
4. **Scambio dei pacchetti**: il router invia il pacchetto a tutti gli altri router della rete (broadcast) e riceve pacchetti analoghi da tutti.

Alla fine ogni router ha una conoscenza "completa" della topologia: ogni nodo, dopo il broadcast collettivo, conosce tutto il grafo e può calcolare i cammini minimi, che sono gli stessi calcolati da tutti gli altri. Ora si può eseguire **Dijkstra**.

Lo pseudocodice

```
LinkState(x):
    N' = {x}
    for all nodes v:
        if v is a neighbor of x then  D(v) = c(x,v)
        else                          D(v) = ∞
    repeat
        find w not in N' such that D(w) is minimum
        add w to N'
        for each neighbor v of w which is not in N':
            D(v) = min( D(v), D(w) + c(w,v) )
    until N' = N
```

Link State (Dijkstra), eseguito dal nodo x

- $D(v)$ conserva la distanza dal nodo corrente a ogni destinazione; N' è l'insieme dei nodi di cui si conosce già il cammino minimo.
- **Inizializzazione**: si assume che tutti i costi siano noti; sono impostate solo le distanze verso i vicini.
- **Costruzione**: si sceglie il nodo w più vicino non ancora in N', si esplora il vicinato di w controllando se il cammino attraverso w è migliore di quello attuale, e si esce quando tutti i nodi sono stati esplorati.

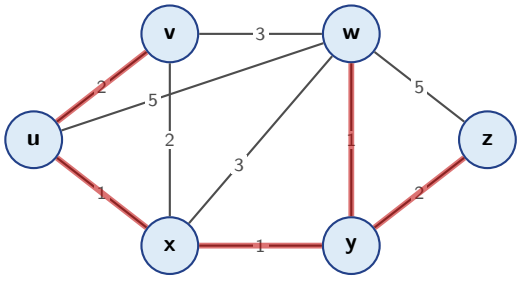
Un esempio completo e la ricostruzione del cammino

Eseguiamo LS sul nodo u del grafo di esempio. Usiamo anche una funzione $p(v)$ che memorizza il **nodo precedente** a v sul cammino migliore trovato finora.

Passo	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	**1, u**	∞	∞
1	ux	**2, u**	4, x		2, x	∞
2	uxv		4, x		**2, x**	∞
3	uxvy		**3, y**			4, y
4	uxvyw					**4, y**
5	uxvywz					

Tabella 25.2 – Dijkstra dal nodo u. In grassetto il nodo scelto a ogni passo (a parità di distanza la scelta è arbitraria: qui v prima di y).

0. Solo u è in N': i vicini hanno la distanza del collegamento diretto (v: 2, w: 5, x: 1), gli altri ∞.
1. Il più vicino è x (1). Da x: w diventa $\min(5,\ 1+3) = 4$ via x; y diventa $1 + 1 = 2$ via x; v resta $\min(2,\ 1+2) = 2$.
2. Si sceglie v (2). Da v: w resta $\min(4,\ 2+3) = 4$.
3. Si sceglie y (2). Da y: w diventa $\min(4,\ 2+1) = 3$ via y; z diventa $2 + 2 = 4$ via y.
4. Si sceglie w (3). Da w: z resta $\min(4,\ 3+5) = 4$.
5. Si sceglie z (4): tutti i nodi sono in N', fine.



Destinazione	Collegamento
v	(u, v)
w	(u, x)
x	(u, x)
y	(u, x)
z	(u, x)

Figura 25.4 – L'albero dei cammini minimi da u e la forwarding table che ne deriva.

Per ricostruire un cammino basta **risalire i predecessori** fino al nodo corrente. Per z: $p(z) = y$, $p(y) = x$, $p(x) = u$, quindi il cammino è u→x→y→z, e il primo hop (quello da scrivere nella forwarding table) è x.

Complessità, pro e contro

A ogni iterazione si controllano tutti i nodi della rete che non sono ancora in N', e se ne aggiunge uno. Il numero totale di controlli è $n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2}$, cioè $O(n^2)$ nel caso peggiore. Con strutture dati più sofisticate (per esempio gli *heap*) si scende a $O(n \log n)$ circa (più precisamente $O((|E| + n) \log n)$).

I messaggi sono l'altra metà del prezzo: lo stato dei collegamenti di ogni nodo deve raggiungere ogni nodo, quindi $O(|N|\,|E|)$ messaggi, e di nuovo a ogni cambiamento.

- ✔ **Convergenza più rapida**: tutte le comunicazioni avvengono contemporaneamente.
- ✔ **Niente count-to-infinity**: lo stato di ogni collegamento si propaga come un fatto.
- ✖ **Sincrono**: un nodo non può calcolare niente finché non è arrivata la mappa; deve prima sentire tutta la rete.

Quando il costo di un collegamento è il suo traffico

E se il costo di un collegamento fosse il carico che porta? Allora l'algoritmo cambia i costi che ha appena letto. Tutti calcolano che il giro in senso orario è più economico e ci si spostano, il che lo rende caro e l'altro lato vuoto; al giro successivo tutti tornano indietro. Il traffico oscilla e non si assesta mai.

- Non è un difetto del link state: qualsiasi algoritmo con una metrica basata sul carico oscilla allo stesso modo.
- Rendere il costo indipendente dal carico sembra inaccettabile, perché evitare i collegamenti congestionati è tutto lo scopo; eppure è ciò che si fa: RIP, OSPF e BGP sono tutti insensibili al carico.
- Un rimedio parziale per ciò che resta: i router non devono eseguire l'algoritmo nello stesso istante. Lasciati a sé si sincronizzano da soli, quindi l'istante di ogni annuncio viene reso casuale.

DV CONTRO LS

DV e LS affrontano il routing in modo complementare: DV sfrutta informazioni **locali** sui vicini, LS richiede informazioni **globali** su tutti i nodi.

	Distance Vector	**Link State**
Informazione	locale (solo i vicini)	globale (tutta la topologia)
Algoritmo	Bellman-Ford	Dijkstra
Complessità dei messaggi	comunicazione solo se cambia un cammino migliore	comunicazione sincrona tra tutti i nodi, più complessa
Velocità di convergenza	lenta; nel frattempo cammini subottimi; count-to-infinity	più rapida
Robustezza	calcoli concatenati: se una tabella è sbagliata, l'errore si propaga a tutti	ogni nodo calcola la propria tabella: più robusto
Esempi reali	RIP	OSPF, IS-IS

Tabella 25.3 – Distance Vector e Link State a confronto.

Un guasto che fece storia

Nel **1997** un router malfunzionante di un piccolo ISP annunciò percorsi errati, che furono propagati a catena: i router della dorsale furono inondati e parte di Internet restò scollegata per diverse ore. È l'esempio tipico di quanto possa costare la fiducia cieca nei calcoli dei vicini.

Nessuno dei due vince, ed **entrambi** sono usati in Internet: il motivo è ciò che viene dopo.

I SISTEMI AUTONOMI

Il modello a grafo ha ignorato due cose.

- **La scala**: Internet contiene centinaia di milioni di router. Una riga per destinazione è memoria impossibile, diffondere ogni cambiamento di collegamento è traffico impossibile, e un distance vector che iterasse su così tanti nodi non convergerebbe mai.
- **L'autonomia**: un ISP vuole eseguire l'algoritmo che preferisce dentro la propria rete, e tenere per sé la forma di quella rete. Un unico algoritmo mondiale gli chiederebbe di pubblicare tutto.

Sistema autonomo (AS)

Una sola mossa risolve entrambi i problemi: raggruppare i router in **sistemi autonomi**, ciascuno sotto un unico controllo amministrativo. Spesso un ISP è un AS; alcuni grandi sono più AS. Un AS è identificato da un numero globalmente unico, l'**ASN**, distribuito dagli stessi registri regionali che distribuiscono gli indirizzi.

Dentro un AS tutti eseguono lo stesso algoritmo e sanno di tutti gli altri: è un protocollo **intra-AS**. Fra AS deve essere qualcos'altro: un protocollo **inter-AS**.

OSPF: link state dentro un AS

- **OSPF** (*Open Shortest Path First*) è un protocollo link state: diffusione degli annunci dello stato dei collegamenti, poi Dijkstra eseguito localmente con il router stesso come radice. *Open* significa che la specifica è pubblica.
- I pesi sono affare dell'amministratore, non del protocollo: tutti a uno danno il routing a numero minimo di salti; pesi inversamente proporzionali alla capacità allontanano il traffico dai collegamenti lenti.
- Un HELLO su ogni collegamento controlla che il vicino sia vivo. Un nuovo annuncio parte ogni volta che un collegamento cambia stato, e comunque almeno ogni 30 minuti anche se non è cambiato niente.
- Gli annunci vanno in broadcast a ogni router dell'AS, non ai vicini: è la mappa dell'intero AS che va ricostruita.
- Si noti dove sta: i messaggi OSPF sono trasportati direttamente da IP, **protocollo 89**. Né TCP né UDP, quindi deve fornire da sé la propria consegna affidabile.

Le aree, e il resto di OSPF

Anche un solo AS può essere troppo grande per una sola mappa. Lo si può tagliare in **aree**: ogni area esegue il proprio algoritmo link state, e un router diffonde i propri annunci solo dentro la propria area. Esattamente un'area è la **dorsale**, e ogni router di bordo d'area le appartiene: attraversare l'AS significa salire fino a un router di bordo, attraversare la dorsale, scendere nell'area di destinazione. È la stessa mossa delle subnet dentro un blocco di indirizzi, un livello più su: nascondere il dettaglio, tenere il riassunto. Quando più cammini verso una destinazione hanno lo stesso costo OSPF può usarli tutti. Gli scambi possono essere autenticati, con una password condivisa o un hash con chiave (un tempo MD5, oggi HMAC-SHA); per default non lo sono, e un annuncio non autenticato si può falsificare, che è un modo di svuotare una rete.

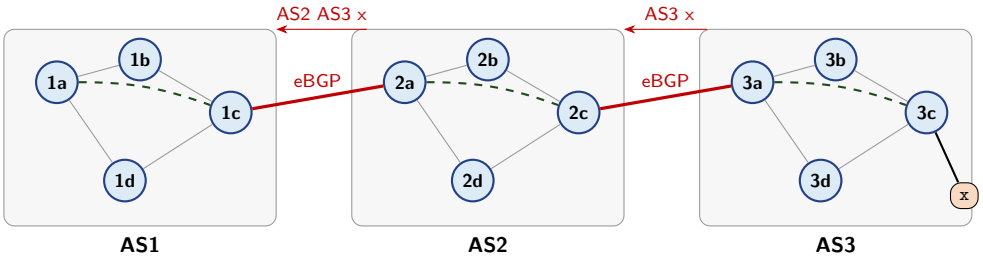
BGP: IL PROTOCOLLO CHE DEVE ESSERE UGUALE PER TUTTI

Un protocollo inter-AS funziona solo se gli AS dei due lati eseguono lo stesso, quindi ce n'è esattamente uno: **BGP** (*Border Gateway Protocol*), e ogni AS di Internet lo parla.

- Le sue destinazioni non sono indirizzi ma **prefissi CIDR**: una rotta verso `138.16.68.0/22` copre 1024 indirizzi in una riga. È qui che i blocchi aggregati ripagano.
- **Primo compito**: imparare dagli AS vicini quali prefissi esistono e attraverso quale sequenza di AS si raggiungono.
- **Secondo compito**: scegliere, fra le rotte imparate per un prefisso, quella da installare; e il criterio non sarà la distanza.
- Senza BGP una subnet è un'isola: esiste, ha indirizzi, e nessun router da nessuna parte ha una riga che corrisponda.

eBGP e iBGP

Gli AS non parlano: parlano i router. Due di loro tengono una connessione TCP semipermanente sulla **porta 179**: un protocollo di routing costruito sul servizio di trasporto già usato. Attraverso un confine è **eBGP**; fra router dello stesso AS è **iBGP**, e la notizia di un prefisso straniero viaggia verso l'interno su di essa.



tratteggiato: iBGP, dentro un AS e non per forza lungo un collegamento fisico

Figura 25.5 – L'annuncio del prefisso `x` viaggia verso sinistra: AS3 lo annuncia ad AS2 come «AS3 x», AS2 lo diffonde dentro di sé con iBGP e lo annuncia ad AS1 come «AS2 AS3 x».

Che cosa porta una rotta

Rotta BGP

Una **rotta** è un prefisso insieme ai suoi **attributi**. Due contano qui:

- **AS-PATH**: la lista degli AS che l'annuncio ha attraversato. Ogni AS antepone il proprio numero prima di passarlo, quindi la lista cresce mentre la notizia viaggia. Rileva anche i cicli: un router che trova il proprio AS già nella lista rifiuta l'annuncio. Nessun conteggio di salti, nessun count-to-infinity.
- **NEXT-HOP**: l'indirizzo IP dell'interfaccia del router che comincia l'AS-PATH, un indirizzo che non appartiene al nostro AS ma sta su una subnet direttamente collegata a esso.

NEXT-HOP è la giuntura fra i due protocolli: per usare una rotta BGP, un router cerca quell'indirizzo nella tabella costruita dal suo protocollo intra-AS.

Scegliere fra le rotte

Un router impara spesso decine di rotte verso lo stesso prefisso. Le regole si applicano in ordine, ciascuna ne elimina alcune, finché ne resta una:

1. **Preferenza locale**: un valore attaccato alla rotta dall'amministratore. È un numero di *policy*, deciso dentro l'AS, e viene **prima** di qualsiasi cosa riguardi la distanza.
2. **AS-PATH più corto**: meno sistemi autonomi attraversati. Non meno router: fra AS non c'è altra nozione di costo.
3. **Patata bollente** (*hot potato*): fra le superstiti, quella il cui NEXT-HOP è più vicino secondo il costo intra-AS. Far uscire il pacchetto dal proprio AS nel modo più economico possibile, senza curarsi del resto del suo viaggio.

La regola 2 prima della 3 è ciò che impedisce a BGP di essere puramente egoista. E due router dello stesso AS possono comunque scegliere AS-PATH diversi per lo stesso prefisso.

La policy decide prima del costo

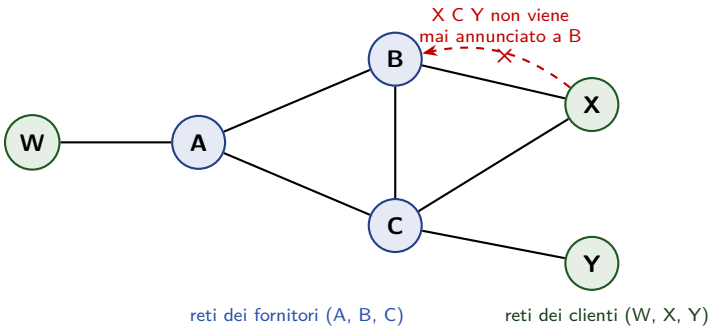


Figura 25.6 – X conosce il cammino attraverso C verso Y, e se lo tiene per sé: B non l'ha mai imparato, quindi non manda mai a X traffico per Y. Nessuna regola vieta il transito: lo fa il silenzio.

- B dovrebbe passare a C il cammino che ha imparato verso W? Trasportare il transito fra A e C costa soldi a B e non gli fa guadagnare niente: è il lato del denaro della domanda lasciata aperta con la neutralità della rete.
- La regola pratica, in nessuno standard: il traffico sulla dorsale di un fornitore deve avere una sorgente o una destinazione in uno dei suoi clienti. Il resto è bilaterale e riservato.

Perché due protocolli

- **Policy**: fra AS domina. Chi può trasportare il traffico di chi è una questione commerciale, e BGP porta gli attributi che permettono di rispondere. Dentro un AS tutto è sotto un'unica amministrazione, e la questione non si pone.
- **Scala**: fra AS le destinazioni sono l'intera Internet. Dentro, se una rete diventa troppo grande, la si può sempre dividere in due AS, e OSPF può dividere ancora in aree.
- **Prestazioni**: fra AS la qualità della rotta è secondaria; un cammino più lungo che soddisfa una policy batte uno più corto che non la soddisfa, e non c'è costo oltre ai salti di AS. Dentro un AS il routing può permettersi di ottimizzare.

La risposta a «quale algoritmo usa Internet» non è quindi un algoritmo, ma una **divisione del lavoro**.

Un indirizzo, molte macchine

Anycast IP

Con l'**anycast** si dà lo stesso indirizzo IP a molte macchine in parti diverse del mondo, e ciascuna lo annuncia con BGP. Un router che riceve più annunci per quell'indirizzo crede che siano cammini diversi verso lo stesso posto, ed esegue la selezione: sceglie la copia più vicina senza sapere che ci sono copie.

È ciò che c'era dietro un fatto lasciato senza spiegazione: la radice del DNS ha tredici indirizzi e più di mille macchine. La query va alla più vicina che ha quell'indirizzo. Il limite: le reti di distribuzione dei contenuti per lo più non lo usano, perché un cambiamento BGP a metà di un trasferimento può consegnare i segmenti successivi della stessa connessione TCP a una macchina diversa, che non ne ha mai sentito parlare.

Mettere un'azienda su Internet

1. Un contratto con un ISP locale: un collegamento fisico dal router gateway dell'azienda a uno dei suoi, e un blocco di indirizzi, per esempio un /24, da assegnare al server Web, al server di posta, al server DNS e al gateway.
2. Un contratto con un *registrar* per un nome di dominio, a cui si dà l'indirizzo del proprio server DNS: il registrar mette quella voce nei server di `.com`, così chiunque può trovare l'azienda per nome.
3. Nel proprio server DNS si associa `www.azienda.com` all'indirizzo del server Web.
4. E il passo senza il quale niente funziona: l'ISP **annuncia il prefisso con BGP**. L'annuncio si propaga verso l'esterno, e alla fine ogni router di Internet ha una riga che corrisponde agli indirizzi dell'azienda, o un aggregato che li contiene.

Indirizzamento, nomi, forwarding e routing, in una sola procedura.

RIEPILOGO

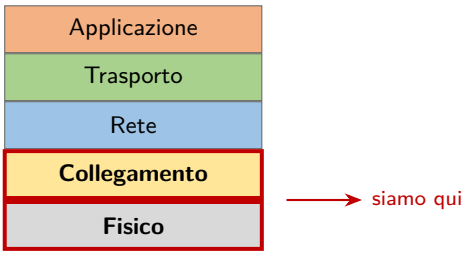
In breve

- Il routing cerca i **cammini a costo minimo** su un grafo $G = (N, E)$ con costi $c(x, y)$; converge quando li ha trovati per tutte le coppie.
- Il **flooding** è robusto ma impraticabile su larga scala.
- **DV** (Bellman-Ford, decentralizzato, asincrono): $d_x(y) = \min_v\{c(x, v) + d_v(y)\}$; convergenza lenta e **count-to-infinity**.
- **LS** (Dijkstra, conoscenza globale): scoperta dei vicini, costi, pacchetti link-state in broadcast, poi Dijkstra con $D(v)$ e $p(v)$; $O(n^2)$.
- LS converge più velocemente ed è più robusto; DV comunica meno. Nessuna delle due metriche segue il traffico, perché un costo che segue il traffico fa oscillare il routing.
- Scala e autonomia dividono Internet in **sistemi autonomi**: OSPF dentro, BGP fra. BGP instrada verso prefissi, con AS-PATH e NEXT-HOP, e decide prima per policy e poi per costo; l'anycast ne è un'applicazione.
- Ogni cammino calcolato è una sequenza di singoli salti, e non ci siamo mai chiesti che cosa sia davvero un salto: come sono incorniciati i bit su un collegamento, e come si chiama lì la macchina all'altro capo. È dove andiamo ora.

Parte VI

Il livello di collegamento

FRAMING E RILEVAZIONE DEGLI ERRORI



DALLA RETE AL COLLEGAMENTO E AL LIVELLO FISICO

Il livello di rete fornisce comunicazione tra due host, *ovunque* si trovino. Il livello di collegamento e il livello fisico forniscono invece comunicazione tra due nodi **direttamente collegati**:

- il **livello di collegamento** (*link layer*) è la parte della pila dedicata alla trasmissione di pacchetti sui collegamenti (canali di trasmissione), **da un nodo al successivo** della rete;
- il **livello fisico** regola la struttura dei collegamenti (mezzo trasmissivo, connettori, tipi di cavo, …) e il modo in cui i bit vengono rappresentati e trasmessi lungo il collegamento.

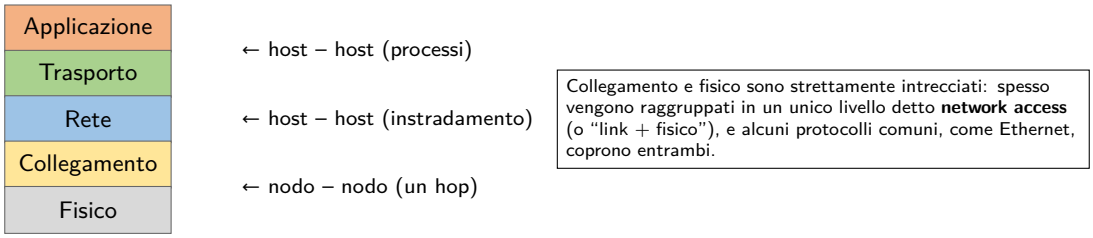


Figura 26.1 – Cosa garantisce ciascun livello.

A differenza dei livelli superiori, questi due livelli sono presenti in **tutti** i pezzi dell'infrastruttura di rete:

- **nodi**: host (PC, server, …), router, switch, hub, access point Wi-Fi;
- **collegamenti**: cablati (rame, fibra ottica) e senza fili (radiofrequenze).

Un esempio di segnale: la codifica NRZ

A questo livello non abbiamo altro che **bit** (zeri e uni) trasmessi su una linea fisica, rappresentati da grandezze fisiche. L'esempio più semplice di codifica è la **NRZ** (*Non-Return to Zero*): il tempo è diviso in intervalli uguali, e in ciascun intervallo il segnale vale $+L$ per rappresentare un 1 e 0 (oppure $-L$) per rappresentare uno 0.

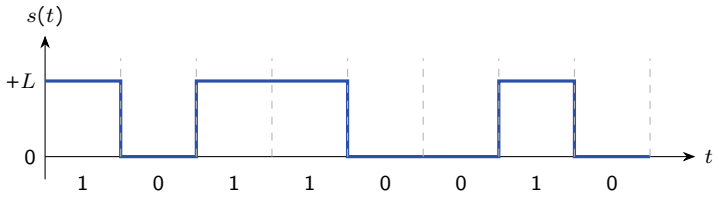


Figura 26.2 – La sequenza 10110010 in codifica NRZ: il tempo è diviso in intervalli uguali.

L'ULTIMO INCAPSULAMENTO

Per consegnare un datagramma IP dall'host sorgente all'host destinazione bisogna farlo "saltare" su ciascuno dei singoli collegamenti e dispositivi del percorso. Per essere trasferiti su un collegamento, i datagrammi vengono incapsulati in **frame** di collegamento, il cui formato dipende dal tipo di collegamento.

Frame

Il livello di collegamento accetta pacchetti dal livello di rete e li incapsula in **frame**, che invia usando il livello fisico; in ricezione il processo è inverso. È l'**ultimo incapsulamento**: i frame vengono convertiti in **segnali** e trasferiti sul collegamento secondo le sue specifiche. I segnali possono essere impulsi elettrici, luce o onde radio.

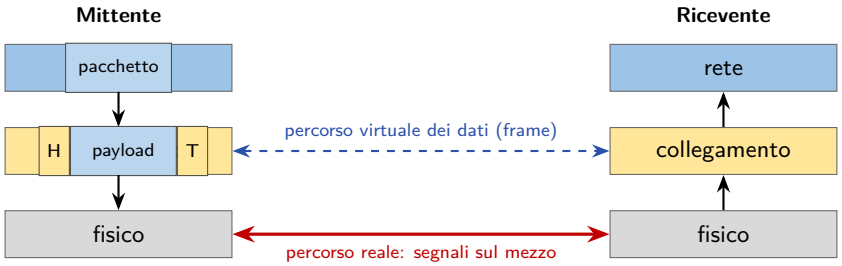


Figura 26.3 – Il frame: header (H), payload (il pacchetto di rete) e trailer (T).

Esempio 1 — Quanti collegamenti per uscire di casa

Per raggiungere Internet, i pacchetti di un portatile in una rete aziendale possono dover attraversare 4 o 5 collegamenti (con e senza fili), un access point, uno switch e 2 o 3 router. Su ciascun collegamento il datagramma viaggia dentro un frame diverso, adatto a quel tipo di collegamento (per esempio un frame Wi-Fi fino all'access point, poi frame Ethernet).

I SERVIZI DEL LIVELLO DI COLLEGAMENTO

Il livello di collegamento può offrire fino a quattro servizi.

1. Framing

Incapsulare i datagrammi in frame, con il datagramma come payload e un header e un trailer specifici. La struttura del frame dipende dai protocolli di collegamento e fisico.

2. Accesso al collegamento (link access)

Definire le regole di accesso al collegamento tramite un protocollo **MAC** (*Medium Access Control*). Le regole dipendono dall'architettura del collegamento:

- nei collegamenti **punto-punto** (un solo mittente e un solo ricevente) il protocollo MAC è semplice o inesistente: il mittente può inviare un frame ogni volta che il collegamento è libero;
- nei collegamenti **broadcast** c'è il **problema dell'accesso multiplo**: il protocollo MAC serve a coordinare le trasmissioni dei molti nodi.

3. Consegna affidabile

Garantire che i frame trasmessi sul collegamento vengano ricevuti.

- Si usa spesso sui collegamenti soggetti a tassi di errore elevati, come quelli **senza fili**, con l'obiettivo di correggere gli errori **localmente**, senza costringere gli estremi a ritrasmettere.
- Può introdurre un forte overhead. Poiché altri protocolli (per esempio TCP) sono già affidabili, non è sempre implementata.

4. Rilevazione e correzione degli errori

L'hardware di collegamento può subire il problema dell'**inversione dei bit** (*bit flipping*). Molti protocolli di collegamento implementano strategie di controllo e correzione più sofisticate (e realizzate in hardware), in aggiunta a quelle dei livelli superiori (i checksum di TCP e IP).

- Poiché non c'è un ulteriore incapsulamento, questi controlli coprono di fatto **l'intero messaggio**.
- I controlli realizzati in hardware possono essere molto più veloci.

L'IMPLEMENTAZIONE: LA NIC

Nei dispositivi di rete (end system, router, …) le funzioni del livello di collegamento sono realizzate sia in software sia, soprattutto, in hardware.

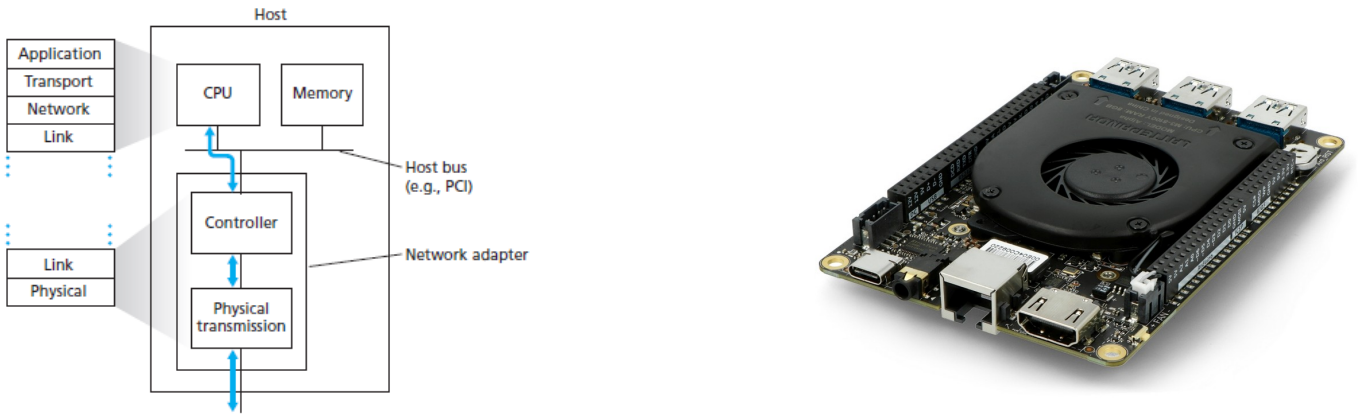


Figura 26.4 – A sinistra lo schema di un adattatore di rete collegato al bus dell'host; a destra un mini-PC (LattePanda) con la NIC integrata sulla scheda madre.

Nei computer ci sono adattatori di rete detti **NIC** (*Network Interface Card*), con un chip specializzato (**controller**) che realizza le funzioni di collegamento. Un tempo erano schede separate dalla scheda madre (inserite negli slot PCI); oggi sono spesso integrate (*LAN-on-motherboard*).

Software e hardware

- **Software**: gira sulla CPU e offre funzioni di alto livello come l'indirizzamento, la gestione di interrupt e hardware, la gestione degli errori, l'incapsulamento e il decapsulamento dei datagrammi.
  - A livello di collegamento c'è un indirizzo aggiuntivo, l'**indirizzo MAC**, che identifica la NIC.
  - I datagrammi vanno letti dalla memoria in incapsulamento (lato mittente) e scritti in memoria in decapsulamento (lato ricevente), perché li usi il protocollo superiore.
- **Hardware**: funziona come un tipico dispositivo di I/O, convertendo i dati nel segnale da trasmettere secondo il protocollo fisico (Ethernet, wireless, …).

Le tecnologie fisiche

Il protocollo di collegamento, e quindi la struttura del frame, dipende dalla tecnologia del collegamento fisico:

- **Ethernet 802.3**: collegamento cablato basato su impulsi elettrici su cavi in rame con 4 doppini intrecciati (il più comune) o su fotoni in cavi di fibra ottica;
- **Wi-Fi 802.11**: collegamento senza fili a radiofrequenza (2,4 GHz, 5 GHz, 6 GHz);
- **Bluetooth**: collegamento senza fili a radiofrequenza (2,4 GHz).

Tipo	Segnale	Distanza	Banda	Punto debole
Senza fili	radio	30–50 m	fino a 1,3 Gb/s	sicurezza, velocità
Doppino in rame	elettrico	30–100 m	1–25 Gb/s	interferenze
Fibra ottica	luce	100–200 km	1–400+ Gb/s	fragilità

Tabella 26.1 – Pro e contro delle tecnologie fisiche (i limiti di distanza e banda cambiano continuamente).

I METODI DI FRAMING

Potremmo trasmettere un flusso ininterrotto di bit? No: sarebbe impossibile **identificare gli errori** e **ritrasmettere** solo le parti danneggiate. Inoltre bisogna distinguere i diversi pacchetti che arrivano come payload dal livello di rete. Il ricevente deve quindi capire **dove inizia e dove finisce** ogni frame. I metodi principali sono quattro:

1. conteggio dei byte (*byte count*);
2. byte di flag con *byte stuffing*;
3. bit di flag con *bit stuffing*;
4. violazioni della codifica fisica.

Conteggio dei byte

Byte count

Ogni frame inizia con un campo che indica il **numero di byte** del frame. È semplice, ma è difficilissimo risincronizzarsi dopo un errore, e in pratica non è mai usato da solo.

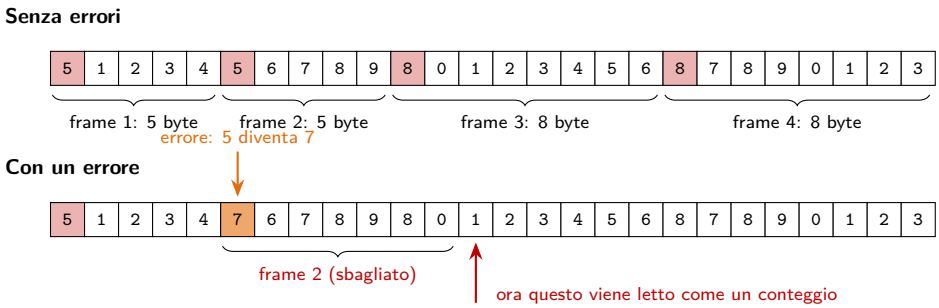


Figura 26.5 – Con il conteggio dei byte un solo errore sul contatore fa perdere la sincronizzazione su tutti i frame successivi.

Byte di flag e byte stuffing

Byte stuffing

Ogni frame è delimitato da uno speciale **byte di flag** (FLAG). Se il byte FLAG compare **nei dati**, va "protetto" inserendo prima di esso un byte di escape (ESC); lo stesso vale per un ESC che compare nei dati. Il ricevente rimuove ogni ESC e considera come dato il byte che lo segue.

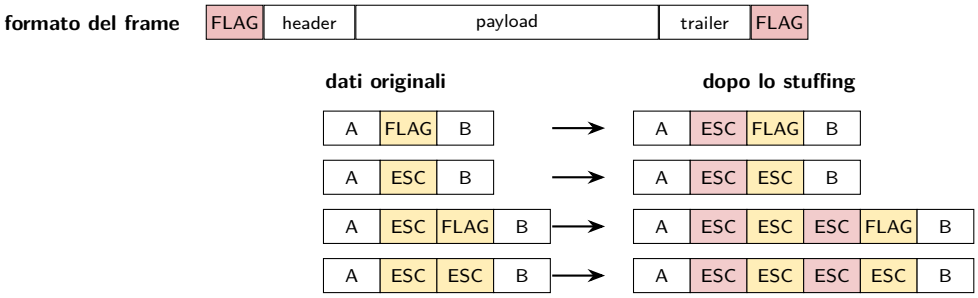


Figura 26.6 – Byte stuffing: ogni FLAG o ESC nei dati viene preceduto da un ESC.

- Il frame diventa **più lungo**, ma dopo un errore è **facile risincronizzarsi**: basta cercare il prossimo FLAG.
- Gli errori non si propagano come nel conteggio dei byte.
- In ricezione: ogni coppia ESC-FLAG diventa un FLAG di dati, ogni coppia ESC-ESC un ESC di dati.

Bit di flag e bit stuffing

Bit stuffing

Il flag che delimita i frame è la sequenza di bit `01111110` (sei 1 consecutivi). Per evitare che compaia nei dati:

- **in trasmissione**, dopo **cinque 1 consecutivi** nei dati si inserisce uno **0**;
- **in ricezione**, ogni **0 che segue cinque 1** viene eliminato.

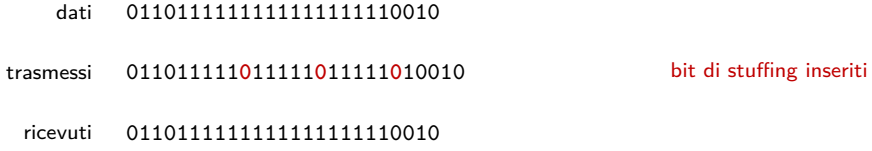


Figura 26.7 – Bit stuffing: dopo cinque 1 si aggiunge uno 0, che il ricevente rimuove.

Si sfrutta la codifica della trasmissione fisica: al livello fisico si usa un simbolo (o una parola) **che non rappresenta dati**. Le violazioni della codifica delimitano i frame, e non serve alcuno stuffing, perché i simboli delimitatori **non possono comparire** nei dati. È il caso, per esempio, della codifica 4B/5B, in cui alcune parole da 5 bit non sono mai usate per i dati.

La rilevazione degli errori

A livello di collegamento si possono rilevare e correggere gli errori a livello di singolo bit. In assenza di informazione aggiuntiva non c'è nulla con cui confrontare il payload: bisogna aggiungerne.

Il problema

Siano D i nostri dati, di d bit. Per proteggerli dagli errori sui bit aggiungiamo al messaggio dei **bit di rilevazione e correzione degli errori**, **EDC** (*Error Detection and Correction*). L'obiettivo è capire se i D' ed EDC' ricevuti differiscono dagli originali D ed EDC e, se possibile, ricostruire gli originali.

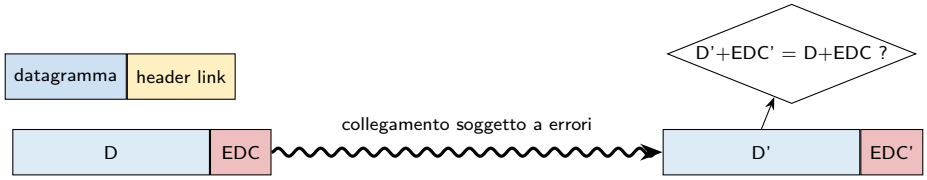


Figura 26.8 – Il ricevente usa EDC' per verificare se D' è integro.

Nessuna garanzia assoluta

Anche usando bit di rilevazione possono restare errori non rilevati, in casi "patologici". Le tecniche migliori riducono questa probabilità, ma al prezzo di più bit aggiuntivi e più calcoli.

Il controllo di parità

Controllo di parità

Il **controllo di parità** è la forma più semplice di rilevazione: si aggiunge al messaggio **un solo bit di parità**, scelto in modo che il numero totale di 1 nei $d+1$ bit sia **pari** (schema a parità pari) o **dispari** (schema a parità dispari).

Messaggio	Bit di parità (pari)	Bit di parità (dispari)
`10101100` (quattro 1)	0	1
`11010000` (tre 1)	1	0

Tabella 26.2 – Bit di parità pari e dispari.

Il ricevente deve solo contare gli 1 nei $d+1$ bit ricevuti: se con uno schema pari trova un numero dispari di 1 (o viceversa), sa che si è verificato **almeno un errore**.

Esempio 2 — Un errore rilevato e uno no

Si invia `10101100` con parità pari: `10101100 0`.

- Si riceve `10100100 0`: tre 1, numero dispari ⇒ **errore rilevato**.
- Si riceve `10100101 0`: quattro 1, numero pari ⇒ due errori, **non rilevati**.

Gli errori arrivano a raffica

La parità funziona solo se si verifica un numero **dispari** di errori. Quanto è probabile? Se gli errori sui bit fossero rari e indipendenti, la probabilità di errori multipli in un pacchetto sarebbe piccolissima. Ma nelle reti non è così: gli errori sono spesso **raggruppati a raffica** (*burst*), cioè sequenze in cui il primo e l'ultimo bit sono sbagliati e quelli in mezzo sono un misto di bit sbagliati e corretti. In condizioni di burst la probabilità di errori non rilevati con la parità singola si avvicina al **50%**: non è davvero utile.

La parità bidimensionale

Parità bidimensionale

Nella **parità bidimensionale** il messaggio viene diviso in n righe e m colonne, e a **ogni riga** e **ogni colonna** si associa un bit di parità.

Figura 26.9 – Parità bidimensionale: dati 4×5 bit, con parità di riga (ultima colonna) e di colonna (ultima riga).

- Il ricevente può rilevare che si è verificato un errore e anche **quale bit** è sbagliato (all'incrocio tra la riga e la colonna con parità errata), e quindi tentare una **correzione**.
- La parità bidimensionale può anche **rilevare** (ma non correggere!) qualsiasi combinazione di **due errori** in un pacchetto.

IL CRC

Cyclic Redundancy Check (CRC)

Il **CRC** è una tecnica di rilevazione degli errori molto diffusa.

- Per inviare d bit di dati D, mittente e ricevente concordano una sequenza di $r+1$ bit detta **generatore** G, con il bit più significativo (a sinistra) uguale a 1.
- Per ogni D il mittente sceglie r bit aggiuntivi, i **bit CRC** (R), da accodare al messaggio, in modo che il messaggio M (cioè D seguito da R) sia **divisibile modulo 2 per** G (resto $= 0$).
- Il ricevente divide il messaggio ricevuto per G: se il resto è zero il messaggio è corretto, altrimenti c'è un errore.

La divisione si esegue allineando G sotto il primo bit a 1 del messaggio e facendo lo **XOR** bit a bit.

L'aritmetica modulo 2

Nel CRC le stringhe di bit sono interpretate come **polinomi a coefficienti binari**: `1011` è $x^3 + x + 1$. L'aritmetica è modulo 2 senza riporti né prestiti: **somma e sottrazione coincidono con lo XOR**.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Perché funziona (con i polinomi)

Accodare r zeri a D equivale a moltiplicare $D(x)$ per x^r. Dividendo per il generatore si ottiene

$$x^r D(x) = Q(x)\,G(x) + R(x)$$

dove Q è il quoziente e R il resto (di grado minore di r). Allora il messaggio trasmesso

$$M(x) = x^r D(x) - R(x) = x^r D(x) + R(x) = Q(x)\,G(x)$$

è **divisibile per** $G(x)$. In binario, M è semplicemente D seguito dagli r bit di R. Se durante la trasmissione si aggiunge un errore $E(x)$, il ricevente trova resto $\neq 0$ a meno che anche $E(x)$ sia divisibile per $G(x)$, cosa che un buon generatore rende molto improbabile.

Un esempio completo

Messaggio di 6 bit con CRC di 3 bit: $d = 6$, $D =$ `101110`, $r = 3$, $G =$ `1001`.

Mittente: calcolo del CRC

D con $r = 3$ zeri: `101110000`

```
  101110000
XOR 1001
  ---------
  00101000
    101000
XOR   1001
  ---------
    001100
XOR     1001
  ---------
     01010
XOR     1001
  ---------
     0011
```

gli ultimi r bit del resto sono il CRC: `011`

Ricevente: verifica

ricevuto D + CRC: `101110011`

```
  101110011
XOR 1001
  ---------
  00101001
    101001
XOR   1001
  ---------
    001101
XOR     1001
  ---------
     01001
XOR     1001
  ---------
     0000
```

resto 0: il messaggio è integro

Figura 26.10 – Calcolo e verifica del CRC con $G =$ `1001`: a ogni passo G si allinea con il primo 1 rimasto.

1. Il mittente accoda $r = 3$ zeri a D: `101110000`.
2. Divide modulo 2 per G: allinea `1001` sotto il primo 1, fa lo XOR, poi si sposta sul primo 1 del risultato, e così via (spostamenti di 0, 2, 2 e 1 posizioni).
3. Il resto, sugli ultimi r bit, è `011`: è il CRC. Si trasmette `101110 011`.
4. Il ricevente divide per G il messaggio ricevuto: il resto è `000`, quindi il messaggio è integro.

Se c'è un errore

Se durante la trasmissione un bit si inverte, per esempio si riceve `100110011`, la divisione dà un resto diverso da zero (qui `001`) e il ricevente scarta il frame. Provate a verificarlo!

I CRC standard

- Sono stati definiti standard internazionali per generatori a 8, 12, 16 e 32 bit ($r = 8, 12, 16, 32$).
- Lo standard **CRC-32**, adottato in diversi protocolli di collegamento IEEE (tra cui Ethernet), usa il generatore a 33 bit:

$$G_{\text{CRC-32}} = \texttt{100000100110000010001110110110111}$$

- Ogni standard CRC rileva **sicuramente** tutti gli errori a raffica di lunghezza inferiore a $r + 1$ bit; per raffiche più lunghe la probabilità di rilevare l'errore è

$$P = 1 - 0{,}5^{r}$$

- La probabilità di rilevare l'errore quindi **cresce con** r: con CRC-32 si perde un errore su circa quattro miliardi.

La distanza di Hamming

Oltre a rilevare gli errori, alcuni codici permettono di **correggerli**.

Codice e distanza di Hamming

Un codice trasforma dati di n bit in **parole di codice** (*codeword*) di $n + k$ bit. La **distanza di Hamming** tra due parole è il numero di bit che è necessario invertire per trasformare una nell'altra, cioè il numero di bit in cui differiscono. La distanza di Hamming **di un codice** è la **minima** distanza tra due parole valide qualsiasi del codice.

Esempio 3 — Distanza tra due parole

$w_1 = \texttt{00110110}$, $w_2 = \texttt{00011010}$. Facendo lo XOR, $w_1 \oplus w_2 = \texttt{00101100}$: i bit diversi sono 3, quindi $d(w_1, w_2) = 3$.

I protocolli di collegamento usano per comunicare parole di lunghezza fissa che **non sono tutte quelle possibili**. Per esempio, delle 2^5 parole possibili di 5 bit se ne usano 2^4 (la metà), come nella codifica **4B/5B**. Ciò consente di rilevare gli errori, e anche di evitare lunghe sequenze di 1 o di 0 (utili per la sincronizzazione).

Esempio 4 — Un codice con distanza 5

Usiamo parole di 10 bit, ma ne usiamo solo 4 (per codificare 2 bit di dati, $n = 2$, $k = 8$):

Dati	Parola di codice
00	`0000000000`
01	`0000011111`
10	`1111100000`
11	`1111111111`

La distanza minima tra due parole è **5**. Se si riceve `0000011111`, è una parola valida. Se si riceve `0100011101`, non è valida: è a distanza 2 da `0000011111` e a distanza maggiore da tutte le altre, quindi si **corregge** in `0000011111` (dato 01). Si è trattato di uno schema didattico (v. Tanenbaum per i veri codici di Hamming, basati su questo principio ma più complicati).

Capacità di rilevazione e correzione

Con un codice di distanza di Hamming L:

- si possono **rilevare** fino a d errori se $L \geq d + 1$ (cioè $d \leq L - 1$): d errori non bastano a trasformare una parola valida in un'altra;
- si possono **correggere** fino a d errori se $L \geq 2d + 1$: la parola ricevuta resta più vicina a quella originale che a qualsiasi altra.

Nell'esempio ($L = 5$): si rilevano fino a 4 errori e se ne correggono fino a 2.

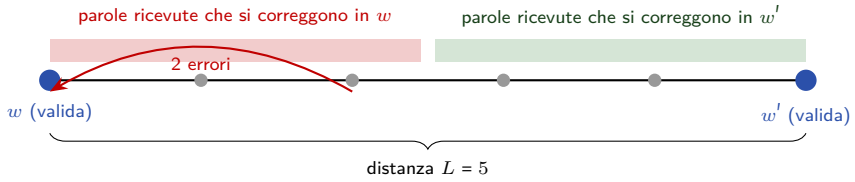


Figura 26.11 – Con distanza 5, una parola con al massimo 2 errori è ancora più vicina all'originale: si può correggere.

RIEPILOGO

In breve

- Collegamento e fisico trasportano frame **da un nodo al successivo**; il frame è l'ultimo incapsulamento e diventa segnale.
- Servizi: **framing**, **accesso al collegamento** (MAC), **consegna affidabile**, **rilevazione e correzione** degli errori. Realizzati soprattutto nella **NIC**.
- Framing: conteggio dei byte (fragile), FLAG con **byte stuffing**, flag `01111110` con **bit stuffing**, violazioni della codifica.
- **Parità** singola (rileva un numero dispari di errori), **bidimensionale** (corregge 1 errore, rileva 2).
- **CRC**: si accodano r bit perché $D \cdot 2^r + R$ sia divisibile modulo 2 per G; rileva le raffiche $< r + 1$.
- Distanza di Hamming L: rileva $L - 1$ errori, ne corregge $\lfloor (L - 1)/2 \rfloor$.

Il sottolivello MAC e l'accesso multiplo

I due tipi di collegamento

Ricordiamo i quattro servizi del livello di collegamento: **framing** (incapsulare e decapsulare i datagrammi), **accesso al collegamento** (regolare l'accesso ai collegamenti condivisi, compito del protocollo MAC), **consegna affidabile** (per lo più facoltativa) e **rilevazione e correzione degli errori** (parità, checksum, CRC). In questo capitolo ci occupiamo del secondo.

I collegamenti da gestire sono essenzialmente di due tipi.

Collegamento punto-punto e broadcast

- Un collegamento **punto-punto** ha un solo mittente a un'estremità e un solo ricevente all'altra. Un protocollo che gestisce collegamenti di questo tipo è il **PPP** (*Point-to-Point Protocol*). Esempio: un cavo Ethernet diretto tra due computer.
- Un collegamento **broadcast** ha più nodi mittenti e riceventi collegati allo **stesso canale condiviso**. Si chiama broadcast perché quando un nodo trasmette un frame, questo viene **ricevuto da tutti** i nodi sul canale. Esempi: il bus Ethernet, Ethernet half-duplex (raro, oggi i cavi sono quasi tutti full-duplex), le LAN wireless.

Figura 27.1 – Nel broadcast tutti ricevono il frame, e solo il destinatario lo accetta.

L'accesso ai collegamenti broadcast va **coordinato** (*problema dell'accesso multiplo*), perché più comunicazioni sullo stesso collegamento possono interferire tra loro.

Le collisioni

Collisione

Il problema principale dei collegamenti broadcast è la **collisione**: se più nodi trasmettono frame **contemporaneamente** sullo stesso canale, i frame si sovrappongono e diventano **incomprensibili**.

I frame in collisione vengono ricevuti da tutti i nodi e scartati come errati (non si fa danno), ma:

- **tutti** i frame trasmessi vanno **persi**;
- l'intervallo di tempo è **sprecato**, perché il canale è stato usato per trasmettere dati inutili.

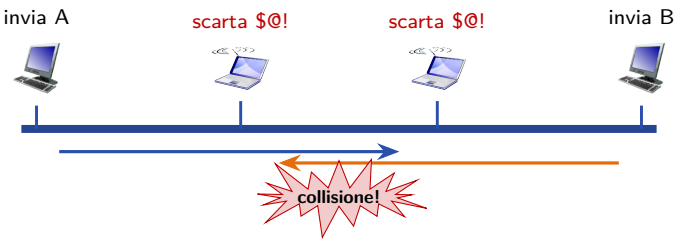


Figura 27.2 – Due trasmissioni contemporanee sullo stesso canale collidono: tutti i nodi ricevono dati illeggibili.

L'accesso al mezzo fisico (come il framing) è regolato dal protocollo **MAC** (*Medium Access Control*), un **sottolivello** del livello di collegamento.

I PROTOCOLLI AD ACCESSO MULTIPLO

Nelle reti i **protocolli ad accesso multiplo** regolano le trasmissioni sui canali broadcast in modo che:

- le collisioni siano **gestite**;
- ogni nodo abbia la possibilità di trasmettere, e nessuno **monopolizzi** il collegamento;
- le connessioni già stabilite non vengano interrotte (per esempio controllando se il collegamento è occupato).

Servono in molti contesti, cablati, senza fili e satellitari, dove centinaia o migliaia di nodi possono comunicare direttamente su un canale broadcast. Ne esistono decine, su tecnologie di collegamento diverse; gli approcci principali sono tre:

- **partizionamento del canale**: la banda viene divisa tra i nodi;
- **accesso casuale**: i nodi "tentano la sorte" per accedere;
- **a turni** (*taking-turns*): i nodi aspettano il proprio turno.

Le proprietà desiderate

Un protocollo ad accesso multiplo per un canale broadcast a R bit/s dovrebbe essere:
- **efficiente**: se un solo nodo deve trasmettere, dovrebbe avere throughput R; se M nodi devono trasmettere, ciascuno dovrebbe avere in media R/M bit/s;
- **decentralizzato**: un nodo "master" sarebbe un single point of failure;
- **semplice e leggero**: si inviano tantissimi frame, non ci può essere overhead.

PROTOCOLLI A PARTIZIONAMENTO DEL CANALE

TDMA

TDMA (Time Division Multiple Access)

Dato un canale con N nodi e velocità R bit/s, il **TDMA** divide il tempo in **trame** (*frame* temporali) e ogni trama in N **slot**. Ogni slot è assegnato a uno degli N nodi: quando un nodo ha un pacchetto da inviare, aspetta il proprio slot. Di solito la dimensione degli slot è scelta in modo che un pacchetto intero possa essere trasmesso in uno slot.

FDMA

FDMA (Frequency Division Multiple Access)

Il **FDMA** divide il canale da R bit/s in **frequenze diverse**, ciascuna con banda R/N, e assegna ogni frequenza a uno degli N nodi.

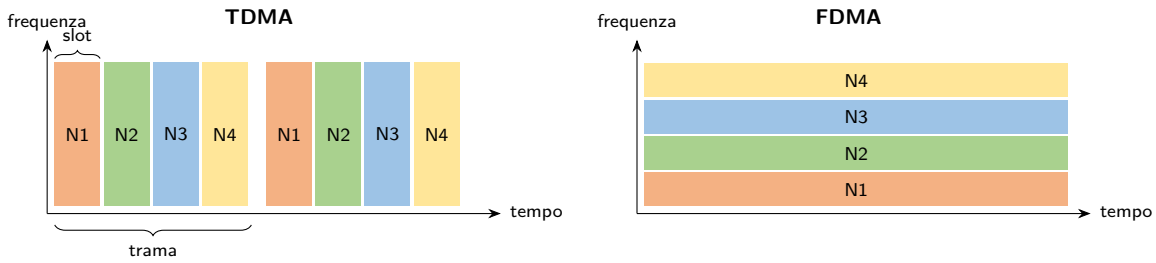


Figura 27.3 – TDMA: a turno, ciascuno con tutta la banda. FDMA: sempre, ciascuno con una fetta della banda.

FDMA e TDMA condividono pro e contro:

- ✔ **evitano le collisioni** e dividono la banda **equamente** tra gli N nodi;
- ✖ un nodo è limitato a una banda di R/N **anche quando è l'unico** ad avere pacchetti da inviare: capacità sprecata.

CDMA

CDMA (Code Division Multiple Access)

Il **CDMA** assegna a ogni nodo un **codice** diverso, con cui codifica e decodifica i bit che invia. Se i codici sono scelti con cura, nelle reti CDMA più nodi possono trasmettere **contemporaneamente, sulle stesse frequenze**, senza interferire.

Esempio 1 — L'idea del CDMA

Siano $c_A = (+1, -1)$ e $c_B = (+1, +1)$ i codici di due stazioni: sono **ortogonali**, cioè il loro prodotto scalare è $c_A \cdot c_B = 1 - 1 = 0$, mentre $c_B \cdot c_B = 2$. Per trasmettere un bit, ogni stazione invia il proprio codice moltiplicato per $+1$ (bit 1) o -1 (bit 0). Sul canale i segnali si **sommano**.

- A invia 1, B invia 1: sul canale arriva $c_A + c_B = (2, 0)$.
- Il ricevente di B calcola $(c_A + c_B) \cdot c_B = c_A \cdot c_B + c_B \cdot c_B = 0 + 2 > 0$: B ha inviato **1**.
- Il ricevente di A calcola $(c_A + c_B) \cdot c_A = 2 + 0 > 0$: anche A ha inviato **1**.

Il contributo dell'altra stazione si annulla grazie all'ortogonalità dei codici (v. Tanenbaum per i dettagli).

- Funziona soprattutto sui canali **senza fili**; è usato da tempo in ambito militare (per le sue proprietà anti-disturbo) e nella telefonia cellulare (per esempio il 3G).
- ✖ È complesso da realizzare.
- ✖ Non scala bene con il numero di nodi.

Il segnale modulato

Sui canali radio i dati non viaggiano "così come sono": si trasmette un segnale in **banda base** (a bassa frequenza) modulando una **frequenza portante** più alta. Il telefono analogico, per esempio, usa la banda 300–3400 Hz. FDMA e CDMA lavorano proprio sulle frequenze portanti e sulla loro condivisione.

PROTOCOLLI AD ACCESSO CASUALE

Accesso casuale

Nei protocolli ad **accesso casuale** un nodo che trasmette usa sempre **tutta la velocità** del canale (R bit/s). Se avviene una collisione (almeno due nodi trasmettono insieme), ogni nodo coinvolto **aspetta un ritardo casuale** prima di ritrasmettere.

Poiché la scelta del ritardo è fatta indipendentemente, è possibile che i due nodi scelgano ritardi abbastanza diversi da permettere a uno dei due di "infilare" il proprio messaggio. Se invece scelgono ritardi simili, avviene una nuova collisione e il processo si ripete.

ALOHA puro

Il primo protocollo ad accesso casuale nacque alle Hawaii (ALOHAnet, anni '70), per collegare le isole via radio:

- ogni stazione trasmette **appena ha un frame** da inviare;
- una stazione centrale ripete ogni frame ricevuto: se non ci sono collisioni, ogni stazione riceve indietro il frame che ha appena trasmesso, e capisce così se è arrivato;
- in caso di collisione si attende un tempo di lunghezza casuale e si ritrasmette.

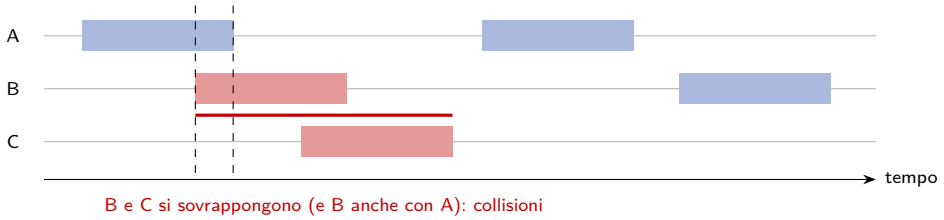


Figura 27.4 – ALOHA puro: basta che due frame si sovrappongano anche in parte perché entrambi vadano persi.

Slotted ALOHA

Slotted ALOHA

- Il tempo è diviso in **slot** (come in TDMA), ciascuno grande abbastanza da contenere un frame.
- I nodi sono **sincronizzati**: ogni nodo trasmette frame solo **all'inizio** di uno slot.
- Se nello slot non c'è collisione, la comunicazione prosegue.
- Se c'è una collisione, ogni nodo coinvolto ritrasmette il frame in ciascuno degli slot successivi **con probabilità** $p \in [0, 1]$ ("lancia una moneta"), finché il frame non viene inviato con successo.

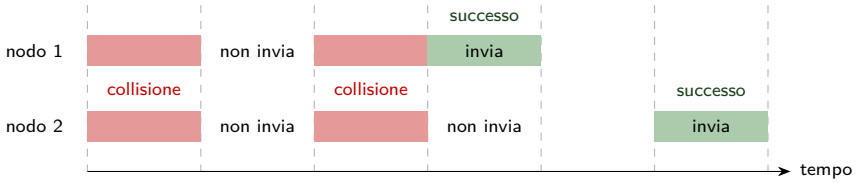


Figura 27.5 – Slotted ALOHA: dopo una collisione, in ogni slot ciascun nodo ritrasmette con probabilità p.

- ✔ Se c'è un solo nodo, usa tutta la velocità R del canale (niente collisioni).
- ✔ È **decentralizzato**: i nodi sono del tutto indipendenti tra loro, a parte la sincronizzazione.
- ✔ È **semplice** da realizzare ed eseguire (la scelta casuale è velocissima).
- ✔ Più collisioni consecutive sono improbabili.

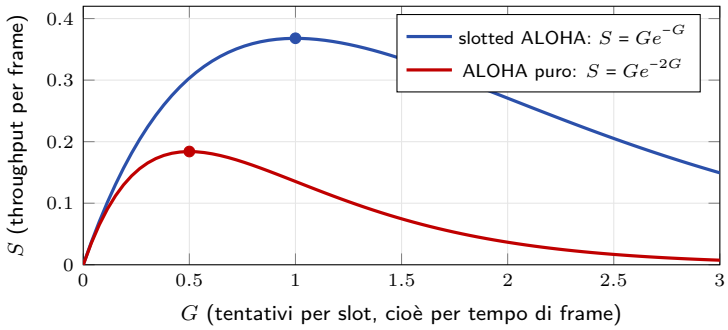


Figura 27.6 – Throughput di ALOHA: al meglio lo slotted ALOHA usa il canale per il 37% del tempo ($1/e$), l'ALOHA puro solo il 18% ($1/2e$).

CSMA e CSMA/CD

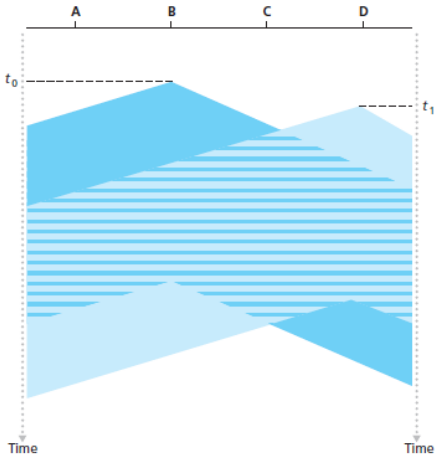
Un punto debole di ALOHA è che si può cominciare a trasmettere (producendo una collisione) anche se il canale **è già occupato**: ce ne accorgiamo solo dall'effetto delle collisioni. La soluzione è **ascoltare** il canale e tentare di trasmettere solo se è libero.

CSMA e CSMA/CD

I protocolli **CSMA** (*Carrier Sense Multiple Access*) e **CSMA/CD** (CSMA con *Collision Detection*) si basano su due principi:

- **carrier sensing**: i nodi ascoltano il canale **prima** di trasmettere. Se un altro nodo sta trasmettendo un frame, aspettano che la trasmissione finisca;
- **collision detection**: i nodi ascoltano il canale **mentre** trasmettono. Se rilevano una collisione, smettono di trasmettere e aspettano un tempo casuale prima di riprovare.

Ma se tutti i nodi ascoltano prima di trasmettere, perché avvengono comunque collisioni? Per il **ritardo di propagazione** del segnale.



- Anche se la propagazione è vicina alla velocità della luce, il segnale impiega del **tempo** per raggiungere gli altri nodi.
- Nella figura (spazio in orizzontale, tempo verso il basso) B comincia a trasmettere al tempo t_0 e il segnale si propaga in entrambe le direzioni. Al tempo t_1 D non ha ancora ricevuto il segnale di B, ascolta, trova il canale libero e **inizia a trasmettere**.
- Poco dopo i due segnali si incontrano: collisione. D se ne accorge solo quando riceve il segnale di B.

Per questo ritardo tra inizio della trasmissione e rilevazione, un nodo può considerare libero un canale che in realtà è già in uso.

Figura 27.7 – Collisione in CSMA dovuta al ritardo di propagazione (da Kurose).

CSMA puro

1. Controlla se il canale è occupato.
2. Se il canale è libero, invia un frame.

In CSMA puro le collisioni **non vengono rilevate**, anche se sono possibili: si capisce che un frame è andato perso solo perché **non arriva l'ACK**.

Varianti di CSMA

Cosa fare se il canale è occupato? Nel **CSMA 1-persistente** si aspetta che si liberi e si trasmette subito (con probabilità 1); nel **non persistente** si attende un tempo casuale e si riascolta; nel p**-persistente** (su canali a slot), quando il canale è libero si trasmette con probabilità p e si rimanda allo slot successivo con probabilità $1 - p$.

CSMA/CD

Il **CSMA/CD**, più sofisticato, è quello implementato in **Ethernet**:

1. controlla se il canale è occupato;
2. se il canale è libero, invia un frame;
3. **durante la trasmissione** controlla eventuali collisioni;
4. se rileva una collisione, **smette subito** di trasmettere e attende un periodo casuale $K \cdot$ (tempo di slot), con $K \in \{0, 1, \ldots, 2^n - 1\}$, dove n è il numero di collisioni già rilevate per quel frame (**binary exponential backoff**).

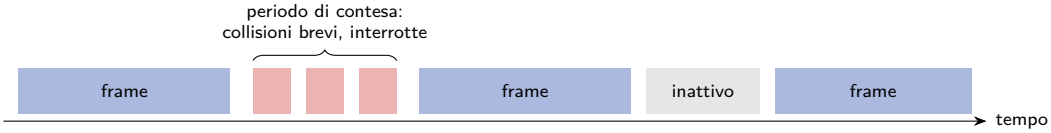


Figura 27.8 – Con la collision detection le collisioni durano pochissimo: le stazioni smettono appena se ne accorgono, e il canale torna disponibile.

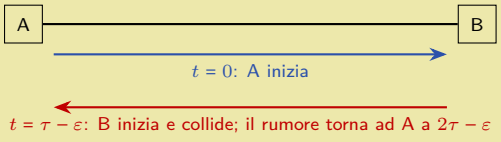
Esempio 2 — Il binary exponential backoff

- Dopo la $\mathbf{1^a}$ collisione: $K \in \{0, 1\}$.
- Dopo la $\mathbf{2^a}$: $K \in \{0, 1, 2, 3\}$.
- Dopo la $\mathbf{3^a}$: $K \in \{0, \ldots, 7\}$; dopo la n-esima: $K \in \{0, \ldots, 2^n - 1\}$ (in Ethernet n si ferma a 10).

Se due stazioni collidono una prima volta, con probabilità $1/2$ scelgono lo stesso K e collidono di nuovo; alla seconda la probabilità scende a $1/4$, poi a $1/8$. La probabilità di riuscire a inviare il frame **cresce con il numero di collisioni**, adattando l'attesa al carico: con poche stazioni le attese sono brevi, con molte si allungano.

La lunghezza minima dei frame in Ethernet

Perché la collision detection funzioni, una stazione deve essere **ancora in trasmissione** quando le arriva il segnale della collisione. Sia τ il tempo di propagazione tra le due stazioni più lontane. Nel caso peggiore A inizia al tempo 0, e il suo segnale arriva a B al tempo $\tau - \varepsilon$, un attimo dopo che B ha iniziato a trasmettere; B rileva subito la collisione, ma il "rumore" della collisione torna ad A solo al tempo $2\tau - \varepsilon$. Quindi il tempo di trasmissione di un frame deve essere di almeno 2τ: per questo Ethernet ha un campo **pad** che allunga artificialmente i frame troppo corti (dati da almeno 46 byte).



Protocolli a turni

Polling

Protocollo di polling

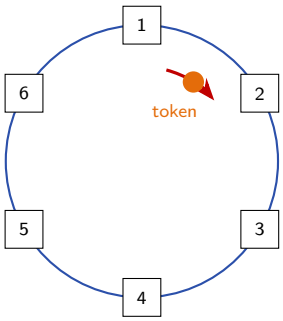
C'è un nodo **master** che, a turno (*round-robin*), seleziona un nodo alla volta a cui permettere di trasmettere (fino a una quantità massima). Il processo si ripete ogni volta che la trasmissione termina. È il caso di Bluetooth.

- ✔ Non ci sono collisioni.
- ✖ C'è un **ritardo di polling** (il tempo per selezionare i nodi).
- ✖ L'approccio è **centralizzato**: il master è un single point of failure.

Token passing

Protocollo a passaggio di token

Non c'è un master: i nodi si scambiano uno speciale frame detto **token** (gettone). Chi riceve il token **può** (ma non deve) trasmettere, poi passa il token al nodo successivo.



La stazione che ha il token, passato lungo l'anello, può trasmettere. Anche i frame di dati viaggiano sull'anello da una stazione alla successiva. L'anello può essere solo **logico**: a livello fisico le stazioni possono essere collegate a un bus, ma a livello logico c'è sempre il passaggio del token.

Figura 27.9 – Token ring: il gettone circola e dà il diritto di trasmettere.

- ✔ Non ci sono collisioni.
- ✔ L'approccio è **decentralizzato**.
- ✖ Ci sono problemi se un nodo **dimentica di rilasciare** il token, monopolizzando il collegamento, o se il token si perde. I protocolli di rilascio hanno diverse varianti, ma è importante che esista un modo per **recuperare un token perso o bloccato** (per esempio se una stazione smette di funzionare mentre lo custodisce).

	Partizionamento	**Accesso casuale**	**A turni**
Esempi	TDMA, FDMA, CDMA	ALOHA, CSMA, CSMA/CD	polling, token passing
Collisioni	no	sì, gestite	no
Carico basso	spreco (banda fissa R/N)	ottimo (tutta la banda)	buono
Carico alto	ottimo, equo	molte collisioni	buono
Punto debole	banda inutilizzata	collisioni	master o token

Tabella 27.1 – Le tre famiglie di protocolli ad accesso multiplo.

Terminali nascosti ed esposti

Nelle reti **senza fili** il carrier sensing non basta: un nodo sente solo le stazioni nel proprio **raggio radio**, e ciò che conta è l'interferenza **presso il ricevente**, non presso il mittente.

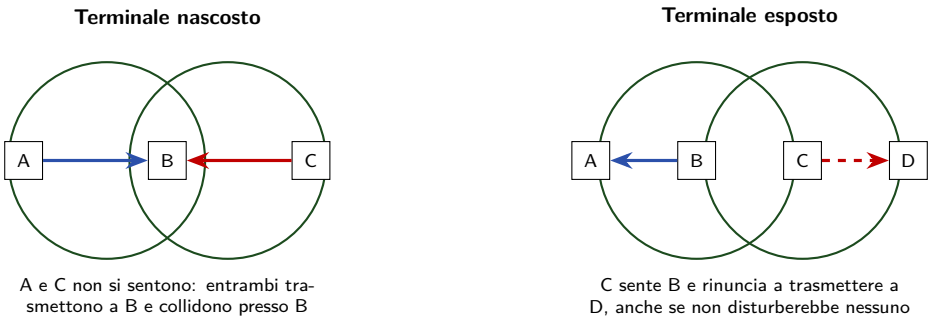


Figura 27.10 – I cerchi indicano il raggio radio di B e C.

- **Terminali nascosti**: mittenti che non riescono a sentirsi a vicenda ma collidono comunque presso il ricevente. A e C trasmettono entrambi a B; poiché A non sente C, il carrier sensing di A dice "libero", e i frame collidono in B. Si vuole evitare questa perdita di efficienza.
- **Terminali esposti**: mittenti che si sentono a vicenda ma potrebbero comunque trasmettere senza problemi a riceventi diversi. B trasmette ad A; C sente B e rimanda la propria trasmissione verso D, anche se D è fuori dalla portata di B. Si vorrebbe permettere questa contemporaneità, che migliora le prestazioni.

MACA

MACA (Multiple Access with Collision Avoidance)

Il protocollo **MACA** concede l'accesso al canale con un breve scambio preliminare:

1. il mittente A invia un breve frame **RTS** (*Request To Send*) a B, indicando la lunghezza del frame da inviare;
2. il ricevente B risponde con un frame **CTS** (*Clear To Send*), anch'esso con la lunghezza;
3. A trasmette il frame.

Chi sente l'RTS resta in silenzio per il tempo del CTS; chi sente il CTS resta in silenzio per tutta la durata del frame.

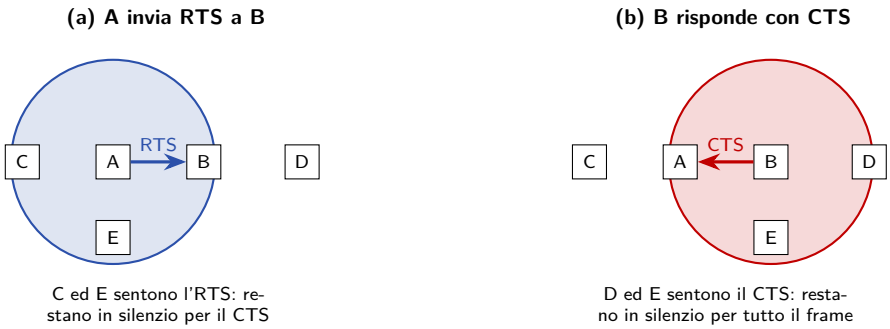


Figura 27.11 – MACA: RTS e CTS "prenotano" il canale attorno al mittente e al ricevente.

- **Terminale nascosto risolto**: D non sente A, ma sente il CTS di B, e quindi non disturba la ricezione di B.
- **Terminale esposto risolto**: C sente l'RTS di A ma non il CTS di B: sa quindi di essere fuori dalla portata del ricevente e, dopo il CTS, può trasmettere senza disturbare.
- Le collisioni restano possibili tra RTS (per esempio due RTS inviati insieme a B), ma riguardano frame piccolissimi: si risolvono con un backoff casuale e costano poco. Una versione evoluta del meccanismo è usata nel Wi-Fi (802.11).

Riepilogo

In breve

- Sui collegamenti **broadcast** trasmissioni contemporanee **collidono**; il sottolivello **MAC** regola l'accesso.
- Proprietà desiderate: R/M a testa (e R se si è soli), decentralizzato, semplice.
- **Partizionamento**: TDMA (slot), FDMA (frequenze), CDMA (codici ortogonali); niente collisioni ma banda fissa R/N.
- **Accesso casuale**: ALOHA puro e slotted (ritrasmissione con probabilità p), CSMA (ascolta prima), CSMA/CD di Ethernet (ascolta anche durante, backoff esponenziale binario $K \in \{0..2^n - 1\}$, frame lunghi almeno 2τ).
- **A turni**: polling (master, Bluetooth) e token passing (token ring).
- Nel wireless: terminali **nascosti** ed **esposti**; **MACA** li affronta con RTS/CTS.

Indirizzi MAC, switch, ARP ed Ethernet

Gli indirizzi MAC

Indirizzo MAC

A livello di collegamento i dispositivi sono identificati dagli **indirizzi MAC** (o **indirizzi fisici**).

- Ogni interfaccia di rete ha un proprio indirizzo MAC (pensato per essere fisso, anche se si può cambiare).
- Ogni produttore ha un proprio insieme di indirizzi MAC (i primi 3 byte identificano il produttore).
- Un indirizzo MAC è lungo **6 byte** (2^{48} indirizzi possibili) e si scrive di solito in esadecimale: `1A:23:F9:CD:06:9B` oppure `1A-23-F9-CD-06-9B`.

	Indirizzo IP	**Indirizzo MAC**
Livello	rete (3)	collegamento (2)
Lunghezza	32 bit (IPv4)	48 bit
Notazione	decimale puntata `192.168.1.10`	esadecimale `1A:23:F9:CD:06:9B`
Portata	**globale** (fuori dalle LAN)	**locale** (solo nella LAN)
Assegnazione	dipende dalla rete in cui ci si trova	dal produttore della scheda
Analogia	l'indirizzo di casa (cambia se traslochi)	il codice fiscale (ti segue ovunque)

Tabella 28.1 – Indirizzi IP e MAC a confronto.

Gli indirizzi MAC sono locali

Tutte le interfacce hanno un indirizzo MAC, ma questo è usato **solo all'interno di una LAN**: un frame non esce dalla propria rete locale. Gli indirizzi IP, invece, fuori dalle LAN sono globali. Mentre il datagramma IP viaggia dalla sorgente alla destinazione mantenendo gli stessi indirizzi IP, gli indirizzi MAC del frame **cambiano a ogni hop**.

Come lavorano gli indirizzi MAC

In una LAN, due interfacce A e B comunicano così:

- A inserisce nel frame l'indirizzo MAC di B e lo trasmette;
- B riceve il frame e confronta il proprio indirizzo MAC con il MAC di destinazione del frame;
- se coincidono il frame viene **accettato**, altrimenti viene **scartato** (e il resto della pila non viene nemmeno coinvolto).

Indirizzo di broadcast

Si possono anche inviare messaggi **broadcast**, accettati da tutti indipendentemente dal MAC. Nelle LAN con indirizzi da 6 byte (come Ethernet e 802.11) l'indirizzo broadcast è una stringa di **48 uni**: `FF-FF-FF-FF-FF-FF`.

In una LAN è del tutto possibile (anzi frequente) che un'interfaccia riceva frame destinati a un'altra: il ruolo dell'indirizzo MAC è **filtrare** i frame non destinati a sé, senza disturbare l'host.

Indirizzi e livelli

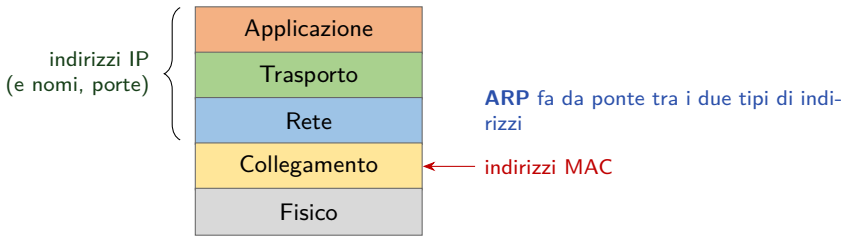


Figura 28.1 – Nella pila IP e MAC convivono: ARP collega gli uni agli altri (una differenza rispetto al modello ISO/OSI puro).

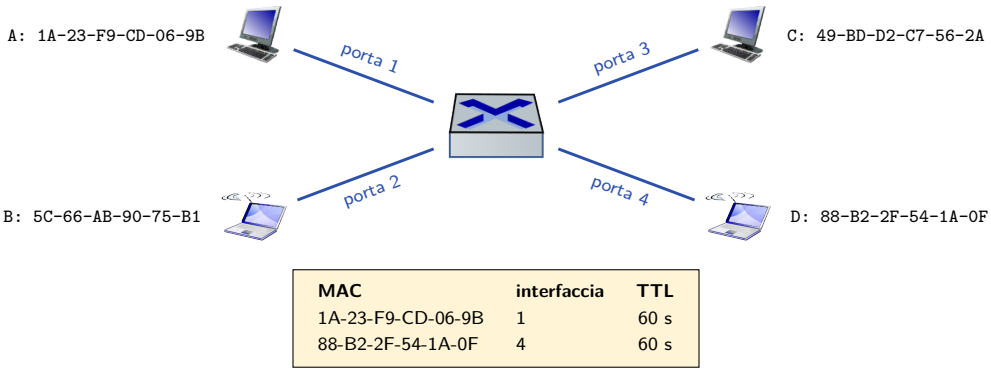
Gli switch

Switch

Gli **switch** sono l'equivalente dei router al livello di collegamento:
- non implementano alcun algoritmo di routing;
- usano **solo indirizzi MAC**, non indirizzi IP.

Il loro compito è ricevere i frame in arrivo e **inoltrarli sui collegamenti in uscita** giusti.

- Lo switch è **trasparente** a host e router della subnet: nessuno gli indirizza frame, e nessuno si accorge che c'è.
- Uno switch ha dei **buffer** sulle interfacce.
- Gli switch hanno **tabelle di inoltro** che associano indirizzi MAC a interfacce.
- La tabella si aggiorna **automaticamente e dinamicamente** man mano che si scoprono nuovi dispositivi (**self-learning**).



MAC	interfaccia	TTL
1A-23-F9-CD-06-9B	1	60 s
88-B2-2F-54-1A-0F	4	60 s

Figura 28.2 – Uno switch e la sua tabella, costruita osservando i frame che arrivano.

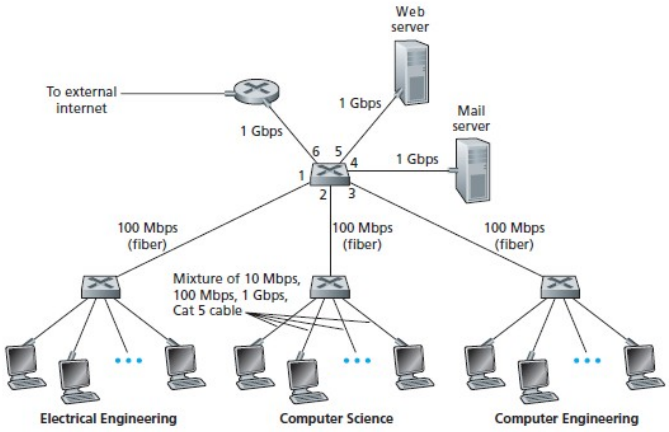
1. Quando arriva un frame sulla porta x, lo switch **impara** che il MAC **sorgente** si trova dietro la porta x e lo registra nella tabella (con un tempo di vita).
2. Poi cerca nella tabella il MAC di **destinazione**:
   - se lo trova e la porta è diversa da x, **inoltra** il frame solo su quella porta;
   - se la porta coincide con x, **scarta** il frame (il destinatario è dallo stesso lato: lo ha già ricevuto);
   - se non lo trova, **inonda** (*flooding*): invia il frame su tutte le porte tranne x.

Esempio 1 — Lo switch impara

Tabella vuota. A (porta 1) invia un frame a D.

- Lo switch impara "A è sulla porta 1". Non conosce D: inonda le porte 2, 3 e 4.
- D risponde ad A. Lo switch impara "D è sulla porta 4"; conosce A: inoltra solo sulla porta 1.
- Da ora in poi i frame tra A e D viaggiano solo tra le porte 1 e 4.

LA LAN COMMUTATA



È comune che le LAN usino uno o più switch per collegare molti dispositivi locali (nell'esempio del Kurose, i dipartimenti di un istituto collegati da switch, con un router verso Internet e i server Web e di posta).

Figura 28.3 – Una rete di istituto con switch organizzati ad albero (da Kurose).

A differenza dei router, gli switch sono **veloci** e **plug-and-play**:

- non c'è alcun algoritmo di routing;
- si considerano solo i primi **2 livelli** della pila.

D'altra parte le LAN commutate sono **limitate in dimensione** e devono essere **strutturate ad albero**:

- gli indirizzi MAC sono difficili da raggruppare (non hanno prefissi legati alla posizione): le tabelle degli switch possono crescere rapidamente;
- non c'è routing: i **cicli** sono difficili da evitare (un frame inondato in un ciclo girerebbe per sempre).

	Switch	Router
Livello	collegamento (2)	rete (3)
Indirizzi	MAC	IP
Tabelle	self-learning	algoritmi di routing
Configurazione	plug-and-play	va configurato
Topologia	albero (niente cicli)	qualsiasi, anche con cicli
Traffico broadcast	lo inoltra	lo blocca (separa i domini)

Tabella 28.2 – Switch e router a confronto.

Il protocollo ARP

I protocolli dei livelli superiori lavorano con gli indirizzi IP, mentre il frame ha bisogno dell'indirizzo MAC del destinatario. Serve quindi tradurre gli indirizzi IP in indirizzi MAC.

ARP (Address Resolution Protocol)

L'**ARP** gestisce la conversione tra indirizzi IP e indirizzi MAC.

- Ogni interfaccia ha un modulo ARP con una **tabella ARP**, che associa ogni IP della LAN a un indirizzo MAC, con un **TTL** (*time to live*) scaduto il quale la voce viene cancellata (tipicamente 5–20 minuti).
- Poiché gli indirizzi MAC sono locali, anche ARP funziona **solo nelle reti locali**.

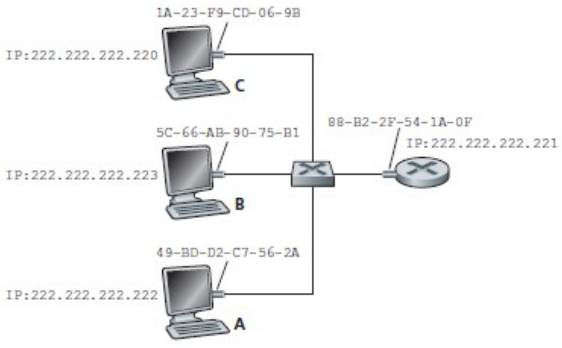
Indirizzo IP	Indirizzo MAC	TTL
222.222.222.221	88-B2-2F-54-1A-0F	13:45:00
222.222.222.223	5C-66-AB-90-75-B1	13:52:00

Tabella 28.3 – Una tabella ARP: la voce scade all'orario indicato.

ARP e DNS

ARP fa per la LAN quello che il DNS fa per Internet: il DNS traduce **nomi** in indirizzi IP (in tutta Internet), ARP traduce **indirizzi IP** in indirizzi MAC (solo nella propria LAN). Sulle proprie macchine si può vedere la tabella con `ip neigh` (o `arp -a`).

Un esempio di ARP



L'host C (`222.222.222.220`) vuole inviare messaggi all'host A (`222.222.222.222`). Per farlo deve conoscere anche il MAC di A.

1. Se A **non è nella tabella**, prima di inviare il messaggio C invia in **broadcast** a tutti i dispositivi della rete un pacchetto **ARP request**: "chi ha l'IP `222.222.222.222`? Lo dica a `222.222.222.220`".
2. Tutti i nodi ricevono il pacchetto, ma **solo** chi ha l'IP cercato risponde, con un messaggio **ARP reply** diretto (*unicast*) che contiene il proprio MAC.
3. C inserisce la coppia IP–MAC nella propria tabella e invia il frame.

Figura 28.4 – ARP in una LAN: tre host e un router collegati da uno switch (da Kurose).

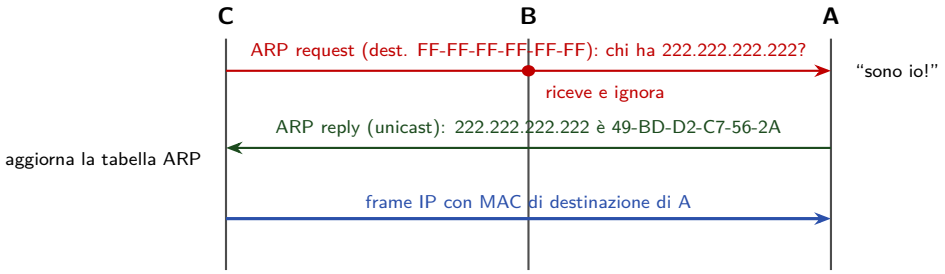


Figura 28.5 – Richiesta in broadcast, risposta in unicast.

ARP poisoning

ARP non prevede alcuna autenticazione: un host **malevolo** può intercettare la richiesta e rispondere con una risposta **falsificata**, associando l'IP cercato al proprio MAC (**ARP poisoning**). I frame destinati alla vittima arrivano così all'attaccante, che può leggerli o modificarli (*man in the middle*). È uno degli attacchi più comuni nelle LAN.

ARP e il gateway

Che succede se l'IP di destinazione è **fuori dalla rete**?

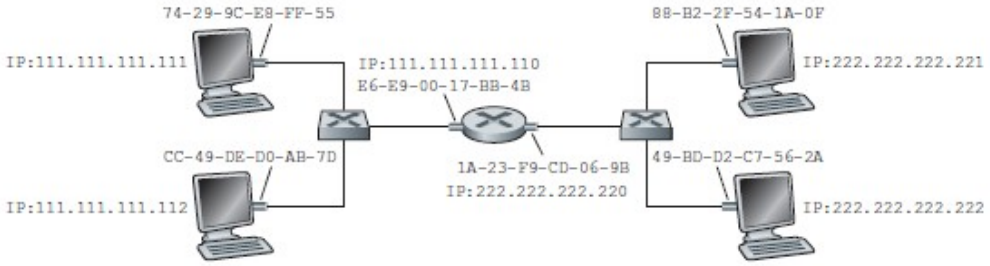


Figura 28.6 – Due subnet (111.111.111.x e 222.222.222.x) collegate da un router con due interfacce, ciascuna con il proprio IP, MAC e tabella ARP.

- Il router che collega le due reti deve avere almeno **2 interfacce** (2 IP, 2 MAC e 2 tabelle ARP), una dentro ciascuna subnet.
- I frame diretti fuori dalla subnet vengono inviati alla **prima interfaccia del router**: l'host usa ARP per il MAC del **router** (il gateway), non per quello del destinatario finale.
- Il router estrae il datagramma, lo sposta sulla seconda interfaccia e, con la seconda tabella ARP, lo invia all'host giusto in un **nuovo frame**.

Esempio 2 — Da A (111.111.111.111) a B (222.222.222.222)

1. A capisce, confrontando indirizzo e maschera, che B è in un'altra subnet: deve passare dal gateway `111.111.111.110`.
2. A usa ARP per trovare il MAC dell'interfaccia del router `111.111.111.110` (`E6-E9-00-17-BB-4B`).
3. A crea un datagramma IP con **IP sorgente A** e **IP destinazione B**, e lo incapsula in un frame con **MAC destinazione = MAC del router**.
4. Il router riceve il frame, estrae il datagramma, vede che B è sulla subnet `222.222.222.0/24`, usa ARP su quell'interfaccia per trovare il MAC di B e crea un nuovo frame con **MAC sorgente = sua seconda interfaccia** e **MAC destinazione = B**.

Tratta	MAC sorgente	MAC destinazione	IP sorgente	IP destinazione
A → router	A	router (if. 1)	A	B
router → B	router (if. 2)	B	A	B

Gli indirizzi IP restano gli stessi da un estremo all'altro; gli indirizzi MAC cambiano a ogni collegamento.

Il frame Ethernet

Ethernet è la tecnologia dominante per le LAN cablate. Nata negli anni '70 come bus condiviso con CSMA/CD, oggi è quasi sempre una topologia a stella con uno switch al centro e collegamenti full-duplex (dove le collisioni non avvengono più).

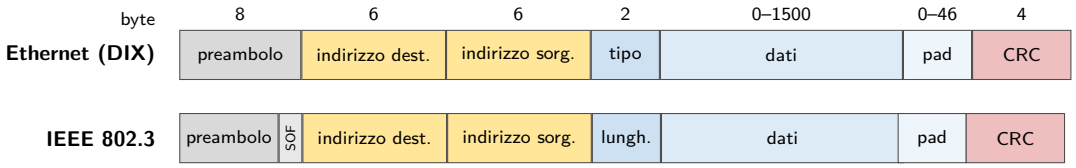


Figura 28.7 – Il frame Ethernet nelle due varianti (DIX e IEEE 802.3).

- **Preambolo** (8 byte): un blocco "sveglia" di bit alternati (`10101010` …, con l'ultimo byte `10101011`) usato per **sincronizzare gli orologi** degli adattatori di sorgente e destinazione.
- **Indirizzo di destinazione** (6 byte): il MAC dell'adattatore di destinazione.
- **Indirizzo sorgente** (6 byte): il MAC dell'adattatore che trasmette il frame nella LAN.
- **Tipo** (2 byte): il protocollo di livello di rete contenuto nel frame. Non c'è solo IP (`0x0800`): per esempio i pacchetti ARP hanno un tipo specifico, `0x0806`. Nella variante 802.3 al suo posto c'è la lunghezza.
- **Dati** (da 46 a 1500 byte): il datagramma IP. Il massimo è la **MTU** di Ethernet: se il datagramma è più grande, va frammentato.
- **Pad**: riempimento per raggiungere la lunghezza minima dei dati (46 byte), necessaria perché il CSMA/CD rilevi le collisioni (tempo di trasmissione di almeno 2τ).
- **CRC** (4 byte): il CRC-32 calcolato sul frame, per rilevare gli errori.

Un servizio senza connessione e inaffidabile

Ethernet è **senza connessione** (nessun handshake con l'adattatore di destinazione) e **inaffidabile**: il ricevente non invia né ACK né NAK. Un frame che non supera il controllo CRC viene semplicemente scartato; se i dati servono davvero, sarà TCP (o l'applicazione) a accorgersi della mancanza e a ritrasmettere.

Riepilogo

In breve

- L'**indirizzo MAC** (48 bit, esadecimale) identifica un'interfaccia ed è **locale**; broadcast = `FF-FF-FF-FF-FF-FF`.
- Gli **switch** inoltrano frame in base al MAC, con tabelle **self-learning**; sono veloci e plug-and-play ma le LAN commutate devono essere ad albero.
- **ARP** traduce IP in MAC: richiesta in broadcast, risposta in unicast, tabella con TTL; vulnerabile all'**ARP poisoning**.
- Verso un'altra subnet il frame va al MAC del **router**: gli IP restano uguali, i MAC cambiano a ogni hop.
- **Frame Ethernet**: preambolo (8), destinazione (6), sorgente (6), tipo (2), dati (46–1500, con pad), CRC (4).

Parte VII

Sicurezza di rete

Attacchi alle reti e fondamenti di crittografia

Perché la sicurezza di rete

Alcune applicazioni scambiano **informazioni sensibili** (credenziali, dati personali, …), e sulle reti pubbliche i messaggi dovrebbero essere protetti: è il motivo per cui quasi ogni protocollo applicativo ha una variante sicura (HTTPS, SMTPS, SSH, FTPS).

Sicurezza di rete

La **sicurezza di rete** è il campo che studia i possibili **attacchi** alle reti e i modi per **prevenirli**.

- Le reti fanno parte della nostra vita (la connessione a Internet è considerata quasi come l'acqua o la corrente), e una grande quantità dei nostri dati sensibili viaggia in rete.
- Esistono molti tipi di attacchi, con scopi e meccanismi diversi.
- Gli attacchi evolvono insieme alla tecnologia e al pubblico delle reti.

I malware

Malware

Un **malware** è un software malevolo che può arrivare su un computer attraverso la rete (un file scaricato, un allegato di posta, …).

Una volta infettato un dispositivo, un malware può fare danni in molti modi:

Tipo	Cosa fa
Adware	costringe il sistema a mostrare pubblicità
Scareware	mostra falsi allarmi per indurre l'utente a scaricare altri malware
Wiper / ransomware	cancella / **cifra** i file (e chiede un riscatto per restituirli)
Spyware	raccoglie informazioni private (password, numeri di documento, …); per esempio i **keylogger** registrano i tasti premuti
Rootkit	ottiene i privilegi di amministratore (root) del sistema
Zombie / botnet	trasforma il dispositivo in uno "schiavo", una testa di ponte per attaccare altri dispositivi

Tabella 29.1 – Alcune famiglie di malware.

I malware di oggi sono spesso **auto-replicanti**: infettato un host, da lì cercano di entrare in altri host su Internet, e da quelli in altri ancora. Si diffondono in due forme:

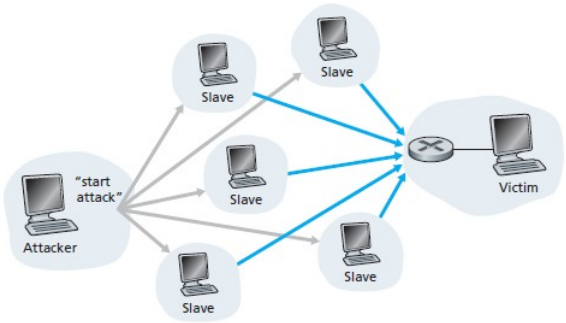
Virus e worm

- Un **virus** richiede una qualche **interazione dell'utente** per infettare il dispositivo. Per esempio un allegato con codice eseguibile malevolo che, una volta aperto, si replica inviando mail simili ai contatti dell'utente. I virus sono spesso camuffati da componenti software legittimi (**cavalli di Troia**, *trojan horses*).
- Un **worm** entra in un dispositivo **senza alcuna interazione** esplicita dell'utente. Per esempio l'utente sta eseguendo un'applicazione di rete vulnerabile che accetta il worm senza intervento; il worm poi scansiona la rete alla ricerca di altri host con la stessa applicazione.

Gli attacchi DoS

Denial of Service (DoS)

Gli attacchi **DoS** (negazione del servizio) sono molto comuni e hanno lo scopo di rendere una rete, un host o un altro pezzo di infrastruttura (server Web, DNS, …) **inutilizzabile dagli utenti legittimi**.



Ci sono tre tipi di attacchi DoS:

1. **Attacco a una vulnerabilità**: si inviano messaggi costruiti ad arte a un'applicazione o a un sistema operativo vulnerabile per bloccarlo o farlo andare in crash.
2. **Bandwidth flooding**: si inonda l'host bersaglio di pacchetti, impedendo a quelli legittimi di raggiungerlo.
3. **Connection flooding**: si aprono moltissime connessioni TCP, mezze aperte o complete, verso il bersaglio, che smette di accettare connessioni legittime (per esempio il *SYN flood*).

Figura 29.1 – Un DDoS (*Distributed DoS*): l'attaccante comanda una botnet di zombie che attaccano la vittima tutti insieme.

Perché "distribuito"

Un solo attaccante ha banda limitata ed è facile da bloccare filtrando il suo indirizzo. Con migliaia di zombie sparsi per il mondo il traffico è enorme e proviene da indirizzi diversi, quindi è molto più difficile da distinguere da quello legittimo.

Il packet sniffing

Packet sniffing

Lo **sniffing** prevede un ricevitore **passivo** (lo *sniffer*) che registra una copia dei pacchetti interessanti di un host bersaglio, cercando di rubare informazioni sensibili (*eavesdropping*, origliare).

- Gli sniffer si possono usare in qualunque rete **broadcast** (cablata o senza fili), semplicemente copiando i pacchetti destinati ad altri invece di scartarli.
- Nelle reti non broadcast uno sniffer può essere inserito in un malware (spyware) che infetta i dispositivi di rete (per esempio i router), in modo che tutto il traffico inoltrato venga anche copiato.
- Poiché gli sniffer sono **passivi** (non immettono traffico nella rete), sono **difficilissimi da rilevare**.
- Per difendersi dallo sniffing si usa la **crittografia**.

Su Internet ci sono molti sniffer liberamente disponibili: un esempio notevole è **Wireshark** (ex Ethereal), che abbiamo usato nelle esercitazioni.

L'IP SPOOFING

IP spoofing

L'**IP spoofing** è la tecnica con cui un host malevolo immette nella rete pacchetti con **indirizzo sorgente falso** (contraffatto).

- Si può usare, insieme alle vulnerabilità di un'applicazione, per attaccare specifici host fingendosi un altro utente.
- Si può usare anche per attacchi DoS (alternativa alle botnet): messaggi da indirizzi sorgente diversi sono più difficili da filtrare.
- Spoofing e sniffing insieme permettono attacchi **man-in-the-middle** (MitM): l'attaccante si mette tra due host che comunicano, fingendosi l'uno con l'altro. I due host credono di parlare tra loro, ma in realtà parlano con l'attaccante.
- Contro lo spoofing servono controlli di **integrità dei messaggi** e di **autenticazione degli estremi**, che permettono di stabilire se un messaggio non è stato modificato e se proviene davvero dalla sorgente dichiarata.

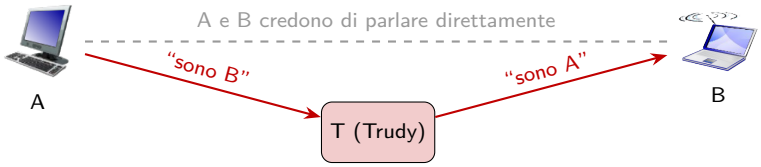


Figura 29.2 – L'attacco man-in-the-middle: T finge di essere B con A e di essere A con B, intercettando i messaggi (spesso aiutandosi con lo spoofing).

LE PROPRIETÀ DI UNA COMUNICAZIONE SICURA

Visti i tipi di attacco, possiamo definire le proprietà che una comunicazione sicura dovrebbe garantire.

Le quattro proprietà

- **Confidenzialità**: solo il mittente e il destinatario previsto devono poter capire il contenuto del messaggio (contro lo *sniffing*).
- **Integrità del messaggio**: il contenuto non deve essere alterato, né in modo malevolo né per errore.
- **Autenticazione degli estremi** (*end-point authentication*): mittente e destinatario devono poter confermare l'identità dell'altra parte (contro lo *spoofing*).
- **Sicurezza operativa**: un'infrastruttura di rete che impedisca agli host malevoli di infilarsi nella comunicazione.

Le prime tre proprietà sono realizzate **via software**; l'ultima (sicurezza operativa) si basa tipicamente su **hardware** specifico (firewall, sistemi di rilevazione delle intrusioni).

Attacco	Proprietà violata	Contromisura
Sniffing	confidenzialità	crittografia
Alterazione dei messaggi	integrità	hash crittografici, MAC
Spoofing, replay	autenticazione	nonce, certificati
DoS, intrusioni, malware	sicurezza operativa	firewall, IDS

Tabella 29.2 – Attacchi, proprietà e contromisure.

Introduzione alla crittografia

Crittografia

Una tecnica **crittografica** permette a un mittente di **camuffare i dati** in modo che siano incomprensibili per un intruso: l'intruso non deve ricavare alcuna informazione dai dati intercettati, mentre il destinatario deve poter recuperare i dati originali.

- Nella sua forma iniziale il messaggio si chiama **testo in chiaro** (*plaintext* o *cleartext*) ed è leggibile da tutti.
- Prima di immetterlo nel canale, il mittente usa un **algoritmo di cifratura** che lo trasforma in una forma illeggibile, il **testo cifrato** (*ciphertext*), che va **decifrato** alla ricezione.

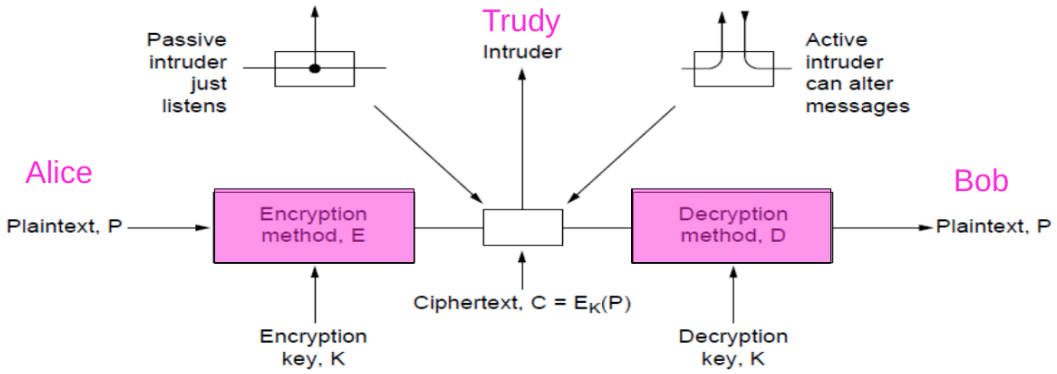


Figura 29.3 – Il modello di cifratura (da Tanenbaum): Alice cifra con la chiave K_E, Bob decifra con K_D; Trudy può ascoltare passivamente o intervenire attivamente.

Con la notazione del Tanenbaum:

- K è la **chiave**, P il messaggio in chiaro (*plaintext*), C il messaggio cifrato (*ciphertext*);
- la cifratura con chiave K si scrive $E_K(P) = C$ e la decifratura $D_K(C) = P$, quindi $D_K(E_K(P)) = P$.

Le chiavi e il principio di Kerckhoffs

Nei sistemi crittografici moderni, compresi quelli di Internet, la tecnica di cifratura è **nota e standard per tutti** (anche per l'intruso). La parte sconosciuta sono le **chiavi di cifratura e decifratura**.

Chiave

Una **chiave** è una stringa alfanumerica che va fornita all'algoritmo di cifratura o decifratura per cifrare o decifrare i messaggi. Le chiavi di cifratura e decifratura possono essere **identiche** (crittografia *simmetrica*) o **diverse** (*asimmetrica*).

Principio di Kerckhoffs

Tutti gli algoritmi devono essere pubblici; solo le chiavi sono segrete. La "sicurezza per oscurità" (*security by obscurity*), cioè tenere segreto l'algoritmo, non funziona mai: gli algoritmi pubblici vengono studiati e attaccati da migliaia di esperti, e quelli che resistono sono davvero robusti.

I problemi della crittoanalisi

Chi cerca di "rompere" un cifrario (il *crittoanalista*) può trovarsi in tre situazioni, dalla più difficile alla più favorevole:

1. **solo testo cifrato** (*ciphertext only*): ha a disposizione solo testi cifrati;
2. **testo in chiaro noto** (*known plaintext*): ha alcuni testi cifrati e i corrispondenti testi in chiaro;
3. **testo in chiaro scelto** (*chosen plaintext*): può cifrare a piacere testi in chiaro scelti da lui.

Un buon cifrario deve resistere anche nel terzo caso.

Cifrari per sostituzione

Cifrario per sostituzione

In un cifrario per **sostituzione** ogni simbolo viene sostituito con un altro. La chiave è la **corrispondenza tra simboli**.

testo in chiaro:	a b c d e f g h i j k l m n o p q r s t u v w x y z
testo cifrato:	Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

Con questa chiave `attacco` diventa `QZZQEEG`. Anche se le chiavi possibili sono $26! \approx 4 \cdot 10^{26}$, per l'alfabeto latino (e altri) questi cifrari sono **facilmente attaccabili**: si sfruttano le **regolarità statistiche** della lingua (in italiano le lettere più frequenti sono e, a, i, o; le doppie sono frequenti; certe coppie di lettere sono comunissime).

Cifrari per trasposizione

Cifrario per trasposizione

In un cifrario per **trasposizione** i simboli **non cambiano**, ma vengono **riordinati**. Nel cifrario a trasposizione per colonne la chiave è una parola senza lettere ripetute: le sue lettere, in ordine alfabetico, danno l'ordine con cui leggere le colonne.

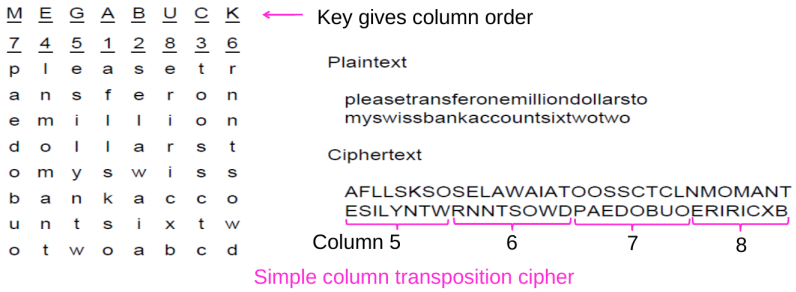


Figura 29.4 – Trasposizione per colonne con chiave MEGABUCK (da Tanenbaum): il testo si scrive per righe e si legge colonna per colonna, nell'ordine indicato dai numeri.

LA CRITTOGRAFIA SIMMETRICA

Crittografia simmetrica

Nella crittografia **simmetrica** c'è **una sola chiave**, usata sia per cifrare sia per decifrare.

- **Cifratura**: il testo in chiaro, insieme alla chiave, viene passato all'algoritmo di cifratura, che genera un testo cifrato che può viaggiare in rete in sicurezza.
- **Decifratura**: il testo cifrato, insieme alla chiave, viene passato all'algoritmo di decifratura, che ricrea il testo in chiaro.

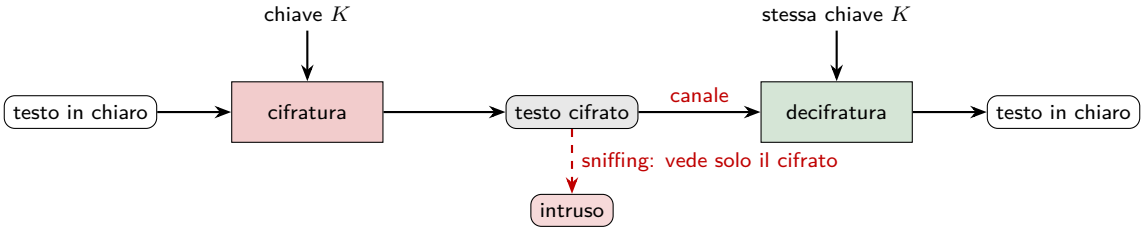


Figura 29.5 – Crittografia simmetrica: la stessa chiave serve a cifrare e a decifrare.

Il problema dello scambio delle chiavi

Come ci si scambia la chiave?

Il problema della crittografia simmetrica (a chiave singola) è che la chiave segreta deve essere **comunicata** (*key exchange problem*).

- Gli host possono usare un **canale sicuro** per scambiarsi la chiave.
- Gli host possono usare un **protocollo** che permetta loro di "convergere" su una chiave condivisa.

Se due parti non riescono a stabilire in modo sicuro la chiave iniziale, non potranno comunicare in sicurezza: un terzo che abbia intercettato la chiave durante lo scambio potrà leggere tutti i messaggi. Inoltre, con N utenti che vogliono parlare a coppie servono $\frac{N(N-1)}{2}$ chiavi, una per ogni coppia (dell'ordine di N^2).

Il one-time pad

One-time pad

Il **one-time pad** è un cifrario **inviolabile**: si sceglie come chiave (il *pad*) una stringa di bit **casuale**, lunga quanto il messaggio, e si cifra facendo lo XOR bit a bit: $C = P \oplus K$. Per decifrare si rifà lo XOR con lo stesso pad: $C \oplus K = P \oplus K \oplus K = P$.

Esempio 1 — Cifrare e decifrare con lo XOR

Con $P = \texttt{0111}$ e $K = \texttt{1101}$:

$$\begin{aligned} &\text{cifratura} && P \oplus K = \texttt{0111} \oplus \texttt{1101} = \texttt{1010} = C \\ &\text{decifratura} && C \oplus K = \texttt{1010} \oplus \texttt{1101} = \texttt{0111} = P \end{aligned}$$

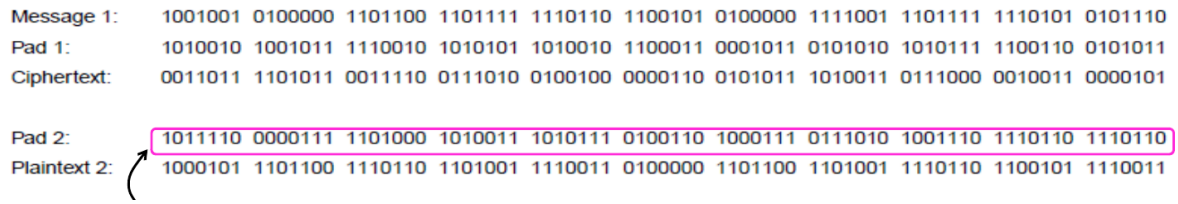


Figura 29.6 – Con il one-time pad, dallo stesso testo cifrato si ottengono testi in chiaro diversi usando pad diversi (da Tanenbaum): senza il pad, ogni messaggio della stessa lunghezza è ugualmente probabile.

Il one-time pad introduce una grande quantità di ridondanza e impedisce che qualsiasi informazione del testo in chiaro P si rifletta nel testo cifrato C. Ha però problemi pratici:

1. **condivisione del pad**: il pad (la chiave) va distribuito in sicurezza;
2. **errori di sincronizzazione**: se il messaggio ha uno spostamento (un bit perso), la decifratura fallisce da lì in poi;
3. il punto vero: il pad ha la **stessa dimensione del messaggio**. Se si ha un modo sicuro di trasmettere il pad, tanto vale trasmettere direttamente il messaggio (salvo casi particolari, come uno scambio di persona del pad su un supporto fisico).

One-time vuol dire una volta sola

Un pad **non deve mai essere usato più di una volta**: se due messaggi sono cifrati con lo stesso pad, $C_1 \oplus C_2 = P_1 \oplus P_2$, e da questo l'intruso può ricavare molto sui due testi in chiaro.

Due principi fondamentali

I due principi della crittografia

1. **Ridondanza**: ogni messaggio cifrato deve contenere una qualche forma di ridondanza. Altrimenti un attaccante potrebbe inviare un messaggio cifrato **a caso**, che dopo la decifratura verrebbe accettato come valido.
2. **Freschezza**: serve un meccanismo per prevenire la **ripetizione** di vecchi messaggi (*replay attack*).

Esempio 2 — Ridondanza e freschezza in pratica

- **Ridondanza**: l'attaccante genera un C a caso e lo manda. Se il destinatario accetta qualsiasi $D_K(C)$, lo scambia per un messaggio valido. Se invece ogni messaggio contiene anche un campo di controllo (per esempio $h = H(m)$, così che si invia $E_K(m, h)$), il destinatario decifra, ricalcola $H(m)$ e verifica la corrispondenza: un C casuale non la supera.
- **Freschezza**: T registra un messaggio cifrato valido da A a B (per esempio "paga 100 euro a T") e lo rimanda a B più tardi. B lo decifra correttamente e lo esegue di nuovo! Si risolve con i **timestamp** (messaggi validi solo per poco tempo) o con numeri usati una sola volta (*nonce*), che vedremo parlando di autenticazione.

Algoritmi a chiave simmetrica: il DES

DES (Data Encryption Standard)

Il **DES** è un algoritmo a chiave simmetrica storico:
- il testo in chiaro viene cifrato a **blocchi di 64 bit**;
- la cifratura avviene in **19 stadi**;
- usa una chiave da **56 bit** (esistono versioni con chiavi più lunghe, come il Triple DES): la versione a 56 bit oggi **non è più considerata sicura**. Il suo successore è l'**AES**.

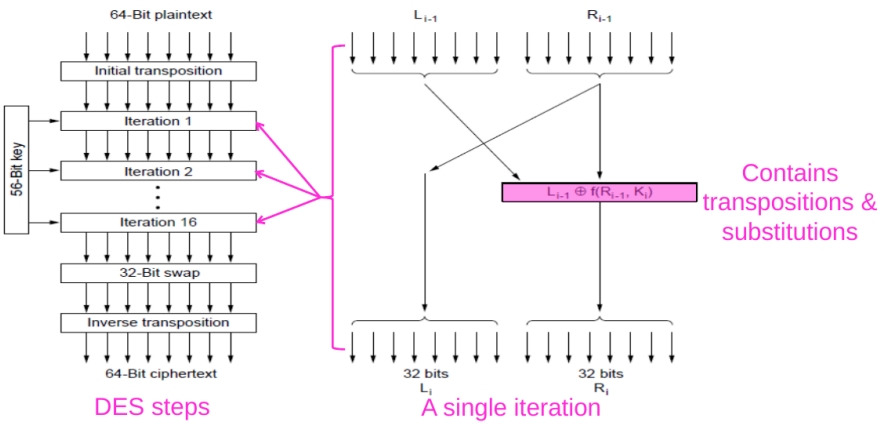


Figura 29.7 – La struttura del DES (da Tanenbaum): a sinistra i 19 stadi, a destra una singola iterazione.

- Il primo e l'ultimo stadio sono **trasposizioni** (una inversa dell'altra); il diciottesimo scambia i due blocchi da 32 bit.
- Nei 16 stadi intermedi, detti iterazioni: i 32 bit di sinistra dell'uscita sono i 32 bit di destra dell'ingresso ($L_i = R_{i-1}$); quelli di destra sono $L_{i-1} \oplus f(R_{i-1}, K_i)$, dove la funzione f **espande**, fa lo **XOR con la chiave** dell'iterazione K_i e applica **sostituzioni** e **trasposizioni**.
- La chiave K_i è diversa per ogni iterazione (derivata dalla chiave principale per trasposizione).

Cifrare blocco per blocco non basta

Il DES eseguito blocco per blocco non è altro che una **sostituzione** di simboli in altri simboli: ogni blocco da 64 bit viene cifrato separatamente, quindi blocchi di testo in chiaro **uguali** producono blocchi cifrati **uguali**. Un attaccante potrebbe riconoscere schemi ripetuti, o scambiare e ricombinare blocchi (per esempio in una busta paga cifrata, sostituire il blocco dello stipendio con quello di un collega più pagato).

CBC (Cipher Block Chaining)

Il **CBC** è una modalità di cifratura molto usata che **concatena** i blocchi con lo XOR:

$$C_0 = E_K(P_0 \oplus IV), \qquad C_i = E_K(P_i \oplus C_{i-1})$$

dove IV è un **vettore di inizializzazione** casuale, diverso per ogni messaggio. Così lo stesso blocco in chiaro produce blocchi cifrati diversi a seconda di tutto ciò che lo precede.

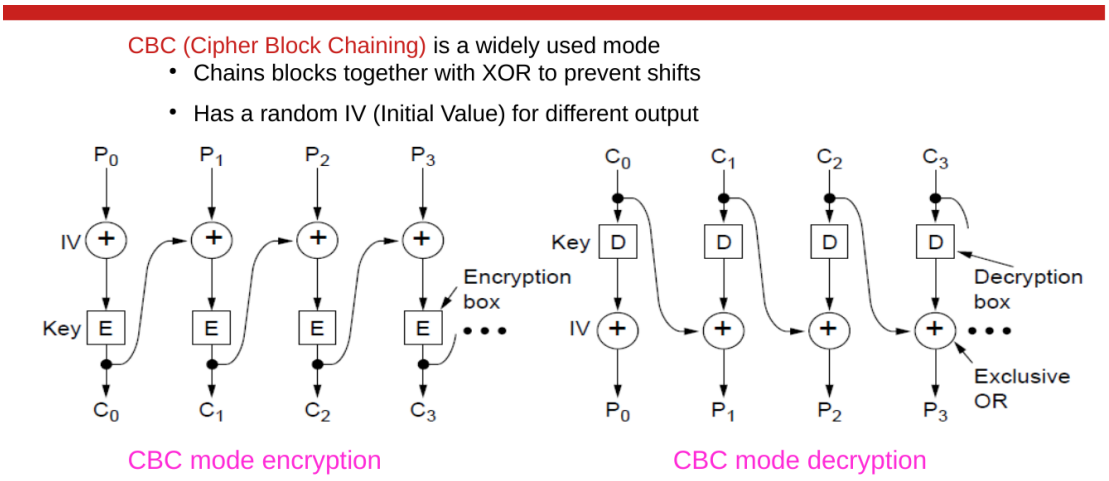


Figura 29.8 – CBC: cifratura (a sinistra) e decifratura (a destra), $P_i = D_K(C_i) \oplus C_{i-1}$.

L'INTEGRITÀ DEI MESSAGGI

Integrità del messaggio

L'**integrità del messaggio** (detta anche *autenticazione del messaggio*) è il problema di verificare che:

1. il messaggio **non sia stato manomesso**;
2. il messaggio sia stato davvero **originato dall'host previsto**.

Si può creare un elemento di controllo, in modo simile al checksum o al CRC. Tipicamente si usa una **funzione hash**, cioè una funzione che associa a dati di dimensione arbitraria valori di dimensione fissa.

Funzione hash crittografica

Una **funzione hash crittografica** è una funzione H che trasforma un messaggio x in una stringa di dimensione fissa $H(x)$ tale che sia **computazionalmente impossibile** trovare un altro messaggio y con $H(y) = H(x)$. Esempi: SHA-256 (e i vecchi MD5 e SHA-1, oggi considerati insicuri).

Primo tentativo: accodare l'hash

1. A crea il messaggio m e calcola l'hash $h = H(m)$.
2. A accoda h al messaggio, creando il messaggio esteso (m, h), e lo invia a B.
3. B riceve (m, h) e calcola $H(m)$. Se $H(m) = h$, il messaggio è integro.

Un approccio ovviamente difettoso

Un intruso può falsificare l'intero messaggio (m, h), creandone uno nuovo "su misura" (m', h') con $h' = H(m')$: è comunque coerente con la funzione hash, e B lo accetta. La funzione H è pubblica, quindi chiunque può calcolarla.

Il MAC: hash con segreto condiviso

Message Authentication Code (MAC)

A e B condividono un **segreto** s (una chiave o una password), una stringa nota solo a loro.

1. A crea la concatenazione $m + s$ e ne calcola l'hash $h = H(m + s)$: è il **MAC** (*Message Authentication Code*).
2. A accoda il MAC al messaggio, creando $(m, H(m + s))$, e lo invia a B.
3. B riceve (m, h) e, conoscendo s, calcola $H(m + s)$. Se coincide con h, il messaggio è integro e viene da A.

Un intruso che invia (m', h') non conosce s: non può calcolare $H(m' + s)$, e B scopre la falsificazione.

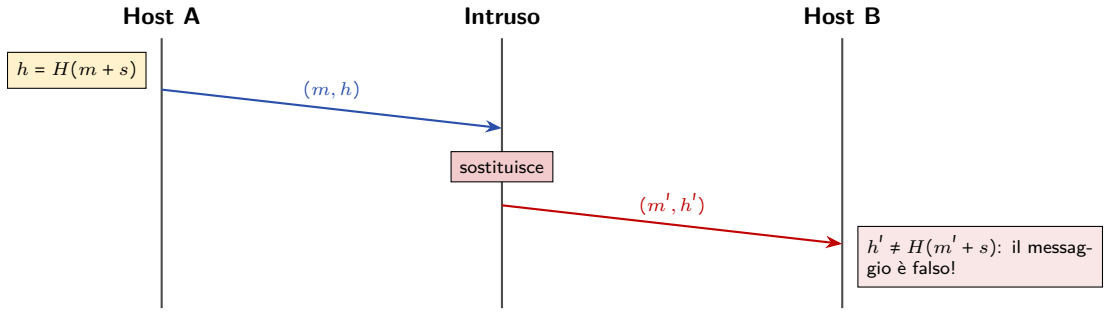


Figura 29.9 – Con il segreto condiviso l'intruso non riesce più a costruire un MAC valido.

Un nome, due significati

Qui MAC significa **Message Authentication Code**, e non ha nulla a che vedere con il *Medium Access Control* (e gli indirizzi MAC) del livello di collegamento.

Come in tutti gli approcci simmetrici, il segreto va **scambiato**: lo si può fare combinando crittografia asimmetrica e certificati.

LA CRITTOGRAFIA ASIMMETRICA

Crittografia asimmetrica (a chiave pubblica)

Nella crittografia **asimmetrica** ci sono **due chiavi** per ogni partecipante: una **pubblica** e una **privata**.

- Un messaggio cifrato con una delle due chiavi va decifrato con l'altra, e viceversa.
- La chiave pubblica può essere inviata su canali non sicuri o pubblicata; la chiave privata è disponibile **solo al proprietario**.

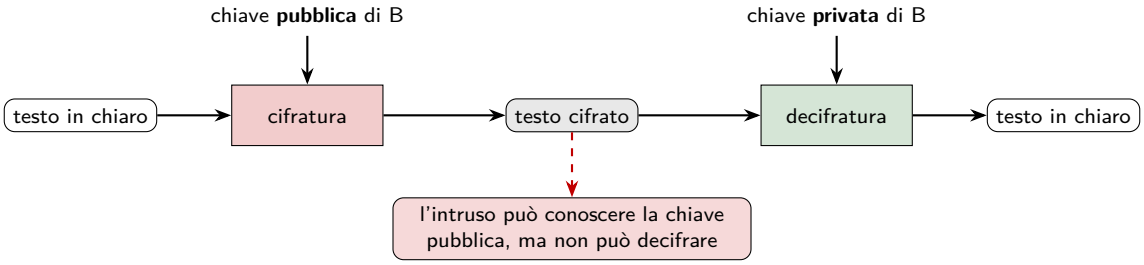


Figura 29.10 – A cifra con la chiave pubblica di B; solo B, con la propria chiave privata, può decifrare.

L'uso tipico: cifrare con la **chiave pubblica** del destinatario e decifrare con la sua **chiave privata**.

- Se l'intruso intercetta la chiave pubblica, non riesce comunque a decifrare i messaggi.
- L'host A usa la chiave pubblica di B per cifrare messaggi che si possono leggere solo con la chiave privata di B, che è solo nelle mani di B.

Pro e contro rispetto alla crittografia simmetrica

✔ La chiave privata **non va distribuita**; basta una coppia di chiavi per utente (invece di N^2 chiavi a coppie).
✖ La sicurezza si basa su proprietà matematiche ed è molto **più lenta** (circa 1000 volte) della crittografia simmetrica.
Per questo la si usa soprattutto per **stabilire chiavi simmetriche di sessione**, con cui poi si cifrano i dati.

Requisiti di un algoritmo a chiave pubblica

1. $D(E(P)) = P$ (correttezza);
2. deve essere estremamente difficile ricavare la chiave privata da quella pubblica;
3. E non deve poter essere violato con un attacco a **testo in chiaro scelto**: l'intruso ha la chiave pubblica, quindi può cifrare tutti i testi in chiaro che vuole.

RSA

RSA

L'**RSA** (Rivest, Shamir, Adleman) è l'algoritmo a chiave pubblica più diffuso. La sua sicurezza si basa sulla difficoltà di **fattorizzare numeri grandi**.

- **Generazione delle chiavi**: si scelgono due numeri primi grandi p e q; si calcolano $n = p \cdot q$ e $z = (p-1)(q-1)$; si sceglie d primo con z; si trova e tale che $e \cdot d \equiv 1 \pmod{z}$. La chiave **pubblica** è (e, n), quella **privata** è (d, n).
- **Cifratura** (per un messaggio di k bit, visto come numero $P < n$): $C = P^e \bmod n$.
- **Decifratura**: $P = C^d \bmod n$.

Esempio 3 — RSA con numeri piccoli (da Tanenbaum)

$p = 3$, $q = 11$: $n = 33$, $z = 2 \cdot 10 = 20$. Scegliamo $d = 7$ (primo con 20) e cerchiamo e con $7e \equiv 1 \pmod{20}$: $e = 3$ ($21 \bmod 20 = 1$). Chiave pubblica $(3, 33)$, privata $(7, 33)$.

- Cifrare $P = 19$: $C = 19^3 \bmod 33 = 6859 \bmod 33 = 28$.
- Decifrare: $P = 28^7 \bmod 33 = 19$.

Con numeri di centinaia di cifre, ricavare d da (e, n) richiederebbe di fattorizzare n: praticamente impossibile.

Diffie-Hellman

Il protocollo **Diffie-Hellman** permette a due parti di stabilire un **segreto condiviso** su un canale pubblico.

1. Alice sceglie un numero segreto x e invia a Bob due numeri pubblici n e g (primi grandi) e il valore $g^x \bmod n$.
2. Bob sceglie un numero segreto y e invia ad Alice $g^y \bmod n$.
3. Alice calcola $(g^y \bmod n)^x \bmod n = g^{xy} \bmod n$; Bob calcola $(g^x \bmod n)^y \bmod n = g^{xy} \bmod n$.

Entrambi ottengono lo stesso segreto $g^{xy} \bmod n$. Un intruso che ascolta conosce n, g, $g^x \bmod n$ e $g^y \bmod n$, ma non può calcolare $g^{xy} \bmod n$ senza conoscere x o y: ricavarli dalle potenze modulari (il *logaritmo discreto*) è praticamente impossibile.

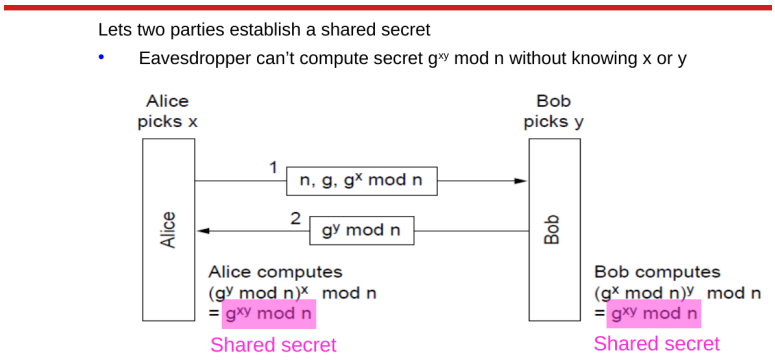
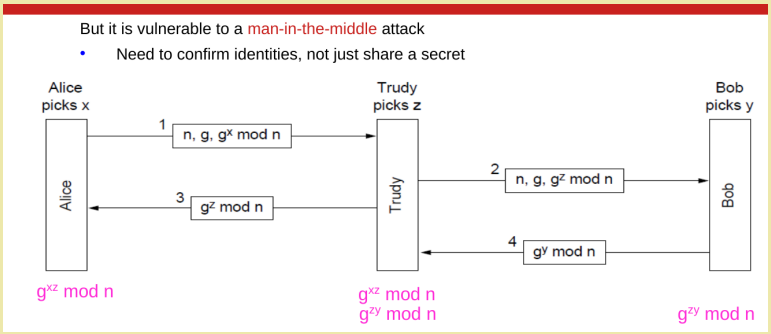


Figura 29.11 – Lo scambio di chiavi Diffie-Hellman.

L'attacco man-in-the-middle a Diffie-Hellman

Diffie-Hellman da solo è vulnerabile a un attacco **man-in-the-middle**: Trudy intercetta $g^x \bmod n$ di Alice e le risponde con un proprio $g^z \bmod n$, poi fa lo stesso con Bob. Alice crede di condividere un segreto con Bob, ma lo condivide con Trudy (g^{xz}), e così Bob (g^{yz}). Trudy decifra, legge e ricifra tutto. Serve **confermare le identità**, non solo condividere un segreto: è il ruolo dei certificati.



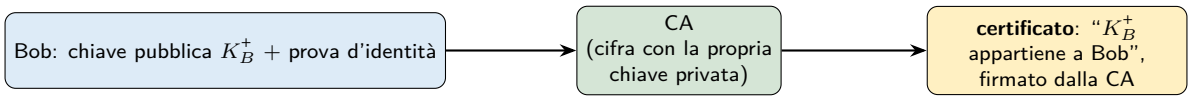
Le autorità di certificazione

Con la crittografia a chiave pubblica è fondamentale verificare che una chiave pubblica appartenga davvero all'entità con cui si vuole comunicare. Altrimenti potremmo avere la chiave di qualcun altro (un attaccante) e cifrare messaggi leggibili da estranei.

Autorità di certificazione (CA)

Il legame tra una chiave pubblica e una certa entità viene tipicamente certificato da una **Certification Authority** (CA), il cui compito è validare le identità ed emettere certificati.

1. Una CA **verifica che un'entità sia chi dice di essere**. Non esiste un protocollo per farlo: bisogna fidarsi che la CA abbia eseguito una verifica sufficientemente rigorosa. Funziona come una selezione naturale: se una CA è inaffidabile nessuno si fida di lei; esistono CA pubbliche e statali ragionevolmente affidabili, ma bisogna comunque fidarsi.
2. Verificata l'identità, la CA crea un **certificato** che lega la chiave pubblica dell'entità alla sua identità. Il certificato contiene la chiave pubblica e un identificativo univoco del proprietario (per esempio un nome o un indirizzo IP), ed è **firmato** digitalmente dalla CA.



Alice verifica la firma con la chiave pubblica della CA (già presente nel browser o nel sistema operativo): se è valida, può fidarsi che K_B^+ sia di Bob.

Figura 29.12 – La CA lega una chiave pubblica a un'identità.

RIEPILOGO

In breve

- Attacchi: **malware** (virus, worm, trojan, ransomware, botnet), **DoS**/DDoS, **sniffing** (passivo), **IP spoofing**, man-in-the-middle.
- Proprietà: **confidenzialità**, **integrità**, **autenticazione**, **sicurezza operativa**.
- **Kerckhoffs**: algoritmi pubblici, chiavi segrete. Cifrari per sostituzione e trasposizione; one-time pad inviolabile ma impraticabile.
- **Simmetrica** (DES, AES; modalità CBC): una chiave, veloce, ma va scambiata. **Asimmetrica** (RSA): chiave pubblica e privata, lenta, usata per scambiare chiavi. **Diffie-Hellman**: segreto condiviso, ma vulnerabile al MitM.
- **Integrità**: hash crittografici; il **MAC** $H(m + s)$ usa un segreto condiviso. Principi: ridondanza e freschezza.
- Le **CA** certificano il legame tra chiave pubblica e identità.

Autenticazione, SSL/TLS e sicurezza operativa

Ricordiamo le quattro proprietà di una comunicazione sicura: **confidenzialità** (contro lo sniffing), **integrità del messaggio**, **autenticazione degli estremi** (contro lo spoofing) e **sicurezza operativa**. Le prime tre sono realizzate via software; l'ultima si basa tipicamente su hardware dedicato. In questo capitolo completiamo l'autenticazione, vediamo come tutto si combina in SSL/TLS e chiudiamo con firewall e IDS.

L'autenticazione degli estremi

Autenticazione degli estremi

L'**autenticazione degli estremi** (*end-point authentication*) è il processo con cui un'entità **dimostra la propria identità** a un'altra entità attraverso una rete.

- Tipicamente un **protocollo di autenticazione** (AP, *Authentication Protocol*) viene eseguito **prima** che le due parti eseguano qualche altro protocollo: serve a stabilire le identità di entrambe prima di cominciare a lavorare.
- Esempi di protocolli che ne hanno bisogno: un trasferimento affidabile (basato su TCP), uno scambio di informazioni di routing (DV o LS), un protocollo di posta (SMTP), …
- Per l'autenticazione ci si può appoggiare alle **CA** viste nel capitolo precedente, ma non basta.

Tentativi progressivi di autenticazione

Supponiamo che l'host A debba autenticarsi presso B prima di lavorare insieme. Costruiamo un protocollo per tentativi, correggendo a ogni passo il difetto del precedente (l'approccio del Kurose, con i protocolli ap1.0, ap2.0, …).

Tentativo 1: controllare l'indirizzo IP

Se A e B hanno già comunicato e l'indirizzo IP di A è sempre lo stesso, B potrebbe autenticare A **controllando l'indirizzo IP** nel datagramma: se è quello noto, si assume che a inviare sia A.

Basta falsificare l'indirizzo

Questa tecnica funziona contro intrusi "ingenui" che cercano di spacciarsi per A, ma non basta: un intruso "meno ingenuo" può **falsificare l'indirizzo** nel pacchetto (spoofing).

Un modo di evitare lo spoofing è sfruttare i **router**: un router può essere configurato per inoltrare solo datagrammi legittimi, cioè con indirizzo sorgente che appartiene davvero agli host dietro di lui.

- Il router deve conoscere gli IP dei propri host (altrimenti gli host sarebbero irraggiungibili).
- Può quindi rilevare i pacchetti il cui indirizzo non è coerente con la sorgente, e scartarli prima che facciano danni.

Purtroppo questa capacità **non è universalmente adottata** né imposta: bisogna assumere che lo spoofing sia possibile.

Tentativo 2: una password segreta

Un altro approccio, il più comune, è usare una **password segreta**: la password funziona come un segreto condiviso tra chi autentica e chi viene autenticato. Gmail, Facebook, Telnet, FTP e moltissimi altri servizi usano l'autenticazione con password. Lo stesso segreto può servire anche al controllo di integrità.

La password si può intercettare

Il primo problema è che l'intruso può **intercettare la password** (eavesdropping) dai messaggi, e usarla insieme a un IP falsificato. Con Telnet, che trasmette tutto in chiaro, basta uno sniffer.

Tentativo 3: una password cifrata

L'intercettazione si evita **cifrando la password**, così che diventi illeggibile per l'intruso. Se si usa una cifratura simmetrica non serve nemmeno una password aggiuntiva: la chiave condivisa stessa, che deve essere un segreto tra i due host, può fare da password.

L'attacco a ripetizione (playback)

La cifratura è una buona cosa, ma non basta: un intruso "molto furbo" può infilarsi nella comunicazione fingendosi A con un **attacco a ripetizione** (*playback* o *replay attack*).

- L'intruso sniffa i pacchetti da A a B cercando di ottenere la versione **cifrata** della password di A.
- Se ci riesce, può **ripetere** la password cifrata a B in una nuova sessione, usandola **senza nemmeno capirla**.
- A differenza dei MitM classici (in tempo reale), sniffing e ripetizione avvengono in momenti separati.

Il problema è che B non riesce a capire se A è "**vivo**", cioè se c'è una sessione attiva in corso tra il vero A e B.

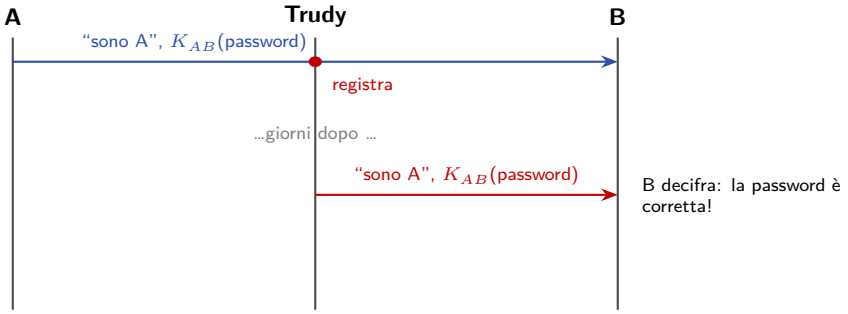


Figura 30.1 – L'attacco playback: Trudy non conosce la password, ma riusa il messaggio cifrato.

Il problema somiglia all'apertura di una connessione TCP, in cui con il three-way handshake i due host si assicurano di essere entrambi vivi. Nel three-way handshake gli host usano **numeri di sequenza iniziali casuali**, perché ritrasmissioni di vecchie connessioni non vengano scambiate per SYN o SYNACK nuovi. Un'idea simile si può usare per l'autenticazione.

La soluzione: il nonce

Nonce

Un **nonce** è un numero casuale (o pseudocasuale) che un protocollo usa **una sola volta nella vita** (*number used once*).

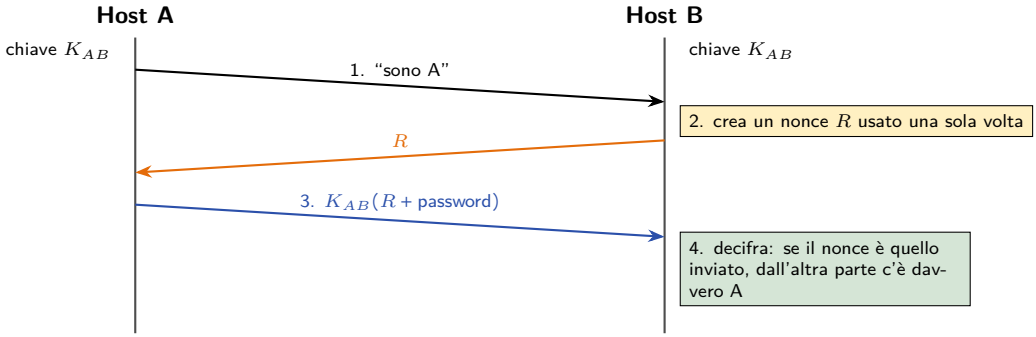


Figura 30.2 – Autenticazione con nonce e chiave simmetrica condivisa.

Supponendo che A e B condividano una chiave simmetrica, il nonce si usa così:

1. A invia a B il messaggio "sono A".
2. B sceglie un nonce R e lo invia ad A.
3. A cifra il nonce insieme alla password con la chiave simmetrica condivisa e rimanda il risultato a B.
4. B decifra il messaggio: se il nonce decifrato coincide con quello che aveva inviato, A è autenticato.

- La password diventa una sorta di **password monouso**: ogni password cifrata è legata al proprio nonce, e non può essere riusata in altre sessioni (da qualcun altro).
- Se password e nonce arrivano correttamente a B, si può ragionevolmente assumere che dietro ci sia A, **vivo** in quel momento.

Con la crittografia asimmetrica

Lo stesso schema funziona con le chiavi pubbliche: B invia il nonce R, A lo cifra con la propria **chiave privata**, e B lo decifra con la chiave **pubblica** di A. Solo A può aver prodotto quel messaggio. Resta però il problema di essere sicuri che la chiave pubblica sia davvero di A: di nuovo servono i **certificati**.

SSL e TLS

Abbiamo visto che diverse tecniche crittografiche si possono usare e integrare per offrire confidenzialità, integrità e autenticazione alle connessioni TCP.

SSL/TLS

Queste tecniche sono realizzate dal **Secure Socket Layer** (SSL), una versione "potenziata" di TCP che offre servizi di sicurezza.

- Una versione leggermente modificata di SSL 3 si chiama **Transport Layer Security** (TLS), ed è quella usata oggi.
- Esiste un equivalente per le comunicazioni basate su UDP, il **Datagram TLS** (DTLS).

- SSL è usato da molti protocolli applicativi: Web, posta, messaggistica istantanea, voice over IP, …
- Spesso la connessione tra il browser e un sito usa **HTTPS** (HTTP + SSL/TLS) invece di HTTP: il lucchetto nella barra degli indirizzi.

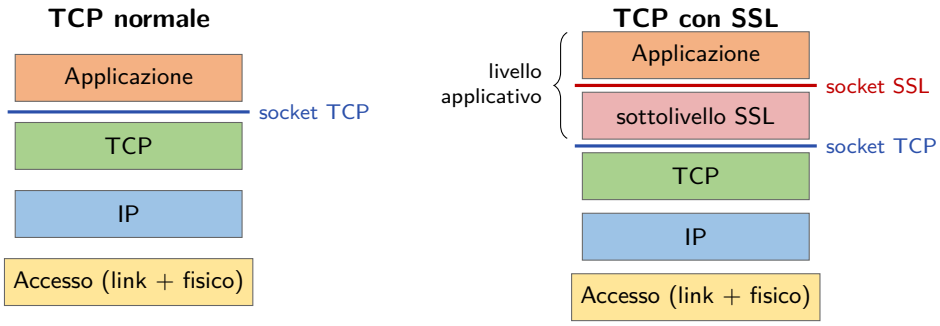


Figura 30.3 – SSL si inserisce tra l'applicazione e TCP: tecnicamente sta nel livello applicativo, ma allo sviluppatore appare come un trasporto sicuro.

- Poiché SSL protegge TCP, può essere usato da **qualsiasi applicazione** che gira su TCP.
- SSL offre un'**API** semplice, con socket simili a quelli di TCP: un'applicazione che vuole usare SSL include le librerie SSL.
- Anche se SSL risiede tecnicamente nel livello applicativo, dal punto di vista dello sviluppatore sembra un protocollo di trasporto che offre i servizi di TCP arricchiti di sicurezza.

SSL ha tre fasi principali: **handshake**, **derivazione delle chiavi** e **trasferimento dati**.

SSL: L'HANDSHAKE

Un client B vuole usare SSL per comunicare con un server A. Nella fase di handshake B deve:

- **stabilire una connessione TCP** con A;
- **verificare che A sia davvero A** (e non un server fasullo);
- **inviare ad A una chiave segreta principale** (*master secret*), il segreto condiviso.

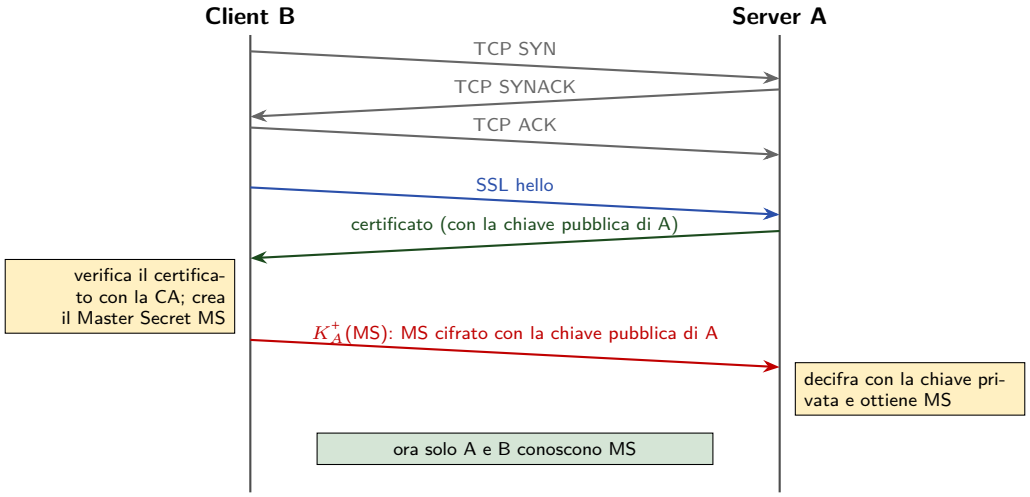


Figura 30.4 – L'handshake SSL (semplificato).

1. Stabilita la connessione TCP, B invia ad A un messaggio **hello**.
2. Il server A risponde con il proprio **certificato**, che contiene la sua **chiave pubblica**.
3. Poiché il certificato è stato certificato da una CA, il client B si fida che la chiave pubblica nel certificato appartenga ad A.
4. B genera un **Master Secret** (MS), che userà solo per questa sessione SSL (è un nonce), lo cifra con la chiave pubblica di A e lo invia ad A.
5. Il server A decifra il messaggio con la propria chiave privata e ottiene MS. Dopo questa fase, **solo B e A** conoscono il master secret della sessione.

Asimmetrica per iniziare, simmetrica per lavorare

SSL usa la crittografia **asimmetrica** (lenta) solo per scambiare in sicurezza il segreto, e poi la crittografia **simmetrica** (veloce) per tutti i dati. È lo schema ibrido tipico di quasi tutti i protocolli sicuri.

SSL: LA DERIVAZIONE DELLE CHIAVI

Poiché MS è un segreto condiviso tra B e A, potrebbe essere usato come chiave simmetrica per tutta la cifratura e il controllo di integrità della sessione. È però più sicuro che A e B usino **chiavi diverse** per la cifratura e per l'integrità, e per i due flussi di dati (da A a B e da B ad A).

Da MS si ricavano **quattro chiavi**:

Chiave	Uso
$E_{B\to A}$	chiave di **cifratura** di sessione per i dati da B ad A
$M_{B\to A}$	chiave **MAC** di sessione per i dati da B ad A
$E_{A\to B}$	chiave di **cifratura** di sessione per i dati da A a B
$M_{A\to B}$	chiave **MAC** di sessione per i dati da A a B

Tabella 30.1 – Le quattro chiavi di sessione derivate dal master secret (MAC = Message Authentication Code).

SSL: IL TRASFERIMENTO DATI

SSL protegge TCP spezzando il flusso di dati dell'applicazione in **record**. Per ciascun record:

1. SSL accoda al record un **MAC**, per il controllo di integrità (record_i + MAC);
2. il record così modificato viene **cifrato**;
3. il record cifrato viene passato a TCP per la trasmissione.

E l'integrità dell'intero flusso?

Così si garantisce l'integrità dei singoli record, ma non dell'intero flusso. Poiché è cifrato solo il payload, un MitM tra A e B può ancora danneggiare il flusso **invertendo o eliminando segmenti**. Per esempio può catturare due segmenti consecutivi, scambiarne l'ordine insieme ai numeri di sequenza TCP, e inviarli al ricevente, che non si accorge dell'inversione finché i dati non arrivano all'applicazione.

Per evitarlo SSL integra anche un **numero di sequenza** nel messaggio prima della cifratura: il vero messaggio cifrato è

$$\text{record}_i + \text{MAC} + \text{numeroDiSequenza}_i$$

dove il MAC è calcolato anche sul numero di sequenza. Conoscendo la procedura e la chiave MAC, il ricevente può certificare l'integrità e l'**ordine** dei messaggi.

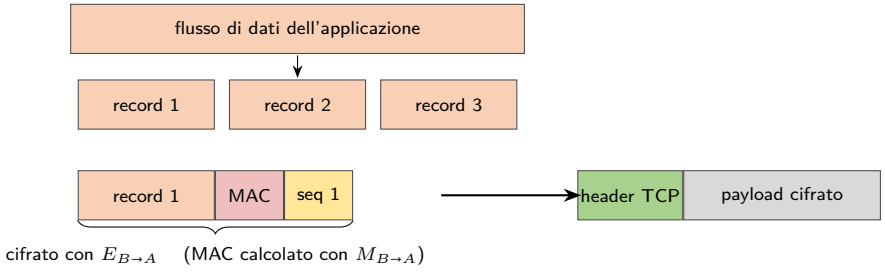
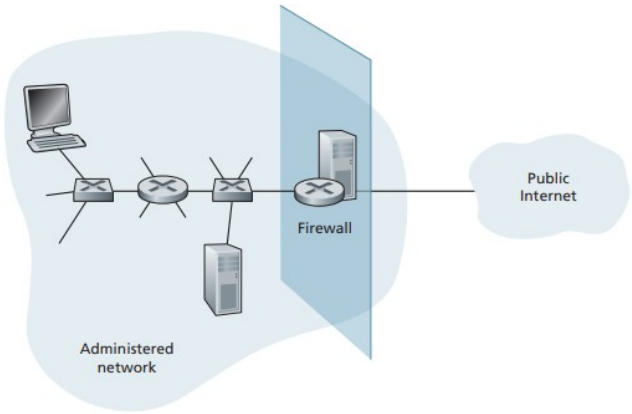


Figura 30.5 – Il trasferimento dati in SSL: record, MAC, numero di sequenza e cifratura, poi TCP.

I FIREWALL

Firewall

Un **firewall** è una combinazione di hardware e software che **isola** (protegge) una rete da Internet, permettendo o negando il passaggio dei pacchetti.



- **Tutto** il traffico in entrata e in uscita dovrebbe passare per il firewall, e solo il traffico **autorizzato** (secondo la politica di sicurezza locale) può passare.
- Il firewall fa da **unico punto di accesso** alla rete pubblica, ma le grandi organizzazioni possono avere più livelli di firewall o firewall distribuiti.
- Il firewall stesso deve essere **immune alle intrusioni**: se viene compromesso, dà un falso senso di sicurezza.

Figura 30.6 – Un firewall tra la rete amministrata e la rete pubblica.

Vediamo tre approcci: il **filtraggio tradizionale** dei pacchetti, il **filtraggio stateful** e l'**application gateway**.

IL FILTRAGGIO TRADIZIONALE DEI PACCHETTI

- Un **filtro di pacchetti** è di solito realizzato sul **gateway**, il router che collega la rete locale all'ISP.
- Esamina **ogni singolo datagramma** e decide se accettarlo o scartarlo in base a regole specificate dall'amministratore. Si possono avere regole diverse per i datagrammi in entrata e in uscita, o per le singole interfacce.

Le regole si basano tipicamente su:

- indirizzo IP sorgente o destinazione;
- tipo di protocollo nel campo del datagramma IP (TCP, UDP, ICMP, …);
- porta sorgente e destinazione;
- bit dei flag TCP (SYN, ACK, …);
- …

Politica	Impostazione del firewall
Solo traffico Web in uscita	scartare tutti i segmenti TCP SYN in uscita con porta di destinazione diversa da 80
Niente streaming (non critico)	scartare il traffico UDP in entrata non essenziale
Non rispondere ai ping dall'esterno	scartare le risposte ICMP ping in uscita (echo reply)
Telnet solo con host fidati	inoltrare i datagrammi Telnet (porta 23) solo da e verso un elenco di indirizzi

Tabella 30.2 – Esempi di politiche e relative regole.

I filtri non proteggono dallo spoofing

Politiche basate su indirizzi e porte non proteggono dallo **spoofing**: un pacchetto con indirizzo sorgente falsificato su quello di un host fidato passa il filtro.

Le access control list

Nei router le regole del firewall sono realizzate con le **access control list** (ACL). Consideriamo l'ACL dell'interfaccia che collega il router all'ISP, per un'organizzazione con indirizzi `222.22/16` che vuole lasciar passare **solo il traffico Web**:

Azione	IP sorgente	IP destinazione	Protocollo	Porta sorg.	Porta dest.	Flag
allow	222.22/16	esterno a 222.22/16	TCP	> 1023	80	qualsiasi
allow	esterno a 222.22/16	222.22/16	TCP	80	> 1023	ACK
allow	222.22/16	esterno a 222.22/16	UDP	> 1023	53	—
allow	esterno a 222.22/16	222.22/16	UDP	53	> 1023	—
deny	tutto	tutto	tutto	tutto	tutto	tutto

Tabella 30.3 – Una ACL che permette solo il traffico Web (le regole si leggono dall'alto: vale la prima che corrisponde).

1. La prima regola lascia **uscire** qualsiasi pacchetto TCP con porta di destinazione 80.
2. La seconda lascia **entrare** qualsiasi pacchetto TCP con porta sorgente 80 e flag **ACK = 1** (tutti i messaggi Web di risposta portano un ACK, mentre un tentativo di aprire una connessione dall'esterno avrebbe SYN senza ACK).
3. La terza e la quarta insieme permettono ai pacchetti **DNS** di uscire ed entrare (senza DNS non si navigherebbe).
4. L'ultima nega tutto il resto.

Il limite: il filtraggio è stateless

Pacchetti ACK falsi

Il filtraggio tradizionale è **senza stato**: esamina ogni pacchetto da solo. Per quanto restrittiva, la tabella precedente lascia entrare **qualsiasi** pacchetto dall'esterno con ACK = 1 e porta sorgente 80, anche se non fa parte di nessuna connessione aperta dall'interno. Pacchetti di questo tipo sono noti per essere usati negli attacchi DoS. La soluzione ingenua sarebbe bloccare anche i pacchetti TCP ACK, ma impedirebbe agli utenti interni di navigare sul Web.

Il filtraggio stateful

Filtro stateful

I **filtri stateful** risolvono il problema tenendo traccia di **tutte le connessioni TCP in corso** in una **tabella delle connessioni**, per capire se il traffico appartiene a una connessione legittima.

- Il firewall riconosce l'**inizio** di una nuova connessione (la sequenza SYN, SYNACK, ACK del three-way handshake) e la sua **fine** (il pacchetto FIN).
- Può anche assumere (in modo conservativo) che la connessione sia finita quando non vede attività per un certo tempo (per esempio 60 secondi).

action	source address	dest address	protocol	source port	dest port	flag bit	check conxion
allow	222.22/16	outside of 222.22/16	TCP	> 1023	80	any	
allow	outside of 222.22/16	222.22/16	TCP	80	> 1023	ACK	X
allow	222.22/16	outside of 222.22/16	UDP	> 1023	53	—	
allow	outside of 222.22/16	222.22/16	UDP	53	> 1023	—	X
deny	all	all	all	all	all	all	

Figura 30.7 – L'ACL di un filtro stateful: la colonna "check connection" indica le regole che richiedono di verificare la tabella delle connessioni. In questo caso sono ammessi solo i pacchetti ACK che appartengono a connessioni già stabilite.

Una tabella stateful ha di solito una colonna "*check connection*" che indica se un pacchetto deve far parte di una connessione già stabilita: un pacchetto con ACK = 1 in arrivo dall'esterno passa solo se nella tabella c'è una connessione aperta dall'interno con gli stessi indirizzi e porte.

L'APPLICATION GATEWAY

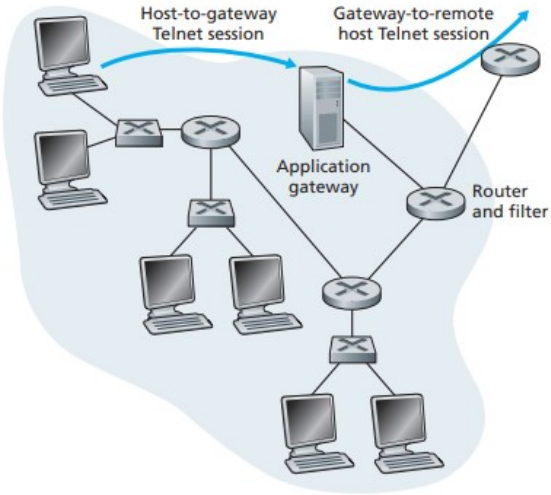
I filtri visti finora controllano soprattutto indirizzi IP e porte. Ma può servire permettere o negare funzionalità in base agli **utenti** o all'**applicazione**, per esempio:

- solo i tecnici possono usare certi protocolli (come Telnet verso l'esterno), negati agli utenti normali;
- all'interno di un protocollo sono permessi solo certi tipi di messaggi o comandi.

Compiti del genere vanno oltre le capacità dei filtri tradizionali e stateful: l'identità degli utenti interni è un dato applicativo, e non compare negli header IP/TCP/UDP.

Application gateway

Un **application gateway** (o *application-level gateway*, ALG) permette di filtrare i pacchetti in base ai **dati di livello applicativo**. È realizzato di solito come un **server separato** che lavora insieme al firewall.



- L'idea è **negare tutte le connessioni** di un certo protocollo (per esempio Telnet) **tranne** quelle da e verso l'application gateway.
- Un utente che vuole usare il protocollo ristretto deve prima **autenticarsi presso il gateway** (nome utente e password).
- Il server controlla se l'utente ha il permesso per quel protocollo.
- Se sì, tutte le richieste e le risposte vengono inoltrate attraverso il gateway (che fa da **proxy**).

Le reti interne hanno spesso più application gateway, per protocolli diversi (Telnet, HTTP, FTP, posta).

Figura 30.8 – Un gateway per Telnet: sessione host–gateway e sessione gateway–host remoto, con un router che filtra tutto il resto.

Gli svantaggi

- Serve un application gateway **diverso per ogni applicazione**.
- C'è una **penalità di prestazioni** dovuta al proxy (soprattutto con molti utenti).
- Il software sugli host deve sapere **come usare** l'application gateway.

Gli Intrusion Detection System

Per rilevare certi tipi di attacco serve un'**ispezione profonda dei pacchetti** (*deep packet inspection*): controllare, anche nel contenuto, schemi di attacco noti o traffico sospetto.

IDS e IPS

Un **Intrusion Detection System** (IDS) è un insieme di dispositivi specializzati che **monitorano la rete** cercando pacchetti sospetti.

- Il suo compito principale è **riconoscere** i pacchetti potenzialmente malevoli e **allertare** l'amministratore di rete (che poi deciderà cosa fare).
- Se può anche **bloccare** (filtrare) i pacchetti sospetti, si parla di **Intrusion Prevention System** (IPS).

Le organizzazioni usano gli IDS per rilevare un'ampia gamma di attacchi (o di potenziali attacchi):

- scansioni della rete o delle porte (per esempio con nmap);
- flooding DoS;
- worm e virus;
- ...

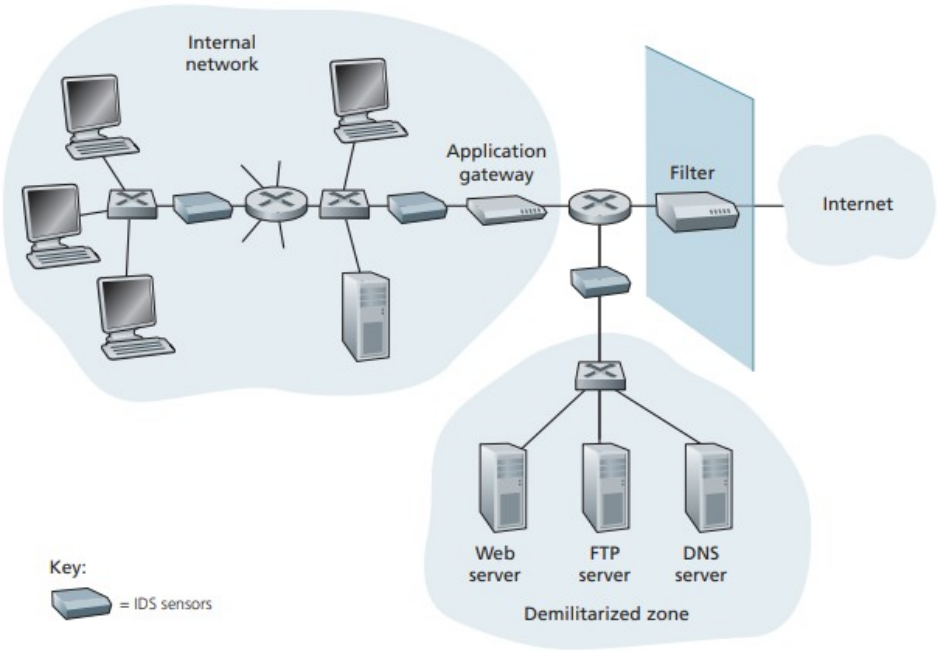


Figura 30.9 – Una rete con filtro, application gateway, sensori IDS e zona demilitarizzata (da Kurose).

Il sistema è spesso composto da:

- uno o più **sensori IDS**;
- un unico **processore IDS centrale**, che raccoglie e integra le informazioni e invia gli allarmi.

Gli IDS possono essere:

- **basati su firme** (*signature-based*): mantengono un ampio database di **firme di attacco**, cioè insiemi di regole legate a un'attività di intrusione (è il tipo più comune). Non riconoscono attacchi nuovi;
- **basati su anomalie** (*anomaly-based*): costruiscono un **profilo del traffico** normale e segnalano i flussi statisticamente insoliti. Possono scoprire attacchi nuovi, ma è difficile distinguere il traffico anomalo da quello semplicemente raro.

LA ZONA DEMILITARIZZATA (DMZ)

Nelle reti grandi capita spesso di dover avere **due livelli di sicurezza** per dispositivi diversi: i server che devono comunicare con il mondo esterno, per esempio, hanno bisogno di un filtraggio più blando.

Zona demilitarizzata (DMZ)

Una **zona demilitarizzata** (DMZ) è una regione a **bassa sicurezza**, con restrizioni limitate, in cui ospitare i server che devono essere raggiungibili dall'esterno (Web, FTP, DNS). Un approccio tipico è mettere la DMZ **tra due firewall**: uno esterno, meno restrittivo, e uno interno, più restrittivo.

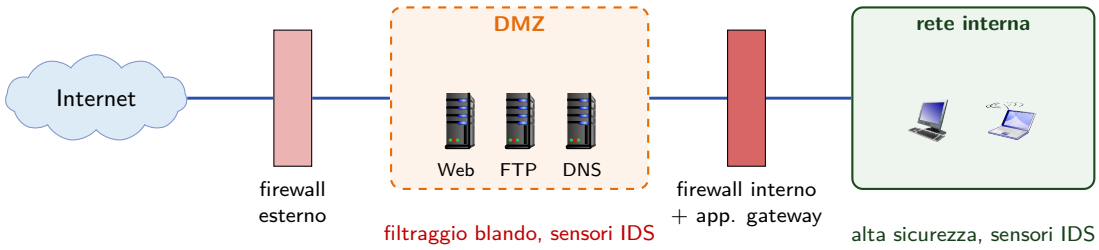


Figura 30.10 – La DMZ tra due firewall.

- La regione ad **alta sicurezza** è protetta da un filtro di pacchetti e da un application gateway, ed è monitorata da sensori IDS.
- La regione a **bassa sicurezza** (la DMZ) è protetta solo dal filtro ed è monitorata dai sensori.

Oltre la sicurezza: la privatezza

Crittografia e firewall proteggono i contenuti, ma chi osserva la rete vede comunque **chi** parla con **chi** (gli indirizzi IP non sono cifrati). Per nascondere anche questo esistono reti di anonimizzazione come **Tor**, che fa passare il traffico cifrato attraverso più nodi, e sistemi come **Whonix** che instradano tutto il traffico di una macchina virtuale attraverso Tor.

Riepilogo

In breve

- **Autenticazione**: l'IP si falsifica, la password si intercetta, la password cifrata si ripete (**playback**); la soluzione è il **nonce**, usato una sola volta.
- **SSL/TLS** protegge TCP: **handshake** (certificato del server, master secret cifrato con la chiave pubblica), **derivazione** di 4 chiavi (cifratura e MAC per ogni direzione), **trasferimento** a record con MAC e numero di sequenza. HTTPS = HTTP + TLS.
- **Firewall**: filtraggio **tradizionale** (ACL su indirizzi, porte, flag; stateless), **stateful** (tabella delle connessioni), **application gateway** (dati applicativi, utenti, proxy).
- **IDS/IPS**: ispezione profonda, basati su firme o su anomalie; la **DMZ** ospita i server pubblici tra due firewall.

Parte VIII

Esercizi

Esercizi sui fondamenti

Questa parte raccoglie gli esercizi svolti, organizzati per livello. Molti riprendono gli esempi visti nei capitoli teorici **con numeri diversi**, così da potersi esercitare davvero; altri sono le esercitazioni di laboratorio proposte a lezione, con lo svolgimento completo. Il consiglio è sempre lo stesso: **provare prima da soli**, e solo dopo leggere la soluzione.

Ritardi e tempi di trasmissione

Richiamo: i ritardi di un pacchetto

Un pacchetto che attraversa un nodo accumula quattro ritardi:

$$d_{\text{nodo}} = d_{\text{elab}} + d_{\text{coda}} + d_{\text{trasm}} + d_{\text{prop}}$$

- d_{elab}: elaborazione (controllo degli errori, scelta dell'uscita), tipicamente microsecondi;
- d_{coda}: attesa in coda, dipende dal traffico;
- $d_{\text{trasm}} = L/R$: tempo per "spingere" sul collegamento tutti gli L bit al rate R. Dipende dalla **dimensione del pacchetto** e dal **rate**, *non* dalla distanza;
- $d_{\text{prop}} = d/s$: tempo che il segnale impiega a percorrere la distanza d alla velocità s (2–$3 \cdot 10^8$ m/s). Dipende dalla **distanza**, *non* dal pacchetto.

Esercizio 1 — Trasmissione contro propagazione

Un pacchetto di $L = 1000$ byte viaggia su un collegamento lungo $d = 2500$ km, con rate $R = 10$ Mb/s e velocità di propagazione $s = 2{,}5 \cdot 10^8$ m/s.

(a) Calcolare il tempo di trasmissione e il tempo di propagazione. Quale dei due domina?

(b) Ripetere il calcolo con $R = 1$ Gb/s. Che cosa cambia?

(c) Quanto è "lungo", in metri, un singolo bit sul cavo nei due casi?

(d) Un collegamento via satellite geostazionario (quota 36 000 km, $s = 3 \cdot 10^8$ m/s) collega due stazioni a terra. Quanto vale il ritardo di propagazione tra le due stazioni?

Svolgimento

(a) Il pacchetto è di $1000 \cdot 8 = 8000$ bit.

$$d_{\text{trasm}} = \frac{L}{R} = \frac{8000}{10^7} = 0{,}8 \text{ ms} \qquad d_{\text{prop}} = \frac{d}{s} = \frac{2{,}5 \cdot 10^6 \text{ m}}{2{,}5 \cdot 10^8 \text{ m/s}} = 10 \text{ ms}$$

Domina la propagazione (12,5 volte più grande); il totale, senza code ed elaborazione, è 10,8 ms.

(b) Con $R = 10^9$ b/s: $d_{\text{trasm}} = 8000/10^9 = 8\ \mu$s, mentre d_{prop} resta 10 ms. Il totale scende solo a 10,008 ms: aumentare la banda **non accorcia la propagazione**.

(c) In un secondo escono R bit che si distribuiscono su s metri, quindi ogni bit occupa s/R metri:

- a 10 Mb/s: $2{,}5 \cdot 10^8/10^7 = 25$ m per bit (il pacchetto intero è lungo 200 km);
- a 1 Gb/s: $2{,}5 \cdot 10^8/10^9 = 25$ cm per bit.

(d) Il segnale sale e scende: $d = 2 \cdot 36\,000$ km $= 7{,}2 \cdot 10^7$ m, quindi $d_{\text{prop}} = 7{,}2 \cdot 10^7/3 \cdot 10^8 = 0{,}24$ s. Un RTT via satellite supera così il mezzo secondo, ed è per questo che le telefonate satellitari hanno quel fastidioso ritardo.

L'analogia dell'autostrada

Pensate a una colonna di 10 auto (i bit) che attraversa un casello (il router) e poi percorre 100 km di autostrada. Il tempo per far passare *tutte* le auto dal casello è il ritardo di **trasmissione**; il tempo che ciascuna impiega a percorrere l'autostrada è il ritardo di **propagazione**. Aprire più caselli (più banda) non rende l'autostrada più corta.

Esercizio 2 — Messaggio intero o pacchetti?

Un messaggio di $8 \cdot 10^6$ bit va dalla sorgente alla destinazione attraverso due router, cioè su $N = 3$ collegamenti da $R = 2$ Mb/s. Si trascurino propagazione, code ed elaborazione.

(a) Quanto tempo serve se il messaggio viene inviato **intero** (commutazione di messaggio, store-and-forward)?

(b) Quanto tempo serve se viene diviso in $P = 800$ pacchetti da 10 000 bit ciascuno? Quando arriva il primo pacchetto?

(c) Ricavare la formula generale e spiegare da dove viene il guadagno.

Svolgimento

(a) Ogni nodo deve ricevere tutto il messaggio prima di inoltrarlo. Ogni collegamento richiede $8 \cdot 10^6/2 \cdot 10^6 = 4$ s, quindi il totale è $3 \cdot 4 = 12$ s.

(b) Ogni pacchetto richiede $L/R = 10^4/2 \cdot 10^6 = 5$ ms per collegamento.

- Il **primo** pacchetto arriva dopo $3 \cdot 5 = 15$ ms.
- La sorgente finisce di trasmettere l'**ultimo** pacchetto dopo $800 \cdot 5$ ms $= 4$ s; da lì servono altri 2 collegamenti, cioè 10 ms. Il messaggio è completo dopo 4,01 s.

(c) La figura mostra il caso di $P = 3$ pacchetti su $N = 3$ collegamenti, con il tempo diviso in intervalli da L/R.

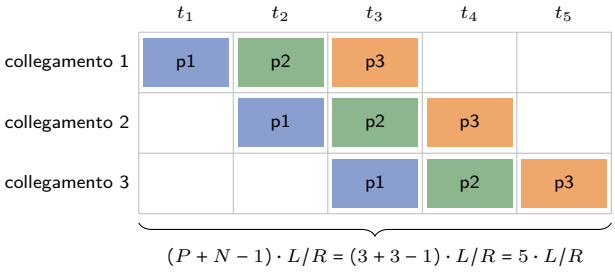


Figura 31.1 – Con i pacchetti i collegamenti lavorano *in parallelo*, come una catena di montaggio.

Il primo pacchetto impiega N intervalli, e ciascuno dei $P - 1$ successivi arriva un intervallo dopo il precedente:

$$T = (P + N - 1) \cdot \frac{L}{R} = (800 + 3 - 1) \cdot 5 \text{ ms} = 4{,}01 \text{ s}$$

Con il messaggio intero, invece, $T = N \cdot P \cdot L/R$: i collegamenti lavorano *uno alla volta*. Dividere in pacchetti permette ai router di lavorare contemporaneamente; inoltre un errore costringe a ritrasmettere solo un pacchetto, e più comunicazioni possono alternarsi sullo stesso collegamento. Il prezzo sono gli header di ogni pacchetto e la necessità di riordinarli a destinazione.

Esercizio 3 — Throughput e collo di bottiglia

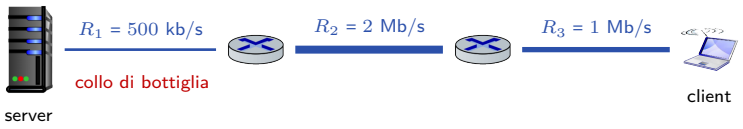
Un server invia un file di 4 MB a un client attraverso tre collegamenti in serie: $R_1 = 500$ kb/s, $R_2 = 2$ Mb/s, $R_3 = 1$ Mb/s.

(a) Qual è il throughput del trasferimento, in assenza di altro traffico?

(b) Quanto dura, approssimativamente, il trasferimento?

(c) Il collegamento centrale viene condiviso in parti uguali da 10 trasferimenti contemporanei. Come cambiano le risposte?

Svolgimento



(a) Il throughput è limitato dal collegamento più lento: $\min\{R_1, R_2, R_3\} = 500$ kb/s.

(b) 4 MB $= 32 \cdot 10^6$ bit, quindi $T = 32 \cdot 10^6 / 5 \cdot 10^5 = 64$ s.

(c) Ogni trasferimento ottiene $R_2/10 = 200$ kb/s sul collegamento centrale, che diventa il nuovo collo di bottiglia: throughput $= \min\{500, 200, 1000\} = 200$ kb/s, e il trasferimento dura $32 \cdot 10^6 / 2 \cdot 10^5 = 160$ s. Il collo di bottiglia non è sempre il collegamento con il rate più basso: dipende anche da **quanto è condiviso**.

Commutazione di circuito e di pacchetto

Esercizio 4 — Quanti utenti su un collegamento

Un collegamento da 1 Mb/s è condiviso da utenti che, quando sono attivi, trasmettono a 100 kb/s; ogni utente è attivo in media il 10% del tempo, in modo indipendente dagli altri.

(a) Con la commutazione di circuito, quanti utenti si possono servire?

(b) Con la commutazione di pacchetto e 35 utenti, qual è la probabilità che esattamente n utenti siano attivi?

(c) Qual è la probabilità che più di 10 utenti siano attivi contemporaneamente? Commentare.

Svolgimento

(a) Ogni circuito riserva 100 kb/s anche quando l'utente tace: $1000/100 = 10$ utenti.

(b) Ogni utente è attivo con probabilità $p = 0{,}1$, indipendentemente dagli altri: il numero di utenti attivi segue una **distribuzione binomiale**

$$P(n \text{ attivi}) = \binom{35}{n} p^n (1-p)^{35-n}$$

Per esempio $P(0) \approx 0{,}025$, $P(3) \approx 0{,}225$ (il valore più probabile), $P(10) \approx 0{,}0013$.

(c) Il collegamento va in sovraccarico solo se ci sono almeno 11 utenti attivi:

$$P(n > 10) = 1 - \sum_{n=0}^{10} \binom{35}{n} 0{,}1^{n}\, 0{,}9^{35-n} \approx 1 - 0{,}99958 = 0{,}00042$$

Circa **4 volte su diecimila**. La commutazione di pacchetto serve quindi 3,5 volte gli utenti del circuito, con prestazioni quasi identiche: è il guadagno della **multiplazione statistica**, che sfrutta il fatto che il traffico dati è *a raffiche* (*bursty*). Il prezzo è che, in quei rari momenti, i pacchetti si accodano e possono andare persi: non c'è più una garanzia.

Esercizio 5 — Un file su un circuito TDM

Quanto tempo serve per inviare un file di 640 000 bit dall'host A all'host B su una rete a commutazione di circuito, se tutti i collegamenti sono da 1,536 Mb/s, ognuno usa TDM con 24 slot, e servono 500 ms per stabilire il circuito? Quanto servirebbe se il collegamento fosse tutto per noi?

Svolgimento

Con TDM a 24 slot, a ogni circuito spetta 1,536/24 = 64 kb/s. Il trasferimento dura 640 000/64 000 = 10 s, a cui si sommano i 0,5 s di instaurazione: **10,5 s**.

Se potessimo usare l'intero collegamento, la trasmissione durerebbe 640 000/1 536 000 ≈ 0,42 s (per collegamento). Il circuito ci **garantisce** i 64 kb/s, ma ci impedisce di usare la capacità lasciata libera dagli altri.

Topologie

Esercizio 6 — Contare collegamenti e porte

Una rete ha $n = 8$ nodi.

(a) Quanti collegamenti servono con topologia a maglia completa, a stella, ad anello e a bus? Quante porte deve avere ogni nodo nella maglia completa?

(b) Si aggiunge un nono nodo: quanti collegamenti in più servono in ciascun caso?

(c) Che cosa succede, in ciascuna topologia, se si guasta un singolo collegamento? E se si guasta il nodo centrale della stella?

Svolgimento

(a) e **(b)**:

Topologia	Collegamenti con 8 nodi	Con 9 nodi	In più
Maglia completa	$\frac{8\cdot 7}{2} = 28$	$\frac{9\cdot 8}{2} = 36$	8
Stella	8 (più il nodo centrale)	9	1
Anello	8	9	1 (se ne toglie uno e se ne aggiungono due)
Bus	1 dorsale + 8 derivazioni	1 + 9	1 derivazione

Tabella 31.1 – Collegamenti richiesti dalle diverse topologie.

Nella maglia completa ogni nodo è collegato a tutti gli altri: servono $n - 1 = 7$ porte per nodo. Aggiungere un nodo costa n collegamenti nuovi e una porta in più su *ogni* nodo esistente: è la crescita quadratica che rende la maglia completa impraticabile per reti grandi.

(c) Guasto di un collegamento:

- **maglia completa**: nessun effetto sulla connettività, esistono altri cammini;
- **stella**: resta isolato solo il nodo di quel collegamento;
- **anello**: se l'anello è unidirezionale la rete si interrompe; se è bidirezionale (o doppio) i messaggi possono girare nell'altro verso;
- **bus**: se si guasta la dorsale la rete si divide in due parti (e il segnale riflesso può disturbare tutti).

Se si guasta il **centro della stella** si ferma tutta la rete: è il suo *single point of failure*.

Esercizio 7 — Perché esistono gli IXP

Sei ISP regionali vogliono scambiarsi traffico direttamente, a coppie. Quanti collegamenti di peering servono? E se si incontrano tutti in un IXP? Ripetere con 50 ISP.

Svolgimento

Collegare tutti con tutti è una maglia completa: $\frac{6 \cdot 5}{2} = 15$ collegamenti. Con un IXP ogni ISP porta un solo collegamento al punto di scambio: 6 collegamenti. Con 50 ISP: $\frac{50 \cdot 49}{2} = 1225$ contro 50. L'IXP trasforma una maglia in una **stella**, ed è il motivo per cui questi punti neutrali di scambio si sono diffusi.

STRATIFICAZIONE E INCAPSULAMENTO

Esercizio 8 — Quanto pesano gli header

Un'applicazione invia messaggi di M byte. Lungo la pila TCP/IP su Ethernet si aggiungono un header TCP da 20 byte, un header IP da 20 byte e 18 byte di header e trailer Ethernet.

(a) Che frazione dei byte trasmessi è occupata dagli header se $M = 100$ byte? E se $M = 1460$ byte?

(b) In una sessione remota ogni tasto premuto invia 1 byte. Che frazione è overhead?

(c) Ricavare la formula per una pila di n strati, ognuno dei quali aggiunge h byte.

Svolgimento

Gli header valgono in tutto $h_{\text{tot}} = 20 + 20 + 18 = 58$ byte.

(a)

$$M = 100: \quad \frac{58}{100 + 58} \approx 36{,}7\% \qquad M = 1460: \quad \frac{58}{1460 + 58} \approx 3{,}8\%$$

(b) $\frac{58}{1 + 58} \approx 98{,}3\%$: per spedire un byte se ne trasmettono 59. I messaggi piccoli sono i più penalizzati dalla stratificazione.

(c) La frazione occupata dagli header è

$$\frac{n\,h}{M + n\,h}$$

che tende a 0 per messaggi grandi e a 1 per messaggi piccoli.

Esercizio 9 — A quale livello?

Per ciascuna voce indicare il livello della pila a cinque strati coinvolto e, dove ha senso, l'unità di dati.

1. Scegliere il cammino di un pacchetto tra i router.
2. Consegnare i dati al processo giusto, individuato da un numero di porta.
3. Delimitare i frame e rilevare gli errori su un singolo collegamento.
4. Trasformare i bit in segnali elettrici o luminosi.
5. Chiedere una pagina Web a un server.
6. Ritrasmettere dati persi lungo il percorso tra i due estremi.
7. Tradurre un nome come `www.unina.it` in un indirizzo IP.
8. Uno switch; un router; un hub (ripetitore).

Svolgimento

#	Livello	Nota
1	Rete	routing; unità: datagramma
2	Trasporto	consegna processo-a-processo; unità: segmento
3	Collegamento	consegna *hop-to-hop*; unità: frame
4	Fisico	unità: bit
5	Applicazione	HTTP; unità: messaggio
6	Trasporto	il trasferimento affidabile è di TCP (end-to-end)
7	Applicazione	il DNS è un'applicazione che offre un servizio alle altre applicazioni
8	Collegamento; Rete; Fisico	lo switch implementa 2 livelli, il router 3, l'hub solo il fisico

Tabella 31.2 – Livelli e unità di dati.

Esercizio 10 — Che servizio serve?

Per ciascuna applicazione scegliere il tipo di servizio più adatto tra i sei visti nel capitolo sulla stratificazione, e dire se ci si aspetta che usi TCP o UDP. (a) scaricare un file ISO; (b) una telefonata VoIP; (c) una query DNS; (d) stampare un libro diviso in pagine; (e) pubblicità inviata in massa; (f) un messaggio con conferma di lettura.

Svolgimento

- **(a)** Flusso di byte affidabile (TCP): non può mancare nemmeno un byte.
- **(b)** Connessione inaffidabile (tipicamente UDP): un pacchetto perso è un piccolo "click", uno in ritardo è inutile.
- **(c)** Richiesta-risposta (UDP): una domanda, una risposta; se si perde, si ripete.
- **(d)** Flusso di messaggi affidabile: i confini tra le pagine vanno preservati.
- **(e)** Datagram inaffidabile: nessuno si preoccupa se qualche messaggio va perso.
- **(f)** Datagram con conferma: niente connessione, ma il mittente vuole sapere che il messaggio è arrivato.

Esercizio 11 — Chi legge quale header

L'host A invia una richiesta HTTP al server B. Il percorso attraversa uno switch, poi un router, poi arriva a B. Per ciascun dispositivo dire fino a quale livello "apre" il pacchetto e quali campi legge o modifica.

Svolgimento

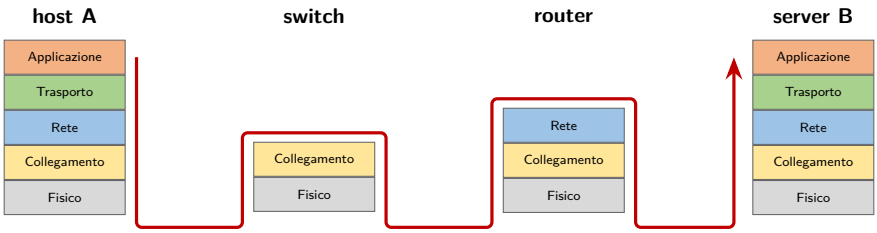


Figura 31.2 – Il pacchetto sale lungo la pila solo fino al livello che il dispositivo implementa.

- **Switch** (livello 2): legge l'header Ethernet, in particolare il **MAC di destinazione**, per scegliere la porta d'uscita. Non modifica il frame e non guarda dentro il payload.
- **Router** (livello 3): rimuove l'header Ethernet, legge l'**IP di destinazione** per scegliere l'uscita, **decrementa il TTL** e ricalcola il checksum IP, poi incapsula il datagramma in un **nuovo frame** con nuovi MAC. Non legge l'header TCP né la richiesta HTTP.
- **Server B**: decapsula tutti gli strati fino alla richiesta HTTP, che consegna al processo server individuato dalla porta di destinazione (80).

Ogni strato tratta il proprio payload come **opaco**: il router trasporta la richiesta HTTP senza sapere che esiste.

Esercizio 12 — Simplex, half-duplex o full-duplex?

Classificare: (a) una trasmissione televisiva; (b) una coppia di walkie-talkie; (c) una telefonata; (d) una tastiera collegata a un computer; (e) un segmento Ethernet condiviso su hub; (f) un collegamento Ethernet commutato moderno.

Svolgimento

(a) simplex; (b) half-duplex (si parla a turno, "passo"); (c) full-duplex; (d) simplex; (e) half-duplex (una stazione alla volta, altrimenti collisione); (f) full-duplex (coppie di fili separate per i due versi, niente collisioni).

Formule da ricordare

- $d_{\text{trasm}} = L/R$; $\quad d_{\text{prop}} = d/s$; $\quad$ lunghezza di un bit sul mezzo $= s/R$.
- P pacchetti su N collegamenti store-and-forward: $T = (P + N - 1) \cdot L/R$.
- Throughput end-to-end $= \min_i R_i$ (tenendo conto della condivisione).
- Maglia completa: $n(n-1)/2$ collegamenti; stella e anello: n.
- Overhead degli header: $n\,h/(M + n\,h)$.

Esercizi sul livello applicativo

Preparare Wireshark

Prima di svolgere gli esercizi di laboratorio serve un Wireshark funzionante e utilizzabile **da utente non-root** (catturare pacchetti come root è sconsigliato: Wireshark analizza dati che arrivano dalla rete, cioè potenzialmente ostili).

```
sudo apt update
sudo apt install wireshark          # rispondere "Sì" alla cattura per utenti non-root

getent group wireshark              # esiste il gruppo wireshark?
groups                              # l'utente ne fa parte?

sudo dpkg-reconfigure wireshark-common      # se serve cambiare la scelta fatta all'installazione
sudo usermod -aG wireshark $USER            # aggiungere l'utente al gruppo
newgrp wireshark                            # (oppure logout e login)

getcap /usr/bin/dumpcap             # deve mostrare cap_net_raw,cap_net_admin=eip
sudo setcap cap_net_raw,cap_net_admin+eip /usr/bin/dumpcap   # solo se mancano i permessi
```

Chi cattura davvero i pacchetti

La cattura non la fa l'interfaccia grafica ma il piccolo programma `dumpcap`, che ha bisogno delle *capabilities* `cap_net_raw` (leggere pacchetti grezzi) e `cap_net_admin` (mettere l'interfaccia in modalità promiscua). Assegnarle solo a lui è più sicuro che eseguire Wireshark come root.

HTTP con Wireshark

Esercizio 1 — Interazione base: GET e risposta

1. Avviare il browser e Wireshark, senza iniziare la cattura. Nel campo del filtro di visualizzazione scrivere `http`.
2. Attendere poco più di un minuto (il motivo sarà chiaro alla fine), poi avviare la cattura.
3. Visitare `http://www.columbia.edu/~fdc/sample.html` e fermare la cattura.

Si vedranno la richiesta GET e la risposta del server. Espandere solo la sezione HTTP (Frame, Ethernet, IP e TCP chiuse) e rispondere:

(a) Quale versione di HTTP usa il browser? E il server?

(b) Quali lingue il browser dichiara di accettare?

(c) Qual è l'indirizzo IP del proprio computer? E quello del server?

(d) Qual è il codice di stato restituito?

(e) Quando è stata modificata l'ultima volta la pagina?

(f) Quanti byte di contenuto sono stati inviati?

(g) Nei byte grezzi ci sono header che non compaiono nella lista dei pacchetti?

Svolgimento

Una cattura tipica ha questo aspetto (i valori esatti dipendono dal browser e dal momento):

```
GET /~fdc/sample.html HTTP/1.1
Host: www.columbia.edu
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: it-IT,it;q=0.9,en-US;q=0.8,en;q=0.7
Accept-Encoding: gzip, deflate
Connection: keep-alive

HTTP/1.1 200 OK
Date: ...
Server: Apache
Last-Modified: ...
Content-Length: ...
Content-Type: text/html
```

(a) La versione del browser è in fondo alla **riga di richiesta**, quella del server all'inizio della **riga di stato**: tipicamente entrambe `HTTP/1.1`. Non devono coincidere per forza: il server risponde con la versione più alta che supporta compatibile con quella del client.

(b) Nell'header `Accept-Language`. I valori `q` (*quality factor*, da 0 a 1) indicano la preferenza relativa: prima l'italiano, poi l'inglese.

(c) Non sono nel messaggio HTTP: si leggono espandendo la sezione *Internet Protocol*. Nella GET *Source* è il proprio host e *Destination* il server; nella risposta sono invertiti.

(d) Nella riga di stato: `200 OK`.

(e) Nell'header `Last-Modified` della risposta.

(f) Nell'header `Content-Length`, che conta i byte del **corpo**, non dell'intero messaggio.

(g) Sì: la vista compatta mostra solo la prima riga, mentre nei byte grezzi si trovano tutti gli header (`Date`, `Server`, `Content-Type`, `User-Agent`, `Accept-Encoding`, e spesso header aggiunti da proxy e cache come `Via`, `X-Cache`, `ETag`).

Perché aspettare un minuto

Il server aggiorna il timestamp `Last-Modified` ogni minuto, così la pagina sembra modificata da meno di un minuto prima del download. È un trucco per **invalidare le cache** (del browser e dei proxy): un GET condizionale con `If-Modified-Since` riceverà sempre `200 OK` con la pagina, e mai `304 Not Modified`.

Esercizio 2 — Un server diverso

Ripetere l'esperimento con `http://scottaaronson.com`. Che differenze si notano nella risposta?

Svolgimento

Le differenze da cercare e commentare sono tipicamente:

- **redirezione**: molti siti rispondono `301 Moved Permanently` o `302 Found`, con un header `Location` che punta alla versione `https://`; si vede quindi una catena di richieste;
- un **software server** diverso (header `Server`, per esempio `nginx` invece di `Apache`);
- **assenza di `Last-Modified`** se la pagina è generata dinamicamente, magari sostituito da un `ETag`;
- `Transfer-Encoding: chunked` al posto di `Content-Length`, quando il server non conosce in anticipo la lunghezza della risposta;
- molte più GET (fogli di stile, script, immagini), spesso su più connessioni in parallelo.

Esercizio 3 — Una risposta divisa in più segmenti

Svuotare la cache del browser, avviare la cattura, visitare di nuovo `http://www.columbia.edu/~fdc/sample.html` e fermare la cattura. Il file HTML è abbastanza lungo da non stare in un solo segmento TCP.

(a) Quante GET ha inviato il browser? In quale pacchetto si trova la GET della pagina?

(b) In quale pacchetto si trovano il codice di stato e la frase della risposta? Quali sono?

(c) Quanti segmenti TCP con dati sono serviti per trasportare la risposta?

Svolgimento

(a) Con la cache vuota c'è una GET per la pagina ed eventualmente altre per gli oggetti collegati (per esempio `favicon.ico`). Il numero di pacchetto è nella prima colonna; il filtro `http.request.method == "GET"` le isola.

(b) È il pacchetto con `HTTP/1.1 200 OK` nella colonna *Info*. Attenzione: **non** è il primo pacchetto della risposta ma l'**ultimo**, perché Wireshark mostra il messaggio HTTP solo quando ha riassemblato tutti i pezzi; i precedenti sono marcati "*TCP segment of a reassembled PDU*".

(c) Selezionando la risposta, la sezione `[N Reassembled TCP Segments]` elenca tutti i segmenti usati, con i loro numeri di pacchetto: il conteggio si legge lì.

Stimare il numero di segmenti a mano

Se il corpo è di L byte, gli header HTTP occupano H byte e la MSS vale 1460 byte, i segmenti di dati sono almeno

$$\left\lceil \frac{L + H}{\text{MSS}} \right\rceil \qquad \text{per esempio} \qquad \left\lceil \frac{30\,000 + 300}{1460} \right\rceil = \lceil 20{,}75 \rceil = 21$$

Il valore osservato può essere maggiore: il server non riempie sempre i segmenti.

Esercizio 4 — Una richiesta e la sua risposta

Un browser invia la richiesta seguente e riceve la risposta riportata sotto.

```
GET /corsi/reti/index.html HTTP/1.1
Host: www.docenti.esempio.it
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml
Accept-Language: it-IT,it;q=0.8,en;q=0.5
Connection: keep-alive
If-Modified-Since: Mon, 14 Sep 2026 08:12:00 GMT

HTTP/1.1 304 Not Modified
Date: Tue, 15 Sep 2026 10:02:11 GMT
Server: Apache
Connection: Keep-Alive
Keep-Alive: timeout=5, max=100
ETag: "3f80f-1b6-5f4e2c1a"
```

(a) Qual è l'URL completo dell'oggetto richiesto?

(b) Il browser chiede una connessione persistente o non persistente?

(c) Si può ricavare dal messaggio l'indirizzo IP del client?

(d) Perché la richiesta contiene `If-Modified-Since`? Quali risposte sono possibili?

(e) L'oggetto è stato trasferito? Che cosa mostrerà il browser?

(f) Per quanto tempo il server terrà aperta la connessione?

Svolgimento

(a) Si ottiene unendo `Host` e il percorso della riga di richiesta: `http://www.docenti.esempio.it/corsi/reti/index.html`.

(b) Persistente: `Connection: keep-alive` (in HTTP/1.1 la connessione è comunque persistente di default).

(c) No: l'indirizzo IP non fa parte del messaggio HTTP ma dell'header del datagramma IP che lo trasporta. Il messaggio applicativo non sa (e non deve sapere) nulla degli strati inferiori.

(d) Il browser (o una cache) possiede già una copia dell'oggetto ricevuta il 14 settembre, e chiede di riceverlo **solo se** è cambiato da allora: è un **GET condizionale**. Il server risponde `200 OK` con il nuovo oggetto se è stato modificato, `304 Not Modified` senza corpo altrimenti.

(e) No: la risposta 304 non ha corpo. Il browser mostra la copia che ha in cache, risparmiando banda e tempo.

(f) `Keep-Alive: timeout=5, max=100`: la connessione viene chiusa dopo 5 secondi di inattività o dopo 100 richieste.

Esercizio 5 — Cookie passo per passo

Un utente visita per la prima volta il sito di un negozio online, poi ci torna il giorno dopo dallo stesso browser. Descrivere gli header relativi ai cookie in ciascun messaggio e che cosa fa il server con il valore del cookie.

Svolgimento

1. **Prima richiesta**: il browser non ha cookie per quel sito, quindi la richiesta non contiene l'header `Cookie`.
2. **Prima risposta**: il server crea un identificativo (per esempio `1678`), crea una riga per quell'utente nel proprio **database di back-end** e risponde con `Set-Cookie: id=1678`.
3. Il browser memorizza la coppia (sito, `id=1678`) nel proprio file dei cookie.
4. **Richieste successive** (anche il giorno dopo): il browser allega `Cookie: id=1678`. Il server consulta il database e "riconosce" l'utente: carrello, preferenze, raccomandazioni.

HTTP resta **senza stato**: lo stato è mantenuto agli estremi (file dei cookie nel browser, database nel server) e il cookie è solo la chiave che li collega.

TEMPI DI RISPOSTA

Esercizio 6 — DNS, connessioni e pipelining

Un utente clicca un link. L'indirizzo IP del server non è in cache: per risolverlo si interrogano in sequenza 3 server DNS, con RTT di 20, 30 e 50 ms. La pagina è composta da un file HTML base e da 8 piccoli oggetti sullo stesso server, con $\mathrm{RTT}_0 = 40$ ms. I tempi di trasmissione sono trascurabili. Calcolare il tempo per ottenere:

(a) il solo file HTML;

(b) la pagina completa con connessioni non persistenti, in sequenza;

(c) la pagina completa con connessioni non persistenti e fino a 5 connessioni parallele;

(d) la pagina completa con una connessione persistente senza pipelining;

(e) la pagina completa con una connessione persistente con pipelining.

Svolgimento

Il DNS costa $20 + 30 + 50 = 100$ ms. Ogni connessione TCP nuova costa 1 RTT di apertura più 1 RTT per richiesta e risposta.

(a) $100 + 2\,\mathrm{RTT}_0 = 100 + 80 =$ **180 ms**.

(b) Ogni oggetto richiede una nuova connessione: $180 + 8 \cdot 2\,\mathrm{RTT}_0 = 180 + 640 =$ **820 ms**.

(c) Gli 8 oggetti si scaricano in due "giri" (5 connessioni, poi 3), ciascuno da $2\,\mathrm{RTT}_0$: $180 + 2 \cdot 80 =$ **340 ms**.

(d) La connessione resta aperta; ogni oggetto costa 1 RTT: $180 + 8 \cdot 40 =$ **500 ms**.

(e) Le 8 richieste partono una dietro l'altra e le risposte arrivano insieme: $180 + 40 =$ **220 ms**.

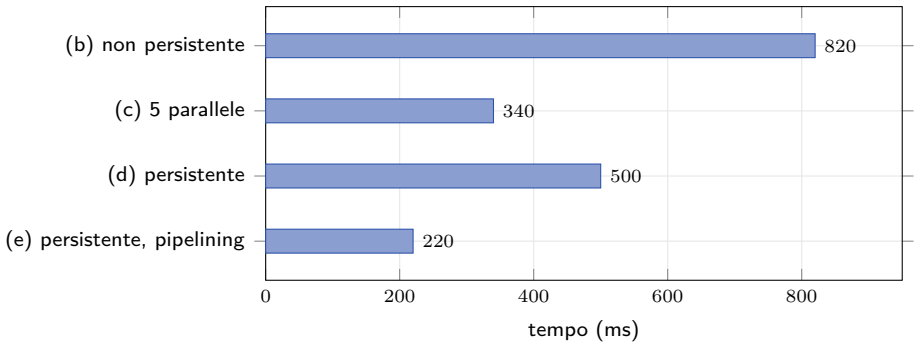


Figura 32.1 – Tempo per scaricare la pagina completa nei quattro casi (DNS incluso).

Esercizio 7 — Distribuire un file: client-server o P2P?

Un file di $F = 15$ Gbit deve essere distribuito a N utenti. Il server ha capacità di upload $u_s = 30$ Mb/s; ogni utente scarica a $d_i = 2$ Mb/s e, nel caso P2P, carica a $u_i = 700$ kb/s. Calcolare il tempo minimo di distribuzione con architettura client-server e P2P, per $N = 10, 100, 1000$.

Svolgimento

Client-server. Il server deve caricare N copie (NF bit, al massimo a u_s), e ogni client non può scaricare più velocemente di $d_{\min}$:

$$D_{\text{cs}} \geq \max\left\{\frac{NF}{u_s},\ \frac{F}{d_{\min}}\right\}$$

P2P. Il server deve caricare almeno *una* copia; il client più lento scarica a $d_{\min}$; e in tutto vanno caricati NF bit, usando la capacità del server *più quella di tutti i peer*:

$$D_{\text{P2P}} \geq \max\left\{\frac{F}{u_s},\ \frac{F}{d_{\min}},\ \frac{NF}{u_s + \sum_i u_i}\right\}$$

Con i dati: $F/u_s = 15 \cdot 10^9 / 3 \cdot 10^7 = 500$ s; $F/d_{\min} = 15 \cdot 10^9 / 2 \cdot 10^6 = 7500$ s.

N	NF/u_s	$NF/(u_s + Nu)$	
10	5 000 s	$150 \cdot 10^9 / 37 \cdot 10^6 \approx 4\,054$ s	$D_{\text{cs}} = 7\,500$ s, $D_{\text{P2P}} = 7\,500$ s
100	50 000 s	$1{,}5 \cdot 10^{12} / 10^8 = 15\,000$ s	$D_{\text{cs}} = 50\,000$ s, $D_{\text{P2P}} = 15\,000$ s
1000	500 000 s	$1{,}5 \cdot 10^{13} / 7{,}3 \cdot 10^8 \approx 20\,548$ s	$D_{\text{cs}} = 500\,000$ s, $D_{\text{P2P}} \approx 20\,548$ s

Tabella 32.1 – Tempi minimi di distribuzione.

Nel client-server il tempo cresce **linearmente** con N; nel P2P ogni nuovo peer porta anche capacità di upload, e il tempo resta limitato: per $N \to \infty$ tende a $F/u = 15 \cdot 10^9 / 7 \cdot 10^5 \approx 21\,429$ s. È la **scalabilità intrinseca** del P2P (la curva vista nel capitolo sulla posta e il P2P).

DNS CON NSLOOKUP

Esercizio 8 — Esplorare un dominio

Eseguire in sequenza, su un dominio a scelta (a lezione `vatican.va`): una query A, una query NS, una query SOA, una query A per il name server principale e infine la query A rivolta **direttamente** al name server principale. Commentare ciascuna risposta.

Svolgimento

Gli output completi per `vatican.va` sono riportati nel capitolo sul DNS; qui li riassumiamo.

Comando	Risultato	Commento
`nslookup vatican.va`	`185.152.70.106`	record A. *Non-authoritative*: risponde il resolver locale (`127.0.0.53`), non un server autorevole
`nslookup -type=NS vatican.va`	`a.nic.va`, `b.nic.va`, `c.nic.va`, `osiris.namex.it`, `seth.namex.it`	i server autorevoli (per nome, non per indirizzo)
`nslookup -type=SOA vatican.va`	origin `a.nic.va`, serial, refresh, …	`a.nic.va` è il primario; nessun glue record in coda
`nslookup -type=A a.nic.va`	`212.77.0.110`	l'indirizzo del primario
`nslookup -type=A vatican.va 212.77.0.110`	`185.152.70.106`	chiesto al server autorevole: dovrebbe essere *authoritative*, ma il flag AA può non essere impostato per configurazione

Tabella 32.2 – Le cinque query su `vatican.va`.

I campi del record SOA

- `serial` (2025062401): versione della zona, incrementata a ogni modifica; i secondari la confrontano per capire se aggiornarsi.
- `refresh` (14400 s = 4 ore): ogni quanto un secondario controlla il primario.
- `retry` (3600 s = 1 ora): dopo quanto riprovare se il controllo fallisce.
- `expire` (1209600 s = 14 giorni): dopo quanto un secondario che non raggiunge il primario smette di considerarsi autorevole.
- `minimum` (3600 s): TTL minimo, usato anche per le risposte negative ("il nome non esiste").
- `mail addr` (`dnsop.net.va`): l'indirizzo dell'amministratore, con il primo punto al posto della @.

Esempio 1 — Il mail server di un dominio

```
$ nslookup -type=MX mit.edu
mit.edu   mail exchanger = 100 mit-edu.mail.protection.outlook.com.

$ nslookup -type=A mit-edu.mail.protection.outlook.com
Address: 52.101.11.3
Address: 52.101.8.44
Address: 52.101.42.14
Address: 52.101.40.1
```

Prima si ottiene il **nome** del mail server (record MX), poi il suo **indirizzo** (record A). Il 100 è la **preferenza**: con più record MX si prova prima quello con il valore più basso. I quattro indirizzi sono un esempio di **distribuzione del carico**: il DNS li restituisce ruotandone l'ordine.

Esempio 2 — Alias e nome canonico

```
$ nslookup www.mit.edu
www.mit.edu               canonical name = www.mit.edu.edgekey.net.
www.mit.edu.edgekey.net   canonical name = e9566.dscb.akamaiedge.net.
Name:    e9566.dscb.akamaiedge.net
Address: 23.50.153.111
```

Una catena di CNAME: `www.mit.edu` è un alias, e il nome canonico finale è un nodo della CDN Akamai. È il servizio di **host aliasing** del DNS, usato dalle CDN per indirizzare ogni utente verso un server vicino.

DNS con Wireshark

Esercizio 9 — Una query standard

Avviare la cattura, eseguire `nslookup <nome_a_piacere>`, fermare la cattura e filtrare con `dns`.

(a) Qual è la porta di destinazione della richiesta? E la porta sorgente della risposta?

(b) A quale indirizzo IP è inviata la richiesta? È il server DNS locale predefinito?

(c) Di che tipo è la query? La richiesta contiene già delle risposte?

(d) Quante domande e quante risposte contiene il messaggio di risposta?

Svolgimento

(a) Entrambe **53**, la porta ben nota del DNS (sezione *User Datagram Protocol*). La porta di destinazione della risposta è invece la porta effimera scelta dal sistema operativo per il client.

(b) Tipicamente `127.0.0.53` (il resolver locale `systemd-resolved`), oppure il router di casa o il server dell'ISP. Si verifica con `cat /etc/resolv.conf` o `resolvectl status`: deve coincidere.

(c) Sezione *Queries*, campo *Type*: `A (Host Address)`. La richiesta ha *Answer RRs* = 0: una domanda non contiene risposte.

(d) *Questions* = 1 (la domanda viene ripetuta nella risposta, così il client può appaiarle) e *Answer RRs* pari al numero di record restituiti, spesso più di uno (CNAME intermedi, più indirizzi).

I campi da guardare nell'header DNS

- **Transaction ID**: appaia richiesta e risposta (ed è ciò che un attaccante dovrebbe indovinare per falsificare una risposta).
- **QR**: 0 per le query, 1 per le risposte.
- **RD** (*Recursion Desired*): il client chiede la risoluzione ricorsiva; **RA** (*Recursion Available*): il server la offre.
- **AA** (*Authoritative Answer*): la risposta viene da un server autorevole per il nome.

Esercizio 10 — Una query NS

Ripetere con `nslookup -type=NS <nome_a_piacere>`.

(a) A quale indirizzo è inviata la richiesta? Quante domande contiene? Contiene risposte?

(b) Nella risposta: quante risposte ci sono e che informazioni contengono? Quanti record aggiuntivi, e di che tipo?

Suggerimento: interrogare anche server in alto nella gerarchia (inclusi i root server) con l'opzione `-norecurse`, e poi senza, confrontando i risultati.

Svolgimento

(a) Lo stesso resolver locale: il tipo di query non cambia il destinatario. Una domanda, di tipo NS, e nessuna risposta.

(b) Tante risposte quanti sono i name server autorevoli del dominio: ognuna contiene il **nome** di un server. Nella sezione *Additional records* ci sono i **glue record**: record A (e AAAA) con gli **indirizzi** di quei server.

Perché servono i glue record

Per risolvere `esempio.it` devo chiedere a `ns1.esempio.it`; ma per contattare `ns1.esempio.it` devo prima risolvere il suo nome, cosa che richiede di chiedere a `ns1.esempio.it`… Il glue record spezza questa **dipendenza circolare** fornendo direttamente l'indirizzo.

L'effetto di `-norecurse`

Con `-norecurse` il flag RD è 0: il server risponde solo con ciò che sa già. Un root server risponde con un **referral** (i name server del TLD, con i glue record), non con l'indirizzo finale. Senza l'opzione il risultato non cambia, perché i root server **non offrono** la ricorsione (RA = 0). Per esempio `nslookup iana.org c.root-servers.net` non restituisce record A, ma i name server del dominio `org`. (Se si è dietro una VPN il risultato può sorprendere: si veda il caso di studio nel capitolo su ICMP.)

Esercizio 11 — Una domanda senza ricorsione, con dig

Prima di chiedere qualcosa a un root server bisogna sapere dov'è: il primo scambio riguarda il server, il secondo va a lui.

```
$ dig +short a.root-servers.net A
198.41.0.4
$ dig +norecurse @198.41.0.4 www.ox.ac.uk
$ dig @198.41.0.4 www.ox.ac.uk
```

(a) Il root server restituisce un indirizzo o un rinvio? Quanti record nella sezione *answer*, quanti nella *authority*?

(b) Che cosa cambia sul filo fra gli ultimi due comandi, e chi fa il lavoro in ciascun caso?

(c) Controllare i flag della risposta: quali ci si aspetta da un root server?

Svolgimento

(a) Un **rinvio** (*referral*): zero record di risposta, nella sezione *authority* i record NS dei server del TLD `uk`, e nella sezione *additional* i loro indirizzi (glue record). Il root server non conosce `www.ox.ac.uk`: sa solo a chi chiedere un livello più in basso.

(b) Sul filo cambia **un bit**: con `+norecurse` il flag RD della domanda vale 0, senza vale 1. La risposta invece è la stessa, perché un root server non fa ricorsione per nessuno (senza l'opzione `dig` lo segnala con `WARNING: recursion requested but not available`): in entrambi i casi il lavoro di seguire i rinvii resta a chi chiede, cioè a `dig` (o al proprio resolver).

(c) Un root server **non imposta mai `ra`**. In un rinvio manca anche `aa`, che segnala una risposta autorevole per il nome chiesto: la si vede chiedendo un nome della zona radice, per esempio `dig +norecurse @198.41.0.4 . NS`. Se nella risposta compare `ra`, a rispondere non è stato il root server ma qualcosa sulla propria linea (un resolver trasparente dell'ISP, un firewall, una VPN) che intercetta il traffico DNS e risponde al suo posto: l'esercizio è falsato, e va ripetuto da un'altra rete.

DNS: DOMANDE D'ESAME

Esercizio 12 — Cambiare l'indirizzo di un server

Si cambia l'indirizzo IP di una macchina di nome `server-1.ortofrutta.it`, che ospita (per esempio) un server FTP. Spiegare come è gestito l'aggiornamento affinché la modifica sia riflessa nel DNS.

Svolgimento

Che la macchina ospiti un server FTP è irrilevante. L'associazione nome–indirizzo è nel server **autorevole**, dove l'amministratore la modifica, ma è **replicata** nelle cache di moltissimi server. Il DNS non avvisa le cache: il punto cruciale è il **TTL**. Nessun aggiornamento attivo; si lascia che i record in cache **scadano**, e alla richiesta successiva vengono sostituiti da quelli aggiornati. Nel frattempo alcuni client riceveranno il vecchio indirizzo: errori temporanei che vengono tollerati.

La strategia pratica

Chi sa di dover cambiare indirizzo **abbassa il TTL** (per esempio a 60 s) qualche giorno prima, fa la modifica, e rialza il TTL a transizione completata. La finestra di incoerenza passa da ore a un minuto.

Esercizio 13 — Risolvere un nome con una cache parziale

Si deve risolvere `feijoada.dsc.utfpr.edu.br` da un portatile in un laboratorio di UniNA, con la cache vuota. Il portatile interroga `dscna2.unina.it`, nella cui cache (per interrogazioni precedenti) c'è un record NS con l'indirizzo del server DNS di `dsc.utfpr.edu.br`. Spiegare come il name server di UniNA gestisce la richiesta e come ci si aspetta che risponda il server di `dsc.utfpr.edu.br`.

Svolgimento

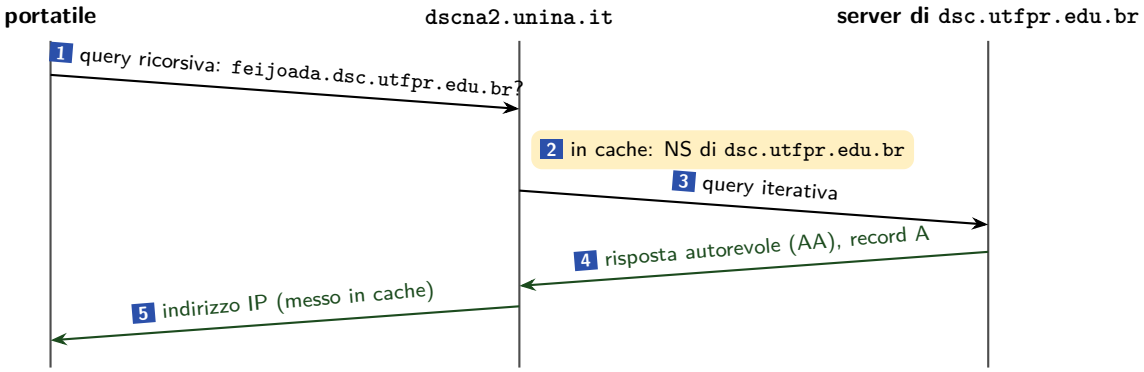


Figura 32.2 – La cache permette di saltare root server, `br`, `edu.br` e `utfpr.edu.br`.

1. Il portatile invia una query **ricorsiva**: vuole l'indirizzo finale (o un errore).
2. `dscna2.unina.it` consulta la cache e trova il record NS (con l'indirizzo) del server della zona `dsc.utfpr.edu.br`: **non deve partire dai root server**, e salta i passi root → `br` → `edu.br` → `utfpr.edu.br`.
3. Invia una query **iterativa** direttamente a quel server, chiedendo il record A di `feijoada.dsc.utfpr.edu.br`.
4. Il server è autorevole per la zona: risponde con una **risposta autorevole** (flag AA) contenente il record A.
5. `dscna2.unina.it` mette la risposta in cache per la durata del TTL e la restituisce al portatile.

Il senso della domanda è il ruolo della cache: un solo record NS risparmia 3–4 scambi. È anche il motivo per cui i root server non collassano: la maggior parte delle risoluzioni non li raggiunge mai.

Esercizio 14 — Contare i messaggi DNS

Un host con cache vuota vuole risolvere `www.cs.esempio.edu`; anche il server DNS locale ha la cache vuota, ed è autorevole per `cs.esempio.edu` un server distinto da quello di `esempio.edu`. Quanti messaggi DNS vengono scambiati in tutto, se il server locale procede in modo **iterativo**? Quanti ne gestisce il root server se invece tutta la catena procede in modo **ricorsivo**?

Svolgimento

In modo iterativo: host → locale (1), locale ↔ root (2), locale ↔ TLD `edu` (2), locale ↔ server di `esempio.edu` (2), locale ↔ server di `cs.esempio.edu` (2), locale → host (1): **10 messaggi**. Il root server gestisce una domanda e una risposta.

In modo ricorsivo i messaggi sono sempre 10, ma ogni server deve **restare in attesa** della risposta del successivo, mantenendo lo stato della richiesta: il root server resterebbe impegnato per tutta la durata della risoluzione. È per questo che i server in alto nella gerarchia rispondono solo in modo iterativo.

Esercizio 15 — Scrivere i resource record

La piccola azienda `ortofrutta.it` ha: un name server `ns1.ortofrutta.it` (`203.0.113.2`); un server `server-1.ortofrutta.it` (`203.0.113.10`) che ospita il sito, raggiungibile come `www.ortofrutta.it`; un mail server `mail.ortofrutta.it` (`203.0.113.25`) che riceve la posta `@ortofrutta.it`. Scrivere i resource record nel server autorevole dell'azienda e quelli da inserire nel server del TLD `it`.

Svolgimento

Nel formato (NAME, VALUE, TYPE), trascurando il TTL:

```
; server autorevole di ortofrutta.it
(ortofrutta.it,           ns1.ortofrutta.it,        NS)
(ns1.ortofrutta.it,       203.0.113.2,              A)
(server-1.ortofrutta.it,  203.0.113.10,             A)
(www.ortofrutta.it,       server-1.ortofrutta.it,   CNAME)
(ortofrutta.it,           mail.ortofrutta.it,       MX)    ; con preferenza, es. 10
(mail.ortofrutta.it,      203.0.113.25,             A)

; server del TLD it
(ortofrutta.it,           ns1.ortofrutta.it,        NS)
(ns1.ortofrutta.it,       203.0.113.2,              A)     ; glue record
```

Il record A di `ns1` nel server del TLD è proprio un **glue record**: senza, nessuno potrebbe contattare il name server dell'azienda.

SOCKET

Esercizio 16 — Chi parte per primo

Con i programmi client e server UDP e TCP visti nella parte sulla programmazione di rete:

(a) che cosa succede se si avvia il client TCP **prima** del server?

(b) e se si avvia il client UDP prima del server?

(c) un server UDP con un solo socket può servire due client? E un server TCP?

(d) che numero di porta usa il client, se non chiama `bind()`?

Svolgimento

(a) `connect()` fallisce subito: il SYN del client raggiunge un host dove nessun processo è in ascolto su quella porta, e il sistema operativo risponde con un segmento **RST**. Il programma riceve l'errore *Connection refused*.

(b) `sendto()` ha successo (UDP non verifica nulla), ma il datagramma viene scartato a destinazione (al più torna un messaggio ICMP *port unreachable*). Il client resta **bloccato per sempre** in `recvfrom()`, a meno di impostare un timeout: con UDP l'affidabilità è compito dell'applicazione.

(c) UDP: sì, lo stesso socket riceve datagrammi da chiunque, e `recvfrom()` dice chi è il mittente a cui rispondere. TCP: il server usa un **welcoming socket** per accettare le connessioni e un **socket dedicato** per ciascun client, restituito da `accept()`.

(d) Una porta **effimera** assegnata automaticamente dal sistema operativo (su Linux tipicamente tra 32768 e 60999).

Esercizio 17 — HTTP

(a) Spiegare quali sono le parti di un URL e l'uso di ciascuna.

(b) Spiegare brevemente il three-way handshake usato per stabilire una connessione HTTP.

(c) Illustrare che cos'è il pipelining in HTTP.

(d) Su quali protocolli di trasporto si basa HTTP?

Le risposte devono essere complete e possono essere concise.

Svolgimento

(a) La struttura è `protocol://[userinfo@]host[:port][/path][?query][#fragment]`:

- **protocol**: come accedere alla risorsa (`http`, `https`, `ftp`…); determina la porta di default;
- **userinfo** (facoltativo, *deprecato*): nome utente e password, che finirebbero in chiaro nei log e nella cronologia;
- **host**: nome (da risolvere con il DNS) o indirizzo IP del server;
- **port** (facoltativa): se omessa, 80 per HTTP e 443 per HTTPS;
- **path**: il percorso della risorsa sul server;
- **query**: dopo `?`, parametri della richiesta (`chiave=valore&…`);
- **fragment**: dopo `#`, una parte della risorsa; è gestito **solo dal client** e non viene mai inviato al server.

(b) Il three-way handshake **non è di HTTP ma di TCP**, su cui HTTP si appoggia:

1. il client invia un **SYN** con un numero di sequenza iniziale casuale `client_isn`, senza dati;
2. il server alloca buffer e variabili e risponde con **SYN-ACK**: ack = `client_isn`+1 e il proprio `server_isn` casuale;
3. il client risponde con un **ACK** (ack = `server_isn`+1), che può già contenere la richiesta HTTP.

L'apertura costa 1 RTT: una richiesta su una connessione nuova costa circa 2 RTT più il tempo di trasmissione.

(c) Con il **pipelining** il client invia più richieste **una dietro l'altra**, senza aspettare le risposte; richiede una connessione persistente e risparmia RTT. Da HTTP/2 richieste e risposte possono anche essere **interlacciate** sulla stessa connessione, con priorità, evitando che un oggetto grande blocchi i successivi.

(d) Tradizionalmente **TCP** (porta 80, o 443 con TLS). Con **HTTP/3** anche **UDP**, attraverso **QUIC**, che reimplementa affidabilità e controllo di congestione sopra UDP.

Formule da ricordare

- Connessione non persistente: $2\,\mathrm{RTT} + t_{\mathrm{trasm}}$ per oggetto; persistente: 1 RTT per oggetto dopo il primo; con pipelining: 1 RTT per tutti.
- Distribuzione di file: $D_{\mathrm{cs}} \geq \max\{NF/u_s,\ F/d_{\min}\}$; $D_{\mathrm{P2P}} \geq \max\{F/u_s,\ F/d_{\min},\ NF/(u_s + \sum u_i)\}$.
- DNS: porta 53 su UDP; la cache e il TTL governano la propagazione delle modifiche.

Esercizi sul livello di trasporto

Il checksum

Esercizio 1 — Checksum con due overflow

Calcolare il checksum UDP di un segmento formato dalle tre parole a 16 bit

`1110011001100110` `1101010101010101` `1000111100001100`

(a) Calcolare il checksum e verificare che il ricevente ottenga sedici 1.

(b) Il checksum rileva sempre un errore su un singolo bit? E su due bit?

Svolgimento

Il procedimento è: somma delle parole, **wrap around** di ogni riporto oltre il sedicesimo bit, **complemento a 1** finale. (Nel capitolo su UDP c'è lo stesso calcolo con parole diverse.)

```
  1110011001100110
+ 1101010101010101
------------------
 11011101110111011      17 bit: overflow!
  1011101110111011
+                1      wrap around
------------------
  1011101110111100
+ 1000111100001100
------------------
 10100101011001000      di nuovo overflow
  0100101011001000
+                1      wrap around
------------------
  0100101011001001      somma
  1011010100110110      CHECKSUM = complemento a 1 della somma
```

Verifica. Il ricevente somma le tre parole (ottenendo di nuovo `0100101011001001`) e il checksum:

```
  0100101011001001
+ 1011010100110110
------------------
  1111111111111111      sedici 1: nessun errore rilevato
```

(b) Un errore su un bit cambia la somma, quindi è **sempre** rilevato. Due errori possono invece compensarsi: se nella stessa posizione un bit passa da 0 a 1 in una parola e da 1 a 0 in un'altra, la somma non cambia. Per esempio, invertendo l'ultimo bit delle prime due parole (…`0110` → …`0111` e …`0101` → …`0100`) il ricevente ottiene ancora sedici 1.

Gli errori tipici

- Buttare via il riporto invece di **risommarlo** (wrap around).
- Consegnare la somma invece del suo **complemento**.
- Dimenticare che, con un numero dispari di byte, si aggiunge un byte di **padding** a zero.

Prestazioni: stop-and-wait e pipelining

Esercizio 2 — Stop-and-wait via satellite

Due stazioni comunicano attraverso un satellite geostazionario, con RTT = 480 ms. Il collegamento ha rate $R = 10$ Mb/s e i pacchetti sono di $L = 1500$ byte; gli ACK hanno tempo di trasmissione trascurabile.

(a) Calcolare l'utilizzo del mittente con lo stop-and-wait e il throughput effettivo.

(b) Ripetere con pipelining di 10 pacchetti.

(c) Quanti pacchetti devono essere "in volo" per usare il canale al 100%? A quanti byte corrispondono?

(d) La receive window di TCP (16 bit) basta? Che cosa serve?

Svolgimento

(a) $L/R = 12\,000/10^7 = 1{,}2$ ms. Il mittente trasmette per 1,2 ms e poi aspetta l'ACK:

$$U = \frac{L/R}{\mathrm{RTT} + L/R} = \frac{1{,}2}{481{,}2} \approx 0{,}0025 = 0{,}25\%$$

Viene consegnato un pacchetto ogni 481,2 ms: throughput = $12\,000/0{,}4812 \approx 24{,}9$ kb/s, su un collegamento da 10 Mb/s.

(b) $U = \dfrac{10 \cdot L/R}{\mathrm{RTT} + L/R} = \dfrac{12}{481{,}2} \approx 2{,}5\%$: dieci volte meglio, ma ancora poco.

(c) Il mittente non si ferma mai se l'ACK del primo pacchetto torna prima che finisca di trasmettere l'N-esimo:

$$N \cdot \frac{L}{R} \geq \mathrm{RTT} + \frac{L}{R} \quad \Rightarrow \quad N \geq \frac{481{,}2}{1{,}2} = 401 \text{ pacchetti}$$

In byte è circa il **bandwidth-delay product**: $R \cdot \mathrm{RTT} = 10^7 \cdot 0{,}48 = 4{,}8 \cdot 10^6$ bit = 600 kB.

(d) No: con 16 bit la finestra annunciabile è al massimo 65 535 byte, circa un decimo di quanto serve. È il motivo dell'opzione TCP **window scaling**, che moltiplica la finestra per 2^k: qui basta $k = 4$ ($65\,535 \cdot 16 \approx 1$ MB).

Il caso del collegamento veloce

Con i valori del capitolo sul trasferimento affidabile (RTT = 30 ms, R = 1 Gb/s, L = 1000 byte) si ottiene $N \geq 30{,}008/0{,}008 \approx 3751$ pacchetti e un bandwidth-delay product di $10^9 \cdot 0{,}03 = 3 \cdot 10^7$ bit = 3,75 MB. Più il collegamento è veloce e lungo, più la finestra deve essere grande.

Go-Back-N e Selective Repeat

Esercizio 3 — Un pacchetto perso

Un mittente con finestra $N = 4$ deve inviare i pacchetti da 0 a 5. Il pacchetto 2 va perso la prima volta; tutto il resto arriva, compresi gli ACK. Descrivere il comportamento di mittente e ricevente con Go-Back-N e con Selective Repeat, e contare le trasmissioni di pacchetti dati.

Svolgimento

Evento	Go-Back-N	Selective Repeat
invio pkt0–pkt3	finestra $[0, 3]$	finestra $[0, 3]$
arrivano pkt0, pkt1	consegna; ACK0, ACK1	consegna; ACK0, ACK1
pkt2 perso	–	–
arriva pkt3	**fuori ordine: scartato**; ACK1 (duplicato)	**bufferizzato**; ACK3
arrivano ACK0, ACK1	finestra $[2, 5]$: invia pkt4, pkt5	finestra $[2, 5]$: invia pkt4, pkt5
arrivano pkt4, pkt5	scartati; ACK1, ACK1	bufferizzati; ACK4, ACK5
scade il timer	di pkt2: **ritrasmette pkt2, pkt3, pkt4, pkt5**	di pkt2: **ritrasmette solo pkt2**
arrivano le ritrasmissioni	consegna 2, 3, 4, 5; ACK2…ACK5	consegna 2, 3, 4, 5 (dal buffer); ACK2
Trasmissioni di dati	6 + 4 = **10**	6 + 1 = **7**

Tabella 33.1 – Lo stesso scenario con i due protocolli.

In Go-Back-N il ricevente è semplicissimo (ricorda solo il numero atteso), ma una sola perdita costa la ritrasmissione di tutta la finestra; in Selective Repeat il ricevente deve gestire un buffer e ACK individuali, ma si ritrasmette solo ciò che manca. Più la finestra è grande e il canale lossy, più la differenza pesa.

Esercizio 4 — Numeri di sequenza e finestra in Selective Repeat

Un Selective Repeat usa numeri di sequenza su 2 bit (da 0 a 3) e una finestra di 3 pacchetti. Mostrare con uno scenario che il ricevente può scambiare un pacchetto vecchio per uno nuovo. Qual è la finestra massima?

Svolgimento

Il mittente invia pkt0, pkt1, pkt2; il ricevente li riceve tutti, li consegna, invia ACK0, ACK1, ACK2 e sposta la sua finestra su $\{3, 0, 1\}$. **Tutti gli ACK vanno persi**. Scade il timer e il mittente ritrasmette il *vecchio* pkt0: il numero 0 cade nella finestra del ricevente, che lo accetta come se fosse il *nuovo* pkt0. Il ricevente non può distinguere i due casi, perché vede solo il numero di sequenza.

L'ambiguità scompare se le finestre "vecchia" e "nuova" del ricevente non si sovrappongono, cioè se la finestra è al più **metà dello spazio dei numeri**: $N \leq 2^k/2 = 2$.

Numeri di sequenza e di acknowledgment in TCP

Esercizio 5 — Una connessione dall'inizio alla fine

Un client apre una connessione TCP con numero di sequenza iniziale 1000; il server sceglie 5000. Il client invia una richiesta di 200 byte, il server risponde con 1000 byte (in un solo segmento), poi il client chiude la connessione. Scrivere seq, ack e flag di tutti i segmenti, assumendo che l'ACK del three-way handshake non trasporti dati.

Svolgimento

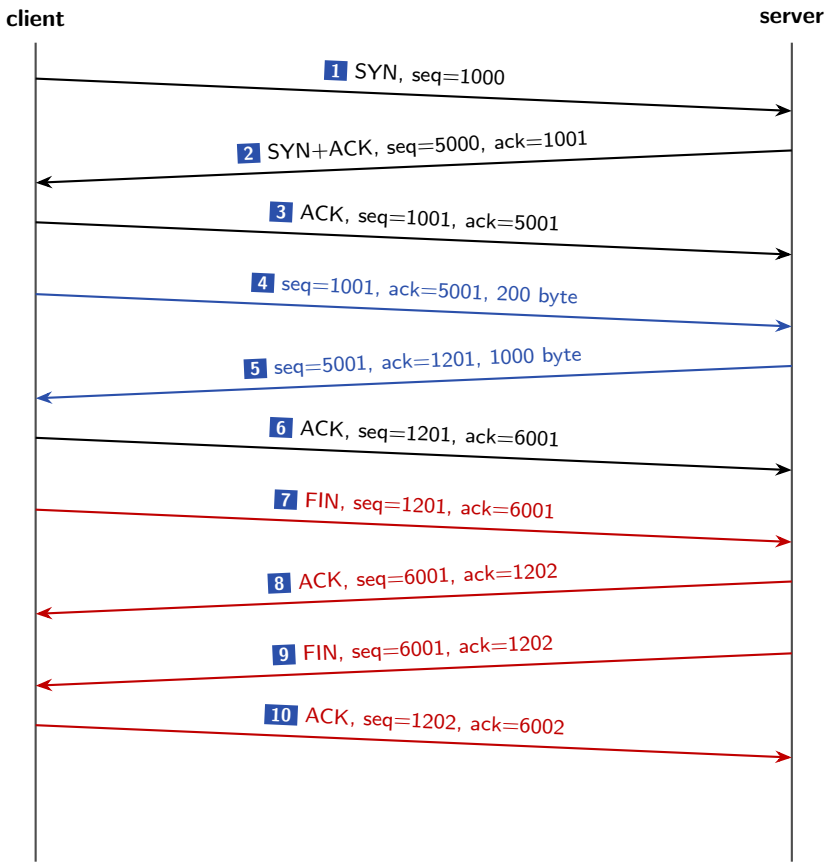


Figura 33.1 – Apertura (nero), dati (blu) e chiusura (rosso) di una connessione.

Le regole applicate:

- **SYN e FIN consumano un numero di sequenza** ciascuno, anche senza dati: per questo il primo byte di dati del client è il 1001 e l'ACK al FIN del client è 1202.
- Un **ACK puro** non consuma numeri: il segmento 3 e il segmento 4 hanno entrambi seq = 1001.
- ack = numero del **prossimo byte atteso** = seq ricevuto + byte ricevuti: dopo 200 byte da 1001 si attende il 1201; dopo 1000 byte da 5001 il 6001.

Esercizio 6 — Segmentare un flusso

Dopo l'handshake (ISN del mittente = 42) un'applicazione invia 12 000 byte con MSS di 1460 byte. Quanti segmenti si formano? Quali sono i numeri di sequenza del primo, del secondo e dell'ultimo? Quale ACK conferma l'intero flusso?

Svolgimento

$\lceil 12\,000/1460 \rceil = \lceil 8{,}22 \rceil = 9$ segmenti: 8 pieni ($8 \cdot 1460 = 11\,680$ byte) e un ultimo da 320 byte. Il SYN ha consumato il numero 42, quindi il primo byte di dati è il 43:

- 1° segmento: seq = 43 (byte 43–1502);
- 2° segmento: seq = 43 + 1460 = 1503;
- k-esimo segmento: seq = $43 + (k-1) \cdot 1460$; il 9°: seq = 43 + 11 680 = 11 723 (byte 11 723–12 042).

L'ACK che conferma tutto è 12 043.

Esercizio 7 — ACK cumulativi

L'host B ha ricevuto da A tutti i byte fino al 249. A invia a B due segmenti consecutivi, di 100 e 60 byte; il primo ha seq = 250. La porta di A è 40000, quella di B è 80. B invia un ACK per ogni segmento ricevuto.

(a) Quali sono seq, porta sorgente e porta destinazione del secondo segmento?

(b) Se il primo segmento arriva prima del secondo, quali sono ack e porte dell'ACK del primo?

(c) Se il secondo arriva per primo, quale ack contiene il suo ACK?

(d) Se il primo segmento va perso e il secondo arriva, che cosa succede?

Svolgimento

(a) seq = 250 + 100 = 350; porta sorgente 40000, destinazione 80.

(b) ack = 350; porta sorgente 80, destinazione 40000 (le porte si invertono).

(c) ack = 250: B ha un "buco" e continua a chiedere il byte 250. È un ACK **duplicato**; il segmento fuori ordine viene di solito bufferizzato.

(d) B risponde al secondo segmento con ack = 250. Allo scadere del timer (o dopo 3 ACK duplicati) A ritrasmette il segmento con seq = 250; B, che aveva bufferizzato il secondo, risponde con ack = 410: un solo ACK **cumulativo** conferma entrambi.

Stima dell'RTT e timeout

Esercizio 8 — Tre campioni

Inizialmente EstimatedRTT = 100 ms e DevRTT = 10 ms. Arrivano in sequenza i campioni 120, 140 e 90 ms. Con $\alpha = 0{,}125$ e $\beta = 0{,}25$ calcolare l'evoluzione di EstimatedRTT, DevRTT e TimeoutInterval, aggiornando DevRTT **prima** di EstimatedRTT (come prescrive l'RFC 6298).

Svolgimento

$$\text{DevRTT} \leftarrow 0{,}75\,\text{DevRTT} + 0{,}25\,|\text{Sample} - \text{EstimatedRTT}| \qquad \text{EstimatedRTT} \leftarrow 0{,}875\,\text{EstimatedRTT} + 0{,}125\,\text{Sample}$$

$$\text{Timeout} = \text{EstimatedRTT} + 4\,\text{DevRTT}$$

Campione	**DevRTT** (ms)	**EstimatedRTT** (ms)	**Timeout** (ms)
120	$0{,}75 \cdot 10 + 0{,}25 \cdot \lvert 120 - 100\rvert = 12{,}5$	$0{,}875 \cdot 100 + 0{,}125 \cdot 120 = 102{,}5$	$102{,}5 + 50 = 152{,}5$
140	$0{,}75 \cdot 12{,}5 + 0{,}25 \cdot \lvert 140 - 102{,}5\rvert = 18{,}75$	$89{,}69 + 17{,}5 = 107{,}19$	$107{,}19 + 75 = 182{,}19$
90	$0{,}75 \cdot 18{,}75 + 0{,}25 \cdot \lvert 90 - 107{,}19\rvert = 18{,}36$	$93{,}79 + 11{,}25 = 105{,}04$	$105{,}04 + 73{,}44 = 178{,}48$

Tabella 33.2 – Evoluzione della stima e del timeout.

Il timeout cresce quando i campioni oscillano (il secondo si scosta di 37,5 ms dalla stima) e si stabilizza quando tornano coerenti: quando si è incerti sull'RTT, si allarga il margine per evitare ritrasmissioni inutili.

L'ordine conta (un po')

Aggiornando prima EstimatedRTT e poi DevRTT (come nelle trasparenze), con il primo campione si ottiene Est = 102,5, Dev = $0{,}75 \cdot 10 + 0{,}25 \cdot |120 - 102{,}5| = 11{,}875$ e Timeout = 150 ms. All'esame si segue l'ordine indicato dal testo, dichiarandolo.

Controllo di flusso

Esercizio 9 — La finestra di ricezione

Il buffer di ricezione di B è RcvBuffer = 4000 byte; in un certo istante LastByteRcvd = 12 500 e LastByteRead = 10 000.

(a) Quanto vale la finestra annunciata ad A?

(b) Se A ha LastByteSent = 12 500 e LastByteAcked = 11 000, quanti byte può ancora inviare?

(c) Che cosa succede se l'applicazione in B smette di leggere?

Svolgimento

(a) Nel buffer ci sono 12 500 − 10 000 = 2500 byte non ancora letti: rwnd = 4000 − 2500 = 1500 byte.

(b) Il vincolo è LastByteSent − LastByteAcked ≤ rwnd. In volo ci sono già 12 500 − 11 000 = 1500 byte, esattamente rwnd: A **non può inviare nulla** finché non arrivano nuovi ACK o una finestra più grande.

(c) LastByteRead resta fermo, il buffer si riempie e rwnd scende a 0: A si ferma. Per non restare bloccato per sempre (B non ha dati da inviare e quindi non manderebbe aggiornamenti), A invia periodicamente **zero window probe** di un byte, le cui risposte riportano la finestra aggiornata.

Controllo di congestione

Esercizio 10 — Evoluzione di cwnd

Un mittente TCP Reno parte con cwnd = 1 MSS e ssthresh = 8 MSS. Tutti gli ACK arrivano regolarmente, tranne che: all'RTT 6 si verificano 3 ACK duplicati; all'RTT 10 scade un timeout. Tracciare cwnd e ssthresh per i primi 12 RTT, indicando lo stato.

Svolgimento

RTT	cwnd	ssthresh	Stato	Evento / azione
1	1	8	slow start	inizio
2	2	8	slow start	crescita esponenziale
3	4	8	slow start	
4	8	8	slow start	cwnd ≥ ssthresh: si passa a congestion avoidance
5	9	8	cong. avoidance	+1 MSS per RTT
6	10	8	cong. avoidance	**3 ACK duplicati**
7	8	5	fast recovery	ssthresh = 10/2 = 5; cwnd = 5 + 3 = 8; ritrasmissione
8	5	5	cong. avoidance	ACK nuovo: cwnd = ssthresh
9	6	5	cong. avoidance	
10	7	5	cong. avoidance	**timeout**
11	1	3	slow start	ssthresh = ⌊7/2⌋ = 3; cwnd = 1
12	2	3	slow start	

Tabella 33.3 – Evoluzione di cwnd e ssthresh (in MSS).

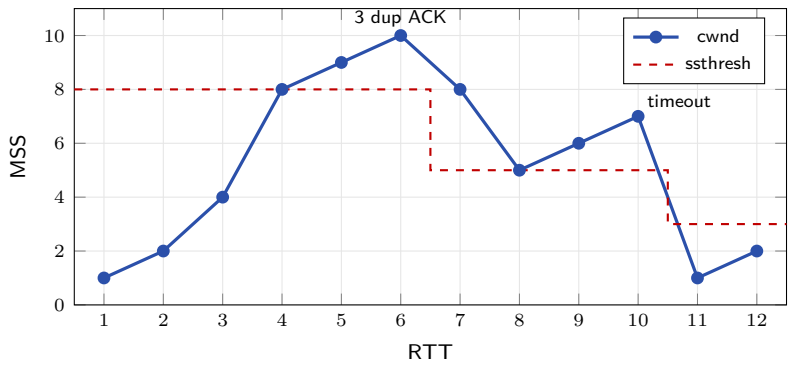
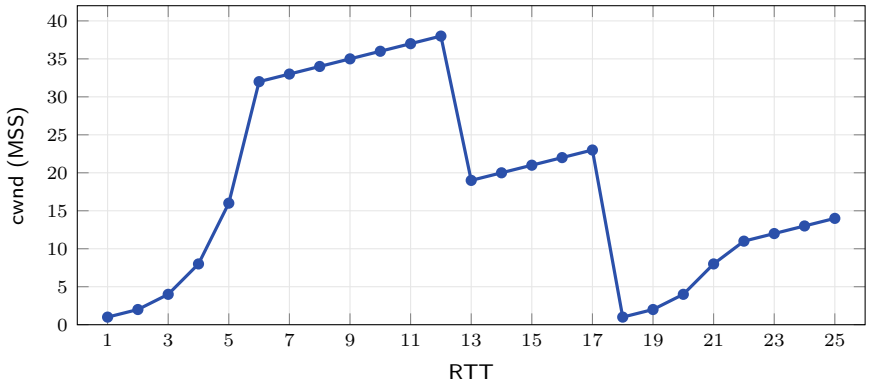


Figura 33.2 – La finestra dell'esercizio: dimezzamento dopo i duplicati, ritorno a 1 dopo il timeout.

Esercizio 11 — Leggere un grafico di cwnd

Il grafico mostra la finestra di congestione di una connessione TCP Reno (in MSS) per 25 RTT.



(a) In quali intervalli è attivo lo slow start? E la congestion avoidance?

(b) Dopo l'RTT 12 e dopo l'RTT 17 viene rilevata una perdita: in che modo, in ciascun caso?

(c) Qual è il valore iniziale di ssthresh? Quanto vale ssthresh all'RTT 13 e all'RTT 18?

(d) In quale RTT viene inviato il 70° segmento?

(e) Che cosa sarebbe successo dopo l'RTT 12 con TCP Tahoe?

Svolgimento

(a) Slow start: RTT 1–6 e 18–22 (la finestra raddoppia). Congestion avoidance: RTT 6–12, 13–17 e 22–25 (+1 MSS per RTT).

(b) Dopo l'RTT 12 la finestra passa da 38 a 19, cioè si **dimezza**: **3 ACK duplicati**. Dopo l'RTT 17 torna a 1: **timeout**.

(c) All'inizio lo slow start si ferma a 32, quindi ssthresh = 32. Dopo i duplicati ssthresh = $38/2 = 19$; dopo il timeout ssthresh = $\lfloor 23/2 \rfloor = 11$ (infatti all'RTT 22 la crescita esponenziale, che darebbe 16, si ferma a 11).

(d) I segmenti inviati si accumulano: dopo gli RTT 1–6 sono $1 + 2 + 4 + 8 + 16 + 32 = 63$; nell'RTT 7 si inviano i segmenti dal 64 al 96. Il 70° parte quindi nell'**RTT 7**.

(e) Tahoe non distingue i due tipi di perdita: anche dopo i 3 ACK duplicati avrebbe riportato cwnd a 1, con ssthresh = 19, ricominciando con lo slow start.

Esercizio 12 — Throughput medio

Una connessione TCP a lungo termine ha MSS di 1460 byte e RTT di 100 ms, e subisce una perdita (3 ACK duplicati) ogni volta che la finestra raggiunge $W = 40$ MSS.

(a) Qual è il throughput medio?

(b) Quanto tempo passa tra una perdita e la successiva?

Svolgimento

(a) Nel regime a dente di sega la finestra oscilla tra $W/2$ e W, con media $0{,}75\,W = 30$ MSS per RTT:

$$\text{throughput} \approx \frac{0{,}75 \cdot W \cdot \text{MSS}}{\text{RTT}} = \frac{30 \cdot 1460 \cdot 8}{0{,}1} \approx 3{,}5 \text{ Mb/s}$$

(b) Dopo ogni perdita la finestra riparte da $W/2 = 20$ e cresce di 1 MSS per RTT fino a 40: servono 20 RTT, cioè **2 secondi**.

TCP con Wireshark

Esercizio 13 — Osservare il controllo di congestione

Avviare la cattura, scaricare un file di prova (`wget http://speedtest.tele2.net/10MB.zip`), fermare la cattura e isolare la connessione con *Follow → TCP Stream*. Individuare apertura e chiusura della connessione, la crescita della finestra, eventuali perdite e aggiornamenti della finestra di ricezione.

Svolgimento

Filtro	Che cosa mostra
`tcp`	tutto il traffico TCP: il three-way handshake (SYN, SYN-ACK, ACK) all'inizio e lo scambio di FIN alla fine
`tcp.analysis.bytes_in_flight`	i byte inviati e non ancora confermati, cioè LastByteSent − LastByteAcked: la finestra effettiva in azione
`tcp.analysis.duplicate_ack`	gli ACK duplicati: dove ce ne sono tre di fila, subito dopo si trova una ritrasmissione veloce
`tcp.analysis.retransmission`	le ritrasmissioni (per timeout o fast retransmit)
`tcp.analysis.window_update`	gli aggiornamenti della finestra di ricezione: il controllo di flusso

Tabella 33.4 – Filtri di Wireshark per analizzare TCP.

Che cosa ci si aspetta di vedere:

- all'inizio i byte in volo crescono rapidissimamente (raddoppiano a ogni RTT): **slow start**;
- poi la crescita diventa lineare: **congestion avoidance**;
- in corrispondenza di ACK duplicati o timeout, cadute nette: il **dente di sega**;
- *Statistics → TCP Stream Graphs → Time Sequence (Stevens)* disegna i numeri di sequenza nel tempo: la **pendenza** è il throughput.

Flusso o congestione?

Se una caduta è preceduta da perdite (ACK duplicati, ritrasmissioni), è **congestione**. Se invece la finestra si appiattisce senza perdite e compaiono `window_update` con valori bassi, è **controllo di flusso**: il ricevente non legge abbastanza in fretta.

Formule da ricordare

- Checksum: somma a 16 bit con wrap around, poi complemento a 1; il ricevente deve ottenere tutti 1.
- Utilizzo: $U = \dfrac{N \cdot L/R}{\text{RTT} + L/R}$; finestra per $U = 1$: $N \geq (\text{RTT} + L/R)/(L/R)$.
- Selective Repeat: finestra $\leq 2^k/2$.
- ack = seq ricevuto + byte di dati; SYN e FIN consumano un numero di sequenza.
- Timeout = EstimatedRTT + 4 DevRTT; rwnd = RcvBuffer − (LastByteRcvd − LastByteRead).
- Reno: 3 dup ACK $\Rightarrow$ ssthresh = cwnd/2, cwnd = ssthresh +3; timeout $\Rightarrow$ ssthresh = cwnd/2, cwnd = 1.
- Throughput medio $\approx 0{,}75\,W/\text{RTT}$.

Esercizi sul livello di rete

Subnet mask e appartenenza alle subnet

Esercizio 1 — Maschere valide e non valide

Quali delle seguenti sono subnet mask valide? Per quelle valide indicare la lunghezza del prefisso.

`255.255.255.192` `255.255.254.0` `255.255.0.255` `255.128.255.0` `255.255.255.252` `255.254.0.0`

Svolgimento

Una maschera identifica un **prefisso**: tutti gli 1 devono essere **contigui e a sinistra**, seguiti solo da 0.

Maschera	Binario	Verdetto
255.255.255.192	11111111 11111111 11111111 11000000	valida, /26
255.255.254.0	11111111 11111111 11111110 00000000	valida, /23
255.255.0.255	11111111 11111111 00000000 11111111	**non valida**: 1 dopo gli 0
255.128.255.0	11111111 10000000 11111111 00000000	**non valida**: 0 in mezzo agli 1
255.255.255.252	11111111 11111111 11111111 11111100	valida, /30
255.254.0.0	11111111 11111110 00000000 00000000	valida, /15

Tabella 34.1 – Verifica delle maschere.

Il trucco veloce

In una maschera valida il byte "di transizione" può valere solo $256 - 2^k$, cioè `0, 128, 192, 224, 240, 248, 252, 254, 255`; tutti i byte prima devono essere 255 e tutti quelli dopo 0.

Esercizio 2 — Stessa subnet o no?

Per ciascuna coppia dire se i due indirizzi appartengono alla stessa subnet.

	Indirizzo 1	Indirizzo 2	Maschera
(a)	192.168.10.130	192.168.10.100	255.255.255.128
(b)	172.20.33.5	172.20.40.9	255.255.240.0
(c)	10.1.1.1	10.2.1.1	255.255.0.0 e poi 255.0.0.0
(d)	200.100.50.77	200.100.50.94	255.255.255.224
(e)	200.100.50.95	200.100.50.97	255.255.255.224

Svolgimento

Si fa l'AND bit a bit di ciascun indirizzo con la maschera e si confrontano i risultati; basta guardare il byte in cui la maschera "finisce".

(a) /25: conta il primo bit dell'ultimo byte. 130 = `10000010`, 100 = `01100100`: **subnet diverse** (192.168.10.128/25 e 192.168.10.0/25).

(b) /20: contano i primi 4 bit del terzo byte. 33 = `00100001`, 40 = `00101000`: **stessa subnet** (172.20.32.0/20).

(c) Con /16 i secondi byte sono diversi (1 e 2): **subnet diverse**. Con /8 conta solo il primo byte (10): **stessa subnet**. La stessa coppia di indirizzi può stare o no nella stessa subnet: dipende dalla maschera.

(d) /27: contano i primi 3 bit dell'ultimo byte. 77 = `01001101`, 94 = `01011110`: **stessa subnet** (200.100.50.64/27).

(e) 95 = `01011111`, 97 = `01100001`: **subnet diverse**. Sono numericamente vicinissimi, ma 95 è il broadcast di .64/27 e 97 è un host di .96/27.

Esercizio 3 — Rete, broadcast e host

Per ciascun indirizzo calcolare indirizzo di rete, indirizzo di broadcast, numero di host utilizzabili e intervallo degli host: (a) `172.16.45.14/20`; (b) `200.100.50.77/27`; (c) `10.0.0.1/30`.

Svolgimento

Rete: bit di host tutti a 0; broadcast: bit di host tutti a 1; host utilizzabili: $2^{32-n} - 2$.

(a) /20: restano 12 bit di host, 4 dei quali nel terzo byte. 45 = `0010 1101`:

- rete: `0010 0000` = 32 ⇒ **172.16.32.0**;
- broadcast: `0010 1111.11111111` ⇒ **172.16.47.255**;
- host: $2^{12} - 2 = 4094$, da 172.16.32.1 a 172.16.47.254.

(b) /27: 5 bit di host nell'ultimo byte. 77 = `010 01101`: rete `010 00000` = 64, broadcast `010 11111` = 95. Rete **200.100.50.64**, broadcast **200.100.50.95**, $2^5 - 2 = 30$ host (da .65 a .94).

(c) /30: 2 bit di host. Rete **10.0.0.0**, broadcast **10.0.0.3**, $2^2 - 2 = 2$ host (.1 e .2). È la dimensione tipica di un collegamento **punto-punto tra due router**: servono esattamente due indirizzi.

Esercizio 4 — Una tavola di routing CIDR (prova d'esame)

Un router che implementa CIDR ha la tavola seguente.

Subnet	Next hop
147.142.168.0/21	L1
147.142.172.0/23	L2
default	R0

Arrivano due pacchetti con destinazione 147.142.203.165 e 147.142.173.85. Descrivere come sono gestiti e inoltrati. (In binario: 147 = `10010011`, 142 = `10001110`, 168 = `10101000`, 172 = `10101100`, 173 = `10101101`, 203 = `11001011`, 165 = `10100101`, 85 = `01010101`.)

Svolgimento

I primi due byte (147.142) coincidono ovunque: tutto si decide sul terzo byte.

```
L1  147.142.168.0/21   10010011 10001110 10101|000 00000000    21 bit di prefisso
L2  147.142.172.0/23   10010011 10001110 1010110|0 00000000    23 bit di prefisso

P1  147.142.203.165    10010011 10001110 11001011 10100101
                                          ^ gia' il secondo bit del terzo byte differisce
P2  147.142.173.85     10010011 10001110 10101101 01010101
                                         10101    = L1  (21 bit uguali)
                                         1010110  = L2  (23 bit uguali)
```

- **147.142.203.165**: il terzo byte `11001011` differisce da quello di L1 (`10101000`) già al secondo bit, quindi non c'è corrispondenza con L1 e, a maggior ragione, con L2 (prefisso più lungo). Il pacchetto va al next hop di **default**, **R0**.
- **147.142.173.85**: i primi 21 bit coincidono con L1 *e* i primi 23 con L2. Ci sono due corrispondenze: per la **longest prefix matching** vince quella più specifica, e il pacchetto va su **L2**.

Esercizio 5 — Perché i router lavorano con le subnet

Spiegare perché i grandi router, per esempio quelli degli ISP, usano CIDR e trattano informazioni su subnet, come `189.103.176.0/20`, invece che su singoli indirizzi IP.

Svolgimento

1. **Dimensione delle tabelle.** Gli indirizzi IPv4 sono $2^{32} \approx 4$ miliardi: una riga per indirizzo sarebbe impossibile da memorizzare. Aggregando, un ISP annuncia un solo prefisso al posto di centinaia di reti: le tabelle dei router di backbone contengono qualche centinaio di migliaia di prefissi.
2. **Velocità.** Il forwarding avviene in hardware in nanosecondi; cercare in una tabella piccola con strutture specializzate (trie, TCAM) è enormemente più rapido.
3. **Flessibilità senza ambiguità.** Con la longest prefix matching i blocchi possono sovrapporsi: un sottoblocco (per esempio di un'organizzazione che cambia ISP) si instrada diversamente aggiungendo una sola riga più specifica, senza rinumerare nulla.

Esercizio 6 — Lookup con prefissi annidati

Un router ha la tabella di inoltro seguente.

Prefisso	Interfaccia
10.20.0.0/16	0
10.20.128.0/17	1
10.20.192.0/18	2
10.20.200.0/21	3
default	4

(a) Quali intervalli di indirizzi copre ciascuna riga?

(b) Su quale interfaccia vengono inoltrati 10.20.5.1, 10.20.130.7, 10.20.201.33, 10.20.210.1 e 10.21.0.1?

Svolgimento

(a) Tutto si gioca sul terzo byte (i primi due devono essere 10.20):

Prefisso	Bit fissati del terzo byte	Intervallo	Interfaccia
/16	nessuno	10.20.0.0 – 10.20.255.255	0
/17	`1xxxxxxx`	10.20.128.0 – 10.20.255.255	1
/18	`11xxxxxx`	10.20.192.0 – 10.20.255.255	2
/21	`11001xxx`	10.20.200.0 – 10.20.207.255	3

I blocchi sono **annidati**: ognuno è contenuto nel precedente, e la regola del prefisso più lungo "scava" i sottoblocchi.

(b)

- **10.20.5.1**: 5 = `00000101`, corrisponde solo al /16 ⇒ **0**.
- **10.20.130.7**: 130 = `10000010`: /16 e /17 sì, /18 no (serve `11`) ⇒ **1**.
- **10.20.201.33**: 201 = `11001001`: /16, /17, /18 e anche /21 (`11001`) ⇒ il più lungo, **3**.
- **10.20.210.1**: 210 = `11010010`: /16, /17, /18 sì; /21 no (`11010` ≠ `11001`) ⇒ **2**.
- **10.21.0.1**: il secondo byte non è 20, nessun prefisso corrisponde ⇒ default, **4**.

Esercizio 7 — Aggregare le rotte

Un ISP serve quattro clienti con i blocchi 192.168.64.0/24, 192.168.65.0/24, 192.168.66.0/24 e 192.168.67.0/24.

(a) Qual è il prefisso aggregato che l'ISP può annunciare?

(b) Il cliente 192.168.66.0/24 passa a un altro ISP portandosi il blocco. Che cosa annunciano i due ISP, e come fa un router esterno a instradare correttamente?

Svolgimento

(a) I terzi byte sono 64 = `010000 00`, 65 = `010000 01`, 66 = `010000 10`, 67 = `010000 11`: i primi 6 bit sono comuni. Il prefisso comune è lungo 16 + 6 = 22 bit: **192.168.64.0/22**, che contiene esattamente i quattro blocchi.

(b) Il primo ISP continua ad annunciare 192.168.64.0/22; il nuovo ISP annuncia **192.168.66.0/24**. Un indirizzo del cliente trasferito corrisponde a entrambi i prefissi, ma per la longest prefix matching vince il /24: i suoi pacchetti vanno al nuovo ISP, tutti gli altri del /22 al vecchio.

Progettare un piano di indirizzamento

Esercizio 8 — Suddividere un blocco CIDR (VLSM)

A un'organizzazione è assegnato il blocco 192.168.0.0/22. Quattro reparti hanno bisogno di 250, 120, 60 e 25 host.

(a) Quanti indirizzi contiene il blocco?

(b) Assegnare a ogni reparto una subnet di dimensione adeguata.

(c) Quanti indirizzi restano liberi?

Svolgimento

(a) $32 - 22 = 10$ bit di host: $2^{10} = 1024$ indirizzi, da 192.168.0.0 a 192.168.3.255.

(b) Si parte dal reparto **più grande** e si sceglie la più piccola potenza di 2 che, tolti 2 indirizzi (rete e broadcast), basta:

Reparto	Host	Blocco	Subnet	Host utilizzabili
A	250	$2^8 = 256$	192.168.0.0/24	.0.1 – .0.254
B	120	$2^7 = 128$	192.168.1.0/25	.1.1 – .1.126
C	60	$2^6 = 64$	192.168.1.128/26	.1.129 – .1.190
D	25	$2^5 = 32$	192.168.1.192/27	.1.193 – .1.222

Tabella 34.2 – Piano di indirizzamento a lunghezza variabile.

Per A un /25 darebbe solo $128 - 2 = 126$ host: non basta, serve il /24 (254 host).

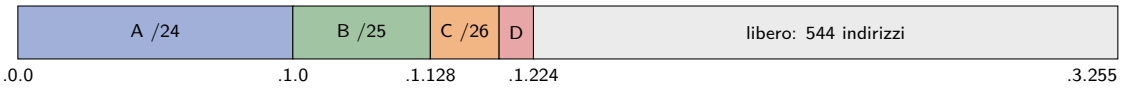


Figura 34.1 – Occupazione del blocco /22.

(c) Usati $256 + 128 + 64 + 32 = 480$; liberi $1024 - 480 = 544$, da 192.168.1.224 a 192.168.3.255.

Perché si parte dal più grande

Una subnet di 2^k indirizzi deve iniziare a un multiplo di 2^k. Assegnando per prime le subnet grandi si evita di "frammentare" lo spazio con blocchi piccoli sparsi, che impedirebbero poi di trovare un intervallo allineato abbastanza grande.

DHCP e NAT

Esercizio 9 — I quattro messaggi del DHCP

Un portatile si collega a una LAN in cui il server DHCP ha indirizzo 192.168.1.1 e offre l'indirizzo 192.168.1.57. Per ciascuno dei quattro messaggi indicare indirizzi IP e porte UDP di sorgente e destinazione, e spiegare perché anche la *request* viaggia in broadcast.

Svolgimento

Messaggio	Sorgente	Destinazione	Contenuto
Discover	0.0.0.0 : 68	255.255.255.255 : 67	"c'è un server DHCP?", con un transaction ID
Offer	192.168.1.1 : 67	255.255.255.255 : 68	`yiaddr` = 192.168.1.57, maschera, durata del lease
Request	0.0.0.0 : 68	255.255.255.255 : 67	"accetto l'offerta di 192.168.1.1"
ACK	192.168.1.1 : 67	255.255.255.255 : 68	conferma: indirizzo, gateway, DNS, lease

Tabella 34.3 – Lo scambio DHCP.

- Il client usa 0.0.0.0 come sorgente perché **non ha ancora un indirizzo**, e la destinazione broadcast perché non sa chi sia il server.
- La *request* è in broadcast perché possono esserci **più server** che hanno fatto un'offerta: così tutti sanno quale è stata accettata, e gli altri possono liberare l'indirizzo che avevano proposto.
- Il client usa ancora 0.0.0.0 fino all'ACK: l'indirizzo è suo solo quando il server lo conferma.

Esercizio 10 — Due host dietro lo stesso NAT

Un router NAT ha indirizzo WAN 138.76.29.7 e serve la LAN 10.0.0.0/24. Gli host 10.0.0.1 e 10.0.0.2 aprono entrambi, nello stesso momento, una connessione verso 128.119.40.186 porta 80, e **il caso vuole** che scelgano entrambi la porta sorgente 3345.

(a) Compilare la tabella di traduzione (il NAT assegna le porte a partire da 5001).

(b) Scrivere sorgente e destinazione dei quattro pacchetti (andata e ritorno, lato LAN e lato WAN) per l'host 10.0.0.2.

(c) Che cosa succede se il NAT non cambiasse la porta?

(d) Che cosa succede a un pacchetto che arriva dall'esterno verso 138.76.29.7:6000?

Svolgimento

(a)

Lato WAN	Lato LAN
138.76.29.7 : 5001	10.0.0.1 : 3345
138.76.29.7 : 5002	10.0.0.2 : 3345

(b)

```
(1) host -> NAT (LAN)       S = 10.0.0.2:3345      D = 128.119.40.186:80
(2) NAT -> Internet (WAN)   S = 138.76.29.7:5002   D = 128.119.40.186:80     sorgente riscritta
(3) Internet -> NAT (WAN)   S = 128.119.40.186:80  D = 138.76.29.7:5002
(4) NAT -> host (LAN)       S = 128.119.40.186:80  D = 10.0.0.2:3345         destinazione riscritta
```

(c) Le due connessioni avrebbero lato WAN la stessa coppia 138.76.29.7:3345: le risposte del server sarebbero indistinguibili e il NAT non saprebbe a quale host consegnarle. Gli host non conoscono le porte scelte dagli altri, quindi è il NAT a garantire l'**unicità** lato WAN. Con 16 bit di porta può gestire oltre 60 000 connessioni contemporanee.

(d) Nella tabella non c'è nessuna riga con porta WAN 6000: il NAT non sa a quale host interno consegnarlo e lo **scarta**. Le connessioni iniziate dall'esterno richiedono meccanismi di *NAT traversal*: port forwarding configurato a mano, UPnP, STUN/TURN, hole punching.

Frammentazione IP

Esercizio 11 — Frammentare un datagramma

Un datagramma IP di 5000 byte totali (20 di header, senza opzioni, e 4980 di dati), con identificativo 777, deve attraversare un collegamento con MTU di 1500 byte.

(a) In quanti frammenti viene diviso? Per ciascuno indicare lunghezza totale, flag MF, offset e identificativo.

(b) Ripetere con MTU di 576 byte.

Svolgimento

(a) Ogni frammento trasporta al più $1500 - 20 = 1480$ byte di dati. Poiché l'offset si esprime in **unità di 8 byte**, i dati di ogni frammento tranne l'ultimo devono essere multipli di 8: $1480/8 = 185$, va bene. $\lceil 4980/1480 \rceil = 4$ frammenti.

#	Dati (byte)	Lunghezza totale	MF	Offset	ID
1	1480	1500	1	0	777
2	1480	1500	1	1480/8 = 185	777
3	1480	1500	1	2960/8 = 370	777
4	540	560	0	4440/8 = 555	777

Tabella 34.4 – Frammenti con MTU di 1500 byte ($1480 \cdot 3 + 540 = 4980$).

(b) Con MTU 576 ci stanno 556 byte di dati, ma 556 non è multiplo di 8: si arrotonda per difetto a 552 ($= 69 \cdot 8$). $\lceil 4980/552 \rceil = 10$ frammenti: nove da 552 byte di dati (lunghezza totale 572, offset $0, 69, 138, \ldots, 552$) e un ultimo da $4980 - 9 \cdot 552 = 12$ byte di dati (lunghezza totale 32, offset $4968/8 = 621$, MF = 0).

Le tre regole della frammentazione

1. L'**identificativo** è copiato identico in tutti i frammenti.
2. L'**offset** è in unità di 8 byte: i dati di ogni frammento (tranne l'ultimo) devono essere multipli di 8.
3. Il flag **More Fragments** è 1 in tutti i frammenti tranne l'ultimo.

Il riassemblaggio avviene solo nell'host di destinazione; se un frammento si perde, si perde l'intero datagramma.

TRACEROUTE, ICMP E IP CON WIRESHARK

Esercizio 12 — Elementi di IPv4

Avviare la cattura e lanciare `traceroute` verso un server (per esempio `traceroute italia.it` oppure `traceroute 8.8.8.8`). Selezionare il primo segmento UDP inviato ed espandere la sezione *Internet Protocol*.

1. Qual è l'indirizzo IP del proprio host?
2. Quanto vale il TTL?
3. Qual è il protocollo di livello superiore?
4. Quanti byte ci sono nell'header IP?
5. Quanti byte ci sono nel payload del datagramma? Come si ricava?
6. Il datagramma è stato frammentato? Come si stabilisce?

Poi, con il filtro `ip.src==<mio_IP> && ip.dst==<IP_server> && udp`:

7. Quali campi restano costanti tra i pacchetti inviati? Perché?
8. Che andamento ha il campo *Identification*?

Infine, con il filtro `ip.dst==<mio_IP> && icmp`:

9. Qual è il protocollo superiore nei datagrammi inviati dai router?
10. I valori di *Identification* dei messaggi ICMP somigliano a quelli dei pacchetti UDP?
11. I TTL dei messaggi ICMP sono simili tra loro?

Svolgimento

1. Nel campo *Source* (lo stesso mostrato da `ip addr`; da non confondere con 127.0.0.1, il loopback).
2. **1**: traceroute parte da TTL 1 per far scadere il pacchetto al primo router.
3. **UDP** (valore 17): traceroute usa datagrammi UDP come sonde.
4. **20 byte**: il campo *Header Length* vale 5, in parole da 32 bit, cioè $5 \cdot 4 = 20$ byte (nessuna opzione).
5. payload = Total Length − Header Length. La dimensione di default dipende dalla versione di traceroute (la prima riga dell'output la dichiara): con una sonda di 40 byte, come indicato nel testo dell'esercitazione, il payload è $40 - 20 = 20$ byte (8 di header UDP e 12 di dati); con le versioni Linux recenti, che usano sonde di 60 byte, è $60 - 20 = 40$ byte.
6. No: il flag *More Fragments* è 0 **e** il *Fragment Offset* è 0, cioè il datagramma è unico. (Spesso è anche impostato *Don't Fragment*.)
7. Restano costanti versione, lunghezza dell'header, indirizzi sorgente e destinazione, protocollo, lunghezza totale, flag e offset: la sonda è sempre la stessa. Cambiano il **TTL** (cresce di 1 a ogni gruppo di sonde), l'**Identification**, la **porta di destinazione UDP** (cresce a ogni sonda) e di conseguenza il **checksum**.
8. Cresce di 1 a ogni datagramma: è il contatore che il sistema operativo usa per distinguere i datagrammi (e riassemblare i frammenti).
9. **ICMP** (valore 1): i router rispondono con *Time Exceeded* (type 11); la destinazione, alla fine, con *Destination Unreachable / Port Unreachable* (type 3, code 3).
10. No: ogni router genera i propri datagrammi con il **proprio** contatore, indipendente dal nostro.
11. No: ogni router invia con il proprio TTL iniziale (tipicamente 64, 128 o 255), e i messaggi arrivano dopo un numero diverso di salti. TTL osservato = TTL iniziale − hop di ritorno: dal valore si possono stimare il sistema operativo del router e la sua distanza.

Esercizio 13 — Frammentazione con traceroute

Ripetere con una sonda da 3000 byte (`traceroute 8.8.8.8 3000`) e il filtro `ip.src==<mio_IP> && ip.dst==<IP_server> && ip`.

(a) Il primo datagramma è stato frammentato?

(b) Quali campi dell'header indicano la frammentazione?

(c) Come si distingue il primo frammento dai successivi?

(d) Quanti byte contiene ciascun datagramma?

Svolgimento

(a) Sì. Con MTU di 1500 byte, un datagramma da 3000 byte (2980 di dati) diventa **tre frammenti**: 1480, 1480 e 20 byte di dati.

(b) Il flag **More Fragments** (1 = "ci sono altri frammenti dopo di me") e il **Fragment Offset** (diverso da 0 = "non sono il primo"). Tutti i frammenti condividono inoltre lo stesso **Identification** (per esempio 28264).

(c)

Frammento	Offset	MF
primo	0	1
intermedio	> 0	1
ultimo	> 0	0

Solo il primo frammento contiene l'header UDP (porte, lunghezza, checksum): gli altri contengono solo dati.

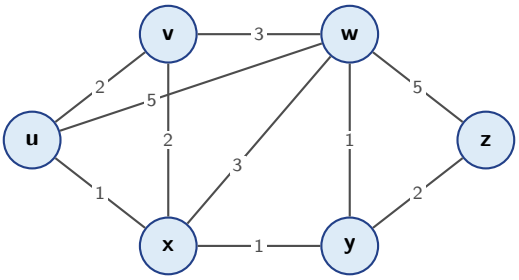
(d) Campo *Total Length*: 1500 per i primi due (20 + 1480), 40 per l'ultimo (20 + 20), che ha offset 2960 byte (370 nel campo).

Nella cattura tutti i frammenti hanno TTL 1 e lo stesso identificativo: ricomposti a destinazione, formerebbero il payload "artificiale" preparato da traceroute. Pur essendo un'applicazione, traceroute non usa il trasporto nel modo consueto: confeziona sonde e legge le risposte ICMP del livello di rete.

ROUTING

Esercizio 14 — Dijkstra a partire da z

Sul grafo in figura (lo stesso usato nel capitolo sul routing) eseguire l'algoritmo Link State a partire dal nodo **z**, riportare la tabella delle iterazioni, disegnare l'albero dei cammini minimi e scrivere la tabella di inoltro di z.



Svolgimento

Per ogni nodo si tiene $D(\cdot)$ (costo corrente) e $p(\cdot)$ (predecessore); a ogni passo entra in N' il nodo con D minimo.

Passo	N'	$D(y), p(y)$	$D(w), p(w)$	$D(x), p(x)$	$D(v), p(v)$	$D(u), p(u)$
0	z	**2, z**	5, z	∞	∞	∞
1	z, y		**3, y**	3, y	∞	∞
2	z, y, w			**3, y**	6, w	8, w
3	z, y, w, x				5, x	**4, x**
4	z, y, w, x, u				**5, x**	
5	z, y, w, x, u, v					

Tabella 34.5 – Link State da z (in grassetto il nodo che entra in N' al passo successivo).

- **Passo 0**: i vicini di z sono y (2) e w (5).
- **Passo 1**: entra y (2). $D(w) = \min\{5,\ 2+1\} = 3$ via y; $D(x) = 2+1 = 3$ via y.
- **Passo 2**: w e x sono a pari merito (3); si sceglie w. $D(v) = 3+3 = 6$; $D(u) = 3+5 = 8$; $D(x) = \min\{3,\ 3+3\} = 3$ invariato.
- **Passo 3**: entra x (3). $D(u) = \min\{8,\ 3+1\} = 4$ via x; $D(v) = \min\{6,\ 3+2\} = 5$ via x.
- **Passo 4**: entra u (4). $D(v) = \min\{5,\ 4+2\} = 5$ invariato.
- **Passo 5**: entra v (5). Fine.

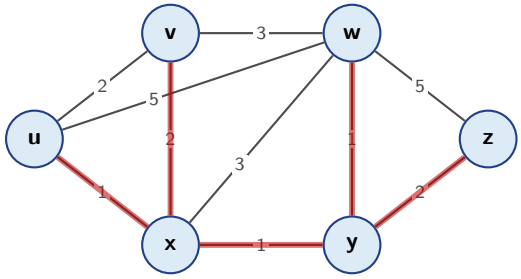


Tabella di z

destinazione	next hop
u	y (costo 4)
v	y (costo 5)
w	y (costo 3)
x	y (costo 3)
y	y (costo 2)

Figura 34.2 – L'albero dei cammini minimi da z e la tabella di inoltro risultante: tutto passa per y.

I cammini si ricostruiscono all'indietro con i predecessori: per esempio $p(v) = x$, $p(x) = y$, $p(y) = z$, quindi il cammino è $z \to y \to x \to v$ di costo $2 + 1 + 2 = 5$.

Esercizio 15 — Verifica con l'equazione di Bellman-Ford

Sullo stesso grafo si sa che $d_v(z) = 5$, $d_w(z) = 3$ e $d_x(z) = 3$. Usare l'equazione di Bellman-Ford per calcolare $d_u(z)$ e il next hop di u verso z, e verificare che il risultato sia coerente con i cammini minimi.

Svolgimento

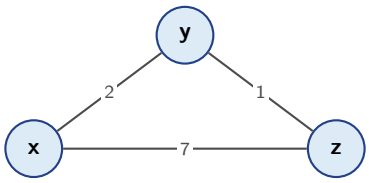
$$d_u(z) = \min_{v \in \text{vicini}(u)} \{c(u,v) + d_v(z)\} = \min\{\underbrace{2+5}_{\text{via v}}, \underbrace{5+3}_{\text{via w}}, \underbrace{1+3}_{\text{via x}}\} = \min\{7, 8, 4\} = 4$$

Il minimo si ottiene via **x**, che è il next hop di u verso z. È coerente con l'esercizio precedente: il cammino minimo tra u e z è $u \to x \to y \to z$, di costo 4 (in un grafo non orientato è lo stesso cammino, percorso al contrario).

Esercizio 16 — Distance Vector su tre nodi

Tre nodi con $c(x,y) = 2$, $c(y,z) = 1$, $c(x,z) = 7$ eseguono Distance Vector. Scrivere i vettori delle distanze all'avvio e dopo ogni scambio, fino alla convergenza.

Svolgimento



All'avvio ogni nodo conosce solo i costi verso i vicini: $D_x = (0, 2, 7)$, $D_y = (2, 0, 1)$, $D_z = (7, 1, 0)$ (nell'ordine x, y, z).

Dopo il primo scambio ogni nodo applica Bellman-Ford con i vettori ricevuti:

$$\begin{aligned} D_x(z) &= \min\{c(x,y) + D_y(z),\ c(x,z) + D_z(z)\} = \min\{2+1,\ 7+0\} = 3 \quad \text{(via y)} \\ D_z(x) &= \min\{c(z,y) + D_y(x),\ c(z,x) + D_x(x)\} = \min\{1+2,\ 7+0\} = 3 \quad \text{(via y)} \\ D_y &= (2, 0, 1) \quad \text{invariato} \end{aligned}$$

Quindi $D_x = (0, 2, 3)$, $D_y = (2, 0, 1)$, $D_z = (3, 1, 0)$. x e z, essendo cambiati, inviano i nuovi vettori; al **secondo scambio** nessun valore cambia e l'algoritmo **converge**: da quel momento nessuno invia più aggiornamenti.

Esercizio 17 — Poisoned reverse contro il count-to-infinity

Si riprenda lo scenario del capitolo sul routing: $c(x,y) = 4$, $c(y,z) = 1$, $c(x,z) = 50$, con z che raggiunge x passando per y. Il costo di x–y sale a 60 e solo y se ne accorge. Mostrare come evolvono $D_y(x)$ e $D_z(x)$ se si usa il **poisoned reverse**.

Svolgimento

Con il poisoned reverse, poiché z raggiunge x *passando per y*, z annuncia a y $D_z(x) = \infty$.

Passo	Calcolo	$D_y(x)$	$D_z(x)$
0	situazione iniziale; z dice a y "$D_z(x) = \infty$"	4 (diretto)	5 (via y)
1	y: $\min\{60,\ 1+\infty\} = 60$	**60** (diretto)	5
2	z riceve 60: $\min\{50,\ 1+60\} = 50$	60	**50** (diretto)
3	ora z non passa più per y: annuncia 50. y: $\min\{60,\ 1+50\} = 51$	**51** (via z)	50
4	y passa per z, quindi annuncia a z $D_y(x) = \infty$; z: $\min\{50,\ \infty\} = 50$	51	50

Tabella 34.6 – Con il poisoned reverse la convergenza richiede pochi scambi invece di oltre 40.

Il ciclo tra y e z non si forma perché y non può più "credere" al vecchio cammino di z, che passava per y stesso. Il limite del poisoned reverse è che funziona per cicli tra **due** nodi: con cicli che coinvolgono tre o più nodi il problema può ripresentarsi.

Formule da ricordare

- Rete = IP AND maschera; broadcast = bit di host a 1; host utilizzabili = $2^{32-n} - 2$.
- Longest prefix matching: tra le righe che corrispondono vince il prefisso più lungo; nessuna $\Rightarrow$ default.
- Frammentazione: dati per frammento = $\lfloor (\text{MTU} - 20)/8 \rfloor \cdot 8$; offset in unità da 8 byte; MF = 0 solo nell'ultimo.
- Dijkstra: $D(v) = \min\{D(v),\ D(w) + c(w,v)\}$; Bellman-Ford: $d_x(y) = \min_v\{c(x,v) + d_v(y)\}$.

Esercizi sul collegamento e sulla sicurezza

Controllo di parità

Esercizio 1 — Bit di parità

Calcolare il bit di parità, con schema pari e con schema dispari, per i messaggi (a) `10101100`, (b) `11010000`, (c) `01111110`. Verificare poi, sul messaggio (a) con parità pari, che un errore singolo venga rilevato e un errore doppio no.

Svolgimento

Si contano gli 1: con parità **pari** il bit rende pari il totale, con parità **dispari** lo rende dispari.

Messaggio	Numero di 1	Parità pari	Parità dispari
`10101100`	4	0	1
`11010000`	3	1	0
`01111110`	6	0	1

```
trasmesso        1010 1100 | 0     4 uni: pari, corretto
un bit invertito 1011 1100 | 0     5 uni: DISPARI -> errore rilevato
due bit invertiti 1011 1000 | 0    4 uni: pari   -> errore NON rilevato
```

La parità singola rileva solo un numero **dispari** di errori. Poiché nelle reti gli errori arrivano spesso a **raffiche** (burst), la probabilità che un errore sfugga può avvicinarsi al 50%.

Esercizio 2 — Parità bidimensionale

Si trasmette il blocco di quattro parole da 5 bit `10101`, `11110`, `01110`, `00101`, con parità pari per riga e per colonna.

(a) Calcolare i bit di parità.

(b) Il bit in riga 2, colonna 3 si inverte. Mostrare come il ricevente lo individua e lo corregge.

(c) Che cosa succede se si invertono due bit sulla stessa riga?

Svolgimento

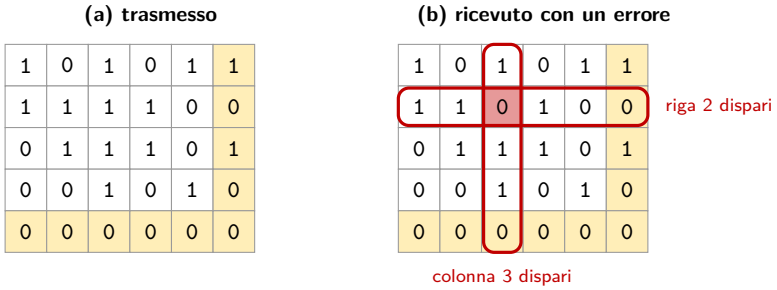


Figura 35.1 – In giallo i bit di parità; l'incrocio della riga e della colonna sbagliate individua il bit errato.

(a) Righe: `10101` ha 3 uni → 1; `11110` ha 4 uni → 0; `01110` ha 3 uni → 1; `00101` ha 2 uni → 0. Colonne: ogni colonna ha un numero pari di 1, quindi i bit di parità di colonna sono tutti 0 (e anche l'angolo, parità della colonna delle parità, è 0).

(b) Il ricevente ricalcola le parità: la riga 2 ha ora 3 uni (dispari) e la colonna 3 ha $1+0+1+1 = 3$ uni (dispari). L'**intersezione** identifica esattamente il bit sbagliato, che viene invertito: errore **corretto** senza ritrasmissione.

(c) Se si invertono i bit in colonna 3 e 4 della riga 2, la riga torna pari (l'errore sfugge al controllo di riga), ma le colonne 3 e 4 risultano dispari: l'errore è **rilevato** ma **non correggibile**, perché non si sa in quale riga siano i bit sbagliati. La parità bidimensionale rileva qualunque combinazione di due errori e ne corregge uno.

IL CRC

Esercizio 3 — Calcolo e verifica del CRC

Si consideri il messaggio D = `1101011111` e il generatore G = `10011` ($r = 4$).

(a) Calcolare il CRC e il messaggio trasmesso.

(b) Verificare lato ricevente che il messaggio sia integro.

(c) Verificare che l'inversione del sesto bit venga rilevata.

(d) Scrivere G e D come polinomi.

Svolgimento

(a) Si aggiungono $r = 4$ zeri a D e si divide modulo 2 per G: a ogni passo si allinea G sotto il primo 1 rimasto e si fa lo XOR (niente riporti né prestiti). Il resto, sugli ultimi r bit, è il CRC.

```
11010111110000   D seguito da 4 zeri
10011            XOR G
─────────────
01001111110000
 10011           XOR G
─────────────
00000011110000
      10011      XOR G
─────────────
00000001101000
       10011     XOR G
─────────────
00000000100100
        10011    XOR G
─────────────
00000000000010
          ^^^^   resto = 0010
```

CRC = 0010; si trasmette $M = D \,\|\, \text{CRC} =$ 1101011111 0010.

(b) Il ricevente divide per G l'intero messaggio ricevuto:

```
11010111110010   ricevuto: D + CRC
10011            XOR G
─────────────
01001111110010
 10011           XOR G
─────────────
00000011110010
      10011      XOR G
─────────────
00000001101010
       10011     XOR G
─────────────
00000000100110
        10011    XOR G
─────────────
00000000000000
          ^^^^   resto = 0000
```

Il resto è zero: nessun errore rilevato.

(c) Si riceve 1101001111 0010:

```
11010011110010   ricevuto con un bit invertito
10011            XOR G
─────────────
01001011110010
 10011           XOR G
─────────────
00000111110010
     10011       XOR G
─────────────
00000011000010
      10011      XOR G
─────────────
00000001011010
       10011     XOR G
─────────────
00000000010110
         10011   XOR G
─────────────
00000000000101
          ^^^^   resto = 0101
```

Il resto 0101 è diverso da zero: **errore rilevato**.

(d) Ogni bit è il coefficiente di una potenza di x: $G(x) = x^4 + x + 1$ e $D(x) = x^9 + x^8 + x^6 + x^4 + x^3 + x^2 + x + 1$. Aggiungere r zeri equivale a moltiplicare per x^r: si calcola $x^r D(x) = Q(x)G(x) + R(x)$ e si trasmette $M(x) = x^r D(x) + R(x)$, che è **divisibile per $G(x)$ per costruzione** (modulo 2 somma e sottrazione coincidono). Il ricevente controlla proprio la divisibilità.

Gli errori tipici nel CRC

- Dimenticare di aggiungere gli r zeri (sono tanti quanti il grado di G, cioè un bit in meno della sua lunghezza).
- Allineare G sotto un bit 0 invece che sotto il primo 1.
- Fare la sottrazione con i prestiti invece dello XOR.
- Prendere come CRC più (o meno) di r bit.

Esercizio 4 — Un CRC da provare

Calcolare il CRC di D = 10011010 con G = 1101.

Svolgimento

```
10011010000   D seguito da 3 zeri
1101          XOR G
────────────
01001010000
 1101         XOR G
────────────
00100010000
  1101        XOR G
────────────
00010110000
   1101       XOR G
────────────
00001100000
    1101      XOR G
────────────
00000001000
       1101   XOR G
────────────
00000000101
        ^^^   resto = 101
```

Si trasmette `10011010 101`.

Esercizio 5 — Che cosa rileva il CRC-32

Il CRC-32 di Ethernet ha un generatore di 33 bit ($r = 32$). Quali burst di errori rileva con certezza? Con quale probabilità rileva un burst più lungo?

Svolgimento

Un CRC con r bit di controllo rileva **tutti** i burst di lunghezza $\leq r$, quindi fino a **32 bit**. Per burst più lunghi, un errore sfugge solo se il resto risulta per caso zero, con probabilità circa 2^{-r}:

$$P(\text{rilevato}) \approx 1 - 0{,}5^{32} = 1 - \frac{1}{4\,294\,967\,296} \approx 0{,}99999999977$$

Un errore non rilevato ogni 4 miliardi circa. Con soli 4 byte di ridondanza, calcolabili in hardware con un registro a scorrimento e qualche porta XOR: ecco perché il CRC è nel trailer dei frame Ethernet.

La distanza di Hamming

Esercizio 6 — Calcolare distanze

Calcolare la distanza di Hamming tra (a) `10110101` e `10011100`; (b) `1111000011110000` e `1010101010101010`.

Svolgimento

Il metodo veloce è contare gli 1 dello XOR:

```
(a)      10110101          (b)      1111000011110000
     XOR 10011100               XOR 1010101010101010
         00101001  -> 3 uni             0101101001011010  -> 8 uni
```

Quindi $d = 3$ e $d = 8$.

Esercizio 7 — Un codice a correzione d'errore

Un codice ha le quattro parole valide `00000`, `01011`, `10101`, `11110`.

(a) Qual è la distanza di Hamming del codice? Quanti errori rileva e quanti ne corregge?

(b) Decodificare le parole ricevute `01111`, `11111` e `01101`.

Svolgimento

(a) Le distanze tra le coppie sono:

`00000`–`01011`	3	`00000`–`10101`	3	`00000`–`11110`	4
`01011`–`10101`	4	`01011`–`11110`	3	`10101`–`11110`	3

La distanza del codice è la minima: $d = 3$. Rileva fino a $d - 1 = 2$ errori e ne corregge fino a $\lfloor (d-1)/2 \rfloor = 1$.

(b) Si decodifica verso la parola valida **più vicina**:

Ricevuta	`00000`	`01011`	`10101`	`11110`	**Decisione**
`01111`	4	**1**	3	2	`01011` (un errore, corretto)
`11111`	5	2	2	**1**	`11110` (un errore, corretto)
`01101`	3	2	2	3	pari merito: errore **rilevato**, non correggibile

Nell'ultimo caso ci sono stati almeno 2 errori: il codice se ne accorge (nessuna parola valida a distanza 0) ma non sa quale correzione scegliere. Con 3 errori potrebbe addirittura "correggere" verso la parola sbagliata.

L'intuizione geometrica

Attorno a ogni parola valida c'è una "sfera" di raggio $\lfloor (d-1)/2 \rfloor$. Se le sfere non si toccano, una parola ricevuta che cade in una sfera si riporta al suo centro senza ambiguità. Per **rilevare** k errori serve $d \geq k + 1$; per **correggerne** k serve $d \geq 2k + 1$.

Esercizio 8 — Bit stuffing

Il flag dei frame è `01111110`. Applicare il bit stuffing ai dati (a) `0111111111111111110` e (b) `01111110`, e mostrare il de-stuffing lato ricevente.

Svolgimento

Regola: in trasmissione, dopo **cinque 1 consecutivi** si inserisce uno **0**; in ricezione, lo 0 che segue cinque 1 si elimina.

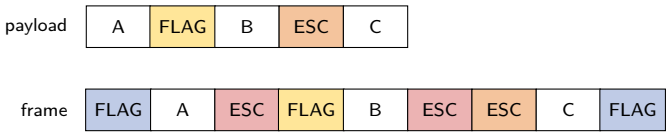
```
(a) dati       0 11111 11111 11111 11 0
    trasmessi  0 11111011111011111011 0      (22 bit: 3 zeri inseriti)

(b) dati       0 11111 1 0
    trasmessi  0 111110 1 0                  (il "flag" nei dati non e' piu' un flag)
```

Il ricevente, scorrendo i bit, elimina ogni 0 che segue cinque 1 e ritrova i dati originali. Grazie allo stuffing nei dati non compaiono mai sei 1 consecutivi, quindi `01111110` delimita i frame senza ambiguità. Nel caso peggiore (dati tutti a 1) l'overhead è di un bit ogni cinque, il 20%.

Esercizio 9 — Byte stuffing

Con FLAG come delimitatore ed ESC come carattere di escape, come si trasmette il payload `A FLAG B ESC C`? Perché anche ESC va preceduto da un ESC?

Svolgimento



Ogni FLAG ed ESC del payload viene preceduto da un ESC (in rosso), e il frame è racchiuso tra due FLAG (in blu). Il ricevente, quando incontra un ESC, lo scarta e prende il byte successivo *così com'è*, anche se è un FLAG. Se un ESC del payload non fosse a sua volta protetto, la sequenza `ESC C` verrebbe interpretata come "C letterale" e l'ESC originale andrebbe perso; e un `ESC FLAG` originale nel payload sarebbe indistinguibile da un FLAG protetto.

Esercizio 10 — Perché il conteggio dei byte non si usa

Un flusso contiene quattro frame, delimitati dal conteggio dei byte (il primo byte di ogni frame ne indica la lunghezza, se stesso compreso): `5 1 2 3 4 | 5 6 7 8 9 | 8 0 1 2 3 4 5 6 | 8 7 8 9 0 1 2 3`. Che cosa succede se il secondo conteggio, 5, si corrompe in 7?

Svolgimento

Il ricevente legge correttamente il primo frame (5 byte). Poi legge `7` come lunghezza e prende `7 6 7 8 9 8 0` come secondo frame; il "terzo frame" inizia quindi dal byte `1`, che viene scambiato per un conteggio, e così via. Il checksum del frame segnalerà l'errore, ma il ricevente **non ha modo di sapere dove inizia il frame successivo**: non riesce più a **risincronizzarsi**, e un solo errore rovina tutti i frame seguenti. Per questo il conteggio dei byte da solo non si usa.

Accesso multiplo

Esercizio 11 — La dimensione minima del frame

Un bus Ethernet è lungo al massimo 2500 m, il segnale si propaga a $2 \cdot 10^8$ m/s e il rate è 10 Mb/s.

(a) Calcolare il tempo di propagazione τ tra gli estremi.

(b) Calcolare la dimensione minima del frame perché CSMA/CD rilevi tutte le collisioni.

(c) Ripetere per un cavo di 1 km a 1 Gb/s. Che cosa se ne conclude?

Svolgimento

(a) $\tau = 2500/(2 \cdot 10^8) = 12{,}5\ \mu$s.

(b) Nel caso peggiore A inizia a trasmettere a $t = 0$, B inizia un istante prima che il segnale di A la raggiunga ($t \approx \tau$), e il "rumore" della collisione torna ad A a $t \approx 2\tau$. A deve essere **ancora in trasmissione** in quel momento:

$$\frac{L_{\min}}{R} \geq 2\tau \quad \Rightarrow \quad L_{\min} \geq 2\tau R = 2 \cdot 12{,}5 \cdot 10^{-6} \cdot 10^7 = 250 \text{ bit}$$

Ethernet fissa il minimo a **512 bit (64 byte)**, con margine per ripetitori e ritardi; se il datagramma è più corto di 46 byte, il campo **pad** allunga il frame fino al minimo.

(c) $\tau = 1000/(2 \cdot 10^8) = 5\ \mu$s; $L_{\min} = 2 \cdot 5 \cdot 10^{-6} \cdot 10^9 = 10^4$ bit = 1250 byte. A velocità alte i frame corti finiscono prima che la collisione possa essere rilevata: il CSMA/CD scala male con la velocità. Per questo le Ethernet veloci funzionano in pratica solo **full-duplex su switch**, dove le collisioni non esistono.

Esercizio 12 — Throughput di ALOHA

Ricavare il throughput massimo di ALOHA puro ($S = Ge^{-2G}$) e di slotted ALOHA ($S = Ge^{-G}$), dove G è il carico offerto per tempo di frame, e spiegare la differenza.

Svolgimento

Slotted ALOHA: $\dfrac{dS}{dG} = e^{-G} - Ge^{-G} = e^{-G}(1-G) = 0 \Rightarrow G = 1, \quad S_{\max} = e^{-1} \approx 0{,}368$ (36,8%).

ALOHA puro: $\dfrac{dS}{dG} = e^{-2G} - 2Ge^{-2G} = e^{-2G}(1-2G) = 0 \Rightarrow G = \tfrac{1}{2}, \quad S_{\max} = \tfrac{1}{2}e^{-1} = \dfrac{1}{2e} \approx 0{,}184$ (18,4%).

Nell'ALOHA puro un frame collide con qualunque frame iniziato nell'intervallo di un tempo di frame *prima* o *dopo* di lui: la finestra di vulnerabilità è di **2** tempi di frame. Negli slot le trasmissioni iniziano solo ai confini, e la finestra si riduce a **1**: dimezzare la vulnerabilità raddoppia il throughput massimo. Il CSMA fa molto meglio di entrambi, perché ascoltando il canale evita la maggior parte delle collisioni: resta solo la finestra dovuta al ritardo di propagazione.

Esercizio 13 — Binary exponential backoff

In Ethernet a 10 Mb/s l'unità di attesa è il tempo di 512 bit.

(a) Dopo la terza collisione di un frame, tra quali valori si sceglie K? Qual è l'attesa massima e quella media?

(b) Qual è l'attesa massima dopo la decima collisione?

(c) Due stazioni hanno appena subito la loro prima collisione. Con che probabilità collidono di nuovo? E dopo la seconda?

Svolgimento

L'unità di attesa è $512/10^7$ s = 51,2 μs.

(a) $K \in \{0, \dots, 2^3 - 1\} = \{0, \dots, 7\}$. Attesa massima $7 \cdot 51{,}2 = 358{,}4$ μs; media $3{,}5 \cdot 51{,}2 = 179{,}2$ μs.

(b) $K \le 2^{10} - 1 = 1023$: attesa massima $1023 \cdot 51{,}2$ μs $\approx 52{,}4$ ms (oltre la decima collisione l'intervallo non cresce più).

(c) Dopo la prima collisione ciascuna sceglie $K \in \{0, 1\}$: scelgono lo stesso valore con probabilità $1/2$. Dopo la seconda $K \in \{0, 1, 2, 3\}$: probabilità $1/4$. In generale $1/2^n$: con l'aumentare delle collisioni l'attesa si allarga e la probabilità di riuscire cresce.

Esercizio 14 — Il difetto di TDMA

Un canale da 1 Mb/s è diviso con TDMA tra 100 stazioni. Qual è il rate di una stazione quando è l'unica ad avere dati da inviare? Come si comporterebbero un protocollo ad accesso casuale e un protocollo a rotazione ideale?

Svolgimento

Con TDMA la stazione usa solo il proprio slot: 1 Mb/s /100 = 10 kb/s, anche se gli altri 99 slot restano vuoti. Viola il primo requisito di un buon protocollo MAC ("se trasmette un solo nodo, deve poter usare tutto R"). Un protocollo ad accesso casuale le lascerebbe usare tutto il canale (non ci sono collisioni con nessuno); uno a rotazione (polling, token) le darebbe quasi tutto il canale, salvo l'overhead del giro di polling o del token.

Switch e ARP

Esercizio 15 — Il self-learning passo per passo

Uno switch a 4 porte appena acceso (tabella vuota) ha collegati: sulla porta 1 un hub con gli host A ed E; sulle porte 2, 3, 4 gli host B, C, D. Transitano, nell'ordine, i frame: A→D, D→A, C→A, B→broadcast, E→A, A→C. Per ciascuno indicare come si aggiorna la tabella e che cosa fa lo switch.

Svolgimento

#	Frame	Tabella dopo il frame	Azione
1	A → D	A:1	D sconosciuto: **flooding** su 2, 3, 4
2	D → A	A:1, D:4	A è sulla porta 1: **inoltro** su 1
3	C → A	A:1, D:4, C:3	inoltro su 1
4	B → FF:FF:FF:FF:FF:FF	A:1, D:4, C:3, B:2	broadcast: flooding su 1, 3, 4
5	E → A	…, E:1	A è sulla stessa porta da cui arriva il frame: **filtraggio** (scartato)
6	A → C	invariata	inoltro su 3

Tabella 35.1 – L'evoluzione della tabella di commutazione.

Lo switch impara sempre dal **MAC sorgente** e decide in base al **MAC destinazione**: *flooding* se non lo conosce (o è broadcast), *inoltro* se è su un'altra porta, *filtraggio* se è sulla stessa porta di arrivo (nel frame 5 A ha già ricevuto il frame dall'hub). Le righe hanno un TTL e scadono se un host resta silenzioso.

Esercizio 16 — Un frame verso una destinazione remota

L'host A, con indirizzo 192.168.1.10/24 e MAC `AA:AA:AA:AA:AA:AA`, invia un datagramma all'host B (10.0.0.5). Il gateway di A è 192.168.1.1, con MAC `BB:BB:BB:BB:BB:BB`; l'interfaccia del router verso la rete di B ha MAC `CC:CC:CC:CC:CC:CC`, e B ha MAC `DD:DD:DD:DD:DD:DD`. Descrivere i passi e gli indirizzi dei due frame.

Svolgimento

1. **A capisce che B è remoto**: 192.168.1.10 ∧ 255.255.255.0 = 192.168.1.0, mentre 10.0.0.5 ∧ 255.255.255.0 = 10.0.0.0. Reti diverse: il frame va consegnato al **gateway**.
2. **A cerca il MAC del gateway** (non di B!) nella tabella ARP; se manca invia una *ARP request* in broadcast ("chi ha 192.168.1.1?") e il router risponde in unicast con `BB:BB:…`.
3. **A invia il frame**; il router lo accetta (il MAC destinazione è il suo), estrae il datagramma, decrementa il TTL, ricalcola il checksum, consulta la forwarding table e, con ARP sull'interfaccia d'uscita, trova il MAC di B.
4. **Il router crea un nuovo frame** verso B.

Tratta	MAC sorgente	MAC destinazione	IP sorgente	IP destinazione
A → router	AA:AA:AA:AA:AA:AA	**BB:BB:BB:BB:BB:BB**	192.168.1.10	10.0.0.5
router → B	CC:CC:CC:CC:CC:CC	**DD:DD:DD:DD:DD:DD**	192.168.1.10	10.0.0.5

La regola: i MAC cambiano a ogni salto (sono quelli del salto corrente), gli IP restano quelli degli estremi (salvo NAT).

Esercizio 17 — ARP poisoning

Spiegare come funziona l'ARP poisoning, perché è così efficace e come ci si difende.

Svolgimento

Un host malevolo M invia ad A risposte ARP **falsificate** (anche non richieste) che associano l'IP del gateway al *proprio* MAC, e fa lo stesso con il gateway per l'IP di A. Da quel momento il traffico tra A e il gateway passa per M, che può leggerlo, modificarlo e inoltrarlo: un **man-in-the-middle** invisibile. Funziona perché ARP **non ha alcuna autenticazione**: chiunque può rispondere, e le tabelle accettano anche risposte non sollecitate (*gratuitous ARP*). Le difese vengono da fuori del protocollo: voci ARP statiche per i nodi critici, *Dynamic ARP Inspection* sugli switch gestiti e, soprattutto, cifratura end-to-end (TLS), che rende inutile l'intercettazione.

CRITTOGRAFIA

Esercizio 18 — Il cifrario di Cesare

Con uno spostamento di $k = 3$ lettere (alfabeto inglese di 26 lettere): (a) cifrare `RETI`; (b) decifrare `SURWRFROOR`; (c) quante chiavi deve provare un attaccante? E con una sostituzione monoalfabetica qualsiasi?

Svolgimento

(a) R→U, E→H, T→W, I→L: `UHWL`.

(b) Si sposta indietro di 3: `PROTOCOLLO`.

(c) Le chiavi utili sono 25: una ricerca esaustiva è immediata. Con una sostituzione monoalfabetica qualsiasi le chiavi sono $26! \approx 4 \cdot 10^{26}$, troppe per la forza bruta; eppure il cifrario si rompe facilmente con l'**analisi delle frequenze** (in italiano E, A, I, O sono le lettere più frequenti), perché ogni lettera in chiaro diventa sempre la stessa lettera cifrata. Uno spazio delle chiavi grande è necessario, ma non sufficiente.

Esercizio 19 — One-time pad

Con $P =$ `10110010` e $K =$ `01101100`: (a) cifrare; (b) decifrare; (c) mostrare perché lo stesso pad non va mai riusato.

Svolgimento

(a) $C = P \oplus K =$ `10110010` $\oplus$ `01101100` $=$ `11011110`.

(b) $C \oplus K =$ `11011110` $\oplus$ `01101100` $=$ `10110010` $= P$: cifrare e decifrare sono la stessa operazione, perché $(P \oplus K) \oplus K = P$.

(c) Se lo stesso K cifra P_1 e P_2, l'attaccante calcola $C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$: il pad **scompare**, e lo XOR di due testi in linguaggio naturale si attacca con metodi statistici. Il one-time pad è inviolabile solo se il pad è casuale, lungo quanto il messaggio e usato **una sola volta**: proprio per questo è poco pratico (va distribuito in modo sicuro un pad grande quanto tutti i messaggi).

Esercizio 20 — Hash e MAC

A invia a B la coppia $(m, H(m))$, dove H è una funzione hash crittografica pubblica. Garantisce l'integrità? Come la garantisce il MAC?

Svolgimento

No. Un intruso intercetta il messaggio, lo sostituisce con m' e calcola lui stesso $H(m')$, perché H è pubblica: B riceve $(m', H(m'))$, la verifica ha successo e la falsificazione passa inosservata.

Con il **MAC** A e B condividono un segreto s: A invia $(m, H(m\|s))$, senza trasmettere s. B ricalcola $H(m\|s)$ e confronta. L'intruso, non conoscendo s, non può produrre il valore corretto per un m' diverso. (Il MAC crittografico, *Message Authentication Code*, non ha nulla a che vedere con il MAC address del livello di collegamento.)

Esercizio 21 — Diffie-Hellman con numeri piccoli

Alice e Bob scelgono pubblicamente $n = 23$ e $g = 5$; Alice sceglie in segreto $x = 6$, Bob $y = 15$.

(a) Calcolare i valori scambiati e il segreto condiviso.

(b) Perché un intercettatore passivo non può ricavarlo (con numeri grandi)?

(c) Come può un attaccante attivo compromettere lo scambio?

Svolgimento

(a) Alice invia $A = 5^6 \bmod 23 = 15625 \bmod 23 = 8$; Bob invia $B = 5^{15} \bmod 23 = 19$. Alice calcola $B^x \bmod n = 19^6 \bmod 23 = 2$; Bob calcola $A^y \bmod n = 8^{15} \bmod 23 = 2$. Il segreto condiviso è $g^{xy} \bmod n = \mathbf{2}$, senza che x o y abbiano mai viaggiato sulla rete.

Calcolare le potenze modulari a mano

Si riduce modulo n a ogni passo, per quadrati successivi. Per $5^{15} \bmod 23$: $5^2 = 25 \equiv 2$; $5^4 \equiv 4$; $5^8 \equiv 16$; $15 = 8+4+2+1$, quindi $5^{15} \equiv 16 \cdot 4 \cdot 2 \cdot 5 = 640 \equiv 640 - 27 \cdot 23 = 19$.

(b) L'intercettatore conosce $n = 23$, $g = 5$, $A = 8$ e $B = 19$. Per ottenere il segreto gli serve x o y, cioè risolvere $5^x \equiv 8 \pmod{23}$: il **logaritmo discreto**, intrattabile quando n ha migliaia di bit (con $n = 23$ basta provare).

(c) Con un **man-in-the-middle**: Trudy intercetta A e invia a Bob il proprio $g^z \bmod n$, e fa lo stesso verso Alice. Stabilisce così un segreto g^{xz} con Alice e uno g^{yz} con Bob, decifra e ricifra tutto, e nessuno dei due se ne accorge. Diffie-Hellman protegge dall'ascolto passivo ma **non autentica**: servono certificati e autorità di certificazione.

Esercizio 22 — RSA con numeri piccoli

Si scelgono $p = 5$ e $q = 7$, con esponente pubblico $e = 5$.

(a) Calcolare n, z e un esponente privato d valido diverso da e.

(b) Cifrare il messaggio $m = 12$ e verificare la decifratura.

Svolgimento

(a) $n = 35$, $z = (p-1)(q-1) = 24$. Serve $e \cdot d \equiv 1 \pmod{24}$: $d = 5$ funzionerebbe ($25 \equiv 1$), ma renderebbe la chiave privata uguale a quella pubblica. Il valore successivo è $d = 29$: $5 \cdot 29 = 145 = 6 \cdot 24 + 1$. Chiave pubblica $(5, 35)$, privata $(29, 35)$.

(b) Cifratura: $c = 12^5 \bmod 35$. $12^2 = 144 \equiv 4$; $12^4 \equiv 16$; $12^5 \equiv 16 \cdot 12 = 192 \equiv 17$. Quindi $c = 17$.

Decifratura: $17^{29} \bmod 35$. $17^2 = 289 \equiv 9$; $17^4 \equiv 81 \equiv 11$; $17^8 \equiv 121 \equiv 16$; $17^{16} \equiv 256 \equiv 11$. Con $29 = 16 + 8 + 4 + 1$: $11 \cdot 16 \cdot 11 \cdot 17 \bmod 35$; $11 \cdot 16 = 176 \equiv 1$, poi $1 \cdot 11 = 11$, poi $11 \cdot 17 = 187 \equiv 12$. Si ritrova $m = 12$.

Ovviamente con $n = 35$ chiunque fattorizza n e ricava d: la sicurezza di RSA viene dall'usare primi di centinaia di cifre.

Esercizio 23 — Il nonce contro il replay

Perché inviare la password cifrata non basta ad autenticarsi? Come risolve il problema un nonce?

Svolgimento

Un attaccante che registra il messaggio con la password cifrata può **rigiocarlo** in una nuova sessione (*playback attack*): non ha bisogno di capirlo, e B non ha modo di sapere se A è davvero "presente". Con il nonce, B sceglie un numero R usato **una sola volta** e lo invia ad A; A risponde con $K_{AB}(R)$. Una risposta registrata in passato riguarda un nonce vecchio $R_1 \neq R_2$, e senza la chiave l'attaccante non può produrre $K_{AB}(R_2)$. È lo stesso principio degli ISN casuali di TCP: un valore imprevedibile e diverso a ogni sessione garantisce che l'interlocutore sia vivo.

FIREWALL

Esercizio 24 — Scrivere una ACL

Un'organizzazione ha la rete 222.22.0.0/16 e un mail server 222.22.1.25. Scrivere le regole di un filtro di pacchetti (tradizionale) che: permettano agli utenti interni di navigare sul Web (HTTP e HTTPS); permettano le query DNS verso l'esterno; permettano la posta in ingresso verso il mail server; blocchino tutto il resto.

Svolgimento

Azione	Sorgente	Destinazione	Prot.	Porta sorg.	Porta dest.	ACK
allow	222.22/16	esterno	TCP	> 1023	80, 443	qualsiasi
allow	esterno	222.22/16	TCP	80, 443	> 1023	1
allow	222.22/16	esterno	UDP	> 1023	53	–
allow	esterno	222.22/16	UDP	53	> 1023	–
allow	esterno	222.22.1.25	TCP	> 1023	25	qualsiasi
allow	222.22.1.25	esterno	TCP	25	> 1023	1
deny	qualsiasi	qualsiasi	qualsiasi	qualsiasi	qualsiasi	qualsiasi

Tabella 35.2 – Una ACL per il filtro tradizionale (le regole si valutano dall'alto; l'ultima blocca tutto il resto).

Il flag **ACK** = 1 sulle regole in ingresso impedisce che dall'esterno si *aprano* connessioni verso le porte alte: il primo segmento di una connessione (SYN) ha ACK = 0. Resta però il limite del filtro **stateless**: un attaccante può inviare segmenti con ACK = 1 e porta sorgente 80 che non appartengono a nessuna connessione, e il filtro li lascerebbe passare. Un filtro **stateful**, che tiene la tabella delle connessioni aperte, li scarterebbe.

Formule da ricordare

- Codice con distanza d: rileva $d-1$ errori, ne corregge $\lfloor (d-1)/2 \rfloor$.
- CRC: si aggiungono r zeri, divisione modulo 2 (XOR), resto di r bit; rileva tutti i burst $\leq r$.
- CSMA/CD: $L_{\min} = 2\tau R$; backoff $K \in \{0, \ldots, 2^n - 1\}$ unità da 512 bit.
- ALOHA: $S_{\max} = 1/(2e) \approx 18\%$; slotted: $1/e \approx 37\%$.
- RSA: $n = pq$, $z = (p-1)(q-1)$, $ed \equiv 1 \pmod{z}$, $c = m^e \bmod n$, $m = c^d \bmod n$.
- Diffie-Hellman: segreto $g^{xy} \bmod n$.

Domande di ripasso

Questo capitolo raccoglie le domande di ripasso proposte durante il corso e una selezione di domande tipiche, organizzate per argomento, con risposte sintetiche. Vale l'avvertenza data in aula: **le risposte devono essere complete e possono essere concise**. Le domande della sessione di ripasso su HTTP (parti di un URL, handshake, pipelining), DNS (cambio di indirizzo, risoluzione con cache) e CIDR (tavola con L1, L2, R0) sono svolte per esteso nei capitoli di esercizi precedenti.

Tre tipi di domande

1. **Definitorie** ("che cos'è X?"): definizione precisa più almeno un esempio.
2. **Di confronto** ("confrontare X e Y"): individuare l'asse lungo cui differiscono (locale/globale, con/senza stato, con/senza connessione, hardware/software) e dirlo esplicitamente.
3. **Di calcolo**: subnetting, longest prefix matching, CRC, checksum, Dijkstra, frammentazione, RTT, finestre. Vanno esercitate a mano: contano i passaggi, non solo il risultato.

Fondamenti

Esercizio 1 — Reti e Internet

(a) Che cos'è una rete di calcolatori? Che cosa significano "autonomi" e "interconnessi"?

(b) Che cos'è un protocollo?

(c) Confrontare commutazione di circuito e commutazione di pacchetto.

(d) Perché si dice che Internet è una "rete di reti"? Che ruolo hanno ISP, IXP e reti dei content provider?

(e) Perché IP è la "vita stretta" della clessidra di Internet?

Svolgimento

(a) Una collezione di dispositivi di calcolo **autonomi** (nessuno comanda gli altri: ognuno ha capacità di calcolo propria) e **interconnessi** (possono scambiarsi informazione, via cavo, fibra, radio…).

(b) Un protocollo definisce il **formato** e l'**ordine** dei messaggi scambiati tra due o più entità, e le **azioni** da compiere quando un messaggio viene inviato o ricevuto.

(c) Nel **circuito** le risorse (una banda in FDM, uno slot in TDM) sono **riservate** per tutta la sessione: prestazioni garantite, ma capacità sprecata quando l'utente tace e costo di instaurazione. Nel **pacchetto** i dati sono divisi in pacchetti che condividono i collegamenti **su richiesta**, con store-and-forward: nessuna garanzia (code, ritardi, perdite), ma molti più utenti serviti grazie alla multiplazione statistica (35 utenti invece di 10 nell'esempio classico). È la scelta vincente per il traffico dati, che è a raffiche.

(d) Perché è l'interconnessione di moltissime reti indipendenti. Gli host si collegano a **ISP di accesso**, che si collegano a ISP regionali e questi a pochi ISP globali (tier 1); gli ISP dello stesso livello si scambiano traffico con accordi di **peering**, spesso negli **IXP**, punti neutrali di scambio che evitano di collegare tutti con tutti. I grandi **content provider** (Google, Microsoft, …) hanno reti proprie che arrivano vicino agli utenti, scavalcando la gerarchia.

(e) Sopra IP ci sono moltissime applicazioni e protocolli di trasporto, sotto moltissime tecnologie di collegamento; in mezzo c'è **un solo** protocollo di rete. Tutto ciò che parla IP può comunicare con tutto il resto: nuove applicazioni e nuove tecnologie fisiche si aggiungono senza cambiare il centro.

Esercizio 2 — Topologie e categorie di reti

(a) Distinguere collegamenti punto-punto e multipunto.

(b) Confrontare le topologie a bus, anello, stella e maglia (costo, guasti, prestazioni).

(c) Distinguere PAN, LAN, MAN e WAN.

(d) Distinguere comunicazione simplex, half-duplex e full-duplex.

Svolgimento

(a) Punto-punto: un collegamento **dedicato** tra esattamente due dispositivi. Multipunto (broadcast): più dispositivi **condividono** lo stesso collegamento e sentono tutti le trasmissioni altrui, quindi serve un protocollo di accesso al mezzo.

(b)

- **Bus**: economico e semplice, ma la dorsale è un punto di guasto unico e le collisioni aumentano con il traffico.
- **Anello**: ogni nodo ripete il segnale; niente collisioni (con token), ma un guasto può interrompere l'anello (se non è doppio).
- **Stella**: n collegamenti, facile da gestire e da estendere, un guasto isola un solo nodo; il centro è il punto critico.
- **Maglia completa**: massima robustezza e prestazioni, ma $n(n-1)/2$ collegamenti: costo quadratico. Nella pratica si usano maglie parziali e topologie ibride.

(c) In base all'estensione: **PAN** (pochi metri, dispositivi di una persona, per esempio Bluetooth); **LAN** (edificio o campus, privata, bassa latenza e pochi errori, tempo di trasmissione nel caso peggiore noto); **MAN** (una città, per esempio le reti via cavo); **WAN** (un paese o un continente: host collegati da una subnet di router e linee di trasmissione, spesso affittate).

(d) Simplex: un solo verso (TV). Half-duplex: entrambi i versi ma a turno (walkie-talkie). Full-duplex: entrambi contemporaneamente (telefono, Ethernet commutata).

Esercizio 3 — Il modello a strati

(a) Elencare i 7 strati del modello ISO/OSI e il compito di ciascuno.

(b) Qual è la differenza tra servizio, interfaccia e protocollo?

(c) Distinguere servizi orientati alla connessione e senza connessione.

(d) Spiegare incapsulamento e decapsulamento.

(e) Perché un router non implementa l'intera pila?

(f) Perché la pila TCP/IP si è imposta sul modello OSI?

(g) Quali vantaggi offre l'architettura a strati, e qual è il suo prezzo?

Svolgimento

(a) Dal basso: **fisico** (trasmissione dei bit sul mezzo), **collegamento** (frame tra nodi adiacenti, rilevazione degli errori, accesso al mezzo), **rete** (instradamento dei pacchetti da host a host), **trasporto** (da processo a processo, affidabilità, controllo di flusso), **sessione** (dialogo e sincronizzazione), **presentazione** (rappresentazione dei dati: codifica, compressione, cifratura), **applicazione** (servizi per l'utente). La pila di Internet ne usa cinque: sessione e presentazione confluiscono nell'applicazione.

(b) Il **servizio** è ciò che uno strato offre a quello superiore (*che cosa* fa); l'**interfaccia** è il modo in cui lo strato superiore vi accede (le primitive); il **protocollo** sono le regole con cui lo strato dialoga con la propria entità paritaria sull'altra macchina (*come* lo fa). Il protocollo si può cambiare senza toccare il servizio.

(c) Orientato alla connessione: si stabilisce una connessione, la si usa, la si chiude (come una telefonata); l'ordine è preservato. Senza connessione: ogni messaggio porta l'indirizzo completo e viaggia per conto suo (come la posta), l'ordine non è garantito. Entrambi possono essere affidabili o no.

(d) Scendendo (mittente) ogni strato aggiunge al payload ricevuto il proprio header (ed eventualmente un trailer); risalendo (destinatario) lo stesso strato lo legge e lo rimuove. Il payload è **opaco**: uno strato non sa, e non deve sapere, che cosa contiene.

(e) Perché il suo compito è la consegna **da host a host**: gli bastano i livelli fisico, collegamento e rete. A quale processo sia destinato il pacchetto non gli interessa; solo gli host implementano tutta la pila.

(f) Per **tempismo** (TCP/IP era già diffuso quando uscirono i protocolli OSI), **progetto** (modello e protocolli complessi, strati squilibriati), **implementazioni** (le prime erano enormi e lente) e **politica** (OSI era percepito come imposto da ministeri e burocrazie), mentre le implementazioni TCP/IP erano gratuite, per esempio in Unix BSD. OSI resta lo standard *de iure* e un ottimo modello concettuale; TCP/IP è lo standard *de facto*.

(g) Vantaggi: una **struttura** esplicita con cui ragionare sul sistema, e soprattutto la **modularità** (si cambia l'implementazione di uno strato, per esempio un driver o una tecnologia di accesso, senza toccare gli altri, purché il servizio resti lo stesso). Prezzo: **maggiore complessità** (interfacce da progettare e implementare, overhead), **ridondanze** (funzioni come il controllo degli errori ripetute in più strati), **rigidità** (le informazioni interne a uno strato sono nascoste; quando servono altrove bisogna riprogettare o ricorrere a soluzioni *quick and dirty*).

Esercizio 4 — Banda e throughput

(a) Distinguere transmission rate, banda e throughput.

(b) Che cosa significa store-and-forward? E cut-through?

Svolgimento

(a) Il **transmission rate** è il massimo che un dispositivo può trasmettere; la **banda** è il massimo che un *cammino* (collegamenti e nodi) può trasportare; il **throughput** è la quantità che sta *effettivamente* fluendo. Vale sempre throughput ≤ banda ≤ rate del dispositivo più lento.

(b) Store-and-forward: un nodo riceve **tutto** il pacchetto prima di inoltrarlo (può controllarne l'integrità, ma aggiunge L/R di ritardo a ogni salto). Cut-through: inizia a inoltrare appena ha letto l'intestazione (meno ritardo, ma può propagare pacchetti danneggiati).

LIVELLO APPLICATIVO

Esercizio 5 — Architetture e protocolli

(a) Confrontare architettura client-server e peer-to-peer.

(b) Perché HTTP è senza stato, e come si recupera lo stato?

(c) Descrivere il caching Web e il GET condizionale.

(d) Perché la posta elettronica usa protocolli diversi?

(e) Descrivere le politiche di BitTorrent.

(f) Perché FTP usa due connessioni? Che differenza c'è tra modalità attiva e passiva?

(g) Qual è la differenza tra un'applicazione e il suo protocollo applicativo?

(h) Quanti indirizzi IP ha un portatile collegato via cavo e via Wi-Fi? A che cosa serve `127.0.0.1`?

Svolgimento

(a) Nel client-server c'è un **server** sempre attivo, con indirizzo fisso e noto, e i client non comunicano direttamente tra loro. Nel P2P i peer sono (idealmente) equivalenti, connessi in modo intermittente, e comunicano direttamente: il sistema scala meglio perché ogni peer porta anche capacità di servizio, ma è più complesso da gestire e pone problemi di sicurezza e affidabilità.

(b) Per semplicità e per poter servire un numero enorme di richieste senza ricordare nulla dei client. Lo stato si recupera con i **cookie**: il server crea un identificativo (`Set-Cookie`), il browser lo memorizza e lo riallega a ogni richiesta (`Cookie`), e il server lo usa come chiave nel proprio database.

(c) Una **cache** (proxy) soddisfa le richieste al posto del server di origine con copie locali: riduce tempi di risposta e traffico sul collegamento verso Internet. Con il **GET condizionale** (`If-Modified-Since`) la cache chiede l'oggetto solo se è cambiato; altrimenti il server risponde `304 Not Modified` senza corpo.

(d) Perché la comunicazione avviene in due tratte diverse: tra mail server si usa **SMTP** (porta 25, protocollo *push*); per leggere la posta dalla propria casella l'utente usa un protocollo di *accesso* (**POP3**, **IMAP** o HTTP per la webmail).

(e) In download: **rarest first**, si chiedono prima i chunk con meno copie tra i vicini. In upload: **tit-for-tat**, si serve chi ci fornisce al ritmo più alto (i migliori 4, rivalutati ogni 10 s), più un vicino scelto a caso ogni 30 s (*optimistic unchoking*) per dare una possibilità ai nuovi peer.

(f) Per tenere separati comandi e dati (segnalazione **fuori banda**): la connessione di **controllo** (porta 21) resta aperta per tutta la sessione, mentre ogni trasferimento (`get`, `put`, anche `ls`) apre e chiude una connessione **dati**. In modalità **attiva** è il server ad aprire la connessione dati verso il client, cosa impossibile se il client è dietro un NAT o un firewall; in modalità **passiva** il client apre entrambe le connessioni.

(g) L'applicazione comprende tutto: per il Web, il formato dei documenti (HTML), i browser, i server. Il protocollo (HTTP) fissa solo il formato e l'ordine dei messaggi tra le parti, ed è l'unica cosa su cui i due capi devono concordare.

(h) Due: un indirizzo appartiene a un'**interfaccia**, non alla macchina, e per il livello di rete il portatile è come due host distinti raggiunti per cammini diversi. `127.0.0.1` (`localhost`) indica la macchina stessa: due processi locali comunicano attraverso di esso come in rete, senza che nulla esca dall'host; serve per provare client e server su un solo computer.

Esercizio 6 — Socket e DNS

(a) Quali sono le differenze nella sequenza di chiamate tra un'applicazione UDP e una TCP?

(b) Da quanti elementi è identificato un socket UDP? E uno TCP?

(c) Perché il server TCP ha bisogno di due socket?

(d) Confrontare query ricorsive e iterative nel DNS.

Svolgimento

(a)

UDP	TCP
`socket()`; `bind()` solo sul server	`socket()`; server: `bind()`, `listen()`, `accept()`
nessuna connessione	client: `connect()` (three-way handshake)
`sendto()` / `recvfrom()` con l'indirizzo a ogni messaggio	`send()` / `read()`: l'indirizzo è fissato dalla connessione
`close()`	`close()` (sul server anche del socket dedicato)

(b) Un socket UDP è identificato dalla coppia (IP, porta) di destinazione: lo stesso socket riceve da client diversi. Un socket TCP connesso dalla quadrupla (IP sorgente, porta sorgente, IP destinazione, porta destinazione).

(c) Il **welcoming socket** resta in ascolto per accettare nuove connessioni; per ogni client `accept()` crea un **socket dedicato**, usato per la comunicazione. Così il server continua ad accettare client mentre serve quelli già connessi.

(d) In una query **ricorsiva** il server interrogato si fa carico dell'intera risoluzione e restituisce la risposta finale; in una **iterativa** risponde con ciò che sa (spesso un rinvio al server successivo). Tipicamente l'host fa una query ricorsiva al server locale, che procede in modo iterativo; i server in alto nella gerarchia non offrono ricorsione, per non tenere stato.

LIVELLO DI TRASPORTO

Esercizio 7 — UDP e TCP

(a) Quali sono le responsabilità del livello di trasporto? Quali copre UDP?

(b) Perché usare UDP invece di TCP?

(c) Qual è la relazione tra MSS e MTU?

(d) Perché il fast retransmit aspetta 3 ACK duplicati e non uno?

Svolgimento

(a) (1) Consegna da processo a processo (multiplexing e demultiplexing con le porte); (2) controllo di integrità (checksum); (3) trasferimento affidabile; (4) controllo di flusso e di congestione. UDP copre solo le prime due; TCP tutte.

(b) Controllo più fine da parte dell'applicazione (utile nel tempo reale), nessun handshake (una query DNS è una domanda e una risposta), nessuno stato di connessione (più client serviti), header minimo (8 byte contro 20).

(c) MSS = MTU − 40 byte (20 di header IP e 20 di header TCP, senza opzioni): con MTU Ethernet di 1500 byte, MSS di 1460 byte.

(d) Perché un ACK duplicato può nascere anche da un semplice **riordino**: se due segmenti arrivano scambiati, il ricevente invia un duplicato prima di ricevere quello mancante. Tre duplicati sono un indizio molto più affidabile di una perdita.

Esercizio 8 — Trasferimento affidabile

(a) Confrontare Go-Back-N e Selective Repeat.

(b) Perché in Selective Repeat la finestra non può superare metà dello spazio dei numeri di sequenza?

(c) Perché il timeout deve essere maggiore dell'RTT, ma non troppo?

Svolgimento

(a) In **GBN** il ricevente accetta solo pacchetti in ordine e invia ACK cumulativi; il mittente, allo scadere del timer (unico, sul pacchetto più vecchio), ritrasmette tutta la finestra. Ricevente semplicissimo, ma molte ritrasmissioni inutili. In **SR** il ricevente bufferizza i pacchetti fuori ordine e conferma ciascuno; il mittente ha un timer per pacchetto e ritrasmette solo quelli persi. Più efficiente, più complesso.

(b) Perché altrimenti, se gli ACK vanno persi, un pacchetto ritrasmesso può cadere nella nuova finestra del ricevente ed essere scambiato per un pacchetto nuovo con lo stesso numero (si veda l'esercizio nel capitolo sul trasporto).

(c) Troppo corto: ritrasmissioni inutili, che aggiungono traffico proprio quando la rete è lenta. Troppo lungo: si reagisce tardi alle perdite. EstimatedRTT + 4 DevRTT dà un margine che si adatta alla variabilità.

Esercizio 9 — Connessione e congestione

(a) Descrivere il problema dei due eserciti e la sua rilevanza per TCP.

(b) Distinguere controllo di flusso e controllo di congestione.

(c) Perché un timeout riporta cwnd a 1 MSS mentre 3 ACK duplicati la dimezzano soltanto?

(d) Perché i numeri di sequenza iniziali sono casuali?

Svolgimento

(a) Due eserciti alleati devono attaccare insieme, ma comunicano con messaggeri che possono essere catturati: qualunque sia il numero di conferme, chi invia l'ultima non sa se è arrivata. Nessun protocollo garantisce l'accordo su un canale inaffidabile. TCP accetta un compromesso: il three-way handshake (e la chiusura con TIME_WAIT) non è perfetto ma è adeguato, e i timer risolvono le situazioni ambigue.

(b) Il **flusso** protegge il **ricevente** dal traboccamento del buffer (rwnd, annunciata dal ricevente); la **congestione** protegge la **rete** (cwnd, stimata dal mittente osservando perdite e ritardi). Il mittente rispetta LastByteSent − LastByteAcked ≤ $\min\{\text{cwnd}, \text{rwnd}\}$.

(c) Un timeout significa che non arriva più nulla: congestione grave, si riparte da capo. Tre ACK duplicati dimostrano che i segmenti successivi stanno arrivando: congestione lieve, basta dimezzare. In entrambi i casi ssthresh = cwnd/2.

(d) Per non confondere segmenti di una vecchia connessione (con le stesse porte) con quelli di una nuova, e per rendere difficile a un attaccante indovinare i numeri e iniettare segmenti falsificati.

Esercizio 10 — IP e indirizzamento

(a) Che cos'è il servizio best effort e perché è sufficiente?

(b) Distinguere forwarding e routing, data plane e control plane.

(c) Perché un indirizzo IP appartiene a un'interfaccia e non a un host?

(d) Perché l'indirizzamento a classi è stato abbandonato?

(e) Perché IPv6 ha eliminato il checksum e la frammentazione nei router?

Svolgimento

(a) La rete "fa del suo meglio" senza garanzie su consegna, ordine, ritardo o banda. Basta perché le garanzie che servono vengono ricostruite agli estremi (TCP, applicazioni): è il principio **end-to-end**, nucleo semplice e intelligenza ai bordi.

(b) Il **forwarding** (data plane) è l'azione locale di spostare un pacchetto dall'ingresso all'uscita giusta: nanosecondi, in hardware. Il **routing** (control plane) calcola i cammini sull'intera rete e riempie le tabelle: secondi, in software.

(c) Perché ogni interfaccia è collegata a una subnet diversa e deve inviare e ricevere datagrammi su quella subnet: un router con tre interfacce ha tre indirizzi, uno per subnet.

(d) Per lo spreco: le classi avevano dimensioni fisse (A 16 milioni, B 65 536, C 256 indirizzi). A chi servivano 300 indirizzi toccava una classe B, sprecandone oltre 65 000. Con CIDR basta un /23 (512 indirizzi).

(e) Il checksum va ricalcolato a ogni salto (il TTL cambia) ed è ridondante, perché collegamento e trasporto controllano già gli errori. La frammentazione nei router è lenta e complessa: in IPv6 un router che riceve un datagramma troppo grande lo scarta e invia un ICMPv6 *Packet Too Big*, e sarà il mittente a ridurre la dimensione.

Esercizio 11 — DHCP, NAT e ICMP

(a) Descrivere i 4 passi di DHCP e spiegare perché usa il broadcast.

(b) Perché il NAT cambia la porta sorgente? Qual è il suo limite fondamentale?

(c) Come funziona traceroute?

Svolgimento

(a) Discover (broadcast, da 0.0.0.0 porta 68 a 255.255.255.255 porta 67), Offer (con l'indirizzo proposto in `yiaddr`), Request (il client accetta un'offerta), ACK (conferma). Il broadcast serve perché l'host non ha ancora un indirizzo e non sa chi sia il server; la request è in broadcast per informare tutti i server di quale offerta è stata scelta.

(b) Perché host diversi possono scegliere la stessa porta: il NAT deve garantire coppie (IP WAN, porta) uniche per distinguere le risposte. Il limite è che le connessioni **iniziate dall'esterno** non trovano una riga nella tabella: servono port forwarding, UPnP, STUN/TURN o hole punching. Inoltre il NAT viola la stratificazione (un dispositivo di rete modifica le porte, che sono di trasporto).

(c) Invia sonde (UDP verso porte improbabili) con TTL = $1, 2, 3, \ldots$ Il router a cui il TTL scade scarta la sonda e risponde con ICMP *Time Exceeded*, rivelando il proprio indirizzo; quando la sonda raggiunge la destinazione, questa risponde con ICMP *Port Unreachable* e traceroute si ferma. Per ogni salto misura il tempo di andata e ritorno; un asterisco indica una risposta mancata.

Esercizio 12 — Routing

(a) Confrontare Distance Vector e Link State.

(b) Spiegare il count-to-infinity e le sue cause.

(c) Perché il flooding, pur trovando sempre il cammino, è impraticabile?

Svolgimento

(a)

	Distance Vector	**Link State**
Conoscenza	locale (vettori dei vicini)	globale (tutto il grafo)
Algoritmo	Bellman-Ford	Dijkstra
Funzionamento	decentralizzato, asincrono	ogni nodo calcola sulla mappa completa
Messaggi	solo ai vicini	link-state packet in flooding a tutti
Convergenza	lenta, count-to-infinity	più rapida
Robustezza	un errore si propaga ai calcoli degli altri	ogni nodo calcola la propria tabella

(b) Il vettore $D_x(y)$ si calcola dai vettori dei vicini, che contengono solo i **costi** e non i **cammini**: un nodo può scegliere un vicino il cui cammino migliore passa per il nodo stesso. Quando un collegamento peggiora, i due nodi si rimandano stime crescenti a piccoli passi finché non si raggiunge il costo alternativo (o all'infinito, se il collegamento cade). Contromisure: poisoned reverse, split horizon, un "infinito" finito (16 in RIP).

(c) Esplora tutti i cammini ed è robustissimo, ma genera una quantità di traffico enorme (che cresce esponenzialmente senza controlli sui duplicati): va bene solo in contesti particolari (diffusione di informazioni a tutti, reti militari), non come routing di Internet.

LIVELLO DI COLLEGAMENTO

Esercizio 13 — Framing ed errori

(a) Elencare i metodi di framing e spiegare perché il conteggio dei byte non si usa.

(b) Perché la parità singola è quasi inutile nelle reti reali?

(c) Che cosa rileva e corregge la parità bidimensionale?

(d) Quanti errori rileva e corregge un codice con distanza di Hamming d?

Svolgimento

(a) Conteggio dei byte, byte di flag con byte stuffing, bit di flag con bit stuffing, violazioni della codifica fisica. Il conteggio non si usa (da solo) perché se il campo di conteggio si corrompe non è più possibile ritrovare l'inizio dei frame successivi.

(b) Rileva solo un numero dispari di errori, e gli errori reali arrivano a raffiche: la probabilità di non accorgersene è vicina al 50%.

(c) Rileva e **corregge** un errore singolo (incrocio di riga e colonna); rileva ma non corregge qualsiasi coppia di errori.

(d) Rileva $d-1$ errori e ne corregge $\lfloor (d-1)/2 \rfloor$: servono distanza $k+1$ per rilevarne k, $2k+1$ per correggerne k.

Esercizio 14 — Accesso multiplo

(a) Quali sono i requisiti ideali di un protocollo ad accesso multiplo?

(b) Qual è il difetto strutturale di TDMA e FDMA?

(c) Se tutti i nodi ascoltano il canale prima di trasmettere, perché ci sono ancora collisioni?

(d) Perché Ethernet ha una dimensione minima dei frame?

(e) Che cosa sono i problemi del terminale nascosto ed esposto, e come li affronta MACA?

Svolgimento

(a) Con un solo nodo attivo, usare tutto il rate R; con M nodi attivi, R/M ciascuno in media; essere **decentralizzato** (niente punto di guasto unico, niente sincronizzazione globale); essere **semplice** ed economico.

(b) Un nodo ha sempre e solo R/N, anche quando tutti gli altri tacciono: viola il primo requisito.

(c) Per il **ritardo di propagazione**: un nodo può trovare libero un canale su cui un altro ha già iniziato a trasmettere, ma il cui segnale non è ancora arrivato.

(d) Perché CSMA/CD rilevi la collisione, il mittente deve essere ancora in trasmissione quando il segnale della collisione torna indietro: il frame deve durare almeno 2τ. Da qui i 64 byte minimi e il campo pad.

(e) Nel wireless **A** e **C** possono non sentirsi pur raggiungendo entrambi **B**: ascoltando il canale lo trovano libero e collidono in B (**terminale nascosto**); viceversa un nodo può rinunciare a trasmettere perché sente una trasmissione che in realtà non disturberebbe il suo destinatario (**terminale esposto**). MACA fa precedere i dati da un breve **RTS**, a cui il destinatario risponde con un **CTS** che indica la durata: chi sente il CTS tace, chi sente solo l'RTS può trasmettere.

Esercizio 15 — Switch e ARP

(a) Confrontare switch e router.

(b) Che cos'è il self-learning?

(c) Perché le LAN commutate devono avere una topologia ad albero (senza cicli)?

(d) Come cambiano IP e MAC di destinazione lungo un percorso con più salti?

(e) Perché servono sia gli indirizzi MAC sia gli indirizzi IP?

Svolgimento

(a) Lo **switch** lavora al livello 2 con indirizzi MAC, è plug-and-play, veloce e senza algoritmo di routing (tabelle per self-learning). Il **router** lavora al livello 3 con indirizzi IP, esegue algoritmi di routing, va configurato, isola i domini di broadcast e supporta qualunque topologia.

(b) Quando riceve un frame, lo switch registra che il **MAC sorgente** è raggiungibile dalla porta di arrivo; per la destinazione inoltra se la conosce, fa flooding se non la conosce, filtra se è sulla stessa porta. Le righe scadono dopo un TTL.

(c) Senza routing, un frame in broadcast (o in flooding) in un ciclo girerebbe per sempre moltiplicandosi: una **tempesta di broadcast**. Per questo le reti commutate usano alberi (in pratica calcolati da un protocollo di spanning tree).

(d) Il MAC di destinazione è quello del **prossimo salto** e cambia a ogni collegamento; l'IP di destinazione è quello della **destinazione finale** e resta invariato (salvo NAT). Ogni router decapsula e reincapsula il datagramma in un nuovo frame.

(e) Il MAC è piatto, assegnato dal costruttore e portabile: identifica l'interfaccia su un collegamento. L'IP è gerarchico e dipende dalla rete a cui si è collegati: permette di aggregare e instradare. Tenerli separati rende gli strati indipendenti (una LAN può trasportare anche protocolli diversi da IP) e consente a una scheda di cambiare rete senza cambiare MAC.

SICUREZZA

Esercizio 16 — Attacchi e crittografia

(a) Distinguere virus e worm.

(b) Elencare i tre tipi di attacco DoS.

(c) Perché lo sniffing è così difficile da rilevare?

(d) Enunciare il principio di Kerckhoffs e spiegarne la ragione.

(e) Perché il one-time pad, pur essendo inviolabile, non si usa in pratica?

(f) Confrontare crittografia simmetrica e asimmetrica.

Svolgimento

(a) Un **virus** richiede un'interazione dell'utente (aprire un allegato, eseguire un programma); un **worm** entra da solo, sfruttando applicazioni di rete vulnerabili, e si propaga automaticamente.

(b) Attacco a una **vulnerabilità** (messaggi costruiti per far bloccare un servizio o il sistema operativo); **bandwidth flooding** (inondare la vittima di pacchetti); **connection flooding** (aprire moltissime connessioni TCP, anche semi-aperte come nel SYN flood).

(c) Perché è **passivo**: non inietta traffico, quindi dall'esterno non lascia tracce. La difesa non è la rilevazione ma la **cifratura**: se non si può impedire l'ascolto, si rende incomprensibile ciò che viene ascoltato.

(d) Gli algoritmi devono essere **pubblici**; solo le **chiavi** sono segrete. Un algoritmo pubblico viene analizzato da tutta la comunità e le sue debolezze emergono e si correggono; un algoritmo segreto sembra sicuro solo finché qualcuno non lo rompe, e non lo si può sapere. La *security through obscurity* non funziona.

(e) Il pad deve essere casuale, lungo quanto i messaggi e mai riusato: va distribuito in modo sicuro, e se esiste un canale sicuro abbastanza capiente per il pad tanto vale usarlo per il messaggio. In più ci sono problemi di sincronizzazione.

(f) Nella **simmetrica** (DES, AES) mittente e destinatario condividono la stessa chiave: veloce, ma la chiave va scambiata in modo sicuro e ne serve una per ogni coppia. Nell'**asimmetrica** (RSA) ognuno ha una chiave pubblica e una privata: niente segreto da scambiare, ma molto più lenta. In pratica si usano insieme: l'asimmetrica per scambiare una **chiave di sessione** simmetrica.

Esercizio 17 — Autenticazione e SSL

(a) Perché cifrare la password non basta ad autenticarsi?

(b) A che cosa serve un'autorità di certificazione?

(c) Descrivere le fasi di SSL/TLS.

(d) Perché SSL mette un numero di sequenza dentro la parte protetta dei record?

Svolgimento

(a) Perché la password cifrata può essere registrata e **rigiocata** (playback attack). Serve un **nonce**: un valore usato una sola volta, scelto dal verificatore, da cifrare con la chiave condivisa.

(b) Garantisce che una chiave pubblica appartenga davvero a chi dice di esserne il proprietario: firma un **certificato** che lega l'identità alla chiave. Senza, un man-in-the-middle potrebbe sostituire la propria chiave (come nell'attacco a Diffie-Hellman).

(c) **Handshake**: dopo la connessione TCP, il server invia il certificato; il client lo verifica, genera un *master secret*, lo cifra con la chiave pubblica del server e lo invia, e i due si scambiano MAC di tutti i messaggi dell'handshake (contro manomissioni). **Derivazione delle chiavi**: dal master secret si ricavano quattro chiavi (cifratura e MAC, per ciascun verso). **Trasferimento dati**: il flusso è diviso in record; a ogni record si aggiunge un MAC, si cifra e si passa a TCP.

(d) Il numero di sequenza TCP è in chiaro e un man-in-the-middle potrebbe riordinare, duplicare o eliminare record. Includendo un numero di sequenza nel calcolo del MAC, ogni alterazione dell'ordine invalida il MAC: si protegge l'integrità dell'intero flusso.

Esercizio 18 — Firewall e IDS

(a) Qual è il limite del filtraggio tradizionale e come lo risolve quello stateful?

(b) Quando serve un application gateway?

(c) Distinguere IDS basati su firme e basati su anomalie.

(d) A che cosa serve una DMZ?

Svolgimento

(a) Il filtro tradizionale è **senza stato**: una regola che lascia entrare i segmenti con ACK = 1 e porta sorgente 80 fa passare anche segmenti che non appartengono ad alcuna connessione (usati negli attacchi). Il filtro **stateful** tiene una tabella delle connessioni (riconosce SYN, SYN-ACK, FIN, e usa timeout di inattività) e ammette solo i pacchetti di connessioni esistenti.

(b) Quando le decisioni dipendono da **utenti** o **contenuti applicativi** (per esempio Telnet solo per alcuni utenti autenticati): queste informazioni non sono negli header IP/TCP/UDP. Il prezzo: un gateway per applicazione, prestazioni più basse, client configurati per usarlo.

(c) Basati su **firme**: confrontano il traffico con un database di attacchi noti; precisi, ma ciechi agli attacchi nuovi. Basati su **anomalie**: costruiscono un profilo del traffico normale e segnalano ciò che se ne discosta; possono scoprire attacchi nuovi, ma producono falsi positivi.

(d) È una zona a sicurezza ridotta, tra un firewall esterno più permissivo e uno interno restrittivo, dove si mettono i server che devono essere raggiungibili da Internet (Web, posta, DNS). Se uno di essi viene compromesso, l'attaccante è ancora fuori dalla rete interna.

Consigli per la preparazione

Gli errori più frequenti

- Confondere consegna **host-to-host** (rete) e **process-to-process** (trasporto).
- Attribuire il three-way handshake a HTTP invece che a TCP.
- Confondere il **MAC address** (livello 2) con il **MAC crittografico** (*Message Authentication Code*).
- Dimenticare che il MAC di destinazione cambia a ogni salto, l'IP di destinazione no.
- Dire che il TTL di IP è un tempo: è un **contatore di salti**.
- Nel CRC: dimenticare gli r zeri, allineare male G, usare la sottrazione con prestiti invece dello XOR.
- Nel checksum: dimenticare il wrap around o il complemento finale.
- Nel subnetting: proporre maschere con bit non contigui; dimenticare che rete e broadcast non sono assegnabili.
- Nella frammentazione: esprimere l'offset in byte invece che in unità da 8 byte, o dimenticare che i dati devono essere multipli di 8.
- Nella stima dell'RTT: non dichiarare l'ordine di aggiornamento di EstimatedRTT e DevRTT.

Come prepararsi

- Per ogni capitolo, rileggere il riquadro “In breve” e provare a spiegarlo ad alta voce senza guardare.
- Rifare gli esercizi di calcolo **a mano**, su carta, controllando i passaggi con le soluzioni solo alla fine.
- Ripetere almeno una volta le esercitazioni con Wireshark: vedere i pacchetti veri fissa i concetti molto più della teoria.
- Per le domande di confronto, preparare piccole tabelle a due colonne (come quelle di questo capitolo).